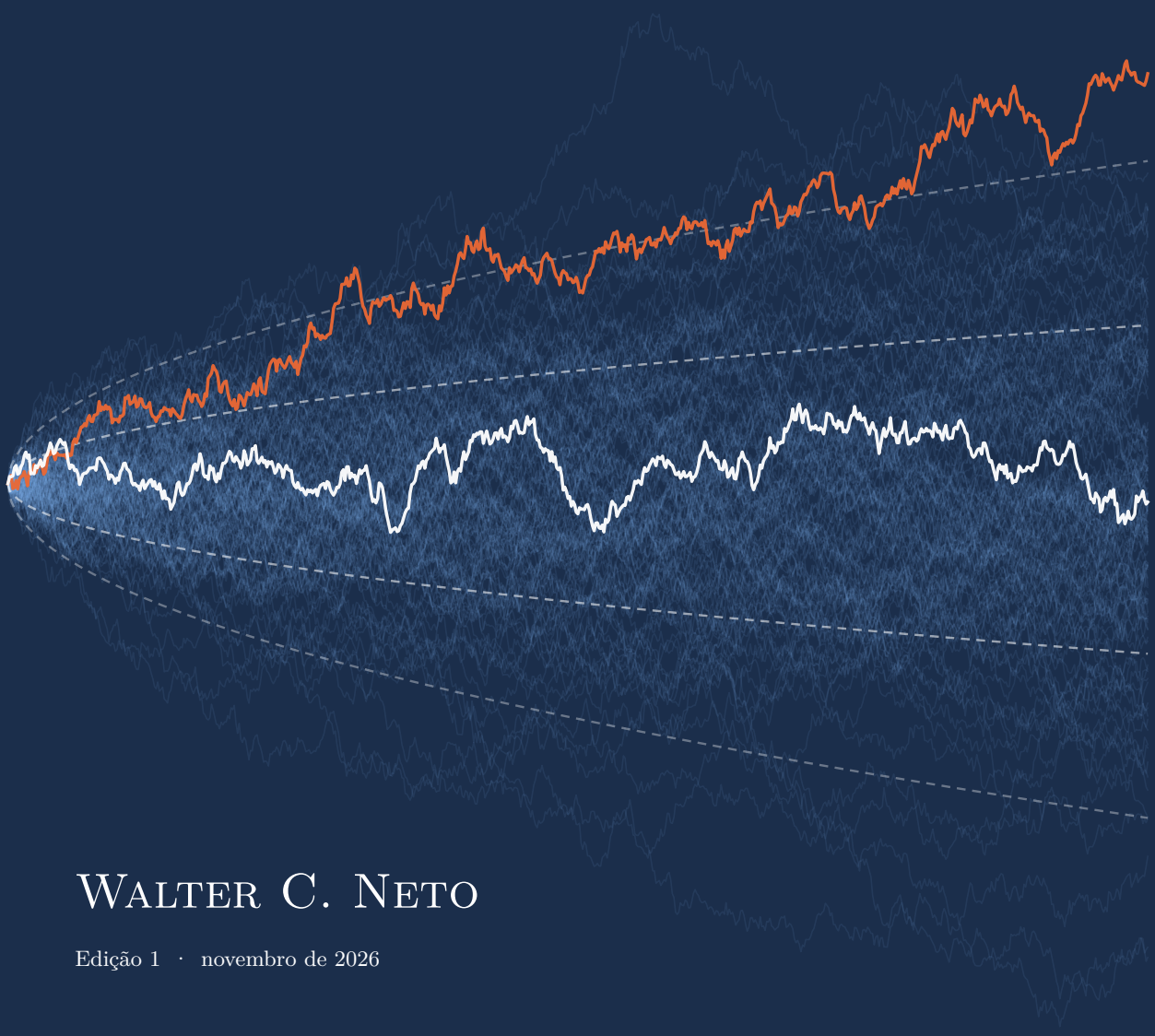
XRiscs

Derivativos com Python

Precificação, calibração e hedge em Python,
do Black-Scholes ao smile, com auxílio de IA



Walter C. Neto

Edição 1 · novembro de 2026

XRiscs

Derivativos com Python

Precificação, calibração e hedge em Python,
do Black-Scholes ao smile, com auxílio de IA

Walter C. Neto

waltercneto.github.io/XRiscs

XRiscs — Derivativos com Python
Walter C. Neto
Edição 1, novembro de 2026. Gerada a partir dos notebooks do repositório `https://github.com/walterCNeto/XRiscs` (commit local).

Textos e figuras: Creative Commons Atribuição 4.0 Internacional (CC BY 4.0). Código: licença MIT. Você pode copiar, adaptar e redistribuir, desde que cite a fonte.

Textos, código e laboratórios desenvolvidos com auxílio de IA (Claude, da Anthropic).

Este é um material de estudo independente. A sequência dos Capítulos 1 a 16 acompanha Hilpisch (2015); não há afiliação com o autor nem com a editora, e nenhum trecho daquele livro é reproduzido. Todos os dados são sintéticos, exceto as taxas dos T-bills do Capítulo 17 (U.S. Treasury, domínio público).

Cada capítulo corresponde a um notebook executável. As verificações automáticas `ok(...)` que aparecem nas saídas foram todas aprovadas na geração desta edição (530 no total).

Capa: trajetórias simuladas de um movimento browniano geométrico, com as bandas de ± 1 e ± 2 desvios-padrão do log-preço.

Sumário

Prefácio vii

I Fundamentos **1**

1 Mercados, arbitragem e precificação de derivativos 3
2 Stylized facts e volatilidade implícita 37
3 A medida neutra ao risco em tempo discreto 73
4 Cálculo estocástico: movimento browniano, Itô e Girsanov 103

II Modelos e métodos de precificação **131**

5 Black-Scholes-Merton: a EDP, a fórmula e as gregas 133
6 A árvore binomial de Cox-Ross-Rubinstein 157
7 Métodos de Fourier I: da função característica à fórmula de Lewis 177
8 Métodos de Fourier II: Carr-Madan, FFT e convolução 197
9 Opções americanas por simulação: Longstaff-Schwartz e o problema dual . . 217

III Precificação calibrada ao mercado **239**

10 Calibração ao mercado com o modelo de saltos de Merton 241
11 O modelo BCC97: volatilidade estocástica, saltos e juros estocásticos 259
12 Monte Carlo: discretização e redução de variância. 277
13 Calibração I: a curva de juros no CIR e as opções no Heston 307
14 Calibração II: saltos, regularização e o BCC97 completo 329
15 Simulação e precificação no modelo BCC97 349
16 Hedge dinâmico e síntese do curso 367

IV Complemento — modelos de juros **387**

17 Hull-White I: medidas, o modelo e as fórmulas fechadas 389
18 Hull-White II: a árvore trinomial 411
19 Simulação de trajetórias de juros: do Vasicek ao HJM 433

V Complemento — sorriso e volatilidade local **457**

20 Árvores implícitas de Derman-Kani-Chriss: construção e visualização interativa . . . 459

A Preparação padrão dos laboratórios . . . 479

B Erratas e notas sobre o livro de Hilpisch . . . 481

Referências . . . 491

Prefácio

Precificação de derivativos em Python: opções sobre índices, da teoria à calibração ao mercado e ao hedge, com complementos sobre modelos de juros e árvores implícitas. São 20 módulos com laboratórios.

Este é um curso de estudo, em português, construído sobre o livro *Derivatives Analytics with Python* (Yves Hilpisch, Wiley, 2015). Ele segue a sequência do livro, do modelo de Black-Scholes-Merton até o modelo geral de Bakshi, Cao e Chen (volatilidade estocástica, saltos e juros estocásticos), e acrescenta a cada passo o que o livro deixa implícito: as deduções completas e um laboratório em Python que confere cada resultado numericamente.

Como o curso é organizado

Cada módulo é um notebook, e cada notebook é um capítulo deste livro com a mesma estrutura:

1. **Objetivos** e a ligação com os módulos anteriores.
2. **Teoria passo a passo**, com as deduções escritas por inteiro.
3. **Laboratórios** em Python: cada afirmação importante é conferida por código, com verificações automáticas (`ok(...)`) que interrompem o notebook se a conta não fechar. São **530 verificações** no curso.
4. **Síntese**, **erratas e notas** sobre o trecho do livro, e **exercícios** com solução.

Todos os dados são **sintéticos**. Isso permite algo que dados reais não permitem: calibrar um modelo onde a resposta é conhecida e ver o que a calibração recupera e o que ela não consegue identificar.

Mapa dos módulos

Módulo	Tema	Livro
1	Mercados, arbitragem e precificação de derivativos	caps. 1-2
2	Stylized facts e volatilidade implícita	cap. 3
3	A medida neutra ao risco em tempo discreto	cap. 4

Módulo	Tema	Livro
4	Cálculo estocástico: movimento browniano, Itô e Girsanov	cap. 4
5	Black-Scholes-Merton: a EDP, a fórmula e as gregas	cap. 5
6	A árvore binomial de Cox-Ross-Rubinstein	cap. 5
7	Métodos de Fourier I: da função característica à fórmula de Lewis	cap. 6
8	Métodos de Fourier II: Carr-Madan, FFT e convolução	cap. 6
9	Opções americanas por simulação: Longstaff-Schwartz e o problema dual	cap. 7
10	Calibração ao mercado com o modelo de saltos de Merton	cap. 8
11	O modelo BCC97: volatilidade estocástica, saltos e juros estocásticos	cap. 9
12	Monte Carlo: discretização e redução de variância	cap. 10
13	Calibração I: a curva de juros no CIR e as opções no Heston	cap. 11
14	Calibração II: saltos, regularização e o BCC97 completo	cap. 11
15	Simulação e precificação no modelo BCC97	cap. 12
16	Hedge dinâmico e síntese do curso	caps. 13-14
17	Hull-White I: medidas, o modelo e as fórmulas fechadas	complemento
18	Hull-White II: a árvore trinomial (e Black-Karasinski)	complemento
19	Simulação de trajetórias de juros: do Vasicek ao HJM	complemento
20	Árvores implícitas de Derman-Kani-Chriss, com widget interativo	complemento

Os Módulos 17 a 19 complementam o livro com modelos de juros. Os dois primeiros tratam do Hull-White, a partir de Wilmott, Clewlow e Strickland e Hull e White: troca de numerário, fórmulas fechadas, caps, estimação de parâmetros e a árvore trinomial com títulos callable e puttable; o Módulo 19 simula e calibra à mesma curva e ao mesmo caplet uma escada de modelos (Vasicek, Ho-Lee, Hull-White, CIR++, Black-Karasinski, G2++ e HJM com componentes principais) e compara o que a calibração não fixa. O Módulo 20 volta às ações: constrói a árvore trinomial implícita de Derman, Kani e Chriss (1996), que reproduz o smile de volatilidade nó a nó, e traz um widget interativo, que roda na própria página, para montar e desenhar a árvore com passos desiguais, nós que acompanham o forward e overrides. O Laboratório 6 do Módulo 17 usa as taxas diárias dos T-bills americanos de 2023 (U.S. Treasury, domínio público), em `dados/`.

A página Erratas e notas reúne, módulo a módulo, as correções ao livro encontradas nos laboratórios: de sinais e fatores em fórmulas a detalhes dos scripts que mudam os resultados numéricos.

Termos em inglês

Alguns termos ficam em inglês porque é assim que o mercado e a literatura os usam, e a tradução atrapalharia mais do que ajudaria: *smile* e *skew* (de volatilidade), *stylized facts*, *payoff*, *strike*, *hedge*, *forward* (taxa, preço, curva e medida *forward*), *spot*, *short rate*, *moment matching*, *override*. Termos com tradução consagrada em português continuam em português: volatilidade implícita e local, gregas, movimento browniano, medida neutra ao risco, numerário.

Pré-requisitos

- Cálculo e probabilidade no nível de um primeiro curso de graduação.
- Python com NumPy, pandas e matplotlib no nível básico.
- Não é preciso conhecer derivativos: o Módulo 1 começa do zero, e o cálculo estocástico é construído nos Módulos 3 e 4.

Como usar

No navegador. Cada página tem um botão de foguete (no topo) para abrir o notebook no Google Colab ou no Binder e executá-lo.

No seu computador.

```
git clone https://github.com/walterCNeto/XRiscs.git
cd XRiscs
conda env create -f environment.yml
conda activate xriscs
jupyter lab
```

Os notebooks estão salvos já executados. Para conferir que todos rodam e passam nas verificações: `pytest -q` (cerca de 15 minutos).

Convenções

- Números em tabelas e saídas com **três casas decimais**, no padrão brasileiro (vírgula decimal).
- Referências como "§11.4.3" ou "(10.6)" são seções e equações do livro.
- Sementes aleatórias fixas: os resultados são reproduzíveis.

Licença e créditos

Código sob licença MIT; textos sob CC BY 4.0. Este é um material de estudo independente: não reproduz o livro, não substitui sua leitura e não tem afiliação com o autor ou a editora. O livro está em Wiley e o site oficial do autor é derivatives-analytics-with-python.com.

Autor: **Walter C. Neto** — waltercneto.github.io

Parte I

Fundamentos

Capítulo 1

Mercados, arbitragem e precificação de derivativos

Introdução

Um derivativo é um contrato cujo pagamento depende do preço de outro ativo: uma opção de compra (call) sobre um índice de ações paga a diferença entre o índice e um preço combinado (o *strike*), se essa diferença for positiva. A pergunta central deste livro é quanto vale hoje um contrato assim.

A resposta clássica não passa por adivinhar o futuro. Num mercado em que se pode comprar e vender livremente o ativo e aplicar ou tomar dinheiro emprestado, é possível montar carteiras que reproduzem o pagamento de muitos derivativos. Se o pagamento pode ser reproduzido, o preço do derivativo tem de ser o custo dessa carteira; qualquer outro preço permitiria lucro sem risco, a chamada **arbitragem**. Quando todo pagamento pode ser reproduzido, o mercado é **completo**, e o preço é único.

Essa construção depende de hipóteses fortes: ausência de custos de transação, negociação contínua, possibilidade de vender a descoberto, ausência de risco de crédito e um modelo que descreva bem o ativo. Os mercados reais são **incompletos**: saltos de preço e volatilidade que muda ao longo do tempo criam riscos que o ativo sozinho não reproduz. Nesses mercados há um intervalo de preços sem arbitragem, e é preciso um critério para escolher um ponto dentro dele.

O critério adotado aqui é o que Hilpisch chama de *market-based valuation*: escolher um modelo e ajustar seus parâmetros para que ele reproduza os preços de opções líquidas negociadas, e então usar o modelo ajustado para precificar e fazer o hedge de contratos menos líquidos. Em português, falaremos em **precificação calibrada ao mercado**. Este capítulo apresenta o vocabulário (payoff, carteira, arbitragem, replicação, completude, calibração) e um mapa de como o resto do livro implementa cada etapa desse processo.

Fundamentos teóricos

Esta seção fixa a linguagem usada no resto do livro. O ponto de partida é um mercado com finitos ativos negociados; a pergunta é quando um preço para um contrato novo é *consistente* com os preços já observados. A resposta, a ausência de arbitragem, organiza

toda a teoria de precificação (Harrison and Kreps, 1979; Björk, 2009).

Mercado, carteiras e arbitragem

Definição 1.1 (Mercado de um período). Seja $\Omega = \{\omega_1, \dots, \omega_K\}$ um conjunto finito de estados com probabilidade $\mathbb{P}(\omega_k) > 0$. Há $N+1$ ativos com preços hoje S_0^i e preços aleatórios $S_1^i(\omega)$ em $t = 1$, $i = 0, \dots, N$. O ativo 0 é a conta remunerada: $S_0^0 = 1$ e $S_1^0 = 1+r$, com $r > -1$. Uma *carteira* é um vetor $\theta \in \mathbb{R}^{N+1}$; seu custo é $V_0(\theta) = \sum_i \theta_i S_0^i$ e seu valor final é $V_1(\theta) = \sum_i \theta_i S_1^i$.

Definição 1.2 (Arbitragem). Uma *arbitragem* é uma carteira com $V_0(\theta) \le 0$, $V_1(\theta) \ge 0$ em todos os estados e $\mathbb{P}(V_1(\theta) > 0) > 0$. O mercado é *livre de arbitragem* se nenhuma carteira desse tipo existe.

Proposição 1.3 (Lei do preço único). *Num mercado livre de arbitragem, duas carteiras com o mesmo valor final em todos os estados têm o mesmo custo hoje.*

Demonstração. Se $V_1(\theta) = V_1(\theta')$ e $V_0(\theta) < V_0(\theta')$, a carteira $\theta - \theta'$, combinada com a aplicação de $V_0(\theta') - V_0(\theta) > 0$ na conta remunerada, custa zero e paga $(1+r)(V_0(\theta') - V_0(\theta)) > 0$ em todos os estados: arbitragem. ∎

Definição 1.4 (Direito contingente e replicação). Um *direito contingente* (ou derivativo) é uma variável aleatória $X = f(\omega)$ paga em $t = 1$. Ele é *replicável* se existe θ com $V_1(\theta) = X$ em todos os estados. O mercado é *completo* se todo direito contingente é replicável.

Corolário 1.5 (Preço por replicação). *Se X é replicável por θ e o mercado é livre de arbitragem, o único preço de X que não cria arbitragem é $V_0(\theta)$.*

O corolário é a forma mais simples de *precificação calibrada ao mercado*: o preço de X é o custo de reproduzi-lo com o que o mercado já precifica. Quando o mercado é incompleto, a replicação deixa de existir para alguns contratos e passa a haver um intervalo de preços livres de arbitragem; o modelo escolhe um ponto nesse intervalo, e é por isso que ele precisa ser *calibrado*.

Modelo, calibração e risco de modelo

Definição 1.6 (Modelo paramétrico e calibração). Um *modelo* é uma família de regras de preço $\{\Pi(\cdot;\vartheta) : \vartheta \in \Theta\}$, $\Theta \subset \mathbb{R}^p$, livres de arbitragem para cada ϑ. Dados preços de mercado $C_1^{\mathrm{mkt}}, \dots, C_m^{\mathrm{mkt}}$ de instrumentos líquidos $X_1, \dots, X_m$, *calibrar* é resolver

$$\hat{\vartheta} \in \arg\min_{\vartheta \in \Theta} \sum_{j=1}^{m} w_j \left(\Pi(X_j;\vartheta) - C_j^{\mathrm{mkt}}\right)^2, \tag{1.1}$$

com pesos $w_j > 0$. O preço de um contrato novo Y é então $\Pi(Y;\hat{\vartheta})$.

Observação 1.7 (Risco de modelo). Dois modelos que reproduzem os mesmos C_j^{mkt} podem dar preços diferentes para Y. A diferença mede o *risco de modelo* do contrato. Ela é nula quando Y é replicável pelos instrumentos de calibração e tende a crescer quando Y depende de aspectos (dinâmica do smile, trajetória, exercício antecipado) que os X_j não revelam.

Paridade put-call

Proposição 1.8 (Paridade put-call). *Sejam C e P uma call e uma put europeias de mesmo strike K e vencimento T sobre um ativo sem dividendos, e $B(0,T)$ o preço de um título que paga 1 em T. Na ausência de arbitragem,*

$$C - P = S_0 - K\,B(0,T). \tag{1.2}$$

Demonstração. A carteira *comprar a call e vender a put* paga $\max(S_T - K, 0) - \max(K - S_T, 0) = S_T - K$ em qualquer estado. A carteira *comprar o ativo e tomar emprestado* $K\,B(0,T)$ paga o mesmo. Pela Proposição 1.3, os custos são iguais. ■

A paridade não depende de nenhum modelo: é uma restrição que qualquer modelo livre de arbitragem precisa respeitar, e é o primeiro teste aplicado aos laboratórios.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulos 1 e 2 do livro.

O que você vai conseguir fazer ao final

1. Escrever o payoff de calls e puts, e dizer quem paga o quê a quem.
2. Explicar por que o valor de uma opção **não** é a média simples do que ela pode pagar.
3. Calcular o valor de uma opção europeia por simulação e pela fórmula de Black-Scholes-Merton (BSM), e conferir uma contra a outra.
4. Separar valor intrínseco de valor no tempo e saber quando este último pode ser negativo.
5. Listar os riscos que movem o preço de uma opção e medir o efeito de cada um.
6. Mostrar, com números, por que o hedge dinâmico funciona num mundo sem saltos e falha num mundo com saltos.
7. Executar, em miniatura, as seis etapas do processo que o resto do curso aprofunda.

Como estudar este notebook

Leia cada bloco de teoria **antes** de rodar a célula seguinte, e tente prever o resultado. Cada laboratório termina com verificações (`ok(...)`): se o notebook roda até o fim, todas as contas foram conferidas.

Neste módulo algumas ferramentas aparecem como **caixa-preta** (a fórmula BSM, a dinâmica lognormal). Isso é proposital: primeiro vemos o que elas fazem, depois as deduzimos do zero nos módulos 3 a 5.

Mapa do curso

Módulo	Tema	Livro
1	**Mercados, arbitragem e precificação de derivativos**	**Caps. 1-2**
2	Stylized facts e volatilidade implícita	Cap. 3
3	A medida neutra ao risco em tempo discreto	Cap. 4
4	Cálculo estocástico: browniano, Itô, Girsanov	Cap. 4
5	BSM: a EDP, a fórmula e as gregas	Cap. 5
6	Árvore binomial CRR e convergência	Cap. 5
7	Métodos de Fourier I: transformada e fórmula de Lewis	Cap. 6
8	Métodos de Fourier II: Carr-Madan, FFT e convolução	Cap. 6
9	Opções americanas: Longstaff-Schwartz e o problema dual	Cap. 7
10	Calibração ao mercado com o modelo de saltos de Merton	Cap. 8
11	O modelo BCC97: Heston + saltos + CIR	Cap. 9
12	Monte Carlo: discretização e redução de variância	Cap. 10
13	Calibração I: curva de juros (CIR) e Heston	Cap. 11
14	Calibração II: saltos e BCC97 completo	Cap. 11
15	Simulação e precificação no modelo BCC97	Cap. 12
16	Hedge dinâmico e síntese do curso	Caps. 13-14
17	Complemento: Hull-White I — medidas, modelo e fórmulas fechadas	—
18	Complemento: Hull-White II — árvore trinomial	—
19	Complemento: simulação de trajetórias de juros, do Vasicek ao HJM	—

Módulo	Tema	Livro
20	Complemento: árvores implícitas de Derman-Kani-Chriss (interativo)	—

As ferramentas do Apêndice A do livro (regressão, splines, otimização, integração) entram no módulo em que são usadas pela primeira vez.

0. Preparação

A célula abaixo importa as bibliotecas e define três utilitários: `f3` (número com três casas decimais e vírgula), `tabela` (mostra um DataFrame nesse formato) e `ok` (verificação com contagem).

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, poisson
from scipy.optimize import brentq, least_squares
from IPython.display import display

print("Ambiente pronto. Exemplo de formato:", f3(1234.5678))
```

```
Ambiente pronto. Exemplo de formato: 1.234,568
```

---

1. O contrato: o que é uma opção

Livro: §2.1

Uma **opção** é um contrato que dá a quem compra um **direito**, sem obrigação:

- a **call** dá o direito de **comprar** o ativo-objeto por um preço combinado;
- a **put** dá o direito de **vender** o ativo-objeto por um preço combinado.

Quatro palavras aparecem o tempo todo:

Termo	Significado
Strike (K)	o preço combinado
Vencimento (T)	a data em que o direito acaba
Titular (*holder*)	quem comprou o direito e pagou o prêmio
Lançador (*writer*)	quem vendeu o direito e assumiu a obrigação

Quanto ao momento de exercício: a **europeia** só pode ser exercida no vencimento; a **americana**, a qualquer momento até o vencimento.

Opções sobre índice são **liquidadas financeiramente**: ninguém entrega "um índice". O lançador apenas paga a diferença em dinheiro.

Passo a passo do payoff

Considere uma call europeia sobre um índice, com $K = 8.000$.

Cenário 1: o índice fecha em 8.200. O titular tem o direito de "comprar" a 8.000 algo que vale 8.200. Ele exerce e recebe a diferença: $8.200 - 8.000 = 200$.

Cenário 2: o índice fecha em 7.800. Comprar a 8.000 algo que vale 7.800 seria perder 200. Como é um direito e não uma obrigação, ele não exerce e recebe zero.

Os dois cenários cabem numa única expressão, o **payoff** (ou valor intrínseco no vencimento):

$$h_T^{\text{call}} = \max(S_T - K,\ 0) \tag{1.3}$$

Para a put, o raciocínio é espelhado:

$$h_T^{\text{put}} = \max(K - S_T,\ 0) \tag{1.4}$$

Repare em duas propriedades que vão importar muito:

1. O payoff **nunca é negativo** para o titular. É por isso que ele paga um prêmio para entrar no contrato.
2. O payoff tem uma **quina** em $S_T = K$. Ele é uma função **convexa**, e veremos que essa convexidade é a origem do valor da volatilidade.

```
def payoff_call(S, K):
    return np.maximum(S - K, 0.0)


def payoff_put(S, K):
    return np.maximum(K - S, 0.0)


K = 8000.0
ok(payoff_call(8200.0, K) == 200.0, "call com índice em 8.200 paga 200")
ok(payoff_call(7800.0, K) == 0.0, "call com índice em 7.800 paga zero")
ok(payoff_put(7800.0, K) == 200.0, "put com índice em 7.800 paga 200")
```

```
OK — call com índice em 8.200 paga 200
OK — call com índice em 7.800 paga zero
OK — put com índice em 7.800 paga 200
```

Laboratório 1.1 — As quatro posições básicas

Uma opção é um **jogo de soma zero** entre as duas pontas: o que o titular recebe, o lançador paga. Comprar e vender calls e puts gera quatro perfis. Antes de rodar, tente desenhar os quatro de cabeça.

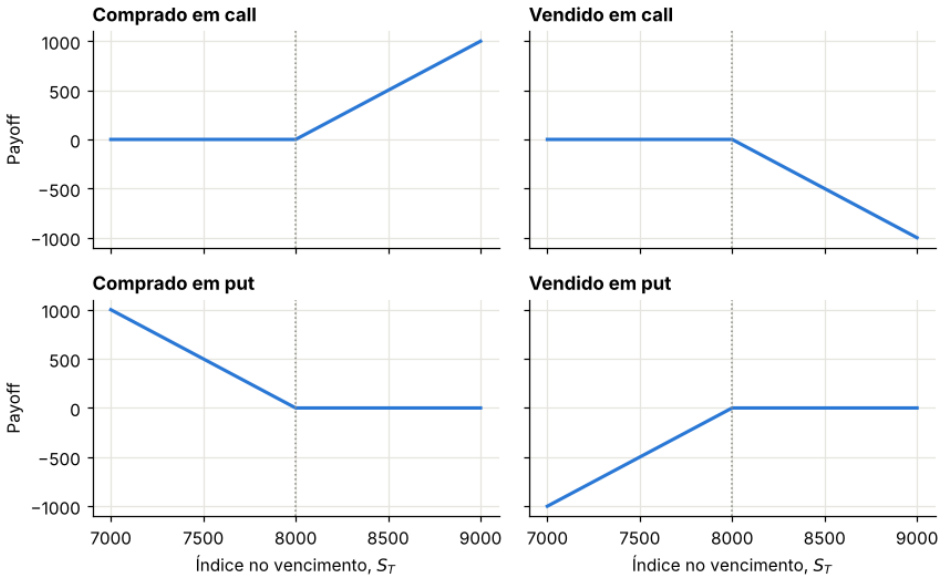
Os gráficos mostram só o payoff no vencimento, sem considerar o prêmio pago ou recebido.

```
S = np.linspace(7000, 9000, 401)
perfis = {
    "Comprado em call": payoff_call(S, K),
```

```
    "Vendido em call": -payoff_call(S, K),
    "Comprado em put": payoff_put(S, K),
    "Vendido em put": -payoff_put(S, K),
}

fig, eixos = plt.subplots(2, 2, figsize=(8, 5), sharex=True, sharey=True)
for ax, (nome, h) in zip(eixos.ravel(), perfis.items()):
    ax.plot(S, h, color=AZUL)
    ax.axvline(K, color=CINZA, lw=1, ls=":")
    ax.set_title(nome)
for ax in eixos[1]:
    ax.set_xlabel("Índice no vencimento, $S_T$")
for ax in eixos[:, 0]:
    ax.set_ylabel("Payoff")
plt.tight_layout()
plt.show()

ok(np.allclose(perfis["Comprado em call"] + perfis["Vendido em call"], 0), "soma zero entre
↪  titular e lançador da call")
ok(np.allclose(perfis["Comprado em put"] + perfis["Vendido em put"], 0), "soma zero entre titular
↪  e lançador da put")
```




```
OK — soma zero entre titular e lançador da call
OK — soma zero entre titular e lançador da put
```

Moneyness: onde a opção está em relação ao strike

Situação	Call	Put
Dentro do dinheiro (ITM)	$S > K$	$S < K$
No dinheiro (ATM)	$S \approx K$	$S \approx K$
Fora do dinheiro (OTM)	$S < K$	$S > K$

Laboratório 1.2 — Uma identidade que vale em qualquer cenário

Compre uma call e venda uma put com o mesmo strike. O payoff combinado é

$$\max(S_T - K, 0) - \max(K - S_T, 0). \tag{1.5}$$

Vamos analisar os dois casos possíveis:

- Se $S_T \geq K$: a call paga $S_T - K$, a put paga zero. Total: $S_T - K$.
- Se $S_T < K$: a call paga zero, a put paga $K - S_T$, e como estamos vendidos nela o total é $-(K - S_T) = S_T - K$.

Nos dois casos o resultado é **exatamente** $S_T - K$, ou seja, o mesmo de um contrato a termo. A quina desaparece. Essa identidade dá origem à **paridade put-call**, que usaremos na Seção 6.

```
sinteticos = payoff_call(S, K) - payoff_put(S, K)
ok(np.allclose(sinteticos, S - K), "call comprada + put vendida = S_T - K em todos os cenários")
```

```
OK — call comprada + put vendida = S_T - K em todos os cenários
```

---

2. Por que o valor não é a média do payoff

Livro: §2.1 (pergunta) e cap. 4 (resposta completa)

A primeira ideia de quase todo mundo é: "o valor da opção é a média do que ela pode pagar, trazida a valor presente." Vamos testar essa ideia no menor mundo possível.

Um mundo com dois cenários

Hoje o índice vale $S_0 = 100$. Daqui a um período ele vale $S_u = 120$ ou $S_d = 80$. A taxa de juros é zero, para simplificar. Temos uma call com $K = 100$:

Cenário	Índice	Payoff da call
Alta	120	20
Baixa	80	0

Se a probabilidade de alta for p, a média do payoff é $20p$. Quanto vale então a call? Com $p = 50\%$ daria 10; com $p = 90\%$ daria 18. Parece razoável.

Mas existe outro caminho: copiar a opção usando ativos que já têm preço.

Passo 1: montar uma carteira que paga o mesmo

Compramos Δ unidades do índice e colocamos B na renda fixa (B negativo significa pegar emprestado). Queremos que a carteira pague o mesmo que a call nos dois cenários:

$$120\,\Delta + B = 20 \qquad \text{(alta)} \tag{1.6}$$

$$80\,\Delta + B = 0 \qquad \text{(baixa)} \tag{1.7}$$

Passo 2: resolver o sistema

Subtraindo a segunda equação da primeira, B some:

$$40\,\Delta = 20 \quad \Rightarrow \quad \Delta = 0{,}5 \tag{1.8}$$

Substituindo na segunda equação: $80 \times 0{,}5 + B = 0$, logo $B = -40$.

Passo 3: ver quanto custa essa carteira hoje

$$V_0 = \Delta\, S_0 + B = 0{,}5 \times 100 - 40 = 10 \tag{1.9}$$

Passo 4: a conclusão

A carteira paga exatamente o mesmo que a call em todos os cenários e custa 10. Logo, **a call tem que valer 10**. Se valesse outra coisa, haveria lucro sem risco.

Note o que **não** usamos: a probabilidade p. O valor não depende dela.

E o que é a probabilidade neutra ao risco?

Existe um número q que faz a média ponderada reproduzir o preço correto. Basta exigir que o índice, em média, renda a taxa livre de risco:

$$q\,S_u + (1-q)\,S_d = (1+r)\,S_0 \quad \Rightarrow \quad q = \frac{(1+r)\,S_0 - S_d}{S_u - S_d} \tag{1.10}$$

Aqui, $q = (100 - 80)/40 = 0{,}5$. Com ele, $V_0 = [q \times 20 + (1-q) \times 0]/(1+r) = 10$.

q não é uma crença sobre o futuro. É um **peso contábil** que resume o custo de replicação. O módulo 3 transforma essa ideia em teorema.

```python
def replicar(S0, Su, Sd, K, r):
    """Replica uma call num mundo de dois cenários e devolve a carteira e o valor."""
    Cu, Cd = max(Su - K, 0.0), max(Sd - K, 0.0)
    delta = (Cu - Cd) / (Su - Sd)
    B = (Cd - delta * Sd) / (1 + r)
    valor = delta * S0 + B
    q = ((1 + r) * S0 - Sd) / (Su - Sd)
    return {"delta": delta, "B": B, "valor": valor, "q": q, "Cu": Cu, "Cd": Cd}


m = replicar(100.0, 120.0, 80.0, 100.0, 0.0)
linhas = []
for p in [0.10, 0.50, 0.90]:
    linhas.append({
        "Prob. de alta p": p,
        "Média simples do payoff": p * m["Cu"] + (1 - p) * m["Cd"],
        "Custo de replicação": m["valor"],
    })
tabela(pd.DataFrame(linhas))

print("Delta:", f3(m["delta"]), "| B:", f3(m["B"]), "| q:", f3(m["q"]))
ok(abs(m["valor"] - 10) < 1e-12, "custo de replicação = 10")
ok(abs((m["q"] * m["Cu"] + (1 - m["q"]) * m["Cd"]) - m["valor"]) < 1e-12, "média sob q reproduz o
↪  custo de replicação")
```

Prob. de alta p	Média simples do payoff	Custo de replicação
0,100	2,000	10,000
0,500	10,000	10,000
0,900	18,000	10,000

```
Delta: 0,500 | B: -40,000 | q: 0,500
OK — custo de replicação = 10
OK — média sob q reproduz o custo de replicação
```

Laboratório 2.1 — Executando a arbitragem

Suponha que alguém, acreditando em $p = 90\%$, aceite pagar 18 pela call. Vendemos a call a 18 e compramos a carteira replicante por 10. O caixa hoje é +8. No vencimento, a carteira paga exatamente o que devemos. Vamos conferir nos dois cenários.

```
preco_venda = 18.0
caixa_hoje = preco_venda - m["valor"]

linhas = []
for nome, St, c in [("Alta", 120.0, m["Cu"]), ("Baixa", 80.0, m["Cd"])]:
    carteira = m["delta"] * St + m["B"]
    linhas.append({"Cenário": nome, "Carteira recebe": carteira, "Call obriga a pagar": c, "Saldo
↪  no vencimento": carteira - c})
df = pd.DataFrame(linhas)
tabela(df)
print("Caixa embolsado hoje:", f3(caixa_hoje))
ok(caixa_hoje > 0 and np.allclose(df["Saldo no vencimento"], 0), "lucro hoje e saldo zero em
↪  qualquer cenário: arbitragem")
```

Cenário	Carteira recebe	Call obriga a pagar	Saldo no vencimento
Alta	20,000	20,000	0,000
Baixa	0,000	0,000	0,000

```
Caixa embolsado hoje: 8,000
OK — lucro hoje e saldo zero em qualquer cenário: arbitragem
```

---

3. O valor hoje de uma opção europeia

Livro: §2.1 e fórmula (5.7) do cap. 5

Passamos agora de dois cenários para infinitos. A receita continua a mesma: **média sob a medida neutra ao risco, descontada pela taxa livre de risco**,

$$C_0 = e^{-rT}\,\mathbb{E}^{Q}\big[\max(S_T - K, 0)\big]. \tag{1.11}$$

Precisamos saber como S_T se distribui. O modelo de referência (benchmark) é o lognormal.

Passo 1: a hipótese

Supomos que o log-retorno até T é normal:

$$S_T = S_0 \exp\left[(r - \tfrac{1}{2}\sigma^2)T + \sigma\sqrt{T}\,Z\right], \qquad Z \sim N(0,1). \tag{1.12}$$

Passo 2: de onde vem aquele $-\frac{1}{2}\sigma^2$?

Sob Q, o índice precisa render r em média, ou seja, $\mathbb{E}^Q[S_T] = S_0 e^{rT}$. Ingenuamente, escreveríamos $S_T = S_0 e^{rT+\sigma\sqrt{T}Z}$. Mas a exponencial é convexa, e a média da exponencial é maior que a exponencial da média. Vamos calcular exatamente quanto, para a constante:

$$\mathbb{E}[e^{aZ}] = \int_{-\infty}^{\infty} e^{az}\,\frac{1}{\sqrt{2\pi}} e^{-z^2/2}\,dz. \tag{1.13}$$

Juntando os expoentes e **completando o quadrado**, $az - \frac{1}{2}z^2 = -\frac{1}{2}(z-a)^2 + \frac{1}{2}a^2$:

$$\mathbb{E}[e^{aZ}] = e^{a^2/2} \underbrace{\int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-(z-a)^2/2}\,dz}_{=1 \text{ (densidade de uma } N(a,1))} = e^{a^2/2}. \tag{1.14}$$

Com $a = \sigma\sqrt{T}$, obtemos $\mathbb{E}[e^{\sigma\sqrt{T}Z}] = e^{\sigma^2 T/2}$. Esse fator extra precisa ser cancelado, e é isso que o termo $-\frac{1}{2}\sigma^2 T$ faz:

$$\mathbb{E}^Q[S_T] = S_0\, e^{rT}\, e^{-\sigma^2 T/2}\, e^{\sigma^2 T/2} = S_0\, e^{rT}. \tag{1.15}$$

Laboratório 3.1 — Conferindo a média por simulação

```
rng = np.random.default_rng(20150211)


def simular_ST(S0, r, sigma, T, n, rng):
    Z = rng.standard_normal(n)
    return S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)


S0, K, T, r, sigma = 8000.0, 8000.0, 1.0, 0.025, 0.20
ST = simular_ST(S0, r, sigma, T, 1_000_000, rng)

teorico = S0 * np.exp(r * T)
sem_ajuste = S0 * np.exp(r * T + 0.5 * sigma**2 * T)
erro_padrao = ST.std(ddof=1) / np.sqrt(ST.size)

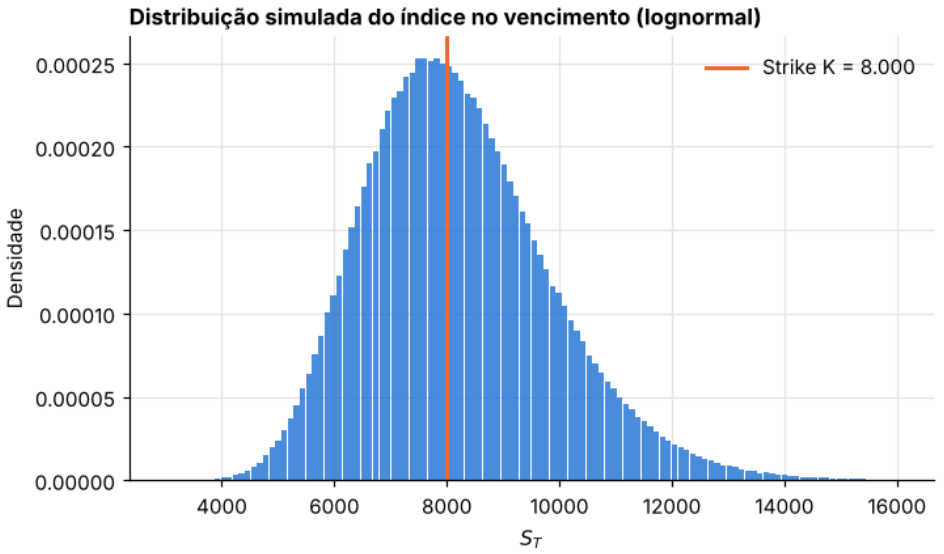
tabela(pd.DataFrame([
    {"Quantidade": "Média simulada de S_T", "Valor": ST.mean()},
    {"Quantidade": "Teoria: S0·exp(rT)", "Valor": teorico},
    {"Quantidade": "O que daria sem o termo −σ²/2", "Valor": sem_ajuste},
    {"Quantidade": "Erro-padrão da simulação", "Valor": erro_padrao},
]))
ok(abs(ST.mean() - teorico) < 4 * erro_padrao, "média simulada compatível com S0·exp(rT)")

fig, ax = plt.subplots()
ax.hist(ST, bins=120, range=(3000, 16000), density=True, color=AZUL, alpha=0.85, rwidth=0.92)
ax.axvline(K, color=LARANJA, lw=2, label=f"Strike K = {K:,.0f}".replace(",", "."))
ax.set_title("Distribuição simulada do índice no vencimento (lognormal)")
ax.set_xlabel("$S_T$")
ax.set_ylabel("Densidade")
```

```
ax.legend()
plt.show()
```

Quantidade	Valor
Média simulada de S_T	8.203,257
Teoria: S0·exp(rT)	8.202,521
O que daria sem o termo $-\sigma^2/2$	8.368,223
Erro-padrão da simulação	1,657

```
OK — média simulada compatível com S0·exp(rT)
```



Observe a assimetria: a distribuição tem cauda longa à direita. O payoff da call corresponde à parte à direita da linha laranja.

Passo 3: o estimador de Monte Carlo

Sorteando I cenários, a média dos payoffs descontados aproxima o valor:

$$\hat{C}_0 = e^{-rT}\,\frac{1}{I}\sum_{i=1}^{I}\max(S_T^{(i)} - K, 0), \qquad \text{erro-padrão} = \frac{\hat{s}}{\sqrt{I}}. \tag{1.16}$$

A divisão por $\sqrt{I}$ é o preço do método: para reduzir o erro à metade, é preciso **quadruplicar** o número de cenários.

Passo 4: a fórmula fechada de Black-Scholes-Merton

No modelo lognormal, aquela integral tem solução exata:

$$C_0 = S_0\,N(d_1) - Ke^{-rT}N(d_2) \tag{1.17}$$

$$d_1 = \frac{\ln(S_0/K) + (r + \tfrac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T} \tag{1.18}$$

em que $N(\cdot)$ é a função de distribuição da normal padrão. Por enquanto é uma caixa-preta; o módulo 5 a deduz por dois caminhos. Mesmo assim, já dá para ler a estrutura:

Termo	Leitura
$N(d_2)$	probabilidade, sob Q, de a opção terminar dentro do dinheiro
$Ke^{-rT}N(d_2)$	valor presente do que se espera **pagar** no exercício
$S_0 N(d_1)$	valor presente do que se espera **receber** de índice, quando há exercício

A put vem da paridade, que deduziremos na Seção 6: $P_0 = Ke^{-rT}N(-d_2) - S_0 N(-d_1)$.

Laboratório 3.2 — Monte Carlo contra a fórmula

```
def bsm(S, K, T, r, sigma, tipo="call"):
    """Valor BSM de call ou put europeia. Aceita arrays em S e K."""
    S, K = np.asarray(S, float), np.asarray(K, float)
    raiz = sigma * np.sqrt(T)
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / raiz
    d2 = d1 - raiz
    if tipo == "call":
        return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)


linhas = []
desconto = np.exp(-r * T)
for k in [6000.0, 7000.0, 8000.0, 9000.0, 10000.0]:
    amostra = desconto * payoff_call(ST, k)
    mc = amostra.mean()
    ep = amostra.std(ddof=1) / np.sqrt(amostra.size)
    fechada = float(bsm(S0, k, T, r, sigma))
    linhas.append({"Strike": k, "Monte Carlo": mc, "Erro-padrão": ep, "BSM": fechada, "Diferença
↪   em erros-padrão": (mc - fechada) / ep})
df = pd.DataFrame(linhas)
tabela(df)
ok((df["Diferença em erros-padrão"].abs() < 4).all(), "Monte Carlo e BSM coincidem dentro de 4
↪   erros-padrão")

d2 = (np.log(S0 / K) + (r - 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
freq = (ST > K).mean()
print("Frequência simulada de S_T > K:", f3(freq), "| N(d2):", f3(norm.cdf(d2)))
ok(abs(freq - norm.cdf(d2)) < 0.002, "N(d2) é a probabilidade neutra ao risco de exercício")
```

Strike	Monte Carlo	Erro-padrão	BSM	Diferença em erros-padrão
6.000,000	2.183,426	1,560	2.182,666	0,487
7.000,000	1.353,073	1,396	1.352,288	0,563
8.000,000	733,489	1,116	733,033	0,409
9.000,000	349,737	0,800	349,675	0,078
10.000,000	149,351	0,526	149,335	0,030

```
OK — Monte Carlo e BSM coincidem dentro de 4 erros-padrão
Frequência simulada de S_T > K: 0,511 | N(d2): 0,510
OK — N(d2) é a probabilidade neutra ao risco de exercício
```

O que move o valor hoje

São quatro fatores de mercado, além do strike:

1. **Nível do índice.** Quanto mais alto, mais provável terminar acima do strike.
2. **Volatilidade.** Maior oscilação aumenta o valor. Parece estranho, porque risco costuma ser ruim; a explicação vem logo abaixo.
3. **Prazo.** Mais tempo, mais chance de o índice ultrapassar o strike.
4. **Taxa de juros.** Essa exige cuidado, e o Laboratório 3.4 mostra por quê.

Por que volatilidade aumenta o valor: a convexidade passo a passo

Considere uma call com $K = 100$ e um índice que fecha em $100 + a$ ou $100 - a$, com 50% de chance cada. A média do índice é sempre 100, seja qual for a. O payoff médio é:

$$\tfrac{1}{2}\max(a, 0) + \tfrac{1}{2}\max(-a, 0) = \tfrac{a}{2}. \tag{1.19}$$

Dobrar a dispersão dobra o payoff médio, **sem alterar a média do índice**. O motivo é a assimetria do contrato: o titular fica com os ganhos da alta e não paga pelas perdas da baixa. Em linguagem matemática, para função convexa h, a desigualdade de Jensen dá $\mathbb{E}[h(S)] \geq h(\mathbb{E}[S])$, e a diferença cresce com a dispersão.

Laboratório 3.3 — Os quatro fatores

Variamos um fator de cada vez a partir de um caso-base com $S_0 = 7.900$, $K = 8.000$, $T = 1$, $r = 2{,}5\%$, $\sigma = 20\%$.

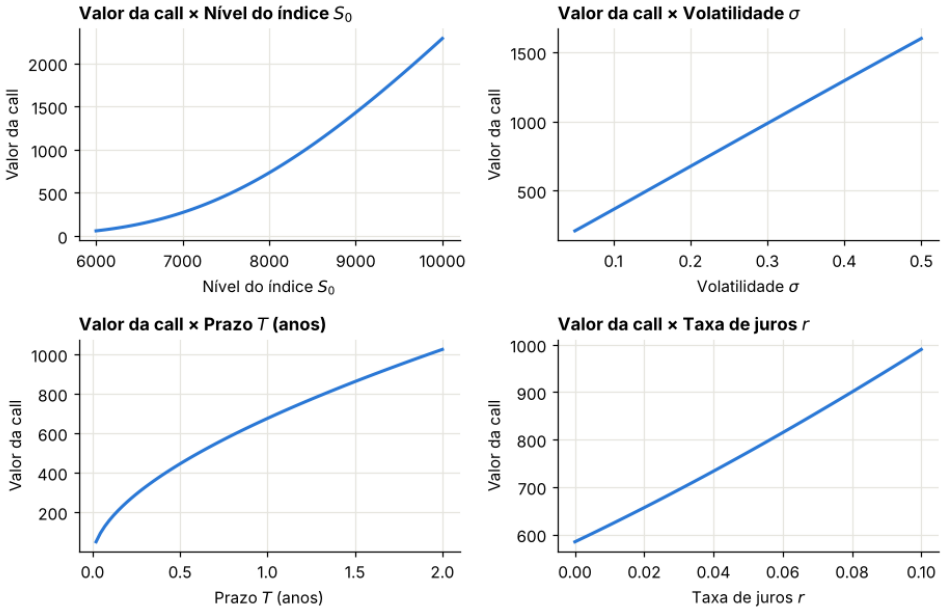
```python
base = dict(S=7900.0, K=8000.0, T=1.0, r=0.025, sigma=0.20)


def varia(par, grade):
    return np.array([float(bsm(**{**base, par: v})) for v in grade])


grades = {
    "S": (np.linspace(6000, 10000, 80), "Nível do índice $S_0$"),
    "sigma": (np.linspace(0.05, 0.50, 80), "Volatilidade $\\sigma$"),
    "T": (np.linspace(0.02, 2.0, 80), "Prazo $T$ (anos)"),
    "r": (np.linspace(0.0, 0.10, 80), "Taxa de juros $r$"),
}
```

```
fig, eixos = plt.subplots(2, 2, figsize=(8.5, 5.6), sharey=False)
for ax, (par, (grade, rotulo)) in zip(eixos.ravel(), grades.items()):
    valores = varia(par, grade)
    ax.plot(grade, valores, color=AZUL)
    ax.set_title(f"Valor da call × {rotulo}")
    ax.set_xlabel(rotulo)
    ax.set_ylabel("Valor da call")
    ok(np.all(np.diff(valores) > 0), f"call é crescente em {par}")
plt.tight_layout()
plt.show()
```

```
OK — call é crescente em S
OK — call é crescente em sigma
OK — call é crescente em T
OK — call é crescente em r
```



Laboratório 3.4 — O efeito dos juros tem duas partes

É comum dizer apenas que "os juros descontam o payoff". Se fosse só isso, juros mais altos **reduziriam** o valor da call, mas o gráfico acima mostrou o contrário. Há dois efeitos em sentidos opostos:

$$C_0 = \underbrace{e^{-rT}}_{\text{desconto: cai com } r} \times \underbrace{\mathbb{E}^Q[\max(S_T - K, 0)]}_{\text{cresce com } r,\ \text{pois } \mathbb{E}^Q[S_T] = S_0 e^{rT}} \tag{1.20}$$

Vamos separar os efeitos ao subir r de 2,5% para 5%.

```
r0, r1 = 0.025, 0.05
args = dict(S=base["S"], K=base["K"], T=base["T"], sigma=base["sigma"])
```

```python
for tipo in ["call", "put"]:
    v0 = float(bsm(r=r0, tipo=tipo, **args))
    v1 = float(bsm(r=r1, tipo=tipo, **args))
    so_desconto = v0 * np.exp(-(r1 - r0) * args["T"])
    efeito_desc = so_desconto - v0
    efeito_dist = v1 - so_desconto
    tabela(pd.DataFrame([{
        "Opção": tipo, "Valor com r=2,5%": v0, "Efeito do desconto": efeito_desc,
        "Efeito da distribuição": efeito_dist, "Valor com r=5%": v1,
    }]))
    if tipo == "call":
        ok(efeito_desc < 0 < efeito_dist and v1 > v0, "na call, o efeito da distribuição supera o
↪  do desconto")
    else:
        ok(efeito_desc < 0 and efeito_dist < 0 and v1 < v0, "na put, os dois efeitos reduzem o
↪  valor")
```

Opção	Valor com r=2,5%	Efeito do desconto	Efeito da distribuição	Valor com r=5%
call	675,358	-16,675	114,866	773,549

```
OK — na call, o efeito da distribuição supera o do desconto
```

Opção	Valor com r=2,5%	Efeito do desconto	Efeito da distribuição	Valor com r=5%
put	577,837	-14,267	-80,185	483,385

```
OK — na put, os dois efeitos reduzem o valor
```

Valor intrínseco e valor no tempo

O valor de uma opção se decompõe em:

$$\text{Valor} = \underbrace{\max(S_0 - K, 0)}_{\text{valor intrínseco}} + \underbrace{(\text{diferença})}_{\text{valor no tempo}} \tag{1.21}$$

O livro afirma que o valor presente fica sempre acima do intrínseco. Para a **call sem dividendos e com** $r \geq 0$, isso é correto, e dá para demonstrar sem modelo algum.

Passo 1. Monte duas carteiras hoje:

- **A:** uma call + Ke^{-rT} em renda fixa.
- **B:** uma unidade do índice.

Passo 2. Compare os valores em T. A renda fixa vira K, então:

Cenário	Carteira A	Carteira B
$S_T \geq K$	$(S_T - K) + K = S_T$	S_T
$S_T < K$	$0 + K = K > S_T$	S_T

Passo 3. A carteira A paga sempre pelo menos o mesmo que B, logo não pode custar menos: $C_0 + Ke^{-rT} \geq S_0$.

Passo 4. Portanto $C_0 \geq S_0 - Ke^{-rT} \geq S_0 - K$, usando $e^{-rT} \leq 1$. Combinado com $C_0 \geq 0$, a call vale pelo menos o intrínseco.

A afirmação não vale para a put europeia. Bem dentro do dinheiro, o titular gostaria de receber K logo para aplicar a juros, mas a europeia o obriga a esperar. O laboratório abaixo mostra valor no tempo **negativo**.

> **Errata do livro:** a legenda da Figura 2.2 diz $K = 9.000$, mas o código e o próprio gráfico usam $K = 8.000$.

Laboratório 3.5 — Valor intrínseco, valor no tempo e o caso da put

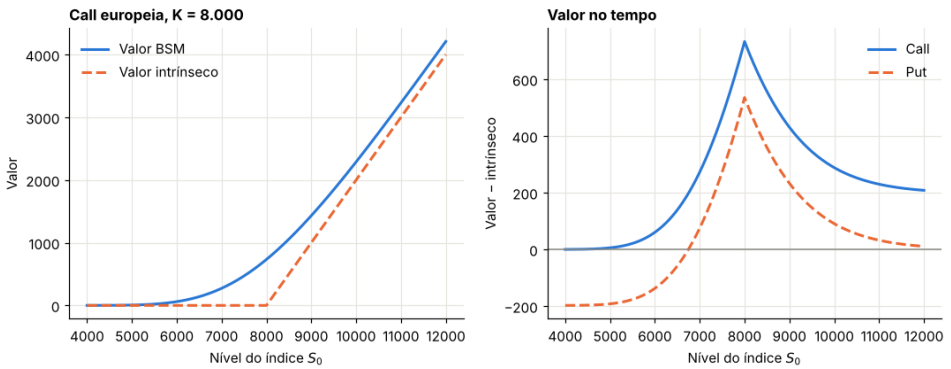
```
K, T, r, sigma = 8000.0, 1.0, 0.025, 0.20
S = np.linspace(4000, 12000, 321)

call = bsm(S, K, T, r, sigma, "call")
put = bsm(S, K, T, r, sigma, "put")
tempo_call = call - payoff_call(S, K)
tempo_put = put - payoff_put(S, K)

fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 4))
a1.plot(S, call, color=AZUL, label="Valor BSM")
a1.plot(S, payoff_call(S, K), color=LARANJA, ls="--", label="Valor intrínseco")
a1.set_title("Call europeia, K = 8.000")
a1.set_xlabel("Nível do índice $S_0$")
a1.set_ylabel("Valor")
a1.legend()

a2.plot(S, tempo_call, color=AZUL, label="Call")
a2.plot(S, tempo_put, color=LARANJA, ls="--", label="Put")
a2.axhline(0, color=CINZA, lw=1)
a2.set_title("Valor no tempo")
a2.set_xlabel("Nível do índice $S_0$")
a2.set_ylabel("Valor − intrínseco")
a2.legend()
plt.tight_layout()
plt.show()

ok(np.all(tempo_call > 0), "valor no tempo da call é sempre positivo (r > 0, sem dividendos)")
ok(tempo_put.min() < 0, "put europeia bem dentro do dinheiro tem valor no tempo negativo")
print("Pior valor no tempo da put:", f3(tempo_put.min()), "| limite teórico K(exp(-rT) − 1):",
↪  f3(K * (np.exp(-r * T) - 1)))
```



```
OK — valor no tempo da call é sempre positivo (r > 0, sem dividendos)
OK — put europeia bem dentro do dinheiro tem valor no tempo negativo
Pior valor no tempo da put: -197,400 | limite teórico K(exp(-rT) − 1): -197,521
```

---

4. Vanilla, exóticos e o que é "o mercado"

Livro: §2.2

Os instrumentos se dividem em dois grupos:

	Vanilla	Exótico
Exemplos	Call e put europeias, listadas	Barreira, asiática, lookback, cliente sob medida
Liquidez	Cotação frequente	Pouca ou nenhuma negociação
Preço	**Observado**	Precisa ser **calculado**

Aqui entra a ideia central do curso. Na precificação teórica, o caminho é:

$$\text{modelo} + \text{parâmetros} \longrightarrow \text{preços}. \tag{1.22}$$

Na **precificação calibrada ao mercado**, o caminho é invertido para os vanilla:

$$\text{preços observados dos vanilla} \longrightarrow \text{parâmetros (calibração)} \longrightarrow \text{preço do exótico}. \tag{1.23}$$

Exigência mínima: qualquer modelo usado para avaliar um exótico deve reproduzir bem os vanilla do mesmo ativo. É uma condição necessária, mas não suficiente, como a Seção 7 vai mostrar.

Ao escolher o modelo, há duas camadas de decisão:

- **Qualitativa:** que comportamento o processo precisa ter? Um índice de ações tende a crescer no longo prazo; juros e volatilidade tendem a voltar a um nível de equilíbrio.
- **Quantitativa:** dada a forma do modelo, quais parâmetros usar? Em finanças não há constantes universais como na física. Essa liberdade é justamente o que permite calibrar.

Laboratório 4.1 — Por que exóticos exigem o caminho inteiro

Três trajetórias terminam **no mesmo ponto**. Para uma opção europeia, são idênticas. Para exóticos, não. Isso explica por que modelos que acertam a distribuição em T podem errar exóticos: o exótico depende da distribuição **da trajetória**.

```python
caminhos = {
    "A: subida suave": np.array([100.0, 102.5, 105.0, 107.5, 110.0]),
    "B: queda forte e recuperação": np.array([100.0, 90.0, 75.0, 95.0, 110.0]),
    "C: alta forte e devolução": np.array([100.0, 120.0, 140.0, 125.0, 110.0]),
}
```

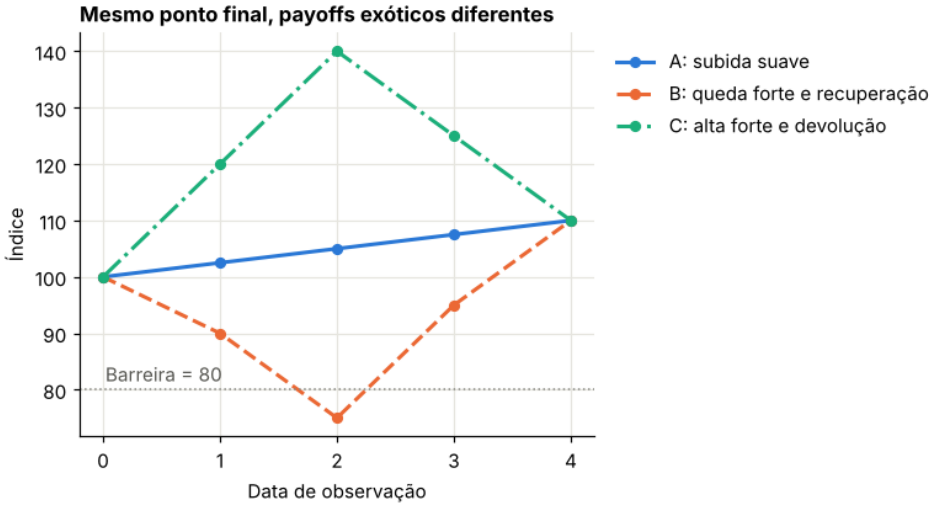
```
K_ex, H = 100.0, 80.0

linhas = []
for nome, p in caminhos.items():
    linhas.append({
        "Trajetória": nome,
        "S_T": p[-1],
        "Call europeia": max(p[-1] - K_ex, 0.0),
        "Barreira knock-out em 80": max(p[-1] - K_ex, 0.0) * float(p.min() > H),
        "Asiática (média)": max(p.mean() - K_ex, 0.0),
        "Lookback (S_T − mínimo)": p[-1] - p.min(),
    })
df = pd.DataFrame(linhas)
tabela(df)

fig, ax = plt.subplots()
for (nome, p), cor, estilo in zip(caminhos.items(), [AZUL, LARANJA, VERDE], ["-", "--", "-."]):
    ax.plot(p, color=cor, ls=estilo, marker="o", markersize=5, label=nome)
ax.axhline(H, color=CINZA, lw=1, ls=":")
ax.text(0.02, H + 1.5, "Barreira = 80", color="#555550")
ax.set_title("Mesmo ponto final, payoffs exóticos diferentes")
ax.set_xlabel("Data de observação")
ax.set_ylabel("Índice")
ax.set_xticks(range(5))
ax.legend(loc="upper left", bbox_to_anchor=(1.01, 1.0))
plt.tight_layout()
plt.show()

ok(df["Call europeia"].nunique() == 1, "a call europeia não distingue as trajetórias")
ok(df["Barreira knock-out em 80"].nunique() > 1 and df["Asiática (média)"].nunique() > 1, "os
↪  exóticos distinguem as trajetórias")
```

Trajetória	S_T	Call europeia	Barreira knock-out em 80	Asiática (média)	Lookback (S_T − mínimo)
A: subida suave	110,000	10,000	10,000	5,000	10,000
B: queda forte e recuperação	110,000	10,000	0,000	0,000	35,000
C: alta forte e devolução	110,000	10,000	10,000	19,000	10,000




```
OK — a call europeia não distingue as trajetórias
OK — os exóticos distinguem as trajetórias
```

5. Os riscos que movem uma opção

Livro: §2.3

Risco	O que é	Onde entra no curso
Preço	oscilação do índice	Delta; módulo 5
Volatilidade	a própria volatilidade é aleatória	Heston; módulos 11-14
Salto	quedas abruptas, que quebram o hedge	Merton; módulos 10 e 11
Juros	afetam desconto e drift	CIR; módulos 11 e 13
Correlação	co-movimentos que mudam em estresse	Parâmetro ρ do Heston; módulo 11
Liquidez	hedge pode não ser executável	Fora do escopo do livro
Crédito	default de emissor	Pouco relevante em índice; fora do escopo

Além desses, dois riscos não são de mercado: o **risco de modelo** (o modelo cobre o fator, mas de forma errada) e o **risco operacional** (a estratégia está certa, mas a execução falha).

Laboratório 5.1 — Reprecificação completa contra aproximação linear

Em risco, é tentador usar a aproximação de primeira ordem:

$$\Delta V \approx \frac{\partial V}{\partial S}\, dS + \frac{\partial V}{\partial \sigma}\, d\sigma + \frac{\partial V}{\partial r}\, dr + \frac{\partial V}{\partial t}\, dt. \tag{1.24}$$

Calculamos as derivadas por diferenças finitas e comparamos com a reprecificação completa, em cenários pequenos e grandes. Fique atento ao erro no cenário de crash.

```
b = dict(S=8000.0, K=8000.0, T=1.0, r=0.025, sigma=0.20)
v0 = float(bsm(**b))


def derivada(par, h):
    return float(bsm(**{**b, par: b[par] + h}) - bsm(**{**b, par: b[par] - h})) / (2 * h)


dV = {"S": derivada("S", 1.0), "sigma": derivada("sigma", 1e-4), "r": derivada("r", 1e-4), "T":
↪  derivada("T", 1e-4)}

cenarios = {
    "Índice −1%": dict(S=-80.0),
    "Índice −10%": dict(S=-800.0),
    "Volatilidade +5 p.p.": dict(sigma=0.05),
    "Juros +100 bps": dict(r=0.01),
    "Passa 1 mês": dict(T=-1 / 12),
```

```
    "Crash: índice −20%, vol +15 p.p.": dict(S=-1600.0, sigma=0.15),
}

linhas = []
for nome, choque in cenarios.items():
    novo = {k: b[k] + choque.get(k, 0.0) for k in b}
    completa = float(bsm(**novo)) - v0
    linear = sum(dV[k] * d for k, d in choque.items())
    linhas.append({"Cenário": nome, "Reprecificação completa": completa, "Aprox. linear": linear,
↪   "Erro da aprox.": linear - completa})
df = pd.DataFrame(linhas)
print("Valor inicial da call:", f3(v0))
tabela(df)

erro = df.set_index("Cenário")["Erro da aprox."].abs()
ok(erro["Crash: índice −20%, vol +15 p.p."] > 5 * erro["Índice −1%"], "o erro da aproximação
↪   linear explode no cenário de crash")
```

```
Valor inicial da call: 733,033
```

Cenário	Reprecificação completa	Aprox. linear	Erro da aprox.
Índice −1%	-46,337	-47,121	-0,783
Índice −10%	-389,352	-471,208	-81,856
Volatilidade +5 p.p.	155,621	155,588	-0,032
Juros +100 bps	40,365	39,790	-0,574
Passa 1 mês	-34,818	-34,221	0,597
Crash: índice −20%, vol +15 p.p.	-295,955	-475,652	-179,697

```
OK — o erro da aproximação linear explode no cenário de crash
```

Leitura. A aproximação linear vai bem para movimentos pequenos e piora muito em movimentos grandes, porque ignora a curvatura (gama) e as derivadas cruzadas. É por isso que stress de carteira de opções se faz com reprecificação completa.

---

6. Hedge: estático e dinâmico

Livro: §2.4

O lançador recebeu o prêmio e agora tem uma obrigação incerta. Ele quer montar algo que pague exatamente o que deve, zerando o risco. Há dois tipos de programa:

- **Estático:** monta-se no início e não se mexe mais, em geral usando outras opções.
- **Dinâmico:** ajusta-se a carteira ao longo do tempo, em geral usando o próprio ativo-objeto.

Hedge estático: a paridade put-call

Voltamos à identidade da Seção 1: call comprada + put vendida paga $S_T - K$ em qualquer cenário. Esse mesmo payoff se obtém com uma unidade do índice e um empréstimo de Ke^{-rT}. Dois investimentos com o mesmo payoff têm o mesmo custo, logo:

$$C_0 - P_0 = S_0 - Ke^{-rT}. \tag{1.25}$$

É uma relação **sem modelo**: não depende de volatilidade nem de lognormalidade.

Hedge dinâmico: o algoritmo

Vendemos uma call e recebemos o prêmio. Em cada data de rebalanceamento:

1. calculamos o delta $N(d_1)$ com o prazo restante;
2. compramos ou vendemos índice para chegar nesse delta, financiando pela conta-caixa;
3. a conta-caixa rende r até a próxima data.

No vencimento, zeramos tudo e pagamos o payoff. O saldo é o **erro de hedge** (P&L).

A teoria diz que, sob as hipóteses do BSM e com rebalanceamento contínuo, esse erro é **zero**. Vamos testar o que acontece com rebalanceamento discreto, e depois fora das hipóteses.

Laboratório 6.1 — Simulador de trajetórias e delta hedge

Criamos um único simulador que serve para os dois mundos, com e sem saltos. O número de saltos em cada passo é Poisson, com intensidade λ, e cada salto tem log-tamanho normal $N(\mu_J, \delta_J^2)$. O termo $\lambda\kappa$, com $\kappa = e^{\mu_J + \delta_J^2/2} - 1$, compensa o crescimento médio causado pelos saltos, para que o drift total continue igual a μ.

```
def simular_caminhos(S0, mu, sigma, T, M, I, rng, salto=None):
    """Trajetórias em M passos e I cenários. salto = (lambda, mu_J, delta_J) ou None."""
    dt = T / M
    Z = rng.standard_normal((M, I))
    lam, comp, J = 0.0, 0.0, 0.0
    if salto is not None:
        lam, mu_j, delta_j = salto
        comp = lam * (np.exp(mu_j + 0.5 * delta_j**2) - 1)
        N = rng.poisson(lam * dt, (M, I))
        J = N * mu_j + np.sqrt(N) * delta_j * rng.standard_normal((M, I))
    incrementos = (mu - comp - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z + J
    log_S = np.vstack([np.zeros(I), np.cumsum(incrementos, axis=0)])
    return S0 * np.exp(log_S)


def delta_call(S, K, tau, r, sigma):
    return norm.cdf((np.log(S / K) + (r + 0.5 * sigma**2) * tau) / (sigma * np.sqrt(tau)))


def pnl_delta_hedge(trajetorias, K, T, r, sigma_h, N):
    """P&L no vencimento de vender a call ao preço BSM e fazer delta hedge com N
    ↪ rebalanceamentos."""
    M = trajetorias.shape[0] - 1
    passo = M // N
    assert passo * N == M, "N precisa dividir M"
    S = trajetorias[::passo]
    dt = T / N

    delta = delta_call(S[0], K, T, r, sigma_h)
    caixa = float(bsm(S[0, 0], K, T, r, sigma_h)) - delta * S[0]
```

```
    for i in range(1, N):
        caixa *= np.exp(r * dt)
        novo = delta_call(S[i], K, T - i * dt, r, sigma_h)
        caixa -= (novo - delta) * S[i]
        delta = novo
    caixa *= np.exp(r * dt)
    return caixa + delta * S[N] - payoff_call(S[N], K)


S0, K, T, r, sigma = 100.0, 100.0, 1.0, 0.025, 0.20
M, I = 252, 40_000
Ns = [6, 12, 21, 42, 84, 252]

rng = np.random.default_rng(7)
mundo_bsm = simular_caminhos(S0, r + 0.06, sigma, T, M, I, rng)

linhas = []
for n in Ns:
    pnl = pnl_delta_hedge(mundo_bsm, K, T, r, sigma, n)
    linhas.append({"Rebalanceamentos N": n, "P&L médio": pnl.mean(), "Desvio-padrão do P&L":
↪   pnl.std(ddof=1), "Pior caso": pnl.min()})
res_bsm = pd.DataFrame(linhas)
print("Prêmio recebido:", f3(float(bsm(S0, K, T, r, sigma))))
tabela(res_bsm)

inclinacao = np.polyfit(np.log(res_bsm["Rebalanceamentos N"]), np.log(res_bsm["Desvio-padrão do
↪   P&L"]), 1)[0]
print("Inclinação log-log do erro de hedge:", f3(inclinacao), "(teoria: −0,500)")
ok(-0.62 < inclinacao < -0.38, "erro de hedge cai na taxa aproximada de 1/√N")
ultimo = res_bsm.iloc[-1]
ok(abs(ultimo["P&L médio"]) < 4 * ultimo["Desvio-padrão do P&L"] / np.sqrt(I), "P&L médio do
↪   hedge diário compatível com zero, apesar de μ ≠ r")
```

```
Prêmio recebido: 9,163
```

Rebalanceamentos N	P&L médio	Desvio-padrão do P&L	Pior caso
6	-0,041	2,685	-18,396
12	-0,022	1,934	-18,193
21	-0,017	1,467	-10,443
42	-0,007	1,044	-7,659
84	-0,002	0,746	-4,955
252	-0,001	0,435	-2,610

```
Inclinação log-log do erro de hedge: -0,488 (teoria: −0,500)
OK — erro de hedge cai na taxa aproximada de 1/√N
OK — P&L médio do hedge diário compatível com zero, apesar de μ ≠ r
```

Duas observações importantes.

1. O erro cai com $1/\sqrt{N}$. Para reduzi-lo à metade, é preciso rebalancear quatro vezes mais, e na prática cada rebalanceamento tem custo.
2. As trajetórias foram simuladas com retorno médio $\mu = r + 6\%$, e não r. O P&L médio continuou compatível com zero. É a mesma lição da Seção 2: **a tendência do ativo não entra no valor da opção**.

Laboratório 6.2 — E quando existem saltos?

Agora o índice pode saltar. Fazemos o mesmo hedge, usando no delta a volatilidade total equivalente $\sigma_h = \sqrt{\sigma^2 + \lambda(\mu_j^2 + \delta_j^2)}$.

```
salto = (0.6, -0.18, 0.12)
sigma_dif = 0.14
lam, mu_j, delta_j = salto
sigma_h = np.sqrt(sigma_dif**2 + lam * (mu_j**2 + delta_j**2))

rng = np.random.default_rng(8)
mundo_saltos = simular_caminhos(S0, r + 0.06, sigma_dif, T, M, I, rng, salto=salto)

linhas = []
for n in Ns:
    pnl = pnl_delta_hedge(mundo_saltos, K, T, r, sigma_h, n)
    linhas.append({"Rebalanceamentos N": n, "P&L médio": pnl.mean(), "Desvio-padrão do P&L":
↪  pnl.std(ddof=1), "Pior caso": pnl.min()})
res_saltos = pd.DataFrame(linhas)
print("Volatilidade total equivalente:", f3(sigma_h))
tabela(res_saltos)

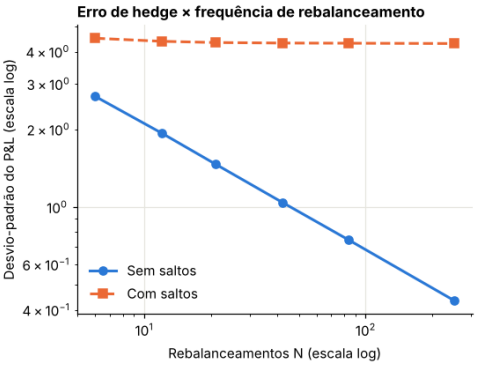
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 4))
a1.loglog(res_bsm["Rebalanceamentos N"], res_bsm["Desvio-padrão do P&L"], color=AZUL, marker="o",
↪  markersize=6, label="Sem saltos")
a1.loglog(res_saltos["Rebalanceamentos N"], res_saltos["Desvio-padrão do P&L"], color=LARANJA,
↪  marker="s", markersize=6, ls="--", label="Com saltos")
a1.set_title("Erro de hedge × frequência de rebalanceamento")
a1.set_xlabel("Rebalanceamentos N (escala log)")
a1.set_ylabel("Desvio-padrão do P&L (escala log)")
a1.legend()

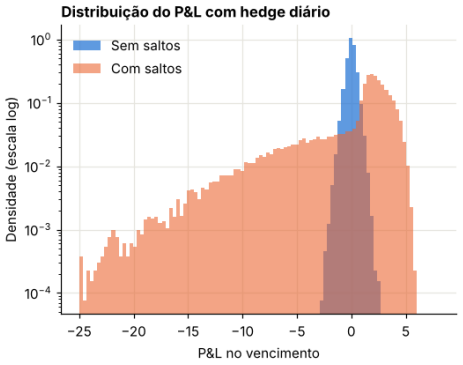
p1 = pnl_delta_hedge(mundo_bsm, K, T, r, sigma, 252)
p2 = pnl_delta_hedge(mundo_saltos, K, T, r, sigma_h, 252)
faixa = (-25, 8)
a2.hist(p1, bins=100, range=faixa, density=True, color=AZUL, alpha=0.75, label="Sem saltos")
a2.hist(p2, bins=100, range=faixa, density=True, color=LARANJA, alpha=0.6, label="Com saltos")
a2.set_yscale("log")
a2.set_title("Distribuição do P&L com hedge diário")
a2.set_xlabel("P&L no vencimento")
a2.set_ylabel("Densidade (escala log)")
a2.legend()
plt.tight_layout()
plt.show()

razao = res_saltos.iloc[-1]["Desvio-padrão do P&L"] / res_bsm.iloc[-1]["Desvio-padrão do P&L"]
print("Razão entre os erros de hedge diário (com/sem saltos):", f3(razao))
ok(razao > 3, "com saltos, o erro de hedge diário é muito maior")
queda_bsm = res_bsm.iloc[0]["Desvio-padrão do P&L"] / res_bsm.iloc[-1]["Desvio-padrão do P&L"]
queda_saltos = res_saltos.iloc[0]["Desvio-padrão do P&L"] / res_saltos.iloc[-1]["Desvio-padrão do
↪  P&L"]
ok(queda_saltos < queda_bsm / 2, "com saltos, rebalancear mais ajuda bem menos")
```

```
Volatilidade total equivalente: 0,218
```

Rebalanceamentos N	P&L médio	Desvio-padrão do P&L	Pior caso
6	0,178	4,507	-44,772
12	0,206	4,385	-45,228
21	0,237	4,337	-37,370
42	0,245	4,318	-35,194
84	0,249	4,311	-35,494
252	0,254	4,299	-34,271






```
Razão entre os erros de hedge diário (com/sem saltos): 9,890
OK — com saltos, o erro de hedge diário é muito maior
OK — com saltos, rebalancear mais ajuda bem menos
```

Por que o hedge não converge? Um salto acontece entre dois rebalanceamentos, de forma que nenhuma frequência consegue antecipá-lo. O mercado é **incompleto**: uma única fonte de hedge (o índice) não dá conta de dois tipos de risco. Só resta **minimizar** o erro, tema do módulo 16.

---

7. O processo completo em miniatura

Livro: §2.5

O resto do curso aprofunda estas seis etapas. Vamos percorrer todas agora, em versão simples:

Etapa	Pergunta	Aqui	Módulos
1. Realidade de mercado	Como o mercado se comporta?	Superfície com assimetria	2
2. Modelo	Qual modelo representa isso?	BSM contra Merton (saltos)	3-5, 11
3. Avaliar vanilla	Como precificar rápido?	Fórmulas fechadas	5-8
4. Calibrar	Quais parâmetros?	Mínimos quadrados	13, 14
5. Avaliar exótico	Quanto vale o produto?	Monte Carlo de uma barreira	12, 15

Etapa	Pergunta	Aqui	Módulos
6. Hedge	Como proteger?	Delta por reprecificação	16

Etapa 1 — O mercado sintético

O livro usa cotações reais do EURO STOXX 50, distribuídas com os scripts na plataforma do autor. Aqui geramos um **mercado sintético** a partir de um modelo conhecido, com um pequeno ruído. A vantagem didática é grande: **sabemos a resposta certa** e podemos verificar se a calibração a recupera.

Etapas 2 e 3 — Dois modelos

Modelo A (BSM): volatilidade constante.

Modelo B (Merton 1976): BSM mais saltos. O preço tem uma lógica simples. **Condicionado** a ter havido exatamente n saltos até T, o log-retorno é normal, ou seja, voltamos ao BSM com parâmetros ajustados:

$$\sigma_n^2 = \sigma^2 + \frac{n\,\delta_J^2}{T}, \qquad r_n = r - \lambda\kappa + \frac{n\ln(1+\kappa)}{T}. \tag{1.26}$$

O valor é a média desses preços BSM, ponderada pela probabilidade de cada n, com uma intensidade ajustada $\lambda' = \lambda(1+\kappa)$:

$$C_0^{M76} = \sum_{n=0}^{\infty} \frac{e^{-\lambda' T}(\lambda' T)^n}{n!}\; C^{BSM}(S_0, K, T, r_n, \sigma_n). \tag{1.27}$$

A dedução completa está no módulo 10. Abaixo, o preço é conferido por Monte Carlo.

```
def merton_call(S, K, T, r, sigma, lam, mu_j, delta_j, n_max=60):
    """Call europeia em Merton (1976), como mistura Poisson de preços BSM."""
    kappa = np.exp(mu_j + 0.5 * delta_j**2) - 1
    lam_q = lam * (1 + kappa)
    total = 0.0
    for n in range(n_max):
        peso = poisson.pmf(n, lam_q * T)
        s_n = np.sqrt(sigma**2 + n * delta_j**2 / T)
        r_n = r - lam * kappa + n * np.log1p(kappa) / T
        total = total + peso * bsm(S, K, T, r_n, s_n)
    return total


verdade = dict(sigma=0.14, lam=0.6, mu_j=-0.18, delta_j=0.12)
S0, r = 100.0, 0.025

rng = np.random.default_rng(99)
teste = simular_caminhos(S0, r, verdade["sigma"], 1.0, 1, 1_000_000, rng, salto=(verdade["lam"],
 ↪  verdade["mu_j"], verdade["delta_j"]))[-1]
amostra = np.exp(-r) * payoff_call(teste, 100.0)
mc, ep = amostra.mean(), amostra.std(ddof=1) / np.sqrt(amostra.size)
fechada = float(merton_call(S0, 100.0, 1.0, r, **verdade))
print("Merton fechada:", f3(fechada), "| Monte Carlo:", f3(mc), "| erro-padrão:", f3(ep))
ok(abs(fechada - mc) < 4 * ep, "fórmula de Merton confere com Monte Carlo")
```

```
prazos = [0.25, 0.5, 1.0]
strikes = np.array([80.0, 85.0, 90.0, 95.0, 100.0, 105.0, 110.0, 115.0, 120.0])
rng = np.random.default_rng(2015)
ruido_rel = 0.003

cotacoes = []
for t_ in prazos:
    precos = merton_call(S0, strikes, t_, r, **verdade) * (1 + rng.normal(0, ruido_rel,
↪   strikes.size))
    cotacoes += [{"T": t_, "K": k, "preco": p} for k, p in zip(strikes, precos)]
mercado = pd.DataFrame(cotacoes)
print("Cotações sintéticas:", len(mercado))
```

```
Merton fechada: 9,426 | Monte Carlo: 9,429 | erro-padrão: 0,013
OK — fórmula de Merton confere com Monte Carlo
Cotações sintéticas: 27
```

Etapa 4 — Calibração

Volatilidade implícita. Em vez de falar em preços, o mercado cota a volatilidade que, colocada no BSM, reproduz o preço observado. Inverter essa fórmula já é uma calibração de um único parâmetro. Como a call é crescente em σ, a raiz é única e podemos achá-la com o método de Brent.

Modelo A: escolhemos σ para acertar a call ATM de 1 ano exatamente.

Modelo B: temos 4 parâmetros e 27 cotações, então o problema é minimizar os erros ao quadrado,

$$\min_{\sigma,\lambda,\mu_J,\delta_J} \sum_{i=1}^{27} \left[C^{M76}(K_i,T_i;\sigma,\lambda,\mu_J,\delta_J) - C_i^{\text{mercado}}\right]^2. \tag{1.28}$$

Esse problema pode ter mínimos locais, então partimos de vários pontos iniciais e ficamos com o melhor. O módulo 13 trata o tema com o devido cuidado.

```
def vol_implicita(preco, S, K, T, r):
    return brentq(lambda s: float(bsm(S, K, T, r, s)) - preco, 1e-4, 3.0)


mercado["vol_impl"] = [vol_implicita(l.preco, S0, l.K, l.T, r) for l in mercado.itertuples()]

atm_1a = mercado.query("T == 1.0 and K == 100.0")["preco"].iloc[0]
sigma_a = vol_implicita(atm_1a, S0, 100.0, 1.0, r)


def erros_b(x):
    sig, lam, mj, dj = x
    saida = []
    for t_, g in mercado.groupby("T"):
        saida.append(merton_call(S0, g["K"].values, t_, r, sig, lam, mj, dj, n_max=30) -
↪   g["preco"].values)
    return np.concatenate(saida)


limites = ([0.01, 0.0, -0.6, 0.001], [0.6, 3.0, 0.1, 0.5])
partidas = [[0.20, 0.3, -0.10, 0.10], [0.12, 1.0, -0.20, 0.20], [0.25, 0.1, -0.30, 0.05], [0.10,
↪   2.0, -0.05, 0.15]]
ajustes = [least_squares(erros_b, x0, bounds=limites) for x0 in partidas]
```

```
melhor = min(ajustes, key=lambda a: a.cost)
print("RMSE por ponto inicial:", [f3(float(np.sqrt(np.mean(a.fun**2)))) for a in ajustes])

calib = dict(zip(["sigma", "lam", "mu_j", "delta_j"], melhor.x))
tabela(pd.DataFrame({
    "Parâmetro": ["σ", "λ", "μ_J", "δ_J"],
    "Verdadeiro": [float(verdade[k]) for k in calib],
    "Calibrado": [float(v) for v in calib.values()],
}))

mercado["modelo_a"] = [float(bsm(S0, l.K, l.T, r, sigma_a)) for l in mercado.itertuples()]
mercado["modelo_b"] = [float(merton_call(S0, l.K, l.T, r, **calib)) for l in mercado.itertuples()]
mercado["vi_b"] = [vol_implicita(l.modelo_b, S0, l.K, l.T, r) for l in mercado.itertuples()]

rmse = lambda col: float(np.sqrt(np.mean((mercado[col] - mercado["preco"]) ** 2)))
tabela(pd.DataFrame([
    {"Modelo": "A: BSM, calibrado só no ATM 1 ano", "σ": sigma_a, "RMSE em preço":
↪   rmse("modelo_a")},
    {"Modelo": "B: Merton, calibrado em tudo", "σ": calib["sigma"], "RMSE em preço":
↪   rmse("modelo_b")},
]))

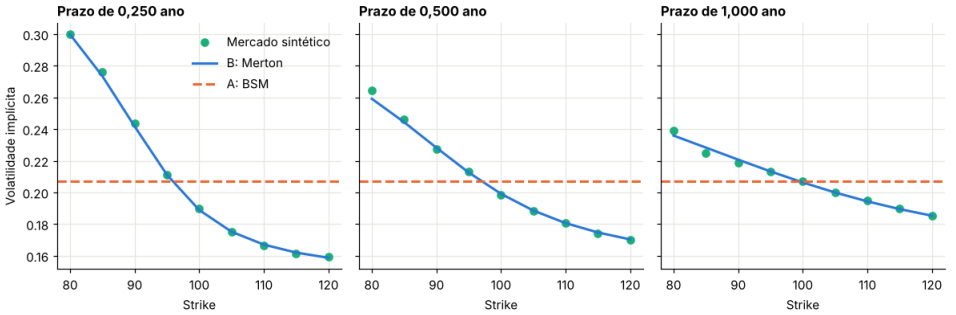
fig, eixos = plt.subplots(1, 3, figsize=(11, 3.8), sharey=True)
for ax, t_ in zip(eixos, prazos):
    g = mercado[mercado["T"] == t_]
    ax.plot(g["K"], g["vol_impl"], "o", color=VERDE, markersize=6, label="Mercado sintético")
    ax.plot(g["K"], g["vi_b"], color=AZUL, label="B: Merton")
    ax.axhline(sigma_a, color=LARANJA, ls="--", label="A: BSM")
    ax.set_title(f"Prazo de {f3(t_)} ano")
    ax.set_xlabel("Strike")
eixos[0].set_ylabel("Volatilidade implícita")
eixos[0].legend()
plt.tight_layout()
plt.show()

ok(abs(float(bsm(S0, 100.0, 1.0, r, sigma_a)) - atm_1a) < 1e-8, "modelo A reproduz exatamente o
↪   ATM de 1 ano")
ok(rmse("modelo_b") < rmse("modelo_a") / 5, "modelo B ajusta a superfície muito melhor que o A")
ok(mercado.query("T == 0.25").sort_values("K")["vol_impl"].iloc[0] > mercado.query("T == 0.25 and
↪   K == 100.0")["vol_impl"].iloc[0], "há skew: strikes baixos têm volatilidade implícita maior")
```

```
RMSE por ponto inicial: ['0,029', '0,029', '0,029', '0,029']
```

Parâmetro	Verdadeiro	Calibrado
σ	0,140	0,143
λ	0,600	0,442
μ_J	-0,180	-0,226
δ_J	0,120	0,099

Modelo	σ	RMSE em preço
A: BSM, calibrado só no ATM 1 ano	0,207	0,449
B: Merton, calibrado em tudo	0,143	0,029




```
OK — modelo A reproduz exatamente o ATM de 1 ano
OK — modelo B ajusta a superfície muito melhor que o A
OK — há skew: strikes baixos têm volatilidade implícita maior
```

O BSM é uma reta horizontal; o mercado tem **skew** (a volatilidade implícita sobe nos strikes baixos). Esse é o primeiro stylized fact que o módulo 2 vai explorar.

Olhe também a tabela de parâmetros. O ajuste em preço é excelente e os quatro pontos iniciais chegam ao mesmo RMSE, mas λ, μ_J e δ_J não coincidem com os verdadeiros. Combinações diferentes de frequência e tamanho de salto geram preços quase iguais, e o ruído basta para deslocar a solução. Esse é o problema da **identificabilidade**: um bom ajuste não prova que os parâmetros estão certos.

Etapa 5 — Avaliar um exótico

Um **down-and-out call** é uma call que **deixa de existir** se o índice tocar a barreira H em qualquer data até o vencimento. Aqui, $K = 100$, $H = 90$, $T = 1$, monitoramento diário.

Antes de usar a simulação, validamos o simulador: em BSM, com barreira contínua, existe solução fechada,

$$C_{DO} = C(S_0, K) - \left(\frac{H}{S_0}\right)^{2r/\sigma^2 - 1} C\left(\frac{H^2}{S_0}, K\right), \tag{1.29}$$

e o monitoramento discreto se aproxima deslocando a barreira: $\tilde{H} = He^{-0{,}5826\,\sigma\sqrt{\Delta t}}$ (Broadie, Glasserman e Kou, 1997).

Etapa 6 — Hedge

Se temos como calcular o preço, temos como calcular o hedge. O delta sai de reprecificar com S_0 deslocado em $\pm 1\%$. Há um atalho útil: como S_t é proporcional a S_0, basta multiplicar as **mesmas** trajetórias, sem nova simulação. Isso elimina quase todo o ruído da diferença.

```python
def down_and_out_fechada(S, K, H, T, r, sigma):
    return float(bsm(S, K, T, r, sigma) - (H / S) ** (2 * r / sigma**2 - 1) * bsm(H**2 / S, K, T,
    ↪  r, sigma))


def avaliar_exoticos(traj, K, H, T, r, escala=1.0):
    """Valor por Monte Carlo da barreira e da put 80, para S0 multiplicado por 'escala'."""
    S = traj * escala
```

```
    desconto = np.exp(-r * T)
    vivo = S.min(axis=0) > H
    bar = desconto * payoff_call(S[-1], K) * vivo
    put = desconto * payoff_put(S[-1], 80.0)
    return bar.mean(), bar.std(ddof=1) / np.sqrt(bar.size), put.mean()


K_b, H_b, T_b, M_b, I_b = 100.0, 90.0, 1.0, 252, 100_000

rng = np.random.default_rng(31)
traj_a = simular_caminhos(S0, r, sigma_a, T_b, M_b, I_b, rng)
traj_b = simular_caminhos(S0, r, calib["sigma"], T_b, M_b, I_b, rng, salto=(calib["lam"],
↪  calib["mu_j"], calib["delta_j"]))
traj_v = simular_caminhos(S0, r, verdade["sigma"], T_b, M_b, I_b, rng, salto=(verdade["lam"],
↪  verdade["mu_j"], verdade["delta_j"]))

H_ajustada = H_b * np.exp(-0.5826 * sigma_a * np.sqrt(T_b / M_b))
referencia = down_and_out_fechada(S0, K_b, H_ajustada, T_b, r, sigma_a)
bar_a, ep_a, _ = avaliar_exoticos(traj_a, K_b, H_b, T_b, r)
print("Validação BSM → fechada com barreira ajustada:", f3(referencia), "| Monte Carlo:",
↪  f3(bar_a), "| erro-padrão:", f3(ep_a))
ok(abs(referencia - bar_a) < 4 * ep_a, "simulador da barreira confere com a solução fechada")

linhas = []
for nome, traj, atm in [
    ("A: BSM", traj_a, float(bsm(S0, 100.0, 1.0, r, sigma_a))),
    ("B: Merton calibrado", traj_b, float(merton_call(S0, 100.0, 1.0, r, **calib))),
    ("Verdade: modelo gerador", traj_v, float(merton_call(S0, 100.0, 1.0, r, **verdade))),
]:
    v, ep, put80 = avaliar_exoticos(traj, K_b, H_b, T_b, r)
    up = avaliar_exoticos(traj, K_b, H_b, T_b, r, 1.01)[0]
    dn = avaliar_exoticos(traj, K_b, H_b, T_b, r, 0.99)[0]
    linhas.append({"Modelo": nome, "Call ATM 1 ano": atm, "Put 80": put80, "Down-and-out": v,
↪  "Erro-padrão": ep, "Delta da barreira": (up - dn) / (2 * 0.01 * S0)})
res = pd.DataFrame(linhas)
tabela(res)

linhas = []
for h in [80.0, 85.0, 90.0]:
    va, ea, _ = avaliar_exoticos(traj_a, K_b, h, T_b, r)
    vv, ev, _ = avaliar_exoticos(traj_v, K_b, h, T_b, r)
    linhas.append({"Barreira H": h, "A: BSM": va, "Verdade": vv, "Diferença": va - vv,
↪  "Erro-padrão da diferença": float(np.hypot(ea, ev))})
sens = pd.DataFrame(linhas)
tabela(sens)
del traj_a, traj_b, traj_v

r_ = res.set_index("Modelo")
ok(abs(r_.loc["A: BSM", "Call ATM 1 ano"] - r_.loc["Verdade: modelo gerador", "Call ATM 1 ano"])
↪  < 0.1, "os modelos concordam no vanilla ATM")
ok(abs(r_.loc["A: BSM", "Put 80"] - r_.loc["Verdade: modelo gerador", "Put 80"]) > 0.4, "o BSM
↪  erra a put fora do dinheiro")
ok(abs(r_.loc["A: BSM", "Down-and-out"] - r_.loc["Verdade: modelo gerador", "Down-and-out"]) > 4
↪  * np.hypot(r_.loc["A: BSM", "Erro-padrão"], r_.loc["Verdade: modelo gerador",
↪  "Erro-padrão"]), "o BSM erra o exótico")
ok(abs(sens["Diferença"]).is_monotonic_increasing, "o erro do BSM cresce quando a barreira se
↪  aproxima do spot")
ok(abs(r_.loc["B: Merton calibrado", "Down-and-out"] - r_.loc["Verdade: modelo gerador",
↪  "Down-and-out"]) < abs(r_.loc["A: BSM", "Down-and-out"] - r_.loc["Verdade: modelo gerador",
↪  "Down-and-out"]), "o modelo calibrado fica mais perto da verdade no exótico")
```

```
Validação BSM → fechada com barreira ajustada: 7,864 | Monte Carlo: 7,749 | erro-padrão: 0,045
OK — simulador da barreira confere com a solução fechada
```

Modelo	Call ATM 1 ano	Put 80	Down-and-out	Erro-padrão	Delta da barreira
A: BSM	9,439	1,029	7,749	0,045	0,737
B: Merton calibrado	9,407	1,565	8,862	0,040	0,722
Verdade: modelo gerador	9,426	1,546	8,889	0,040	0,713

Barreira H	A: BSM	Verdade	Diferença	Erro-padrão da diferença
80,000	9,248	9,390	-0,142	0,061
85,000	8,876	9,291	-0,415	0,061
90,000	7,749	8,889	-1,140	0,060

```
OK — os modelos concordam no vanilla ATM
OK — o BSM erra a put fora do dinheiro
OK — o BSM erra o exótico
OK — o erro do BSM cresce quando a barreira se aproxima do spot
OK — o modelo calibrado fica mais perto da verdade no exótico
```

O que estas tabelas ensinam. Os dois modelos concordam na call ATM, instrumento usado para calibrar o BSM. Mesmo assim, discordam na put fora do dinheiro e no exótico, e o delta também muda. Acertar um preço vanilla não garante acertar o resto: o modelo precisa reproduzir **a superfície inteira** e ter a **dinâmica** adequada.

A segunda tabela mostra que o tamanho do erro depende do produto. Com a barreira em 80, longe do spot e numa região onde a call já pagaria pouco, a diferença entre os modelos é pequena. Com a barreira em 90, ela fica grande, porque o valor passa a depender de **quantas trajetórias tocam a barreira**.

Uma ressalva para guardar: aqui, o Merton leva vantagem porque o mercado foi gerado por ele. No mundo real, nenhum modelo é o verdadeiro, e calibrar bem os vanilla não elimina o risco de modelo. Dois modelos que ajustam igualmente bem os vanilla ainda podem dar preços diferentes ao exótico. Medir essa dispersão é parte central da validação.

---

8. Síntese

- O payoff é convexo, e essa convexidade dá valor à volatilidade.
- O valor vem do **custo de replicação**, e não da média sob probabilidades reais. A tendência do ativo não entra.
- Sob Q: valor = média descontada do payoff. No caso lognormal, isso resulta na fórmula BSM.
- Juros têm dois efeitos opostos: aumentam a call e reduzem a put.
- Valor no tempo pode ser negativo em puts europeias.
- Exóticos dependem da trajetória, por isso o modelo precisa acertar a dinâmica.
- O hedge dinâmico converge a $1/\sqrt{N}$ num mundo sem saltos e deixa de convergir com saltos.
- Precificação calibrada ao mercado segue seis etapas: mercado, modelo, vanilla, calibração, exótico e hedge.

9. Exercícios

Exercício 1. Com $S_0 = 100$, $S_u = 120$, $S_d = 80$, $r = 5\%$ no período e call com $K = 110$, calcule à mão Δ, B, q e o valor. Confira com `replicar`.

Solução

$C_u = 10$, $C_d = 0$. $\Delta = 10/40 = 0{,}250$. $B = (0 - 0{,}25 \times 80)/1{,}05 = -19{,}048$. Valor $= 25 - 19{,}048 = 5{,}952$. Pela medida neutra: $q = (105 - 80)/40 = 0{,}625$, e $0{,}625 \times 10/1{,}05 = 5{,}952$.

Exercício 2. Confira numericamente a paridade put-call para 5 strikes. Por que o resultado não depende de σ?

Solução

Calcule `bsm(S,K,T,r,s,'call') - bsm(S,K,T,r,s,'put') - (S - K*np.exp(-r*T))`: o resultado é zero, a menos de erro numérico, para qualquer `s`. A paridade vem de igualar payoffs em todos os cenários, sem hipótese sobre a distribuição.

Exercício 3. A volatilidade sempre aumenta o valor de uma opção europeia vanilla? E de uma down-and-out? Teste com `down_and_out_fechada`, usando $S_0 = 100$, $K = 100$, $H = 95$.

Solução

Na vanilla, sempre aumenta. Na down-and-out há dois efeitos opostos: mais volatilidade dá mais payoff, mas também mais chance de tocar a barreira. Com esses parâmetros e $r = 2{,}5\%$, entre $\sigma = 10\%$ e $\sigma = 50\%$ a vanilla vai de 5,295 para 20,756, enquanto a barreira vai só de 4,338 para 5,010: quase todo o ganho é consumido. Em outros produtos, como a up-and-out call com spot perto da barreira, o segundo efeito domina e o valor **cai** com a volatilidade.

Exercício 4. Modifique `pnl_delta_hedge` para cobrar um custo de 0,1% sobre o valor negociado em cada ajuste. Como o P&L médio e o desvio-padrão se comportam quando N aumenta?

Solução

Subtraia `custo * np.abs(novo - delta) * S[i]` da conta-caixa a cada ajuste, incluindo a montagem e o encerramento. O desvio-padrão continua caindo, mas o custo total cresce aproximadamente com $\sqrt{N}$, deixando o P&L médio cada vez mais negativo. Há portanto uma frequência ótima, que depende da aversão a risco. Esse é o problema de Leland (1985).

Exercício 5. Calibre o modelo A a toda a superfície, em vez de só ao ATM de 1 ano. O preço do exótico melhora? O que muda no ajuste da call ATM?

Solução

Use `least_squares` com um único parâmetro, σ. A superfície fica melhor em média, mas a call ATM deixa de ser reproduzida exatamente, e o exótico continua dependendo de uma dinâmica sem saltos. Um parâmetro só não consegue representar o skew.

```
print(f"Módulo 1 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 1 concluído: 37 verificações aprovadas.
```

Capítulo 2

Stylized facts e volatilidade implícita

Introdução

Antes de escolher um modelo, é preciso saber o que ele deve reproduzir. Os **stylized facts** são regularidades estatísticas encontradas em praticamente todos os mercados financeiros líquidos, independentemente do país, do período ou do ativo: caudas mais pesadas que as da normal, assimetria entre altas e baixas, períodos de volatilidade alta e baixa que se agrupam, e correlação negativa entre retornos e volatilidade.

O modelo de Black-Scholes, ponto de partida da teoria, supõe retornos normais com volatilidade constante, e portanto ignora todos esses fatos. O mercado de opções sabe disso: se os operadores usassem Black-Scholes à risca, todas as opções de um mesmo vencimento seriam negociadas com a mesma volatilidade. Não são. Ao inverter a fórmula para obter a **volatilidade implícita** de cada opção, aparece uma curva, o **smile** de volatilidade, que nos índices de ações costuma ser inclinada (o **skew**): strikes baixos, que protegem contra quedas, são mais caros.

Este capítulo liga as duas pontas. Primeiro medimos os stylized facts em séries de preços; depois mostramos como cada um deles deixa uma marca no smile. Essa ponte justifica os ingredientes dos modelos dos capítulos seguintes: saltos para as caudas e a assimetria de curto prazo, volatilidade estocástica para o agrupamento e correlação negativa para o skew.

Fundamentos teóricos

Retornos e seus momentos

Definição 2.1 (Retornos). Para preços $S_0, S_1, \ldots$ observados em datas igualmente espaçadas, o *retorno simples* é $R_t = S_t/S_{t-1} - 1$ e o *log-retorno* é $r_t = \ln(S_t/S_{t-1})$. Log-retornos são aditivos no tempo: $\ln(S_T/S_0) = \sum_{t=1}^{T} r_t$.

Definição 2.2 (Assimetria e curtose). Para uma variável X com média μ e desvio-padrão σ, a *assimetria* é $\mathbb{E}[(X-\mu)^3]/\sigma^3$ e a *curtose* é $\kappa = \mathbb{E}[(X-\mu)^4]/\sigma^4$. Para a normal, assimetria 0 e $\kappa = 3$; o *excesso de curtose* é $\kappa - 3$.

Definição 2.3 (Volatilidade realizada). Com n log-retornos por ano, a volatilidade realizada anualizada é $\hat{\sigma} = \sqrt{n}\,\mathrm{dp}(r_t)$. A regra da raiz do tempo supõe retornos não correlacionados e de variância constante: $\mathrm{Var}(\sum_{t=1}^{h} r_t) = h\,\mathrm{Var}(r_t)$.

Stylized facts

Os stylized facts são regularidades estatísticas observadas em praticamente todos os mercados líquidos (Cont, 2001). Os que importam para a avaliação de opções são:

1. **Caudas pesadas:** o excesso de curtose dos log-retornos diários é positivo e grande; a normal subestima a frequência de movimentos extremos.
2. **Assimetria de ganhos e perdas:** em índices de ações, grandes quedas são mais frequentes que grandes altas.
3. **Agrupamento de volatilidade:** $\mathrm{corr}(|r_t|, |r_{t+h}|)$ é positiva e decai lentamente em h, embora $\mathrm{corr}(r_t, r_{t+h}) \approx 0$.
4. **Efeito alavancagem:** $\mathrm{corr}(r_t, \sigma^2_{t+h}) < 0$; quedas antecedem aumentos de volatilidade.
5. **Normalidade agregada:** em horizontes longos, a distribuição dos log-retornos se aproxima da normal.

Cada fato motiva um ingrediente dos modelos dos capítulos seguintes: saltos (caudas e assimetria de curto prazo), volatilidade estocástica (agrupamento) e correlação negativa entre preço e volatilidade (alavancagem).

Volatilidade implícita

Seja $C^{\mathrm{BS}}(\sigma) = C^{\mathrm{BS}}(S_0, K, T, r, \sigma)$ o preço de Black-Scholes de uma call europeia (Capítulo 5).

Proposição 2.4 (Existência e unicidade da volatilidade implícita). *Para $S_0, K, T > 0$, a função $\sigma \mapsto C^{\mathrm{BS}}(\sigma)$ é contínua e estritamente crescente em $(0, \infty)$, com*

$$\lim_{\sigma \downarrow 0} C^{\mathrm{BS}}(\sigma) = (S_0 - Ke^{-rT})^+, \qquad \lim_{\sigma \to \infty} C^{\mathrm{BS}}(\sigma) = S_0. \tag{2.1}$$

Logo, para todo preço C no intervalo de não arbitragem $((S_0 - Ke^{-rT})^+, S_0)$ existe um único $\sigma_{\mathrm{imp}} > 0$ com $C^{\mathrm{BS}}(\sigma_{\mathrm{imp}}) = C$.

Demonstração. A derivada é o vega, $\partial C^{\mathrm{BS}}/\partial\sigma = S_0\sqrt{T}\,\phi(d_1) > 0$, onde ϕ é a densidade normal; logo a função é estritamente crescente. Quando $\sigma \downarrow 0$, $d_1, d_2 \to \pm\infty$ conforme o sinal de $\ln(S_0/K) + rT$, e o preço tende ao valor intrínseco descontado. Quando $\sigma \to \infty$, $d_1 \to +\infty$ e $d_2 \to -\infty$, e o preço tende a S_0. O resultado segue do teorema do valor intermediário. ∎

Definição 2.5 (Smile e superfície de volatilidade). A função $(K, T) \mapsto \sigma_{\mathrm{imp}}(K, T)$ obtida invertendo preços de mercado é a *superfície de volatilidade implícita*; um corte com T fixo é o *smile*. É comum parametrizá-la pela *log-moneyness* $m = \ln(K/F_T)$, com $F_T = S_0 e^{rT}$.

Observação 2.6. Se o modelo de Black-Scholes fosse correto, a superfície seria plana. O skew e a convexidade do smile são a forma como o mercado de opções incorpora os stylized facts acima. A vol implícita é uma *unidade de preço*, não uma estimativa da volatilidade futura.

Numericamente, a inversão usa Newton–Raphson, $\sigma_{k+1} = \sigma_k - (C^{\mathrm{BS}}(\sigma_k) - C)/\mathrm{vega}(\sigma_k)$, que converge quadraticamente perto da raiz; longe do dinheiro o vega é pequeno e métodos de intervalo (bissecção, Brent) são mais robustos (Gatheral, 2006).

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulo 3 do livro.

O que você vai conseguir fazer ao final

1. Distinguir volatilidade histórica, realizada, instantânea e implícita, e saber o que cada uma mede.
2. Calcular quanta informação uma série realmente contém sobre a média e sobre a volatilidade.
3. Testar se uma série é normal, e entender o que o teste diz e o que não diz.
4. Reconhecer os stylized facts dos índices e saber **qual mecanismo de modelo gera cada um**.
5. Calcular volatilidade implícita de forma robusta, sabendo por que certos métodos falham.
6. Ler uma superfície de volatilidade, verificar se ela tem arbitragem e extrair dela a densidade neutra ao risco.
7. Justificar, a partir dos dados, por que o modelo geral do curso tem três componentes.

O método deste módulo

O livro analisa dados reais do DAX, do EURO STOXX 50 e da Euribor. Aqui usamos **mundos sintéticos**, em que ligamos um mecanismo de cada vez. Isso permite responder a uma pergunta que dados reais não respondem: *qual parte do modelo produz qual fato?*

O que aparece como caixa-preta neste módulo: a simulação de volatilidade estocástica (módulo 12) e a fórmula de preços por Fourier (módulos 7 e 11). Ambas são conferidas numericamente aqui.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from scipy.stats import norm, skew, kurtosis, skewtest, kurtosistest, normaltest
from scipy.optimize import brentq, fsolve
from scipy.integrate import simpson
from IPython.display import display

RAMPA = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281", "#0d366b"]

def bsm(S, K, T, r, sigma):
    """Call europeia no BSM. Aceita arrays em S e K."""
    S, K = np.asarray(S, float), np.asarray(K, float)
    raiz = sigma * np.sqrt(T)
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / raiz
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - raiz)

DIAS = 252
print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. O vocabulário: retornos, volatilidades e momentos

Livro: §3.2

1.1 Por que retorno logarítmico

Dada uma série de preços $S_0, S_1, \dots, S_N$, há duas formas de medir a variação:

$$R_n = \frac{S_n}{S_{n-1}} - 1 \quad \text{(simples)}, \qquad r_n = \ln \frac{S_n}{S_{n-1}} \quad \text{(log)}. \tag{2.2}$$

Para movimentos pequenos, os dois são quase iguais, pois $\ln(1+x) \approx x$. A vantagem do log aparece ao somar no tempo. Escrevendo a soma:

$$\sum_{n=1}^{N} r_n = \ln \frac{S_1}{S_0} + \ln \frac{S_2}{S_1} + \cdots + \ln \frac{S_N}{S_{N-1}} = \ln \frac{S_N}{S_0}. \tag{2.3}$$

Os termos intermediários se cancelam: **log-retornos somam**. Retornos simples não têm essa propriedade, como mostra o laboratório.

```
precos = np.array([100.0, 110.0, 99.0])
simples = precos[1:] / precos[:-1] - 1
logret = np.diff(np.log(precos))

tabela(pd.DataFrame({
    "Medida": ["Dia 1", "Dia 2", "Soma dos dois dias", "Retorno efetivo do período"],
    "Simples": [simples[0], simples[1], simples.sum(), precos[-1] / precos[0] - 1],
    "Log": [logret[0], logret[1], logret.sum(), np.log(precos[-1] / precos[0])],
}))
ok(abs(logret.sum() - np.log(precos[-1] / precos[0])) < 1e-12, "log-retornos somam exatamente")
ok(abs(simples.sum() - (precos[-1] / precos[0] - 1)) > 0.005, "retornos simples não somam")
```

Medida	Simples	Log
Dia 1	0,100	0,095
Dia 2	-0,100	-0,105
Soma dos dois dias	0,000	-0,010
Retorno efetivo do período	-0,010	-0,010

```
OK — log-retornos somam exatamente
OK — retornos simples não somam
```

1.2 Quatro volatilidades diferentes

Conceito	O que é	Observável?
Histórica	desvio-padrão amostral dos log-retornos numa janela	sim, olhando para trás
Realizada	variância acumulada dos retornos, atualizada a cada dia	sim, olhando para trás
Instantânea	o coeficiente σ_t da equação do processo	não: é parâmetro do modelo
Implícita	o σ que faz o BSM reproduzir o preço da opção	sim, mas só existe se há preço de opção

A histórica se calcula assim:

$$\hat{\mu} = \frac{1}{N}\sum_{n=1}^{N} r_n, \qquad \hat{\sigma} = \sqrt{\frac{1}{N-1}\sum_{n=1}^{N}(r_n - \hat{\mu})^2}. \tag{2.4}$$

A realizada, na forma usada nos scripts do livro, não subtrai a média:

$$\hat{\sigma}^2_{\text{real}} = \frac{252}{N}\sum_{n=1}^{N} r_n^2. \tag{2.5}$$

Em dados diários, a diferença entre as duas é desprezível, porque a média diária é muito menor que o desvio.

Anualização, passo a passo. Se os retornos diários são independentes e têm variância σ_d^2, a variância da soma de 252 dias é a soma das variâncias, $252\,\sigma_d^2$. Tirando a raiz:

$$\sigma_{\text{ano}} = \sigma_d\sqrt{252}. \tag{2.6}$$

Atenção: o que escala com o tempo é a **variância**. A raiz de 252 só é correta porque supusemos independência. Se os retornos tivessem autocorrelação, a fórmula mudaria.

1.3 Assimetria e curtose

$$\hat{s} = \frac{\frac{1}{N}\sum (r_n - \hat{\mu})^3}{\left[\frac{1}{N}\sum (r_n - \hat{\mu})^2\right]^{3/2}}, \qquad \hat{k} = \frac{\frac{1}{N}\sum (r_n - \hat{\mu})^4}{\left[\frac{1}{N}\sum (r_n - \hat{\mu})^2\right]^{2}} - 3. \tag{2.7}$$

- **Assimetria** $\hat{s}$: como elevamos ao cubo, os desvios mantêm o sinal. Negativa indica cauda esquerda maior. Na normal, é zero.
- **Curtose em excesso** $\hat{k}$: a quarta potência amplifica os valores extremos. Na normal, a razão vale 3, por isso se subtrai 3. Positiva indica **caudas pesadas**.

Laboratório 1.1 — Quanta informação existe numa série?

Uma pergunta de que depende tudo o mais: com Y anos de dados diários, qual a precisão das estimativas?

Média anualizada. A média dos log-retornos é $\hat{\mu} = (\ln S_N - \ln S_0)/N$: só os pontos inicial e final importam. Anualizando, o erro-padrão é

$$\mathrm{ep}(\hat{\mu}_{\text{ano}}) = \frac{\sigma}{\sqrt{Y}}. \tag{2.8}$$

Observar **com mais frequência não ajuda**. Só observar **por mais tempo**.

Volatilidade. Para dados normais, a variância amostral tem $\mathrm{Var}(s^2) = 2\sigma^4/(N-1)$. Como $s = \sqrt{s^2}$, a regra de propagação de erro, $\mathrm{ep}(s) \approx \mathrm{ep}(s^2)/(2\sigma)$, dá

$$\mathrm{ep}(\hat{\sigma}) \approx \frac{\sigma}{\sqrt{2(N-1)}}. \tag{2.9}$$

Aqui, sim, N é o número de observações. **Observar com mais frequência ajuda muito.**

```
sigma_v = 0.20
rng = np.random.default_rng(301)
linhas = []
for anos in [1, 5, 10, 30]:
    N = anos * DIAS
    amostra = rng.normal(0.05 / DIAS, sigma_v / np.sqrt(DIAS), (N, 2000))
    medias = amostra.mean(axis=0) * DIAS
    vols = amostra.std(axis=0, ddof=1) * np.sqrt(DIAS)
    linhas.append({
        "Anos": anos,
        "Média: ep teórico": sigma_v / np.sqrt(anos),
        "Média: ep simulado": medias.std(ddof=1),
        "Vol.: ep teórico": sigma_v / np.sqrt(2 * (N - 1)),
        "Vol.: ep simulado": vols.std(ddof=1),
    })
df = pd.DataFrame(linhas)
tabela(df)
ok(np.allclose(df["Média: ep teórico"], df["Média: ep simulado"], rtol=0.08), "erro da média
↪ segue σ/√Y")
ok(np.allclose(df["Vol.: ep teórico"], df["Vol.: ep simulado"], rtol=0.08), "erro da volatilidade
↪ segue σ/√(2N)")
```

Anos	Média: ep teórico	Média: ep simulado	Vol.: ep teórico	Vol.: ep simulado
1	0,200	0,199	0,009	0,009
5	0,089	0,088	0,004	0,004
10	0,063	0,062	0,003	0,003
30	0,037	0,037	0,002	0,002

```
OK — erro da média segue σ/√Y
OK — erro da volatilidade segue σ/√(2N)
```

Leitura. Mesmo com 30 anos de dados, o erro da média anual fica em torno de 3,651 pontos percentuais, enquanto o da volatilidade é irrisório já com 10 anos. A tendência de um ativo é quase impossível de estimar; a volatilidade, não. Felizmente, como vimos no Módulo 1, é a volatilidade que importa para o valor da opção.

---

2. O caso de referência: o Browniano geométrico

Livro: §3.3

O modelo BSM supõe a equação diferencial estocástica

$$dS_t = rS_t\,dt + \sigma S_t\,dZ_t, \tag{2.10}$$

cuja versão exata em tempo discreto, que já usamos no Módulo 1, é

$$S_t = S_{t-\Delta t}\exp\left[(r - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\,z_t\right], \qquad z_t \sim N(0,1). \tag{2.11}$$

Log-retornos são normais, independentes e têm volatilidade constante. Simulamos 10 anos com os parâmetros do livro: $S_0 = 100$, $r = 5\%$, $\sigma = 20\%$.

Como ler um gráfico Q-Q

Ordenamos os retornos da amostra e os colocamos contra os quantis que uma normal teria nas mesmas posições. Se a amostra for normal, os pontos ficam sobre uma reta. Se as pontas se afastarem da reta, as caudas são diferentes das da normal.

```python
def estatisticas(r):
    """Estatísticas de uma série de log-retornos diários."""
    return {
        "Média anualizada": r.mean() * DIAS,
        "Volatilidade anualizada": r.std(ddof=1) * np.sqrt(DIAS),
        "Assimetria": float(skew(r)),
        "p-valor assimetria": float(skewtest(r).pvalue),
        "Curtose em excesso": float(kurtosis(r)),
        "p-valor curtose": float(kurtosistest(r).pvalue),
        "p-valor normalidade": float(normaltest(r).pvalue),
        "Volatilidade realizada": float(np.sqrt(DIAS * np.mean(r**2))),
    }

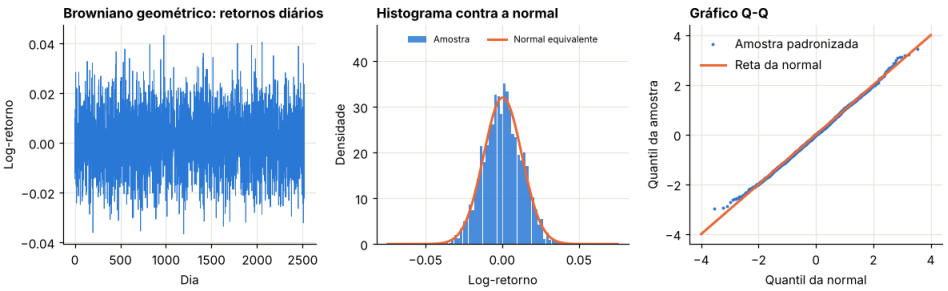

S0, r_gbm, sigma_gbm, anos = 100.0, 0.05, 0.20, 10
dt = 1 / DIAS
rng = np.random.default_rng(100)
r_g = (r_gbm - 0.5 * sigma_gbm**2) * dt + sigma_gbm * np.sqrt(dt) * rng.standard_normal(anos *
↪  DIAS)
S_g = S0 * np.exp(np.cumsum(r_g))

tabela(pd.DataFrame({"Estatística": list(estatisticas(r_g)), "Valor":
↪  list(estatisticas(r_g).values())}))
```

```python
def painel_distribuicao(r, titulo):
    fig, (a1, a2, a3) = plt.subplots(1, 3, figsize=(11, 3.5))
    a1.plot(r, color=AZUL, lw=0.6)
    a1.set_title(f"{titulo}: retornos diários")
    a1.set_xlabel("Dia")
    a1.set_ylabel("Log-retorno")
    mu, s = r.mean(), r.std(ddof=1)
    x = np.linspace(mu - 6 * s, mu + 6 * s, 300)
    a2.hist(r, bins=80, range=(x[0], x[-1]), density=True, color=AZUL, alpha=0.85, rwidth=0.92,
↪   label="Amostra")
    a2.plot(x, norm.pdf(x, mu, s), color=LARANJA, label="Normal equivalente")
    a2.set_title("Histograma contra a normal")
    a2.set_xlabel("Log-retorno")
    a2.set_ylabel("Densidade")
    a2.set_ylim(0, a2.get_ylim()[1] * 1.3)
    a2.legend(loc="upper center", ncol=2, fontsize=7.5)
    p = (np.arange(1, r.size + 1) - 0.5) / r.size
    q = norm.ppf(p)
    a3.plot(q, np.sort((r - mu) / s), ".", color=AZUL, markersize=3, label="Amostra padronizada")
    a3.plot([-4, 4], [-4, 4], color=LARANJA, label="Reta da normal")
    a3.set_title("Gráfico Q-Q")
    a3.set_xlabel("Quantil da normal")
    a3.set_ylabel("Quantil da amostra")
    a3.legend()
    plt.tight_layout()
    plt.show()


painel_distribuicao(r_g, "Browniano geométrico")
ok(abs(estatisticas(r_g)["Volatilidade anualizada"] - sigma_gbm) < 0.012, "volatilidade amostral
↪  próxima da instantânea")
```

Estatística	Valor
Média anualizada	0,109
Volatilidade anualizada	0,198
Assimetria	0,069
p-valor assimetria	0,159
Curtose em excesso	-0,106
p-valor curtose	0,282
p-valor normalidade	0,208
Volatilidade realizada	0,198




```
OK — volatilidade amostral próxima da instantânea
```

Laboratório 2.1 — O que um teste de normalidade realmente diz

Os três testes têm como hipótese nula que a amostra é normal. O p-valor responde: *se a amostra fosse normal, qual a chance de ver algo tão extremo?*

Dois erros comuns:

- p-valor alto **não prova** normalidade. Só diz que os dados não têm evidência contra ela.
- Mesmo com dados perfeitamente normais, o teste rejeita com frequência igual ao nível escolhido. Com nível de 5%, **uma em cada vinte amostras normais será rejeitada**.

Vamos verificar simulando 2.000 séries de 10 anos, todas normais de verdade.

```
rng = np.random.default_rng(102)
series = rng.normal(0, sigma_gbm * np.sqrt(dt), (anos * DIAS, 2000))
taxas = {
    "Teste de assimetria": (skewtest(series, axis=0).pvalue < 0.05).mean(),
    "Teste de curtose": (kurtosistest(series, axis=0).pvalue < 0.05).mean(),
    "Teste conjunto": (normaltest(series, axis=0).pvalue < 0.05).mean(),
}
tabela(pd.DataFrame({"Teste": list(taxas), "Taxa de rejeição a 5%": [float(v) for v in
↪  taxas.values()]}))
ok(all(0.03 < v < 0.07 for v in taxas.values()), "testes rejeitam séries normais em cerca de 5%
↪  dos casos")
```

Teste	Taxa de rejeição a 5%
Teste de assimetria	0,045
Teste de curtose	0,047
Teste conjunto	0,048

```
OK — testes rejeitam séries normais em cerca de 5% dos casos
```

Laboratório 2.2 — Cuidado com a correlação móvel

O livro calcula a média móvel e a volatilidade móvel em janelas de 252 dias, e depois a **correlação móvel entre as duas**, usada como indício do efeito alavancagem.

O problema: uma janela de 252 dias desliza um dia por vez, então as duas séries são muito **sobrepostas**. Dentro de uma janela de correlação, restam pouquíssimas observações independentes. Isso produz oscilações violentas **mesmo quando não existe relação alguma**.

No browniano geométrico, retorno e volatilidade são independentes, então a correlação verdadeira é zero. Vamos medir o que a estatística mostra.

```
rng = np.random.default_rng(103)
amostras = rng.normal(0, sigma_gbm * np.sqrt(dt), (anos * DIAS, 100))

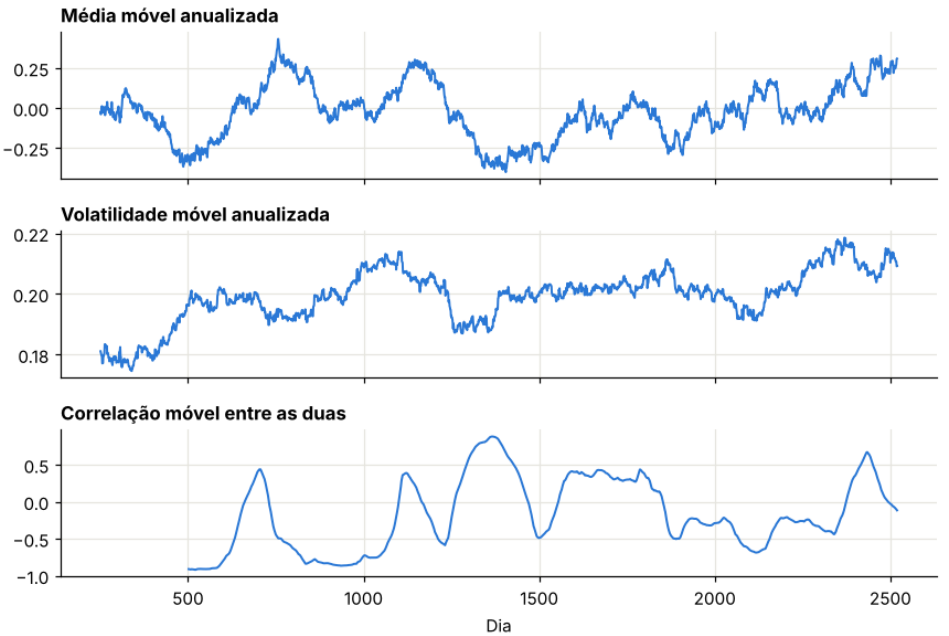
resumo = []
for j in range(amostras.shape[1]):
    s = pd.Series(amostras[:, j])
    media_movel = s.rolling(DIAS).mean() * DIAS
    vol_movel = s.rolling(DIAS).std() * np.sqrt(DIAS)
    corr = media_movel.rolling(DIAS).corr(vol_movel).dropna()
```

```
    resumo.append({"média": corr.mean(), "mínimo": corr.min(), "máximo": corr.max(), "fração
↪   |corr|>0,5": (corr.abs() > 0.5).mean()})
    if j == 0:
        exemplo = (media_movel, vol_movel, corr)
resumo = pd.DataFrame(resumo)
tabela(pd.DataFrame({"Média das 100 trajetórias":
↪   resumo.mean()}).reset_index(names="Estatística"))

fig, eixos = plt.subplots(3, 1, figsize=(8, 5.5), sharex=True)
for ax, serie, nome in zip(eixos, exemplo, ["Média móvel anualizada", "Volatilidade móvel
↪   anualizada", "Correlação móvel entre as duas"]):
    ax.plot(serie, color=AZUL, lw=1.3)
    ax.set_title(nome)
eixos[-1].set_xlabel("Dia")
plt.tight_layout()
plt.show()

ok(abs(resumo["média"].mean()) < 0.1, "em média, a correlação móvel fica perto de zero")
ok(resumo["fração |corr|>0,5"].mean() > 0.2 and resumo["máximo"].mean() > 0.6, "mesmo sem relação
↪   alguma, a correlação móvel atinge valores altos")
```

Estatística	Média das 100 trajetórias
média	-0,006
mínimo	-0,810
máximo	0,820
fração |corr|>0,5	0,389




```
OK — em média, a correlação móvel fica perto de zero
OK — mesmo sem relação alguma, a correlação móvel atinge valores altos
```

Leitura. Valores próximos de ± 1 não servem, sozinhos, como evidência de relação.

Na próxima seção usamos uma medida melhor, sem sobreposição: a correlação entre o retorno de hoje e o retorno ao quadrado de amanhã.

---

3. Stylized facts, um mecanismo de cada vez

Livro: §3.4

Comparados com o browniano geométrico, retornos de índice apresentam:

Fato	Descrição
Volatilidade estocástica	a volatilidade varia ao longo do tempo, sem previsão precisa
Agrupamento	dias de alta volatilidade tendem a vir juntos
Reversão à média	a volatilidade volta para um nível de longo prazo
Efeito alavancagem	queda no preço tende a vir com alta da volatilidade
Caudas pesadas	movimentos grandes são mais frequentes que na normal
Saltos	há movimentos abruptos grandes demais para um processo contínuo

O gerador de mundos

Para saber qual mecanismo gera cada fato, simulamos cinco mundos, todos com a **mesma variância total anual** de $0{,}22^2$:

Mundo	Mecanismo ligado
M0	nenhum: browniano geométrico
M1	só saltos
M2	volatilidade estocástica, com $\rho = 0$
M3	volatilidade estocástica, com $\rho = -0{,}7$
M4	volatilidade estocástica, $\rho = -0{,}7$, e saltos

A dinâmica geral é a do modelo de Heston com saltos:

$$d\ln S_t = (\mu - \lambda\bar{\kappa} - \tfrac{1}{2}v_t)dt + \sqrt{v_t}\,dZ_t^1 + J_t\,dN_t \tag{2.12}$$

$$dv_t = \kappa(\theta - v_t)\,dt + \nu\sqrt{v_t}\,dZ_t^2, \qquad dZ_t^1\,dZ_t^2 = \rho\,dt \tag{2.13}$$

Como ler a segunda equação:

- $\kappa(\theta - v_t)$ é o termo de **reversão à média**: se $v_t > \theta$, a tendência é de queda.
- ν é a **volatilidade da volatilidade**.
- ρ é a **correlação** entre os choques do preço e da variância. Com $\rho < 0$, um choque negativo no preço tende a vir com um choque positivo na variância.

Os saltos chegam como um processo de Poisson com intensidade λ, e cada log-salto $J \sim N(\mu_J, \delta_J^2)$. A variância anual dos saltos é $\lambda(\mu_J^2 + \delta_J^2)$; é ela que descontamos de θ para manter a variância total igual em todos os mundos.

A simulação aqui é um esquema de Euler simples com passo diário. O módulo 12 trata das sutilezas.

```
V_TOTAL = 0.22**2
LAM, MU_J, DELTA_J = 1.0, -0.04, 0.05
V_SALTO = LAM * (MU_J**2 + DELTA_J**2)
KAPPA, NU, RHO = 3.0, 0.45, -0.7

MUNDOS = {
    "M0: browniano": dict(theta=V_TOTAL, kappa=0.0, nu=0.0, rho=0.0, lam=0.0),
    "M1: só saltos": dict(theta=V_TOTAL - V_SALTO, kappa=0.0, nu=0.0, rho=0.0, lam=LAM),
    "M2: vol. estoc., ρ=0": dict(theta=V_TOTAL, kappa=KAPPA, nu=NU, rho=0.0, lam=0.0),
    "M3: vol. estoc., ρ<0": dict(theta=V_TOTAL, kappa=KAPPA, nu=NU, rho=RHO, lam=0.0),
    "M4: completo": dict(theta=V_TOTAL - V_SALTO, kappa=KAPPA, nu=NU, rho=RHO, lam=LAM),
}


def gerar_mundo(cfg, passos, trajetorias, rng, mu=0.08, dt=1 / DIAS, guardar=True):
    """Simula log-retornos num mundo de Heston com saltos (Euler com truncamento total)."""
    theta, kappa, nu, rho, lam = cfg["theta"], cfg["kappa"], cfg["nu"], cfg["rho"], cfg["lam"]
    comp = lam * (np.exp(MU_J + 0.5 * DELTA_J**2) - 1)
    v = np.full(trajetorias, theta, dtype=float)
    acum = np.zeros(trajetorias)
    ret, var = [], []
    raiz_dt = np.sqrt(dt)
    for _ in range(passos):
        z1 = rng.standard_normal(trajetorias)
        z2 = rho * z1 + np.sqrt(1 - rho**2) * rng.standard_normal(trajetorias)
        vp = np.maximum(v, 0.0)
        salto = 0.0
        if lam > 0:
            n = rng.poisson(lam * dt, trajetorias)
            salto = n * MU_J + np.sqrt(n) * DELTA_J * rng.standard_normal(trajetorias)
        incr = (mu - comp - 0.5 * vp) * dt + np.sqrt(vp) * raiz_dt * z1 + salto
        v = v + kappa * (theta - vp) * dt + nu * np.sqrt(vp) * raiz_dt * z2
        if guardar:
            ret.append(incr)
            var.append(vp)
        else:
            acum += incr
    return (np.array(ret), np.array(var)) if guardar else acum


def acf(x, defasagens):
    """Autocorrelação média entre colunas de x, nas defasagens indicadas."""
    x = x - x.mean(axis=0)
    den = (x**2).sum(axis=0)
    return np.array([np.mean((x[:-d] * x[d:]).sum(axis=0) / den) for d in defasagens])


def corr_alavancagem(r):
    """Correlação entre o retorno de hoje e o retorno ao quadrado de amanhã, média entre
    ↪ colunas."""
    a, b = r[:-1], r[1:] ** 2
    a = a - a.mean(axis=0)
    b = b - b.mean(axis=0)
    return np.mean((a * b).sum(axis=0) / np.sqrt((a**2).sum(axis=0) * (b**2).sum(axis=0)))
```

```
rng = np.random.default_rng(304)
PASSOS, TRAJ = 10 * DIAS, 400
dados = {nome: gerar_mundo(cfg, PASSOS, TRAJ, rng)[0] for nome, cfg in MUNDOS.items()}

linhas = []
for nome, r in dados.items():
    vol_movel = pd.DataFrame(r[:, :100]).rolling(DIAS).std().dropna().values * np.sqrt(DIAS)
    linhas.append({
        "Mundo": nome,
        "Vol. anual": float(np.mean(r.std(axis=0, ddof=1)) * np.sqrt(DIAS)),
        "Assimetria": float(skew(r, axis=0).mean()),
        "Curtose em excesso": float(kurtosis(r, axis=0).mean()),
        "ACF de r², defasagem 1": float(acf(r**2, [1])[0]),
        "ACF de r², defasagem 21": float(acf(r**2, [21])[0]),
        "Corr(r hoje, r² amanhã)": float(corr_alavancagem(r)),
        "Amplitude da vol. móvel": float(np.mean(vol_movel.max(axis=0) - vol_movel.min(axis=0))),
    })
res = pd.DataFrame(linhas).set_index("Mundo")
tabela(res.reset_index())

ok(np.allclose(res["Vol. anual"], 0.22, atol=0.012), "os cinco mundos têm a mesma volatilidade
↪  total")
ok(abs(res.loc["M0: browniano", "Curtose em excesso"]) < 0.05, "M0 não tem caudas pesadas")
ok(res.loc["M1: só saltos", "Curtose em excesso"] > 1, "saltos geram caudas pesadas")
ok(res.loc["M2: vol. estoc., ρ=0", "Curtose em excesso"] > 0.5, "volatilidade estocástica também
↪  gera caudas pesadas")
ok(abs(res.loc["M1: só saltos", "ACF de r², defasagem 1"]) < 0.02, "saltos sozinhos não geram
↪  agrupamento")
ok(res.loc["M2: vol. estoc., ρ=0", "ACF de r², defasagem 1"] > 0.05, "volatilidade estocástica
↪  gera agrupamento")
ok(abs(res.loc["M2: vol. estoc., ρ=0", "Corr(r hoje, r² amanhã)"]) < 0.02, "com ρ = 0 não há
↪  efeito alavancagem")
ok(res.loc["M3: vol. estoc., ρ<0", "Corr(r hoje, r² amanhã)"] < -0.03, "ρ negativo gera efeito
↪  alavancagem")
ok(res.loc["M1: só saltos", "Assimetria"] < res.loc["M0: browniano", "Assimetria"] - 0.2, "saltos
↪  negativos geram assimetria negativa")
```

Mundo	Vol. anual	Assimetria	Curtose em excesso	ACF de r², defasagem 1	ACF de r², defasagem 21	Corr(r hoje, r² amanhã)	Amplitude da vol. móvel
M0: browniano	0,220	0,004	-0,001	-0,001	-0,000	-0,001	0,044
M1: só saltos	0,220	-0,524	4,771	0,001	0,000	0,000	0,069
M2: vol. estoc., ρ=0	0,219	-0,013	1,878	0,154	0,117	-0,002	0,204
M3: vol. estoc., ρ<0	0,219	-0,003	1,836	0,152	0,114	-0,047	0,202
M4: completo	0,218	-0,580	6,929	0,076	0,056	-0,030	0,209

```
OK — os cinco mundos têm a mesma volatilidade total
OK — M0 não tem caudas pesadas
OK — saltos geram caudas pesadas
OK — volatilidade estocástica também gera caudas pesadas
OK — saltos sozinhos não geram agrupamento
OK — volatilidade estocástica gera agrupamento
OK — com ρ = 0 não há efeito alavancagem
OK — ρ negativo gera efeito alavancagem
OK — saltos negativos geram assimetria negativa
```

Como ler a tabela. Os mundos têm a mesma volatilidade total, mas comportamentos muito diferentes. A tabela mostra um mapa:

- **Caudas pesadas** têm duas origens possíveis: saltos (M1) ou volatilidade estocástica (M2). Olhar só para a curtose não permite distingui-las.
- **Agrupamento** vem da volatilidade estocástica (M2). Saltos independentes não

o produzem.
- **Efeito alavancagem** exige $\rho < 0$ (M3).

Dois detalhes do mundo M4. A curtose fica próxima da que o livro mede no DAX (7,206), o que torna o mundo um bom substituto nesse quesito; a assimetria, porém, é negativa, enquanto o DAX do período tem assimetria próxima de zero. E a autocorrelação de r^2 é menor que em M3: os saltos aumentam a variabilidade de r^2 sem criar dependência no tempo, diluindo a medida.

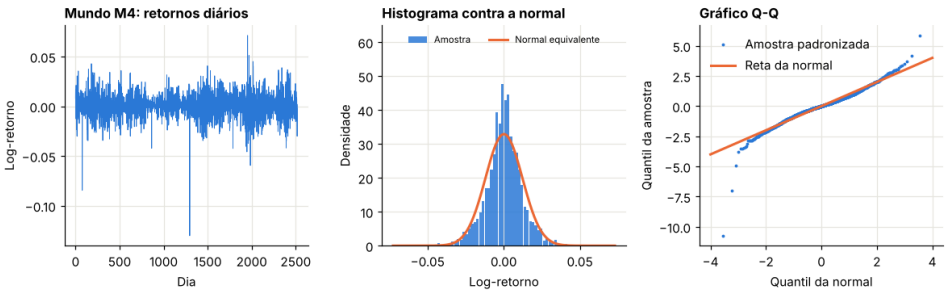
Laboratório 3.1 — As trajetórias e os gráficos diagnósticos

Abaixo, a série M4, em que todos os mecanismos estão ligados, nos mesmos gráficos usados para o browniano. Em seguida, o decaimento da autocorrelação e a medida de alavancagem para todos os mundos.

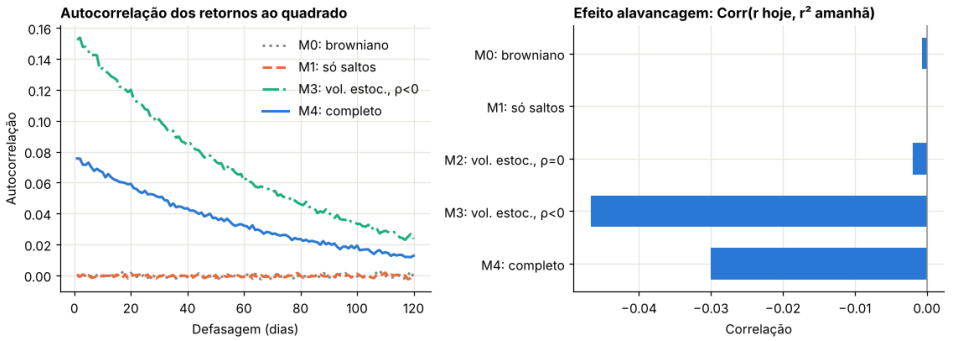
```
r_m4 = dados["M4: completo"][:, 0]
painel_distribuicao(r_m4, "Mundo M4")
tabela(pd.DataFrame({"Estatística": list(estatisticas(r_m4)), "Valor":
↪  list(estatisticas(r_m4).values())}))

defas = np.arange(1, 121)
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10.5, 3.9))
cores = {"M0: browniano": CINZA, "M1: só saltos": LARANJA, "M3: vol. estoc., ρ<0": VERDE, "M4:
↪  completo": AZUL}
estilos = {"M0: browniano": ":", "M1: só saltos": "--", "M3: vol. estoc., ρ<0": "-.", "M4:
↪  completo": "-"}
for nome in cores:
    a1.plot(defas, acf(dados[nome] ** 2, defas), color=cores[nome], ls=estilos[nome], label=nome)
a1.set_title("Autocorrelação dos retornos ao quadrado")
a1.set_xlabel("Defasagem (dias)")
a1.set_ylabel("Autocorrelação")
a1.legend()

nomes = list(res.index)
valores = res["Corr(r hoje, r² amanhã)"].values
a2.barh(nomes[::-1], valores[::-1], color=AZUL, height=0.6)
a2.axvline(0, color=CINZA, lw=1)
a2.set_title("Efeito alavancagem: Corr(r hoje, r² amanhã)")
a2.set_xlabel("Correlação")
plt.tight_layout()
plt.show()
```



Estatística	Valor
Média anualizada	0,035
Volatilidade anualizada	0,193
Assimetria	-0,596
p-valor assimetria	0,000
Curtose em excesso	7,347
p-valor curtose	0,000
p-valor normalidade	0,000
Volatilidade realizada	0,193



Laboratório 3.2 — Reversão à média: recuperando κ dos dados

No modelo de Heston, a autocorrelação de r_t^2 decai exponencialmente com a defasagem τ:

$$\mathrm{ACF}(\tau) \approx A\, e^{-\kappa \tau}. \tag{2.14}$$

Tirando o logaritmo, obtemos uma reta: $\ln \mathrm{ACF}(\tau) = \ln A - \kappa\tau$. A inclinação revela κ. A partir dele, a **meia-vida** de um choque de variância, tempo para perder metade do efeito, resolve $e^{-\kappa t} = \frac{1}{2}$, ou seja, $t = \ln 2/\kappa$.

Note que os saltos alteram A, mas não a taxa de decaimento, pois são independentes no tempo.

```
lags = np.arange(1, 121)
acf_m4 = acf(dados["M4: completo"] ** 2, lags)
incl = np.polyfit(lags / DIAS, np.log(acf_m4), 1)[0]
kappa_hat = -incl

tabela(pd.DataFrame([
    {"Grandeza": "κ verdadeiro", "Valor": KAPPA},
    {"Grandeza": "κ estimado pelo decaimento da ACF", "Valor": float(kappa_hat)},
    {"Grandeza": "Meia-vida verdadeira (dias)", "Valor": float(np.log(2) / KAPPA * DIAS)},
    {"Grandeza": "Meia-vida estimada (dias)", "Valor": float(np.log(2) / kappa_hat * DIAS)},
]))
ok(KAPPA * 0.7 < kappa_hat < KAPPA * 1.6, "o decaimento da autocorrelação dá a ordem de grandeza
↪  de κ")
```

Grandeza	Valor
κ verdadeiro	3,000
κ estimado pelo decaimento da ACF	3,863
Meia-vida verdadeira (dias)	58,224
Meia-vida estimada (dias)	45,215

```
OK — o decaimento da autocorrelação dá a ordem de grandeza de κ
```

A estimativa acerta a ordem de grandeza, mas sai acima do valor verdadeiro. Parte disso é efeito de amostra finita: ao subtrair a média de cada série de 10 anos, a autocorrelação nas defasagens longas fica subestimada e a curva parece cair mais rápido. A mesma família de viés reaparece na Seção 7.

Laboratório 3.3 — Quantos saltos existem?

O livro chama de salto qualquer retorno diário acima de $\pm 5\%$, reconhecendo que o critério é heurístico. Vamos aplicar o mesmo critério e depois um melhor.

Contagem por limiar. Sob a normal, $P(r < -0{,}05) = N\big((-0{,}05 - \hat{\mu})/\hat{\sigma}\big)$. Usamos primeiro as estatísticas que o próprio livro imprime para o DAX, com 31 ocorrências observadas em 2.557 dias.

> **Errata do livro:** o texto escreve $P(r_n > -0{,}05)$ onde deveria ser $P(r_n > +0{,}05)$. Além disso, as probabilidades que ele informa não fecham com a média e o desvio que ele próprio imprime, como a tabela mostra. A conclusão, porém, não muda.

```
mu_dax, s_dax, n_dax = 0.000348, 0.013761, 2557
p_baixo = norm.cdf((-0.05 - mu_dax) / s_dax)
p_alto = norm.sf((0.05 - mu_dax) / s_dax)

tabela(pd.DataFrame({
    "Cauda": ["r < −5%", "r > +5%"],
    "Prob. por 10.000 dias, valor do livro": [2.911, 3.402],
    "Prob. por 10.000 dias, recalculada": [float(p_baixo * 1e4), float(p_alto * 1e4)],
    "Ocorrências esperadas, valor do livro": [0.0002911 * n_dax, 0.0003402 * n_dax],
    "Ocorrências esperadas, recalculadas": [float(p_baixo * n_dax), float(p_alto * n_dax)],
}))
print("Ocorrências observadas no DAX (duas caudas), segundo o livro: 31")
```

Cauda	Prob. por 10.000 dias, valor do livro	Prob. por 10.000 dias, recalculada	Ocorrências esperadas, valor do livro	Ocorrências esperadas, recalculadas
r < −5%	2,911	1,267	0,744	0,324
r > +5%	3,402	1,542	0,870	0,394

```
Ocorrências observadas no DAX (duas caudas), segundo o livro: 31
```

Esperava-se menos de uma ocorrência por cauda, e houve 31 no total. Isso é evidência de caudas pesadas, mas não prova a existência de saltos. Nos mundos sintéticos, dá para verificar: **volatilidade estocástica sozinha também produz retornos acima de 5%**.

Medida melhor: variação bipotência. A variância realizada soma todos os movimentos ao quadrado:

$$RV = \sum_n r_n^2 \;\longrightarrow\; \int \sigma_t^2 \, dt + \sum J^2. \tag{2.15}$$

Já a variação bipotência multiplica retornos **vizinhos** em valor absoluto:

$$BV = \frac{\pi}{2} \sum_n |r_n| \, |r_{n-1}| \;\longrightarrow\; \int \sigma_t^2 \, dt. \tag{2.16}$$

A ideia: um salto é raro, então um retorno enorme fica ao lado de um retorno normal, e o produto continua pequeno. O salto entra em RV inteiro, mas quase não entra em BV. Assim, $(RV - BV)/RV$ estima a parcela da variância devida a saltos. O fator $\pi/2$ corrige o fato de que $\mathbb{E}|Z| = \sqrt{2/\pi}$.

```
linhas = []
limiar = 0.05
for nome, r in dados.items():
    rv = (r**2).sum(axis=0)
    bv = (np.pi / 2) * (np.abs(r[1:]) * np.abs(r[:-1])).sum(axis=0)
    parcela = 1 - bv / rv
    n_ext = (np.abs(r) > limiar).sum(axis=0)
    verdade = (MUNDOS[nome]["lam"] * (MU_J**2 + DELTA_J**2)) / V_TOTAL
    linhas.append({"Mundo": nome, "Retornos acima de ±5% por década": float(n_ext.mean()),
 ↪  "Parcela de saltos verdadeira": float(verdade), "Parcela estimada (RV−BV)/RV":
 ↪  float(parcela.mean())})
sal = pd.DataFrame(linhas).set_index("Mundo")
tabela(sal.reset_index())

ok(sal.loc["M3: vol. estoc., ρ<0", "Retornos acima de ±5% por década"] > 5 * max(sal.loc["M0:
 ↪  browniano", "Retornos acima de ±5% por década"], 0.1), "volatilidade estocástica sem salto
 ↪  gera muitos retornos acima de 5%")
ok(abs(sal.loc["M3: vol. estoc., ρ<0", "Parcela estimada (RV−BV)/RV"]) < 0.04, "a bipotência não
 ↪  confunde volatilidade estocástica com salto")
ok(abs(sal.loc["M4: completo", "Parcela estimada (RV−BV)/RV"] - sal.loc["M4: completo", "Parcela
 ↪  de saltos verdadeira"]) < 0.06, "a bipotência recupera a parcela de saltos no mundo completo")
```

Mundo	Retornos acima de ±5% por década	Parcela de saltos verdadeira	Parcela estimada (RV−BV)/RV
M0: browniano	0,833	0,000	0,001
M1: só saltos	4,935	0,085	0,054
M2: vol. estoc., ρ=0	11,227	0,000	0,004
M3: vol. estoc., ρ<0	11,012	0,000	0,006
M4: completo	13,455	0,085	0,065

```
OK — volatilidade estocástica sem salto gera muitos retornos acima de 5%
OK — a bipotência não confunde volatilidade estocástica com salto
OK — a bipotência recupera a parcela de saltos no mundo completo
```

Leitura. A bipotência acerta o zero nos mundos sem saltos, em que a contagem por limiar enganaria. Nos mundos com saltos, ela subestima um pouco: com dados diários, saltos pequenos se confundem com a difusão. Há estimativas melhores com dados intradiários.

4. O mercado de opções: bid, ask e o limite da precisão

Livro: §3.5.1

Opções não têm um preço, mas ao menos dois: o **bid**, pelo qual o formador de mercado compra, e o **ask**, pelo qual ele vende. Os dados que o livro apresenta, de opções sobre ações do índice Dow Jones entre 1996 e 2010, mostram spread médio de 0,227 dólar, cerca de 7,920% do preço médio, e de 18,720% nas opções de curto prazo fora do dinheiro.

A consequência prática é direta: **não faz sentido exigir de um modelo precisão maior do que a do próprio mercado**. Um erro de calibração dentro do spread não é erro.

Traduzindo o spread para volatilidade

A aproximação de primeira ordem é:

$$\Delta\sigma \approx \frac{\Delta C}{\text{vega}}. \tag{2.17}$$

O vega é pequeno nas opções muito fora do dinheiro e de prazo curto. Logo, o **mesmo spread em preço corresponde a uma incerteza enorme em volatilidade**. Esse é o motivo pelo qual a calibração costuma ponderar os erros, tema do módulo 13.

Laboratório 4.1 — Incerteza em volatilidade implícita

Partimos de $S_0 = 100$, $r = 2{,}5\%$, volatilidade de 20%, e meio-spread de 0,05. Para cada opção, medimos a distância entre as volatilidades implícitas calculadas no bid e no ask. Quando o preço é menor que o meio-spread, o bid seria zero e a volatilidade inferior não existe: a célula fica `n/d`.

```
def iv_brent(preco, S, K, T, r):
    """Volatilidade implícita por bracketing. Devolve NaN fora dos limites de não arbitragem."""
    piso = max(S - K * np.exp(-r * T), 0.0)
    if not (piso < preco < S):
        return np.nan
    return brentq(lambda s: float(bsm(S, K, T, r, s)) - preco, 1e-5, 8.0, xtol=1e-10)


S0, r, sigma_ref, meio = 100.0, 0.025, 0.20, 0.05
prazos = [("1 mês", 1 / 12), ("3 meses", 0.25), ("1 ano", 1.0)]
strikes = [85.0, 90.0, 95.0, 100.0, 105.0, 110.0, 115.0, 125.0]

linhas = {}
for rot, T in prazos:
    col = []
    for K in strikes:
        c = float(bsm(S0, K, T, r, sigma_ref))
        baixo = iv_brent(c - meio, S0, K, T, r) if c > meio else np.nan
        alto = iv_brent(c + meio, S0, K, T, r)
        col.append((alto - baixo) * 100)
    linhas[rot] = col
larg = pd.DataFrame(linhas, index=[f"K = {k:.0f}" for k in strikes])
print("Largura bid-ask em pontos percentuais de volatilidade:")
tabela(larg.reset_index(names="Strike"))
```

```
ok(larg.loc["K = 100", "1 mês"] > larg.loc["K = 100", "1 ano"], "no dinheiro, prazo curto tem
↪  incerteza maior")
ok(larg.loc["K = 115", "1 ano"] > larg.loc["K = 100", "1 ano"], "longe do dinheiro, a incerteza
↪  em volatilidade aumenta")
```

```
Largura bid-ask em pontos percentuais de volatilidade:
```

Strike	1 mês	3 meses	1 ano
K = 85	n/d	2,300	0,429
K = 90	5,908	0,990	0,333
K = 95	1,369	0,610	0,281
K = 100	0,870	0,505	0,257
K = 105	1,178	0,538	0,251
K = 110	3,122	0,714	0,259
K = 115	n/d	1,147	0,280
K = 125	n/d	5,860	0,373

```
OK — no dinheiro, prazo curto tem incerteza maior
OK — longe do dinheiro, a incerteza em volatilidade aumenta
```

---

5. Volatilidade implícita: definição e cálculo

Livro: §3.5.2

A volatilidade implícita σ^{imp} é o número que resolve

$$C^{BSM}(S_0, K, T, r, \sigma^{imp}) = C^{\text{mercado}}. \tag{2.18}$$

Mais do que uma estimativa de volatilidade futura, ela é **uma forma de cotar preço**, numa unidade comparável entre strikes e prazos. Ao longo do livro e do curso, "implícita" sempre significa implícita no BSM.

A solução existe e é única? Passo a passo

Passo 1: limites. Quando $\sigma \to 0$, a call tende a $\max(S_0 - Ke^{-rT}, 0)$. Quando $\sigma \to \infty$, tende a S_0. Logo, um preço entre esses limites está no contradomínio da função.

Passo 2: derivada em relação a σ (vega). Primeiro, uma identidade útil. Calculando a razão das densidades normais:

$$\frac{\varphi(d_2)}{\varphi(d_1)} = \exp\left(\frac{d_1^2 - d_2^2}{2}\right) = \exp\left(d_1\sigma\sqrt{T} - \frac{\sigma^2 T}{2}\right) = \exp\left(\ln(S_0/K) + rT\right) = \frac{S_0 e^{rT}}{K}. \tag{2.19}$$

Logo, $S_0\,\varphi(d_1) = Ke^{-rT}\,\varphi(d_2)$. Agora derivamos $C = S_0 N(d_1) - Ke^{-rT}N(d_2)$:

$$\frac{\partial C}{\partial \sigma} = S_0\varphi(d_1)\frac{\partial d_1}{\partial \sigma} - Ke^{-rT}\varphi(d_2)\frac{\partial d_2}{\partial \sigma}. \tag{2.20}$$

Pela identidade, os dois coeficientes são iguais. Como $d_2 = d_1 - \sigma\sqrt{T}$, temos $\partial d_1/\partial\sigma - \partial d_2/\partial\sigma = \sqrt{T}$, e sobra

$$\text{vega} = S_0\,\varphi(d_1)\,\sqrt{T} > 0. \tag{2.21}$$

Passo 3: conclusão. A call é contínua e **estritamente crescente** em σ, então cada preço válido corresponde a exatamente uma volatilidade.

Passo 4: put e call têm a mesma implícita. A paridade $C - P = S_0 - Ke^{-rT}$ não depende de volatilidade. Se σ^* reprecifica a call de mercado, então o mesmo σ^* reprecifica a put que respeita a paridade.

Como calcular: Newton e seu ponto fraco

O método de Newton atualiza

$$\sigma_{n+1} = \sigma_n - \frac{C(\sigma_n) - C^{\text{mercado}}}{\text{vega}(\sigma_n)}. \tag{2.22}$$

Com um chute ruim e vega perto de zero, a divisão gera um passo gigantesco. O livro usa `fsolve` partindo sempre de $\sigma = 0{,}2$.

Existe um ponto de partida melhor. Para encontrá-lo, derivamos o vega. Sabendo que $\partial d_1/\partial\sigma = -d_2/\sigma$:

$$\frac{\partial\,\text{vega}}{\partial\sigma} = \text{vega}\cdot\frac{d_1 d_2}{\sigma}. \tag{2.23}$$

Com $m = \ln(S_0/K) + rT$, vale $d_1 d_2 = \dfrac{m^2}{\sigma^2 T} - \dfrac{\sigma^2 T}{4}$. Essa expressão muda de sinal em

$$\sigma^* = \sqrt{\frac{2|m|}{T}}. \tag{2.24}$$

Abaixo de σ^*, a função é convexa; acima, côncava. Partindo exatamente do ponto de inflexão, Newton converge de forma **monotônica** (Manaster e Koehler, 1982). O chute deixa de ser arbitrário.

Laboratório 5.1 — Corrida entre os métodos

Geramos preços de um conjunto amplo de opções com volatilidade conhecida e tentamos recuperá-la.

```python
def vega(S, K, T, r, s):
    d1 = (np.log(S / K) + (r + 0.5 * s**2) * T) / (s * np.sqrt(T))
    return S * norm.pdf(d1) * np.sqrt(T)


def newton_iv(preco, S, K, T, r, chute, max_iter=100):
    s = chute
    for _ in range(max_iter):
        if not np.isfinite(s) or s <= 0:
            return np.nan
        dif = float(bsm(S, K, T, r, s)) - preco
        if abs(dif) < 1e-11:
            return s
        v = vega(S, K, T, r, s)
```

```
        if v < 1e-300:
            return np.nan
        s = s - dif / v
    return np.nan


def chute_mk(S, K, T, r):
    return np.sqrt(2 * abs(np.log(S / K) + r * T) / T) if K != S * np.exp(r * T) else 0.2


def fsolve_iv(preco, S, K, T, r, chute=0.2):
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        return float(fsolve(lambda x: float(bsm(S, K, T, r, x[0])) - preco, [chute])[0])


casos = []
for K in np.arange(50.0, 251.0, 10.0):
    for T in [0.02, 0.1, 0.5, 1.0, 2.0]:
        for s in [0.05, 0.1, 0.2, 0.4, 0.8, 1.5]:
            p = float(bsm(100.0, K, T, 0.025, s))
            piso = max(100.0 - K * np.exp(-0.025 * T), 0.0)
            if p - piso > 1e-6:
                casos.append((p, K, T, s))

metodos = {
    "fsolve com chute 0,2 (livro)": lambda p, K, T: fsolve_iv(p, 100.0, K, T, 0.025),
    "Newton com chute 0,2": lambda p, K, T: newton_iv(p, 100.0, K, T, 0.025, 0.2),
    "Newton com chute de inflexão": lambda p, K, T: newton_iv(p, 100.0, K, T, 0.025,
    ↪  chute_mk(100.0, K, T, 0.025)),
    "Brent com intervalo": lambda p, K, T: iv_brent(p, 100.0, K, T, 0.025),
}
linhas = []
acertos = {}
for nome, func in metodos.items():
    with np.errstate(all="ignore"):
        acerto = np.array([abs(func(p, K, T) - s) < 1e-4 for p, K, T, s in casos])
    acertos[nome] = acerto.mean()
    linhas.append({"Método": nome, "Casos": len(casos), "Taxa de sucesso (%)":
↪  float(acerto.mean() * 100)})
tabela(pd.DataFrame(linhas))

ok(acertos["Newton com chute 0,2"] < 0.95, "Newton com chute fixo falha em parte dos casos")
ok(acertos["Newton com chute de inflexão"] > 0.999, "Newton a partir da inflexão resolve
↪  praticamente todos")
ok(acertos["Brent com intervalo"] > 0.999, "Brent com intervalo resolve praticamente todos")
```

Método	Casos	Taxa de sucesso (%)
fsolve com chute 0,2 (livro)	403	87,097
Newton com chute 0,2	403	52,854
Newton com chute de inflexão	403	100,000
Brent com intervalo	403	100,000

```
OK — Newton com chute fixo falha em parte dos casos
OK — Newton a partir da inflexão resolve praticamente todos
OK — Brent com intervalo resolve praticamente todos
```

6. A superfície de volatilidade

Livro: §3.5.2 e Figura 3.11

Se o BSM fosse o modelo verdadeiro, todas as opções teriam a mesma implícita. Na prática, ela varia com o strike e com o prazo. Essa função é a **superfície de volatilidade**.

Os dois fatos que o livro destaca:

- **Smile/skew:** a implícita depende do strike. Em índices de ações, ela é maior nos strikes baixos.
- **Estrutura a termo:** o skew é mais acentuado nos prazos curtos.

> **Errata do livro:** o texto diz que, para calls, as implícitas **fora** do dinheiro são maiores que as ATM. A Figura 3.11 do próprio livro mostra o oposto: as implícitas altas estão nos strikes baixos, em que as calls estão **dentro** do dinheiro (e as puts, fora). A frase seria correta se dissesse *puts*.

O motor de preços

Geramos a superfície pelo mesmo modelo do mundo M4, sob a medida neutra ao risco. Dois avisos:

- Por simplicidade, usamos os mesmos parâmetros de volatilidade nas medidas real e neutra. Na realidade, implícitas costumam ficar acima da volatilidade realizada, o chamado prêmio de risco de variância.
- O preço vem da fórmula de Lewis (2001), uma integral da função característica do log-preço, que é uma caixa-preta até os módulos 7 e 11:

$$C_0 = S_0 - \frac{\sqrt{S_0 K}\, e^{-rT}}{\pi} \int_0^\infty \mathrm{Re}\Big[e^{iuk}\, \phi(u - \tfrac{i}{2})\Big] \frac{du}{u^2 + \frac{1}{4}}, \qquad k = \ln \frac{S_0}{K}. \tag{2.25}$$

Antes de usá-la, conferimos com a fórmula BSM e com Monte Carlo.

```
def cf_bates(u, T, r, p):
    """Função característica de ln(S_T/S_0) no modelo de Heston com saltos, sob Q."""
    comp = p["lam"] * (np.exp(p["mu_j"] + 0.5 * p["delta_j"] ** 2) - 1)
    deriva = np.exp(1j * u * (r - comp) * T)
    salto = np.exp(p["lam"] * T * (np.exp(1j * u * p["mu_j"] - 0.5 * u**2 * p["delta_j"] ** 2) -
↪   1))
    if p["nu"] == 0:
        return deriva * salto * np.exp(-0.5 * p["v0"] * (1j * u + u**2) * T)
    xi = p["kappa"] - p["rho"] * p["nu"] * 1j * u
    d = np.sqrt(xi**2 + p["nu"] ** 2 * (1j * u + u**2))
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
    A = p["kappa"] * p["theta"] / p["nu"] ** 2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / p["nu"] ** 2 * (1 - e) / (1 - g * e)
    return deriva * salto * np.exp(A + B * p["v0"])


U = np.linspace(1e-8, 700.0, 14001)


def call_lewis(S, K, T, r, p):
```

```
    """Call europeia pela integral de Lewis (2001). Aceita array em K."""
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(S / K)[:, None]
    fator = np.exp(1j * U * k) * cf_bates(U - 0.5j, T, r, p)
    integral = simpson(fator.real / (U**2 + 0.25), x=U, axis=1)
    return S - np.sqrt(S * K) * np.exp(-r * T) / np.pi * integral


def params(nome):
    c = MUNDOS[nome]
    return dict(v0=c["theta"], theta=c["theta"], kappa=c["kappa"], nu=c["nu"], rho=c["rho"],
     ↪  lam=c["lam"], mu_j=MU_J, delta_j=DELTA_J)


P_M4 = params("M4: completo")
S0, r = 100.0, 0.025

# Conferência 1: sem volatilidade estocástica e sem saltos, Lewis tem que dar BSM
p_flat = params("M0: browniano")
Kt = np.array([80.0, 100.0, 120.0])
ok(np.allclose(call_lewis(S0, Kt, 0.5, r, p_flat), bsm(S0, Kt, 0.5, r, np.sqrt(V_TOTAL)),
 ↪  atol=1e-7), "Lewis coincide com BSM no caso sem saltos e sem vol. estocástica")

# Conferência 2: no modelo completo, comparar com Monte Carlo
rng = np.random.default_rng(306)
X = gerar_mundo(MUNDOS["M4: completo"], DIAS, 300_000, rng, mu=r, guardar=False)
ST = S0 * np.exp(X)
linhas = []
for K in [80.0, 100.0, 120.0]:
    a = np.exp(-r) * np.maximum(ST - K, 0)
    mc, ep = a.mean(), a.std(ddof=1) / np.sqrt(a.size)
    lw = float(call_lewis(S0, K, 1.0, r, P_M4)[0])
    linhas.append({"Strike": K, "Lewis": lw, "Monte Carlo": mc, "Erro-padrão": ep, "Diferença em
 ↪  erros-padrão": (mc - lw) / ep})
cmp_mc = pd.DataFrame(linhas)
tabela(cmp_mc)
ok((cmp_mc["Diferença em erros-padrão"].abs() < 4).all(), "Lewis confere com Monte Carlo no
 ↪  modelo completo")
```

```
OK — Lewis coincide com BSM no caso sem saltos e sem vol. estocástica
```

Strike	Lewis	Monte Carlo	Erro-padrão	Diferença em erros-padrão
80,000	23,725	23,753	0,033	0,860
100,000	9,654	9,671	0,023	0,726
120,000	2,245	2,251	0,011	0,566

```
OK — Lewis confere com Monte Carlo no modelo completo
```

Laboratório 6.1 — A superfície do mundo completo

Desenhamos as fatias da superfície por prazo, usando no eixo horizontal o **log-moneyness a termo**, $x = \ln(K/F)$ com $F = S_0 e^{rT}$. Esse eixo torna os prazos comparáveis. Os prazos em dias são os usados no livro, somados a dois mais longos.

Medimos o skew pela inclinação da implícita em torno de $x = 0$:

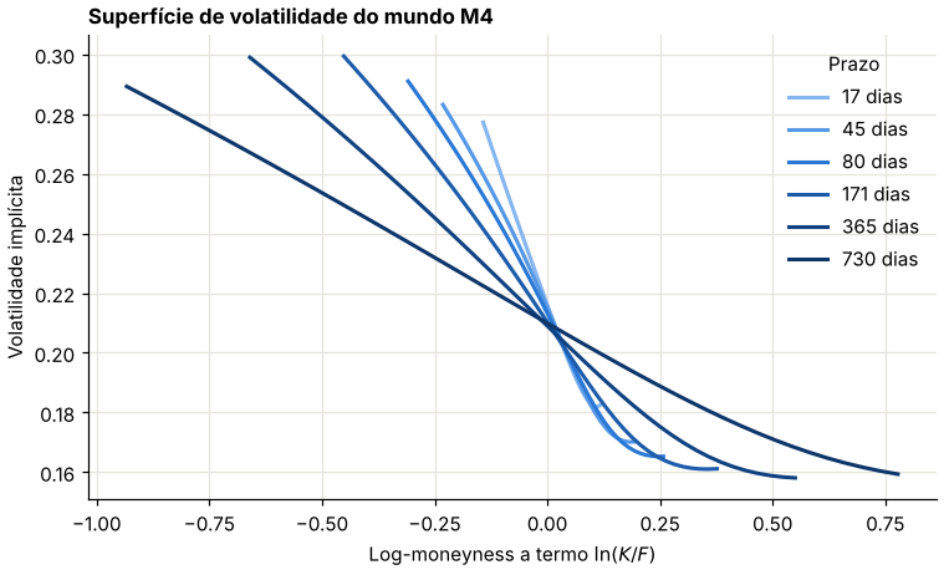
$$\text{skew}(T) = \frac{\sigma^{imp}(x = +h) - \sigma^{imp}(x = -h)}{2h}, \quad h = 0{,}02. \tag{2.26}$$

```python
def iv_superficie(x, T, p):
    """Implícita BSM em log-moneyness a termo x, para um vetor de x."""
    F = S0 * np.exp(r * T)
    K = F * np.exp(x)
    c = call_lewis(S0, K, T, r, p)
    return np.array([iv_brent(ci, S0, Ki, T, r) if ci > 1e-7 else np.nan for ci, Ki in zip(c, K)])


dias = [17, 45, 80, 171, 365, 730]
fig, ax = plt.subplots(figsize=(8, 4.4))
for d, cor in zip(dias, RAMPA):
    T = d / 365
    largura = 0.22 * np.sqrt(T)
    x = np.linspace(-3 * largura, 2.5 * largura, 45)
    ax.plot(x, iv_superficie(x, T, P_M4), color=cor, label=f"{d} dias")
ax.set_title("Superfície de volatilidade do mundo M4")
ax.set_xlabel("Log-moneyness a termo $\\ln(K/F)$")
ax.set_ylabel("Volatilidade implícita")
ax.legend(title="Prazo")
plt.show()


def skew_atm(T, p, h=0.02):
    v = iv_superficie(np.array([-h, h]), T, p)
    return (v[1] - v[0]) / (2 * h)


grade_T = np.array([17, 30, 45, 80, 120, 171, 365, 730]) / 365
s_t = np.array([skew_atm(T, P_M4) for T in grade_T])
tabela(pd.DataFrame({"Prazo (dias)": [int(round(t * 365)) for t in grade_T], "Skew ATM": s_t}))
incl_skew = np.polyfit(np.log(grade_T), np.log(-s_t), 1)[0]
print("Inclinação log-log do skew em relação ao prazo:", f3(incl_skew))
ok(np.all(s_t < 0), "skew negativo em todos os prazos")
ok(np.all(np.diff(np.abs(s_t)) < 0), "o skew diminui em módulo à medida que o prazo aumenta")
```



Prazo (dias)	Skew ATM
17	-0,411
30	-0,376
45	-0,348
80	-0,302
120	-0,264
171	-0,227
365	-0,146
730	-0,087

```
Inclinação log-log do skew em relação ao prazo: -0,402
OK — skew negativo em todos os prazos
OK — o skew diminui em módulo à medida que o prazo aumenta
```

Laboratório 6.2 — Qual mecanismo desenha qual forma?

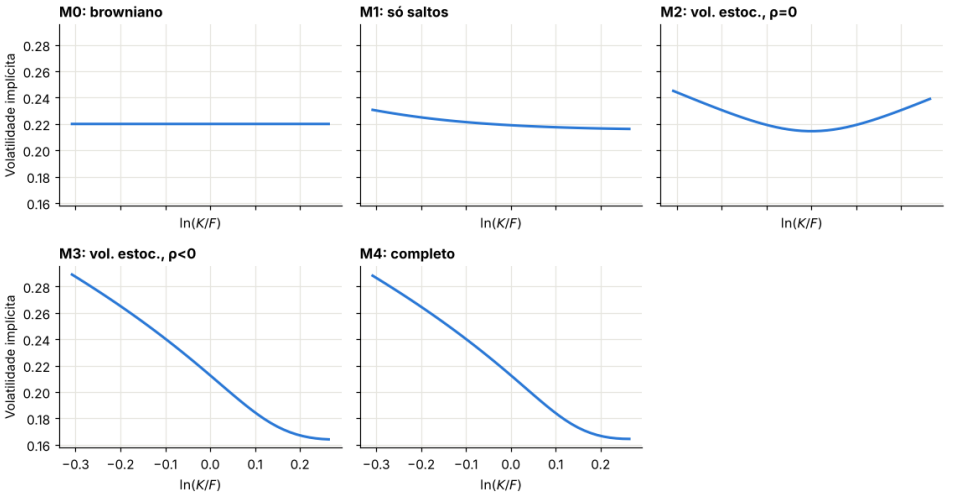
Voltamos aos cinco mundos da Seção 3, agora olhando para as opções, sempre em prazo de 3 meses. Compare com a tabela da Seção 3: cada mecanismo que deixava marca nos retornos deixa também uma marca na superfície.

```
T_f = 0.25
larg = 0.22 * np.sqrt(T_f)
x = np.linspace(-2.8 * larg, 2.4 * larg, 41)

fig, eixos = plt.subplots(2, 3, figsize=(10.5, 5.6), sharey=True, sharex=True)
eixos = eixos.ravel()
resultado = []
for ax, nome in zip(eixos, MUNDOS):
    p = params(nome)
    v = iv_superficie(x, T_f, p)
    ax.plot(x, v, color=AZUL)
    ax.set_title(nome)
    ax.set_xlabel("$\\ln(K/F)$")
```

```
    resultado.append({"Mundo": nome, "Skew ATM": skew_atm(T_f, p), "Curvatura":
↪  float((iv_superficie(np.array([-0.1]), T_f, p)[0] + iv_superficie(np.array([0.1]), T_f, p)[0]
↪  - 2 * iv_superficie(np.array([0.0]), T_f, p)[0]) / 0.01)})
eixos[0].set_ylabel("Volatilidade implícita")
eixos[3].set_ylabel("Volatilidade implícita")
eixos[5].axis("off")
plt.tight_layout()
plt.show()

mec = pd.DataFrame(resultado).set_index("Mundo")
tabela(mec.reset_index())
ok(abs(mec.loc["M0: browniano", "Skew ATM"]) < 1e-3 and abs(mec.loc["M0: browniano",
↪  "Curvatura"]) < 1e-2, "BSM tem superfície plana")
ok(abs(mec.loc["M2: vol. estoc., ρ=0", "Skew ATM"]) < 0.01 and mec.loc["M2: vol. estoc., ρ=0",
↪  "Curvatura"] > 0.02, "vol. estocástica com ρ = 0 gera smile simétrico")
ok(mec.loc["M3: vol. estoc., ρ<0", "Skew ATM"] < -0.1, "ρ negativo inclina o smile")
ok(mec.loc["M1: só saltos", "Skew ATM"] < -0.01, "saltos negativos também geram skew")
ok(mec.loc["M4: completo", "Skew ATM"] < mec.loc["M3: vol. estoc., ρ<0", "Skew ATM"] * 0.5, "o
↪  mundo completo mantém skew pronunciado")
```



Mundo	Skew ATM	Curvatura
M0: browniano	-0,000	0,000
M1: só saltos	-0,019	0,085
M2: vol. estoc., ρ=0	0,000	0,957
M3: vol. estoc., ρ<0	-0,290	-0,042
M4: completo	-0,291	-0,118

```
OK — BSM tem superfície plana
OK — vol. estocástica com ρ = 0 gera smile simétrico
OK — ρ negativo inclina o smile
OK — saltos negativos também geram skew
OK — o mundo completo mantém skew pronunciado
```

Laboratório 6.3 — Saltos e correlação em prazos diferentes

Comparamos o skew dos dois mecanismos isolados em um prazo curto e em um longo.

```
linhas = []
for d in [17, 365]:
    T = d / 365
    s1 = skew_atm(T, params("M1: só saltos"))
    s3 = skew_atm(T, params("M3: vol. estoc., ρ<0"))
    linhas.append({"Prazo (dias)": d, "Skew só saltos": s1, "Skew só correlação": s3, "Razão
↪  saltos/correlação": s1 / s3})
dec = pd.DataFrame(linhas)
tabela(dec)
ok(dec["Razão saltos/correlação"].iloc[0] > dec["Razão saltos/correlação"].iloc[1], "o peso
↪  relativo dos saltos no skew é maior no prazo curto")
```

Prazo (dias)	Skew só saltos	Skew só correlação	Razão saltos/correlação
17	-0,065	-0,346	0,188
365	-0,005	-0,156	0,033

```
OK — o peso relativo dos saltos no skew é maior no prazo curto
```

O mapa completo. Volatilidade estocástica cria **curvatura**. A correlação negativa **inclina** a curva. Saltos negativos também inclinam, com um peso que cresce nos prazos curtos.

Complemento: a superfície não pode ter qualquer forma

O conjunto de preços precisa satisfazer condições de não arbitragem, que não dependem de modelo:

Condição	Forma	Por quê
Monotonicidade	C decresce com K	Comprar por menos nunca pode valer menos
Borboleta	C é convexa em K	Uma borboleta tem payoff ≥ 0, então seu custo é ≥ 0
Calendário	$w(x,T) = \sigma^2(x,T)\,T$ cresce com T	A variância total não pode encolher, a x fixo

Da convexidade à densidade. Derivando $C(K) = e^{-rT}\int_K^\infty (s-K)\,q(s)\,ds$ duas vezes em relação a K:

$$\frac{\partial C}{\partial K} = -e^{-rT}\int_K^\infty q(s)\,ds, \qquad \frac{\partial^2 C}{\partial K^2} = e^{-rT} q(K). \tag{2.27}$$

Logo, $q(K) = e^{rT}\,\partial^2 C/\partial K^2$. Esse é o resultado de **Breeden e Litzenberger (1978)**: os preços de opções revelam a própria distribuição neutra ao risco. E a borboleta ter custo não negativo equivale a $q \geq 0$.

Laboratório 6.4 — Densidade implícita e detecção de arbitragem

```
T_d = 0.5
F = S0 * np.exp(r * T_d)
h_k = 0.5
K_g = np.arange(20.0, 260.0 + h_k, h_k)
C_g = call_lewis(S0, K_g, T_d, r, P_M4)
q = np.exp(r * T_d) * (C_g[2:] - 2 * C_g[1:-1] + C_g[:-2]) / h_k**2
K_i = K_g[1:-1]

vol_atm = iv_superficie(np.array([0.0]), T_d, P_M4)[0]
lognormal = norm.pdf((np.log(K_i / F) + 0.5 * vol_atm**2 * T_d) / (vol_atm * np.sqrt(T_d))) /
↪  (K_i * vol_atm * np.sqrt(T_d))

integral = float(np.sum(q) * h_k)
media = float(np.sum(K_i * q) * h_k)
p_cauda_q = float(np.sum(q[K_i < 80]) * h_k)
p_cauda_ln = float(np.sum(lognormal[K_i < 80]) * h_k)
tabela(pd.DataFrame([
    {"Grandeza": "Integral da densidade", "Valor": integral},
    {"Grandeza": "Média da densidade", "Valor": media},
    {"Grandeza": "Forward teórico", "Valor": float(F)},
    {"Grandeza": "P(S_T < 80): densidade implícita", "Valor": p_cauda_q},
    {"Grandeza": "P(S_T < 80): lognormal com vol. ATM", "Valor": p_cauda_ln},
]))

fig, ax = plt.subplots()
ax.plot(K_i, q, color=AZUL, label="Densidade implícita (M4)")
ax.plot(K_i, lognormal, color=LARANJA, ls="--", label="Lognormal com vol. ATM")
ax.set_xlim(50, 150)
ax.set_title("Densidade neutra ao risco extraída dos preços, prazo de 6 meses")
ax.set_xlabel("$S_T$")
ax.set_ylabel("Densidade")
ax.legend()
plt.show()

ok(abs(integral - 1) < 0.01, "a densidade implícita integra 1")
ok(abs(media - F) < 0.15, "a média da densidade é o forward")
ok(p_cauda_q > p_cauda_ln * 1.2, "a cauda esquerda implícita é mais pesada que a lognormal")


def verificar(C, K, tol=1e-8):
    """Conta violações de monotonicidade e de convexidade numa fatia de calls."""
    dC = np.diff(C) / np.diff(K)
    return {"monotonicidade": int((dC > tol).sum()), "borboleta": int((np.diff(dC) < -tol).sum())}


K_v = np.arange(60.0, 141.0, 1.0)
T_v = 0.25
C_limpo = call_lewis(S0, K_v, T_v, r, P_M4)
iv_limpa = np.array([iv_brent(c, S0, k, T_v, r) for c, k in zip(C_limpo, K_v)])

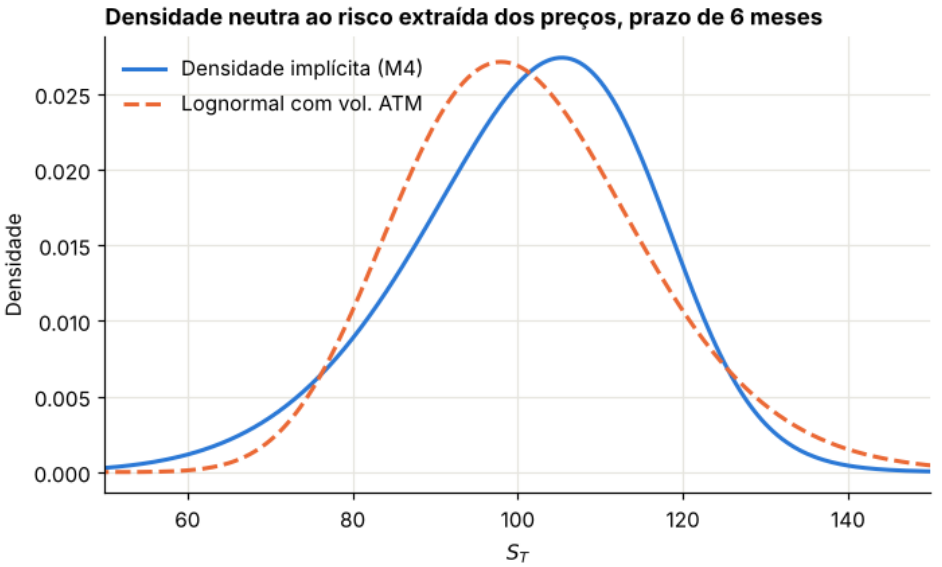
iv_ruim = iv_limpa.copy()
iv_ruim[K_v == 100.0] += 0.02
C_ruim = np.array([float(bsm(S0, k, T_v, r, s)) for k, s in zip(K_v, iv_ruim)])

print("Fatia sem interferência:", verificar(C_limpo, K_v))
print("Fatia com +2 p.p. em um único strike:", verificar(C_ruim, K_v))
ok(sum(verificar(C_limpo, K_v).values()) == 0, "a superfície do modelo não tem arbitragem de
↪  strike")
```

```
ok(verificar(C_ruim, K_v)["borboleta"] > 0, "um erro de 2 p.p. em um único strike cria borboleta
↪  negativa")

x_c = np.linspace(-0.2, 0.15, 15)
w = np.array([iv_superficie(x_c, d / 365, P_M4) ** 2 * d / 365 for d in dias])
ok(np.all(np.diff(w, axis=0) > 0), "a variância total cresce com o prazo: sem arbitragem de
↪  calendário")
```

Grandeza	Valor
Integral da densidade	1,000
Média da densidade	101,258
Forward teórico	101,258
P(S_T < 80): densidade implícita	0,087
P(S_T < 80): lognormal com vol. ATM	0,063




```
OK — a densidade implícita integra 1
OK — a média da densidade é o forward
OK — a cauda esquerda implícita é mais pesada que a lognormal
```

```
Fatia sem interferência: {'monotonicidade': 0, 'borboleta': 0}
Fatia com +2 p.p. em um único strike: {'monotonicidade': 0, 'borboleta': 1}
OK — a superfície do modelo não tem arbitragem de strike
OK — um erro de 2 p.p. em um único strike cria borboleta negativa
```

```
OK — a variância total cresce com o prazo: sem arbitragem de calendário
```

7. Taxas de juros curtas

Livro: §3.6

Taxas são a base do desconto em qualquer modelo. A evidência empírica é menos consensual do que no caso das ações, mas o livro destaca quatro fatos:

Fato	Descrição
Positividade	taxas nominais em geral são positivas
Aleatoriedade	movem-se de forma não previsível
Reversão à média	não sobem nem caem indefinidamente
Estrutura a termo	prazos diferentes têm taxas diferentes

> **Aviso de data.** O livro é de 2015. Depois da sua publicação, as taxas Euribor ficaram negativas por vários anos. A positividade não é uma lei, e modelos como o CIR, que usaremos, não conseguem representar taxas negativas sem modificações.

O modelo CIR

Cox, Ingersoll e Ross (1985) propõem

$$dr_t = \kappa_r(\theta_r - r_t)\,dt + \sigma_r\sqrt{r_t}\,dZ_t. \tag{2.28}$$

É a mesma equação usada para a variância no Heston. O termo $\sqrt{r_t}$ faz o choque encolher à medida que a taxa se aproxima de zero, e a condição de Feller, $2\kappa_r\theta_r \geq \sigma_r^2$, garante que o zero não seja atingido.

Para comparação, o modelo de **Vasicek** tem choque constante:

$$dr_t = \kappa_r(\theta_r - r_t)\,dt + \sigma_v\,dZ_t. \tag{2.29}$$

Sua distribuição é normal, e por isso pode ficar negativa.

A estrutura a termo no CIR

O preço de um título zero-cupom tem solução fechada, deduzida no módulo 11, $P(0,\tau) = A(\tau)e^{-B(\tau)r_0}$, com

$$\gamma = \sqrt{\kappa_r^2 + 2\sigma_r^2}, \qquad B(\tau) = \frac{2(e^{\gamma\tau}-1)}{(\gamma+\kappa_r)(e^{\gamma\tau}-1)+2\gamma}, \tag{2.30}$$

$$A(\tau) = \left[\frac{2\gamma\, e^{(\kappa_r+\gamma)\tau/2}}{(\gamma+\kappa_r)(e^{\gamma\tau}-1)+2\gamma}\right]^{2\kappa_r\theta_r/\sigma_r^2}. \tag{2.31}$$

A taxa zero até τ é $y(\tau) = -\ln P(0,\tau)/\tau$. Se $r_0 < \theta_r$, a curva tende a ser ascendente; se $r_0 > \theta_r$, tende a ser invertida.

Laboratório 7.1 — Positividade, curva e reversão à média

Simulamos o CIR pela transição exata, que usa a distribuição qui-quadrado não central (módulo 12).

```
KR, TR, SR = 1.5, 0.015, 0.12
print("Condição de Feller 2κθ − σ²:", f3(2 * KR * TR - SR**2))


def simular_cir(r0, kappa, theta, sigma, passos, trajetorias, rng, dt=1 / DIAS):
    c = sigma**2 * (1 - np.exp(-kappa * dt)) / (4 * kappa)
    df = 4 * kappa * theta / sigma**2
    saida = np.zeros((passos + 1, trajetorias))
    saida[0] = r0
    for t in range(1, passos + 1):
        nc = saida[t - 1] * np.exp(-kappa * dt) / c
        saida[t] = c * rng.noncentral_chisquare(df, nc, trajetorias)
    return saida


def simular_vasicek(r0, kappa, theta, sigma_v, passos, trajetorias, rng, dt=1 / DIAS):
    a = np.exp(-kappa * dt)
    s = sigma_v * np.sqrt((1 - a**2) / (2 * kappa))
    saida = np.zeros((passos + 1, trajetorias))
    saida[0] = r0
    for t in range(1, passos + 1):
        saida[t] = theta + (saida[t - 1] - theta) * a + s * rng.standard_normal(trajetorias)
    return saida


def zero_cir(tau, r0, kappa, theta, sigma):
    g = np.sqrt(kappa**2 + 2 * sigma**2)
    den = (g + kappa) * (np.exp(g * tau) - 1) + 2 * g
    B = 2 * (np.exp(g * tau) - 1) / den
    A = (2 * g * np.exp((kappa + g) * tau / 2) / den) ** (2 * kappa * theta / sigma**2)
    return A * np.exp(-B * r0)


rng = np.random.default_rng(307)
sigma_v = SR * np.sqrt(TR)
cir = simular_cir(TR, KR, TR, SR, 10 * DIAS, 400, rng)
vas = simular_vasicek(TR, KR, TR, sigma_v, 10 * DIAS, 400, rng)

tabela(pd.DataFrame([
    {"Modelo": "CIR", "Média": float(cir[DIAS:].mean()), "Desvio-padrão":
↪  float(cir[DIAS:].std()), "Trajetórias que ficaram negativas (%)": float((cir.min(axis=0) <
↪  0).mean() * 100), "Tempo em taxa negativa (%)": float((cir < 0).mean() * 100)},
    {"Modelo": "Vasicek", "Média": float(vas[DIAS:].mean()), "Desvio-padrão":
↪  float(vas[DIAS:].std()), "Trajetórias que ficaram negativas (%)": float((vas.min(axis=0) <
↪  0).mean() * 100), "Tempo em taxa negativa (%)": float((vas < 0).mean() * 100)},
]))
ok((cir >= 0).all(), "CIR nunca fica negativo")
ok((vas < 0).mean() > 0.005, "Vasicek passa parte do tempo em taxa negativa")

# Conferência da fórmula da curva por Monte Carlo
r_mc = simular_cir(0.03, KR, TR, SR, 2 * DIAS, 100_000, rng)
integ = (r_mc[:-1] + r_mc[1:]).sum(axis=0) * 0.5 / DIAS
fd = np.exp(-integ)
mc_p, ep_p = fd.mean(), fd.std(ddof=1) / np.sqrt(fd.size)
ana = float(zero_cir(2.0, 0.03, KR, TR, SR))
print("Título zero de 2 anos: fórmula", f3(ana), "| Monte Carlo", f3(mc_p), "| erro-padrão",
↪  f3(ep_p))
ok(abs(ana - mc_p) < 4 * ep_p + 1e-4, "fórmula do título zero confere com Monte Carlo")

fig, (a1, a2) = plt.subplots(1, 2, figsize=(10.5, 3.9))
```

```
for j, estilo in zip(range(3), ["-", "--", "-."]):
    a1.plot(np.arange(cir.shape[0]) / DIAS, cir[:, j] * 100, lw=1.2, color=[AZUL, LARANJA,
↪  VERDE][j], ls=estilo, label=f"Trajetória {j + 1}")
a1.axhline(TR * 100, color=CINZA, lw=1, ls=":")
a1.set_title("Short rate no CIR: reversão à média")
a1.set_xlabel("Anos")
a1.set_ylabel("Taxa (% ao ano)")
a1.set_ylim(0, a1.get_ylim()[1] * 1.18)
a1.legend(loc="upper center", ncol=3, fontsize=8)

taus = np.linspace(0.02, 10, 200)
for r0, cor, estilo, rot in [(0.005, AZUL, "-", "Taxa inicial abaixo do longo prazo"), (0.04,
↪  LARANJA, "--", "Taxa inicial acima do longo prazo")]:
    a2.plot(taus, -np.log(zero_cir(taus, r0, KR, TR, SR)) / taus * 100, color=cor, ls=estilo,
↪  label=rot)
a2.set_title("Estrutura a termo no CIR")
a2.set_xlabel("Prazo (anos)")
a2.set_ylabel("Taxa zero (% ao ano)")
a2.legend()
plt.tight_layout()
plt.show()

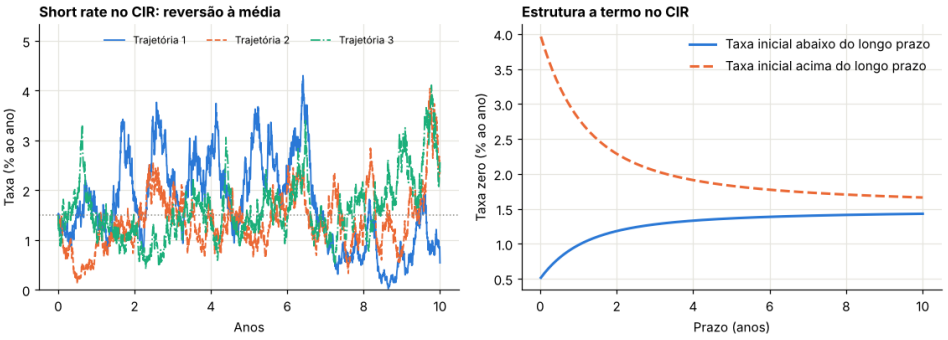
y_baixo = -np.log(zero_cir(taus, 0.005, KR, TR, SR)) / taus
y_alto = -np.log(zero_cir(taus, 0.04, KR, TR, SR)) / taus
ok(y_baixo[-1] > y_baixo[0] and y_alto[-1] < y_alto[0], "curva ascendente com taxa baixa e
↪  invertida com taxa alta")
```

```
Condição de Feller 2κθ − σ²: 0,031
```

Modelo	Média	Desvio-padrão	Trajetórias que ficaram negativas (%)	Tempo em taxa negativa (%)
CIR	0,015	0,008	0,000	0,000
Vasicek	0,015	0,009	87,000	3,631

```
OK — CIR nunca fica negativo
OK — Vasicek passa parte do tempo em taxa negativa
```

```
Título zero de 2 anos: fórmula 0,961 | Monte Carlo 0,961 | erro-padrão 0,000
OK — fórmula do título zero confere com Monte Carlo
```




```
OK — curva ascendente com taxa baixa e invertida com taxa alta
```

Laboratório 7.2 — Reversão à média é difícil de medir

Uma estimativa simples de κ_r regride a variação na taxa sobre o nível da taxa:

$$\Delta r_t = a + b\, r_{t-1} + \varepsilon_t, \qquad \hat{\kappa}_r = -\frac{\hat{b}}{\Delta t}. \tag{2.32}$$

Esse estimador é **viesado para cima** em amostras finitas: a média amostral absorve parte do movimento, e a série parece voltar mais rápido do que realmente volta. É um problema clássico em séries persistentes.

```
rng = np.random.default_rng(308)
amostras_cir = simular_cir(TR, KR, TR, SR, 10 * DIAS, 500, rng)
x = amostras_cir[:-1]
y = np.diff(amostras_cir, axis=0)
xd, yd = x - x.mean(axis=0), y - y.mean(axis=0)
b = (xd * yd).sum(axis=0) / (xd**2).sum(axis=0)
k_hat = -b * DIAS

tabela(pd.DataFrame([
    {"Grandeza": "κ verdadeiro", "Valor": KR},
    {"Grandeza": "Média das estimativas (10 anos)", "Valor": float(k_hat.mean())},
    {"Grandeza": "Desvio-padrão das estimativas", "Valor": float(k_hat.std(ddof=1))},
]))
ok(k_hat.mean() > KR * 1.1, "a estimativa de κ por regressão é viesada para cima")
```

Grandeza	Valor
κ verdadeiro	1,500
Média das estimativas (10 anos)	2,053
Desvio-padrão das estimativas	0,764

```
OK — a estimativa de κ por regressão é viesada para cima
```

---

8. Dos fatos ao modelo

Livro: §3.7

Fato observado	Exigência sobre o modelo	Componente	Módulos
Volatilidade varia, agrupa e reverte	variância estocástica com reversão à média	Heston (1993)	11-14
Efeito alavancagem	correlação negativa entre preço e variância	ρ do Heston	11-14
Smile (curvatura)	volatilidade da volatilidade	ν do Heston	11-14

Fato observado	Exigência sobre o modelo	Componente	Módulos
Skew acentuado nos prazos curtos, movimentos abruptos	saltos	Merton (1976)	10, 11, 14
Taxa aleatória, positiva e com curva	short rate com reversão à média	CIR (1985)	11, 13

A combinação é o modelo **BCC97**, de Bakshi, Cao e Chen (1997), que será o modelo geral do curso.

9. Síntese

- Log-retornos somam no tempo; a variância escala com o tempo se os retornos forem independentes.
- A média é quase impossível de estimar; a volatilidade, não.
- Curtose alta não identifica a causa: tanto saltos quanto volatilidade estocástica a produzem.
- Correlações móveis sobrepostas enganam. Medidas com observações não sobrepostas são mais confiáveis.
- A variação bipotência separa a variância contínua da variância de saltos.
- A volatilidade implícita é única, mas o método importa: o chute no ponto de inflexão resolve os casos difíceis.
- Volatilidade estocástica cria curvatura; correlação negativa e saltos negativos inclinam a superfície.
- A superfície está sujeita a restrições de não arbitragem, e sua curvatura em K é a densidade neutra ao risco.

10. Erratas do livro neste capítulo

Local	Problema
Comentário 4 após a saída da GBM	cita p-valor conjunto de 0,426, enquanto a saída impressa mostra 0,374
§3.4.2, contagem de saltos	escreve $P(r_n > -0{,}05)$; deveria ser $+0{,}05$
§3.4.2, contagem de saltos	as probabilidades informadas (2,911 e 3,402 por 10.000 dias) não fecham com $\hat{\mu}$ e $\hat{\sigma}$ impressos, que dão 1,267 e 1,542 por 10.000 dias
§3.5.2, primeira bolha	diz que calls OTM têm implícita mais alta; a Figura 3.11 mostra isso para calls ITM (puts OTM)

11. Exercícios

Exercício 1. O desvio diário impresso no livro para o DAX é 0,013761. Calcule a volatilidade anualizada com 252 dias.

Solução

$0{,}013761 \times \sqrt{252} = 0{,}218$, valor coerente com o que o livro imprime.

Exercício 2. Com dados semanais em vez de diários, ao longo do mesmo período, por que fator cresce o erro-padrão da volatilidade? E o da média?

Solução

Volatilidade: o erro é proporcional a $1/\sqrt{N}$, e N cai pelo fator $252/52$, de forma que o erro cresce por $\sqrt{252/52} = 2{,}201$. Média: não muda, pois depende apenas da extensão do período.

Exercício 3. Mostre, usando a paridade, que a derivada de uma put em relação à volatilidade é igual ao vega da call.

Solução

Derivando $P = C - S_0 + Ke^{-rT}$ em relação a σ, os dois últimos termos não dependem de σ. Logo, $\partial P/\partial\sigma = \partial C/\partial\sigma$.

Exercício 4. Em `P_M4`, altere `rho` para $-0{,}3$ e recalcule `skew_atm` em 17 e 365 dias. Onde o efeito é maior, em termos relativos? Por quê?

Solução

O skew passa de $-0{,}411$ para $-0{,}216$ em 17 dias (queda de 47,445%) e de $-0{,}146$ para $-0{,}067$ em 365 dias (queda de 54,110%). O efeito relativo é maior no prazo longo: em prazo curto, parte do skew vem dos saltos, que não dependem de ρ; no prazo longo, o peso dos saltos se reduz e a correlação domina.

Exercício 5. Refaça o Laboratório 7.2 com 30 anos de dados. O que acontece com o viés? E por que aumentar a frequência dos dados não adiantaria?

Solução

O viés diminui, aproximadamente na proporção inversa do período. Aumentar a frequência não adianta porque, como na média de um ativo, a informação sobre a reversão vem do número de ciclos observados, ou seja, da extensão do período.

```
print(f"Módulo 2 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 2 concluído: 47 verificações aprovadas.
```

Capítulo 3

A medida neutra ao risco em tempo discreto

Introdução

A ideia de que o preço de um derivativo é o custo de sua carteira replicante leva a uma consequência surpreendente: o preço pode ser escrito como uma média dos pagamentos futuros, descontada pela taxa livre de risco, desde que a média seja calculada com probabilidades **diferentes** das reais. Essas probabilidades formam a **medida neutra ao risco**.

O nome confunde muita gente. Não se supõe que os investidores sejam indiferentes ao risco; a medida neutra ao risco é um artifício contábil, um conjunto de pesos que resume os preços de mercado. Sob esses pesos, todo ativo rende em média a taxa livre de risco, e isso basta para precificar por média.

O capítulo constrói esse resultado no ambiente mais simples possível, com um número finito de estados e de datas, onde tudo pode ser provado com álgebra linear: o **primeiro teorema fundamental** (não há arbitragem se e somente se existe uma medida neutra ao risco) e o **segundo** (o mercado é completo se e somente se essa medida é única). O modelo binomial aparece como caso particular, e os preços de Arrow-Debreu, preços de receber 1 num único estado, mostram de onde vêm essas probabilidades. Em tempo contínuo, o Capítulo 4 repete a mesma lógica com outras ferramentas.

Fundamentos teóricos

Medidas martingais equivalentes

Mantemos o mercado de um período do Capítulo 1, com estados $\omega_1, \dots, \omega_K$ e conta remunerada de taxa r.

Definição 3.1 (Medida martingal equivalente). Uma medida $\mathbb{Q}$ em Ω é *equivalente* a $\mathbb{P}$ se $\mathbb{Q}(\omega_k) > 0$ para todo k. Ela é uma *medida martingal equivalente* (MME) se, para todo ativo i,

$$S_0^i = \frac{1}{1+r}\,\mathbb{E}^{\mathbb{Q}}[S_1^i]. \tag{3.1}$$

Teorema 3.2 (Primeiro teorema fundamental). *O mercado é livre de arbitragem se, e somente se, existe uma medida martingal equivalente.*

Demonstração. ($\Leftarrow$) Se $\mathbb{Q}$ é MME, então $V_0(\theta) = \mathbb{E}^{\mathbb{Q}}[V_1(\theta)]/(1+r)$ para toda carteira. Se $V_1(\theta) \geq 0$ com $V_1(\theta) > 0$ em algum estado, como $\mathbb{Q}$ dá peso positivo a todos os estados, $V_0(\theta) > 0$: não há arbitragem.

($\Rightarrow$) Considere o conjunto $\mathcal{G} = \{G(\theta) \in \mathbb{R}^K : \theta \in \mathbb{R}^N\}$ dos ganhos descontados $G_k(\theta) = \sum_{i\geq 1} \theta_i(S_1^i(\omega_k)/(1+r) - S_0^i)$, um subespaço vetorial. A ausência de arbitragem equivale a $\mathcal{G} \cap \mathbb{R}^K_+ = \{0\}$. O simplex $\Delta = \{x \in \mathbb{R}^K_+ : \sum x_k = 1\}$ é compacto, convexo e disjunto de $\mathcal{G}$; pelo teorema da separação de hiperplanos existe $q \in \mathbb{R}^K$ com $q \cdot g = 0$ para todo $g \in \mathcal{G}$ e $q \cdot x > 0$ para todo $x \in \Delta$. A segunda condição dá $q_k > 0$; normalizando, $\mathbb{Q}(\omega_k) = q_k / \sum_j q_j$ é uma medida equivalente, e a primeira condição é exatamente (3.1). ∎

O teorema vale em tempo discreto com qualquer espaço de estados (Dalang et al., 1990) e, com a noção adequada de arbitragem, em tempo contínuo (Harrison and Pliska, 1981; Delbaen and Schachermayer, 1994).

Teorema 3.3 (Segundo teorema fundamental). *Num mercado livre de arbitragem, o mercado é completo se, e somente se, a MME é única.*

Esboço. Completude significa que a matriz de payoffs $A_{ki} = S_1^i(\omega_k)$ tem posto K. As MME são as soluções positivas de um sistema linear de $N+1$ equações em K incógnitas; o sistema tem solução única se e somente se o posto é K. ∎

Corolário 3.4 (Fórmula de precificação neutra ao risco). *Se X é replicável, seu único preço livre de arbitragem é*

$$\Pi_0(X) = \frac{1}{1+r}\,\mathbb{E}^{\mathbb{Q}}[X], \tag{3.2}$$

para qualquer MME $\mathbb{Q}$. Se X não é replicável, os preços livres de arbitragem formam o intervalo $(\inf_{\mathbb{Q}} \mathbb{E}^{\mathbb{Q}}[X], \sup_{\mathbb{Q}} \mathbb{E}^{\mathbb{Q}}[X])/(1+r)$.

O modelo binomial

Proposição 3.5 (Binomial de um período). *Se $S_1 \in \{uS_0, dS_0\}$ com $0 < d < u$, o mercado com o ativo e a conta remunerada é livre de arbitragem se, e somente se, $d < 1 + r < u$. Nesse caso a MME é única,*

$$q = \frac{1+r-d}{u-d}, \tag{3.3}$$

e o mercado é completo: um payoff (X_u, X_d) é replicado por $\Delta = (X_u - X_d)/((u-d)S_0)$ unidades do ativo e $B = (uX_d - dX_u)/((u-d)(1+r))$ na conta.

Demonstração. A condição (3.1) para o ativo é $qu + (1-q)d = 1 + r$, cuja solução é a expressão de q; ela está em $(0,1)$ se e somente se $d < 1+r < u$, e o Teorema 3.2 conclui. A replicação resolve o sistema $\Delta u S_0 + B(1+r) = X_u$, $\Delta d S_0 + B(1+r) = X_d$. ∎

Definição 3.6 (Árvore binomial de n períodos). Em cada período, $S_{t+1} = uS_t$ com probabilidade q e $S_{t+1} = dS_t$ com probabilidade $1-q$, independentemente do passado.

Após n períodos, $S_n = S_0 u^j d^{n-j}$, com j binomial. Por indução para trás, o preço de um payoff europeu $f(S_n)$ é

$$\Pi_0 = \frac{1}{(1+r)^n} \sum_{j=0}^{n} \binom{n}{j} q^j (1-q)^{n-j} f(S_0 u^j d^{n-j}). \tag{3.4}$$

Observação 3.7 (Martingais e mudança de numerário). Em (3.1), o preço descontado $S_t/(1+r)^t$ é um $\mathbb{Q}$-martingal. A conta remunerada é o *numerário*. Qualquer ativo de preço positivo pode servir de numerário, com uma medida própria; essa ideia, retomada no Capítulo 17, é a base das medidas forward (Geman et al., 1995).

Observação 3.8 (Preços de Arrow-Debreu). O *título de Arrow-Debreu* do estado k paga 1 se ω_k ocorre e 0 caso contrário (Arrow, 1964). Seu preço é $\lambda_k = \mathbb{Q}(\omega_k)/(1+r)$, e qualquer payoff vale $\sum_k \lambda_k X(\omega_k)$. Os preços de Arrow-Debreu reaparecem nas árvores dos Capítulos 18 e 20, onde são calculados por indução para a frente.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** seções 4.1 a 4.4 e 4.7 do livro. A parte de tempo contínuo fica para o Módulo 4.

O que você vai conseguir fazer ao final

1. Modelar incerteza e informação com objetos concretos: estados, eventos, álgebras e filtrações.
2. Calcular esperanças condicionais e testar se um processo é martingale.
3. Mudar de medida de probabilidade com a derivada de Radon-Nikodym.
4. Escrever carteiras, estratégias autofinanciadas e processos de ganho.
5. Enunciar e **entender a demonstração** do Teorema Fundamental, na versão de um período.
6. Precificar qualquer derivativo atingível e obter sua carteira replicante, inclusive com juros aleatórios.
7. Reconhecer um mercado incompleto e calcular o intervalo de preços sem arbitragem.

Por que um módulo inteiro em mundo finito

Com um número finito de cenários, toda a teoria vira **soma e álgebra linear**. Cada definição pode ser verificada no computador, célula a célula. A intuição obtida aqui é a que levamos para o tempo contínuo, no próximo módulo, em que a matemática é bem mais pesada.

No Módulo 1 vimos a ideia em dois cenários: o preço vem do custo de replicação e existe uma probabilidade q que o resume. Agora transformamos essa observação em teoria.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
from itertools import combinations, product

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import linprog
from IPython.display import display

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. Incerteza: estados, eventos e probabilidade

Livro: §4.2, definições 1 a 4

1.1 O ponto de partida: os estados

Fixamos um horizonte com datas $t = 0, 1, \dots, T$. Hoje não sabemos como o mundo estará em T, mas conhecemos a lista de possibilidades. Essa lista é o **espaço de estados** Ω, e cada elemento ω é uma história completa.

Nossa economia-exemplo tem dois períodos, em cada um dos quais o índice sobe (`u`) ou desce (`d`):

$$\Omega = \{uu,\ ud,\ du,\ dd\}. \tag{3.5}$$

1.2 Eventos e álgebras

Um **evento** é um subconjunto de Ω: por exemplo, "subiu no primeiro período" é $\{uu, ud\}$.

Nem todo evento precisa ser observável. Uma **álgebra** $\mathcal{F}$ é uma família de eventos que representa o que é observável. Ela precisa satisfazer três condições de bom senso:

1. $\Omega \in \mathcal{F}$: sempre sei que algo aconteceu.
2. Se $E \in \mathcal{F}$, então $E^c \in \mathcal{F}$: se consigo verificar que E ocorreu, sei também quando não ocorreu.
3. Uniões finitas de eventos de $\mathcal{F}$ estão em $\mathcal{F}$.

Toda álgebra num Ω finito vem de uma partição. Basta pegar os menores eventos não vazios da álgebra, chamados **átomos**: eles dividem Ω sem sobreposição, e a álgebra é formada por todas as uniões possíveis deles. Por isso, no computador, representamos a informação pela partição.

Com m átomos, cada um pode entrar ou não na união: são 2^m eventos.

1.3 Probabilidade e esperança

Uma **medida de probabilidade** atribui um número em $[0,1]$ a cada evento, de forma aditiva para eventos disjuntos e com $P(\Omega)=1$. Num mundo finito, basta dar $P(\omega)$ em cada estado.

Duas medidas são **equivalentes** se concordam sobre o que é impossível: $P(E)=0 \Leftrightarrow Q(E)=0$. Elas podem discordar completamente das chances, mas não sobre o que pode ou não acontecer.

Uma **variável aleatória** é uma função de Ω nos números reais, e sua esperança é

$$\mathbb{E}^{P}[X] = \sum_{\omega \in \Omega} P(\omega)\, X(\omega). \tag{3.6}$$

Laboratório 1.1 — Álgebras no computador

```
OMEGA = ["uu", "ud", "du", "dd"]


def algebra(particao):
    """Álgebra gerada por uma partição: todas as uniões possíveis de átomos."""
    atomos = [frozenset(a) for a in particao]
    saida = set()
    for k in range(len(atomos) + 1):
        for comb in combinations(atomos, k):
            saida.add(frozenset().union(*comb))
    return saida


def eh_algebra(F, omega):
    """Confere as três condições da definição de álgebra."""
    tudo = frozenset(omega)
    if tudo not in F:
        return False
    if any(tudo - E not in F for E in F):
        return False
    return all(A | B in F for A in F for B in F)


F0 = [["uu", "ud", "du", "dd"]]
F1 = [["uu", "ud"], ["du", "dd"]]
F2 = [["uu"], ["ud"], ["du"], ["dd"]]

linhas = []
for nome, p in [("F0: nada se sabe", F0), ("F1: conhece o 1º movimento", F1), ("F2: conhece tudo",
↪  F2)]:
    A = algebra(p)
    linhas.append({"Informação": nome, "Átomos": len(p), "Eventos na álgebra": len(A), "É
↪  álgebra?": eh_algebra(A, OMEGA)})
tabela(pd.DataFrame(linhas))
ok(all(l["É álgebra?"] and l["Eventos na álgebra"] == 2 ** l["Átomos"] for l in linhas), "cada
↪  partição gera uma álgebra com 2^m eventos")

falsa = {frozenset(), frozenset(OMEGA), frozenset({"uu"})}
ok(not eh_algebra(falsa, OMEGA), "uma família sem o complemento de {uu} não é álgebra")
```

Informação	Átomos	Eventos na álgebra	É álgebra?
F0: nada se sabe	1	2	True
F1: conhece o 1º movimento	2	4	True
F2: conhece tudo	4	16	True

```
OK — cada partição gera uma álgebra com 2^m eventos
OK — uma família sem o complemento de {uu} não é álgebra
```

2. Informação ao longo do tempo

Livro: §4.2, definições 5 a 9

2.1 Filtração

A incerteza não se resolve de uma só vez. Uma **filtração** é uma sequência crescente de álgebras,

$$\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \cdots \subseteq \mathcal{F}_T, \tag{3.7}$$

em que $\mathcal{F}_t$ representa o que se sabe na data t. Crescente significa que **ninguém esquece**. Em geral, $\mathcal{F}_0 = \{\emptyset, \Omega\}$ (nada se sabe) e $\mathcal{F}_T$ contém todos os eventos (tudo se sabe).

Em linguagem de partições: de uma data para a outra, os átomos **se dividem**.

2.2 Mensurabilidade e adaptação

Dizer que X é $\mathcal{F}_t$**-mensurável** significa que X é conhecida na data t. Na partição, a condição é concreta: X **é constante em cada átomo de** $\mathcal{F}_t$.

Um processo (S_t) é **adaptado** se S_t é $\mathcal{F}_t$-mensurável para todo t: o preço de hoje é conhecido hoje.

2.3 Tempo de parada

Um **tempo de parada** é uma data aleatória τ em que a decisão de parar usa **apenas informação disponível até aquele momento**:

$$\{\tau \le t\} \in \mathcal{F}_t \quad \text{para todo } t. \tag{3.8}$$

"Vender na primeira vez que o índice cair" é um tempo de parada. "Vender no dia do máximo" não é, porque só se sabe qual foi o máximo depois. Esse conceito será central para as opções americanas no Módulo 9.

Laboratório 2.1 — A economia-exemplo

Tomamos $S_0 = 100$, com alta de 20% ou queda de 20% por período, e juros de 5% por período. As probabilidades reais são $p = 70\%$ de alta, independentes.

```python
def particoes(T):
    """Partições da filtração natural de uma árvore binária de T períodos."""
    omega = ["".join(c) for c in product("ud", repeat=T)]
```

```
    saida = []
    for t in range(T + 1):
        grupos = {}
        for w in omega:
            grupos.setdefault(w[:t], []).append(w)
        saida.append(list(grupos.values()))
    return omega, saida


def arvore(T, S0, u, d):
    """Preços por trajetória: linhas são estados e colunas são datas."""
    omega, _ = particoes(T)
    S = pd.DataFrame(index=omega, columns=range(T + 1), dtype=float)
    for w in omega:
        v = S0
        S.loc[w, 0] = v
        for t, c in enumerate(w, start=1):
            v *= u if c == "u" else d
            S.loc[w, t] = v
    return S


def medida(omega, p):
    return pd.Series({w: np.prod([p if c == "u" else 1 - p for c in w]) for w in omega})


def mensuravel(X, particao):
    return all(np.ptp(X.loc[a].values) < 1e-12 for a in particao)

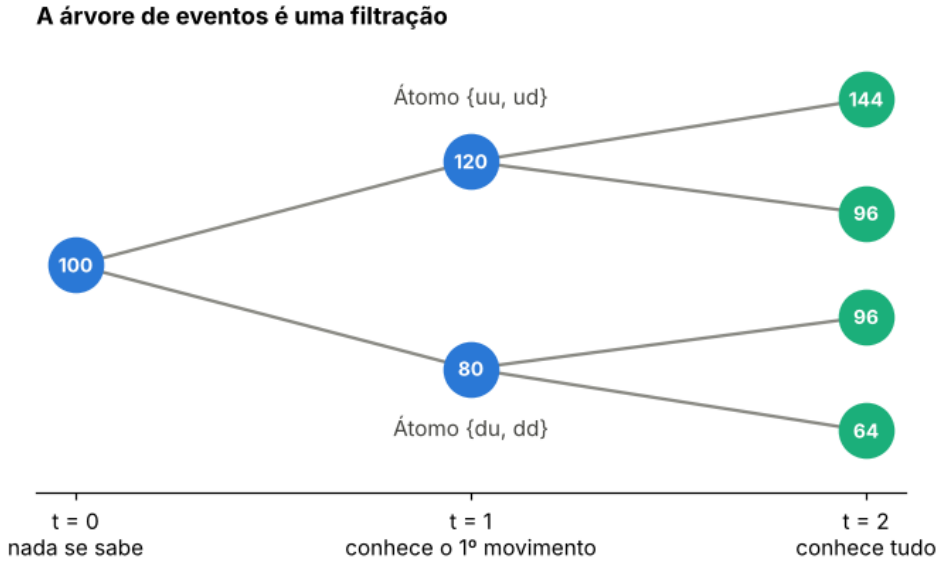

T, S0, u, d, r, p = 2, 100.0, 1.2, 0.8, 0.05, 0.7
omega, filtracao = particoes(T)
S = arvore(T, S0, u, d)
P = medida(omega, p)
tabela(S.reset_index(names="Estado"))

fig, ax = plt.subplots(figsize=(7.2, 3.8))
nos = {"": (0, 0), "u": (1, 1), "d": (1, -1), "uu": (2, 1.6), "ud": (2, 0.5), "du": (2, -0.5),
↪  "dd": (2, -1.6)}
for pai, (x0, y0) in nos.items():
    for m in "ud":
        filho = pai + m
        if filho in nos:
            ax.plot([x0, nos[filho][0]], [y0, nos[filho][1]], color=CINZA, lw=1.5, zorder=1)
for no, (x, y) in nos.items():
    t = len(no)
    w = (no + "uu")[:T]
    ax.scatter([x], [y], s=620, color=AZUL if t < 2 else VERDE, zorder=2)
    ax.text(x, y, f"{S.loc[w, t]:.0f}", ha="center", va="center", color="white",
↪  fontweight="bold", fontsize=9, zorder=3)
ax.text(1, 1.55, "Átomo {uu, ud}", ha="center", color="#444440")
ax.text(1, -1.65, "Átomo {du, dd}", ha="center", color="#444440")
ax.set_xticks([0, 1, 2])
ax.set_xticklabels(["t = 0\nnada se sabe", "t = 1\nconhece o 1º movimento", "t = 2\nconhece
↪  tudo"])
ax.set_yticks([])
ax.grid(False)
ax.spines["left"].set_visible(False)
ax.set_ylim(-2.2, 2.2)
ax.set_title("A árvore de eventos é uma filtração")
```

```
plt.show()

ok(mensuravel(S[1], filtracao[1]) and mensuravel(S[2], filtracao[2]), "o preço é adaptado à
↪  filtração")
ok(not mensuravel(S[2], filtracao[1]), "o preço de t = 2 não é conhecido em t = 1")
```

Estado	0	1	2
uu	100,000	120,000	144,000
ud	100,000	120,000	96,000
du	100,000	80,000	96,000
dd	100,000	80,000	64,000




```
OK — o preço é adaptado à filtração
OK — o preço de t = 2 não é conhecido em t = 1
```

Laboratório 2.2 — O preço de hoje não basta como informação

Em nossa árvore, $S_2(ud) = S_2(du) = 96$. Se alguém só observar S_2, não saberá qual caminho ocorreu: a álgebra gerada por S_2 tem 3 átomos, e não 4. Já observando todo o histórico (S_1, S_2), todos os estados se separam.

É por isso que produtos dependentes da trajetória exigem controlar o caminho, e não só o preço final.

```
def particao_gerada(serie):
    """Partição da álgebra gerada por uma variável (ou tupla de variáveis)."""
    grupos = {}
    for w, v in serie.items():
        grupos.setdefault(v if isinstance(v, tuple) else round(float(v), 9), []).append(w)
    return list(grupos.values())


so_final = particao_gerada(S[2])
```

```
historico = pd.Series({w: (round(S.loc[w, 1], 9), round(S.loc[w, 2], 9)) for w in omega})
com_historico = particao_gerada(historico)

print("Só o preço final:", so_final)
print("Todo o histórico:", com_historico)
ok(len(so_final) == 3 and len(com_historico) == 4, "o histórico contém mais informação do que o
↪  preço final")


def eh_tempo_de_parada(tau, filt):
    """Testa se {tau <= t} é união de átomos de F_t para todo t."""
    for t, part in enumerate(filt):
        evento = set(tau.index[tau <= t])
        for a in part:
            if not (set(a) <= evento or not (set(a) & evento)):
                return False
    return True


primeira_queda = pd.Series({w: (w.index("d") + 1 if "d" in w else T) for w in omega})
dia_do_maximo = pd.Series({w: int(np.argmax(S.loc[w].values)) for w in omega})
tabela(pd.DataFrame({"Primeira queda": primeira_queda, "Dia do máximo":
↪  dia_do_maximo}).reset_index(names="Estado"))
ok(eh_tempo_de_parada(primeira_queda, filtracao), "'primeira queda' é um tempo de parada")
ok(not eh_tempo_de_parada(dia_do_maximo, filtracao), "'dia do máximo' exige olhar o futuro: não é
↪  tempo de parada")
```

```
Só o preço final: [['uu'], ['ud', 'du'], ['dd']]
Todo o histórico: [['uu'], ['ud'], ['du'], ['dd']]
OK — o histórico contém mais informação do que o preço final
```

Estado	Primeira queda	Dia do máximo
uu	2	2
ud	2	1
du	1	0
dd	1	0

```
OK — 'primeira queda' é um tempo de parada
OK — 'dia do máximo' exige olhar o futuro: não é tempo de parada
```

---

3. Esperança condicional e martingales

Livro: §4.2, definições 10 e 11

3.1 Esperança condicional, passo a passo

Na data t já sabemos em que átomo estamos, e a melhor previsão usa apenas os estados daquele átomo, com as probabilidades reponderadas. Para cada átomo A de $\mathcal{F}_t$:

$$\mathbb{E}_t^P[X](\omega) = \frac{\sum_{\eta \in A} P(\eta)\, X(\eta)}{P(A)} \qquad \text{para } \omega \in A. \tag{3.9}$$

Note que **a esperança condicional é, ela própria, uma variável aleatória**: uma média diferente para cada átomo.

A definição formal do livro exige duas propriedades:

1. $\mathbb{E}_t^P[X]$ é $\mathcal{F}_t$-mensurável (só depende do que se sabe em t);
2. para todo evento $E \in \mathcal{F}_t$: $\mathbb{E}^P[\mathbb{E}_t^P[X]\,\mathbf{1}_E] = \mathbb{E}^P[X\,\mathbf{1}_E]$ (o valor médio em cada evento conhecido é preservado).

Dessas decorre a **lei das esperanças iteradas**: para $s \le t$, $\mathbb{E}_s\big[\mathbb{E}_t[X]\big] = \mathbb{E}_s[X]$. Em palavras: minha previsão de hoje sobre minha previsão de amanhã é a minha previsão de hoje.

3.2 Martingale

Um processo adaptado é um **martingale** sob Q se

$$\mathbb{E}_t^Q[S_{t+s}] = S_t. \tag{3.10}$$

É o modelo matemático do jogo justo: a melhor previsão para qualquer data futura é o valor atual.

Basta verificar um passo. Se $\mathbb{E}_t[S_{t+1}] = S_t$ para todo t, a lei das esperanças iteradas dá $\mathbb{E}_t[S_{t+2}] = \mathbb{E}_t\big[\mathbb{E}_{t+1}[S_{t+2}]\big] = \mathbb{E}_t[S_{t+1}] = S_t$, e assim por diante.

Laboratório 3.1 — Conferindo definições

Testamos o preço **descontado** $\tilde{S}_t = S_t/(1+r)^t$ sob duas medidas: a real ($p = 70\%$) e aquela em que q é escolhido para zerar o ganho esperado do preço descontado:

$$q = \frac{(1+r) - d}{u - d}. \tag{3.11}$$

```
def E_cond(X, particao, prob):
    """Esperança condicional de X dada a informação da partição, sob a medida prob."""
    Y = pd.Series(index=X.index, dtype=float)
    for a in particao:
        Y.loc[a] = float((X.loc[a] * prob.loc[a]).sum() / prob.loc[a].sum())
    return Y


def eh_martingale(X, filt, prob):
    """X é DataFrame com colunas t. Testa o passo de um período em todas as datas."""
    return all(np.allclose(E_cond(X[t + 1], filt[t], prob), X[t]) for t in range(len(filt) - 1))


B = pd.DataFrame({t: (1 + r) ** t for t in range(T + 1)}, index=omega)
S_desc = S / B
q = (1 + r - d) / (u - d)
Q = medida(omega, q)

print("q =", f3(q), "| p =", f3(p))
X = S[2]
EX = E_cond(X, filtracao[1], P)
ok(mensuravel(EX, filtracao[1]), "propriedade 1: a esperança condicional é mensurável em F_1")
ok(all(abs(float((EX[list(E)] * P[list(E)]).sum()) - float((X[list(E)] * P[list(E)]).sum())) <
 ↪  1e-10 for E in algebra(filtracao[1]) if E), "propriedade 2: as médias são preservadas em todo
 ↪  evento de F_1")
```

```
ok(abs(float((EX * P).sum()) - float((X * P).sum())) < 1e-10, "lei das esperanças iteradas")
ok(eh_martingale(S_desc, filtracao, Q), "o preço descontado é martingale sob Q")
ok(not eh_martingale(S_desc, filtracao, P), "o preço descontado não é martingale sob P")
```

```
q = 0,625 | p = 0,700
OK — propriedade 1: a esperança condicional é mensurável em F_1
OK — propriedade 2: as médias são preservadas em todo evento de F_1
OK — lei das esperanças iteradas
OK — o preço descontado é martingale sob Q
OK — o preço descontado não é martingale sob P
```

3.3 Trocando de medida: a derivada de Radon-Nikodym

Se Q é equivalente a P, definimos, estado a estado,

$$L(\omega) = \frac{Q(\omega)}{P(\omega)}. \tag{3.12}$$

Uma esperança sob Q pode, então, ser calculada sob P. Basta multiplicar e dividir por $P(\omega)$:

$$\mathbb{E}^Q[X] = \sum_\omega Q(\omega)X(\omega) = \sum_\omega P(\omega)\frac{Q(\omega)}{P(\omega)}X(\omega) = \mathbb{E}^P[L\,X]. \tag{3.13}$$

Duas consequências que o livro não desenvolve, mas que usaremos no próximo módulo para o teorema de Girsanov:

Processo de densidade. $L_t = \mathbb{E}_t^P[L]$ é um martingale sob P, com $L_0 = 1$.

Fórmula de Bayes. Para esperanças condicionais, não basta multiplicar por L; é preciso normalizar:

$$\mathbb{E}_t^Q[X] = \frac{\mathbb{E}_t^P[L\,X]}{L_t}. \tag{3.14}$$

Para ver o porquê, fixe um átomo A de $\mathcal{F}_t$. Do lado esquerdo, a definição dá $\sum_A QX/Q(A)$. Do lado direito, o numerador é $\sum_A PLX/P(A) = \sum_A QX/P(A)$, e o denominador é $L_t = Q(A)/P(A)$. Dividindo, $P(A)$ se cancela e os dois lados coincidem.

```
L = Q / P
Lt = pd.DataFrame({t: E_cond(L, filtracao[t], P) for t in range(T + 1)})
tabela(pd.DataFrame({"P": P, "Q": Q, "L = Q/P": L}).reset_index(names="Estado"))

ok(abs(float((Q * S[2]).sum()) - float((P * L * S[2]).sum())) < 1e-10, "E^Q[X] = E^P[L X]")
ok(eh_martingale(Lt, filtracao, P) and abs(float(Lt[0].iloc[0]) - 1) < 1e-12, "L_t é martingale
↪  sob P e começa em 1")
bayes = E_cond(L * S[2], filtracao[1], P) / Lt[1]
ok(np.allclose(bayes, E_cond(S[2], filtracao[1], Q)), "fórmula de Bayes para esperança
↪  condicional")
```

Estado	P	Q	L = Q/P
uu	0,490	0,391	0,797
ud	0,210	0,234	1,116
du	0,210	0,234	1,116
dd	0,090	0,141	1,562

```
OK — E^Q[X] = E^P[L X]
OK — L_t é martingale sob P e começa em 1
OK — fórmula de Bayes para esperança condicional
```

---

4. O mercado: carteiras e estratégias

Livro: §4.3

4.1 Ativos

O mercado tem $K+1$ ativos. O ativo 0 é o **título** (*bond*), com $S_0^0 = 1$, que serve de **numerário**. Ele é sem risco no sentido de que S_t^0 é conhecido já em $t-1$. O **fator de desconto** é $\beta_t = 1/S_t^0$.

Isso inclui juros aleatórios: o juro do próximo período é desconhecido hoje, mas é conhecido no início do período. Veremos um exemplo na Seção 8.

4.2 Estratégia de negociação

Uma **estratégia** é um processo $\phi_t = (\phi_t^0, \dots, \phi_t^K)$ com a quantidade de cada ativo no período $(t-1, t]$. Ela precisa ser **previsível**: ϕ_t é $\mathcal{F}_{t-1}$-mensurável, ou seja, a carteira do período é escolhida **antes** de se conhecer o movimento do período.

O valor da estratégia é $V_t = \phi_t \cdot S_t$ e o ganho acumulado é

$$G_t = \sum_{i=1}^{t} \phi_i \cdot (S_i - S_{i-1}). \tag{3.15}$$

4.3 Autofinanciamento

Na data t, a carteira velha vale $\phi_t \cdot S_t$. Ao montar a nova, sem entrada nem saída de recursos, a condição é

$$\phi_t \cdot S_t = \phi_{t+1} \cdot S_t. \tag{3.16}$$

Isso equivale a $V_t = V_0 + G_t$: **a riqueza só muda por ganhos de mercado**.

4.4 O lema que sustenta toda a teoria

Vamos descontar, com $\tilde{S}_t = \beta_t S_t$ e $\tilde{V}_t = \beta_t V_t$. Multiplicando a condição de autofinanciamento por β_t, ela continua válida em termos descontados: $\phi_t \cdot \tilde{S}_t = \phi_{t+1} \cdot \tilde{S}_t$. Usando isso na variação de um período:

$$\tilde{V}_{t+1} - \tilde{V}_t = \phi_{t+1} \cdot \tilde{S}_{t+1} - \phi_t \cdot \tilde{S}_t = \phi_{t+1} \cdot \tilde{S}_{t+1} - \phi_{t+1} \cdot \tilde{S}_t = \phi_{t+1} \cdot (\tilde{S}_{t+1} - \tilde{S}_t). \tag{3.17}$$

Somando no tempo:

$$\tilde{V}_T = \tilde{V}_0 + \sum_{t=1}^{T} \phi_t \cdot (\tilde{S}_t - \tilde{S}_{t-1}). \tag{3.18}$$

Agora suponha que $\tilde{S}$ é martingale sob Q. Como ϕ_t é previsível, sai da esperança condicional:

$$\mathbb{E}^Q_{t-1}[\phi_t \cdot (\tilde{S}_t - \tilde{S}_{t-1})] = \phi_t \cdot \mathbb{E}^Q_{t-1}[\tilde{S}_t - \tilde{S}_{t-1}] = 0. \tag{3.19}$$

Conclusão: **o valor descontado de qualquer estratégia autofinanciada previsível também é martingale sob** Q. Uma aposta justa, com quantidade escolhida de antemão, continua justa, seja qual for a regra.

> **Errata do livro:** na prova da Proposição 1 (§4.7.2), a soma aparece com o termo $\phi_t \cdot (\beta_t S_t - \beta_{t+1} S_{t+1})$, para $t = 2, \ldots, T$. O índice correto é o da expressão acima: $\phi_t \cdot (\beta_t S_t - \beta_{t-1} S_{t-1})$. Da forma impressa, a soma usaria S_{T+1}, que não existe.

Laboratório 4.1 — Não dá para vencer um jogo justo, a menos que se olhe o futuro

Simulamos um martingale simples, com passos de ± 1 e chances iguais, em 20 períodos. Comparamos estratégias previsíveis com uma que "espia" o movimento do período.

```
rng = np.random.default_rng(403)
periodos, cenarios = 20, 400_000
dX = rng.choice([-1.0, 1.0], size=(cenarios, periodos))


def ganhos(fi):
    return np.cumsum(fi * dX, axis=1)


constante = np.ones_like(dX)
momento = np.hstack([np.ones((cenarios, 1)), np.sign(dX[:, :-1])])
reversao = -momento

dobrar = np.ones_like(dX)
for t in range(1, periodos):
    perdeu = dX[:, t - 1] < 0
    dobrar[:, t] = np.where(perdeu, np.minimum(dobrar[:, t - 1] * 2, 64.0), 1.0)

espiando = np.sign(dX)

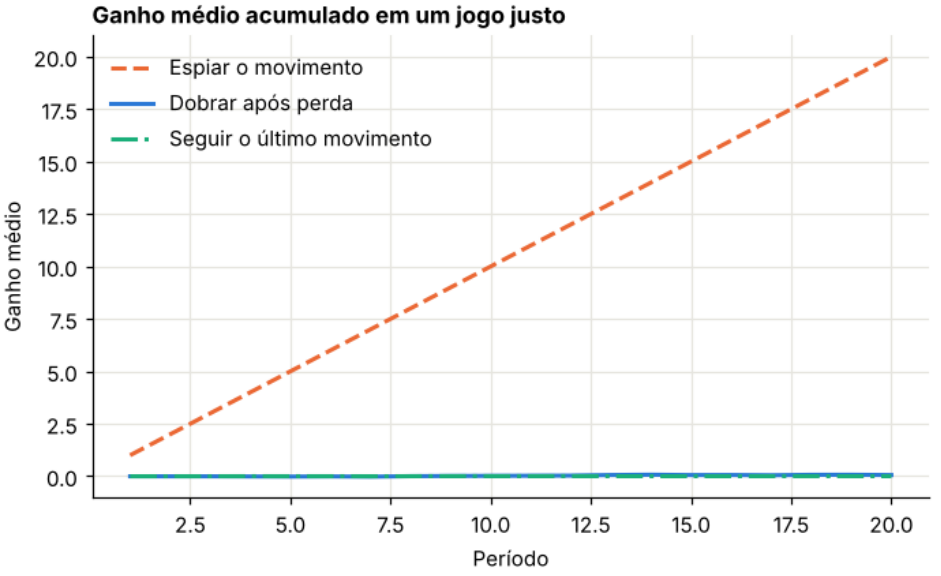
estrategias = {
    "Comprar e manter": constante,
    "Seguir o último movimento": momento,
    "Apostar contra o último movimento": reversao,
    "Dobrar após perda (limite 64)": dobrar,
    "Espiar o movimento do período": espiando,
}

linhas, curvas = [], {}
for nome, fi in estrategias.items():
    g = ganhos(fi)
    curvas[nome] = g.mean(axis=0)
    gf = g[:, -1]
    linhas.append({"Estratégia": nome, "Previsível?": nome != "Espiar o movimento do período",
↪   "Ganho médio final": gf.mean(), "Erro-padrão": gf.std(ddof=1) / np.sqrt(cenarios)})
res = pd.DataFrame(linhas)
tabela(res)
```

```
fig, ax = plt.subplots()
ax.plot(range(1, periodos + 1), curvas["Espiar o movimento do período"], color=LARANJA, ls="--",
↪  label="Espiar o movimento")
ax.plot(range(1, periodos + 1), curvas["Dobrar após perda (limite 64)"], color=AZUL,
↪  label="Dobrar após perda")
ax.plot(range(1, periodos + 1), curvas["Seguir o último movimento"], color=VERDE, ls="-.",
↪  label="Seguir o último movimento")
ax.set_title("Ganho médio acumulado em um jogo justo")
ax.set_xlabel("Período")
ax.set_ylabel("Ganho médio")
ax.legend()
plt.show()

prev = res[res["Previsível?"]]
ok((prev["Ganho médio final"].abs() < 4 * prev["Erro-padrão"]).all(), "nenhuma regra previsível
↪  altera o ganho médio de um jogo justo")
ok(res.loc[~res["Previsível?"], "Ganho médio final"].iloc[0] > 19.9, "sem previsibilidade, o jogo
↪  deixa de ser justo")
```

Estratégia	Previsível?	Ganho médio final	Erro-padrão
Comprar e manter	True	-0,007	0,007
Seguir o último movimento	True	0,001	0,007
Apostar contra o último movimento	True	-0,001	0,007
Dobrar após perda (limite 64)	True	0,055	0,059
Espiar o movimento do período	False	20,000	0,000




```
OK — nenhuma regra previsível altera o ganho médio de um jogo justo
OK — sem previsibilidade, o jogo deixa de ser justo
```

Observação. A estratégia de dobrar a aposta aparenta ganhar quase sempre, mas com limite de exposição o ganho médio é zero: os muitos ganhos pequenos são exatamente compensados por raras perdas grandes. Sem limite, surgem problemas em tempo contínuo, e a noção de estratégia admissível no Módulo 4 serve para excluí-los.

5. Arbitragem

Livro: §4.4, definições 21 a 26

5.1 Direitos contingentes

Um **direito contingente** é um pagamento $A_T(\omega) \geq 0$ em T, que depende do estado. Opções são exemplos. O direito é **atingível** se existe uma estratégia autofinanciada com $V_T = A_T$: esse direito é **redundante** e o preço dele deve ser V_0.

5.2 O que é arbitragem

Arbitragem é um almoço grátis: uma estratégia autofinanciada com

$$V_0 = 0, \qquad V_T \geq 0 \text{ em todos os estados}, \qquad \mathbb{E}^P[V_T] > 0. \tag{3.20}$$

Não custa nada, nunca dá prejuízo e tem chance de lucro. Note que as probabilidades reais só importam para dizer **o que é possível**: basta que $P(\omega) > 0$.

> **Errata do livro, que afeta o entendimento.** O livro define estratégia admissível (Def. 19) como aquela com valor limitado inferiormente, $V_t \geq -\alpha$, e arbitragem (Def. 25) como estratégia admissível com $V_0 = 0$ e $\mathbb{E}[V_T] > 0$. Num mundo finito, **toda** estratégia é limitada inferiormente. Pela letra do livro, comprar o índice com dinheiro emprestado seria arbitragem sempre que o retorno esperado superasse a taxa de juros, o que contradiz o próprio Teorema 1. No artigo original de Harrison e Pliska, admissível significa $V_t \geq 0$, e é isso que faz a definição funcionar. A prova do Lema 1, no livro, já parte de $V_t \geq 0$.

Laboratório 5.1 — A definição do livro, ao pé da letra

Em nossa economia, sabemos que existe medida martingale. Portanto, não pode haver arbitragem. Mas vejamos a estratégia "comprar o índice com dinheiro emprestado".

```
fi_S, fi_B = 1.0, -100.0
V0 = fi_S * S0 + fi_B * 1.0
VT = fi_S * S[2] + fi_B * B[2]

tabela(pd.DataFrame({"Valor final": VT, "Prob. real": P}).reset_index(names="Estado"))
print("Custo inicial:", f3(V0), "| Valor esperado final:", f3(float((VT * P).sum())), "| Pior
↪  caso:", f3(float(VT.min())))

ok(V0 == 0 and (VT * P).sum() > 0 and VT.min() > -np.inf, "satisfaz a Definição 25 como impressa")
ok(VT.min() < 0, "mas pode dar prejuízo: não é arbitragem")
ok(abs(float((VT / B[2] * Q).sum())) < 1e-10, "e sob Q seu valor esperado descontado é zero")
```

Estado	Valor final	Prob. real
uu	33,750	0,490
ud	-14,250	0,210
du	-14,250	0,210
dd	-46,250	0,090

```
Custo inicial: 0,000 | Valor esperado final: 6,390 | Pior caso: -46,250
OK — satisfaz a Definição 25 como impressa
OK — mas pode dar prejuízo: não é arbitragem
OK — e sob Q seu valor esperado descontado é zero
```

5.3 Arbitragem fraca: o Lema 1 com números

Pode haver uma estratégia com $V_0 = 0$ e $V_T \geq 0$ que, no meio do caminho, passe por valor **negativo**. A definição de Harrison e Pliska exige $V_t \geq 0$ em todas as datas, e o livro chama essa versão mais frouxa de **arbitragem fraca**. O Lema 1 diz que não há diferença essencial: de uma arbitragem fraca sempre se extrai uma arbitragem no sentido forte.

A ideia da prova. Olhe para o último instante em que o valor fica negativo, e o evento em que isso ocorre. Não faça nada até lá. Nesse instante e nesse evento, comece a estratégia, pois o déficit já não é mais carregado: o valor inicial passa a ser zero e o resultado final só melhora.

Exemplo com $r = 0$. O índice começa em 100 e vai a 110 ou 90. Em d, porém, os dois sucessores são 95 e 92, ambos acima de 90: há uma oportunidade naquele nó.

> **Errata menor:** na prova do Lema 1, em "$\psi_u(\omega) \equiv 0$ if $u < t$ as well as $\omega \notin E$", deveria ser $u > t$.

```python
est = ["uu", "ud", "du", "dd"]
S_lema = pd.DataFrame({0: 100.0, 1: [110.0, 110.0, 90.0, 90.0], 2: [120.0, 100.0, 95.0, 92.0]},
 ↪  index=est)

# Estratégia fraca: comprar 1 unidade a crédito; em d, comprar mais 4; em u, zerar.
fi_S1 = pd.Series(1.0, index=est)
fi_B1 = pd.Series(-100.0, index=est)
fi_S2 = pd.Series({"uu": 0.0, "ud": 0.0, "du": 5.0, "dd": 5.0})
fi_B2 = fi_B1 + (fi_S1 - fi_S2) * S_lema[1]

V1 = fi_S1 * S_lema[1] + fi_B1
V2 = fi_S2 * S_lema[2] + fi_B2

# Estratégia corrigida: nada até t=1; em d, entra com a mesma posição, mais o déficit em caixa.
a = V1["dd"]
psi_S2 = pd.Series({"uu": 0.0, "ud": 0.0, "du": 5.0, "dd": 5.0})
psi_B2 = pd.Series({"uu": 0.0, "ud": 0.0, "du": fi_B2["du"] - a, "dd": fi_B2["dd"] - a})
W1 = pd.Series({"uu": 0.0, "ud": 0.0, "du": float(psi_S2["du"] * 90.0 + psi_B2["du"]), "dd":
 ↪  float(psi_S2["dd"] * 90.0 + psi_B2["dd"])})
W2 = psi_S2 * S_lema[2] + psi_B2

tabela(pd.DataFrame({"V1 fraca": V1, "V2 fraca": V2, "V1 corrigida": W1, "V2 corrigida":
 ↪  W2}).reset_index(names="Estado"))
ok(V1.min() < 0 and V2.min() >= 0 and V2.max() > 0, "a estratégia fraca passa por valor negativo,
 ↪  mas termina não negativa")
ok(W1.abs().max() < 1e-9 and W2.min() >= 0 and W2.max() > 0, "a estratégia corrigida é arbitragem
 ↪  em sentido forte")
```

Estado	V1 fraca	V2 fraca	V1 corrigida	V2 corrigida
uu	10,000	10,000	0,000	0,000
ud	10,000	10,000	0,000	0,000
du	-10,000	15,000	0,000	25,000
dd	-10,000	0,000	0,000	10,000

```
OK — a estratégia fraca passa por valor negativo, mas termina não negativa
OK — a estratégia corrigida é arbitragem em sentido forte
```

---

6. O Teorema Fundamental

Livro: §4.4, Teorema 1 e §4.7.3

Enunciado

No modelo de mercado discreto, são equivalentes:

1. Não há oportunidades de arbitragem.
2. Existe uma medida Q, equivalente a P, sob a qual os preços descontados são martingales.
3. Existe um sistema de preços lineares consistente com o mercado.

A versão de um período, passo a passo

Considere n ativos e m estados. O ativo j custa p_j hoje e paga $D_{\omega j}$ no estado ω. A matriz D tem m linhas e n colunas. Uma carteira θ custa $p \cdot \theta$ e paga $D\theta$.

Definição de preços de estado. Um vetor $\psi \in \mathbb{R}^m$, com todos os componentes positivos, tal que

$$p = D^\top \psi, \quad \text{isto é,} \quad p_j = \sum_\omega \psi_\omega D_{\omega j}. \tag{3.21}$$

ψ_ω é o **preço hoje de receber 1 no estado** ω (título de Arrow-Debreu).

Parte fácil: preços de estado eliminam arbitragem. Se $D\theta \geq 0$ e pelo menos um componente é positivo, e ψ é estritamente positivo, então

$$p \cdot \theta = (D^\top \psi) \cdot \theta = \psi \cdot D\theta > 0. \tag{3.22}$$

Ou seja, uma carteira com pagamento não negativo e não nulo **tem custo positivo**. Não há almoço grátis.

Parte difícil: sem arbitragem existem preços de estado. Aqui está a ideia geométrica.

Passo 1. Cada carteira gera o vetor $(-p \cdot \theta,\ D\theta)$ em $\mathbb{R}^{1+m}$: fluxo de caixa hoje e em cada estado. O conjunto desses vetores é um **subespaço linear** M.

Passo 2. Arbitragem significa que M toca o octante positivo $\mathbb{R}^{1+m}_+$ em algum ponto diferente da origem.

Passo 3. Sem arbitragem, M e o octante só se encontram na origem. Pelo **teorema do hiperplano separador**, existe um vetor (ψ_0, ψ) estritamente positivo e **ortogonal a** M.

Passo 4. Ortogonal a M significa, para todo θ:

$$-\psi_0\, p \cdot \theta + \psi \cdot D\theta = 0. \tag{3.23}$$

Como vale para todo θ, temos $\psi_0\, p = D^\top \psi$. Normalizando por ψ_0, chegamos a $p = D^\top \psi$.

De preços de estado a medida martingale

Seja o título, que custa 1 e paga $1 + r$ em todos os estados. Aplicando $p = D^\top \psi$:

$$1 = (1 + r) \sum_\omega \psi_\omega. \tag{3.24}$$

Logo, a soma dos ψ_ω é o fator de desconto. Defina

$$Q(\omega) = (1 + r)\, \psi_\omega. \tag{3.25}$$

São números positivos que somam 1: uma probabilidade. Para qualquer ativo,

$$p_j = \sum_\omega \psi_\omega D_{\omega j} = \frac{1}{1+r} \sum_\omega Q(\omega) D_{\omega j} = \frac{\mathbb{E}^Q[D_j]}{1+r}. \tag{3.26}$$

A probabilidade neutra ao risco é apenas o preço de estado normalizado. Essa é a relação da Proposição 1 do livro entre sistema de preços e medida martingale.

A geometria com dois estados

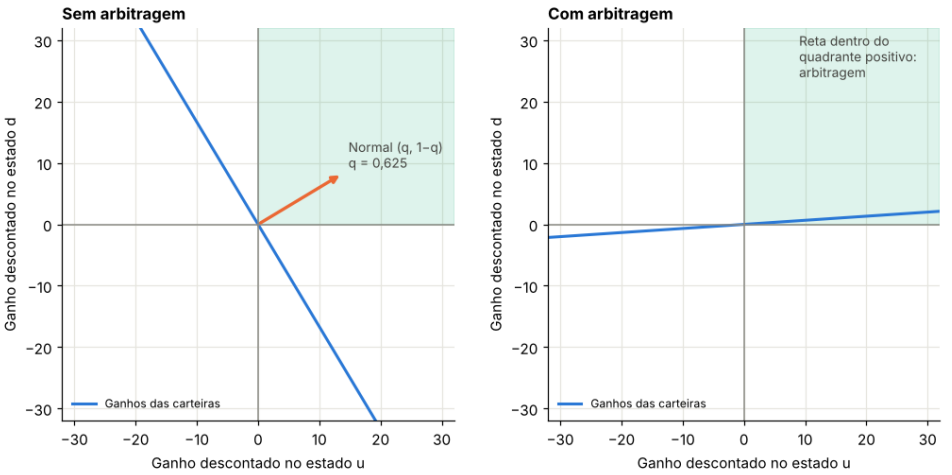
Com um ativo arriscado e dois estados, o ganho descontado de uma carteira com θ unidades é $\theta\,(\Delta\tilde{S}_u, \Delta\tilde{S}_d)$: todas as carteiras formam uma **reta pela origem**.

- Se a reta atravessa os quadrantes sem entrar no positivo, não há arbitragem, e a normal à reta dá $(q, 1-q)$.
- Se a reta entra no quadrante positivo, há arbitragem, e nenhuma normal tem componentes positivos.

```
def painel(ax, Su, Sd, r, titulo):
    du_, dd_ = Su / (1 + r) - 100, Sd / (1 + r) - 100
    lim = 32
    ax.axhspan(0, lim, xmin=0.5, xmax=1, color=VERDE, alpha=0.14)
    esc = 2 * lim / max(abs(du_), abs(dd_))
    ax.plot([-esc * du_, esc * du_], [-esc * dd_, esc * dd_], color=AZUL, label="Ganhos das
↪   carteiras")
    ax.axhline(0, color=CINZA, lw=1)
    ax.axvline(0, color=CINZA, lw=1)
    if du_ * dd_ < 0:
        q = -dd_ / (du_ - dd_)
        ax.annotate("", xy=(22 * q, 22 * (1 - q)), xytext=(0, 0),
↪   arrowprops=dict(arrowstyle="-|>", color=LARANJA, lw=2.2))
        ax.text(22 * q + 1, 22 * (1 - q) + 1, f"Normal (q, 1−q)\nq = {f3(q)}", color="#444440",
↪   fontsize=9)
    else:
        ax.text(9, 24, "Reta dentro do\nquadrante positivo:\narbitragem", color="#444440",
↪   fontsize=9)
    ax.set_xlim(-lim, lim)
    ax.set_ylim(-lim, lim)
    ax.set_aspect("equal")
```

```
    ax.set_title(titulo)
    ax.set_xlabel("Ganho descontado no estado u")
    ax.set_ylabel("Ganho descontado no estado d")
    ax.legend(loc="lower left", fontsize=8)


fig, (a1, a2) = plt.subplots(1, 2, figsize=(9.5, 4.6))
painel(a1, 120.0, 80.0, 0.05, "Sem arbitragem")
painel(a2, 120.0, 106.0, 0.05, "Com arbitragem")
plt.tight_layout()
plt.show()
```



Laboratório 6.1 — Teste de força do teorema em mercados aleatórios

Sorteamos centenas de mercados, com números aleatórios de ativos e estados, e para cada um respondemos duas perguntas **de forma independente**, por programação linear:

- **Existe arbitragem?** Procuramos θ limitado com $D\theta \geq 0$, $p \cdot \theta \leq 0$ e benefício positivo.
- **Existem preços de estado positivos?** Procuramos ψ com $D^\top \psi = p$ e o maior piso ε possível.

O teorema diz que **exatamente uma** das respostas é sim, em todos os mercados.

```
def achar_arbitragem(D, p):
    """Maximiza o benefício de uma carteira limitada com Dθ >= 0 e p·θ <= 0."""
    m, n = D.shape
    A_ub = np.vstack([-D, p])
    b_ub = np.zeros(m + 1)
    c = -(D.sum(axis=0) - p)
    res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=[(-1, 1)] * n, method="highs")
    return -res.fun, res.x


def achar_precos_estado(D, p):
```

```
    """Maximiza o piso ε de um vetor ψ com D'ψ = p."""
    m, n = D.shape
    c = np.zeros(m + 1)
    c[-1] = -1.0
    A_eq = np.hstack([D.T, np.zeros((n, 1))])
    A_ub = np.hstack([-np.eye(m), np.ones((m, 1))])
    res = linprog(c, A_ub=A_ub, b_ub=np.zeros(m), A_eq=A_eq, b_eq=p, bounds=[(None, None)] * m +
↪   [(None, 1.0)], method="highs")
    if res.status != 0:
        return -np.inf, None
    return res.x[-1], res.x[:-1]


rng = np.random.default_rng(406)
tol = 1e-8
tot = {"sem arbitragem": 0, "com arbitragem": 0, "contradições": 0}
for i in range(800):
    m, n = rng.integers(2, 7), rng.integers(1, 6)
    D = rng.uniform(0.0, 3.0, (m, n))
    if i % 2 == 0:
        p = D.T @ rng.uniform(0.05, 1.0, m)
    else:
        p = rng.uniform(0.0, 3.0, n)
    benef, _ = achar_arbitragem(D, p)
    eps, _ = achar_precos_estado(D, p)
    arb, psi = benef > tol, eps > tol
    if arb == psi:
        tot["contradições"] += 1
    tot["com arbitragem" if arb else "sem arbitragem"] += 1

print(tot)
ok(tot["contradições"] == 0, "em todos os mercados sorteados, há arbitragem ou há preços de
↪   estado, nunca os dois")
ok(min(tot["com arbitragem"], tot["sem arbitragem"]) > 100, "o teste cobriu bem os dois casos")
```

```
{'sem arbitragem': 540, 'com arbitragem': 260, 'contradições': 0}
OK — em todos os mercados sorteados, há arbitragem ou há preços de estado, nunca os dois
OK — o teste cobriu bem os dois casos
```

---

7. Precificação e replicação em vários períodos

Livro: §4.4, Corolário 1 e Proposição 2

7.1 A fórmula

Se o mercado é livre de arbitragem e A_T é atingível por ϕ, o lema da Seção 4.4 dá: $\tilde{V}$ é martingale sob qualquer Q. Logo,

$$\beta_t V_t(\phi) = \mathbb{E}_t^Q[\beta_T A_T] \quad \Longrightarrow \quad A_t = \beta_t^{-1}\,\mathbb{E}_t^Q[\beta_T A_T]. \tag{3.27}$$

Três conclusões do Corolário 1:

- O preço é **único**: todas as estratégias que geram A_T têm o mesmo custo.
- O preço **não depende de qual** Q se escolhe, se o direito for atingível.
- A probabilidade real não aparece.

7.2 De um período para vários: nó a nó

Um modelo em árvore é uma sucessão de mini-mercados de um período. Em cada nó, calculamos a probabilidade neutra da alta,

$$q(\text{nó}) = \frac{S_{\text{nó}}\,(S^0_{t+1}/S^0_t) - S_{\text{desce}}}{S_{\text{sobe}} - S_{\text{desce}}}. \tag{3.28}$$

A medida Q de um estado é o produto das probabilidades dos ramos. Há arbitragem se e só se, em algum nó, q sai do intervalo $(0,1)$.

A carteira replicante se calcula **de trás para frente**: em cada nó, resolvemos o sistema 2\$×\$2 do Módulo 1, tendo como alvo o valor da carteira no período seguinte.

Laboratório 7.1 — Um produto dependente da trajetória

Três períodos, opção asiática: paga $\max(\bar{S}-K,0)$ com $\bar{S}$ a média de S_1, S_2, S_3 e $K = 100$. O algoritmo abaixo serve para **qualquer** payoff numa árvore binária.

```
def medida_Q(S, B, filt):
    """Constrói Q pelos ramos. Devolve (Q, menor q, maior q)."""
    T_ = len(filt) - 1
    Q_ = pd.Series(1.0, index=S.index)
    qs = []
    for t in range(T_):
        for a in filt[t]:
            filhos = [f for f in filt[t + 1] if set(f) <= set(a)]
            sobe = max(filhos, key=lambda f: S.loc[f[0], t + 1])
            desce = min(filhos, key=lambda f: S.loc[f[0], t + 1])
            w = a[0]
            q_no = (S.loc[w, t] * B.loc[w, t + 1] / B.loc[w, t] - S.loc[desce[0], t + 1]) /
 ↪  (S.loc[sobe[0], t + 1] - S.loc[desce[0], t + 1])
            qs.append(q_no)
            Q_.loc[sobe] *= q_no
            Q_.loc[desce] *= 1 - q_no
    return Q_, min(qs), max(qs)


def replicar(A, S, B, filt):
    """Calcula valor e carteira replicante de trás para frente."""
    T_ = len(filt) - 1
    V = pd.DataFrame(index=S.index, columns=range(T_ + 1), dtype=float)
    V[T_] = A
    fi_s = pd.DataFrame(index=S.index, columns=range(1, T_ + 1), dtype=float)
    fi_b = fi_s.copy()
    for t in range(T_, 0, -1):
        for a in filt[t - 1]:
            filhos = [f for f in filt[t] if set(f) <= set(a)]
            w1, w2 = filhos[0][0], filhos[1][0]
            M = np.array([[S.loc[w1, t], B.loc[w1, t]], [S.loc[w2, t], B.loc[w2, t]]])
            th = np.linalg.solve(M, [V.loc[w1, t], V.loc[w2, t]])
            fi_s.loc[a, t], fi_b.loc[a, t] = th
            V.loc[a, t - 1] = th[0] * S.loc[a[0], t - 1] + th[1] * B.loc[a[0], t - 1]
    return V, fi_s, fi_b


T3 = 3
om3, fil3 = particoes(T3)
S3 = arvore(T3, 100.0, 1.2, 0.8)
```

```
B3 = pd.DataFrame({t: 1.05**t for t in range(T3 + 1)}, index=om3)
media = S3[[1, 2, 3]].mean(axis=1)
A3 = np.maximum(media - 100.0, 0.0)

Q3, qmin, qmax = medida_Q(S3, B3, fil3)
V3, fs3, fb3 = replicar(A3, S3, B3, fil3)
beta3 = 1 / B3

tabela(pd.DataFrame({"Média": media, "Payoff": A3, "Q": Q3}).reset_index(names="Estado"))
print("Valor hoje:", f3(float(V3[0].iloc[0])), "| Pela esperança sob Q:", f3(float((beta3[T3] *
↪  A3 * Q3).sum())))

ok(qmin > 0 and qmax < 1, "em todos os nós, 0 < q < 1: não há arbitragem")
ok(all(mensuravel(fs3[t], fil3[t - 1]) and mensuravel(fb3[t], fil3[t - 1]) for t in range(1, T3 +
↪  1)), "a carteira replicante é previsível")
ok(all(np.allclose(fs3[t] * S3[t] + fb3[t] * B3[t], fs3[t + 1] * S3[t] + fb3[t + 1] * B3[t]) for
↪  t in range(1, T3)), "a carteira replicante é autofinanciada")
ok(np.allclose(fs3[T3] * S3[T3] + fb3[T3] * B3[T3], A3), "a carteira replica o payoff em todos os
↪  estados")
ok(all(np.allclose(beta3[t] * V3[t], E_cond(beta3[T3] * A3, fil3[t], Q3)) for t in range(T3 + 1)),
↪  "Proposição 2: β_t V_t = E_t^Q[β_T A_T] em todas as datas")
```

Estado	Média	Payoff	Q
uuu	145,600	45,600	0,244
uud	126,400	26,400	0,146
udu	110,400	10,400	0,146
udd	97,600	0,000	0,088
duu	97,067	0,000	0,146
dud	84,267	0,000	0,088
ddu	73,600	0,000	0,088
ddd	65,067	0,000	0,053

```
Valor hoje: 14,274 | Pela esperança sob Q: 14,274
OK — em todos os nós, 0 < q < 1: não há arbitragem
OK — a carteira replicante é previsível
OK — a carteira replicante é autofinanciada
OK — a carteira replica o payoff em todos os estados
OK — Proposição 2: β_t V_t = E_t^Q[β_T A_T] em todas as datas
```

Laboratório 7.2 — Por que P não aparece

Variamos a probabilidade real da alta. O preço justo e a carteira replicante não mudam. O que muda é o retorno esperado da opção: quem compra opção num mundo otimista espera ganhar mais, mas o preço hoje é o mesmo.

```
linhas = []
for p_ in [0.2, 0.5, 0.7, 0.9]:
    P_ = medida(om3, p_)
    esperado = float((A3 * P_).sum()) / 1.05**3
    linhas.append({"Prob. real de alta": p_, "Payoff médio descontado a r": esperado, "Preço de
↪  replicação": float(V3[0].iloc[0]), "Retorno esperado da opção (%)": (float((A3 * P_).sum()) /
↪  float(V3[0].iloc[0]) - 1) * 100})
df = pd.DataFrame(linhas)
tabela(df)
ok(df["Preço de replicação"].nunique() == 1 and df["Payoff médio descontado a r"].nunique() == 4,
↪  "o preço não depende de P; a média sob P depende")
```

Prob. real de alta	Payoff médio descontado a r	Preço de replicação	Retorno esperado da opção (%)
0,200	1,332	14,274	-89,194
0,500	8,898	14,274	-27,839
0,700	18,184	14,274	47,478
0,900	31,291	14,274	153,778

```
OK — o preço não depende de P; a média sob P depende
```

---

8. Juros aleatórios e títulos de Arrow-Debreu

Livro: §4.3 e final da §4.4

Até aqui, juros foram constantes. A teoria só exige que o juro do período seja conhecido no início dele. Com juros aleatórios, β_T **entra dentro da esperança**, e já não é possível separar o desconto do payoff.

Título de Arrow-Debreu é o direito que paga 1 em um único estado. Aplicando a fórmula de preços:

$$\mathrm{AD}(\omega) = \mathbb{E}^Q[\beta_T \, \mathbf{1}_\omega] = \beta_T(\omega)\, Q(\omega). \tag{3.29}$$

Todo direito é uma combinação deles, e o preço é a soma: $A_0 = \sum_\omega A_T(\omega)\,\mathrm{AD}(\omega)$.

Laboratório 8.1 — Dois períodos com juros aleatórios

O juro do primeiro período é 5%. No segundo, vai a 8% se o índice subiu e a 2% se caiu. Os preços do índice são dados na árvore.

```
est = ["uu", "ud", "du", "dd"]
S_r = pd.DataFrame({0: 100.0, 1: [120.0, 120.0, 90.0, 90.0], 2: [150.0, 100.0, 110.0, 80.0]},
 ↪  index=est)
B_r = pd.DataFrame({0: 1.0, 1: 1.05, 2: [1.05 * 1.08, 1.05 * 1.08, 1.05 * 1.02, 1.05 * 1.02]},
 ↪  index=est)
_, fil_r = particoes(2)

Q_r, q1, q2 = medida_Q(S_r, B_r, fil_r)
beta_r = 1 / B_r
AD = beta_r[2] * Q_r
A_r = np.maximum(S_r[2] - 100.0, 0.0)
V_r, _, _ = replicar(A_r, S_r, B_r, fil_r)

tabela(pd.DataFrame({"β₂": beta_r[2], "Q": Q_r, "Preço Arrow-Debreu": AD, "Payoff da call":
 ↪  A_r}).reset_index(names="Estado"))

zero = float(AD.sum())
ingenuo = 1 / (1.05 * (1 + float((Q_r[["uu", "ud"]].sum() * 0.08 + Q_r[["du", "dd"]].sum() *
 ↪  0.02))))
print("Título zero de 2 períodos:", f3(zero), "| Descontando pelo juro médio:", f3(ingenuo))
print("Call: Arrow-Debreu", f3(float((A_r * AD).sum())), "| Replicação",
 ↪  f3(float(V_r[0].iloc[0])))

print("A diferença entre as duas primeiras é pequena aqui, mas não é zero: a média dos descontos
 ↪  não é o desconto do juro médio.")
ok(mensuravel(B_r[2], fil_r[1]), "o título do período 2 é conhecido em t = 1 (previsível)")
ok(eh_martingale(S_r / B_r, fil_r, Q_r), "preço descontado por juros aleatórios é martingale sob
 ↪  Q")
```

```
ok(abs(float((A_r * AD).sum()) - float(V_r[0].iloc[0])) < 1e-10, "preço por Arrow-Debreu coincide
↪  com o de replicação")
ok(abs(float((S_r[2] * AD).sum()) - 100.0) < 1e-10, "os títulos Arrow-Debreu reprecificam o
↪  índice")
```

Estado	β_2	Q	Preço Arrow-Debreu	Payoff da call
uu	0,882	0,296	0,261	50,000
ud	0,882	0,204	0,180	0,000
du	0,934	0,197	0,184	10,000
dd	0,934	0,303	0,283	0,000

```
Título zero de 2 períodos: 0,908 | Descontando pelo juro médio: 0,907
Call: Arrow-Debreu 14,887 | Replicação 14,887
A diferença entre as duas primeiras é pequena aqui, mas não é zero: a média dos descontos não é o desconto do
↪  juro médio.
OK — o título do período 2 é conhecido em t = 1 (previsível)
OK — preço descontado por juros aleatórios é martingale sob Q
OK — preço por Arrow-Debreu coincide com o de replicação
OK — os títulos Arrow-Debreu reprecificam o índice
```

---

9. Mercados incompletos

Livro: §4.4, definições 23 e 27, Proposição 3

9.1 Completude

O mercado é **completo** se todo direito contingente é atingível. A Proposição 3 (o Segundo Teorema Fundamental) diz:

$$\text{sem arbitragem:} \qquad \text{mercado completo} \iff Q \text{ é único.} \tag{3.30}$$

No caso de um período: completude significa que D tem **posto igual ao número de estados**. Faltam ativos quando há mais estados que ativos independentes.

Olhe para as equações $D^\top \psi = p$: são n equações em m incógnitas. Se $m >$ posto(D), há infinitas soluções. Cada uma é uma medida martingale.

9.2 Um mundo com três estados

Um período, índice em $S_0 = 100$ podendo ir a 120, 100 ou 80, e $r = 2\%$. Temos só índice e título: 3 estados, 2 ativos. As condições sobre Q são duas (soma 1 e preço do índice), logo sobra um grau de liberdade: um **segmento** de medidas martingale.

9.3 Intervalo de preços e super-replicação

Se um direito não é atingível, sem arbitragem o preço pode ser qualquer número no intervalo

$$\Big(\inf_Q \mathbb{E}^Q[\beta_T A_T],\ \sup_Q \mathbb{E}^Q[\beta_T A_T] \Big). \tag{3.31}$$

O lado do vendedor tem outra interpretação. O **custo de super-replicação** é a menor quantia que permite montar uma carteira que paga **pelo menos** A_T em todo estado. Pela dualidade de programação linear, ele é igual ao supremo acima:

$$\min_{\theta} \{p \cdot \theta : D\theta \geq A\} = \max_{\psi \geq 0} \{\psi \cdot A : D^{\top}\psi = p\}. \tag{3.32}$$

É o resultado por trás do comentário do Módulo 1: em um mercado incompleto, só resta minimizar o erro ou pagar a super-replicação.

Laboratório 9.1 — Uma call no mercado trinomial

```
r9 = 0.02
Sf = np.array([120.0, 100.0, 80.0])
D2 = np.column_stack([np.full(3, 1 + r9), Sf])
p2 = np.array([1.0, 100.0])
A9 = np.maximum(Sf - 100.0, 0.0)

print("Posto de D:", np.linalg.matrix_rank(D2), "| Número de estados:", D2.shape[0])


def faixa_Q(payoff, D, p, desc):
    """Mínimo e máximo do valor descontado sob o fecho das medidas martingale."""
    m = D.shape[0]
    aeq, beq = D.T * desc, p
    bai = linprog(payoff * desc, A_eq=aeq, b_eq=beq, bounds=[(0, 1)] * m, method="highs")
    alt = linprog(-payoff * desc, A_eq=aeq, b_eq=beq, bounds=[(0, 1)] * m, method="highs")
    return bai.fun, -alt.fun, bai.x, alt.x


def super_replicacao(payoff, D, p):
    n = D.shape[1]
    r_ = linprog(p, A_ub=-D, b_ub=-payoff, bounds=[(None, None)] * n, method="highs")
    return r_.fun, r_.x


def sub_replicacao(payoff, D, p):
    n = D.shape[1]
    r_ = linprog(-p, A_ub=D, b_ub=payoff, bounds=[(None, None)] * n, method="highs")
    return -r_.fun, r_.x


desc = 1 / (1 + r9)
bx, al, Qb, Qa = faixa_Q(A9, D2, p2, desc)
sup_c, th_sup = super_replicacao(A9, D2, p2)
sub_c, th_sub = sub_replicacao(A9, D2, p2)

tabela(pd.DataFrame([
    {"Grandeza": "Mínimo sob as medidas martingale", "Valor": float(bx)},
    {"Grandeza": "Custo de sub-replicação", "Valor": float(sub_c)},
    {"Grandeza": "Máximo sob as medidas martingale", "Valor": float(al)},
    {"Grandeza": "Custo de super-replicação", "Valor": float(sup_c)},
]))

ok(np.linalg.matrix_rank(D2) < D2.shape[0], "mercado incompleto: faltam ativos")
ok(abs(al - sup_c) < 1e-8 and abs(bx - sub_c) < 1e-8, "dualidade: extremos sob Q coincidem com
↪  super e sub-replicação")
ok((D2 @ th_sup >= A9 - 1e-9).all(), "a carteira de super-replicação cobre o payoff em todo
↪  estado")
```

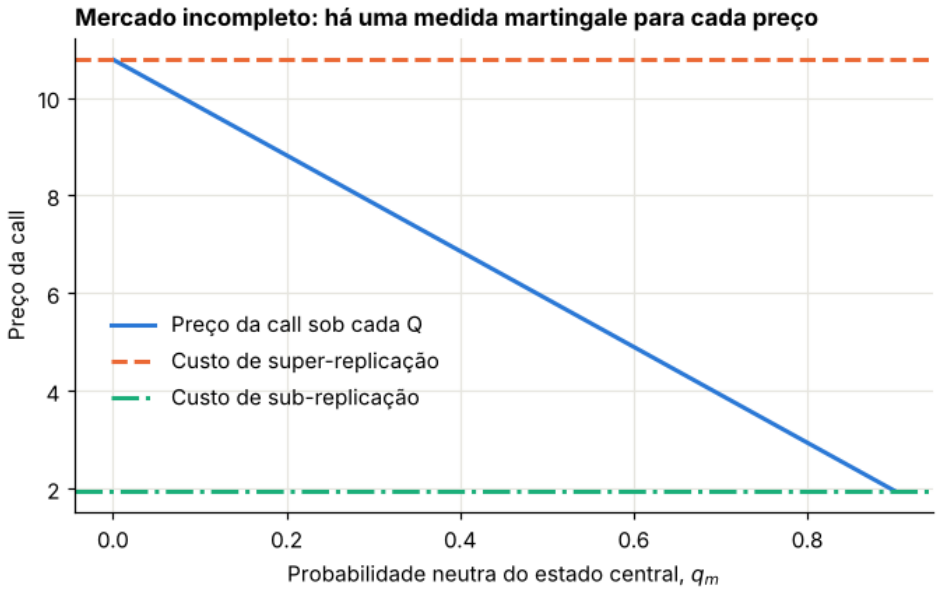
```python
# Parametrização do segmento de medidas: q_m é o grau de liberdade
qm_grade = np.linspace(1e-3, 1.0, 400)
qu = ((1 + r9) * 100 - 80 - 20 * qm_grade) / 40
qd = 1 - qu - qm_grade
validos = (qu > 0) & (qd > 0)
grade_valida = qm_grade[validos]
preco_call = desc * 20 * qu[validos]

fig, ax = plt.subplots()
ax.plot(grade_valida, preco_call, color=AZUL, label="Preço da call sob cada Q")
ax.axhline(sup_c, color=LARANJA, ls="--", label="Custo de super-replicação")
ax.axhline(sub_c, color=VERDE, ls="-.", label="Custo de sub-replicação")
ax.set_title("Mercado incompleto: há uma medida martingale para cada preço")
ax.set_xlabel("Probabilidade neutra do estado central, $q_m$")
ax.set_ylabel("Preço da call")
ax.legend(loc="center left", bbox_to_anchor=(0.02, 0.32))
plt.show()
```

```
Posto de D: 2 | Número de estados: 3
```

Grandeza	Valor
Mínimo sob as medidas martingale	1,961
Custo de sub-replicação	1,961
Máximo sob as medidas martingale	10,784
Custo de super-replicação	10,784

```
OK — mercado incompleto: faltam ativos
OK — dualidade: extremos sob Q coincidem com super e sub-replicação
OK — a carteira de super-replicação cobre o payoff em todo estado
```



Laboratório 9.2 — Completando o mercado com uma opção negociada

Suponha agora que uma call de strike 100 seja **negociada** a um preço observado dentro do intervalo. A matriz D ganha uma coluna, atinge posto 3 e a medida Q passa a ser única. Agora qualquer outro direito tem preço único.

Pare um instante nessa ideia, porque ela é **a precificação calibrada ao mercado em sua forma mais pura**. O mercado não escolhe Q pela teoria, mas pelos preços das opções líquidas. No Módulo 1, chamamos isso de calibração.

```
preco_obs = 0.5 * (bx + al)
D3 = np.column_stack([D2, A9])
p3 = np.array([1.0, 100.0, preco_obs])

psi = np.linalg.solve(D3.T, p3)
Q3e = psi * (1 + r9)
A_novo = np.maximum(110.0 - Sf, 0.0)
theta = np.linalg.solve(D3, A_novo)
por_q = float(desc * Q3e @ A_novo)
por_carteira = float(p3 @ theta)

tabela(pd.DataFrame({"Estado": ["120", "100", "80"], "Preço de estado ψ": psi, "Q": Q3e}))
print("Put 110: pela medida", f3(por_q), "| pela carteira replicante", f3(por_carteira))
print("Sem a call negociada, a put 110 poderia estar em:", [f3(float(x)) for x in faixa_Q(A_novo,
↪  D2, p2, desc)[:2]])

ok(np.linalg.matrix_rank(D3) == 3, "com a call negociada, o mercado fica completo")
ok((Q3e > 0).all() and abs(Q3e.sum() - 1) < 1e-12, "Q é único e estritamente positivo")
ok(abs(por_q - por_carteira) < 1e-10, "preço pela medida coincide com o preço pela carteira")
```

Estado	Preço de estado ψ	Q
120	0,319	0,325
100	0,441	0,450
80	0,221	0,225

```
Put 110: pela medida 11,029 | pela carteira replicante 11,029
Sem a call negociada, a put 110 poderia estar em: ['8,824', '13,235']
OK — com a call negociada, o mercado fica completo
OK — Q é único e estritamente positivo
OK — preço pela medida coincide com o preço pela carteira
```

---

10. Síntese

- Ω lista as histórias; uma filtração descreve como a informação chega; partições tornam tudo computável.
- Martingale é o jogo justo. Uma estratégia previsível sobre um martingale continua sendo um martingale.
- Radon-Nikodym e a fórmula de Bayes permitem trocar de medida; serão a base do Girsanov.
- Sem arbitragem existe Q, e Q é o preço de estado normalizado.
- Direito atingível tem preço único: $A_t = \beta_t^{-1}\mathbb{E}_t^Q[\beta_T A_T]$.
- Com juros aleatórios, β_T fica dentro da esperança.

- Mercado completo equivale a Q único. Em mercado incompleto, o preço é um intervalo, e os preços das opções negociadas ajudam a fixar Q.

11. Erratas do livro neste trecho

Local	Problema
Defs. 19 e 25	admissibilidade com $V_t \geq -\alpha$ torna "arbitragem" qualquer estratégia alavancada com ganho esperado positivo; o correto (Harrison-Pliska) é $V_t \geq 0$
Prova do Lema 1	"$u < t$" na definição de ψ deveria ser "$u > t$"
Prova da Prop. 1	a soma usa $\beta_{t+1}S_{t+1}$; o correto é $\beta_{t-1}S_{t-1}$
Prova da Prop. 1	"trading strategy $\zeta \in T$" deveria ser ϕ
Nota 13	refere "condição 3 da definição 3"; o certo é definição 1 (álgebra)

12. Exercícios

Exercício 1. Uma partição tem 3 átomos. Quantos eventos tem a álgebra gerada? Liste-os para $\{\{a\}, \{b\}, \{c, d\}\}$.

Solução

$2^3 = 8$: $\emptyset$, $\{a\}$, $\{b\}$, $\{c, d\}$, $\{a, b\}$, $\{a, c, d\}$, $\{b, c, d\}$, Ω.

Exercício 2. Numa árvore de dois períodos com $u = 1{,}1$, $d = 0{,}9$, $r = 2\%$, $S_0 = 100$, calcule à mão q e o preço da call com $K = 100$.

Solução

$q = (1{,}02 - 0{,}9)/0{,}2 = 0{,}600$. Só uu paga: $S_{uu} = 121$, payoff 21. Preço $= 0{,}600^2 \times 21/1{,}02^2 = 7{,}266$.

Exercício 3. Mostre que $L_t = \mathbb{E}_t^P[L]$ é martingale sob P.

Solução

Pela lei das esperanças iteradas, para $s \leq t$: $\mathbb{E}_s^P[L_t] = \mathbb{E}_s^P[\mathbb{E}_t^P[L]] = \mathbb{E}_s^P[L] = L_s$.

Exercício 4. Num modelo de um período, suponha $d > 1 + r$. Monte uma arbitragem explícita. Qual a relação com o valor de q?

Solução

Pegue 1 de título emprestado e compre o índice: valor inicial zero, e valor final $S_0(d-(1+r)) > 0$ no pior caso. Nesse caso, $q = (1+r-d)/(u-d) < 0$: não existe probabilidade.

Exercício 5. No Laboratório 9.1, troque o índice por um mercado com $r = 0$. Mostre que o intervalo de preços da call com $K = 100$ vira $(0, 10)$.

Solução

Com $r = 0$, as condições são $q_u = q_d$ e $q_m = 1 - 2q_u$, com $q_u \in (0, 1/2)$. A call paga 20 só no estado u: seu preço é $20q_u \in (0, 10)$.

```
print(f"Módulo 3 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 3 concluído: 40 verificações aprovadas.
```

Capítulo 4

Cálculo estocástico: movimento browniano, Itô e Girsanov

Introdução

Para levar a teoria do Capítulo 3 ao tempo contínuo, é preciso uma matemática própria para trajetórias aleatórias que não são suaves. O **movimento browniano** é o modelo básico desse tipo de trajetória: contínuo, mas com oscilações em qualquer escala de tempo, a ponto de não ter derivada em ponto algum.

Essa irregularidade muda as regras do cálculo. Uma integral em relação ao movimento browniano, a **integral de Itô**, depende de onde se avalia o integrando em cada intervalo, e a escolha financeiramente correta, decidir a posição antes de conhecer o movimento seguinte, faz da integral um martingal. A regra da cadeia também muda: a **fórmula de Itô** ganha um termo de segunda ordem, porque o quadrado de um incremento browniano não é desprezível. É desse termo que vem o $-\frac{1}{2}\sigma^2$ que aparece em toda a teoria de opções.

A terceira peça é o **teorema de Girsanov**, que mostra como trocar a medida de probabilidade alterando apenas a deriva do processo. É ele que transforma a dinâmica observada de um ativo na dinâmica neutra ao risco usada para precificar. O capítulo apresenta as três ferramentas com simulações que as tornam concretas: o leitor vê a variação quadrática convergir, a correção de Itô aparecer na média e a mudança de medida reponderar trajetórias.

Fundamentos teóricos

Fixamos um espaço de probabilidade $(\Omega, \mathcal{F}, \mathbb{P})$ com uma filtração $(\mathcal{F}_t)_{t\geq 0}$ que satisfaz as condições usuais. As referências para esta seção são Shreve (2004b), Karatzas and Shreve (1991) e Øksendal (2003).

Movimento browniano

Definição 4.1 (Movimento browniano). Um processo $(W_t)_{t\geq 0}$ adaptado é um *movimento browniano padrão* se (i) $W_0 = 0$; (ii) para $0 \leq s < t$, o incremento $W_t - W_s$

é independente de $\mathcal{F}_s$; (iii) $W_t - W_s \sim N(0, t-s)$; (iv) as trajetórias $t \mapsto W_t(\omega)$ são contínuas.

Proposição 4.2 (Limite do passeio aleatório). *Seja $X_1, X_2, \ldots$ i.i.d. com $\mathbb{P}(X_i = \pm 1) = \frac{1}{2}$ e $W_t^{(n)} = n^{-1/2} \sum_{i \le \lfloor nt \rfloor} X_i$. Para cada t, $W_t^{(n)} \Rightarrow N(0,t)$, e o processo $W^{(n)}$ converge em distribuição para o movimento browniano (teorema de Donsker).*

Proposição 4.3 (Variação quadrática). *Para uma partição $0 = t_0 < \cdots < t_n = t$ com malha $\|\pi\| = \max_k (t_{k+1} - t_k) \to 0$,*

$$\sum_{k=0}^{n-1} \left(W_{t_{k+1}} - W_{t_k} \right)^2 \xrightarrow{\;L^2\;} t. \tag{4.1}$$

Demonstração. Seja Q_π a soma. Como $\mathbb{E}[(\Delta W_k)^2] = \Delta t_k$, $\mathbb{E}[Q_\pi] = t$. Pela independência dos incrementos e $\operatorname{Var}((\Delta W_k)^2) = 2\Delta t_k^2$, $\operatorname{Var}(Q_\pi) = 2\sum_k \Delta t_k^2 \le 2\|\pi\| \, t \to 0$. ∎

A proposição é a origem da regra heurística $(dW_t)^2 = dt$. Ela implica que as trajetórias brownianas têm variação total infinita em qualquer intervalo, e por isso a integral $\int H \, dW$ não pode ser definida trajetória a trajetória como uma integral de Stieltjes.

Integral de Itô

Definição 4.4 (Integral de Itô). Para um processo *simples* $H_t = \sum_k \xi_k \mathbf{1}_{(t_k, t_{k+1}]}(t)$, com ξ_k limitada e $\mathcal{F}_{t_k}$-mensurável, define-se $\int_0^T H_t \, dW_t = \sum_k \xi_k (W_{t_{k+1}} - W_{t_k})$. Para H adaptado com $\mathbb{E}\int_0^T H_t^2 \, dt < \infty$, a integral é o limite em L^2 das integrais de processos simples que aproximam H.

Teorema 4.5 (Propriedades da integral de Itô). *Para H como acima, $I_t = \int_0^t H_s \, dW_s$ é um martingal contínuo com $\mathbb{E}[I_t] = 0$ e vale a* isometria de Itô

$$\mathbb{E}\Big[\Big(\int_0^t H_s \, dW_s \Big)^2 \Big] = \mathbb{E} \int_0^t H_s^2 \, ds. \tag{4.2}$$

O integrando é avaliado no *início* de cada intervalo. É essa escolha, e não outra, que torna a integral um martingal, e ela corresponde à realidade financeira: a posição ξ_k é decidida antes de se conhecer o movimento seguinte.

Fórmula de Itô

Definição 4.6 (Processo de Itô). Um processo de Itô é $X_t = X_0 + \int_0^t \mu_s \, ds + \int_0^t \sigma_s \, dW_s$, que se escreve $dX_t = \mu_t \, dt + \sigma_t \, dW_t$; sua variação quadrática é $d\langle X \rangle_t = \sigma_t^2 \, dt$.

Teorema 4.7 (Fórmula de Itô). *Se $f \in C^{1,2}$, então*

$$df(t, X_t) = \Big(\partial_t f + \mu_t \, \partial_x f + \tfrac{1}{2} \sigma_t^2 \, \partial_{xx} f \Big) dt + \sigma_t \, \partial_x f \, dW_t. \tag{4.3}$$

Esboço. Expanda f por Taylor até a segunda ordem em cada intervalo de uma partição. Os termos $(\Delta t)^2$ e $\Delta t \, \Delta W$ somam zero no limite; os termos $(\Delta W)^2$ somam $\int \sigma^2 \, \partial_{xx} f \, dt$ pela Proposição 4.3. ∎

Exemplo 4.8 (Movimento browniano geométrico). A EDE $dS_t = \mu S_t\,dt + \sigma S_t\,dW_t$ tem solução única

$$S_t = S_0 \exp\Big((\mu - \tfrac{1}{2}\sigma^2)t + \sigma W_t \Big). \tag{4.4}$$

Basta aplicar (4.3) a $f(x) = \ln x$: $d\ln S_t = (\mu - \frac{1}{2}\sigma^2)\,dt + \sigma\,dW_t$. O termo $-\frac{1}{2}\sigma^2$ é a correção de Itô; ela garante $\mathbb{E}[S_t] = S_0 e^{\mu t}$.

Mudança de medida

Teorema 4.9 (Girsanov). *Seja γ adaptado com $\mathbb{E}\big[\exp\big(\frac{1}{2}\int_0^T \gamma_s^2 ds\big)\big] < \infty$ (condição de Novikov) e*

$$Z_T = \exp\Big(-\int_0^T \gamma_s\,dW_s - \tfrac{1}{2}\int_0^T \gamma_s^2\,ds \Big). \tag{4.5}$$

Então $\mathbb{E}[Z_T] = 1$, $d\mathbb{Q} = Z_T\,d\mathbb{P}$ define uma medida equivalente, e $\widetilde{W}_t = W_t + \int_0^t \gamma_s\,ds$ é um movimento browniano sob $\mathbb{Q}$.

Corolário 4.10 (Medida neutra ao risco no modelo lognormal). *No movimento browniano geométrico com taxa livre de risco r, escolha o* preço de mercado do risco *$\gamma = (\mu - r)/\sigma$. Sob a $\mathbb{Q}$ do Teorema 4.9, $dS_t = rS_t\,dt + \sigma S_t\,d\widetilde{W}_t$, e o preço descontado $e^{-rt}S_t$ é um $\mathbb{Q}$-martingal.*

Teorema 4.11 (Representação de martingais). *Se $(\mathcal{F}_t)$ é a filtração gerada por W e M é um martingal de quadrado integrável, existe H adaptado com $M_t = M_0 + \int_0^t H_s\,dW_s$.*

A representação de martingais é a versão contínua do Segundo Teorema Fundamental: no modelo de um fator browniano, todo payoff de quadrado integrável é replicável, e o integrando H dá a estratégia de hedge (Harrison and Pliska, 1981).

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** seção 4.5 do livro e o lema de Itô das seções 5.6.1 e 9.7.1.

O que você vai conseguir fazer ao final

1. Explicar o que é um movimento browniano e por que ele não tem derivada.
2. Entender por que a integral estocástica depende do ponto de avaliação, e por que se usa o de Itô.
3. Aplicar o lema de Itô, sabendo de onde vem o termo extra.
4. Trocar de medida com o teorema de Girsanov e conectar isso à derivada de Radon-Nikodym do Módulo 3.
5. Mostrar por que μ desaparece do preço de uma opção, agora em tempo contínuo.
6. Explicar por que estratégias precisam de uma condição de admissibilidade.
7. Dizer quando um modelo contínuo é completo, e o que muda com volatilidade estocástica.

Como este módulo se relaciona com o livro

O livro enuncia os resultados de tempo contínuo em alto nível e remete às referências, e **não trata do teorema de Girsanov**. Aqui construímos os mesmos resultados de forma intuitiva, partindo do mundo discreto do Módulo 3 e **conferindo cada passo numericamente**. A abordagem é heurística no rigor, mas correta nos resultados; para as demonstrações completas, veja Björk (2004) ou Shreve (2004).

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, kurtosis
from IPython.display import display

def bsm_call(S, K, T, r, sigma):
    d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - sigma * np.sqrt(T))

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. Do passeio aleatório ao movimento browniano

Livro: §4.5, Definição 33

1.1 A escala correta

Comece com o jogo justo do Módulo 3: em cada passo, sobe ou desce com chances iguais. Queremos encurtar os passos, **mantendo uma variância finita**. Para n passos em $[0, 1]$, com duração $\Delta t = 1/n$ e tamanho a:

$$\operatorname{Var}\Big(\sum_{i=1}^{n} X_i\Big) = n\,a^2. \tag{4.6}$$

Para essa variância valer $t = 1$, é preciso $a = \sqrt{\Delta t}$. Esse é o coração de tudo o que vem:

$$\boxed{\text{um movimento de duração } \Delta t \text{ tem tamanho } \sqrt{\Delta t}.} \tag{4.7}$$

Como $\sqrt{\Delta t} \gg \Delta t$ para Δt pequeno, o movimento aleatório domina a tendência em escalas curtas.

Pelo teorema central do limite, a soma tende à normal. A versão para trajetórias inteiras é o **teorema de Donsker**: o passeio aleatório reescalado converge para um processo contínuo, o movimento browniano.

1.2 Definição

Um **movimento browniano padrão** (Z_t) é um processo adaptado que satisfaz:

1. $Z_0 = 0$;
2. o incremento $Z_t - Z_s$ é independente de $\mathcal{F}_s$, para $s < t$ (o futuro não depende do passado);
3. $Z_t - Z_s \sim N(0, t-s)$;
4. as trajetórias são contínuas.

Laboratório 1.1 — A convergência

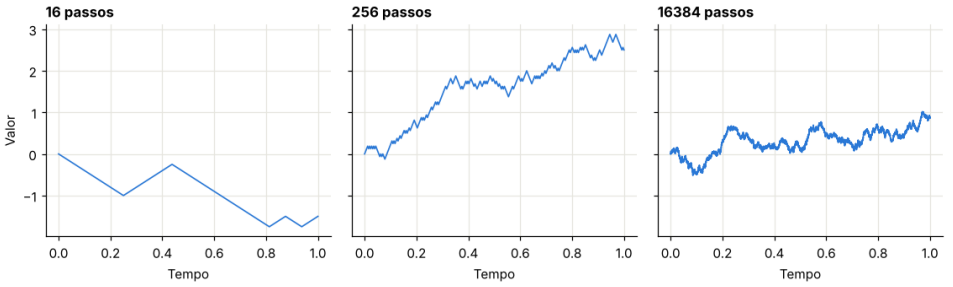
A soma de n passos $\pm\sqrt{1/n}$ pode ser amostrada diretamente pela distribuição binomial. Para um passeio de n passos, a teoria dá curtose em excesso igual a $-2/n$.

```
rng = np.random.default_rng(501)
linhas = []
for n in [1, 4, 16, 64, 1024, 65536]:
    W = (2 * rng.binomial(n, 0.5, 2_000_000) - n) / np.sqrt(n)
    linhas.append({"Passos n": n, "Variância": W.var(), "Curtose em excesso": float(kurtosis(W)),
↪   "Teoria: -2/n": -2 / n, "P(W ≤ 1)": (W <= 1 + 1e-12).mean()})
df = pd.DataFrame(linhas)
tabela(df)
print("Φ(1) =", f3(norm.cdf(1)))
ok(np.allclose(df["Variância"], 1, atol=0.005), "com passo √Δt, a variância é 1 em qualquer n")
ok(np.allclose(df["Curtose em excesso"], df["Teoria: -2/n"], atol=0.02), "a curtose do passeio
↪   segue -2/n")
ok(abs(df["P(W ≤ 1)"].iloc[-1] - norm.cdf(1)) < 0.003, "para n grande, a distribuição é a normal")

rng = np.random.default_rng(502)
fig, eixos = plt.subplots(1, 3, figsize=(10.5, 3.3), sharey=True)
for ax, n in zip(eixos, [16, 256, 16384]):
    passos = rng.choice([-1.0, 1.0], size=n) * np.sqrt(1 / n)
    ax.plot(np.linspace(0, 1, n + 1), np.r_[0.0, np.cumsum(passos)], color=AZUL, lw=1.1)
    ax.set_title(f"{n} passos")
    ax.set_xlabel("Tempo")
eixos[0].set_ylabel("Valor")
plt.tight_layout()
plt.show()
```

Passos n	Variância	Curtose em excesso	Teoria: −2/n	P(W ≤ 1)
1	1,000	-2,000	-2,000	1,000
4	1,000	-0,501	-0,500	0,938
16	0,998	-0,122	-0,125	0,895
64	1,001	-0,031	-0,031	0,869
1024	0,999	-0,006	-0,002	0,849
65536	0,999	0,004	-0,000	0,843

```
Φ(1) = 0,841
OK — com passo √Δt, a variância é 1 em qualquer n
OK — a curtose do passeio segue −2/n
OK — para n grande, a distribuição é a normal
```



---

2. As propriedades estranhas do browniano

2.1 Martingale

Se $s < t$, usando a independência dos incrementos:

$$\mathbb{E}_s[Z_t] = \mathbb{E}_s[Z_s + (Z_t - Z_s)] = Z_s + 0 = Z_s. \tag{4.8}$$

Outro martingale importante é $Z_t^2 - t$. De fato, escrevendo $Z_t = Z_s + \Delta$:

$$\mathbb{E}_s[Z_t^2] = Z_s^2 + 2Z_s\,\mathbb{E}_s[\Delta] + \mathbb{E}_s[\Delta^2] = Z_s^2 + (t - s). \tag{4.9}$$

Subtraindo t dos dois lados, resulta $\mathbb{E}_s[Z_t^2 - t] = Z_s^2 - s$.

2.2 Covariância

Para $s \le t$: $\mathbb{E}[Z_s Z_t] = \mathbb{E}[Z_s(Z_s + \Delta)] = \mathbb{E}[Z_s^2] + 0 = s$. Logo, $\operatorname{Cov}(Z_s, Z_t) = \min(s, t)$.

2.3 Variação total infinita

Particione $[0, T]$ em n intervalos de tamanho Δt. A soma dos valores absolutos tem esperança

$$\mathbb{E}\sum_i |\Delta Z_i| = n\sqrt{\Delta t}\,\sqrt{2/\pi} = \sqrt{\frac{2Tn}{\pi}} \xrightarrow[n\to\infty]{} \infty. \tag{4.10}$$

Em palavras: a distância percorrida pelo browniano em qualquer intervalo é infinita. Por isso, Z **não é derivável em lugar nenhum** e não se pode escrever a integral $\int H\,dZ$ como uma integral ordinária.

2.4 Variação quadrática finita

Agora eleve ao quadrado:

$$Q_n = \sum_i (\Delta Z_i)^2. \tag{4.11}$$

Cada termo tem média Δt e variância $2\Delta t^2$. Somando n termos independentes:

$$\mathbb{E}[Q_n] = n\Delta t = T, \qquad \mathrm{Var}(Q_n) = 2n\Delta t^2 = \frac{2T^2}{n} \to 0. \tag{4.12}$$

A variância vai a zero, então Q_n deixa de ser aleatório: converge para o **número** T. É a principal surpresa do cálculo estocástico e é escrita como

$$\boxed{(dZ_t)^2 = dt.} \tag{4.13}$$

Uma função suave faz o oposto: variação total finita e variação quadrática zero.

Laboratório 2.1 — Propriedades e variação

```
rng = np.random.default_rng(503)
M_, I_ = 100, 200_000
dZ = np.sqrt(1 / M_) * rng.standard_normal((M_, I_))
Z = np.cumsum(dZ, axis=0)
t25, t75 = 24, 74

tabela(pd.DataFrame([
    {"Grandeza": "Cov(Z(0,25), Z(0,75))", "Simulado": float(np.mean(Z[t25] * Z[t75])), "Teoria":
↪  0.25},
    {"Grandeza": "Corr de incrementos disjuntos", "Simulado": float(np.corrcoef(Z[t25], Z[t75] -
↪  Z[t25])[0, 1]), "Teoria": 0.0},
    {"Grandeza": "E[Z(1)² − 1]", "Simulado": float(np.mean(Z[-1] ** 2 - 1)), "Teoria": 0.0},
    {"Grandeza": "E[(Z(1)²−1) | Z(0,75)>0] − E[(Z(0,75)²−0,75) | Z(0,75)>0]", "Simulado":
↪  float(np.mean((Z[-1] ** 2 - 1)[Z[t75] > 0]) - np.mean((Z[t75] ** 2 - 0.75)[Z[t75] > 0])),
↪  "Teoria": 0.0},
]))
ok(abs(np.mean(Z[t25] * Z[t75]) - 0.25) < 0.005, "Cov(Z_s, Z_t) = min(s, t)")
ok(abs(np.corrcoef(Z[t25], Z[t75] - Z[t25])[0, 1]) < 0.01, "incrementos disjuntos são não
↪  correlacionados")
ok(abs(np.mean((Z[-1] ** 2 - 1)[Z[t75] > 0]) - np.mean((Z[t75] ** 2 - 0.75)[Z[t75] > 0])) < 0.015,
↪  "Z² − t é martingale, inclusive condicionado ao passado")
del Z, dZ

rng = np.random.default_rng(504)
K_max = 22
n_fino = 2**K_max
incr = np.sqrt(1 / n_fino) * rng.standard_normal(n_fino)
trajeto = np.r_[0.0, np.cumsum(incr)]
grade_fina = np.linspace(0, 1, n_fino + 1)
suave = np.sin(2 * np.pi * grade_fina)

linhas = []
for k in [4, 8, 12, 16, 20, 22]:
    p = 2 ** (K_max - k)
    z = trajeto[::p]
    s = suave[::p]
    n = 2**k
    linhas.append({"n": n, "VT browniano": float(np.abs(np.diff(z)).sum()), "Teoria √(2n/π)":
↪  float(np.sqrt(2 * n / np.pi)), "VQ browniano": float((np.diff(z) ** 2).sum()), "VT seno":
↪  float(np.abs(np.diff(s)).sum()), "VQ seno": float((np.diff(s) ** 2).sum())})
var = pd.DataFrame(linhas)
tabela(var)

fig, ax = plt.subplots()
ax.loglog(var["n"], var["VT browniano"], color=AZUL, marker="o", markersize=6, label="Variação
↪  total: browniano")
```

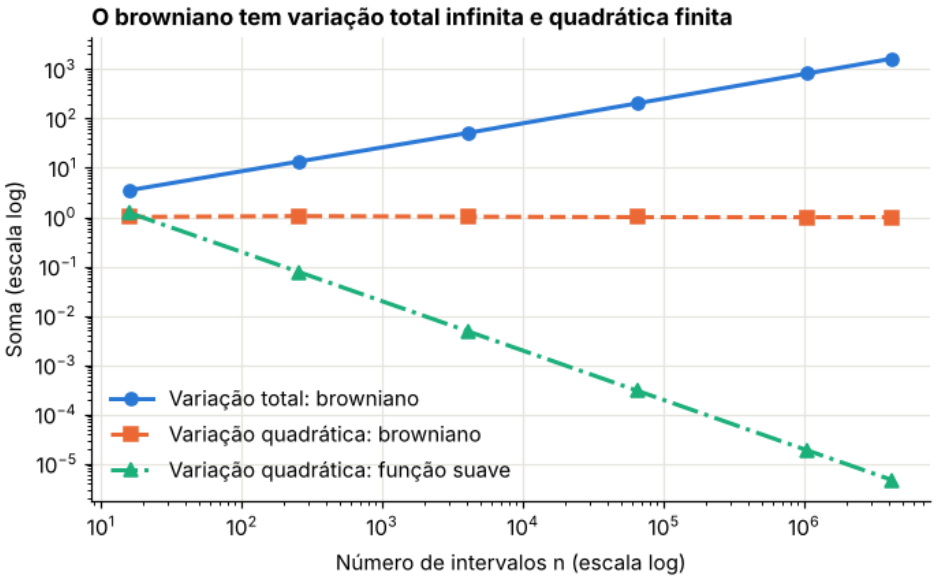
```
ax.loglog(var["n"], var["VQ browniano"], color=LARANJA, marker="s", markersize=6, ls="--",
 ↪  label="Variação quadrática: browniano")
ax.loglog(var["n"], var["VQ seno"], color=VERDE, marker="^", markersize=6, ls="-.",
 ↪  label="Variação quadrática: função suave")
ax.set_title("O browniano tem variação total infinita e quadrática finita")
ax.set_xlabel("Número de intervalos n (escala log)")
ax.set_ylabel("Soma (escala log)")
ax.legend()
plt.show()

ult = var.iloc[-1]
ok(abs(ult["VT browniano"] / ult["Teoria √(2n/π)"] - 1) < 0.01, "a variação total cresce como
 ↪  √(2n/π)")
ok(abs(ult["VQ browniano"] - 1) < 0.01, "a variação quadrática converge para T = 1")
ok(ult["VQ seno"] < 1e-4 and abs(ult["VT seno"] - 4) < 1e-3, "em função suave, a variação
 ↪  quadrática some")
```

Grandeza	Simulado	Teoria
Cov(Z(0,25), Z(0,75))	0,251	0,250
Corr de incrementos disjuntos	0,002	0,000
$\mathrm{E}[\mathrm{Z}(1)^2 - 1]$	0,000	0,000
$\mathrm{E}[(\mathrm{Z}(1)^2-1) \mid \mathrm{Z}(0{,}75)>0] - \mathrm{E}[(\mathrm{Z}(0{,}75)^2-0{,}75) \mid \ldots$	0,001	0,000

```
OK — Cov(Z_s, Z_t) = min(s, t)
OK — incrementos disjuntos são não correlacionados
OK — Z² − t é martingale, inclusive condicionado ao passado
```

n	VT browniano	Teoria $\sqrt{(2n/\pi)}$	VQ browniano	VT seno	VQ seno
16	3,523	3,192	1,012	4,000	1,218
256	13,441	12,766	1,052	4,000	0,077
4096	51,247	51,065	1,024	4,000	0,005
65536	204,107	204,258	1,002	4,000	0,000
1048576	816,699	817,034	0,999	4,000	0,000
4194304	1.633,949	1.634,068	1,000	4,000	0,000




```
OK — a variação total cresce como √(2n/π)
OK — a variação quadrática converge para T = 1
OK — em função suave, a variação quadrática some
```

---

3. A integral de Itô

Livro: §4.5, definições 28 a 31

3.1 O problema

Queremos dar sentido ao ganho de uma estratégia, $\int_0^T H_s\, dZ_s$, o análogo contínuo de G_t no Módulo 3. Tentamos o limite de somas:

$$\sum_i H(\tau_i)\,(Z_{t_{i+1}} - Z_{t_i}), \qquad \tau_i \in [t_i, t_{i+1}]. \tag{4.14}$$

Numa integral ordinária, a escolha de τ_i não importa. Aqui importa muito. Veja com $H = Z$ e denote $a = Z_{t_i}$, $b = Z_{t_{i+1}}$. Usando duas identidades algébricas:

$$a(b-a) = \tfrac{1}{2}\big[(b^2-a^2) - (b-a)^2\big], \qquad b(b-a) = \tfrac{1}{2}\big[(b^2-a^2) + (b-a)^2\big]. \tag{4.15}$$

Somando no tempo, os termos b^2-a^2 telescopam, e os termos $(b-a)^2$ dão a variação quadrática:

- **Ponto esquerdo:** $\sum a(b-a) \to \frac{1}{2}Z_T^2 - \frac{1}{2}T$.
- **Ponto direito:** $\sum b(b-a) \to \frac{1}{2}Z_T^2 + \frac{1}{2}T$.
- **Ponto médio** (Stratonovich): $\to \frac{1}{2}Z_T^2$.

A diferença é exatamente a variação quadrática, que seria zero num mundo suave.

3.2 Por que usar o ponto esquerdo

Porque é o único coerente com a negociação: você decide a posição **antes** do movimento. É exatamente a condição de previsibilidade do Módulo 3. A integral de Itô com integrando previsível é o ganho de uma estratégia factível, e herda a propriedade central:

$$\mathbb{E}\Big[\int_0^T H_s\, dZ_s\Big] = 0. \tag{4.16}$$

O livro constrói a integral com as **estratégias simples previsíveis**: posições constantes entre datas. A integral geral é o limite delas.

3.3 Isometria de Itô

Para H previsível e quadrado-integrável:

$$\mathbb{E}\Big[\Big(\int_0^T H_s\, dZ_s\Big)^2\Big] = \mathbb{E}\Big[\int_0^T H_s^2\, ds\Big]. \tag{4.17}$$

A prova na forma simples. Abrindo o quadrado de $\sum_i H_i \Delta Z_i$:

- Termos cruzados, $i < j$: $\mathbb{E}[H_i \Delta Z_i H_j \Delta Z_j] = \mathbb{E}[H_i \Delta Z_i H_j\, \mathbb{E}_{t_j}[\Delta Z_j]] = 0$.
- Termos quadrados: $\mathbb{E}[H_i^2 \Delta Z_i^2] = \mathbb{E}[H_i^2\, \mathbb{E}_{t_i}[\Delta Z_i^2]] = \mathbb{E}[H_i^2]\Delta t$.

Note que os dois passos usam H_i previsível.

Laboratório 3.1 — O ponto de avaliação muda o resultado

```
rng = np.random.default_rng(505)
n, paths = 4096, 2000
meio = np.sqrt(1 / (2 * n)) * rng.standard_normal((2 * n, paths))
Zf = np.vstack([np.zeros(paths), np.cumsum(meio, axis=0)])
inc = Zf[2::2] - Zf[:-2:2]

somas = {
    "Esquerdo (Itô)": ((Zf[:-2:2] * inc).sum(axis=0), 0.5 * Zf[-1] ** 2 - 0.5),
    "Médio (Stratonovich)": ((Zf[1::2] * inc).sum(axis=0), 0.5 * Zf[-1] ** 2),
    "Direito": ((Zf[2::2] * inc).sum(axis=0), 0.5 * Zf[-1] ** 2 + 0.5),
}
linhas = []
for nome, (soma, limite) in somas.items():
    linhas.append({"Ponto de avaliação": nome, "Média da soma": float(soma.mean()), "Raiz do erro
↪  quadrático contra o limite": float(np.sqrt(np.mean((soma - limite) ** 2)))})
res = pd.DataFrame(linhas).set_index("Ponto de avaliação")
tabela(res.reset_index())
ok(all(res["Raiz do erro quadrático contra o limite"] < 0.03), "as três somas convergem para seus
↪  limites teóricos")
ok(abs(res.loc["Esquerdo (Itô)", "Média da soma"]) < 0.07, "só a integral de Itô tem média zero")
ok(res.loc["Direito", "Média da soma"] > 0.9, "o ponto direito espia o futuro e tem média T")
del Zf, meio, inc
```

Ponto de avaliação	Média da soma	Raiz do erro quadrático contra o limite
Esquerdo (Itô)	-0,004	0,011
Médio (Stratonovich)	0,497	0,008
Direito	0,997	0,011

```
OK — as três somas convergem para seus limites teóricos
OK — só a integral de Itô tem média zero
OK — o ponto direito espia o futuro e tem média T
```

Laboratório 3.2 — A isometria

Para $H = Z$: $\mathbb{E}\int_0^1 Z_s^2\,ds = \int_0^1 s\,ds = \frac{1}{2}$.

```
rng = np.random.default_rng(506)
n, paths = 250, 300_000
dt = 1 / n
Z, integral, energia = np.zeros(paths), np.zeros(paths), np.zeros(paths)
for _ in range(n):
    d = np.sqrt(dt) * rng.standard_normal(paths)
    integral += Z * d
    energia += Z**2 * dt
    Z += d
tabela(pd.DataFrame([
    {"Grandeza": "E[∫Z dZ]", "Simulado": float(integral.mean()), "Teoria": 0.0},
    {"Grandeza": "E[(∫Z dZ)²]", "Simulado": float((integral**2).mean()), "Teoria": 0.5 * (1 - 1 /
↪   n)},
    {"Grandeza": "E[∫Z² ds]", "Simulado": float(energia.mean()), "Teoria": 0.5 * (1 - 1 / n)},
]))
ok(abs((integral**2).mean() - energia.mean()) < 0.01, "isometria de Itô")
ok(abs(integral.mean()) < 0.005, "a integral de Itô tem média zero")
```

Grandeza	Simulado	Teoria
$\mathrm{E}[\int \mathrm{Z}\ \mathrm{dZ}]$	-0,000	0,000
$\mathrm{E}[(\int \mathrm{Z}\ \mathrm{dZ})^2]$	0,496	0,498
$\mathrm{E}[\int \mathrm{Z}^2\ \mathrm{ds}]$	0,498	0,498

```
OK — isometria de Itô
OK — a integral de Itô tem média zero
```

3.4 Quando "martingale" falha: as estratégias de dobrar

A integral de Itô com integrando sem controle é apenas um **martingale local**, e isso permite uma construção perturbadora. Considere

$$X_t = \int_0^t \frac{dZ_s}{\sqrt{1-s}}, \qquad t < 1. \tag{4.18}$$

Sua variância é $\int_0^t \frac{ds}{1-s} = -\ln(1-t)$, que tende a infinito quando $t \to 1$. X se comporta como um browniano num relógio acelerado, $s = -\ln(1-t)$. Um browniano atinge qualquer nível, cedo ou tarde, com probabilidade 1. Logo:

Comece com zero e pare ao atingir $+1$: antes de $t = 1$, você terá ganho 1 com certeza.

É o análogo contínuo da estratégia de dobrar a aposta. O problema está no caminho: o valor pode cair arbitrariamente antes de voltar. Isso exige crédito ilimitado.

A solução é restringir o conjunto de estratégias. **Admissível** (Def. 35 do livro) significa que o valor nunca cai abaixo de um limite $-\alpha$. Com esse limite, a estratégia para em $+1$ ou em $-\alpha$. Pelo teorema de parada de martingales, $\mathbb{E}[\text{ganho}] = 0$, ou seja,

$$1 \cdot P(+1) - \alpha\,(1 - P(+1)) = 0 \quad \Rightarrow \quad P(+1) = \frac{\alpha}{1+\alpha}. \tag{4.19}$$

O almoço grátis desaparece.

Laboratório 3.3 — Limites de crédito

Primeiro, a probabilidade de já ter alcançado $+1$ em cada data, sem limite. Pelo princípio da reflexão, para um browniano no relógio s: $P(\max \geq 1) = 2(1 - \Phi(1/\sqrt{s}))$. Depois, a simulação com limite de crédito.

```
linhas = []
for rotulo, falta in [("0,500", 0.5), ("0,100", 0.1), ("0,001", 1e-3), ("10⁻⁹", 1e-9), ("10⁻³⁰",
↪  1e-30)]:
    s = -np.log(falta)
    linhas.append({"Tempo restante 1 − t": rotulo, "Relógio s": float(s), "P(atingiu +1 até t)":
↪  float(2 * (1 - norm.cdf(1 / np.sqrt(s))))})
tabela(pd.DataFrame(linhas))

rng = np.random.default_rng(507)
ds = 0.002
linhas = []
for alfa in [1.0, 3.0, 9.0]:
    n = 40_000
    x = np.zeros(n)
    ativo = np.arange(n)
    ganho = np.zeros(n)
    while ativo.size:
        x[ativo] += np.sqrt(ds) * rng.standard_normal(ativo.size)
        cima, baixo = x[ativo] >= 1, x[ativo] <= -alfa
        ganho[ativo[cima]], ganho[ativo[baixo]] = 1.0, -alfa
        ativo = ativo[~(cima | baixo)]
    linhas.append({"Limite de crédito α": alfa, "P(+1): simulada": float((ganho > 0).mean()),
↪  "P(+1): teoria α/(1+α)": alfa / (1 + alfa), "Ganho médio": float(ganho.mean()),
↪  "Erro-padrão": float(ganho.std(ddof=1) / np.sqrt(n))})
lim = pd.DataFrame(linhas)
tabela(lim)
ok(np.allclose(lim["P(+1): simulada"], lim["P(+1): teoria α/(1+α)"], atol=0.012), "com limite de
↪  crédito, P(+1) = α/(1+α)")
ok((lim["Ganho médio"].abs() < 4 * lim["Erro-padrão"] + 0.02).all(), "com limite de crédito, o
↪  ganho médio é zero")
```

Tempo restante 1 − t	Relógio s	P(atingiu +1 até t)
0,500	0,693	0,230
0,100	2,303	0,510
0,001	6,908	0,704
10^{-9}	20,723	0,826
10^{-30}	69,078	0,904

Limite de crédito α	P(+1): simulada	P(+1): teoria $\alpha/(1+\alpha)$	Ganho médio	Erro-padrão
1,000	0,502	0,500	0,003	0,005
3,000	0,747	0,750	-0,014	0,009
9,000	0,898	0,900	-0,022	0,015

```
OK — com limite de crédito, P(+1) = α/(1+α)
OK — com limite de crédito, o ganho médio é zero
```

4. O lema de Itô

Livro: §5.6.1 e §9.7.1

4.1 De onde vem

Seja $f(S,t)$ suave e $dS_t = m_t\,dt + v_t\,dZ_t$. A expansão de Taylor de segunda ordem é

$$df = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S}dS + \frac{1}{2}\frac{\partial^2 f}{\partial S^2}(dS)^2 + \dots \tag{4.20}$$

Calculamos $(dS)^2 = m^2\,dt^2 + 2mv\,dt\,dZ + v^2\,(dZ)^2$. Usamos as **regras de multiplicação**, que resumem a Seção 2:

$\times$	dt	dZ
dt	0	0
dZ	0	dt

Por ordem de grandeza: dt^2 e $dt\,dZ \sim dt^{3/2}$ são desprezíveis; $(dZ)^2 = dt$ não. Sobra $(dS)^2 = v^2\,dt$, e resulta o **lema de Itô**:

$$\boxed{df = \Big(\frac{\partial f}{\partial t} + m\frac{\partial f}{\partial S} + \frac{1}{2}v^2\frac{\partial^2 f}{\partial S^2}\Big)dt + v\frac{\partial f}{\partial S}\,dZ.} \tag{4.21}$$

No cálculo comum, o termo de segunda ordem desaparece. Aqui, como $(dZ)^2$ tem ordem dt, ele **sobrevive**. Esse é o único ingrediente novo.

4.2 Exemplo 1: $f = Z^2$

Com $m = 0$, $v = 1$, $f' = 2Z$, $f'' = 2$:

$$d(Z^2) = 2Z\,dZ + dt \quad\Longrightarrow\quad \int_0^T Z\,dZ = \tfrac{1}{2}Z_T^2 - \tfrac{1}{2}T. \tag{4.22}$$

Foi o resultado que obtivemos da soma à esquerda.

4.3 Exemplo 2: resolver o browniano geométrico

Seja $dS = \mu S\,dt + \sigma S\,dZ$. Aplicamos Itô a $f = \ln S$, com $f' = 1/S$, $f'' = -1/S^2$, $m = \mu S$, $v = \sigma S$:

$$d\ln S = \Big(\mu S\cdot\frac{1}{S} - \frac{1}{2}\sigma^2 S^2\cdot\frac{1}{S^2}\Big)dt + \sigma S\cdot\frac{1}{S}\,dZ = (\mu - \tfrac{1}{2}\sigma^2)dt + \sigma\,dZ. \tag{4.23}$$

O lado direito não tem S, então a integração é imediata:

$$S_T = S_0\exp\Big[(\mu - \tfrac{1}{2}\sigma^2)T + \sigma Z_T\Big]. \tag{4.24}$$

Fica enfim justificado o termo $-\frac{1}{2}\sigma^2$ que usamos como caixa-preta desde o Módulo 1. Ele é o **termo de Itô**.

4.4 Versão multidimensional

Com vários processos, as regras ganham $dZ^1 dZ^2 = \rho\, dt$, e surgem termos cruzados. Para $f(X, Y)$:

$$df = f_X\, dX + f_Y\, dY + \tfrac{1}{2} f_{XX} (dX)^2 + \tfrac{1}{2} f_{YY} (dY)^2 + f_{XY}\, dX\, dY. \tag{4.25}$$

O caso $f = XY$ é a **regra do produto**: $d(XY) = X\, dY + Y\, dX + dX\, dY$.

Por que vale a atenção: a dinâmica do modelo geral (BCC97) tem índice, variância e juros simultaneamente, e no Módulo 11 precisaremos exatamente dessa versão.

Laboratório 4.1 — Conferindo o lema trajetória a trajetória

Com $f(z) = e^z$, temos $f' = f'' = e^z$. Comparamos três quantidades **em cada trajetória**: $f(Z_T) - f(0)$, a soma de Itô com correção e a soma sem correção.

```
rng = np.random.default_rng(508)
n_max, paths = 2**15, 1000
base = np.sqrt(1 / n_max) * rng.standard_normal((n_max, paths))
Z_ = np.vstack([np.zeros(paths), np.cumsum(base, axis=0)])

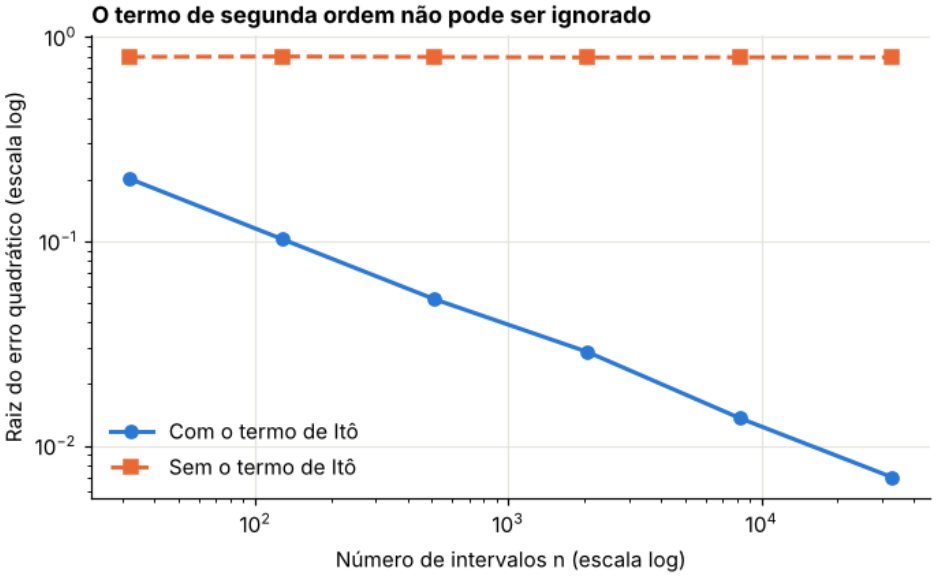
linhas = []
for n in [2**k for k in (5, 7, 9, 11, 13, 15)]:
    p = n_max // n
    z = Z_[::p]
    dz = np.diff(z, axis=0)
    fl = np.exp(z[:-1])
    com = (fl * dz).sum(axis=0) + 0.5 * fl.sum(axis=0) / n
    sem = (fl * dz).sum(axis=0)
    alvo = np.exp(z[-1]) - 1
    linhas.append({"n": n, "Erro com o termo de Itô": float(np.sqrt(np.mean((com - alvo) ** 2))),
↪   "Erro sem o termo de Itô": float(np.sqrt(np.mean((sem - alvo) ** 2)))})
ito = pd.DataFrame(linhas)
tabela(ito)
incl = np.polyfit(np.log(ito["n"]), np.log(ito["Erro com o termo de Itô"]), 1)[0]
print("Inclinação log-log do erro com correção:", f3(incl))

fig, ax = plt.subplots()
ax.loglog(ito["n"], ito["Erro com o termo de Itô"], color=AZUL, marker="o", markersize=6,
↪   label="Com o termo de Itô")
ax.loglog(ito["n"], ito["Erro sem o termo de Itô"], color=LARANJA, marker="s", markersize=6,
↪   ls="--", label="Sem o termo de Itô")
ax.set_title("O termo de segunda ordem não pode ser ignorado")
ax.set_xlabel("Número de intervalos n (escala log)")
ax.set_ylabel("Raiz do erro quadrático (escala log)")
ax.legend()
plt.show()

ok(-0.6 < incl < -0.4, "com o termo de Itô, o erro cai como 1/√n")
ok(ito["Erro sem o termo de Itô"].iloc[-1] > 0.5, "sem o termo de Itô, o erro não some")
del base, Z_
```

n	Erro com o termo de Itô	Erro sem o termo de Itô
32	0,202	0,797
128	0,102	0,801
512	0,052	0,796
2048	0,029	0,792
8192	0,014	0,794
32768	0,007	0,793

```
Inclinação log-log do erro com correção: -0,483
```




```
OK — com o termo de Itô, o erro cai como 1/√n
OK — sem o termo de Itô, o erro não some
```

Laboratório 4.2 — A solução exata contra o esquema de Euler, e a regra do produto

O esquema de Euler discretiza a equação diretamente: $S_{i+1} = S_i(1+\mu\Delta t+\sigma\Delta Z_i)$. Com os mesmos choques, comparamos com a solução exata. O erro trajetória a trajetória é o chamado **erro forte**.

Em seguida, a regra do produto para dois brownianos com correlação ρ: como $\mathbb{E}[d(XY)] = \mathbb{E}[dX\,dY] = \rho\,dt$, temos $\mathbb{E}[X_T Y_T] = \rho T$.

```
rng = np.random.default_rng(509)
mu, sigma, S0 = 0.08, 0.3, 100.0
n_max, paths = 2**12, 20_000
ZT = None
linhas = []
inc_base = np.sqrt(1 / n_max) * rng.standard_normal((n_max, paths))
exato = S0 * np.exp((mu - 0.5 * sigma**2) + sigma * inc_base.sum(axis=0))
```

```
for n in [4, 16, 64, 256, 1024, 4096]:
    p = n_max // n
    S = np.full(paths, S0)
    for i in range(n):
        S = S * (1 + mu / n + sigma * inc_base[i * p : (i + 1) * p].sum(axis=0))
    linhas.append({"Passos n": n, "Erro forte de Euler": float(np.sqrt(np.mean((S - exato) **
↪  2))), "Média Euler": float(S.mean())})
eul = pd.DataFrame(linhas)
tabela(eul)
inc_e = np.polyfit(np.log(eul["Passos n"].iloc[2:]), np.log(eul["Erro forte de Euler"].iloc[2:]),
↪  1)[0]
print("Inclinação do erro forte:", f3(inc_e), "| Média da solução exata nesta amostra:",
↪  f3(float(exato.mean())), "| E[S_T] teórico:", f3(S0 * np.exp(mu)))
ok(-0.62 < inc_e < -0.38, "Euler tem erro forte de ordem 1/√n")
del inc_base

rng = np.random.default_rng(510)
rho, n, paths = -0.6, 250, 400_000
X, Y, p1, p2 = np.zeros(paths), np.zeros(paths), np.zeros(paths), np.zeros(paths)
for _ in range(n):
    a = np.sqrt(1 / n) * rng.standard_normal(paths)
    b = rho * a + np.sqrt(1 - rho**2) * np.sqrt(1 / n) * rng.standard_normal(paths)
    p1 += X * b
    p2 += Y * a
    X += a
    Y += b
prod_ok = np.sqrt(np.mean((X * Y - (p1 + p2 + rho)) ** 2))
print("E[X·Y]:", f3(float(np.mean(X * Y))), "| ρ·T:", f3(rho), "| Raiz do erro da regra do
↪  produto:", f3(float(prod_ok)))
ok(abs(np.mean(X * Y) - rho) < 0.01, "E[X_T Y_T] = ρT")
ok(prod_ok < 0.08, "regra do produto: X_T Y_T ≈ ∫X dY + ∫Y dX + ρT")
```

Passos n	Erro forte de Euler	Média Euler
4	3,611	107,944
16	1,782	108,005
64	0,894	108,024
256	0,449	108,022
1024	0,227	108,026
4096	0,112	108,026

```
Inclinação do erro forte: -0,499 | Média da solução exata nesta amostra: 108,026 | E[S_T] teórico: 108,329
OK — Euler tem erro forte de ordem 1/√n
```

```
E[X·Y]: -0,602 | ρ·T: -0,600 | Raiz do erro da regra do produto: 0,074
OK — E[X_T Y_T] = ρT
OK — regra do produto: X_T Y_T ≈ ∫X dY + ∫Y dX + ρT
```

---

5. Saltos e a noção de semimartingale

Livro: §4.5, definições 30 a 34

5.1 Que processos podem ser preços

O ganho $\int \phi \, dS$ só está bem definido se S for um **semimartingale**. Felizmente, a classe é grande. A caracterização prática (Def. 32) é:

$$S_t = S_0 + \underbrace{M_t}_{\text{martingale local}} + \underbrace{A_t}_{\text{variação finita}}. \tag{4.26}$$

Tudo o que usaremos no curso cabe aqui. Para o browniano geométrico: em $\ln S_t$, $A_t = (\mu - \tfrac{1}{2}\sigma^2)t$ e $M_t = \sigma Z_t$.

5.2 Processo de Poisson

O processo de **Lévy** (Def. 34) generaliza o browniano mantendo incrementos independentes e estacionários. O exemplo que usaremos é o processo de Poisson N_t, que conta eventos que chegam à taxa λ:

$$P(N_t = k) = \frac{e^{-\lambda t}(\lambda t)^k}{k!}, \qquad \mathbb{E}[N_t] = \operatorname{Var}(N_t) = \lambda t. \tag{4.27}$$

N_t não é martingale, pois só sobe. Sua **versão compensada**, $N_t - \lambda t$, é.

5.3 Compensando os saltos

Se um salto multiplica S por e^J, com $\bar{\kappa} = \mathbb{E}[e^J - 1]$, o crescimento médio causado pelos saltos é $\lambda\bar{\kappa}$ por unidade de tempo. Para que o retorno total seja r, tiramos essa parcela do drift:

$$\frac{dS}{S_-} = (r - \lambda\bar{\kappa})\,dt + \sigma\,dZ + (e^J - 1)\,dN. \tag{4.28}$$

Essa é a origem do termo $\lambda\bar{\kappa}$ usado no Módulo 1. A regra $dN\,dZ = 0$ do livro diz que saltos e browniano não interagem. Em Itô, o salto entra pela diferença exata $f(S_- e^J) - f(S_-)$, e não pela derivada.

Laboratório 5.1 — A compensação

```
rng = np.random.default_rng(511)
r, sig, T = 0.03, 0.15, 1.0
lam, mu_j, delta_j = 1.5, -0.1, 0.12
n = 2_000_000
kappa = np.exp(mu_j + 0.5 * delta_j**2) - 1
N = rng.poisson(lam * T, n)
J = N * mu_j + np.sqrt(N) * delta_j * rng.standard_normal(n)
Zt = np.sqrt(T) * rng.standard_normal(n)
com = np.exp((r - lam * kappa - 0.5 * sig**2) * T + sig * Zt + J)
sem = np.exp((r - 0.5 * sig**2) * T + sig * Zt + J)

tabela(pd.DataFrame([
    {"Grandeza": "E[N_T]", "Simulado": float(N.mean()), "Teoria": lam * T},
    {"Grandeza": "Var[N_T]", "Simulado": float(N.var()), "Teoria": lam * T},
    {"Grandeza": "E[S_T/S_0] com compensação", "Simulado": float(com.mean()), "Teoria":
↪   float(np.exp(r * T))},
    {"Grandeza": "E[S_T/S_0] sem compensação", "Simulado": float(sem.mean()), "Teoria":
↪   float(np.exp((r + lam * kappa) * T))},
]))
```

```
ok(abs(N.mean() - lam * T) < 0.005 and abs(N.var() - lam * T) < 0.01, "Poisson tem média e
↪  variância iguais a λT")
ok(abs(com.mean() - np.exp(r * T)) < 0.002, "o termo λκ deixa o preço descontado em martingale")
```

Grandeza	Simulado	Teoria
E[N_T]	1,499	1,500
Var[N_T]	1,500	1,500
E[S_T/S_0] com compensação	1,030	1,030
E[S_T/S_0] sem compensação	0,902	0,902

```
OK — Poisson tem média e variância iguais a λT
OK — o termo λκ deixa o preço descontado em martingale
```

---

6. Mudança de medida: o teorema de Girsanov

Conteúdo além do livro; base da precificação neutra ao risco

No Módulo 3, Q se obtinha de P reponderando estados com $L = Q/P$. Em tempo contínuo, cada trajetória individual tem probabilidade zero, mas a ideia sobrevive: L é a **densidade** de Q em relação a P.

6.1 Uma única normal

Seja $Z \sim N(0,1)$ sob P. Queremos reponderar de modo que passe a ter média $-\theta$. A densidade-alvo sobre a original é

$$L(z) = \frac{\varphi(z+\theta)}{\varphi(z)} = \exp\left(-\theta z - \tfrac{1}{2}\theta^2\right). \tag{4.29}$$

(Basta expandir $(z+\theta)^2 = z^2 + 2\theta z + \theta^2$ no expoente.) Veja que a mudança de medida desloca a média **sem alterar a variância**.

6.2 Da árvore ao contínuo

Tome um passeio com passos $\pm\sqrt{\Delta t}$, com probabilidade $\frac{1}{2}$ em cada direção sob P. Sob Q, a probabilidade de alta é $\frac{1}{2}(1 - \theta\sqrt{\Delta t})$. O passo tem média

$$\tfrac{1}{2}(1-\theta\sqrt{\Delta t})\sqrt{\Delta t} - \tfrac{1}{2}(1+\theta\sqrt{\Delta t})\sqrt{\Delta t} = -\theta\Delta t. \tag{4.30}$$

Temos um drift de $-\theta$ por unidade de tempo, criado **apenas reponderando caminhos**. O peso de cada passo é $L_i = 1 - \theta\,\varepsilon_i\sqrt{\Delta t}$, com $\varepsilon_i = \pm 1$. Para a trajetória completa:

$$\ln L_T = \sum_i \ln\left(1 - \theta\varepsilon_i\sqrt{\Delta t}\right) \approx \sum_i \left(-\theta\varepsilon_i\sqrt{\Delta t} - \tfrac{1}{2}\theta^2\Delta t\right) = -\theta Z_T - \tfrac{1}{2}\theta^2 T. \tag{4.31}$$

O $-\frac{1}{2}\theta^2 T$ vem do termo de segunda ordem do logaritmo: a mesma origem do termo de Itô.

6.3 O teorema

Seja θ constante e defina em $[0,T]$ uma medida Q por

$$\frac{dQ}{dP} = L_T = \exp\big(-\theta Z_T - \tfrac{1}{2}\theta^2 T\big). \tag{4.32}$$

Então $Z_t^Q = Z_t + \theta t$ é um movimento browniano **sob** Q.

Prova, em passos. Basta mostrar que, para $s < t$, o incremento de Z^Q tem distribuição normal $N(0, t-s)$ e independe de $\mathcal{F}_s$, sob Q. Sejam $\Delta = t - s$ e $\Delta Z = Z_t - Z_s$.

Passo 1. Pela fórmula de Bayes do Módulo 3, usando $L_t = \mathbb{E}_t^P[L_T] = \exp(-\theta Z_t - \tfrac{1}{2}\theta^2 t)$:

$$\mathbb{E}_s^Q\big[e^{u(Z_t^Q - Z_s^Q)}\big] = \mathbb{E}_s^P\left[\frac{L_t}{L_s}\, e^{u(\Delta Z + \theta\Delta)}\right]. \tag{4.33}$$

Passo 2. Substitua $L_t/L_s = \exp(-\theta\Delta Z - \tfrac{1}{2}\theta^2\Delta)$ e agrupe os expoentes:

$$= \mathbb{E}_s^P\Big[\exp\big((u-\theta)\Delta Z\big)\Big]\exp\big((u\theta - \tfrac{1}{2}\theta^2)\Delta\big). \tag{4.34}$$

Passo 3. ΔZ é independente de $\mathcal{F}_s$ e normal sob P. Pela fórmula $\mathbb{E}[e^{aZ}] = e^{a^2/2}$ do Módulo 1:

$$= \exp\big(\tfrac{1}{2}(u-\theta)^2\Delta + (u\theta - \tfrac{1}{2}\theta^2)\Delta\big) = \exp\big(\tfrac{1}{2}u^2\Delta\big). \tag{4.35}$$

Passo 4. É a função geradora de momentos de uma $N(0,\Delta)$, sem dependência de $\mathcal{F}_s$. A distribuição fica determinada, e o incremento é independente do passado.

Em uma frase: mudar de medida muda o **drift**, mas não muda a **volatilidade**.

Laboratório 6.1 — A árvore converge para a densidade exponencial

```
rng = np.random.default_rng(512)
theta, T = 0.8, 1.0
linhas = []
for n in [4, 16, 64, 256, 1024, 4096]:
    dt = T / n
    N_up = rng.binomial(n, 0.5, 300_000)
    N_dw = n - N_up
    ZT_ = (N_up - N_dw) * np.sqrt(dt)
    arv = (1 - theta * np.sqrt(dt)) ** N_up * (1 + theta * np.sqrt(dt)) ** N_dw
    expo = np.exp(-theta * ZT_ - 0.5 * theta**2 * T)
    linhas.append({"Passos n": n, "Média do peso": float(arv.mean()), "Média ponderada de Z_T":
↪   float(np.mean(arv * ZT_)), "Teoria: −θT": -theta * T, "Erro do peso contra a exponencial":
↪   float(np.sqrt(np.mean((arv - expo) ** 2)))})
c = pd.DataFrame(linhas)
tabela(c)
ok(np.allclose(c["Média do peso"], 1, atol=0.01), "o peso tem média 1: é uma densidade")
ok(np.allclose(c["Média ponderada de Z_T"], -theta * T, atol=0.015), "a reponderação cria um
↪   drift −θ")
ok(c["Erro do peso contra a exponencial"].iloc[-1] < 0.05 and c["Erro do peso contra a
↪   exponencial"].iloc[-1] < c["Erro do peso contra a exponencial"].iloc[1], "o peso da árvore
↪   converge para a densidade exponencial")
```

Passos n	Média do peso	Média ponderada de Z_T	Teoria: $-\theta T$	Erro do peso contra a exponencial
4	1,000	-0,799	-0,800	0,065
16	1,000	-0,800	-0,800	0,019
64	1,001	-0,802	-0,800	0,005
256	0,998	-0,796	-0,800	0,001
1024	0,998	-0,793	-0,800	0,000
4096	1,001	-0,803	-0,800	0,000

```
OK — o peso tem média 1: é uma densidade
OK — a reponderação cria um drift −θ
OK — o peso da árvore converge para a densidade exponencial
```

Laboratório 6.2 — Mesmas trajetórias, outra medida

Simulamos Z sob P e calculamos a distribuição de $Z_T^Q = Z_T + \theta T$, sem peso e com peso L_T. Com peso, ela deve ser uma normal padrão.

Há um custo prático, relevante para Monte Carlo: quanto mais distante Q fica de P, mais a massa se concentra em poucas trajetórias. Como $\mathbb{E}^P[L^2] = e^{\theta^2 T}$, o **tamanho efetivo da amostra** é

$$\frac{\mathrm{ESS}}{n} = \frac{(\sum L_i)^2}{n \sum L_i^2} \approx e^{-\theta^2 T}. \tag{4.36}$$

```python
rng = np.random.default_rng(513)
n = 2_000_000
Z_P = rng.standard_normal(n)
L_ = np.exp(-theta * Z_P - 0.5 * theta**2)
Z_Q = Z_P + theta

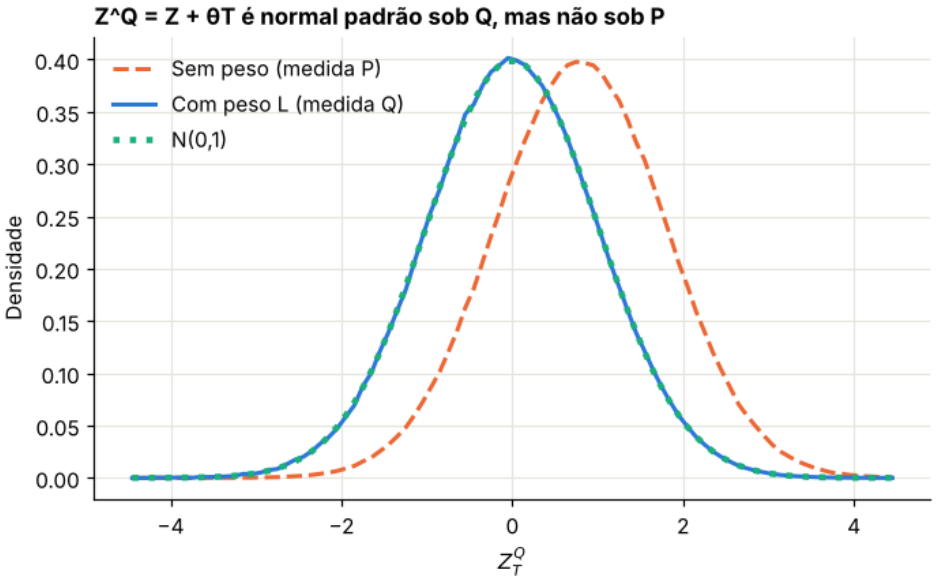
bins = np.linspace(-4.5, 4.5, 91)
h_sem, _ = np.histogram(Z_Q, bins=bins, density=True)
h_com, _ = np.histogram(Z_Q, bins=bins, weights=L_, density=True)
meio = 0.5 * (bins[1:] + bins[:-1])

fig, ax = plt.subplots()
ax.plot(meio, h_sem, color=LARANJA, ls="--", label="Sem peso (medida P)")
ax.plot(meio, h_com, color=AZUL, label="Com peso L (medida Q)")
ax.plot(meio, norm.pdf(meio), color=VERDE, ls=":", lw=3, label="N(0,1)")
ax.set_title("Z^Q = Z + θT é normal padrão sob Q, mas não sob P")
ax.set_xlabel("$Z^Q_T$")
ax.set_ylabel("Densidade")
ax.legend()
plt.show()

m1 = float(np.mean(L_ * Z_Q))
m2 = float(np.mean(L_ * Z_Q**2))
tabela(pd.DataFrame([
    {"Grandeza": "E^Q[Z^Q]", "Simulado": m1, "Teoria": 0.0},
    {"Grandeza": "E^Q[(Z^Q)²]", "Simulado": m2, "Teoria": 1.0},
]))
ok(abs(m1) < 0.005 and abs(m2 - 1) < 0.01, "sob Q, Z^Q tem média 0 e variância 1")

linhas = []
for th in [0.2, 0.5, 1.0, 2.0, 3.0]:
    L2 = np.exp(-th * Z_P - 0.5 * th**2)
    ess = float(L2.sum() ** 2 / (n * (L2**2).sum()))
    linhas.append({"θ√T": th, "ESS/n simulado": ess, "Teoria e^(−θ²T)": float(np.exp(-th**2))})
e_ = pd.DataFrame(linhas)
```

```
tabela(e_)
ok(np.allclose(e_["ESS/n simulado"].iloc[:4], e_["Teoria e^(-θ²T)"].iloc[:4], atol=0.01), "o
↪  tamanho efetivo da amostra segue e^(-θ²T)")
```



Grandeza	Simulado	Teoria
E^Q[Z^Q]	0,000	0,000
E^Q[(Z^Q)²]	1,001	1,000

```
OK — sob Q, Z^Q tem média 0 e variância 1
```

$\theta\sqrt{T}$	ESS/n simulado	Teoria e^$(-\theta^2 T)$
0,200	0,961	0,961
0,500	0,779	0,779
1,000	0,366	0,368
2,000	0,018	0,018
3,000	0,000	0,000

```
OK — o tamanho efetivo da amostra segue e^(-θ²T)
```

6.4 Aplicação ao mercado: o preço de mercado do risco

Sob P: $dS = \mu S\,dt + \sigma S\,dZ$. Definimos

$$\theta = \frac{\mu - r}{\sigma}, \tag{4.37}$$

o **preço de mercado do risco**: o excesso de retorno por unidade de volatilidade. Substituindo $dZ = dZ^Q - \theta\,dt$:

$$dS = \mu S\,dt + \sigma S\,(dZ^Q - \theta\,dt) = (\mu - \sigma\theta)S\,dt + \sigma S\,dZ^Q = rS\,dt + \sigma S\,dZ^Q. \quad (4.38)$$

Pronto: sob Q, o drift é r. Falta conferir que o preço descontado é martingale. Pela regra do produto, com $\beta_t = e^{-rt}$ e $d\beta = -r\beta\,dt$:

$$d(\beta S) = \beta\,dS - r\beta S\,dt = \sigma\,\beta S\,dZ^Q. \quad (4.39)$$

Não há termo em dt, logo é um martingale. **E μ desapareceu.** O mesmo resultado dos Módulos 1 e 3, agora em tempo contínuo.

6.5 Fórmula de precificação

Para um direito atingível, as equações (4.2) e (4.3) do livro dão

$$V_0 = \mathbb{E}^Q[\beta_T A_T], \qquad V_t = \beta_t^{-1}\,\mathbb{E}_t^Q[\beta_T A_T]. \quad (4.40)$$

O argumento do livro é igual ao do Módulo 3: $V_T = V_0 + \int \phi\,dS$, e a esperança da integral é zero.

> **Observação sobre o livro.** A passagem que justifica (4.1) afirma que a integral tem esperança zero "pela propriedade de martingale de S". Isso exige mais: como vimos na Seção 3.4, a integral de um integrando admissível é, em geral, apenas um martingale local, o que dá $\mathbb{E}^Q[V_T] \le V_0$. A igualdade vale quando V é um martingale verdadeiro, condição que se impõe às estratégias de replicação.

Laboratório 6.3 — Quatro formas de precificar uma call

Parâmetros: $S_0 = 100$, $K = 105$, $T = 1$, $r = 3\%$, $\mu = 12\%$, $\sigma = 20\%$.

```
S0, K, T, r, mu, sigma = 100.0, 105.0, 1.0, 0.03, 0.12, 0.20
theta = (mu - r) / sigma
rng = np.random.default_rng(514)
n = 2_000_000
Z_ = np.sqrt(T) * rng.standard_normal(n)
fator = np.exp(-r * T)

ST_P = S0 * np.exp((mu - 0.5 * sigma**2) * T + sigma * Z_)
ST_Q = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * Z_)
L_T = np.exp(-theta * Z_ - 0.5 * theta**2 * T)
pay_P = fator * np.maximum(ST_P - K, 0)
pay_Q = fator * np.maximum(ST_Q - K, 0)
pay_L = pay_P * L_T

referencia = float(bsm_call(S0, K, T, r, sigma))
linhas = []
for nome, a in [("Simular sob Q", pay_Q), ("Simular sob P e ponderar por L", pay_L), ("Simular
↪  sob P sem ponderar (errado)", pay_P)]:
    linhas.append({"Método": nome, "Valor": float(a.mean()), "Erro-padrão": float(a.std(ddof=1) /
↪  np.sqrt(n)), "Distância em erros-padrão": float((a.mean() - referencia) / (a.std(ddof=1) /
↪  np.sqrt(n)))})
linhas.append({"Método": "Fórmula BSM", "Valor": referencia, "Erro-padrão": 0.0, "Distância em
↪  erros-padrão": 0.0})
pr = pd.DataFrame(linhas).set_index("Método")
tabela(pr.reset_index())
```

```
print("θ =", f3(theta), "| E^Q[βS_T] por peso:", f3(float(np.mean(fator * ST_P * L_T))), "|
↪  E^P[βS_T]:", f3(float(np.mean(fator * ST_P))))

ok(abs(pr.loc["Simular sob Q", "Distância em erros-padrão"]) < 4, "simular sob Q reproduz BSM")
ok(abs(pr.loc["Simular sob P e ponderar por L", "Distância em erros-padrão"]) < 4, "simular sob P
↪  com peso de Radon-Nikodym reproduz BSM")
ok(abs(pr.loc["Simular sob P sem ponderar (errado)", "Distância em erros-padrão"]) > 20, "a média
↪  sob P, sem peso, não é o preço")
ok(abs(np.mean(fator * ST_P * L_T) - S0) < 0.05, "o preço descontado é martingale sob Q")
```

Método	Valor	Erro-padrão	Distância em erros-padrão
Simular sob Q	7,125	0,009	-0,371
Simular sob P e ponderar por L	7,127	0,005	-0,231
Simular sob P sem ponderar (errado)	12,699	0,012	469,697
Fórmula BSM	7,128	0,000	0,000

```
θ = 0,450 | E^Q[βS_T] por peso: 100,008 | E^P[βS_T]: 109,413
OK — simular sob Q reproduz BSM
OK — simular sob P com peso de Radon-Nikodym reproduz BSM
OK — a média sob P, sem peso, não é o preço
OK — o preço descontado é martingale sob Q
```

7. O Teorema Fundamental e a completude em tempo contínuo

Livro: §4.5, teoremas 2 e 3, Proposição 4

7.1 NA não basta: NFLVR

Em tempo contínuo, "não há arbitragem" (NA) não é suficiente para garantir uma medida martingale. Pode haver uma sequência de estratégias que, no limite, gera lucro sem risco, sem que nenhuma seja, individualmente, arbitragem. Delbaen e Schachermayer (1994) estabeleceram a condição certa, **NFLVR** (*no free lunch with vanishing risk*): nenhuma sequência de estratégias admissíveis aproxima um almoço grátis com risco que tende a zero.

Mundo discreto (Módulo 3)	Mundo contínuo
soma $\sum \phi_t \Delta S_t$	integral de Itô $\int \phi \, dS$
ϕ previsível	ϕ previsível e admissível
qualquer processo adaptado	semimartingale
não arbitragem (NA)	NFLVR
medida martingale	medida martingale **local** em geral
$L = Q/P$	densidade de Radon-Nikodym, Girsanov

Imprecisão do livro. O Teorema 3 afirma a existência de uma medida **martingale** para semimartingales em geral. O teorema de Delbaen e Schachermayer (1994) vale nessa forma para S limitado; para S localmente limitado, a medida é martingale **local**; no caso totalmente geral, é sigma-martingale (1998).

7.2 Completude: conte as fontes de risco

A regra prática: mercado completo ⇔ tantos ativos arriscados negociados quantas fontes independentes de risco.

Em Girsanov, cada browniano precisa de seu θ. Cada ativo negociado impõe uma equação $\theta = (\mu - r)/\sigma$. Sobrando browniano sem equação, θ fica livre.

Modelo	Fontes de risco	Ativos arriscados	Medida Q
BSM	1 browniano	1	Única
Merton (1976)	1 browniano + saltos	1	Infinitas
Heston (1993)	2 brownianos	1	Infinitas (preço do risco de volatilidade livre)
BCC97	3 brownianos + saltos	1	Infinitas

É o mesmo cenário do mercado trinomial do Módulo 3: os preços das opções negociadas selecionam a medida. Por isso, a calibração é o tema central da terceira parte do curso.

Laboratório 7.1 — Em Heston, várias medidas martingale e vários preços

Sob uma mudança de medida no segundo browniano com preço de risco proporcional à raiz da variância, os parâmetros da variância passam a ser $\kappa^* = \kappa + \lambda$ e $\theta^* = \kappa\theta/(\kappa + \lambda)$. **Toda escolha de λ mantém o preço descontado como martingale**, mas muda o preço da opção.

```
def heston_terminal(S0, r, v0, kappa, theta_v, nu, rho, T, passos, paths, rng):
    """Euler com truncamento total, guardando apenas o valor terminal."""
    dt = T / passos
    v = np.full(paths, v0)
    x = np.zeros(paths)
    for _ in range(passos):
        z1 = rng.standard_normal(paths)
        z2 = rho * z1 + np.sqrt(1 - rho**2) * rng.standard_normal(paths)
        vp = np.maximum(v, 0)
        x += (r - 0.5 * vp) * dt + np.sqrt(vp * dt) * z1
        v += kappa * (theta_v - vp) * dt + nu * np.sqrt(vp * dt) * z2
    return S0 * np.exp(x)


S0, K, T, r = 100.0, 100.0, 1.0, 0.03
v0, kappa, theta_v, nu, rho = 0.04, 2.0, 0.04, 0.4, -0.6
rng = np.random.default_rng(515)
paths = 300_000
linhas = []
for lam in [-1.0, 0.0, 1.0, 3.0]:
    ks = kappa + lam
    ts = kappa * theta_v / ks
    S_ = heston_terminal(S0, r, v0, ks, ts, nu, rho, T, 200, paths, rng)
    desc = np.exp(-r * T)
    m_s = desc * S_
```

```
    c_ = desc * np.maximum(S_ - K, 0)
    linhas.append({"λ": lam, "κ*": ks, "θ*": ts, "E^Q[βS_T]": float(m_s.mean()), "Erro-padrão":
↪   float(m_s.std(ddof=1) / np.sqrt(paths)), "Call ATM": float(c_.mean()), "Erro-padrão da call":
↪   float(c_.std(ddof=1) / np.sqrt(paths))})
h = pd.DataFrame(linhas)
tabela(h)
ok(((h["E^Q[βS_T]"] - S0).abs() < 4 * h["Erro-padrão"] + 0.02).all(), "todas as medidas mantêm o
↪   preço descontado como martingale")
ok(h["Call ATM"].max() - h["Call ATM"].min() > 10 * h["Erro-padrão da call"].max(), "o preço da
↪   call muda com a medida: mercado incompleto")
```

λ	κ*	θ*	E^Q[βS_T]	Erro-padrão	Call ATM	Erro-padrão da call
-1,000	1,000	0,080	100,008	0,040	10,269	0,024
0,000	2,000	0,040	100,001	0,035	9,098	0,021
1,000	3,000	0,027	100,009	0,031	8,293	0,019
3,000	5,000	0,016	99,979	0,025	7,165	0,016

```
OK — todas as medidas mantêm o preço descontado como martingale
OK — o preço da call muda com a medida: mercado incompleto
```

---

8. Síntese

- O movimento de duração Δt tem tamanho $\sqrt{\Delta t}$. Daí vêm a variação total infinita e $(dZ)^2 = dt$.
- A integral de Itô usa o ponto esquerdo, porque corresponde a uma posição decidida antes do movimento.
- O lema de Itô é Taylor com um termo a mais, e é ele que gera o $-\frac{1}{2}\sigma^2$.
- Sem restrição de crédito, estratégias de dobrar geram ganho certo em tempo contínuo; admissibilidade resolve o problema.
- Saltos entram no drift via compensação $\lambda\bar{\kappa}$.
- Girsanov muda o drift e não muda a volatilidade.
- μ desaparece: sob Q, o drift é r.
- Contando fontes de risco e ativos sabemos se Q é única; quando não é, os preços das opções a selecionam.

9. Notas sobre o livro neste trecho

Local	Comentário
Justificativa de (4.1)	a esperança da integral é zero só se V for martingale verdadeiro; admissibilidade garante apenas $\mathbb{E}^Q[V_T] \le V_0$
Teorema 3	a medida é martingale local (sigma-martingale, no caso mais geral), e não martingale, para S não limitado

Local	Comentário
Def. 37 (arbitragem contínua)	exige $V_T \geq 0$, corrigindo a Def. 25 do caso discreto
Geral	o teorema de Girsanov não é tratado; o livro passa direto à dinâmica sob Q

10. Exercícios

Exercício 1. Use o lema de Itô em $f = Z^3$ para expressar $\int_0^T Z^2\,dZ$.

Solução

$d(Z^3) = 3Z^2\,dZ + 3Z\,dt$. Logo, $\int_0^T Z^2\,dZ = \frac{1}{3}Z_T^3 - \int_0^T Z_s\,ds$.

Exercício 2. Para o browniano geométrico, use Itô em $f = S^2$ e obtenha $\mathbb{E}[S_T^2]$.

Solução

$d(S^2) = 2S\,dS + (dS)^2 = (2\mu + \sigma^2)S^2\,dt + 2\sigma S^2\,dZ$. Tomando esperanças e resolvendo a equação ordinária: $\mathbb{E}[S_T^2] = S_0^2\,e^{(2\mu+\sigma^2)T}$.

Exercício 3. Mostre que $E_t = \exp(\theta Z_t - \frac{1}{2}\theta^2 t)$ satisfaz $dE = \theta E\,dZ$.

Solução

Com $f(z,t) = e^{\theta z - \theta^2 t/2}$: $f_t = -\frac{1}{2}\theta^2 f$, $f_z = \theta f$, $f_{zz} = \theta^2 f$. Itô dá $df = (-\frac{1}{2}\theta^2 + \frac{1}{2}\theta^2)f\,dt + \theta f\,dZ = \theta f\,dZ$.

Exercício 4. Mostre que $\mathrm{Var}^P(L_T) = e^{\theta^2 T} - 1$.

Solução

$\mathbb{E}[L_T^2] = \mathbb{E}[e^{-2\theta Z_T - \theta^2 T}] = e^{-\theta^2 T}\,e^{2\theta^2 T} = e^{\theta^2 T}$. Como $\mathbb{E}[L_T] = 1$, a variância é $e^{\theta^2 T} - 1$.

Exercício 5. Por que a volatilidade estimada com dados reais pode ser usada, em princípio, sob Q, mas o retorno médio não?

Solução

Girsanov troca o drift e preserva a volatilidade: a variação quadrática é uma propriedade das trajetórias, igual sob medidas equivalentes. O drift, ao contrário, é substituído por r. No BSM, isso permite usar σ histórico; na prática, a volatilidade futura difere da passada, e o mercado cobra prêmio de risco de variância.

```
print(f"Módulo 4 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 4 concluído: 34 verificações aprovadas.
```

Parte II

Modelos e métodos de precificação

Capítulo 5

Black-Scholes-Merton: a EDP, a fórmula e as gregas

Introdução

O modelo de Black, Scholes e Merton, de 1973, é o ponto de referência de toda a teoria de derivativos. Ele supõe que o preço do ativo segue um movimento browniano geométrico, com volatilidade constante, e que se pode negociar continuamente sem custos.

Sob essas hipóteses, uma opção pode ser replicada de forma perfeita combinando o ativo e a conta remunerada, rebalanceando a carteira a cada instante. Dessa replicação sai uma **equação diferencial parcial** que o preço da opção precisa satisfazer, e nela a taxa de retorno esperada do ativo desaparece: o preço depende apenas da volatilidade. Resolvendo a equação, ou calculando a esperança neutra ao risco do Capítulo 4, obtém-se a **fórmula de Black-Scholes**.

As derivadas do preço em relação às variáveis do problema, as **gregas** (delta, gama, vega, theta e rô), medem as sensibilidades da opção e orientam o hedge. O capítulo deduz a equação, a fórmula e as gregas e verifica cada uma numericamente. Black-Scholes não é o modelo final deste livro, mas é a linguagem em que o mercado cota opções (pela volatilidade implícita) e o referencial contra o qual medimos os modelos mais ricos.

Fundamentos teóricos

Estratégias autofinanciadas e a EDP

Definição 5.1 (Estratégia autofinanciada). Uma estratégia (Δ_t, β_t) em ações e conta remunerada $B_t = e^{rt}$ tem valor $V_t = \Delta_t S_t + \beta_t B_t$. Ela é *autofinanciada* se $dV_t = \Delta_t\, dS_t + \beta_t\, dB_t$, isto é, se toda variação de valor vem de ganhos de mercado, sem aportes nem retiradas.

Teorema 5.2 (Black-Scholes-Merton; Black and Scholes, 1973; Merton, 1973). *Suponha $dS_t = \mu S_t\, dt + \sigma S_t\, dW_t$, taxa r constante e ausência de dividendos e custos. Seja $V(t,S) \in C^{1,2}$ o preço de um derivativo europeu de payoff $h(S_T)$. Se não há arbitragem,*

V satisfaz

$$\partial_t V + rS\,\partial_S V + \tfrac{1}{2}\sigma^2 S^2\,\partial_{SS} V - rV = 0, \qquad V(T,S) = h(S). \tag{5.1}$$

Demonstração. Monte a carteira $\Pi = V - \Delta S$ com $\Delta = \partial_S V$. Pela fórmula de Itô, $d\Pi = (\partial_t V + \frac{1}{2}\sigma^2 S^2 \partial_{SS} V)dt$, sem termo em dW: a carteira é localmente sem risco. Na ausência de arbitragem ela rende a taxa livre de risco, $d\Pi = r\Pi\,dt = r(V - S\partial_S V)\,dt$. Igualando as duas expressões obtém-se (5.1). Note que μ desaparece. ■

Teorema 5.3 (Feynman-Kac). *Sob condições de crescimento em h, a solução de* (5.1) *é*

$$V(t,S) = e^{-r(T-t)}\,\mathbb{E}^{\mathbb{Q}}\big[h(S_T)\,\big|\,S_t = S\big], \qquad dS_u = rS_u\,du + \sigma S_u\,d\widetilde{W}_u. \tag{5.2}$$

Esboço. Aplique Itô a $e^{-ru}V(u,S_u)$ sob $\mathbb{Q}$: a EDP anula o termo em du, e o processo é um martingal local; com as condições de crescimento ele é um martingal, e igualar seus valores em t e em T dá a fórmula. ■

A fórmula

Proposição 5.4 (Fórmula de Black-Scholes). *Para a call europeia de strike K e vencimento T, com $\tau = T - t$,*

$$C(t,S) = S\,N(d_1) - Ke^{-r\tau}N(d_2), \tag{5.3}$$

$$d_{1,2} = \frac{\ln(S/K) + (r \pm \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}, \tag{5.4}$$

e, pela paridade put-call, $P = Ke^{-r\tau}N(-d_2) - S\,N(-d_1)$.

Demonstração. Pelo Teorema 5.3, $C = e^{-r\tau}\mathbb{E}^{\mathbb{Q}}[(S_T - K)^+]$ com $\ln S_T \sim N(\,\ln S + (r - \frac{1}{2}\sigma^2)\tau, \sigma^2\tau)$. Escreva $\mathbb{E}[(S_T - K)^+] = \mathbb{E}[S_T \mathbf{1}_{S_T > K}] - K\,\mathbb{Q}(S_T > K)$. O segundo termo é $K\,N(d_2)$. No primeiro, a mudança de medida com densidade $S_T/\mathbb{E}[S_T]$ (o próprio ativo como numerário) desloca a média do log em $\sigma^2\tau$ e dá $Se^{r\tau}N(d_1)$. ■

Gregas

Proposição 5.5 (Sensibilidades da call). *Com ϕ a densidade normal padrão,*

$$\Delta = \partial_S C = N(d_1), \qquad \Gamma = \partial_{SS} C = \frac{\phi(d_1)}{S\sigma\sqrt{\tau}}, \tag{5.5}$$

$$\mathcal{V} = \partial_\sigma C = S\sqrt{\tau}\,\phi(d_1), \qquad \rho = \partial_r C = K\tau e^{-r\tau}N(d_2), \tag{5.6}$$

$$\Theta = \partial_t C = -\frac{S\phi(d_1)\sigma}{2\sqrt{\tau}} - rKe^{-r\tau}N(d_2). \tag{5.7}$$

A demonstração usa a identidade $S\,\phi(d_1) = Ke^{-r\tau}\phi(d_2)$, que elimina os termos que vêm de derivar d_1 e d_2. Substituindo as gregas em (5.1) obtém-se a relação $\Theta + \frac{1}{2}\sigma^2 S^2\Gamma + rS\Delta - rC = 0$: numa carteira com delta neutro, o ganho de *theta* paga a convexidade (*gamma*).

Observação 5.6 (Hipóteses). O teorema supõe negociação contínua, sem custos, volatilidade constante e trajetórias contínuas. Cada hipótese é relaxada em capítulos posteriores: saltos (Capítulo 10), volatilidade estocástica e juros estocásticos (Capítulo 11), hedge discreto (Capítulo 16). Para custos de transação, veja Leland (1985).

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** seções 5.1 a 5.3 do livro. A árvore binomial (§5.4) fica para o Módulo 6.

O que você vai conseguir fazer ao final

1. Deduzir a equação de Black-Scholes por replicação, sem supor neutralidade ao risco.
2. Deduzir a fórmula por dois caminhos independentes e ver por que eles coincidem.
3. Resolver a EDP numericamente, inclusive para payoffs sem fórmula.
4. Deduzir e usar as cinco gregas.
5. Explicar a relação entre gama, teta e volatilidade realizada.
6. Medir o que acontece quando se faz hedge com a volatilidade errada.

Onde estamos

Nos módulos anteriores, a fórmula BSM foi caixa-preta. Agora temos as ferramentas para abri-la: Itô, replicação e medida neutra ao risco.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import mpmath as mp
import sympy as sy
from scipy.linalg import solve_banded
from scipy.stats import norm
from IPython.display import display

RAMPA3 = ["#86b6ef", "#2a78d6", "#0d366b"]

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. O modelo

Livro: §5.2.1

Dois ativos negociados. O título sem risco cresce à taxa constante r:

$$dB_t = rB_t\,dt. \tag{5.8}$$

O índice segue um browniano geométrico. Partimos da dinâmica **real**, sob P, com retorno esperado μ:

$$dS_t = \mu S_t\,dt + \sigma S_t\,dZ_t. \tag{5.9}$$

> O livro já parte da dinâmica neutra ao risco, com r no lugar de μ. Preferimos começar com μ, porque assim a EDP mostra por si só que o retorno esperado não importa. Na dedução com r, esse resultado teria sido embutido na hipótese.

Hipóteses de mercado perfeito: negociação contínua, sem custos, sem restrições a posições, volatilidade e juros constantes, sem dividendos. Estamos num índice, e o tratamento de dividendos é o tema do Exercício 3.

---

2. A equação de Black-Scholes

Livro: §5.2.2

Queremos o valor de um direito cujo preço depende apenas de S e t, $V(S,t)$.

Passo 1: aplicar Itô a V

$$dV = \Big(V_t + \mu S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\Big)dt + \sigma S V_S\,dZ. \tag{5.10}$$

Passo 2: montar a carteira replicante

Compramos ϕ_t unidades do índice e ψ_t títulos, com valor $\Pi = \phi S + \psi B$. Pela condição de **autofinanciamento** (Módulo 3), a variação do valor vem só dos ganhos de mercado:

$$d\Pi = \phi\,dS + \psi\,dB = (\phi\mu S + \psi r B)dt + \phi\sigma S\,dZ. \tag{5.11}$$

Passo 3: igualar os termos aleatórios

Queremos $\Pi = V$ em todos os instantes. Para os coeficientes de dZ coincidirem:

$$\phi\,\sigma S = \sigma S\,V_S \quad\Longrightarrow\quad \boxed{\phi = V_S = \Delta.} \tag{5.12}$$

Passo 4: igualar os termos em dt

Substituindo $\phi = V_S$:

$$V_S \mu S + \psi r B = V_t + \mu S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}. \tag{5.13}$$

O termo $\mu S V_S$ aparece nos dois lados e se cancela. Sobra:

$$\psi r B = V_t + \tfrac{1}{2}\sigma^2 S^2 V_{SS}. \tag{5.14}$$

Passo 5: eliminar ψ

Como $\Pi = V$, temos $\psi B = V - SV_S$. Substituindo:

$$r(V - SV_S) = V_t + \tfrac{1}{2}\sigma^2 S^2 V_{SS}. \tag{5.15}$$

Rearrumando, chegamos à **equação de Black-Scholes**:

$$\boxed{V_t + rS\,V_S + \tfrac{1}{2}\sigma^2 S^2\,V_{SS} - rV = 0.} \tag{5.16}$$

Observe três pontos:

1. μ **sumiu**. O motivo é o cancelamento do Passo 4: o hedge elimina a exposição à tendência.
2. A equação vale para **qualquer** derivativo cujo valor dependa só de (S, t). O que distingue uma call de uma put é a condição final $V(S, T) = h(S)$.
3. O termo em S tem coeficiente r, não μ. Foi essa substituição que deu origem à ideia de precificação neutra ao risco.

   Errata importante do livro. Na passagem para a equação (5.6), o livro escreve $r(V - \partial V/\partial S)$ em vez de $r(V - S\,\partial V/\partial S)$, e a própria equação (5.6) está impressa sem o fator rS no primeiro termo, como $\frac{\partial V}{\partial S} + \ldots$. A forma correta é a do quadro acima. O Laboratório 2.1 mostra que a versão impressa não é satisfeita pela fórmula.

Observação sobre a dedução do livro. Ele usa a carteira $\Pi = V - \Delta S$ e trata Δ como constante no intervalo. Essa carteira não é autofinanciada, e a conta só funciona porque Δ é mantido fixo em $d\Pi$. O caminho por replicação, seguido acima, é o de Merton e dispensa esse cuidado.

2.1 Lendo a equação: gama e teta

Com $\Delta = V_S$, $\Gamma = V_{SS}$ e $\Theta = V_t$, temos

$$\Theta + \tfrac{1}{2}\sigma^2 S^2 \Gamma = r\,(V - S\Delta). \tag{5.17}$$

Com juros zero, $\Theta = -\frac{1}{2}\sigma^2 S^2 \Gamma$: **o que o titular ganha em convexidade, perde em passagem do tempo**. Veremos o significado disso em P&L na Seção 7.

2.2 Da EDP à esperança: Feynman-Kac

Aplique Itô a $e^{-rt}V(S_t, t)$ sob Q, em que $dS = rS\,dt + \sigma S\,dZ^Q$:

$$d(e^{-rt}V) = e^{-rt}(V_t + rSV_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS} - rV)dt + e^{-rt}\sigma S V_S\,dZ^Q. \tag{5.18}$$

O parêntese é a própria EDP, igual a zero. Sobra um martingale. Portanto $e^{-rt}V_t = \mathbb{E}_t^Q[e^{-rT}h(S_T)]$. **A EDP e a esperança são a mesma coisa, escrita de duas formas.** Este é o teorema de Feynman-Kac.

Laboratório 2.1 — A fórmula satisfaz a EDP?

Escrevemos a fórmula de forma simbólica, derivamos e avaliamos o resíduo com 50 dígitos em pontos aleatórios.

```
S_s, K_s, tau_s, r_s, sig_s = sy.symbols("S K tau r sigma", positive=True)
d1_s = (sy.log(S_s / K_s) + (r_s + sig_s**2 / 2) * tau_s) / (sig_s * sy.sqrt(tau_s))
d2_s = d1_s - sig_s * sy.sqrt(tau_s)
N_s = lambda x: (1 + sy.erf(x / sy.sqrt(2))) / 2
C_s = S_s * N_s(d1_s) - K_s * sy.exp(-r_s * tau_s) * N_s(d2_s)

# Em t, com tau = T - t: V_t = -V_tau
Vt, Vs, Vss = -sy.diff(C_s, tau_s), sy.diff(C_s, S_s), sy.diff(C_s, S_s, 2)
correta = Vt + r_s * S_s * Vs + sig_s**2 * S_s**2 * Vss / 2 - r_s * C_s
impressa = Vs + sig_s**2 * S_s**2 * Vss / 2 + Vt - r_s * C_s

rng = np.random.default_rng(601)
linhas = []
for _ in range(6):
    p = {S_s: rng.uniform(50, 150), K_s: rng.uniform(50, 150), tau_s: rng.uniform(0.1, 2), r_s:
↪   rng.uniform(0.01, 0.1), sig_s: rng.uniform(0.1, 0.5)}
    linhas.append({"S": float(p[S_s]), "K": float(p[K_s]), "Resíduo na EDP correta":
↪   abs(float(correta.evalf(50, subs=p))), "Resíduo em (5.6) como impressa":
↪   abs(float(impressa.evalf(50, subs=p)))})
res = pd.DataFrame(linhas)
tabela(res)
ok((res["Resíduo na EDP correta"] < 1e-30).all(), "a fórmula BSM satisfaz a EDP com 50 dígitos")
ok((res["Resíduo em (5.6) como impressa"] > 1e-3).all(), "a equação como impressa no livro não é
↪   satisfeita")
```

S	K	Resíduo na EDP correta	Resíduo em (5.6) como impressa
132,546	50,277	0,000	9,666
110,102	108,779	0,000	2,151
98,423	98,129	0,000	5,143
108,703	116,086	0,000	2,807
118,032	51,157	0,000	9,989
67,668	51,857	0,000	1,681

```
OK — a fórmula BSM satisfaz a EDP com 50 dígitos
OK — a equação como impressa no livro não é satisfeita
```

---

3. A fórmula, por dois caminhos

Livro: §5.2.3

Caminho A: integrando a esperança

Para uma call com prazo restante $\tau = T - t$, sob Q:

$$S_T = S \exp\left[(r - \tfrac{1}{2}\sigma^2)\tau + \sigma\sqrt{\tau}\, z\right], \quad z \sim N(0,1). \tag{5.19}$$

Passo 1: quando a call paga? $S_T > K$ equivale a $z > -d_2$, com

$$d_2 = \frac{\ln(S/K) + (r - \tfrac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}. \tag{5.20}$$

Passo 2: separar o valor em dois termos.

$$C = e^{-r\tau}\int_{-d_2}^{\infty}(S_T(z)-K)\varphi(z)\,dz = \underbrace{e^{-r\tau}\int_{-d_2}^{\infty}S_T(z)\varphi(z)\,dz}_{\text{I}} - \underbrace{Ke^{-r\tau}\int_{-d_2}^{\infty}\varphi(z)\,dz}_{\text{II}}. \tag{5.21}$$

Passo 3: termo II. Pela simetria da normal, $\int_{-d_2}^{\infty}\varphi = N(d_2)$. Logo, $\text{II} = Ke^{-r\tau}N(d_2)$.

Passo 4: termo I. Substituindo S_T, o fator $e^{-r\tau}$ cancela $e^{r\tau}$:

$$\text{I} = S\int_{-d_2}^{\infty}\exp\big(-\tfrac{1}{2}\sigma^2\tau+\sigma\sqrt{\tau}\,z-\tfrac{1}{2}z^2\big)\frac{dz}{\sqrt{2\pi}}. \tag{5.22}$$

Completando o quadrado, $-\frac{1}{2}\sigma^2\tau+\sigma\sqrt{\tau}z-\frac{1}{2}z^2 = -\frac{1}{2}(z-\sigma\sqrt{\tau})^2$. Com $w = z-\sigma\sqrt{\tau}$:

$$\text{I} = S\int_{-d_2-\sigma\sqrt{\tau}}^{\infty}\varphi(w)\,dw = S\,N(d_2+\sigma\sqrt{\tau}) = S\,N(d_1). \tag{5.23}$$

Resultado:

$$\boxed{C = S\,N(d_1) - Ke^{-r\tau}N(d_2), \qquad d_1 = d_2+\sigma\sqrt{\tau}.} \tag{5.24}$$

O significado de $N(d_1)$. No Passo 4, o que apareceu foi a densidade reponderada pelo fator $S_T/(Se^{r\tau})$. É uma mudança de medida de Girsanov: é a medida em que o **índice é o numerário**. Assim, $N(d_2)$ é a probabilidade de exercício sob Q, e $N(d_1)$ é a probabilidade de exercício sob a medida do índice.

Caminho B: resolvendo a EDP

Passo 1: variável de tempo restante. Com $\tau = T-t$, temos $V_t = -V_\tau$, e o problema vira uma condição **inicial** $V(S,0) = h(S)$.

Passo 2: tirar o desconto e o drift. Mude variáveis:

$$V(S,\tau) = e^{-r\tau}u(x,\tau), \qquad x = \ln S + (r-\tfrac{1}{2}\sigma^2)\tau. \tag{5.25}$$

Passo 3: calcular as derivadas, pela regra da cadeia:

$$V_\tau = e^{-r\tau}\big[-ru+u_\tau+(r-\tfrac{1}{2}\sigma^2)u_x\big], \quad V_S = e^{-r\tau}\frac{u_x}{S}, \quad V_{SS} = e^{-r\tau}\frac{u_{xx}-u_x}{S^2}. \tag{5.26}$$

Passo 4: substituir em $V_\tau = rSV_S + \frac{1}{2}\sigma^2S^2V_{SS} - rV$:

$$-ru+u_\tau+(r-\tfrac{1}{2}\sigma^2)u_x = ru_x+\tfrac{1}{2}\sigma^2(u_{xx}-u_x)-ru. \tag{5.27}$$

Tudo se cancela, exceto:

$$\boxed{u_\tau = \tfrac{1}{2}\sigma^2\,u_{xx}.} \tag{5.28}$$

É a **equação do calor**: a BSM é difusão pura depois da mudança de variáveis.

Passo 5: solução. A solução da equação do calor é a convolução da condição inicial com um núcleo gaussiano:

$$u(x,\tau)=\int_{-\infty}^{\infty} h(e^{y})\,\frac{1}{\sqrt{2\pi\sigma^{2}\tau}}\exp\Big[-\frac{(x-y)^{2}}{2\sigma^{2}\tau}\Big]dy. \tag{5.29}$$

Com $y = x+\sigma\sqrt{\tau}\,z$, tem-se $e^{y} = S\exp[(r-\frac{1}{2}\sigma^{2})\tau+\sigma\sqrt{\tau}z]$. **É exatamente a integral do Caminho A.** Duas teorias, uma só conta.

A put

Compre índice e put, e venda a call. Em T o resultado é K em qualquer cenário: se $S_T<K$, tem-se $S_T+(K-S_T)$; se $S_T\geq K$, tem-se $S_T-(S_T-K)$. Logo, $S+P-C=Ke^{-r\tau}$, de onde, usando $1-N(x)=N(-x)$,

$$P = Ke^{-r\tau}N(-d_2)-S\,N(-d_1). \tag{5.30}$$

Laboratório 3.1 — Verificações

A opção de referência do livro: $S_0=K=100$, $T=1$, $r=5\%$, $\sigma=20\%$.

```
def d12(S, K, tau, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * tau) / (sig * np.sqrt(tau))
    return d1, d1 - sig * np.sqrt(tau)


def bsm(S, K, tau, r, sig, tipo="call"):
    d1, d2 = d12(S, K, tau, r, sig)
    if tipo == "call":
        return S * norm.cdf(d1) - K * np.exp(-r * tau) * norm.cdf(d2)
    return K * np.exp(-r * tau) * norm.cdf(-d2) - S * norm.cdf(-d1)


S0, K0, T0, r0, s0 = 100.0, 100.0, 1.0, 0.05, 0.20
C0, P0 = float(bsm(S0, K0, T0, r0, s0)), float(bsm(S0, K0, T0, r0, s0, "put"))

rng = np.random.default_rng(602)
n = 4_000_000
z = rng.standard_normal(n)
ST = S0 * np.exp((r0 - 0.5 * s0**2) * T0 + s0 * np.sqrt(T0) * z)
amostra = np.exp(-r0 * T0) * np.maximum(ST - K0, 0)
ep = amostra.std(ddof=1) / np.sqrt(n)

d1, d2 = d12(S0, K0, T0, r0, s0)
peso = ST / (S0 * np.exp(r0 * T0))
tabela(pd.DataFrame([
    {"Grandeza": "Call pela fórmula", "Valor": C0},
    {"Grandeza": "Call por Monte Carlo", "Valor": float(amostra.mean())},
    {"Grandeza": "Put pela fórmula", "Valor": P0},
    {"Grandeza": "C − P", "Valor": C0 - P0},
    {"Grandeza": "S − K·exp(−rT)", "Valor": S0 - K0 * np.exp(-r0 * T0)},
    {"Grandeza": "N(d2)", "Valor": float(norm.cdf(d2))},
    {"Grandeza": "P(S_T>K) sob Q, simulada", "Valor": float((ST > K0).mean())},
    {"Grandeza": "N(d1)", "Valor": float(norm.cdf(d1))},
    {"Grandeza": "P(S_T>K) sob a medida do índice, simulada", "Valor": float(np.mean(peso * (ST >
 ↪  K0)))},
]))
ok(abs(C0 - 10.450584) < 1e-5 and abs(P0 - 5.573526) < 1e-5, "valores coincidem com a referência
 ↪  do livro (cerca de 10,4 e 5,6)")
ok(abs(amostra.mean() - C0) < 4 * ep, "Monte Carlo confere com a fórmula")
```

```
ok(abs((C0 - P0) - (S0 - K0 * np.exp(-r0 * T0))) < 1e-10, "paridade put-call")
ok(abs((ST > K0).mean() - norm.cdf(d2)) < 0.001, "N(d2) é a probabilidade de exercício sob Q")
ok(abs(np.mean(peso * (ST > K0)) - norm.cdf(d1)) < 0.001, "N(d1) é a probabilidade de exercício
↪  sob a medida do índice")
```

Grandeza	Valor
Call pela fórmula	10,451
Call por Monte Carlo	10,444
Put pela fórmula	5,574
C − P	4,877
S − K·exp(−rT)	4,877
N(d2)	0,560
P(S_T>K) sob Q, simulada	0,560
N(d1)	0,637
P(S_T>K) sob a medida do índice, simulada	0,637

```
OK — valores coincidem com a referência do livro (cerca de 10,4 e 5,6)
```

```
OK — Monte Carlo confere com a fórmula
OK — paridade put-call
OK — N(d2) é a probabilidade de exercício sob Q
OK — N(d1) é a probabilidade de exercício sob a medida do índice
```

Laboratório 3.2 — Dependência dos parâmetros

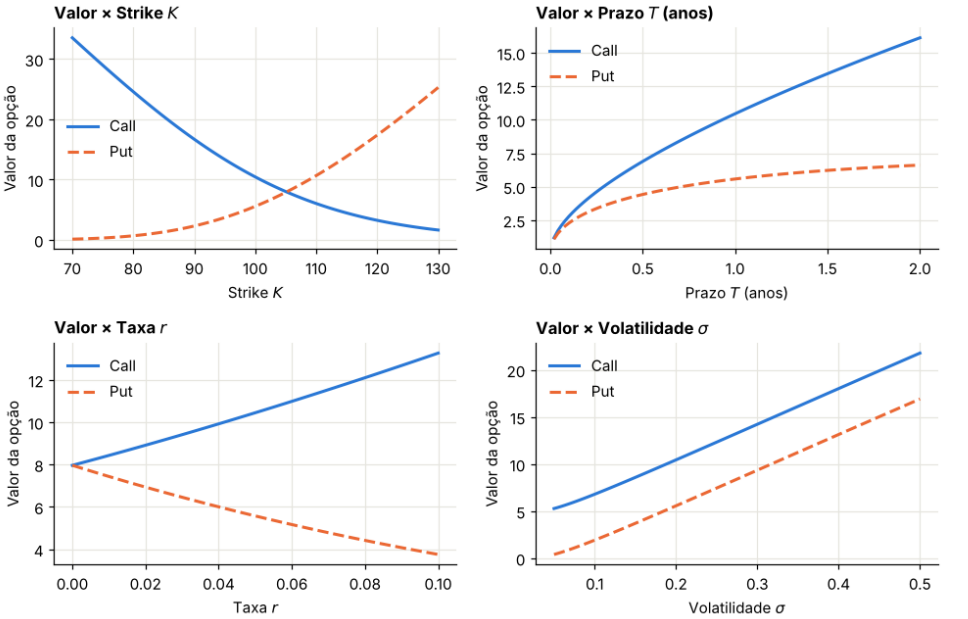
Reproduzimos as Figuras 5.1 e 5.2 do livro em um único painel. Note como o valor da put cai para prazos longos, quando dentro do dinheiro, e como o sinal do efeito dos juros difere entre call e put.

> **Errata menor:** no item 1 da lista do livro, "*$K > 100$ for the put, OTM*" deveria ser $K < 100$.

```
base = dict(S=S0, K=K0, tau=T0, r=r0, sig=s0)
grades = {
    "K": (np.linspace(70, 130, 121), "Strike $K$"),
    "tau": (np.linspace(0.02, 2.0, 100), "Prazo $T$ (anos)"),
    "r": (np.linspace(0.0, 0.10, 100), "Taxa $r$"),
    "sig": (np.linspace(0.05, 0.5, 100), "Volatilidade $\\sigma$"),
}
fig, eixos = plt.subplots(2, 2, figsize=(9, 6))
for ax, (par, (g, rot)) in zip(eixos.ravel(), grades.items()):
    c = np.array([float(bsm(**{**base, par: v})) for v in g])
    p = np.array([float(bsm(**{**base, par: v}, tipo="put")) for v in g])
    ax.plot(g, c, color=AZUL, label="Call")
    ax.plot(g, p, color=LARANJA, ls="--", label="Put")
    ax.set_title(f"Valor × {rot}")
    ax.set_xlabel(rot)
    ax.set_ylabel("Valor da opção")
    ax.legend()
plt.tight_layout()
plt.show()

prazos = np.linspace(0.05, 30, 300)
put_itm = bsm(50.0, 100.0, prazos, 0.05, 0.2, "put")
```

```
ok(put_itm[-1] < put_itm[0], "put europeia bem dentro do dinheiro pode perder valor com o prazo")
```




```
OK — put europeia bem dentro do dinheiro pode perder valor com o prazo
```

---

4. Resolvendo a EDP no computador

Conteúdo além do livro; útil para payoffs sem fórmula

4.1 Montagem em log-preço

Com $x = \ln S$ e prazo restante τ, a EDP vira

$$V_\tau = aV_{xx} + bV_x - rV, \qquad a = \tfrac{1}{2}\sigma^2, \quad b = r - \tfrac{1}{2}\sigma^2. \tag{5.31}$$

Numa malha com passo Δx, aproximamos as derivadas por diferenças centrais. Em cada ponto interior:

$$\mathcal{L}V_j = \ell V_{j-1} + cV_j + uV_{j+1}, \quad \ell = \frac{a}{\Delta x^2} - \frac{b}{2\Delta x}, \quad c = -\frac{2a}{\Delta x^2} - r, \quad u = \frac{a}{\Delta x^2} + \frac{b}{2\Delta x}. \tag{5.32}$$

4.2 Crank-Nicolson

A média entre os passos de tempo n e $n+1$ dá erro de segunda ordem:

$$\Big(I - \frac{\Delta\tau}{2}\mathcal{L}\Big)V^{n+1} = \Big(I + \frac{\Delta\tau}{2}\mathcal{L}\Big)V^{n}. \tag{5.33}$$

Em cada passo, resolve-se um sistema **tridiagonal**, de custo linear.

Cuidado: a quina do payoff. Crank-Nicolson não amortece oscilações de alta frequência, e a quina em K as gera. A solução usual, de **Rannacher**, troca os primeiros passos por passos implícitos, que suavizam a condição inicial.

Nas bordas, usamos o comportamento assintótico: para uma call, $V \to 0$ embaixo e $V \to S - Ke^{-r\tau}$ em cima.

Laboratório 4.1 — Convergência e gregas de graça

Colocamos o nó central exatamente em $\ln S_0$ e, ao dobrar a malha, medimos o erro. Em segunda ordem, o erro deve cair por um fator próximo de 4.

```
def cn(payoff, borda, S0, K, T, r, sig, J, nt, rannacher=4):
    """Crank-Nicolson em log-preço, com passos implícitos iniciais. Devolve malha e valores."""
    L = 6 * sig * np.sqrt(T) + abs(np.log(K / S0))
    dx = L / J
    x = np.log(S0) + dx * np.arange(-J, J + 1)
    Sg = np.exp(x)
    V = payoff(Sg)
    a, b = 0.5 * sig**2, r - 0.5 * sig**2
    l_, c_, u_ = a / dx**2 - b / (2 * dx), -2 * a / dx**2 - r, a / dx**2 + b / (2 * dx)
    dtau = T / nt
    passos = [(dtau / 2, 1.0)] * rannacher + [(dtau, 0.5)] * (nt - rannacher // 2)
    tau = 0.0
    for h, th in passos:
        tau_novo = tau + h
        lo, hi = borda(Sg[0], tau_novo), borda(Sg[-1], tau_novo)
        lo_v, hi_v = borda(Sg[0], tau), borda(Sg[-1], tau)
        interior = V[1:-1]
        rhs = interior + (1 - th) * h * (l_ * V[:-2] + c_ * interior + u_ * V[2:])
        rhs[0] += th * h * l_ * lo
        rhs[-1] += th * h * u_ * hi
        ab = np.zeros((3, interior.size))
        ab[0, 1:] = -th * h * u_
        ab[1, :] = 1 - th * h * c_
        ab[2, :-1] = -th * h * l_
        novo = solve_banded((1, 1), ab, rhs)
        V = np.r_[lo, novo, hi]
        tau = tau_novo
    return x, V, J, dx


pay_call = lambda S: np.maximum(S - K0, 0.0)
borda_call = lambda S, tau: 0.0 if S < K0 else S - K0 * np.exp(-r0 * tau)

linhas = []
for J in [100, 200, 400, 800, 1600]:
    x, V, c_, dx = cn(pay_call, borda_call, S0, K0, T0, r0, s0, J, J)
    linhas.append({"Malha J": J, "Valor numérico": float(V[c_]), "Erro": abs(float(V[c_]) - C0)})
conv = pd.DataFrame(linhas)
conv["Razão do erro"] = conv["Erro"].shift(1) / conv["Erro"]
tabela(conv)

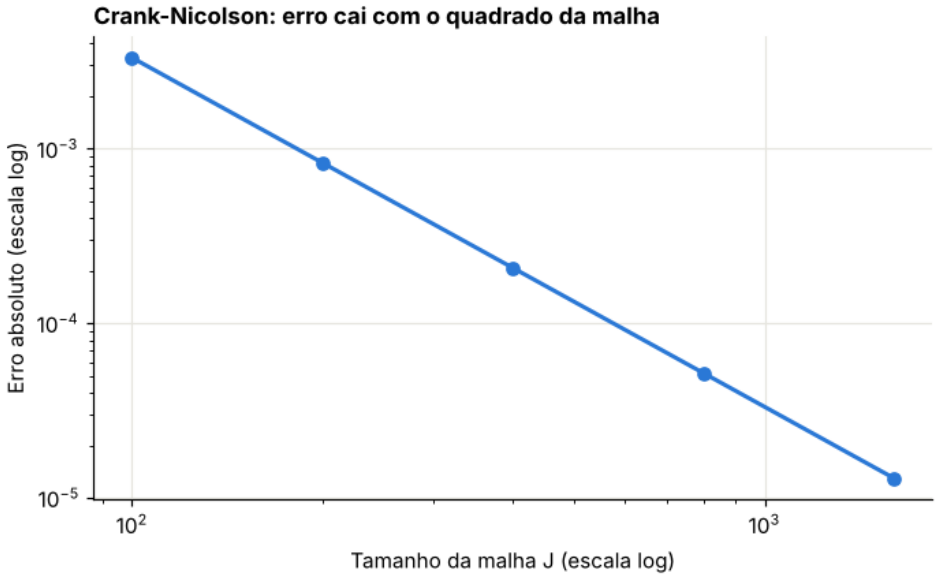
fig, ax = plt.subplots()
ax.loglog(conv["Malha J"], conv["Erro"], color=AZUL, marker="o", markersize=6)
ax.set_title("Crank-Nicolson: erro cai com o quadrado da malha")
ax.set_xlabel("Tamanho da malha J (escala log)")
ax.set_ylabel("Erro absoluto (escala log)")
```

```
plt.show()

ok(3.2 < conv["Razão do erro"].iloc[-1] < 4.8, "convergência de segunda ordem")
ok(conv["Erro"].iloc[-1] < 1e-3, "o erro final é menor que 0,001")

# Gregas a partir da malha
J = 1600
x, V, c_, dx = cn(pay_call, borda_call, S0, K0, T0, r0, s0, J, J)
Vx = (V[c_ + 1] - V[c_ - 1]) / (2 * dx)
Vxx = (V[c_ + 1] - 2 * V[c_] + V[c_ - 1]) / dx**2
delta_pde = Vx / S0
gama_pde = (Vxx - Vx) / S0**2
d1, _ = d12(S0, K0, T0, r0, s0)
tabela(pd.DataFrame([
    {"Grega": "Delta", "Malha da EDP": float(delta_pde), "Fórmula": float(norm.cdf(d1))},
    {"Grega": "Gama", "Malha da EDP": float(gama_pde), "Fórmula": float(norm.pdf(d1) / (S0 * s0 *
↪  np.sqrt(T0)))},
]))
ok(abs(delta_pde - norm.cdf(d1)) < 1e-4 and abs(gama_pde - norm.pdf(d1) / (S0 * s0 *
↪  np.sqrt(T0))) < 1e-4, "delta e gama saem da própria malha")
```

Malha J	Valor numérico	Erro	Razão do erro
100	10,447	0,003	n/d
200	10,450	0,001	4,002
400	10,450	0,000	4,001
800	10,451	0,000	4,000
1600	10,451	0,000	4,000




```
OK — convergência de segunda ordem
OK — o erro final é menor que 0,001
```

Grega	Malha da EDP	Fórmula
Delta	0,637	0,637
Gama	0,019	0,019

```
OK — delta e gama saem da própria malha
```

Laboratório 4.2 — Um payoff diferente

A mesma EDP resolve qualquer payoff que dependa apenas de S_T: basta trocar a condição final. Testamos com uma **digital** de 1 mês, que paga 1 se $S_T > K$, cuja solução é $e^{-r\tau}N(d_2)$. Usamos passos de tempo grandes de propósito, para expor o problema: em descontinuidades, é ainda mais importante amortecer os primeiros passos. A coluna conta quantas vezes a curva de valores muda de direção; o valor correto é monotônico.

```
pay_dig = lambda S: np.where(S > K0, 1.0, np.where(np.isclose(S, K0), 0.5, 0.0))
borda_dig = lambda S, tau: 0.0 if S < K0 else float(np.exp(-r0 * tau))
T_dig = 1 / 12
ref_dig = float(np.exp(-r0 * T_dig) * norm.cdf(d12(S0, K0, T_dig, r0, s0)[1]))

linhas = []
for ran, nome in [(0, "Crank-Nicolson puro"), (4, "Com passos implícitos iniciais")]:
    x, V, c_, dx = cn(pay_dig, borda_dig, S0, K0, T_dig, r0, s0, 800, 20, rannacher=ran)
    osc = int(np.sum(np.diff(np.sign(np.diff(V))) != 0))
    linhas.append({"Esquema": nome, "Valor": float(V[c_]), "Fórmula": ref_dig, "Erro":
↪   abs(float(V[c_]) - ref_dig), "Mudanças de direção na malha": osc})
d_ = pd.DataFrame(linhas).set_index("Esquema")
tabela(d_.reset_index())
ok(d_.loc["Com passos implícitos iniciais", "Erro"] < 2e-3, "o solver numérico reproduz a
↪   digital")
ok(d_.loc["Crank-Nicolson puro", "Mudanças de direção na malha"] > d_.loc["Com passos implícitos
↪   iniciais", "Mudanças de direção na malha"], "sem amortecimento, a solução oscila perto do
↪   strike")
ok(d_.loc["Crank-Nicolson puro", "Erro"] > 100 * d_.loc["Com passos implícitos iniciais",
↪   "Erro"], "sem amortecimento, o erro no strike é enorme")
```

Esquema	Valor	Fórmula	Erro	Mudanças de direção na malha
Crank-Nicolson puro	0,898	0,515	0,383	4
Com passos implícitos iniciais	0,517	0,515	0,001	0

```
OK — o solver numérico reproduz a digital
OK — sem amortecimento, a solução oscila perto do strike
OK — sem amortecimento, o erro no strike é enorme
```

---

5. As gregas

Livro: §5.3

As gregas medem a sensibilidade do valor aos parâmetros. Usamos a identidade do Módulo 2, que simplifica tudo:

$$S\,\varphi(d_1) = Ke^{-r\tau}\varphi(d_2). \tag{5.34}$$

Delta

$$\frac{\partial C}{\partial S} = N(d_1) + \left[S\varphi(d_1) - Ke^{-r\tau}\varphi(d_2)\right]\frac{\partial d_1}{\partial S} = N(d_1). \tag{5.35}$$

O colchete é zero pela identidade. (Usamos também $\partial d_2/\partial S = \partial d_1/\partial S$.) Para a put, pela paridade, $\Delta_P = \Delta_C - 1 = -N(-d_1)$.

Gama

$$\Gamma = \frac{\partial \Delta}{\partial S} = \varphi(d_1)\frac{\partial d_1}{\partial S} = \frac{\varphi(d_1)}{S\sigma\sqrt{\tau}}. \tag{5.36}$$

Pela paridade, é igual para call e put.

Vega

Como visto no Módulo 2: $\mathcal{V} = S\varphi(d_1)\sqrt{\tau}$. É igual para call e put.

Rô

$\partial C/\partial r$ recebe duas contribuições. O colchete com a identidade se cancela outra vez, e sobra a derivada do desconto: $\rho_C = K\tau e^{-r\tau}N(d_2)$. Para a put, $\rho_P = -K\tau e^{-r\tau}N(-d_2)$.

Teta

Em vez de derivar de novo, usamos a própria EDP: $\Theta = rC - rS\Delta - \frac{1}{2}\sigma^2 S^2\Gamma$. Substituindo:

$$\Theta_C = -\frac{S\varphi(d_1)\sigma}{2\sqrt{\tau}} - rKe^{-r\tau}N(d_2). \tag{5.37}$$

Para a put: $\Theta_P = -\dfrac{S\varphi(d_1)\sigma}{2\sqrt{\tau}} + rKe^{-r\tau}N(-d_2)$.

O teta é a variação do valor por unidade de tempo decorrido. Para uma call, é sempre negativo com $r \geq 0$. Para uma put bem dentro do dinheiro, pode ser positivo, resultado do valor no tempo negativo do Módulo 1.

Onde o gama é máximo

O senso comum diz que no dinheiro. Vamos conferir. Derivando $\ln\Gamma = \ln\varphi(d_1) - \ln S - \ln(\sigma\sqrt{\tau})$ em relação a S:

$$-d_1\frac{1}{S\sigma\sqrt{\tau}} - \frac{1}{S} = 0 \quad\Longrightarrow\quad d_1 = -\sigma\sqrt{\tau} \quad\Longrightarrow\quad S^* = K\exp\left[-(r + \tfrac{3}{2}\sigma^2)\tau\right]. \tag{5.38}$$

Próximo do strike para prazos curtos, mas abaixo dele para prazos longos.

Laboratório 5.1 — Fórmulas contra diferenças finitas

```
def gregas(S, K, tau, r, sig, tipo="call"):
    d1, d2 = d12(S, K, tau, r, sig)
    gama = norm.pdf(d1) / (S * sig * np.sqrt(tau))
    vega = S * norm.pdf(d1) * np.sqrt(tau)
    decaimento = -S * norm.pdf(d1) * sig / (2 * np.sqrt(tau))
    if tipo == "call":
        return {"delta": norm.cdf(d1), "gama": gama, "teta": decaimento - r * K * np.exp(-r * tau)
        ↪  * norm.cdf(d2), "ro": K * tau * np.exp(-r * tau) * norm.cdf(d2), "vega": vega}
    return {"delta": norm.cdf(d1) - 1, "gama": gama, "teta": decaimento + r * K * np.exp(-r * tau)
    ↪  * norm.cdf(-d2), "ro": -K * tau * np.exp(-r * tau) * norm.cdf(-d2), "vega": vega}


def dif_finita(S, K, tau, r, sig, tipo):
    f = lambda **k: float(bsm(**{**dict(S=S, K=K, tau=tau, r=r, sig=sig), **k}, tipo=tipo))
    h = 1e-4
    return {
        "delta": (f(S=S + h) - f(S=S - h)) / (2 * h),
        "gama": (f(S=S + 1e-2) - 2 * f() + f(S=S - 1e-2)) / 1e-4,
        "teta": -(f(tau=tau + h) - f(tau=tau - h)) / (2 * h),
        "ro": (f(r=r + h) - f(r=r - h)) / (2 * h),
        "vega": (f(sig=sig + h) - f(sig=sig - h)) / (2 * h),
    }


pontos = [(100.0, 100.0, 1.0), (90.0, 100.0, 0.25), (120.0, 100.0, 2.0)]
linhas = []
maior = 0.0
for tipo in ("call", "put"):
    for S_, K_, t_ in pontos:
        a, b = gregas(S_, K_, t_, r0, s0, tipo), dif_finita(S_, K_, t_, r0, s0, tipo)
        maior = max(maior, max(abs(float(a[k]) - b[k]) for k in a))
        linhas.append({"Tipo": tipo, "S": S_, "Prazo": t_, **{k: float(v) for k, v in a.items()}})
tabela(pd.DataFrame(linhas))
print("Maior diferença entre fórmula e diferenças finitas:", f3(maior))
ok(maior < 1e-3, "as fórmulas das gregas coincidem com as diferenças finitas")

d = gregas(S0, K0, T0, r0, s0)
ok(abs(float(d["teta"]) - (r0 * C0 - r0 * S0 * float(d["delta"]) - 0.5 * s0**2 * S0**2 *
↪  float(d["gama"]))) < 1e-10, "o teta é o que a EDP determina")
ok(float(gregas(40.0, 100.0, 1.0, 0.05, 0.2, "put")["teta"]) > 0, "put bem dentro do dinheiro tem
↪  teta positivo")

Sg = np.linspace(50, 160, 2201)
for tau_ in [1 / 12, 1.0, 2.0]:
    g_ = gregas(Sg, 100.0, tau_, 0.05, 0.3)["gama"]
    ok(abs(Sg[np.argmax(g_)] - 100.0 * np.exp(-(0.05 + 1.5 * 0.3**2) * tau_)) < 0.1, f"pico do
↪  gama em S* para prazo {f3(tau_)}")
```

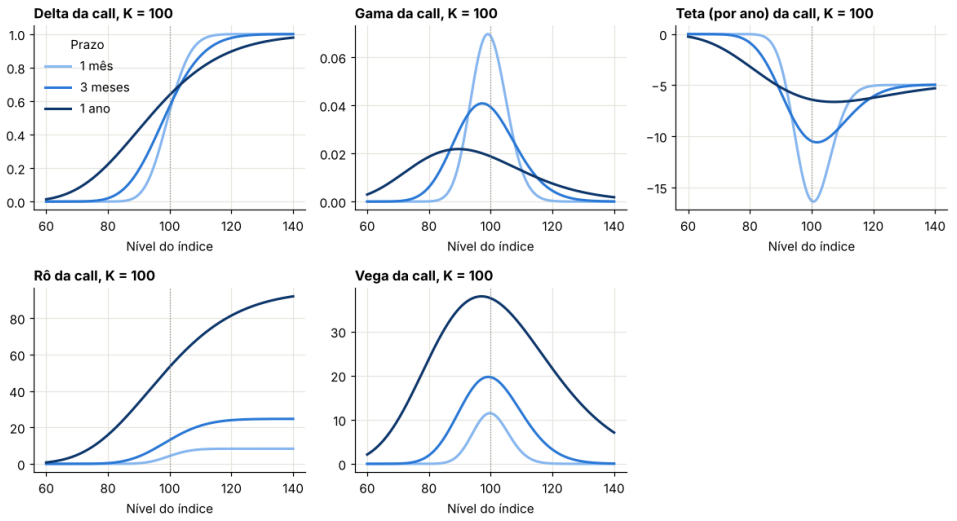
Tipo	S	Prazo	delta	gama	teta	ro	vega
call	100,000	1,000	0,637	0,019	-6,414	53,232	37,524
call	90,000	0,250	0,190	0,030	-5,691	4,046	12,204
call	120,000	2,000	0,873	0,006	-5,407	145,535	35,368
put	100,000	1,000	-0,363	0,019	-1,658	-41,890	37,524
put	90,000	0,250	-0,810	0,030	-0,753	-20,643	12,204
put	120,000	2,000	-0,127	0,006	-0,883	-35,433	35,368

```
Maior diferença entre fórmula e diferenças finitas: 0,000
OK — as fórmulas das gregas coincidem com as diferenças finitas
OK — o teta é o que a EDP determina
OK — put bem dentro do dinheiro tem teta positivo
OK — pico do gama em S* para prazo 0,083
OK — pico do gama em S* para prazo 1,000
OK — pico do gama em S* para prazo 2,000
```

Laboratório 5.2 — O formato das gregas

Em vez das superfícies 3D do livro, usamos fatias por prazo, para comparar os formatos na mesma escala de índice.

```python
Sg = np.linspace(60, 140, 321)
prazos_g = [(1 / 12, "1 mês"), (0.25, "3 meses"), (1.0, "1 ano")]
fig, eixos = plt.subplots(2, 3, figsize=(10.8, 6))
eixos = eixos.ravel()
nomes = {"delta": "Delta", "gama": "Gama", "teta": "Teta (por ano)", "ro": "Rô", "vega": "Vega"}
for ax, (k, rot) in zip(eixos, nomes.items()):
    for (t_, nome), cor in zip(prazos_g, RAMPA3):
        ax.plot(Sg, gregas(Sg, 100.0, t_, r0, s0)[k], color=cor, label=nome)
    ax.set_title(f"{rot} da call, K = 100")
    ax.set_xlabel("Nível do índice")
    ax.axvline(100, color=CINZA, lw=0.8, ls=":")
eixos[0].legend(title="Prazo")
eixos[5].axis("off")
plt.tight_layout()
plt.show()
```



Leitura. O gama e o teta se concentram perto do strike nos prazos curtos: são os riscos do vencimento próximo. O vega e o rô crescem com o prazo.

6. Aproximação por gregas

Uma expansão de Taylor em todas as variáveis, até segunda ordem nos termos relevantes, é:

$$\Delta V \approx \Delta\, dS + \tfrac{1}{2}\Gamma\, dS^2 + \mathcal{V}\, d\sigma + \tfrac{1}{2}\text{Volga}\, d\sigma^2 + \text{Vanna}\, dS\, d\sigma + \Theta\, dt + \rho\, dr. \tag{5.39}$$

As duas gregas cruzadas são

$$\text{Vanna} = \frac{\partial^2 V}{\partial S\, \partial \sigma} = -\varphi(d_1)\frac{d_2}{\sigma}, \qquad \text{Volga} = \frac{\partial^2 V}{\partial \sigma^2} = \mathcal{V}\,\frac{d_1 d_2}{\sigma}. \tag{5.40}$$

No Módulo 1, a aproximação linear errou feio no cenário de crash. Vamos ver o quanto cada termo recupera. A abordagem padronizada do FRTB segue a mesma lógica com delta, vega e curvatura; lá, a curvatura é calculada por reprecificação sob choque, justamente porque a expansão falha em movimentos grandes.

Laboratório 6.1 — Quanto cada termo explica

```
K_, S_, t_, r_, s_ = 8000.0, 8000.0, 1.0, 0.025, 0.20
d1, d2 = d12(S_, K_, t_, r_, s_)
g0 = {k: float(v) for k, v in gregas(S_, K_, t_, r_, s_).items()}
vanna = float(-norm.pdf(d1) * d2 / s_)
volga = float(g0["vega"] * d1 * d2 / s_)
v0 = float(bsm(S_, K_, t_, r_, s_))

cenarios = {"Índice −1%": (-80.0, 0.0), "Índice −10%": (-800.0, 0.0), "Vol +5 p.p.": (0.0, 0.05),
 ↪  "Crash: −20% e vol +15 p.p.": (-1600.0, 0.15)}
linhas = []
for nome, (dS, ds) in cenarios.items():
    completa = float(bsm(S_ + dS, K_, t_, r_, s_ + ds)) - v0
    primeira = g0["delta"] * dS + g0["vega"] * ds
    com_gama = primeira + 0.5 * g0["gama"] * dS**2
    completa2 = com_gama + 0.5 * volga * ds**2 + vanna * dS * ds
    linhas.append({"Cenário": nome, "Reprecificação": completa, "1ª ordem": primeira, "+ Gama":
 ↪  com_gama, "+ Volga e Vanna": completa2, "Erro final": completa2 - completa})
tay = pd.DataFrame(linhas).set_index("Cenário")
tabela(tay.reset_index())
nom, med = "Crash: −20% e vol +15 p.p.", "Índice −10%"
ok(abs(tay.loc[med, "Erro final"]) < 0.1 * abs(tay.loc[med, "1ª ordem"] - tay.loc[med,
 ↪  "Reprecificação"]), "em choque moderado, a segunda ordem recupera mais de 90% do erro")
ok(abs(tay.loc[nom, "Erro final"]) > 100, "no crash, a expansão continua errando muito")
```

Cenário	Reprecificação	1ª ordem	+ Gama	+ Volga e Vanna	Erro final
Índice −1%	-46,337	-47,121	-46,343	-46,343	-0,005
Índice −10%	-389,352	-471,208	-393,414	-393,414	-4,062
Vol +5 p.p.	155,621	155,588	155,588	155,698	0,077
Crash: −20% e vol +15 p.p.	-295,955	-475,652	-164,475	-151,821	144,133

```
OK — em choque moderado, a segunda ordem recupera mais de 90% do erro
OK — no crash, a expansão continua errando muito
```

Leitura. No choque de 10%, o gama resolve quase todo o erro. No crash, a expansão de segunda ordem troca um erro por outro: sai de uma perda exagerada para uma perda

subestimada. Um choque de 20% com 15 pontos de volatilidade está fora do raio em que Taylor funciona, e nem Volga e Vanna salvam. A conclusão prática é a do Módulo 1: **stress se faz com reprecificação completa**.

---

7. O que o delta hedge realmente faz

7.1 A carteira localmente sem risco

Numa pequena variação, a carteira $V - \Delta S$ não tem risco de primeira ordem, e sobra

$$dV - \Delta\, dS \approx \Theta\, dt + \tfrac{1}{2}\Gamma\,(dS)^2. \tag{5.41}$$

Em um passo, o desvio-padrão de um retorno não protegido escala com $\sqrt{\Delta t}$. O da carteira protegida escala com Δt.

7.2 Gama, teta e volatilidade realizada

Considere a posição comprada na opção, com hedge. O retorno em cada dia, em versão com $r = 0$, vale

$$\text{P\&L} \approx \Theta\, dt + \tfrac{1}{2}\Gamma S^2\Big(\frac{dS}{S}\Big)^2 = \tfrac{1}{2}\Gamma S^2\Big[\Big(\frac{dS}{S}\Big)^2 - \sigma^2\, dt\Big]. \tag{5.42}$$

O titular ganha se o quadrado do retorno superar a variância precificada, e perde caso contrário. Comprar opção com hedge é apostar na volatilidade realizada, contra a implícita.

7.3 Hedge com volatilidade errada

Compramos a call a uma implícita σ_i, e a volatilidade verdadeira é $\sigma_r > \sigma_i$. Há duas possibilidades de hedge:

- **Delta com σ_r:** o lucro é determinístico, e igual a $C(\sigma_r) - C(\sigma_i)$.
- **Delta com σ_i:** o lucro em valor presente é $\int_0^T e^{-rt}\tfrac{1}{2}\Gamma^i S^2(\sigma_r^2 - \sigma_i^2)\, dt$, uma quantidade **dependente da trajetória**: ela é maior se o índice ficar perto do strike, onde o gama é grande.

A teoria diz que acertar a volatilidade rende um lucro certo, enquanto usar a implícita rende um lucro aleatório. Mas esse lucro aleatório nunca fica negativo, porque o integrando é sempre positivo. Ao menos em tempo contínuo.

Laboratório 7.1 — Um passo

```
rng = np.random.default_rng(603)
n = 2_000_000
linhas = []
for dias in [16, 8, 4, 2, 1]:
    dt = dias / 252
    z = rng.standard_normal(n)
    S1 = S0 * np.exp((0.08 - 0.5 * s0**2) * dt + s0 * np.sqrt(dt) * z)
    dV = bsm(S1, K0, T0 - dt, r0, s0) - C0
```

```
    d_inic = float(gregas(S0, K0, T0, r0, s0)["delta"])
    prot = dV - d_inic * (S1 - S0)
    linhas.append({"Dias": dias, "Desvio da opção": dV.std(), "Desvio da opção protegida":
↪   prot.std()})
um = pd.DataFrame(linhas)
tabela(um)
inc_nao = np.polyfit(np.log(um["Dias"]), np.log(um["Desvio da opção"]), 1)[0]
inc_sim = np.polyfit(np.log(um["Dias"]), np.log(um["Desvio da opção protegida"]), 1)[0]
print("Inclinação sem hedge:", f3(inc_nao), "| Com hedge:", f3(inc_sim))
ok(abs(inc_nao - 0.5) < 0.03 and abs(inc_sim - 1.0) < 0.05, "sem hedge o risco escala com √Δt;
↪   com hedge, com Δt")
```

Dias	Desvio da opção	Desvio da opção protegida
16	3,277	0,339
8	2,293	0,169
4	1,613	0,084
2	1,137	0,042
1	0,803	0,021

```
Inclinação sem hedge: 0,507 | Com hedge: 1,002
OK — sem hedge o risco escala com √Δt; com hedge, com Δt
```

Laboratório 7.2 — Hedge com a volatilidade certa e com a errada

Compramos a call a $\sigma_i = 20\%$, mas o índice oscila a $\sigma_r = 30\%$, com $\mu = 12\%$. Hedge diário, em 40 mil trajetórias. Descontamos o P&L para a data 0.

```
def delta_call(S, K, tau, r, sig):
    return norm.cdf(d12(S, K, tau, r, sig)[0])


def hedge_comprado(S, K, T, r, sig_preco, sig_hedge):
    """P&L descontado de comprar a call a sig_preco e vender delta calculado com sig_hedge."""
    M = S.shape[0] - 1
    dt = T / M
    d0 = delta_call(S[0], K, T, r, sig_hedge)
    caixa = -float(bsm(S[0, 0], K, T, r, sig_preco)) + d0 * S[0]
    d_ant = d0
    for i in range(1, M):
        caixa *= np.exp(r * dt)
        novo = delta_call(S[i], K, T - i * dt, r, sig_hedge)
        caixa += (novo - d_ant) * S[i]
        d_ant = novo
    caixa *= np.exp(r * dt)
    final = caixa - d_ant * S[M] + np.maximum(S[M] - K, 0)
    return np.exp(-r * T) * final


rng = np.random.default_rng(604)
M, I = 252, 40_000
sig_i, sig_r, mu_r = 0.20, 0.30, 0.12
Z = rng.standard_normal((M, I))
S = S0 * np.exp(np.vstack([np.zeros(I), np.cumsum((mu_r - 0.5 * sig_r**2) / M + sig_r * np.sqrt(1
↪   / M) * Z, axis=0)]))
```

```
pnl_r = hedge_comprado(S, K0, T0, r0, sig_i, sig_r)
pnl_i = hedge_comprado(S, K0, T0, r0, sig_i, sig_i)
ref = float(bsm(S0, K0, T0, r0, sig_r) - bsm(S0, K0, T0, r0, sig_i))

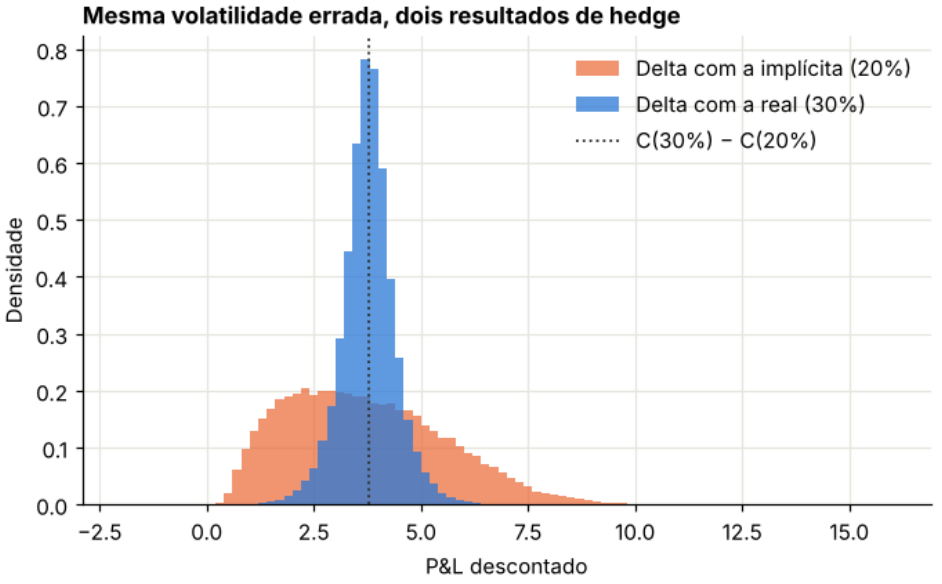
tabela(pd.DataFrame([
    {"Hedge": "Delta com a volatilidade real (30%)", "P&L médio": float(pnl_r.mean()),
↪  "Desvio-padrão": float(pnl_r.std(ddof=1)), "Pior caso": float(pnl_r.min()), "Melhor caso":
↪  float(pnl_r.max())},
    {"Hedge": "Delta com a volatilidade implícita (20%)", "P&L médio": float(pnl_i.mean()),
↪  "Desvio-padrão": float(pnl_i.std(ddof=1)), "Pior caso": float(pnl_i.min()), "Melhor caso":
↪  float(pnl_i.max())},
]))
print("Lucro teórico C(30%) − C(20%):", f3(ref))

faixa = (-2, 16)
fig, ax = plt.subplots()
ax.hist(pnl_i, bins=90, range=faixa, density=True, color=LARANJA, alpha=0.7, label="Delta com a
↪  implícita (20%)")
ax.hist(pnl_r, bins=90, range=faixa, density=True, color=AZUL, alpha=0.75, label="Delta com a
↪  real (30%)")
ax.axvline(ref, color="#333330", lw=1.2, ls=":", label="C(30%) − C(20%)")
ax.set_title("Mesma volatilidade errada, dois resultados de hedge")
ax.set_xlabel("P&L descontado")
ax.set_ylabel("Densidade")
ax.legend()
plt.show()

ok(abs(pnl_r.mean() - ref) < 0.05, "com a volatilidade real, o lucro médio é C(σ_r) − C(σ_i)")
ok(pnl_i.std() > 2 * pnl_r.std(), "com a volatilidade implícita, o lucro é bem mais incerto")
ok(np.mean(pnl_i > 0) > 0.95, "mesmo assim, o lucro com a implícita quase sempre é positivo")
```

Hedge	P&L médio	Desvio-padrão	Pior caso	Melhor caso
Delta com a volatilidade real (30%)	3,777	0,622	0,498	9,233
Delta com a volatilidade implícita (20%)	3,713	1,836	0,300	12,717

```
Lucro teórico C(30%) − C(20%): 3,781
```




```
OK — com a volatilidade real, o lucro médio é C(σ_r) − C(σ_i)
OK — com a volatilidade implícita, o lucro é bem mais incerto
OK — mesmo assim, o lucro com a implícita quase sempre é positivo
```

Leitura. Em tempo contínuo, o hedge com a volatilidade real daria lucro exatamente constante. O desvio que resta nessa linha vem do rebalanceamento diário, o erro de $1/\sqrt{N}$ do Módulo 1. A diferença entre as duas linhas é o efeito da dependência da trajetória: as duas têm médias próximas, mas o desvio é cerca de três vezes maior com a implícita.

7.4 Neutralizar o gama

O erro de rebalanceamento vem do gama. Para reduzi-lo, acrescenta-se **outra opção** à carteira: escolhemos a quantidade n de opção auxiliar de modo que o gama total seja zero e, depois, ajustamos o delta com o índice:

$$n = \frac{\Gamma_{\text{alvo}}}{\Gamma_{\text{aux}}}, \qquad \phi = \Delta_{\text{alvo}} - n\,\Delta_{\text{aux}}. \tag{5.43}$$

Laboratório 7.3 — Delta contra delta-gama, com rebalanceamento semanal

Vendemos uma call de 3 meses. A auxiliar é uma call de 1 ano, com o mesmo strike.

```python
def erro_hedge(S, K, T, r, sig, K_aux=None, T_aux=None):
    """Erro de hedge final de uma call vendida. Se K_aux é dado, faz delta-gama."""
    M = S.shape[0] - 1
    dt = T / M
    n_aux, phi = np.zeros(S.shape[1]), np.zeros(S.shape[1])
    caixa = np.full(S.shape[1], float(bsm(S[0, 0], K, T, r, sig)))
    for i in range(M):
```

```
        tau = T - i * dt
        s_i = S[i]
        ga, de = gregas(s_i, K, tau, r, sig)["gama"], gregas(s_i, K, tau, r, sig)["delta"]
        if K_aux is not None:
            aux = gregas(s_i, K_aux, T_aux - i * dt, r, sig)
            novo_n = ga / aux["gama"]
            novo_phi = de - novo_n * aux["delta"]
            caixa -= (novo_n - n_aux) * bsm(s_i, K_aux, T_aux - i * dt, r, sig)
            n_aux = novo_n
        else:
            novo_phi = de
        caixa -= (novo_phi - phi) * s_i
        phi = novo_phi
        caixa *= np.exp(r * dt)
    fim = caixa + phi * S[M] - np.maximum(S[M] - K, 0)
    if K_aux is not None:
        fim += n_aux * bsm(S[M], K_aux, T_aux - T, r, sig)
    return fim


rng = np.random.default_rng(605)
M, I = 13, 60_000
Z = rng.standard_normal((M, I))
S = S0 * np.exp(np.vstack([np.zeros(I), np.cumsum((0.1 - 0.5 * s0**2) * 0.25 / M + s0 *
 ↪  np.sqrt(0.25 / M) * Z, axis=0)]))
e_d = erro_hedge(S, K0, 0.25, r0, s0)
e_g = erro_hedge(S, K0, 0.25, r0, s0, K_aux=K0, T_aux=1.0)
tabela(pd.DataFrame([
    {"Hedge": "Só delta", "Erro médio": float(e_d.mean()), "Desvio-padrão":
 ↪  float(e_d.std(ddof=1))},
    {"Hedge": "Delta e gama", "Erro médio": float(e_g.mean()), "Desvio-padrão":
 ↪  float(e_g.std(ddof=1))},
]))
ok(e_g.std() < 0.5 * e_d.std(), "neutralizar o gama reduz o erro de hedge em mais da metade")
```

Hedge	Erro médio	Desvio-padrão
Só delta	-0,009	0,924
Delta e gama	-0,001	0,231

```
OK — neutralizar o gama reduz o erro de hedge em mais da metade
```

---

8. Síntese

- A EDP sai da replicação: o hedge com $\Delta = V_S$ cancela o aleatório e, junto, a tendência.
- A fórmula resulta de uma integral lognormal ou da equação do calor; é a mesma conta, e Feynman-Kac explica por quê.
- $N(d_2)$ é a probabilidade de exercício sob Q; $N(d_1)$, sob a medida do índice.
- O teta é determinado pela EDP a partir do delta e do gama.
- Gama e teta são dois lados da mesma moeda: comprar opção com hedge é apostar em volatilidade realizada contra implícita.
- Hedge com a volatilidade certa dá lucro certo; com a implícita, dá lucro aleatório.
- A EDP numérica estende o modelo para payoffs sem fórmula.

9. Erratas do livro neste trecho

Local	Problema
Passagem para (5.6)	$r(V - \partial V/\partial S)$ deveria ser $r(V - S\,\partial V/\partial S)$
Equação (5.6)	falta o fator rS no primeiro termo: o correto é $rS\,\partial V/\partial S$
§5.2.2	a carteira $\Pi = V - \Delta S$ não é autofinanciada; a dedução usa Δ fixo no intervalo
§5.2.3, item 1	"$K > 100$ for the put, OTM" deveria ser $K < 100$

10. Exercícios

Exercício 1. Mostre que $V = S$ e $V = e^{rt}$ satisfazem a EDP. O que eles representam?

Solução

$V = S$: $V_t = 0$, $V_S = 1$, $V_{SS} = 0$, e $rS - rS = 0$. $V = e^{rt}$: $V_t = re^{rt}$, e $re^{rt} - re^{rt} = 0$. São os dois ativos negociados.

Exercício 2. Sem refazer contas, deduza o Δ da carteira formada por comprar uma call e vender uma put, com o mesmo strike. Por que o resultado é tão simples?

Solução

$\Delta_C - \Delta_P = N(d_1) - (N(d_1) - 1) = 1$. Pela paridade, a carteira equivale a um termo: $S - Ke^{-r\tau}$, com delta 1.

Exercício 3. Se o índice paga dividendo contínuo com taxa y, a EDP troca r por $r - y$ no termo em V_S. Qual a fórmula da call?

Solução

$C = Se^{-y\tau}N(d_1) - Ke^{-r\tau}N(d_2)$, com $d_1 = \big[\ln(S/K) + (r - y + \frac{1}{2}\sigma^2)\tau\big]/(\sigma\sqrt{\tau})$. O índice é trocado pelo seu valor sem dividendos, $Se^{-y\tau}$.

Exercício 4. Deduza a condição para a qual o valor de uma call fica com Θ igual a $-rKe^{-r\tau}$ quando $\sigma \to 0$, com a opção dentro do dinheiro a termo.

Solução

Com $\sigma \to 0$ e $S > Ke^{-r\tau}$, temos $d_1, d_2 \to \infty$, $\varphi(d_1) \to 0$ e $N(d_2) \to 1$. Sobra $\Theta = -rKe^{-r\tau}$: a opção vale $S - Ke^{-r\tau}$, e só o desconto do strike muda com o tempo.

Exercício 5. Em `hedge_comprado`, faça o caso $\sigma_r < \sigma_i$ (por exemplo, compre a 30% num mundo a 20%). O que muda?

Solução

Os sinais se invertem: com o delta a σ_r, o prejuízo é quase determinístico e igual a $C(\sigma_r) - C(\sigma_i) < 0$. Com o delta a σ_i, o prejuízo é dependente de trajetória, em geral maior próximo do strike.

```
print(f"Módulo 5 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 5 concluído: 27 verificações aprovadas.
```

Capítulo 6

A árvore binomial de Cox-Ross-Rubinstein

Introdução

A árvore binomial de Cox, Ross e Rubinstein, de 1979, é a forma mais intuitiva de precificar opções: em cada período o preço do ativo só pode subir ou descer por fatores fixos, e o valor da opção é obtido de trás para a frente, nó a nó, com as probabilidades neutras ao risco do Capítulo 3.

Com os fatores de alta e baixa escolhidos para reproduzir a volatilidade, a árvore converge para Black-Scholes quando o número de passos cresce, e isso a torna uma ponte didática entre o modelo discreto e o contínuo. Mas sua vantagem prática é outra: ela lida naturalmente com o **exercício antecipado**. Em cada nó, compara-se o valor de continuar com o valor de exercer agora, e fica-se com o maior. É assim que se precificam opções americanas, que podem ser exercidas a qualquer momento até o vencimento.

O capítulo implementa a árvore, estuda sua convergência (inclusive as oscilações típicas do erro), extrai gregas e mostra a fronteira de exercício de uma put americana.

Fundamentos teóricos

A parametrização CRR e a convergência

Definição 6.1 (Árvore de Cox-Ross-Rubinstein). Divida $[0,T]$ em M passos de tamanho $\Delta t = T/M$ e tome

$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = 1/u, \qquad q = \frac{e^{r\Delta t} - d}{u - d}. \tag{6.1}$$

O preço no nó (m, j), com j altas em m passos, é $S_{m,j} = S_0 u^j d^{m-j} = S_0 u^{2j-m}$. Pela Proposição 3.5, a árvore é livre de arbitragem se $d < e^{r\Delta t} < u$, o que vale para Δt pequeno (Cox et al., 1979).

Proposição 6.2 (Moment matching). *Com a escolha acima, o log-retorno de um passo $X = \ln(S_{m+1}/S_m)$ satisfaz, sob $\mathbb{Q}$,*

$$\mathbb{E}[X] = (r - \tfrac{1}{2}\sigma^2)\Delta t + O(\Delta t^{3/2}), \qquad \operatorname{Var}(X) = \sigma^2 \Delta t + O(\Delta t^2). \tag{6.2}$$

Demonstração. $X = \pm\sigma\sqrt{\Delta t}$ com probabilidades q e $1-q$, logo $\mathbb{E}[X] = (2q-1)\sigma\sqrt{\Delta t}$ e $\mathrm{Var}(X) = 4q(1-q)\sigma^2\Delta t$. Expandindo $e^{r\Delta t}$ e $e^{\pm\sigma\sqrt{\Delta t}}$ até a ordem Δt, $q = \frac{1}{2} + \frac{(r-\sigma^2/2)}{2\sigma}\sqrt{\Delta t} + O(\Delta t)$, de onde seguem as duas expressões. ■

Teorema 6.3 (Convergência para Black-Scholes). *Para um payoff europeu h contínuo e de crescimento no máximo linear, o preço binomial* (3.4) *com a parametrização CRR converge para o preço de Black-Scholes quando $M \to \infty$.*

Esboço. $\ln(S_M/S_0)$ é soma de M variáveis i.i.d. com média e variância dadas pela proposição anterior. Pelo teorema central do limite de Lindeberg para arranjos triangulares, $\ln(S_M/S_0) \Rightarrow N((r-\frac{1}{2}\sigma^2)T, \sigma^2 T)$, que é a lei de $\ln(S_T/S_0)$ sob $\mathbb{Q}$ no modelo de Black-Scholes. A integrabilidade uniforme de $h(S_M)$ permite passar o limite para dentro da esperança. ■

Observação 6.4 (Ordem e oscilação). Para a call europeia, o erro é $O(1/M)$, mas não monótono: ele oscila conforme a posição do strike em relação aos nós finais. Médias de árvores com M e $M+1$ passos, ou o posicionamento de um nó no strike, suavizam a convergência.

Exercício antecipado

Definição 6.5 (Opção americana na árvore). Uma opção americana de payoff h pode ser exercida em qualquer nó. Seu valor é dado pela indução para trás

$$V_{M,j} = h(S_{M,j}), \qquad V_{m,j} = \max\Big\{h(S_{m,j}),\ e^{-r\Delta t}(qV_{m+1,j+1} + (1-q)V_{m+1,j})\Big\}. \quad (6.3)$$

Teorema 6.6 (Envelope de Snell). *$V_m = \max_\tau \mathbb{E}^{\mathbb{Q}}\big[e^{-r(\tau-m)\Delta t}h(S_\tau)\,\big|\,\mathcal{F}_m\big]$, onde o máximo é sobre os tempos de parada $\tau \in \{m,\dots,M\}$. O processo descontado $e^{-rm\Delta t}V_m$ é o menor supermartingal que domina o payoff descontado, e $\tau^* = \min\{m : V_m = h(S_m)\}$ é um tempo de parada ótimo.*

Proposição 6.7 (Call americana sem dividendos). *Se $r \geq 0$ e o ativo não paga dividendos, a call americana vale o mesmo que a europeia (Merton, 1973).*

Demonstração. Para $t < T$, $C_t^{\text{eur}} \geq S_t - Ke^{-r(T-t)} \geq S_t - K$, pela paridade put-call e por $P \geq 0$. Logo o valor de continuação sempre supera o de exercício, e nunca é ótimo exercer antes. ■

Para a put, o exercício antecipado tem valor: quando S é baixo, receber K hoje e aplicar a r vale mais que esperar. A região de exercício $\{S \leq S^*(t)\}$ é delimitada pela *fronteira de exercício* $S^*(t)$, crescente em t, com $S^*(T) = K$.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** seção 5.4 e o Apêndice A.2.2 do livro.

O que você vai conseguir fazer ao final

1. Calibrar uma árvore por moment matching, e ver surgir o termo $-\frac{1}{2}\sigma^2$.
2. Implementar a árvore de três formas e entender as diferenças de custo.
3. Mostrar que a árvore converge para a fórmula de BSM.
4. Explicar as oscilações do erro, e reduzi-las.
5. Extrair gregas da árvore sem reprecificar.
6. Precificar opções americanas e encontrar a fronteira de exercício.
7. Reconhecer a árvore como um método de diferenças finitas.

Onde estamos

No Módulo 3, construímos árvores que **não recombinam**, com 2^M estados. Isso permite qualquer produto dependente da trajetória, mas fica inviável rapidamente. A árvore CRR faz $u \cdot d = 1$ para que os caminhos se reencontrem: são apenas $M + 1$ nós na data final.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
from time import perf_counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, binom
from IPython.display import display

def d12(S, K, tau, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * tau) / (sig * np.sqrt(tau))
    return d1, d1 - sig * np.sqrt(tau)

def bsm(S, K, tau, r, sig, tipo="call"):
    d1, d2 = d12(S, K, tau, r, sig)
    if tipo == "call":
        return S * norm.cdf(d1) - K * np.exp(-r * tau) * norm.cdf(d2)
    return K * np.exp(-r * tau) * norm.cdf(-d2) - S * norm.cdf(-d1)

S0, K0, T0, r0, s0 = 100.0, 100.0, 1.0, 0.05, 0.20
C_BSM = float(bsm(S0, K0, T0, r0, s0))
print("Ambiente pronto. Call BSM de referência:", f3(C_BSM))
```

```
Ambiente pronto. Call BSM de referência: 10,451
```

---

1. O modelo e sua calibração

Livro: §5.4

1.1 A estrutura

Dividimos $[0, T]$ em M intervalos de $\Delta t = T/M$. Em cada passo, o índice é multiplicado por u (alta) ou d (queda). Sob Q, a probabilidade de alta é q. Como no Módulo 3, a condição de martingale do índice descontado fixa q:

$$q\,u + (1-q)\,d = e^{r\Delta t} \quad \Longrightarrow \quad q = \frac{e^{r\Delta t} - d}{u - d}. \tag{6.4}$$

1.2 Como escolher u e d

Queremos que a árvore tenha, em cada passo, a volatilidade certa. CRR escolhe

$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = \frac{1}{u}. \tag{6.5}$$

Vejamos por que isso funciona, usando log-retornos. O log-retorno do passo vale $\pm a$, com $a = \sigma\sqrt{\Delta t}$.

Variância do passo:

$$\mathrm{Var}^{Q} = (2a)^2\, q(1-q). \tag{6.6}$$

Expansão de q. Com Taylor, $u = 1 + a + \frac{a^2}{2} + \ldots$, $d = 1 - a + \frac{a^2}{2} - \ldots$ e $e^{r\Delta t} = 1 + r\Delta t + \ldots$:

$$q = \frac{a + r\Delta t - \frac{a^2}{2} + \ldots}{2a + \ldots} = \frac{1}{2} + \frac{r\Delta t - \frac{1}{2}\sigma^2\Delta t}{2\sigma\sqrt{\Delta t}} + \ldots \tag{6.7}$$

Consequências. Com $q \to \frac{1}{2}$, a variância tende a $4a^2 \cdot \frac{1}{4} = \sigma^2\Delta t$. E a média do log-retorno é

$$(2q-1)\,a = \frac{(r - \frac{1}{2}\sigma^2)\Delta t}{\sigma\sqrt{\Delta t}} \cdot \sigma\sqrt{\Delta t} = (r - \tfrac{1}{2}\sigma^2)\Delta t. \tag{6.8}$$

O termo $-\frac{1}{2}\sigma^2$ reaparece **sem que ninguém o tenha colocado**: vem apenas da condição de martingale, a mesma lógica de Itô.

1.3 Condição de não arbitragem

Precisamos de $0 < q < 1$, ou seja, $d < e^{r\Delta t} < u$. Com $u = e^{\sigma\sqrt{\Delta t}}$, isso exige $|r|\sqrt{\Delta t} < \sigma$:

$$M > \frac{r^2 T}{\sigma^2}. \tag{6.9}$$

Em juros altos e volatilidade baixa, uma árvore grosseira pode ter arbitragem, e essa condição não é detalhe: num cenário de 13% ao ano com volatilidade de 5%, são necessários mais de 6 passos para cada ano.

Laboratório 1.1 — Reproduzindo a Tabela 5.1 do livro

```
def params_crr(T, r, sig, M):
    dt = T / M
    u = np.exp(sig * np.sqrt(dt))
    d = 1 / u
```

```
    q = (np.exp(r * dt) - d) / (u - d)
    return dt, u, d, q


def arvore_completa(S0, K, T, r, sig, M):
    """Devolve as matrizes de índice e de valor, para inspeção de árvores pequenas."""
    dt, u, d, q = params_crr(T, r, sig, M)
    S = np.full((M + 1, M + 1), np.nan)
    V = np.full((M + 1, M + 1), np.nan)
    for t in range(M + 1):
        for j in range(t + 1):
            S[j, t] = S0 * u ** (t - j) * d**j
    V[:, M] = np.maximum(S[:, M] - K, 0)
    desc = np.exp(-r * dt)
    for t in range(M - 1, -1, -1):
        for j in range(t + 1):
            V[j, t] = desc * (q * V[j, t + 1] + (1 - q) * V[j + 1, t + 1])
    return S, V, q


Sa, Va, q4 = arvore_completa(S0, K0, T0, r0, s0, 4)
rot = ["t=0", "Δt", "2Δt", "3Δt", "T"]
print("Índice:")
tabela(pd.DataFrame(Sa, columns=rot))
print("Call:")
tabela(pd.DataFrame(Va, columns=rot))
print("q =", f3(float(q4)))

ok(abs(q4 - 0.5378) < 5e-4, "q coincide com o livro")
ok(abs(Va[0, 0] - 9.97) < 0.005 and abs(Va[0, 3] - 36.23) < 0.005 and abs(Sa[0, 4] - 149.18) <
↪  0.005, "valores coincidem com a Tabela 5.1")

linhas = []
for M in [4, 16, 64, 256, 1024]:
    dt, u, d, q = params_crr(T0, r0, s0, M)
    a = s0 * np.sqrt(dt)
    linhas.append({"M": M, "q": float(q), "Média do log-passo (×M)": float((2 * q - 1) * a * M),
↪  "Teoria (r−σ²/2)T": (r0 - 0.5 * s0**2) * T0, "Variância do log-passo (×M)": float(4 * a**2 *
↪  q * (1 - q) * M), "Teoria σ²T": s0**2 * T0})
mom = pd.DataFrame(linhas)
tabela(mom)
ok(abs(mom["Média do log-passo (×M)"].iloc[-1] - mom["Teoria (r−σ²/2)T"].iloc[-1]) < 1e-3, "o
↪  drift converge para r − σ²/2")
ok(abs(mom["Variância do log-passo (×M)"].iloc[-1] - s0**2 * T0) < 1e-3, "a variância converge
↪  para σ²T")

_, _, _, q_mau = params_crr(1.0, 0.13, 0.05, 4)
_, _, _, q_bom = params_crr(1.0, 0.13, 0.05, 12)
print("r=13%, σ=5%: q com M=4:", f3(float(q_mau)), "| com M=12:", f3(float(q_bom)))
ok(q_mau > 1 and 0 < q_bom < 1, "árvores grosseiras podem violar a condição de não arbitragem")
```

```
Índice:
```

t=0	Δt	2Δt	3Δt	T
100,000	110,517	122,140	134,986	149,182
n/d	90,484	100,000	110,517	122,140
n/d	n/d	81,873	90,484	100,000
n/d	n/d	n/d	74,082	81,873
n/d	n/d	n/d	n/d	67,032

```
Call:
```

t=0	Δt	2Δt	3Δt	T
9,971	15,922	24,609	36,228	49,182
n/d	3,317	6,246	11,759	22,140
n/d	n/d	0,000	0,000	0,000
n/d	n/d	n/d	0,000	0,000
n/d	n/d	n/d	n/d	0,000

```
q = 0,538
OK — q coincide com o livro
OK — valores coincidem com a Tabela 5.1
```

M	q	Média do log-passo (×M)	Teoria (r−σ²/2)T	Variância do log-passo (×M)	Teoria σ²T
4	0,538	0,030	0,030	0,040	0,040
16	0,519	0,030	0,030	0,040	0,040
64	0,509	0,030	0,030	0,040	0,040
256	0,505	0,030	0,030	0,040	0,040
1024	0,502	0,030	0,030	0,040	0,040

```
OK — o drift converge para r − σ²/2
OK — a variância converge para σ²T
r=13%, σ=5%: q com M=4: 1,154 | com M=12: 0,874
OK — árvores grosseiras podem violar a condição de não arbitragem
```

Errata do livro: a nota da Tabela 5.1 diz "the true value of the European **put** option is 10.45". A tabela é da **call**; a put vale 5,574.

---

2. Três implementações

Livro: Apêndice A.2.2

Há três formas equivalentes de obter o mesmo número, com custos bem diferentes.

A. Laços duplos. A forma literal do algoritmo. Custo de $M^2/2$ operações em Python puro.

B. Vetorizada. A cada passo para trás, uma única operação sobre o vetor de valores:

$$V^t = e^{-r\Delta t}\left[q\, V^{t+1}_{1:} + (1-q)\, V^{t+1}_{:-1}\right]. \tag{6.10}$$

Mesmo custo teórico, mas em C. Só é preciso guardar a última coluna: memória $O(M)$.

C. Soma binomial fechada. Para payoffs que só dependem de S_T, a árvore tem solução direta. O nó com j altas tem probabilidade binomial:

$$C_0 = e^{-rT} \sum_{j=0}^{M} \binom{M}{j} q^j (1-q)^{M-j} \max\left(S_0 u^j d^{M-j} - K, 0\right). \tag{6.11}$$

Custo $O(M)$. Não serve para americanas, nem para produtos dependentes da trajetória.

2.1 Fechando a conta

Seja a o menor j com $S_0 u^j d^{M-j} > K$. Separando o payoff em dois termos, como no Módulo 5:

$$C_0 = S_0 \, \mathcal{B}(a; M, q') - K e^{-rT} \mathcal{B}(a; M, q), \qquad q' = q\, u\, e^{-r\Delta t}, \tag{6.12}$$

em que $\mathcal{B}$ é a cauda superior da binomial. q' é a probabilidade de alta **sob a medida em que o índice é o numerário**, assim como $N(d_1)$. Pelo teorema central do limite, $\mathcal{B}(a; M, q) \to N(d_2)$ e $\mathcal{B}(a; M, q') \to N(d_1)$. Logo, a árvore tende à fórmula de BSM.

Laboratório 2.1 — Corrida de implementações

```python
def crr_laco(S0, K, T, r, sig, M):
    dt, u, d, q = params_crr(T, r, sig, M)
    desc = np.exp(-r * dt)
    V = [max(S0 * u**j * d ** (M - j) - K, 0.0) for j in range(M + 1)]
    for t in range(M - 1, -1, -1):
        for j in range(t + 1):
            V[j] = desc * (q * V[j + 1] + (1 - q) * V[j])
    return V[0]


def crr_vetor(S0, K, T, r, sig, M):
    dt, u, d, q = params_crr(T, r, sig, M)
    desc = np.exp(-r * dt)
    j = np.arange(M + 1)
    V = np.maximum(S0 * u**j * d ** (M - j) - K, 0.0)
    for _ in range(M):
        V = desc * (q * V[1:] + (1 - q) * V[:-1])
    return float(V[0])


def crr_fechada(S0, K, T, r, sig, M):
    dt, u, d, q = params_crr(T, r, sig, M)
    a = int(np.floor(np.log(K / (S0 * d**M)) / np.log(u / d))) + 1
    a = min(max(a, 0), M + 1)
    q_ = q * u * np.exp(-r * dt)
    return float(S0 * binom.sf(a - 1, M, q_) - K * np.exp(-r * T) * binom.sf(a - 1, M, q))


linhas = []
for M in [100, 500, 2000]:
    regs = {}
    for nome, f in [("Laços", crr_laco), ("Vetorizada", crr_vetor), ("Fechada", crr_fechada)]:
        t0 = perf_counter()
        v = f(S0, K0, T0, r0, s0, M)
```

```
        regs[nome] = (v, (perf_counter() - t0) * 1000)
    linhas.append({"M": M, "Valor": regs["Fechada"][0], "Laços (ms)": regs["Laços"][1],
↪   "Vetorizada (ms)": regs["Vetorizada"][1], "Fechada (ms)": regs["Fechada"][1]})
    ok(abs(regs["Laços"][0] - regs["Vetorizada"][0]) < 1e-9 and abs(regs["Fechada"][0] -
↪   regs["Vetorizada"][0]) < 1e-8, f"as três implementações coincidem para M = {M}")
corrida = pd.DataFrame(linhas)
tabela(corrida)
ok(corrida["Vetorizada (ms)"].iloc[-1] < corrida["Laços (ms)"].iloc[-1], "a vetorização é mais
↪   rápida que os laços")

d1, d2 = d12(S0, K0, T0, r0, s0)
linhas = []
for M in [10, 100, 1000, 10000]:
    dt, u, d, q = params_crr(T0, r0, s0, M)
    a = int(np.floor(np.log(K0 / (S0 * d**M)) / np.log(u / d))) + 1
    linhas.append({"M": M, "Cauda sob q": float(binom.sf(a - 1, M, q)), "N(d2)":
↪   float(norm.cdf(d2)), "Cauda sob q'": float(binom.sf(a - 1, M, q * u * np.exp(-r0 * dt))),
↪   "N(d1)": float(norm.cdf(d1))})
cau = pd.DataFrame(linhas)
tabela(cau)
ok(abs(cau["Cauda sob q"].iloc[-1] - norm.cdf(d2)) < 0.005 and abs(cau["Cauda sob q'"].iloc[-1] -
↪   norm.cdf(d1)) < 0.005, "as caudas binomiais convergem para N(d2) e N(d1)")
```

```
OK — as três implementações coincidem para M = 100
OK — as três implementações coincidem para M = 500
```

```
OK — as três implementações coincidem para M = 2000
```

M	Valor	Laços (ms)	Vetorizada (ms)	Fechada (ms)
100	10,431	1,233	0,368	0,307
500	10,447	24,746	1,246	0,218
2000	10,450	419,164	6,405	0,280

```
OK — a vetorização é mais rápida que os laços
```

M	Cauda sob q	N(d2)	Cauda sob q’	N(d1)
10	0,437	0,560	0,518	0,637
100	0,599	0,560	0,674	0,637
1000	0,547	0,560	0,625	0,637
10000	0,564	0,560	0,641	0,637

```
OK — as caudas binomiais convergem para N(d2) e N(d1)
```

---

3. Convergência e seus padrões

Livro: §5.4 e Figuras 5.8 e 5.9

3.1 Por que o erro oscila

O valor da árvore é uma soma sobre nós. Se K cair sobre um nó ou entre dois, o resultado muda. Variar M desloca os nós em relação a K, e o erro oscila.

- Com $K = S_0$, um nó central só existe se M for **par**. Com M ímpar, K cai no meio de dois nós.
- É isso que aparece nas figuras do livro: usando passos de 20 em 20, todo M é par e o valor parece vir por baixo. Com passos de 25 em 25, M alterna entre par e ímpar e o valor oscila.

  Sobre o texto do livro. Ele atribui a oscilação a uma "parametrização numérica diferente". Não há outra parametrização: é o mesmo modelo, e a diferença é apenas a paridade de M em relação à posição do strike.

Para K fora do dinheiro, o padrão é mais complicado, com um **dente de serra** irregular.

3.2 Três melhorias

Média de dois vizinhos. Com M e $M+1$, os erros têm sinais opostos no ATM, e a média cancela boa parte.

Suavização e Richardson (BBSR). O payoff tem quina, e é ela que causa as oscilações. Broadie e Detemple propõem trocar o último passo pelo valor BSM em Δt. A quina desaparece e o erro fica liso, o que permite a extrapolação de Richardson. Se o erro é c/M:

$$V_M = V + \frac{c}{M}, \quad V_{M/2} = V + \frac{2c}{M} \quad \Longrightarrow \quad V = 2V_M - V_{M/2}. \tag{6.13}$$

Leisen-Reimer. Em vez de casar momentos, escolhe-se q e q' de modo que as caudas binomiais aproximem $N(d_2)$ e $N(d_1)$ diretamente, pela inversão de Peizer-Pratt, com M ímpar:

$$h(z) = \frac{1}{2} + \frac{\operatorname{sinal}(z)}{2}\sqrt{1 - \exp\left[-\left(\frac{z}{M+\frac{1}{3}+\frac{0{,}1}{M+1}}\right)^2 (M+\tfrac{1}{6})\right]}, \tag{6.14}$$

com $q = h(d_2)$, $q' = h(d_1)$, $u = e^{r\Delta t}q'/q$ e $d = (e^{r\Delta t} - qu)/(1-q)$.

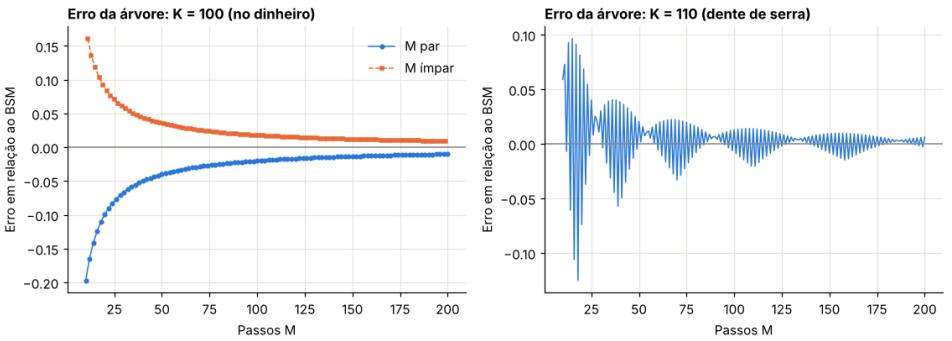
Laboratório 3.1 — O padrão do erro

```
Ms = np.arange(10, 201)
err_atm = np.array([crr_fechada(S0, 100.0, T0, r0, s0, int(M)) for M in Ms]) - float(bsm(S0,
↪  100.0, T0, r0, s0))
err_otm = np.array([crr_fechada(S0, 110.0, T0, r0, s0, int(M)) for M in Ms]) - float(bsm(S0,
↪  110.0, T0, r0, s0))

fig, (a1, a2) = plt.subplots(1, 2, figsize=(10.5, 3.9))
a1.plot(Ms[Ms % 2 == 0], err_atm[Ms % 2 == 0], color=AZUL, marker="o", markersize=3, lw=1,
↪  label="M par")
a1.plot(Ms[Ms % 2 == 1], err_atm[Ms % 2 == 1], color=LARANJA, marker="s", markersize=3, lw=1,
↪  ls="--", label="M ímpar")
a1.axhline(0, color=CINZA, lw=1)
a1.set_title("Erro da árvore: K = 100 (no dinheiro)")
a1.set_xlabel("Passos M")
```

```
a1.set_ylabel("Erro em relação ao BSM")
a1.legend()
a2.plot(Ms, err_otm, color=AZUL, lw=1)
a2.axhline(0, color=CINZA, lw=1)
a2.set_title("Erro da árvore: K = 110 (dente de serra)")
a2.set_xlabel("Passos M")
a2.set_ylabel("Erro em relação ao BSM")
plt.tight_layout()
plt.show()

ok(np.all(err_atm[Ms % 2 == 0] < 0), "com M par e K no dinheiro, a árvore fica abaixo do BSM")
ok(np.all(err_atm[Ms % 2 == 1] > 0), "com M ímpar e K no dinheiro, a árvore fica acima do BSM")
trocas = int(np.sum(np.diff(np.sign(err_otm)) != 0))
print("Trocas de sinal do erro para K = 110:", trocas)
ok(trocas > 5, "fora do dinheiro, o erro troca de sinal irregularmente")
```




```
OK — com M par e K no dinheiro, a árvore fica abaixo do BSM
OK — com M ímpar e K no dinheiro, a árvore fica acima do BSM
Trocas de sinal do erro para K = 110: 140
OK — fora do dinheiro, o erro troca de sinal irregularmente
```

Laboratório 3.2 — Comparando os métodos

```
def recursao(V, u, d, q, dt, r, M):
    desc = np.exp(-r * dt)
    for _ in range(M):
        V = desc * (q * V[1:] + (1 - q) * V[:-1])
    return float(V[0])


def bbs(S0, K, T, r, sig, M):
    """Árvore com o último passo substituído pelo valor BSM."""
    dt, u, d, q = params_crr(T, r, sig, M)
    j = np.arange(M)
    S = S0 * u**j * d ** (M - 1 - j)
    return recursao(bsm(S, K, dt, r, sig), u, d, q, dt, r, M - 1)


def bbsr(S0, K, T, r, sig, M):
    return 2 * bbs(S0, K, T, r, sig, M) - bbs(S0, K, T, r, sig, M // 2)


def leisen_reimer(S0, K, T, r, sig, M):
    M = M + 1 if M % 2 == 0 else M
```

```
    dt = T / M
    d1, d2 = d12(S0, K, T, r, sig)

    def h(z):
        return 0.5 + np.sign(z) * 0.5 * np.sqrt(1 - np.exp(-((z / (M + 1 / 3 + 0.1 / (M + 1))) **
        ↪  2) * (M + 1 / 6)))

    q, q_ = h(d2), h(d1)
    u = np.exp(r * dt) * q_ / q
    d = (np.exp(r * dt) - q * u) / (1 - q)
    j = np.arange(M + 1)
    return recursao(np.maximum(S0 * u**j * d ** (M - j) - K, 0.0), u, d, q, dt, r, M)


def media_viz(S0, K, T, r, sig, M):
    return 0.5 * (crr_fechada(S0, K, T, r, sig, M) + crr_fechada(S0, K, T, r, sig, M + 1))

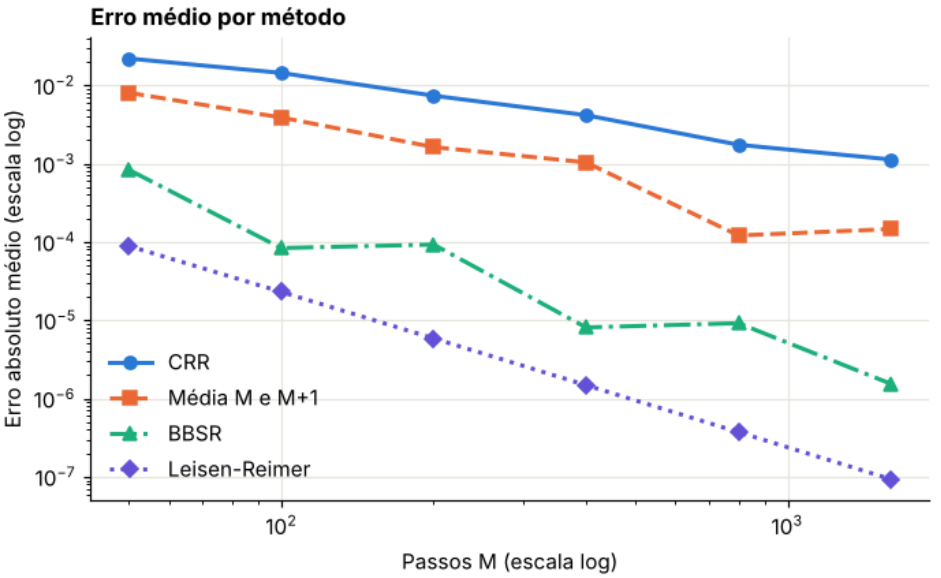

metodos = {"CRR": crr_fechada, "Média M e M+1": media_viz, "BBSR": bbsr, "Leisen-Reimer":
↪  leisen_reimer}
linhas = []
grade_M = [50, 100, 200, 400, 800, 1600]
for K_ in [90.0, 100.0, 115.0]:
    ref = float(bsm(S0, K_, T0, r0, s0))
    for nome, f in metodos.items():
        for M in grade_M:
            linhas.append({"Strike": K_, "Método": nome, "M": M, "Erro": abs(f(S0, K_, T0, r0, s0,
↪  M) - ref)})
er = pd.DataFrame(linhas)
media_err = er.groupby(["Método", "M"])["Erro"].mean().unstack(0)[list(metodos)]
print("Erro absoluto médio em 3 strikes (×1000):")
tabela((media_err * 1000).reset_index())

fig, ax = plt.subplots()
for (nome, cor, est, mark) in [("CRR", AZUL, "-", "o"), ("Média M e M+1", LARANJA, "--", "s"),
↪  ("BBSR", VERDE, "-.", "^"), ("Leisen-Reimer", "#6250d6", ":", "D")]:
    ax.loglog(media_err.index, media_err[nome], color=cor, ls=est, marker=mark, markersize=6,
↪  label=nome)
ax.set_title("Erro médio por método")
ax.set_xlabel("Passos M (escala log)")
ax.set_ylabel("Erro absoluto médio (escala log)")
ax.legend()
plt.show()

ultimo = media_err.iloc[-1]
ok(ultimo["Leisen-Reimer"] < ultimo["CRR"] / 100, "Leisen-Reimer reduz o erro em pelo menos duas
↪  ordens de grandeza")
ok(ultimo["BBSR"] < ultimo["CRR"] / 20, "BBSR reduz bastante o erro")
```

```
Erro absoluto médio em 3 strikes (×1000):
```

M	CRR	Média M e M+1	BBSR	Leisen-Reimer
50	22,036	8,053	0,837	0,089
100	14,439	3,888	0,084	0,023
200	7,380	1,628	0,093	0,006
400	4,165	1,038	0,008	0,001
800	1,745	0,121	0,009	0,000
1600	1,130	0,146	0,002	0,000




```
OK — Leisen-Reimer reduz o erro em pelo menos duas ordens de grandeza
OK — BBSR reduz bastante o erro
```

---

4. Gregas a partir da árvore

Uma das vantagens da árvore: os nós das primeiras datas já contêm a informação das sensibilidades, sem reprecificar.

Delta, com os dois nós de Δt:

$$\Delta \approx \frac{V_u - V_d}{S_0(u-d)}. \tag{6.15}$$

Gama, com os três nós de $2\Delta t$, como variação do delta:

$$\Gamma \approx \frac{\dfrac{V_{uu}-V_{ud}}{S_{uu}-S_{ud}} - \dfrac{V_{ud}-V_{dd}}{S_{ud}-S_{dd}}}{\frac{1}{2}(S_{uu}-S_{dd})}. \tag{6.16}$$

Teta: como $ud = 1$, o nó central de $2\Delta t$ tem o mesmo índice que a raiz:

$$\Theta \approx \frac{V_{ud} - V_0}{2\Delta t}. \tag{6.17}$$

Laboratório 4.1 — Gregas da árvore

```
def arvore_gregas(S0, K, T, r, sig, M, tipo="call", americana=False):
    """Árvore vetorizada que devolve o valor, as gregas e a fronteira de exercício (para put)."""
    dt, u, d, q = params_crr(T, r, sig, M)
    desc = np.exp(-r * dt)
    sinal = 1.0 if tipo == "call" else -1.0
    j = np.arange(M + 1)
    V = np.maximum(sinal * (S0 * u**j * d ** (M - j) - K), 0.0)
    guardados, fronteira = {}, np.full(M + 1, np.nan)
    fronteira[M] = K
    for t in range(M - 1, -1, -1):
        V = desc * (q * V[1:] + (1 - q) * V[:-1])
        if americana:
            j = np.arange(t + 1)
            St = S0 * u**j * d ** (t - j)
            intr = np.maximum(sinal * (St - K), 0.0)
            exerce = intr > V + 1e-12
            V = np.where(exerce, intr, V)
            if tipo == "put" and exerce.any():
                fronteira[t] = St[exerce].max()
        if t <= 2:
            guardados[t] = V.copy()
    V0 = float(guardados[0][0])
    Vd, Vu = guardados[1]
    Vdd, Vud, Vuu = guardados[2]
    delta = (Vu - Vd) / (S0 * (u - d))
    Suu, Sud, Sdd = S0 * u**2, S0, S0 * d**2
    gama = ((Vuu - Vud) / (Suu - Sud) - (Vud - Vdd) / (Sud - Sdd)) / (0.5 * (Suu - Sdd))
    teta = (Vud - V0) / (2 * dt)
    return {"valor": V0, "delta": delta, "gama": gama, "teta": teta, "fronteira": fronteira}


g = arvore_gregas(S0, K0, T0, r0, s0, 2000)
d1, d2 = d12(S0, K0, T0, r0, s0)
exato = {
    "delta": float(norm.cdf(d1)),
    "gama": float(norm.pdf(d1) / (S0 * s0 * np.sqrt(T0))),
    "teta": float(-S0 * norm.pdf(d1) * s0 / (2 * np.sqrt(T0)) - r0 * K0 * np.exp(-r0 * T0) *
    ↪  norm.cdf(d2)),
}
tabela(pd.DataFrame([{"Grega": k, "Árvore (M=2000)": float(g[k]), "BSM": v} for k, v in
↪  exato.items()]))
ok(abs(g["delta"] - exato["delta"]) < 2e-3, "delta da árvore")
ok(abs(g["gama"] - exato["gama"]) < 1e-3, "gama da árvore")
ok(abs(g["teta"] - exato["teta"]) < 0.02, "teta da árvore")
```

Grega	Árvore (M=2000)	BSM
delta	0,637	0,637
gama	0,019	0,019
teta	-6,416	-6,414

```
OK — delta da árvore
OK — gama da árvore
OK — teta da árvore
```

5. Opções americanas

Livro: §5.4; o Módulo 9 aprofunda com simulação

Agora vem a principal vantagem prática. Em cada nó, o titular de uma opção americana escolhe o melhor entre exercer e continuar:

$$V_{\text{nó}} = \max\Big(\underbrace{h(S_{\text{nó}})}_{\text{exercer}},\ \underbrace{e^{-r\Delta t}\big[qV_{\text{sobe}} + (1-q)V_{\text{desce}}\big]}_{\text{continuar}}\Big). \tag{6.18}$$

É a **programação dinâmica**. Bastou acrescentar um `max` à recursão.

5.1 Call americana sem dividendos

Do Módulo 1: uma call europeia vale pelo menos $S - Ke^{-r\tau}$. Com $r > 0$, isso é maior que o valor de exercício $S - K$. Logo, **nunca compensa exercer antes**: vivo vale mais que morto. Portanto, call americana sem dividendos é igual à europeia.

5.2 Put americana

Para a put o argumento se inverte. Se a put estiver muito dentro do dinheiro, o titular prefere receber K já e aplicar. Existe uma **fronteira de exercício** $S^*(t)$: abaixo dela, o exercício é ótimo. Com $r = 0$, não há esse incentivo, e a put americana é igual à europeia.

Laboratório 5.1 — Put americana

Usamos os parâmetros do exemplo clássico de Longstaff e Schwartz: $S_0 = 36$, $K = 40$, $T = 1$, $r = 6\%$, $\sigma = 20\%$.

```
S_a, K_a, T_a, r_a, s_a = 36.0, 40.0, 1.0, 0.06, 0.20
linhas = []
for M in [50, 100, 500, 1000, 2000, 4000]:
    am = arvore_gregas(S_a, K_a, T_a, r_a, s_a, M, "put", True)["valor"]
    eu = arvore_gregas(S_a, K_a, T_a, r_a, s_a, M, "put", False)["valor"]
    linhas.append({"M": M, "Americana": am, "Europeia": eu, "Prêmio de exercício": am - eu})
tb = pd.DataFrame(linhas)
tabela(tb)
print("BSM europeia:", f3(float(bsm(S_a, K_a, T_a, r_a, s_a, "put"))))

ok(abs(tb["Americana"].iloc[-1] - tb["Americana"].iloc[-2]) < 1e-3, "o valor americano converge")
ok((tb["Americana"] > tb["Europeia"]).all(), "a put americana vale mais que a europeia")

call_am = arvore_gregas(100.0, 100.0, 1.0, 0.05, 0.2, 500, "call", True)["valor"]
call_eu = arvore_gregas(100.0, 100.0, 1.0, 0.05, 0.2, 500, "call", False)["valor"]
ok(abs(call_am - call_eu) < 1e-9, "call americana sem dividendos é igual à europeia")
p_am0 = arvore_gregas(S_a, K_a, T_a, 0.0, s_a, 500, "put", True)["valor"]
p_eu0 = arvore_gregas(S_a, K_a, T_a, 0.0, s_a, 500, "put", False)["valor"]
ok(abs(p_am0 - p_eu0) < 1e-9, "com juros zero, a put americana é igual à europeia")

M_f = 4000
res_f = arvore_gregas(S_a, K_a, T_a, r_a, s_a, M_f, "put", True)
ts = np.arange(M_f + 1) * T_a / M_f
fr = pd.Series(res_f["fronteira"]).ffill().values

premio = []
grade_r = np.linspace(0.0, 0.15, 16)
```

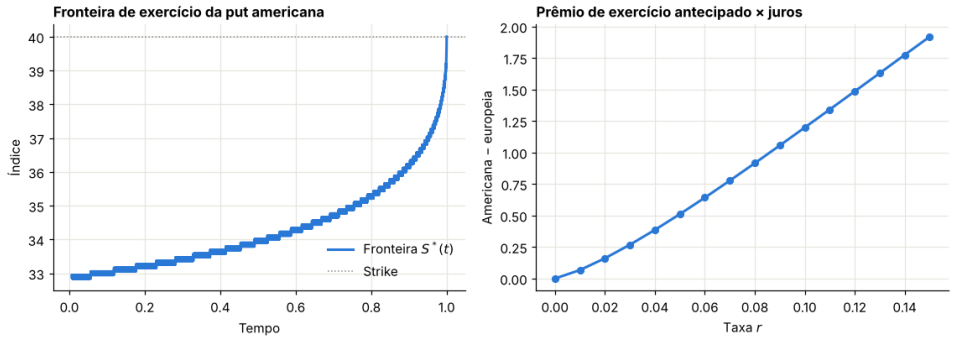
```
for r_ in grade_r:
    premio.append(arvore_gregas(S_a, K_a, T_a, float(r_), s_a, 800, "put", True)["valor"] -
↪   arvore_gregas(S_a, K_a, T_a, float(r_), s_a, 800, "put", False)["valor"])

fig, (a1, a2) = plt.subplots(1, 2, figsize=(10.5, 3.9))
a1.plot(ts[5:], fr[5:], color=AZUL, label="Fronteira $S^*(t)$")
a1.axhline(K_a, color=CINZA, lw=1, ls=":", label="Strike")
a1.set_title("Fronteira de exercício da put americana")
a1.set_xlabel("Tempo")
a1.set_ylabel("Índice")
a1.legend()
a2.plot(grade_r, premio, color=AZUL, marker="o", markersize=5)
a2.set_title("Prêmio de exercício antecipado × juros")
a2.set_xlabel("Taxa $r$")
a2.set_ylabel("Americana − europeia")
plt.tight_layout()
plt.show()

ok(fr[int(0.9 * M_f)] > fr[int(0.1 * M_f)] and fr[-2] < K_a + 1e-9, "a fronteira sobe com o tempo
↪   e termina perto do strike")
ok(np.all(np.diff(premio) > 0), "o prêmio de exercício cresce com os juros")
```

M	Americana	Europeia	Prêmio de exercício
50	4,484	3,840	0,645
100	4,488	3,848	0,640
500	4,486	3,844	0,643
1000	4,487	3,845	0,642
2000	4,487	3,844	0,642
4000	4,487	3,844	0,642

```
BSM europeia: 3,844
OK — o valor americano converge
OK — a put americana vale mais que a europeia
OK — call americana sem dividendos é igual à europeia
OK — com juros zero, a put americana é igual à europeia
```




```
OK — a fronteira sobe com o tempo e termina perto do strike
OK — o prêmio de exercício cresce com os juros
```

Laboratório 5.2 — Exercício apenas em algumas datas (bermudana)

Uma opção **bermudana** só pode ser exercida em datas específicas. É o que um algoritmo de simulação como o do Módulo 9 realmente precifica, com um número finito de datas de exercício. Vale a pena ter uma noção do efeito.

```
def bermudana(S0, K, T, r, sig, M, datas):
    """Put que só pode ser exercida em 'datas' datas igualmente espaçadas."""
    dt, u, d, q = params_crr(T, r, sig, M)
    desc = np.exp(-r * dt)
    j = np.arange(M + 1)
    V = np.maximum(K - S0 * u**j * d ** (M - j), 0.0)
    passo = M // datas
    for t in range(M - 1, -1, -1):
        V = desc * (q * V[1:] + (1 - q) * V[:-1])
        if t % passo == 0 and t > 0:
            j = np.arange(t + 1)
            V = np.maximum(V, K - S0 * u**j * d ** (t - j))
    return float(V[0])


M_b = 4800
linhas = [{"Datas de exercício": n, "Valor": bermudana(S_a, K_a, T_a, r_a, s_a, M_b, n)} for n in
↪  [1, 2, 4, 12, 50, 400, 4800]]
ber = pd.DataFrame(linhas)
tabela(ber)
ok(np.all(np.diff(ber["Valor"]) > 0), "mais datas de exercício aumentam o valor")
ok(abs(ber["Valor"].iloc[0] - float(bsm(S_a, K_a, T_a, r_a, s_a, "put"))) < 2e-3, "com uma única
↪  data, a bermudana é a europeia")
```

Datas de exercício	Valor
1	3,844
2	4,199
4	4,362
12	4,450
50	4,478
400	4,486
4800	4,487

```
OK — mais datas de exercício aumentam o valor
OK — com uma única data, a bermudana é a europeia
```

Sobre a referência. O artigo de Longstaff e Schwartz trabalha com 50 datas de exercício por ano e publica 4,478 para este caso, valor que coincide com a linha de 50 datas acima. O livro usa como referência 4,486, de uma árvore de 500 passos. Essa diferença de 0,009 é o efeito das datas de exercício, e não erro de método. Lembre-se desse detalhe quando validarmos algoritmos no Módulo 9.

6. Payoffs arbitrários

Uma árvore serve para qualquer payoff em T, mas a qualidade da convergência depende do payoff. Uma **digital** tem salto, e não apenas quina. O teste abaixo mostra que o erro é muito maior que na call.

Laboratório 6.1 — A digital na árvore

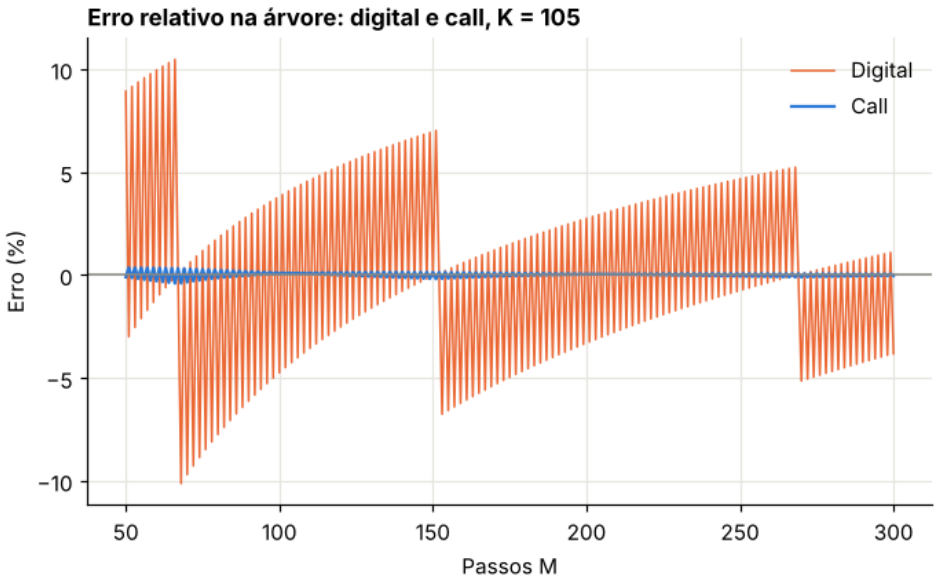
```
def payoff_tree(pay, S0, T, r, sig, M):
    dt, u, d, q = params_crr(T, r, sig, M)
    j = np.arange(M + 1)
    return recursao(pay(S0 * u**j * d ** (M - j)), u, d, q, dt, r, M)


K_d = 105.0
ref_dig = float(np.exp(-r0 * T0) * norm.cdf(d12(S0, K_d, T0, r0, s0)[1]))
pay_d = lambda S: np.where(S > K_d, 1.0, 0.0)
ref_cal = float(bsm(S0, K_d, T0, r0, s0))
pay_c = lambda S: np.maximum(S - K_d, 0.0)

Ms2 = np.arange(50, 301)
e_d = np.array([payoff_tree(pay_d, S0, T0, r0, s0, int(M)) for M in Ms2]) - ref_dig
e_c = np.array([payoff_tree(pay_c, S0, T0, r0, s0, int(M)) for M in Ms2]) - ref_cal

fig, ax = plt.subplots()
ax.plot(Ms2, e_d / ref_dig * 100, color=LARANJA, lw=1, label="Digital")
ax.plot(Ms2, e_c / ref_cal * 100, color=AZUL, lw=1.5, label="Call")
ax.axhline(0, color=CINZA, lw=1)
ax.set_title("Erro relativo na árvore: digital e call, K = 105")
ax.set_xlabel("Passos M")
ax.set_ylabel("Erro (%)")
ax.legend()
plt.show()

print("Maior erro relativo (%): digital", f3(float(np.max(np.abs(e_d / ref_dig)) * 100)), "|
↪  call", f3(float(np.max(np.abs(e_c / ref_cal)) * 100)))
ok(np.max(np.abs(e_d / ref_dig)) > 3 * np.max(np.abs(e_c / ref_cal)), "um payoff descontínuo tem
↪  erro relativo bem maior")
```




```
Maior erro relativo (%): digital 10,481 | call 0,395
OK — um payoff descontínuo tem erro relativo bem maior
```

---

7. A árvore é um método de diferenças finitas

Escreva a recursão CRR com V_j^t o valor no nó de log-preço x_j:

$$V_j^t = e^{-r\Delta t}[q\, V_{j+1}^{t+\Delta t} + (1-q)\, V_{j-1}^{t+\Delta t}]. \tag{6.19}$$

Vamos expandir o lado direito em Taylor, em torno de $(x, t+\Delta t)$, com passo $\Delta x = \sigma\sqrt{\Delta t}$:

$$qV_{j+1} + (1-q)V_{j-1} = V + (2q-1)\Delta x\, V_x + \tfrac{1}{2}\Delta x^2\, V_{xx} + \ldots \tag{6.20}$$

Usando $(2q-1)\Delta x = (r - \frac{1}{2}\sigma^2)\Delta t$ e $\Delta x^2 = \sigma^2\Delta t$, e expandindo $e^{-r\Delta t} \approx 1 - r\Delta t$:

$$V^t = V^{t+\Delta t} + \Delta t[(r - \tfrac{1}{2}\sigma^2)V_x + \tfrac{1}{2}\sigma^2 V_{xx} - rV] + \ldots \tag{6.21}$$

Dividindo por Δt e tomando o limite:

$$V_t + (r - \tfrac{1}{2}\sigma^2)V_x + \tfrac{1}{2}\sigma^2 V_{xx} - rV = 0. \tag{6.22}$$

É a EDP de Black-Scholes em log-preço, a mesma que resolvemos no Módulo 5. A árvore é um esquema explícito de diferenças finitas, com a escolha particular $\Delta x^2 = \sigma^2\Delta t$, que fica no limite de estabilidade. Isso explica também a ordem de convergência $1/M$: é a do esquema explícito no tempo.

---

8. Síntese

- A árvore CRR é a do Módulo 3, com $ud = 1$ para recombinar.
- O termo $-\frac{1}{2}\sigma^2$ surge sozinho da condição de martingale.
- A soma binomial converge para BSM, e q' é o análogo discreto de $N(d_1)$.
- O erro oscila conforme a posição de K nos nós; BBSR e Leisen-Reimer eliminam quase todo o erro.
- Os primeiros nós dão as gregas de graça.
- Basta um `max` para precificar americanas: call sem dividendos é igual à europeia, e a put tem fronteira de exercício.
- Valores de referência por simulação costumam ser bermudanos.
- A árvore é um esquema explícito de diferenças finitas da EDP.

9. Notas sobre o livro neste trecho

Local	Comentário
Nota da Tabela 5.1	diz “put”; a tabela é de call
§5.4, após a Figura 5.8	atribui a oscilação a “parametrização diferente”; o motivo é a paridade de M em relação à posição de K

10. Exercícios

Exercício 1. Para $M = 2$, com $S_0 = K = 100$, $T = 1$, $r = 5\%$, $\sigma = 20\%$, calcule à mão a call.

Solução

$\Delta t = 0{,}500$, $u = e^{0{,}141} = 1{,}152$, $d = 0{,}868$, $q = (1{,}025 - 0{,}868)/0{,}284 = 0{,}554$. Em T: $S_{uu} = 132{,}690$, payoff 32,690; os demais não pagam. Valor: $e^{-0{,}05} \times 0{,}554^2 \times 32{,}690 = 9{,}541$.

Exercício 2. Por que os nós intermediários não precisam ser guardados na versão vetorizada?

Solução

Cada coluna só depende da seguinte. Para opções americanas, os preços do nó são recalculados por fórmula em cada data, de forma que não há necessidade de manter a matriz inteira.

Exercício 3. Modifique `arvore_gregas` para índice que paga dividendo contínuo y. Com $y > 0$, a call americana pode ser exercida antes?

Solução

Troque q por $(e^{(r-y)\Delta t} - d)/(u-d)$. A desigualdade $C \geq S - Ke^{-r\tau}$ vira $C \geq Se^{-y\tau} - Ke^{-r\tau}$, que pode ser menor que $S - K$: o exercício antecipado passa a ser possível.

Exercício 4. Calcule o número mínimo de passos para que $q \in (0,1)$ com $r = 15\%$, $\sigma = 10\%$, $T = 2$.

Solução

$M > r^2 T/\sigma^2 = 0{,}0225 \times 2/0{,}01 = 4{,}5$. Logo, $M \geq 5$.

Exercício 5. Por que uma árvore recombinante não serve diretamente para uma opção asiática? O que seria necessário?

Solução

O payoff depende da média do caminho, e caminhos diferentes que se reencontram num nó têm médias diferentes. Seria preciso guardar a média como variável de estado adicional, em cada nó, ou recorrer à simulação de Monte Carlo.

```
print(f"Módulo 6 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 6 concluído: 27 verificações aprovadas.
```

Capítulo 7

Métodos de Fourier I: da função característica à fórmula de Lewis

Introdução

Muitos modelos realistas não têm uma fórmula fechada para o preço das opções, mas têm uma fórmula fechada para outra coisa: a **função característica** do log-preço, a transformada de Fourier de sua distribuição. É o caso dos modelos com saltos e de volatilidade estocástica usados no restante do livro.

A ideia dos métodos de Fourier é precificar a partir dessa transformada. O preço de uma call é uma integral do pagamento contra a densidade do ativo; pela fórmula de Parseval, ele pode ser escrito como uma integral da transformada do pagamento contra a função característica. A **fórmula de Lewis** faz isso de forma particularmente estável e reduz a precificação a uma integral unidimensional, rápida o bastante para ser repetida milhares de vezes durante uma calibração.

O capítulo constrói a função característica dos modelos de Black-Scholes, Merton e Heston, deduz a fórmula de Lewis e confere numericamente seus preços contra Black-Scholes e contra simulação.

Fundamentos teóricos

Função característica

Definição 7.1 (Função característica). A função característica de uma variável aleatória X é $\varphi_X(u) = \mathbb{E}[e^{iuX}]$, $u \in \mathbb{R}$. Ela existe sempre, determina a lei de X e, quando X tem densidade f integrável, $f(x) = \frac{1}{2\pi}\int_{\mathbb{R}} e^{-iux}\varphi_X(u)\,du$. A definição se estende a $u \in \mathbb{C}$ na faixa onde $\mathbb{E}[e^{-\operatorname{Im}(u)X}] < \infty$.

Exemplo 7.2 (Modelos usados no livro). Com $X_T = \ln(S_T/S_0)$ sob $\mathbb{Q}$:

1. Black-Scholes: $\varphi(u) = \exp\left(iu(r - \tfrac{1}{2}\sigma^2)T - \tfrac{1}{2}\sigma^2 u^2 T\right)$.
2. Merton com saltos (Merton, 1976): saltos de intensidade λ e tamanho $\ln(1+J) \sim$

$N(\mu_J, \delta^2)$,

$$\varphi(u) = \exp\left(T\left[iu\omega - \tfrac{1}{2}\sigma^2 u^2 + \lambda(e^{iu\mu_J - \delta^2 u^2/2} - 1)\right]\right), \tag{7.1}$$

com $\omega = r - \frac{1}{2}\sigma^2 - \lambda(e^{\mu_J + \delta^2/2} - 1)$ escolhido para que $e^{-rt}S_t$ seja martingal.

3. Heston (Heston, 1993): forma exponencial-afim $\varphi(u) = \exp\left(A(u,T) + B(u,T)v_0\right)$, com A e B soluções de equações de Riccati (Capítulo 11).

Teorema 7.3 (Processos de Lévy). *Se X é um processo de Lévy, $\varphi_{X_t}(u) = e^{t\psi(u)}$, com o expoente característico dado pela fórmula de Lévy-Khintchine*

$$\psi(u) = i\gamma u - \tfrac{1}{2}\sigma^2 u^2 + \int_{\mathbb{R}} \left(e^{iux} - 1 - iux\mathbf{1}_{|x|<1}\right)\nu(dx). \tag{7.2}$$

O modelo de Merton é Lévy (difusão mais Poisson composto); o de Heston não é, mas é *afim*, e também tem função característica em forma fechada (Duffie et al., 2000).

A fórmula de Lewis

Teorema 7.4 (Lewis, 2001). *Seja φ a função característica de $X_T = \ln(S_T/S_0)$ sob $\mathbb{Q}$ e suponha que $\varphi(u - \frac{i}{2})$ é bem definida. Então a call europeia vale*

$$C_0 = S_0 - \frac{\sqrt{S_0 K}\, e^{-rT/2}}{\pi} \int_0^\infty \operatorname{Re}\left[e^{iuk}\,\varphi(u - \tfrac{i}{2})\right] \frac{du}{u^2 + \frac{1}{4}}, \qquad k = \ln(S_0/K). \tag{7.3}$$

Esboço. Escreva o payoff em log-preço, $w(x) = (e^x - K)^+$, e sua transformada generalizada $\hat{w}(z) = \int e^{izx} w(x)\,dx = -K^{iz+1}/(z^2 - iz)$, válida para $\operatorname{Im} z > 1$. Pela fórmula de Parseval, $C_0 = \frac{e^{-rT}}{2\pi}\int \hat{w}(z)\,\varphi_{\ln S_T}(-z)\,dz$ ao longo de uma reta $\operatorname{Im} z = c$. Desloque a reta para $c = \frac{1}{2}$; ao cruzar o polo em $z = i$ aparece o resíduo S_0. O integrando restante é par na parte real, o que dá o fator 2 e a integral em $[0, \infty)$ (Lewis, 2001). ■

Observação 7.5 (Por que $u - \frac{i}{2}$). A linha $\operatorname{Im} z = \frac{1}{2}$ fica no meio da faixa $0 < \operatorname{Im} z < 1$ onde a transformada de um contrato *covered call* existe e onde φ é finita para qualquer modelo com primeiro momento: $\mathbb{E}[S_T^{1/2}] < \infty$. O integrando decai como u^{-2}, e a integral converge rapidamente.

Integração numérica

A integral (7.3) é calculada por quadratura em $[0, U]$. O truncamento em U tem erro controlado pelo decaimento de $|\varphi(u - \frac{i}{2})|$; nos modelos de difusão, o decaimento é gaussiano e U moderado basta; com saltos e prazos curtos, o decaimento é mais lento e U precisa crescer. A quadratura adaptativa é robusta, mas cara para muitos strikes; o Capítulo 8 troca a quadratura por uma transformada rápida.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** seções 6.1 a 6.4.1 do livro. Carr-Madan, FFT e convolução ficam para o Módulo 8.

O que você vai conseguir fazer ao final

1. Calcular a transformada de Fourier de funções simples e usar suas propriedades.
2. Entender por que a função característica é o objeto certo para precificar.
3. Deduzir a função característica de modelos com saltos.
4. Deduzir a fórmula de Lewis passo a passo, incluindo a integral no plano complexo.
5. Avaliá-la numericamente, sabendo como escolher limite e malha.
6. Identificar quando o método perde precisão.

O problema

Toda opção europeia é uma integral do payoff contra a densidade neutra ao risco:

$$C_0 = e^{-rT} \int_0^\infty \max(s - K, 0)\, q(s)\, ds. \tag{7.4}$$

Fora do BSM, a densidade q em geral **não tem fórmula**. Mas em Merton, Heston e BCC97, a **função característica** tem. A ideia desta família de métodos é trocar a integral na variável de preço por uma integral na variável de frequência, em que os dois ingredientes são conhecidos.

Foi o que usamos como caixa-preta no Módulo 2.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
from time import perf_counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import mpmath as mp
from scipy.integrate import quad, simpson
from scipy.stats import norm, poisson
from IPython.display import display

RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

def bsm(S, K, T, r, sig, tipo="call"):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    d2 = d1 - sig * np.sqrt(T)
    if tipo == "call":
        return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

1. A transformada de Fourier

Livro: §6.3

1.1 Definição

Usaremos a convenção do livro. A **transformada de Fourier** de f é

$$\hat{f}(u) = \int_{-\infty}^{\infty} e^{iux} f(x)\, dx, \tag{7.5}$$

e a fórmula de **inversão** recupera a função:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} e^{-iux} \hat{f}(u)\, du. \tag{7.6}$$

Intuição: $e^{iux} = \cos(ux) + i\sin(ux)$ é uma oscilação de frequência u. $\hat{f}(u)$ mede quanto dessa frequência existe em f. Inverter é recompor a função somando todas as frequências.

1.2 Exemplo: a gaussiana

Seja $f(x) = \varphi(x)$, a densidade normal padrão. Completamos o quadrado no expoente:

$$iux - \tfrac{1}{2}x^2 = -\tfrac{1}{2}(x - iu)^2 - \tfrac{1}{2}u^2. \tag{7.7}$$

Assim,

$$\hat{\varphi}(u) = e^{-u^2/2} \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-(x-iu)^2/2}\, dx = e^{-u^2/2}. \tag{7.8}$$

A integral restante vale 1, mesmo com o deslocamento complexo. A transformada de uma gaussiana é outra gaussiana. Note a relação inversa entre as larguras: **estreita em x significa larga em u**. É a origem da dificuldade para opções de prazo curto.

1.3 A propriedade que importa

Se X e Y são independentes com densidades f e g, a densidade de $X+Y$ é a **convolução**

$$(f * g)(z) = \int f(x)\, g(z - x)\, dx. \tag{7.9}$$

Convoluções são difíceis. Mas, na transformada, viram **produto**:

$$\widehat{f * g}(u) = \hat{f}(u)\, \hat{g}(u). \tag{7.10}$$

Prova, com a troca de variável $y = z - x$:

$$\int e^{iuz} \int f(x) g(z-x)\, dx\, dz = \int e^{iux} f(x) \int e^{iu(z-x)} g(z-x)\, dz\, dx = \hat{f}(u)\hat{g}(u). \tag{7.11}$$

Essa é a razão de tudo funcionar. Log-preços são somas de muitos incrementos independentes: a densidade é uma convolução intratável; a transformada, um produto simples.

1.4 Relação de Parseval

Para funções f, g de quadrado integrável:

$$\int f(x)\,\overline{g(x)}\,dx = \frac{1}{2\pi}\int \hat{f}(u)\,\overline{\hat{g}(u)}\,du. \tag{7.12}$$

Um **produto interno** em x é igual a um produto interno em u. Note o paralelo com a precificação: o valor da opção é um produto interno entre payoff e densidade. Daí, transformamos os dois.

Laboratório 1.1 — Propriedades, numericamente

```
x = np.linspace(-12, 12, 12001)


def transformada(f_val, u):
    return simpson(np.exp(1j * u * x) * f_val, x=x)


u_g = np.linspace(-6, 6, 25)
f_a = norm.pdf(x)
num = np.array([transformada(f_a, u) for u in u_g])
ok(np.max(np.abs(num - np.exp(-(u_g**2) / 2))) < 1e-8, "a transformada da gaussiana é e^(−u²/2)")

g1, g2 = norm.pdf(x, 0.5, 0.7), norm.pdf(x, -1.0, 1.2)
conv = np.convolve(g1, g2, mode="same") * (x[1] - x[0])
referencia = norm.pdf(x, -0.5, np.hypot(0.7, 1.2))
ok(np.max(np.abs(conv - referencia)) < 1e-3, "convolução de normais é a normal da soma")
us = np.array([0.3, 1.0, 2.5])
lado_e = np.array([transformada(referencia, u) for u in us])
lado_d = np.array([transformada(g1, u) * transformada(g2, u) for u in us])
ok(np.max(np.abs(lado_e - lado_d)) < 1e-8, "a transformada da convolução é o produto das
↪  transformadas")

u_p = np.linspace(-30, 30, 6001)
hat1 = np.array([transformada(g1, u) for u in u_p[::10]])
hat2 = np.array([transformada(g2, u) for u in u_p[::10]])
esq = simpson(g1 * g2, x=x)
dir_ = (simpson(hat1 * np.conj(hat2), x=u_p[::10]) / (2 * np.pi)).real
print("Parseval: lado em x =", f3(float(esq)), "| lado em u =", f3(float(dir_)))
ok(abs(esq - dir_) < 1e-8, "relação de Parseval")
```

```
OK — a transformada da gaussiana é e^(−u²/2)
OK — convolução de normais é a normal da soma
OK — a transformada da convolução é o produto das transformadas
```

```
Parseval: lado em x = 0,160 | lado em u = 0,160
OK — relação de Parseval
```

---

2. Funções características

Livro: §6.3, Definição 41

2.1 Definição e propriedades

A **função característica** de X é a transformada de Fourier de sua densidade:

$$\phi(u) = \mathbb{E}[e^{iuX}]. \tag{7.13}$$

Propriedades que vamos usar:

Propriedade	Motivo
$\phi(0) = 1$	a densidade integra 1
$\lvert\phi(u)\rvert \le 1$ para u real	$\lvert e^{iux}\rvert = 1$
$\phi(-u) = \overline{\phi(u)}$ para u real	X é real
X, Y independentes: $\phi_{X+Y} = \phi_X \phi_Y$	seção anterior
Sempre existe para u real	ao contrário da função geradora de momentos

2.2 Cumulantes

Tomando o logaritmo, obtemos os **cumulantes** c_n, que somam em variáveis independentes:

$$\ln \mathbb{E}[e^{tX}] = \sum_{n\ge 1} c_n \frac{t^n}{n!}, \qquad c_1 = \text{média}, \quad c_2 = \text{variância}. \tag{7.14}$$

Em termos dos cumulantes: assimetria $= c_3/c_2^{3/2}$ e curtose em excesso $= c_4/c_2^2$. Como a soma de n termos iid tem cumulantes multiplicados por n, a assimetria cai com $1/\sqrt{n}$ e a curtose com $1/n$. É o que vimos com o passeio aleatório do Módulo 4, e explica por que o smile do Merton se achata com o prazo.

2.3 Argumento complexo e condição de martingale

Para o que vem a seguir, precisamos de ϕ em **argumentos complexos**. Escrevendo $u = -i$:

$$\phi(-i) = \mathbb{E}[e^{i(-i)X}] = \mathbb{E}[e^{X}]. \tag{7.15}$$

Vamos sempre escrever $S_T = S_0 e^{rT + X_T}$. O índice descontado é martingale se e só se

$$\boxed{\phi(-i) = 1.} \tag{7.16}$$

Atenção: com argumento complexo a integral pode divergir. ϕ só existe numa **faixa** de $\mathrm{Im}(u)$, determinada pelos momentos de X. Para a normal, a faixa é o plano inteiro. Para modelos como o Heston, ela é limitada, e isso tem consequências práticas.

2.4 Dois exemplos

BSM. $X_T = -\frac{1}{2}\sigma^2 T + \sigma Z_T$, normal com média $-\frac{1}{2}\sigma^2 T$ e variância $\sigma^2 T$:

$$\phi^{BSM}(u) = \exp\left[-\tfrac{1}{2}\sigma^2 T\,(iu + u^2)\right]. \tag{7.17}$$

Merton (1976). $X_T = \omega T + \sigma Z_T + \sum_{j=1}^{N_T} J_j$, com $J_j \sim N(\mu_J, \delta^2)$ e N_T Poisson. Calculamos a parte de saltos **condicionando** em N_T:

$$\mathbb{E}\big[e^{iu\sum J_j}\big] = \sum_{n=0}^{\infty} e^{-\lambda T}\frac{(\lambda T)^n}{n!}\,\phi_J(u)^n = \exp\big[\lambda T(\phi_J(u)-1)\big], \tag{7.18}$$

usando a série da exponencial, com $\phi_J(u) = e^{iu\mu_J - u^2\delta^2/2}$. Juntando os termos:

$$\phi^{M76}(u) = \exp\Big\{T\Big[iu\,\omega - \tfrac{1}{2}\sigma^2u^2 + \lambda\big(e^{iu\mu_J - u^2\delta^2/2} - 1\big)\Big]\Big\}. \tag{7.19}$$

O termo ω é o que garante a condição de martingale. Exigindo $\phi(-i) = 1$:

$$\omega + \tfrac{1}{2}\sigma^2 + \lambda\big(e^{\mu_J + \delta^2/2} - 1\big) = 0 \quad\Longrightarrow\quad \omega = -\tfrac{1}{2}\sigma^2 - \lambda\bar{\kappa}. \tag{7.20}$$

Laboratório 2.1 — Recuperando densidade e momentos

Invertemos a função característica de Merton para obter a densidade, e comparamos com a mistura exata de normais. Depois calculamos os cumulantes com derivadas em alta precisão.

```
def cf_bsm(u, T, sigma):
    return np.exp(-0.5 * sigma**2 * T * (1j * u + u**2))


def cf_merton(u, T, sigma, lam, mu, delta):
    omega = -0.5 * sigma**2 - lam * (np.exp(mu + 0.5 * delta**2) - 1)
    return np.exp(T * (1j * u * omega - 0.5 * sigma**2 * u**2 + lam * (np.exp(1j * u * mu - 0.5 *
    ↪  u**2 * delta**2) - 1)))


M76 = dict(sigma=0.15, lam=0.8, mu=-0.12, delta=0.10)
T1 = 1.0
ok(abs(cf_merton(0.0, T1, **M76) - 1) < 1e-14, "φ(0) = 1")
ok(abs(cf_merton(-1j, T1, **M76) - 1) < 1e-14, "condição de martingale: φ(−i) = 1")
ok(abs(cf_bsm(-1j, T1, 0.2) - 1) < 1e-14, "condição de martingale para o BSM")

xg = np.linspace(-1.2, 0.9, 421)
ug = np.linspace(-120, 120, 48001)
fi = cf_merton(ug, T1, **M76)
dens_cf = np.array([simpson((np.exp(-1j * ug * x_) * fi).real, x=ug) for x_ in xg]) / (2 * np.pi)

omega = -0.5 * M76["sigma"] ** 2 - M76["lam"] * (np.exp(M76["mu"] + 0.5 * M76["delta"] ** 2) - 1)
dens_mix = np.zeros_like(xg)
for n in range(40):
    dens_mix += poisson.pmf(n, M76["lam"] * T1) * norm.pdf(xg, omega * T1 + n * M76["mu"],
↪  np.sqrt(M76["sigma"] ** 2 * T1 + n * M76["delta"] ** 2))

fig, (a1, a2) = plt.subplots(1, 2, figsize=(10.5, 3.9))
ur = np.linspace(-30, 30, 1201)
fi_r = cf_merton(ur, T1, **M76)
a1.plot(ur, fi_r.real, color=AZUL, label="Parte real")
a1.plot(ur, fi_r.imag, color=LARANJA, ls="--", label="Parte imaginária")
a1.set_title("Função característica de Merton")
a1.set_xlabel("$u$")
a1.set_ylabel("$\\phi(u)$")
a1.legend()
a2.plot(xg, dens_mix, color=VERDE, lw=4, alpha=0.55, label="Mistura de normais")
```

```
a2.plot(xg, dens_cf, color=AZUL, lw=1.4, label="Inversão de φ")
a2.set_title("Densidade do log-retorno")
a2.set_xlabel("$X_T$")
a2.set_ylabel("Densidade")
a2.legend()
plt.tight_layout()
plt.show()
ok(np.max(np.abs(dens_cf - dens_mix)) < 1e-8, "a inversão de φ reproduz a densidade exata")

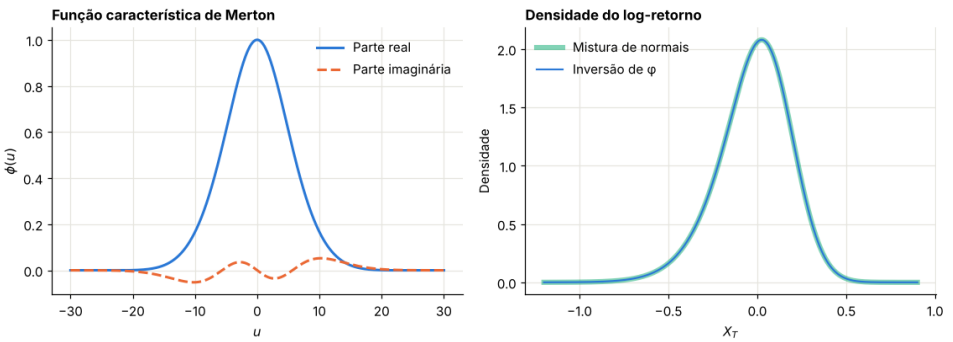

def cumulantes(cf, n_max=4):
    """Derivadas do logaritmo de E[exp(tX)] em t = 0, por mpmath com alta precisão."""
    mp.mp.dps = 40
    psi = lambda t: mp.log(cf(t))
    return [float(mp.diff(psi, 0, n)) for n in range(1, n_max + 1)]


def mgf_merton(t, T=T1, sigma=M76["sigma"], lam=M76["lam"], mu=M76["mu"], delta=M76["delta"]):
    omega = -sigma**2 / 2 - lam * (mp.e ** (mu + delta**2 / 2) - 1)
    return mp.e ** (T * (omega * t + sigma**2 * t**2 / 2 + lam * (mp.e ** (mu * t + delta**2 *
    ↪  t**2 / 2) - 1)))


c1, c2, c3, c4 = cumulantes(mgf_merton)
s, l_, m_, d_ = M76["sigma"], M76["lam"], M76["mu"], M76["delta"]
teoria = [omega * T1 + l_ * m_ * T1, (s**2 + l_ * (m_**2 + d_**2)) * T1, l_ * T1 * (m_**3 + 3 * m_
↪  * d_**2), l_ * T1 * (m_**4 + 6 * m_**2 * d_**2 + 3 * d_**4)]
tabela(pd.DataFrame({"Cumulante": ["c1", "c2", "c3", "c4"], "Da função característica": [c1, c2,
↪  c3, c4], "Fórmula direta": teoria}))
ok(np.allclose([c1, c2, c3, c4], teoria, rtol=1e-10), "os cumulantes da função característica
↪  batem com as fórmulas")

rng = np.random.default_rng(801)
N = rng.poisson(l_ * T1, 3_000_000)
X_ = omega * T1 + s * np.sqrt(T1) * rng.standard_normal(N.size) + N * m_ + np.sqrt(N) * d_ *
↪  rng.standard_normal(N.size)
print("Assimetria teórica:", f3(c3 / c2**1.5), "| Curtose teórica:", f3(c4 / c2**2))
ok(abs(np.var(X_) - c2) < 2e-4, "a variância simulada confere")
```

```
OK — φ(0) = 1
OK — condição de martingale: φ(−i) = 1
OK — condição de martingale para o BSM
```



```
OK — a inversão de φ reproduz a densidade exata
```

Cumulante	Da função característica	Fórmula direta
c1	-0,020	-0,020
c2	0,042	0,042
c3	-0,004	-0,004
c4	0,001	0,001

```
OK — os cumulantes da função característica batem com as fórmulas
```

```
Assimetria teórica: -0,495 | Curtose teórica: 0,621
```

```
OK — a variância simulada confere
```

---

3. A fórmula de Lewis, passo a passo

Livro: §6.4.1

Passo 1: transformar o payoff

Com $s = \ln S_T$, o payoff é $C_T(s) = \max(e^s - K, 0)$. Note um problema: **ele não é integrável**, pois cresce sem limite. A transformada só existe se u tiver parte imaginária suficientemente grande, que produz um amortecimento.

Seja $u = u_r + iu_i$. Como $|e^{ius}| = e^{-u_i s}$, a integral converge em $s \to +\infty$ se $u_i > 1$. Calculando:

$$\hat{C}_T(u) = \int_{\ln K}^{\infty} e^{ius}(e^s - K)\,ds = \Big[\frac{e^{(iu+1)s}}{iu+1} - K\frac{e^{ius}}{iu}\Big]_{\ln K}^{\infty}. \tag{7.21}$$

Com $u_i > 1$, o limite superior é zero. No limite inferior, $e^{(iu+1)\ln K} = K^{iu+1}$ e $K\,e^{iu\ln K} = K^{iu+1}$:

$$\hat{C}_T(u) = K^{iu+1}\Big(\frac{1}{iu} - \frac{1}{iu+1}\Big) = \frac{K^{iu+1}}{iu(iu+1)} = -\frac{K^{iu+1}}{u^2 - iu}. \tag{7.22}$$

Passo 2: inverter e trocar a ordem

Escrevemos o payoff pela inversão, numa reta com $u_i > 1$, e aplicamos a esperança:

$$C_0 = e^{-rT}\,\mathbb{E}^Q\Big[\frac{1}{2\pi}\int e^{-ius}\hat{C}_T(u)\,du\Big] = \frac{e^{-rT}}{2\pi}\int \mathbb{E}^Q\big[e^{-ius}\big]\hat{C}_T(u)\,du. \tag{7.23}$$

Passo 3: colocar a função característica

Como $s = \ln S_T = y + X_T$, com $y = \ln S_0 + rT$:

$$\mathbb{E}^Q[e^{-ius}] = e^{-iuy}\,\phi(-u). \tag{7.24}$$

Com $k = \ln(S_0/K) + rT$, juntamos K^{iu} com e^{-iuy}, ficando e^{-iuk}:

$$\boxed{C_0 = -\frac{Ke^{-rT}}{2\pi}\int_{-\infty+iu_i}^{\infty+iu_i} e^{-iuk}\,\phi(-u)\,\frac{du}{u^2-iu}, \qquad u_i > 1.} \tag{7.25}$$

Passo 4: atravessar um polo

O denominador é $u^2 - iu = u(u-i)$: há polos em $u = 0$ e $u = i$. A reta original está acima de ambos. Queremos descer para $0 < u_i < 1$, onde $\phi(u - i/2)$ é melhor comportada.

O **teorema dos resíduos** diz que, ao mover a reta através de um polo, a integral muda de $2\pi i$ vezes o resíduo. O resíduo em $u = i$ é

$$\mathrm{Res}(i) = \lim_{u\to i}(u-i)\Big[-\frac{Ke^{-rT}}{2\pi}\frac{e^{-iuk}\phi(-u)}{u(u-i)}\Big] = -\frac{Ke^{-rT}}{2\pi}\cdot\frac{e^k\,\phi(-i)}{i}. \tag{7.26}$$

Usando $e^k = (S_0/K)e^{rT}$, $\phi(-i) = 1$ (martingale!) e $1/i = -i$, sobra $\mathrm{Res}(i) = \dfrac{iS_0}{2\pi}$.
A integral em $u_i > 1$ é a integral em $u_i \in (0,1)$ menos $2\pi i\,\mathrm{Res}(i) = -S_0$. Portanto,

$$\boxed{C_0 = S_0 - \frac{Ke^{-rT}}{2\pi}\int_{-\infty+iu_i}^{\infty+iu_i} e^{-iuk}\,\phi(-u)\,\frac{du}{u^2-iu}, \qquad 0 < u_i < 1.} \tag{7.27}$$

O termo S_0 aparece por causa da condição de martingale. É um bom sinal: é a parte que vem de $\phi(-i) = 1$.

Passo 5: escolher a reta simétrica

Tomamos $u_i = \frac{1}{2}$, ou seja, $u = z + i/2$, com z real. Calculemos o denominador:

$$(z+\tfrac{i}{2})^2 - i(z+\tfrac{i}{2}) = z^2 + iz - \tfrac{1}{4} - iz + \tfrac{1}{2} = z^2 + \tfrac{1}{4}. \tag{7.28}$$

Um real positivo. O fator exponencial é $e^{-i(z+i/2)k} = e^{-izk}e^{k/2}$, e

$$Ke^{-rT}e^{k/2} = Ke^{-rT}\sqrt{\frac{S_0}{K}}e^{rT/2} = \sqrt{S_0K}\,e^{-rT/2}. \tag{7.29}$$

Portanto,

$$C_0 = S_0 - \frac{\sqrt{S_0K}\,e^{-rT/2}}{2\pi}\int_{-\infty}^{\infty} e^{-izk}\,\phi(-z-i/2)\,\frac{dz}{z^2+1/4}. \tag{7.30}$$

Passo 6: integrar só em $z > 0$

A integral é de uma função $h(z)$ com $h(-z) = \overline{h(z)}$. Assim, ela é igual a $2\int_0^\infty \mathrm{Re}[h(z)]\,dz$. Trocando $z \to -z$ na expressão, chega-se à **fórmula de Lewis**:

$$\boxed{C_0 = S_0 - \frac{\sqrt{S_0 K}\, e^{-rT/2}}{\pi} \int_0^\infty \mathrm{Re}\big[e^{izk}\,\phi(z - i/2)\big]\,\frac{dz}{z^2 + 1/4}.} \tag{7.31}$$

Errata do livro. Na prova da Proposição 6, o livro escreve $(z - i/2)^2 - (z - i/2)i = z^2 - 2zi + 1/4$, e usa esse denominador na equação seguinte. Há dois erros: a substituição deveria ser na variável real, com $u + i/2$, e o resultado da expressão impressa nem é o indicado (daria $z^2 - 2iz - 3/4$). A conta correta é a do Passo 5, e a equação final (6.5) está certa.

Caso BSM: a integral é real

Com $u = z - i/2$: $iu + u^2 = iz + \frac{1}{2} + z^2 - iz - \frac{1}{4} = z^2 + \frac{1}{4}$. Logo, $\phi^{BSM}(z - i/2) = e^{-\frac{1}{2}\sigma^2 T(z^2 + 1/4)}$, um número **real**, e a fórmula de Lewis fica uma integral de $\cos(zk)$ contra uma gaussiana amortecida. O integrando em BSM decai de forma gaussiana, e essa velocidade depende de $\sigma^2 T$.

Laboratório 3.1 — Conferindo a transformada do payoff

```
K = 2.0
s_g = np.linspace(np.log(K), np.log(K) + 90, 1_800_001)
linhas = []
for u in [0.5 + 1.5j, 2.0 + 2.0j, -3.0 + 1.2j, 7.0 + 3.0j]:
    num = simpson(np.exp(1j * u * s_g) * (np.exp(s_g) - K), x=s_g)
    fec = -(K ** (1j * u + 1)) / (u**2 - 1j * u)
    linhas.append({"u": str(u), "Numérica (real)": num.real, "Fechada (real)": fec.real,
    "Numérica (imag)": num.imag, "Fechada (imag)": fec.imag, "Erro relativo (×10⁶)": abs(num -
    fec) / abs(fec) * 1e6})
pt = pd.DataFrame(linhas)
tabela(pt)
ok((pt["Erro relativo (×10⁶)"] < 1).all(), "a transformada do payoff confere numericamente, com
    u_i > 1")
```

u	Numérica (real)	Fechada (real)	Numérica (imag)	Fechada (imag)	Erro relativo ($\times 10^6$)
(0.5+1.5j)	0,074	0,074	0,628	0,628	0,000
(2+2j)	-0,078	-0,078	-0,011	-0,011	0,000
(-3+1.2j)	0,006	0,006	0,089	0,089	0,049
(7+3j)	0,002	0,002	0,004	0,004	0,000

```
OK — a transformada do payoff confere numericamente, com u_i > 1
```

Laboratório 3.2 — Mover a reta muda o resultado de forma exata

Calculamos a integral em retas diferentes. O teorema dos resíduos prevê:

- $u_i > 1$: a integral dá a **call**, sem o termo S_0;
- $0 < u_i < 1$: a integral mais S_0 dá a call;

- $u_i < 0$: a reta passou também pelo polo em $u = 0$, e a integral dá a **put**.

```
def integral_reta(S, K, T, r, cf, ui, U=300.0, n=120001):
    """Calcula -K exp(-rT)/(2π) ∫ exp(-iuk) φ(-u)/(u²-iu) du em u = z + i·ui."""
    k = np.log(S / K) + r * T
    z = np.linspace(-U, U, n)
    u = z + 1j * ui
    h = np.exp(-1j * u * k) * cf(-u) / (u**2 - 1j * u)
    return float((-K * np.exp(-r * T) / (2 * np.pi) * simpson(h, x=z)).real)

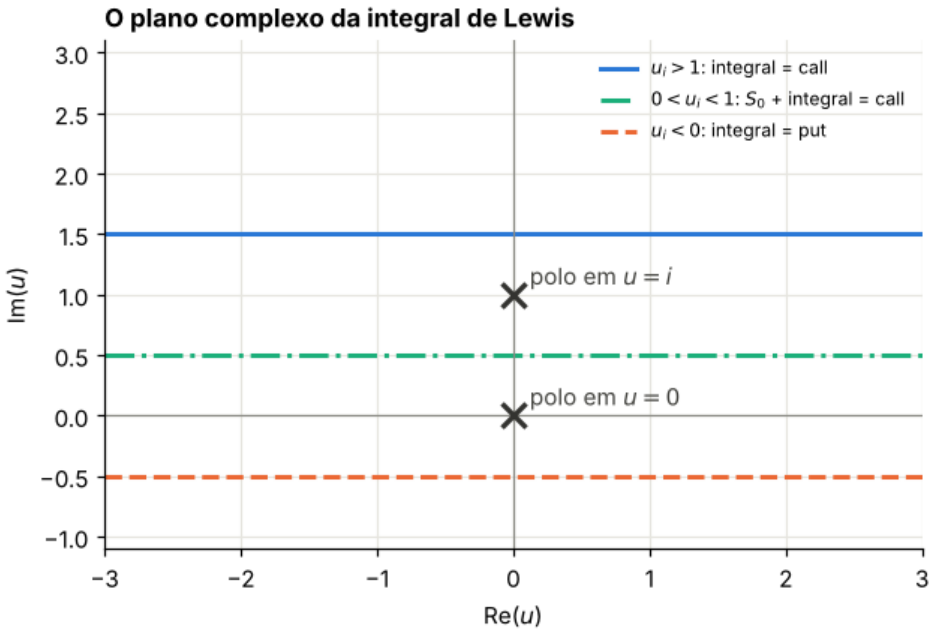

S0, K0, T0, r0, s0 = 100.0, 95.0, 1.0, 0.04, 0.2
cf_b = lambda u: cf_bsm(u, T0, s0)
C_ref, P_ref = float(bsm(S0, K0, T0, r0, s0)), float(bsm(S0, K0, T0, r0, s0, "put"))

linhas = []
for ui in [2.5, 1.5, 0.75, 0.5, 0.25, -0.5, -1.5]:
    I = integral_reta(S0, K0, T0, r0, cf_b, ui)
    if ui > 1:
        prev, nome, ref = I, "Integral = call", C_ref
    elif ui > 0:
        prev, nome, ref = I + S0, "S0 + integral = call", C_ref
    else:
        prev, nome, ref = I, "Integral = put", P_ref
    linhas.append({"u_i": ui, "Regra": nome, "Resultado": prev, "Referência BSM": ref, "Erro":
↪   abs(prev - ref)})
ret = pd.DataFrame(linhas)
tabela(ret)
ok((ret["Erro"] < 1e-8).all(), "o teorema dos resíduos prevê exatamente o valor em cada faixa")

fig, ax = plt.subplots(figsize=(6.4, 4.0))
for ui, rot, cor, est in [(1.5, "$u_i > 1$: integral = call", AZUL, "-"), (0.5, "$0 < u_i < 1$:
↪   $S_0$ + integral = call", VERDE, "-."), (-0.5, "$u_i < 0$: integral = put", LARANJA, "--")]:
    ax.plot([-3, 3], [ui, ui], color=cor, ls=est, label=rot)
ax.scatter([0, 0], [0, 1], marker="x", s=110, color="#333330", linewidths=2.2, zorder=4)
ax.text(0.12, 1.08, "polo em $u = i$", color="#444440")
ax.text(0.12, 0.08, "polo em $u = 0$", color="#444440")
ax.axhline(0, color=CINZA, lw=0.8)
ax.axvline(0, color=CINZA, lw=0.8)
ax.set_xlim(-3, 3)
ax.set_ylim(-1.1, 3.1)
ax.set_title("O plano complexo da integral de Lewis")
ax.set_xlabel("Re($u$)")
ax.set_ylabel("Im($u$)")
ax.legend(loc="upper right", fontsize=8)
plt.show()
```

u_i	Regra	Resultado	Referência BSM	Erro
2,500	Integral = call	12,757	12,757	0,000
1,500	Integral = call	12,757	12,757	0,000
0,750	S0 + integral = call	12,757	12,757	0,000
0,500	S0 + integral = call	12,757	12,757	0,000
0,250	S0 + integral = call	12,757	12,757	0,000
-0,500	Integral = put	4,032	4,032	0,000
-1,500	Integral = put	4,032	4,032	0,000

```
OK — o teorema dos resíduos prevê exatamente o valor em cada faixa
```



---

4. Avaliando a integral

Livro: §6.4.1, final

Na prática, resta escolher: até onde integrar, e com que malha.

4.1 O integrando

Para BSM: $h(z) = \cos(zk)\, e^{-\frac{1}{2}\sigma^2 T(z^2+1/4)}/(z^2+1/4)$. O decaimento é regido pela combinação $\sigma^2 T$, ou seja, a variância total. Para **prazos curtos**, a densidade em x é estreita, e a transformada em z é larga: é preciso integrar mais longe.

Uma regra simples para truncar em BSM: queremos $e^{-\frac{1}{2}\sigma^2 T U^2} \leq \varepsilon$, ou seja,

$$U \approx \frac{1}{\sigma\sqrt{T}}\sqrt{2\ln(1/\varepsilon)}. \tag{7.32}$$

Para modelos com saltos, a parte de difusão ainda é quem garante o decaimento rápido.

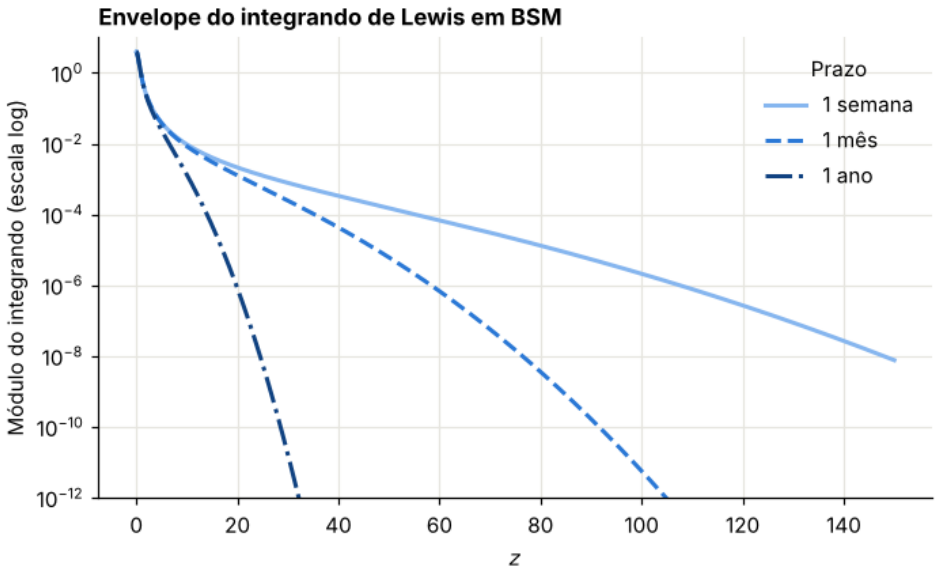
Laboratório 4.1 — O integrando e os limites

```python
def lewis(S, K, T, r, cf, U=None, n=None):
    """Fórmula de Lewis vetorial em K. cf é a função característica de X_T (martingale)."""
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(S / K) + r * T
    z = np.linspace(0.0, U, n)
    h = np.exp(1j * z * k[:, None]) * cf(z - 0.5j)
```

```
    integral = simpson(h.real / (z**2 + 0.25), x=z, axis=1)
    return S - np.sqrt(S * K) * np.exp(-0.5 * r * T) / np.pi * integral


fig, ax = plt.subplots()
z = np.linspace(0, 150, 3001)
for T_, cor, est, rot in zip([1 / 52, 1 / 12, 1.0], RAMPA, ["-", "--", "-."], ["1 semana", "1
↪  mês", "1 ano"]):
    ax.semilogy(z, np.exp(-0.5 * 0.2**2 * T_ * (z**2 + 0.25)) / (z**2 + 0.25), color=cor, ls=est,
↪  label=rot)
ax.set_ylim(1e-12, 10)
ax.set_title("Envelope do integrando de Lewis em BSM")
ax.set_xlabel("$z$")
ax.set_ylabel("Módulo do integrando (escala log)")
ax.legend(title="Prazo")
plt.show()

K_t = np.array([85.0, 100.0, 115.0])
linhas = []
for rot, T_ in [("1 semana", 1 / 52), ("1 mês", 1 / 12), ("6 meses", 0.5), ("1 ano", 1.0), ("2
↪  anos", 2.0)]:
    ref = bsm(100.0, K_t, T_, 0.04, 0.2)
    cf_ = lambda u: cf_bsm(u, T_, 0.2)
    U_min = None
    for U in 2.0 ** np.arange(1, 13):
        v = lewis(100.0, K_t, T_, 0.04, cf_, U=U, n=max(int(U * 20) * 2 + 1, 4001))
        if np.max(np.abs(v - ref)) < 1e-6:
            U_min = U
            break
    linhas.append({"Prazo": rot, "Menor U com erro < 10⁻⁶ (grade de 2ⁿ)": float(U_min), "Regra
↪  1/(σ√T)·√(2·ln(10⁶))": float(np.sqrt(2 * np.log(1e6)) / (0.2 * np.sqrt(T_)))})
tu = pd.DataFrame(linhas)
tabela(tu)
ok(tu.iloc[:, 1].is_monotonic_decreasing, "prazos menores exigem limites de integração maiores")
ok((tu.iloc[:, 1] <= 2.1 * tu.iloc[:, 2]).all(), "a regra de truncamento indica a ordem de
↪  grandeza correta")
```



Prazo	Menor U com erro $< 10^{-6}$ (grade de 2^n)	Regra $1/(\sigma\sqrt{T})\cdot\sqrt{(2\cdot\ln(10^6))}$
1 semana	256,000	189,527
1 mês	128,000	91,046
6 meses	64,000	37,169
1 ano	32,000	26,283
2 anos	32,000	18,585

```
OK — prazos menores exigem limites de integração maiores
OK — a regra de truncamento indica a ordem de grandeza correta
```

Laboratório 4.2 — Velocidade e precisão, com vários modelos

Aplicamos a mesma função a BSM, Merton e Heston com saltos (a função característica deste último é a do Módulo 2, e será deduzida no Módulo 11). Conferimos com referências independentes: BSM, a série de Merton e Monte Carlo. Em seguida, comparamos custos para 1.000 opções.

```python
def merton_serie(S, K, T, r, sigma, lam, mu, delta, n_max=60):
    """Série de Poisson do Módulo 1, referência independente."""
    kap = np.exp(mu + 0.5 * delta**2) - 1
    lq = lam * (1 + kap)
    tot = 0.0
    for n in range(n_max):
        sn = np.sqrt(sigma**2 + n * delta**2 / T)
        rn = r - lam * kap + n * np.log1p(kap) / T
        tot = tot + poisson.pmf(n, lq * T) * bsm(S, K, T, rn, sn)
    return tot


def cf_bates(u, T, v0, kappa, theta, nu, rho, lam, mu, delta):
    comp = lam * (np.exp(mu + 0.5 * delta**2) - 1)
    salto = np.exp(lam * T * (np.exp(1j * u * mu - 0.5 * u**2 * delta**2) - 1))
    deriva = np.exp(-1j * u * comp * T)
    xi = kappa - rho * nu * 1j * u
    d = np.sqrt(xi**2 + nu**2 * (1j * u + u**2))
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
    A = kappa * theta / nu**2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / nu**2 * (1 - e) / (1 - g * e)
    return deriva * salto * np.exp(A + B * v0)


S_, r_, T_ = 100.0, 0.03, 0.5
K_m = np.array([70.0, 85.0, 100.0, 115.0, 130.0])
cf_m = lambda u: cf_merton(u, T_, **M76)
v_lw = lewis(S_, K_m, T_, r_, cf_m, U=250.0, n=20001)
v_sr = merton_serie(S_, K_m, T_, r_, **M76)
tabela(pd.DataFrame({"Strike": K_m, "Lewis": v_lw, "Série de Poisson": v_sr, "Diferença (×10⁶)":
 ↪  (v_lw - v_sr) * 1e6}))
ok(np.max(np.abs(v_lw - v_sr)) < 1e-7, "Lewis coincide com a série de Merton")

B97 = dict(v0=0.04, kappa=2.0, theta=0.04, nu=0.4, rho=-0.6, lam=0.5, mu=-0.1, delta=0.1)
ok(abs(cf_bates(-1j, 1.0, **B97) - 1) < 1e-12, "martingale em Heston com saltos: φ(−i) = 1")

rng = np.random.default_rng(802)
n_p, passos = 400_000, 250
```

```
dt = T_ / passos
comp = B97["lam"] * (np.exp(B97["mu"] + 0.5 * B97["delta"] ** 2) - 1)
v, x_ = np.full(n_p, B97["v0"]), np.zeros(n_p)
for _ in range(passos):
    z1 = rng.standard_normal(n_p)
    z2 = B97["rho"] * z1 + np.sqrt(1 - B97["rho"] ** 2) * rng.standard_normal(n_p)
    vp = np.maximum(v, 0)
    nj = rng.poisson(B97["lam"] * dt, n_p)
    x_ += (r_ - comp - 0.5 * vp) * dt + np.sqrt(vp * dt) * z1 + nj * B97["mu"] + np.sqrt(nj) *
↪   B97["delta"] * rng.standard_normal(n_p)
    v += B97["kappa"] * (B97["theta"] - vp) * dt + B97["nu"] * np.sqrt(vp * dt) * z2
ST = S_ * np.exp(x_)

cf_h = lambda u: cf_bates(u, T_, **B97)
v_h = lewis(S_, K_m, T_, r_, cf_h, U=250.0, n=20001)
linhas = []
for k_, vv in zip(K_m, v_h):
    a = np.exp(-r_ * T_) * np.maximum(ST - k_, 0)
    linhas.append({"Strike": k_, "Lewis": float(vv), "Monte Carlo": float(a.mean()),
↪   "Erro-padrão": float(a.std(ddof=1) / np.sqrt(n_p)), "Distância em erros-padrão":
↪   float((a.mean() - vv) / (a.std(ddof=1) / np.sqrt(n_p)))})
h_ = pd.DataFrame(linhas)
tabela(h_)
ok((h_["Distância em erros-padrão"].abs() < 4).all(), "Lewis confere com Monte Carlo em Heston
↪   com saltos")

K_mil = np.linspace(60, 140, 1000)
t0 = perf_counter(); lewis(S_, K_mil, T_, r_, cf_h, U=200.0, n=4001); t_vet = perf_counter() - t0


def lewis_quad(K):
    k = np.log(S_ / K) + r_ * T_
    f = lambda z: (np.exp(1j * z * k) * cf_h(z - 0.5j)).real / (z**2 + 0.25)
    return S_ - np.sqrt(S_ * K) * np.exp(-0.5 * r_ * T_) / np.pi * quad(f, 0, 200, limit=200)[0]


t0 = perf_counter(); [lewis_quad(k) for k in K_mil[:20]]; t_quad = (perf_counter() - t0) * 50

t0 = perf_counter()
a = np.exp(-r_ * T_) * np.maximum(ST - K_mil[:50, None], 0.0).mean(axis=1)
t_mc = (perf_counter() - t0) * 20

tabela(pd.DataFrame({"Método para 1.000 opções": ["Lewis vetorial", "Lewis com quad adaptativo
↪   (extrapolado)", "Monte Carlo com trajetórias prontas: só o cálculo dos payoffs
↪   (extrapolado)"], "Tempo (ms)": [t_vet * 1e3, t_quad * 1e3, t_mc * 1e3]}))
ok(t_vet < t_quad, "a versão vetorial é bem mais rápida que a integração adaptativa opção a
↪   opção")
```

Strike	Lewis	Série de Poisson	Diferença ($\times 10^6$)
70,000	31,151	31,151	0,000
85,000	17,220	17,220	-0,000
100,000	6,247	6,247	-0,000
115,000	1,165	1,165	0,000
130,000	0,110	0,110	-0,000

```
OK — Lewis coincide com a série de Merton
OK — martingale em Heston com saltos: φ(−i) = 1
```

Strike	Lewis	Monte Carlo	Erro-padrão	Distância em erros-padrão
70,000	31,256	31,234	0,023	-0,948
85,000	17,606	17,589	0,020	-0,848
100,000	6,803	6,796	0,014	-0,505
115,000	1,265	1,265	0,006	-0,054
130,000	0,111	0,111	0,002	-0,023

```
OK — Lewis confere com Monte Carlo em Heston com saltos
```

Método para 1.000 opções	Tempo (ms)
Lewis vetorial	137,090
Lewis com quad adaptativo (extrapolado)	1.373,611
Monte Carlo com trajetórias prontas: só o cálc...	1.989,091

```
OK — a versão vetorial é bem mais rápida que a integração adaptativa opção a opção
```

---

5. Limites de precisão

A fórmula de Lewis calcula o preço como S_0 menos uma grande parcela. Para opções **muito fora do dinheiro**, os dois termos quase se cancelam, e o erro de arredondamento pesa mais no resultado. É um fenômeno clássico de **cancelamento catastrófico**. Numa call, o preço é pequeno; numa put com strike baixo, a paridade agrava o problema.

Laboratório 5.1 — O preço fica pequeno, o erro não

```
Ks = np.array([100.0, 110.0, 120.0, 130.0, 140.0, 150.0])
T_o, r_o, sig_o = 1 / 12, 0.03, 0.2
cf_o = lambda u: cf_bsm(u, T_o, sig_o)
v = lewis(100.0, Ks, T_o, r_o, cf_o, U=600.0, n=60001)
v_ruim = lewis(100.0, Ks, T_o, r_o, cf_o, U=60.0, n=2001)
ref = bsm(100.0, Ks, T_o, r_o, sig_o)
pc = pd.DataFrame({"Strike": Ks, "Preço BSM (×10⁶)": ref * 1e6, "Malha fina: erro (×10¹²)":
↪  np.abs(v - ref) * 1e12, "Malha fina: erro relativo (%)": np.abs(v - ref) / ref * 100, "Malha
↪  grossa: erro relativo (%)": np.abs(v_ruim - ref) / ref * 100})
tabela(pc)
ok(pc["Malha fina: erro (×10¹²)"].max() < 100, "com malha fina, o erro absoluto fica abaixo de
↪  10⁻¹⁰")
ok(pc["Malha grossa: erro relativo (%)"].iloc[-1] > 1 and pc["Malha grossa: erro relativo
↪  (%)"].iloc[0] < 1e-2, "com malha grossa, o erro relativo nas caudas é grande, mesmo sendo
↪  desprezível no dinheiro")
```

Strike	Preço BSM ($\times 10^6$)	Malha fina: erro ($\times 10^{12}$)	Malha fina: erro relativo (%)	Malha grossa: erro relativo (%)
100,000	2.427.098,631	0,028	0,000	0,004
110,000	138.045,658	0,004	0,000	0,060
120,000	1.600,465	0,014	0,000	2,053
130,000	4,541	0,011	0,000	1.204,908
140,000	0,004	0,005	0,000	839.225,289
150,000	0,000	0,006	0,429	4.004.758.061,695

```
OK — com malha fina, o erro absoluto fica abaixo de 10⁻¹⁰
OK — com malha grossa, o erro relativo nas caudas é grande, mesmo sendo desprezível no dinheiro
```

Leitura. O erro absoluto é minúsculo em todos os strikes, e é isso que costuma ser observado. Mas o que importa nas caudas é o erro relativo: com uma malha que parece boa para a opção no dinheiro, o preço de uma opção muito fora do dinheiro pode sair sem nenhum dígito correto. Se você calcular uma volatilidade implícita a partir desse preço, o resultado será ruim. O Módulo 8 apresenta a transformada específica para fora do dinheiro.

---

6. Gregas pela função característica

Como ϕ não depende de S_0, derivadas em S_0 entram só no prefator e em k. Derivando dentro da integral:

$$\frac{\partial}{\partial S_0}\Big[\sqrt{S_0 K}\, e^{izk}\Big] = \sqrt{\frac{K}{S_0}}\, e^{izk}\Big(\frac{1}{2} + iz\Big). \tag{7.33}$$

Logo,

$$\Delta = 1 - \frac{1}{\pi}\sqrt{\frac{K}{S_0}}\, e^{-rT/2} \int_0^\infty \mathrm{Re}\Big[e^{izk}\phi(z - i/2)\big(\tfrac{1}{2} + iz\big)\Big]\frac{dz}{z^2 + 1/4}. \tag{7.34}$$

É a **mesma integral**, com um fator a mais. Custo extra quase zero e sem erro de diferenças finitas.

Laboratório 6.1 — Delta

```python
def delta_lewis(S, K, T, r, cf, U, n):
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(S / K) + r * T
    z = np.linspace(0.0, U, n)
    h = np.exp(1j * z * k[:, None]) * cf(z - 0.5j) * (0.5 + 1j * z)
    integral = simpson(h.real / (z**2 + 0.25), x=z, axis=1)
    return 1 - np.sqrt(K / S) * np.exp(-0.5 * r * T) / np.pi * integral


Kd = np.array([85.0, 100.0, 115.0])
T_d, r_d, s_d = 0.5, 0.03, 0.2
dl = delta_lewis(100.0, Kd, T_d, r_d, lambda u: cf_bsm(u, T_d, s_d), 200.0, 20001)
d1 = (np.log(100.0 / Kd) + (r_d + 0.5 * s_d**2) * T_d) / (s_d * np.sqrt(T_d))
ok(np.max(np.abs(dl - norm.cdf(d1))) < 1e-8, "em BSM, o delta de Lewis é N(d1)")

cf_d = lambda u: cf_bates(u, T_d, **B97)
dl_h = delta_lewis(100.0, Kd, T_d, r_d, cf_d, 300.0, 30001)
h = 0.01
dif = (lewis(100.0 + h, Kd, T_d, r_d, cf_d, U=300.0, n=30001) - lewis(100.0 - h, Kd, T_d, r_d,
↪  cf_d, U=300.0, n=30001)) / (2 * h)
tabela(pd.DataFrame({"Strike": Kd, "Delta integral": dl_h, "Delta por diferenças finitas": dif}))
ok(np.max(np.abs(dl_h - dif)) < 1e-6, "em Heston com saltos, o delta integral coincide com o de
↪  diferenças finitas")
```

```
OK — em BSM, o delta de Lewis é N(d1)
```

Strike	Delta integral	Delta por diferenças finitas
85,000	0,893	0,893
100,000	0,634	0,634
115,000	0,220	0,220

```
OK — em Heston com saltos, o delta integral coincide com o de diferenças finitas
```

---

7. Síntese

- A transformada converte convoluções em produtos. Por isso, ϕ é simples mesmo quando a densidade não é.
- $\phi(-i) = 1$ é a condição de martingale.
- A fórmula de Lewis tem três ingredientes: transformada do payoff em $u_i > 1$, Parseval/Fubini e um resíduo em $u = i$.
- Mover a reta troca call, call menos S_0 e put, exatamente como o teorema dos resíduos prevê.
- Prazos curtos exigem integrar mais longe.
- Nas caudas, o erro relativo domina.
- O delta sai da mesma integral.

8. Notas sobre o livro neste trecho

Local	Comentário
Prova da Prop. 6	a expressão $(z - i/2)^2 - (z - i/2)i = z^2 - 2zi + 1/4$ está errada; o denominador correto, com a reta em $u_i = 1/2$, é $z^2 + 1/4$. A fórmula (6.5) está certa
Geral	a existência de ϕ em argumentos complexos exige uma faixa de analiticidade, que o texto não discute

9. Exercícios

Exercício 1. Mostre que, para $Y = aX + b$, a função característica é $\phi_Y(u) = e^{iub}\phi_X(au)$.

Solução

$\mathbb{E}[e^{iu(aX+b)}] = e^{iub}\,\mathbb{E}[e^{i(au)X}] = e^{iub}\phi_X(au)$.

Exercício 2. Use a relação do Exercício 1 e a função $e^{-u^2/2}$ para obter a função característica de $N(\mu, \sigma^2)$.

Solução

$\phi(u) = e^{iu\mu - \sigma^2 u^2/2}$.

Exercício 3. Calcule a curtose em excesso de X_T no Merton, em função de T, e explique o achatamento do smile com o prazo.

Solução

$c_4/c_2^2 = \lambda T\, m_4/[T(\sigma^2+\lambda m_2)]^2 = \dfrac{\lambda m_4}{T(\sigma^2+\lambda m_2)^2}$, com $m_2 = \mu_J^2 + \delta^2$ e $m_4 = \mu_J^4 + 6\mu_J^2\delta^2 + 3\delta^4$. Cai com $1/T$: em prazos longos, a distribuição se aproxima da normal.

Exercício 4. Por que a escolha $u_i = 1/2$ é natural para a fórmula de Lewis?

Solução

É o meio da faixa entre os polos $u = 0$ e $u = i$. O denominador vira o número real positivo $z^2 + 1/4$, e o integrando fica afastado dos dois polos, o que beneficia a integração numérica.

Exercício 5. Derive, da mesma forma que o delta, uma integral para o gama.

Solução

Derivando de novo: $\dfrac{\partial}{\partial S_0}\left[\sqrt{K/S_0}\,e^{izk}(\tfrac{1}{2}+iz)\right] = \sqrt{K}\,S_0^{-3/2}\,e^{izk}\,(\tfrac{1}{2}+iz)(iz-\tfrac{1}{2})$, e $(\tfrac{1}{2}+iz)(iz-\tfrac{1}{2}) = -(z^2+\tfrac{1}{4})$. O denominador se cancela: $\Gamma = \dfrac{\sqrt{K}\,e^{-rT/2}}{\pi S_0^{3/2}}\displaystyle\int_0^\infty \mathrm{Re}[e^{izk}\phi(z-i/2)]\,dz$.

```
print(f"Módulo 7 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 7 concluído: 22 verificações aprovadas.
```

Capítulo 8

Métodos de Fourier II: Carr-Madan, FFT e convolução

Introdução

A fórmula de Lewis calcula uma integral por strike. Quando se precisa de uma superfície inteira de preços, é mais eficiente calcular muitos strikes de uma vez. O método de **Carr e Madan** reescreve o preço da call, depois de um amortecimento exponencial, como uma transformada de Fourier no log-strike, e essa transformada pode ser calculada para toda uma grade de strikes com a **FFT** (transformada rápida de Fourier), em tempo quase linear.

O preço da velocidade é uma série de escolhas numéricas: o fator de amortecimento, o tamanho da grade, a relação entre o espaçamento em frequência e o espaçamento em strike, a regra de integração. O capítulo estuda cada uma e mostra quando o método erra.

Por fim, a mesma lógica se estende a opções com exercício antecipado em datas discretas: entre duas datas de exercício, o valor de continuação é uma **convolução** da função valor com a densidade do retorno, e convoluções também são calculadas rapidamente por FFT.

Fundamentos teóricos

O método de Carr e Madan

O preço da call como função do log-strike $k = \ln K$, $C_T(k) = e^{-rT}\mathbb{E}^{\mathbb{Q}}[(e^{s_T} - e^k)^+]$ com $s_T = \ln S_T$, não é integrável em k: tende a S_0 quando $k \to -\infty$. Carr e Madan amortecem a função antes de transformá-la (Carr and Madan, 1999).

Proposição 8.1 (Transformada da call amortecida). *Para $\alpha > 0$ com $\mathbb{E}[S_T^{\alpha+1}] < \infty$, a função $c_T(k) = e^{\alpha k} C_T(k)$ é integrável, e sua transformada $\psi_T(v) = \int e^{ivk} c_T(k)\, dk$ vale*

$$\psi_T(v) = \frac{e^{-rT}\,\varphi_{s_T}(v - (\alpha+1)i)}{\alpha^2 + \alpha - v^2 + i(2\alpha+1)v}. \tag{8.1}$$

Logo

$$C_T(k) = \frac{e^{-\alpha k}}{\pi} \int_0^{\infty} \operatorname{Re}\left[e^{-ivk}\psi_T(v)\right] dv. \tag{8.2}$$

Demonstração. Troque a ordem das integrais (Fubini, justificado pela condição de momento): $\psi_T(v) = e^{-rT} \int q(s) \int_{-\infty}^{s} e^{(\alpha+iv)k}(e^s - e^k)\, dk\, ds$, onde q é a densidade de s_T. A integral interna vale $e^{(\alpha+1+iv)s} \big/ ((\alpha+iv)(\alpha+1+iv))$, e a integral externa é $\varphi_{s_T}(v-(\alpha+1)i)$. O denominador expandido é (8.1). A inversão usa que c_T é real, de modo que $\psi_T(-v) = \overline{\psi_T(v)}$. ■

Observação 8.2 (Escolha de α). α grande demais viola a condição de momento ou amplifica erros numéricos em $e^{-\alpha k}$; pequeno demais deixa c_T pouco amortecida e a integral mal condicionada. Valores entre 1 e 2 funcionam para os modelos do livro. Para opções muito fora do dinheiro, Carr e Madan propõem uma variante com sinh que evita o cancelamento.

Da integral à FFT

Definição 8.3 (Transformada discreta de Fourier). Para $x \in \mathbb{C}^N$, a DFT é $X_m = \sum_{j=0}^{N-1} e^{-2\pi i j m/N} x_j$, $m = 0, \dots, N-1$. A FFT calcula todos os X_m em $O(N \log N)$ operações (Cooley and Tukey, 1965), contra $O(N^2)$ da soma direta.

Proposição 8.4 (Discretização de Carr-Madan). *Com a malha $v_j = \eta j$ e a malha de log-strikes $k_m = -b + \lambda m$, $m = 0, \dots, N-1$, a regra de quadratura com pesos w_j dá*

$$C_T(k_m) \approx \frac{e^{-\alpha k_m}}{\pi} \operatorname{Re} \sum_{j=0}^{N-1} e^{-2\pi i j m/N}\, e^{ibv_j} \psi_T(v_j)\, \eta\, w_j, \tag{8.3}$$

que é uma DFT se, e somente se, $\lambda\eta = 2\pi/N$. Com $b = N\lambda/2$, a malha de strikes fica centrada no dinheiro.

A restrição $\lambda\eta = 2\pi/N$ é o compromisso central do método: uma malha fina em v (pequeno η, integral precisa) força uma malha grossa em k (strikes espaçados), e vice-versa. Com pesos de Simpson, $w_j = \frac{1}{3}(3 + (-1)^{j+1} - \delta_{j0})$, o erro de quadratura cai de $O(\eta^2)$ para $O(\eta^4)$ em integrandos suaves.

Convolução e exercício antecipado

Proposição 8.5 (Valor de continuação como convolução). *Num modelo com incrementos independentes do log-preço (Lévy), o valor de continuação de uma opção bermudana entre duas datas de exercício é*

$$C(t_m, x) = e^{-r\Delta t} \int V(t_{m+1}, x + y)\, f(y)\, dy, \tag{8.4}$$

uma convolução da função valor com a densidade f do incremento. No domínio de Fourier, a convolução vira produto: $\widehat{C} = e^{-r\Delta t}\, \widehat{V} \cdot \overline{\varphi}$, calculável com duas FFTs por data (Lord et al., 2008).

Esse é o método de convolução (CONV): ele herda a velocidade da FFT e trata o exercício antecipado tomando o máximo entre o valor de continuação e o payoff em cada

data. A precisão depende do amortecimento, do tamanho do domínio e da malha, os mesmos parâmetros de Carr-Madan.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** seções 6.4.2 a 6.6 do livro.

O que você vai conseguir fazer ao final

1. Deduzir a fórmula de Carr-Madan e ver que ela é a fórmula de Lewis em outra reta.
2. Escolher o parâmetro de amortecimento α.
3. Tratar opções muito fora do dinheiro com a transformada do valor no tempo.
4. Entender a transformada discreta, as raízes da unidade e o algoritmo FFT.
5. Precificar uma grade inteira de strikes com FFT, e saber quando isso compensa.
6. Ver a árvore binomial como uma convolução.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
from time import perf_counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import simpson
from scipy.interpolate import CubicSpline
from scipy.optimize import minimize_scalar
from scipy.stats import norm, poisson
from IPython.display import display

def bsm(S, K, T, r, sig, tipo="call"):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    d2 = d1 - sig * np.sqrt(T)
    if tipo == "call":
        return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)

def cf_bsm(u, T, sigma):
    """Função característica de X_T, parte martingale do log-retorno."""
    return np.exp(-0.5 * sigma**2 * T * (1j * u + u**2))

def cf_merton(u, T, sigma, lam, mu, delta):
    omega = -0.5 * sigma**2 - lam * (np.exp(mu + 0.5 * delta**2) - 1)
    return np.exp(T * (1j * u * omega - 0.5 * sigma**2 * u**2 + lam * (np.exp(1j * u * mu - 0.5 *
    ↪  u**2 * delta**2) - 1)))

def lewis(S, K, T, r, cf, U, n):
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(S / K) + r * T
    z = np.linspace(0.0, U, n)
```

```
    h = np.exp(1j * z * k[:, None]) * cf(z - 0.5j)
    return S - np.sqrt(S * K) * np.exp(-0.5 * r * T) / np.pi * simpson(h.real / (z**2 + 0.25),
    ↪  x=z, axis=1)

S0, K0, T0, r0, s0 = 100.0, 100.0, 1.0, 0.05, 0.20
print("Ambiente pronto. Call BSM:", f3(float(bsm(S0, K0, T0, r0, s0))))
```

```
Ambiente pronto. Call BSM: 10,451
```

---

1. A abordagem de Carr-Madan

Livro: §6.4.2

1.1 Mudança de ponto de vista

Lewis transformou o payoff em relação ao log-preço. Carr e Madan (1999) transformam **o preço da opção em relação ao log-strike** $k = \ln K$. Se a integral for sobre os strikes, a inversão entrega o preço para muitos strikes simultaneamente, e é isso que permitirá usar a FFT.

1.2 O problema da integrabilidade

Quando $k \to -\infty$, a call tende a S_0, e não a zero. A função $C(k)$ não é integrável. A solução é amortecer:

$$c(k) = e^{\alpha k}\, C(k), \qquad \alpha > 0. \tag{8.5}$$

1.3 Dedução

Seja $s = \ln S_T$ e q sua densidade. A transformada de c é

$$\psi(v) = \int_{-\infty}^{\infty} e^{ivk}\, e^{\alpha k}\, e^{-rT} \int_{k}^{\infty} (e^{s} - e^{k})\, q(s)\, ds\, dk. \tag{8.6}$$

Passo 1: trocar a ordem (Fubini). A região $\{s > k\}$ equivale a $k \in (-\infty, s)$:

$$\psi(v) = e^{-rT} \int_{-\infty}^{\infty} q(s) \Big[\int_{-\infty}^{s} e^{(iv+\alpha)k} (e^{s} - e^{k})\, dk \Big] ds. \tag{8.7}$$

Passo 2: integrar em k. Como $\alpha > 0$, o limite inferior anula:

$$\int_{-\infty}^{s} e^{(iv+\alpha)k} (e^{s} - e^{k})\, dk = e^{s} \frac{e^{(iv+\alpha)s}}{iv+\alpha} - \frac{e^{(iv+\alpha+1)s}}{iv+\alpha+1} = \frac{e^{(iv+\alpha+1)s}}{(iv+\alpha)(iv+\alpha+1)}. \tag{8.8}$$

Passo 3: reconhecer a função característica. Note que $e^{(iv+\alpha+1)s} = e^{i(v-(\alpha+1)i)s}$. Multiplicando o denominador:

$$\boxed{\psi(v) = \frac{e^{-rT}\, \phi_s(v - (\alpha+1)i)}{\alpha^2 + \alpha - v^2 + i(2\alpha+1)v}.} \tag{8.9}$$

Passo 4: inverter. Como $C(k)$ é real, a transformada satisfaz $\psi(-v) = \overline{\psi(v)}$, e

$$C(k) = \frac{e^{-\alpha k}}{\pi} \int_0^{\infty} \mathrm{Re}\left[e^{-ivk}\psi(v)\right] dv. \tag{8.10}$$

(O livro omite o Re na equação (6.6), mas ele é necessário.)

1.4 Quem é Carr-Madan, em relação a Lewis?

Observe o argumento de ϕ: $v - (\alpha + 1)i$. Compare com a integral de Lewis, $\phi(-u)$ em $u = z + iu_i$, com $u_i > 1$. Fazendo $u = -v + i(\alpha + 1)$, **é a mesma integral, numa reta com** $u_i = \alpha + 1$. Carr-Madan é a fórmula original de Lewis antes de atravessar o polo em $u = i$, e com um nome diferente para u_i.

1.5 O parâmetro α

Há duas restrições.

- **Limite superior:** $\phi_s(v - (\alpha + 1)i)$ precisa existir, ou seja, $\mathbb{E}[S_T^{\alpha+1}] < \infty$. Em Heston, o momento pode explodir.
- **Precisão:** α pequeno deixa o integrando próximo do polo em $v = 0$; α grande amplifica $e^{-\alpha k}$.

Carr e Madan sugerem algo em torno de $\frac{1}{4}$ do limite; o livro usa $\alpha = 1{,}5$.

Laboratório 1.1 — Carr-Madan em uma reta só

```
def carr_madan(S, K, T, r, cf, alpha, U=400.0, n=40001):
    """Carr-Madan com integração direta. cf é a função característica de X_T."""
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(K / S)
    v = np.linspace(0.0, U, n)
    w = v - (alpha + 1) * 1j
    fi = np.exp(1j * w * r * T) * cf(w)
    psi = np.exp(-r * T) * fi / (alpha**2 + alpha - v**2 + 1j * (2 * alpha + 1) * v)
    return S * np.exp(-alpha * k) / np.pi * simpson((np.exp(-1j * v * k[:, None]) * psi).real,
    ↪  x=v, axis=1)


Kt = np.array([80.0, 100.0, 120.0])
ref = bsm(S0, Kt, T0, r0, s0)
cf_ = lambda u: cf_bsm(u, T0, s0)
linhas = []
for al in [0.1, 0.25, 0.5, 1.0, 1.5, 3.0, 6.0, 12.0, 25.0]:
    v = carr_madan(S0, Kt, T0, r0, cf_, al, U=80.0, n=1601)
    linhas.append({"α": al, "Erro K=80 (×10⁶)": abs(v[0] - ref[0]) * 1e6, "Erro K=100 (×10⁶)":
↪  abs(v[1] - ref[1]) * 1e6, "Erro K=120 (×10⁶)": abs(v[2] - ref[2]) * 1e6})
al_t = pd.DataFrame(linhas)
tabela(al_t)
v_fina = carr_madan(S0, Kt, T0, r0, cf_, 1.5)
v_lw = lewis(S0, Kt, T0, r0, cf_, 400.0, 40001)
print("Carr-Madan (α = 1,5):", [f3(float(x)) for x in v_fina], "| Lewis:", [f3(float(x)) for x in
↪  v_lw])

ok(np.max(np.abs(v_fina - ref)) < 1e-9 and np.max(np.abs(v_lw - ref)) < 1e-9, "Carr-Madan e Lewis
↪  coincidem com o BSM")
```

```
ok(al_t["Erro K=100 (×10⁶)"].iloc[0] > 10 * al_t["Erro K=100 (×10⁶)"].iloc[4], "α muito pequeno
↪  aumenta o erro, com a mesma malha")
ok(al_t["Erro K=100 (×10⁶)"].iloc[-1] > 10 * al_t["Erro K=100 (×10⁶)"].iloc[4], "α muito grande
↪  também aumenta o erro")
```

α	**Erro K=80 ($\times 10^6$)**	**Erro K=100 ($\times 10^6$)**	**Erro K=120 ($\times 10^6$)**
0,100	61.899,573	61.899,573	61.899,573
0,250	5,023	5,023	5,023
0,500	0,000	0,000	0,000
1,000	0,000	0,000	0,000
1,500	0,000	0,000	0,000
3,000	0,000	0,000	0,000
6,000	0,000	0,000	0,000
12,000	0,000	0,000	0,000
25,000	0,001	0,000	0,000

```
Carr-Madan (α = 1,5): ['24,589', '10,451', '3,247'] | Lewis: ['24,589', '10,451', '3,247']
OK — Carr-Madan e Lewis coincidem com o BSM
OK — α muito pequeno aumenta o erro, com a mesma malha
OK — α muito grande também aumenta o erro
```

1.6 Fora do dinheiro: transformar o valor no tempo

Vimos no Módulo 7 que uma call muito fora do dinheiro é obtida como a diferença de dois números grandes. Carr e Madan propõem transformar diretamente o **valor no tempo**, normalizado com $S_0 = 1$:

$$z(k) = \begin{cases} \text{put de strike } e^k, & k < 0, \\ \text{call de strike } e^k, & k > 0. \end{cases} \tag{8.11}$$

Sua transformada é

$$\varsigma(v) = e^{-rT}\Big(\frac{1}{1+iv} - \frac{e^{rT}}{iv} - \frac{\phi_s(v-i)}{v^2 - iv}\Big). \tag{8.12}$$

Como z tem pico em $k = 0$, um amortecimento $\sinh(\alpha k)$ ajuda:

$$\gamma(v) = \frac{\varsigma(v - i\alpha) - \varsigma(v + i\alpha)}{2}, \qquad z(k) = \frac{1}{\sinh(\alpha k)}\,\frac{1}{2\pi}\int_{-\infty}^{\infty} e^{-ivk}\gamma(v)\,dv. \tag{8.13}$$

Mas há um custo. Os dois primeiros termos de ς decaem apenas como $1/v$, e γ, como $1/v^2$, mesmo que ϕ caia rapidamente. Truncar a integral custa caro.

Uma alternativa simples é manter a transformada da call e escolher α **por strike**. Lord e Kahl propõem o valor que minimiza o módulo do integrando em $v = 0$, ou seja, que coloca a reta de integração perto do ponto de sela. Sem subtração de números grandes e sem cauda lenta.

Laboratório 1.2 — Precisão em opções muito fora do dinheiro

Repetimos o caso difícil do Módulo 7: prazo de um mês, malha grossa ($U = 60$), strikes longe do dinheiro. Depois, repetimos a transformada OTM com $U = 1.000$.

```python
def valor_tempo(S, K, T, r, cf, alpha=1.1, U=60.0, n=2001):
    """Valor no tempo pela transformada OTM de Carr-Madan (put se K<S, call se K>S)."""
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(K / S)
    v = np.linspace(-U, U, 2 * n - 1)

    def vs(w):
        w_i = w - 1j
        fi = np.exp(1j * w_i * r * T) * cf(w_i)
        return np.exp(-r * T) * (1 / (1 + 1j * w) - np.exp(r * T) / (1j * w) - fi / (w**2 - 1j * w))

    gama = 0.5 * (vs(v - 1j * alpha) - vs(v + 1j * alpha))
    integ = simpson((np.exp(-1j * v * k[:, None]) * gama).real, x=v, axis=1)
    return S * integ / (2 * np.pi * np.sinh(alpha * k))


Ks = np.array([110.0, 120.0, 130.0, 140.0, 150.0])
T_o, r_o, sig_o = 1 / 12, 0.03, 0.2
cf_o = lambda u: cf_bsm(u, T_o, sig_o)
ref = bsm(100.0, Ks, T_o, r_o, sig_o)
v_lewis = lewis(100.0, Ks, T_o, r_o, cf_o, 60.0, 2001)
v_otm = valor_tempo(100.0, Ks, T_o, r_o, cf_o, U=60.0, n=2001)
v_otm_g = valor_tempo(100.0, Ks, T_o, r_o, cf_o, U=1000.0, n=100001)


def alfa_otimo(S, K, T, r, cf, a_max=500.0):
    """Escolhe α minimizando o módulo do integrando em v = 0 (Lord e Kahl)."""
    k = np.log(K / S)

    def objetivo(a):
        w = -(a + 1) * 1j
        return -a * k + np.log(np.abs(np.exp(1j * w * r * T) * cf(w))) - np.log(a**2 + a)

    return minimize_scalar(objetivo, bounds=(0.1, a_max), method="bounded").x


alfas = np.array([alfa_otimo(100.0, K_, T_o, r_o, cf_o) for K_ in Ks])
v_cm_f = carr_madan(100.0, Ks, T_o, r_o, cf_o, 1.5, U=60.0, n=2001)
v_cm_o = np.array([carr_madan(100.0, K_, T_o, r_o, cf_o, a_, U=60.0, n=2001)[0] for K_, a_ in
 ↪  zip(Ks, alfas)])
cmp_ = pd.DataFrame({"Strike": Ks, "Preço BSM (×10⁶)": ref * 1e6, "α ótimo": alfas, "Lewis: erro
 ↪  relativo (%)": np.abs(v_lewis - ref) / ref * 100, "Carr-Madan α=1,5: erro (%)": np.abs(v_cm_f
 ↪  - ref) / ref * 100, "Carr-Madan α ótimo: erro (%)": np.abs(v_cm_o - ref) / ref * 100, "OTM,
 ↪  U=60: erro (%)": np.abs(v_otm - ref) / ref * 100, "OTM, U=1000: erro (%)": np.abs(v_otm_g -
 ↪  ref) / ref * 100})
tabela(cmp_)
ok(np.all(np.abs(v_cm_o - ref) / ref < 1e-3), "com α escolhido por strike, o erro relativo fica
 ↪  abaixo de 0,1% em toda a cauda")
ok(abs(v_lewis[-1] - ref[-1]) / ref[-1] > 1e4 * abs(v_cm_o[-1] - ref[-1]) / ref[-1], "no strike
 ↪  mais distante, o α ótimo é muito melhor que Lewis")
ok(np.all(np.abs(v_otm_g - ref) < np.abs(v_otm - ref) / 100), "a transformada OTM está certa, mas
 ↪  exige um limite de integração muito maior")
```

Strike	Preço BSM ($\times 10^6$)	α ótimo	Lewis: erro relativo (%)	Carr-Madan α=1,5: erro (%)	Carr-Madan α ótimo: erro (%)	OTM, U=60: erro (%)	OTM, U=1000: erro (%)
110,000	138.045,658	41,598	0,060	0,036	0,009	9,357	0,008
120,000	1.600,465	62,909	2,053	0,221	0,025	204,065	0,133
130,000	4,541	84,517	1.204,908	929,709	0,034	87.344,076	24,496
140,000	0,004	105,360	839.225,289	71.940,607	0,040	27.553.573,988	18.100,411
150,000	0,000	125,164	4.004.758.061,695	1.684.406.833,965	0,043	61.848.295.674,124	31.735.471,218

```
OK — com α escolhido por strike, o erro relativo fica abaixo de 0,1% em toda a cauda
OK — no strike mais distante, o α ótimo é muito melhor que Lewis
OK — a transformada OTM está certa, mas exige um limite de integração muito maior
```

Leitura. A coluna de α fixo mostra que, com malha grossa, Carr-Madan sofre o mesmo problema de Lewis. A transformada OTM está correta, e melhora muito com U, mas a cauda lenta a torna cara. A escolha de α por strike é a solução mais econômica, e o α ótimo cresce com a distância do strike.

---

2. Da integral à soma: séries de Fourier

Livro: §6.5.1

2.1 Série de Fourier

Uma função periódica em $[-L, L]$ se escreve como soma de oscilações:

$$f(x) = \sum_{n=-\infty}^{\infty} c_n \, e^{in\pi x/L}, \qquad c_n = \frac{1}{2L} \int_{-L}^{L} f(x) \, e^{-in\pi x/L} \, dx. \tag{8.14}$$

Quando $L \to \infty$, as frequências $k_n = n\pi/L$ ficam cada vez mais próximas, a soma vira integral e chegamos à transformada de Fourier do Módulo 7.

> **Errata do livro:** ao apresentar a forma complexa, o texto escreve $e^{iux} = e^{u_r x}\cos(u_r x) + ie^{u_r x}\sin(u_i x)$. O correto é $e^{iux} = e^{-u_i x}\big[\cos(u_r x) + i\sin(u_r x)\big]$: o fator exponencial vem da parte imaginária e o oscilador, da parte real.

2.2 Regularidade e velocidade

A rapidez com que os coeficientes caem depende da suavidade de f:

Função	Decaimento de c_n
Com salto	$1/n$
Contínua, com quina	$1/n^2$
Suave	Mais rápido que qualquer potência

Para $f(x) = |x|$ em $[-\pi, \pi]$, com quina, a série é

$$|x| = \frac{\pi}{2} + \sum_{n=1}^{\infty} \frac{2((-1)^n - 1)}{\pi n^2} \cos nx. \tag{8.15}$$

Esse quadro explica o que vimos nas árvores (Módulo 6): payoff com quina (call) converge melhor do que payoff com salto (digital).

Laboratório 2.1 — Aproximando funções

```
x = np.linspace(-np.pi, np.pi, 8001)

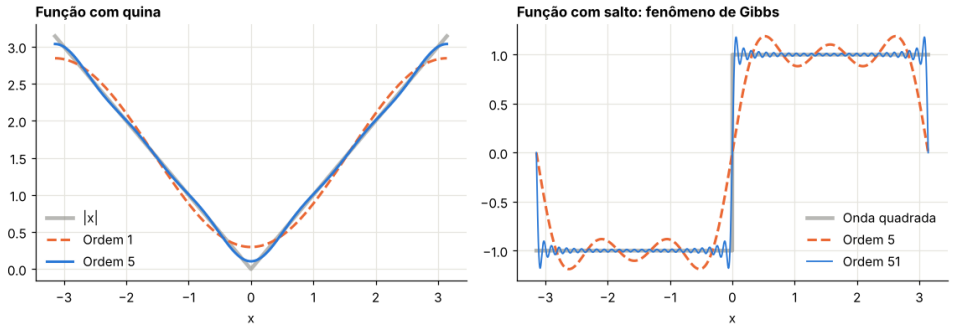

def serie_abs(x, n):
    soma = np.full_like(x, np.pi / 2)
    for m in range(1, n + 1):
```

```
        soma += 2 * ((-1) ** m - 1) / (np.pi * m**2) * np.cos(m * x)
    return soma


def serie_quad(x, n):
    soma = np.zeros_like(x)
    for m in range(1, n + 1, 2):
        soma += 4 / (np.pi * m) * np.sin(m * x)
    return soma


fig, (a1, a2) = plt.subplots(1, 2, figsize=(10.5, 3.8))
a1.plot(x, np.abs(x), color=CINZA, lw=3, alpha=0.6, label="|x|")
a1.plot(x, serie_abs(x, 1), color=LARANJA, ls="--", label="Ordem 1")
a1.plot(x, serie_abs(x, 5), color=AZUL, label="Ordem 5")
a1.set_title("Função com quina")
a1.set_xlabel("x")
a1.legend()
a2.plot(x, np.sign(x), color=CINZA, lw=3, alpha=0.6, label="Onda quadrada")
a2.plot(x, serie_quad(x, 5), color=LARANJA, ls="--", label="Ordem 5")
a2.plot(x, serie_quad(x, 51), color=AZUL, lw=1.2, label="Ordem 51")
a2.set_title("Função com salto: fenômeno de Gibbs")
a2.set_xlabel("x")
a2.legend(loc="lower right")
plt.tight_layout()
plt.show()

linhas = []
for n in [5, 25, 125, 625]:
    e_abs = np.max(np.abs(serie_abs(x, n) - np.abs(x)))
    e_q = np.max(serie_quad(x, n) - 1)
    linhas.append({"Ordem": n, "Erro máximo: |x|": float(e_abs), "Pico acima de 1 na onda
↪  quadrada": float(e_q)})
s_ = pd.DataFrame(linhas)
tabela(s_)
ok(s_["Erro máximo: |x|"].iloc[-1] < s_["Erro máximo: |x|"].iloc[0] / 50, "a série converge
↪  uniformemente na função com quina")
ok(s_["Pico acima de 1 na onda quadrada"].iloc[-1] > 0.15, "na função com salto, o pico não some
↪  ao aumentar a ordem")
```



Ordem	Erro máximo: |x|	Pico acima de 1 na onda quadrada
5	0,105	0,188
25	0,024	0,179
125	0,005	0,179
625	0,001	0,175

```
OK — a série converge uniformemente na função com quina
OK — na função com salto, o pico não some ao aumentar a ordem
```

---

3. Transformada discreta e FFT

Livro: §6.5.2 e §6.6.3

3.1 Raízes da unidade

Seja $z_n = e^{-2\pi i/n}$. As potências $z_n^0, z_n^1, \dots, z_n^{n-1}$ são n pontos igualmente espaçados no círculo unitário. Tem-se a soma geométrica:

$$\sum_{j=0}^{n-1} \left(z_n^k\right)^j = \begin{cases} n, & k \equiv 0 \pmod n, \\ 0, & \text{caso contrário.} \end{cases} \tag{8.16}$$

Prova: se $w = z_n^k \neq 1$, a soma é $(w^n - 1)/(w - 1) = 0$, pois $w^n = 1$. Se $w = 1$, cada termo vale 1. Essa é a **ortogonalidade** que faz tudo funcionar.

3.2 A DFT e sua inversa

Dada a sequência $a_0, \dots, a_{n-1}$, sua transformada discreta é

$$b_k = \sum_{j=0}^{n-1} a_j \, z_n^{kj}, \qquad \text{e a inversa,} \qquad a_l = \frac{1}{n} \sum_{k=0}^{n-1} b_k \, z_n^{-lk}. \tag{8.17}$$

A prova da inversa é uma aplicação direta da ortogonalidade: substituindo b_k, obtém-se $\frac{1}{n}\sum_j a_j \sum_k z_n^{k(j-l)}$, em que a soma interna vale n se $j = l$ e 0 se não.

> **Erratas do livro.** (i) As somas das raízes são escritas a partir de $j = 1$ até $n - 1$; para valerem 0 ou n, devem começar em $j = 0$. (ii) A definição da inversa está sem o fator $1/n$, indispensável, e presente implicitamente nas fórmulas seguintes. (iii) Com $z_n = e^{-2\pi i/n}$, as potências positivas giram em sentido **horário**, e não anti-horário como diz o texto.

3.3 Por que é “rápida”

A DFT direta custa $O(n^2)$. O algoritmo de **Cooley-Tukey** usa n par e separa os termos pares e ímpares:

$$b_k = \sum_{j=0}^{n/2-1} a_{2j} \, z_n^{2jk} + z_n^k \sum_{j=0}^{n/2-1} a_{2j+1} \, z_n^{2jk}. \tag{8.18}$$

Como $z_n^2 = z_{n/2}$, **os dois somatórios são DFTs de tamanho** $n/2$. Além disso, $z_n^{k+n/2} = -z_n^k$, de modo que a segunda metade dos resultados reutiliza os mesmos cálculos. A recursão dá $T(n) = 2T(n/2) + O(n)$, ou seja, $O(n \log n)$.

Laboratório 3.1 — As três coisas

```
fig, eixos = plt.subplots(1, 2, figsize=(8.4, 4.2))
for ax, n in zip(eixos, [5, 30]):
    z = np.exp(-2j * np.pi * np.arange(n) / n)
    for p in z:
        ax.plot([0, p.real], [0, p.imag], color=AZUL, lw=1)
    ax.scatter(z.real, z.imag, color=LARANJA, s=30, zorder=3)
    ax.set_aspect("equal")
    ax.set_title(f"Raízes da unidade, n = {n}")
    ax.set_xlabel("Parte real")
    ax.set_ylabel("Parte imaginária")
plt.tight_layout()
plt.show()

n = 12
soma_0 = np.sum(np.exp(-2j * np.pi * 0 * np.arange(n) / n))
somas = [abs(np.sum(np.exp(-2j * np.pi * k * np.arange(n) / n))) for k in range(1, n)]
ok(abs(soma_0 - n) < 1e-12 and max(somas) < 1e-12, "ortogonalidade: a soma vale n para k = 0 e
↪  zero nos demais")
parcial = abs(np.sum(np.exp(-2j * np.pi * 1 * np.arange(1, n) / n)))
print("Soma de j = 1 a n−1, para k = 1 (como no livro):", f3(float(parcial)), "(e não 0)")
ok(abs(parcial - 1) < 1e-12, "a soma a partir de j = 1 vale 1 em módulo, e não zero")

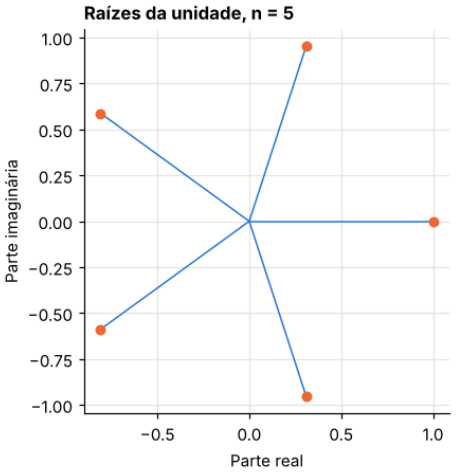

def dft_direta(a):
    n = a.size
    z = np.exp(-2j * np.pi * np.arange(n)[:, None] * np.arange(n) / n)
    return z @ a

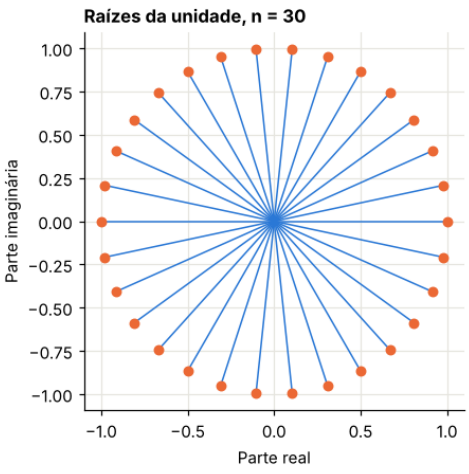

def fft_recursiva(a):
    n = a.size
    if n == 1:
        return a.copy()
    pares, impares = fft_recursiva(a[::2]), fft_recursiva(a[1::2])
    w = np.exp(-2j * np.pi * np.arange(n // 2) / n) * impares
    return np.concatenate([pares + w, pares - w])


rng = np.random.default_rng(901)
a = rng.standard_normal(256) + 1j * rng.standard_normal(256)
ok(np.allclose(dft_direta(a), np.fft.fft(a)) and np.allclose(fft_recursiva(a), np.fft.fft(a)),
↪  "DFT direta, recursiva e numpy coincidem")
ok(np.allclose(np.fft.ifft(np.fft.fft(a)), a), "a inversa, com o fator 1/n, recupera a sequência")
ok(not np.allclose(np.fft.ifft(np.fft.fft(a)) * a.size, a), "sem o fator 1/n, a inversa não
↪  recupera a sequência")

linhas = []
for n in [256, 1024, 4096]:
    a = rng.standard_normal(n) + 0j
    regs = {}
    for nome, f in [("Direta", dft_direta), ("Recursiva", fft_recursiva), ("numpy", np.fft.fft)]:
        t0 = perf_counter()
        f(a)
        regs[nome] = (perf_counter() - t0) * 1e3
    linhas.append({"n": n, "Direta (ms)": regs["Direta"], "FFT em Python (ms)": regs["Recursiva"],
↪  "FFT numpy (ms)": regs["numpy"]})
vel = pd.DataFrame(linhas)
tabela(vel)
```

```
ok(vel["FFT numpy (ms)"].iloc[-1] < vel["Direta (ms)"].iloc[-1] / 20, "a FFT é muito mais rápida
↪  que a DFT direta")
```






```
OK — ortogonalidade: a soma vale n para k = 0 e zero nos demais
Soma de j = 1 a n−1, para k = 1 (como no livro): 1,000 (e não 0)
OK — a soma a partir de j = 1 vale 1 em módulo, e não zero
OK — DFT direta, recursiva e numpy coincidem
OK — a inversa, com o fator 1/n, recupera a sequência
OK — sem o fator 1/n, a inversa não recupera a sequência
```

n	Direta (ms)	FFT em Python (ms)	FFT numpy (ms)
256	1,725	1,078	0,022
1024	43,040	4,069	0,035
4096	512,657	16,439	0,073

```
OK — a FFT é muito mais rápida que a DFT direta
```

---

4. Precificando uma grade de strikes com FFT

Livro: §6.5.2 e §6.6.1

4.1 Passo a passo

Passo 1: discretizar a integral. Com $v_j = \eta j$, para $j = 0, \dots, N-1$:

$$C(k) \approx \frac{e^{-\alpha k}}{\pi} \sum_{j=0}^{N-1} e^{-iv_j k}\, \psi(v_j)\, \eta. \tag{8.19}$$

Passo 2: escolher uma grade de log-strikes. $k_u = -b + \varepsilon u$, para $u = 0, \dots, N-1$, com $b = \frac{1}{2} N\varepsilon$.

Passo 3: separar o produto. $e^{-iv_j k_u} = e^{ibv_j}\, e^{-i\varepsilon\eta j u}$.

Passo 4: forçar o formato da FFT. Para que $e^{-i\varepsilon\eta ju} = e^{-2\pi iju/N}$, impomos

$$\varepsilon\,\eta = \frac{2\pi}{N}. \tag{8.20}$$

Essa é a restrição de todo o método: uma grade fina em v (preciso na integral) implica grade grossa em k (poucos strikes úteis), e vice-versa.

Passo 5: acrescentar pesos de Simpson, $\frac{\eta}{3}(3 + (-1)^{j+1} - \delta_{j,0})$, com j a partir de 0, os pesos sendo $1, 4, 2, 4, \ldots$. Resulta

$$C(k_u) \approx \frac{e^{-\alpha k_u}}{\pi}\,\mathrm{Re}\sum_{j=0}^{N-1} e^{-2\pi iju/N}\, e^{ibv_j}\,\psi(v_j)\,\frac{\eta}{3}(3 + (-1)^{j+1} - \delta_{j,0}). \tag{8.21}$$

4.2 Quando usar

O atrativo é ter N preços ao custo de uma FFT. Mas, como o livro observa citando Kahl, as cotações de mercado raramente estão nos pontos da grade. Em calibração, isso acrescenta um erro de interpolação. Em geral, a integração direta (Lewis) é mais adequada; a FFT é mais útil para precificar densidades inteiras.

Laboratório 4.1 — FFT com os parâmetros do livro

Usamos $N = 4096$ e $\varepsilon = 1/150$, como no livro, que relata cerca de cinco dígitos para a FFT, contra 12 da integração direta. Há uma explicação: o script do livro usa a transformada OTM quando $S_0 < 0{,}95K$, e, como vimos, essa transformada tem cauda lenta. Com $v_{\max} \approx 942$, o erro absoluto fica em torno de 10^{-5}. Aqui usamos a transformada da call em toda a grade.

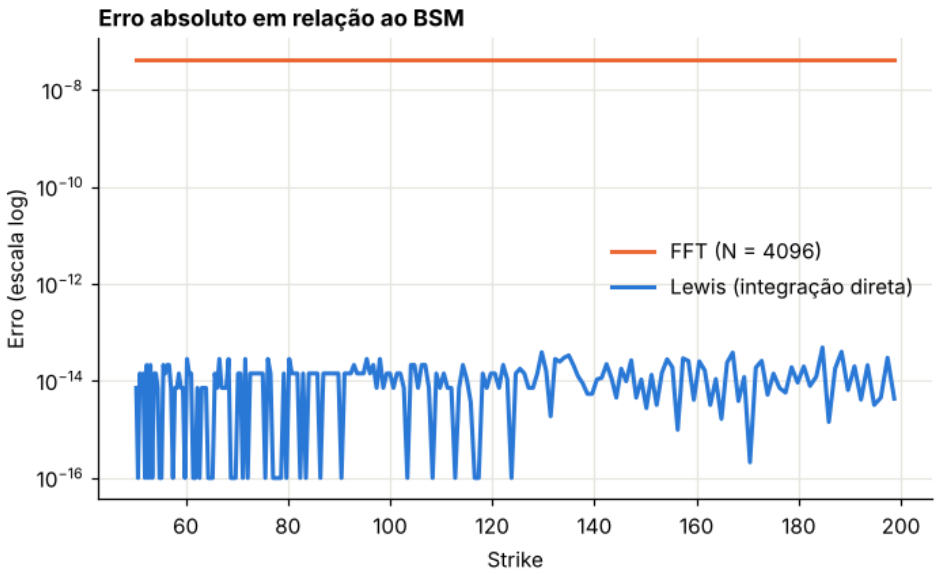
```
def cm_fft(S, T, r, cf, alpha=1.5, N=4096, eps=1 / 150, k_ref=0.0):
    """Devolve grade de strikes e calls. k_ref coloca ln(K/S) = k_ref em u = N/2."""
    eta = 2 * np.pi / (N * eps)
    b = 0.5 * N * eps - k_ref
    j = np.arange(N)
    v = eta * j
    w = v - (alpha + 1) * 1j
    fi = np.exp(1j * w * r * T) * cf(w)
    psi = np.exp(-r * T) * fi / (alpha**2 + alpha - v**2 + 1j * (2 * alpha + 1) * v)
    peso = (3 + (-1.0) ** (j + 1) - (j == 0)) * eta / 3
    y = np.fft.fft(np.exp(1j * b * v) * psi * peso).real
    k = -b + eps * j
    return S * np.exp(k), S * np.exp(-alpha * k) / np.pi * y


Kg, Cg = cm_fft(S0, T0, r0, lambda u: cf_bsm(u, T0, s0))
mascara = (Kg > 50) & (Kg < 200)
er_fft = np.abs(Cg[mascara] - bsm(S0, Kg[mascara], T0, r0, s0))
er_lw = np.abs(lewis(S0, Kg[mascara], T0, r0, lambda u: cf_bsm(u, T0, s0), 400.0, 40001) - bsm(S0,
 ↪  Kg[mascara], T0, r0, s0))

fig, ax = plt.subplots()
ax.semilogy(Kg[mascara], er_fft + 1e-16, color=LARANJA, label="FFT (N = 4096)")
ax.semilogy(Kg[mascara], er_lw + 1e-16, color=AZUL, label="Lewis (integração direta)")
ax.set_title("Erro absoluto em relação ao BSM")
ax.set_xlabel("Strike")
ax.set_ylabel("Erro (escala log)")
ax.legend()
```

```
plt.show()

print("Erro máximo FFT (×10⁶):", f3(float(er_fft.max() * 1e6)), "| Erro máximo Lewis (×10⁹):",
↪  f3(float(er_lw.max() * 1e9)))
print("Espaçamento da grade de strikes perto do dinheiro:", f3(float(Kg[2049] - Kg[2048])))
ok(er_fft.max() < 1e-6, "com a transformada da call, a FFT supera os cinco dígitos do livro")
ok(er_lw.max() < er_fft.max() / 100, "a integração direta é muito mais precisa")
```




```
Erro máximo FFT (×10⁶): 0,043 | Erro máximo Lewis (×10⁹): 0,000
Espaçamento da grade de strikes perto do dinheiro: 0,669
OK — com a transformada da call, a FFT supera os cinco dígitos do livro
OK — a integração direta é muito mais precisa
```

Laboratório 4.2 — Interpolação, velocidade e modelo com saltos

```
K_alvo = np.array([87.3, 94.1, 101.7, 108.2, 119.9])
M76 = dict(sigma=0.15, lam=0.8, mu=-0.12, delta=0.10)
T_m, r_m = 0.5, 0.03
cf_m = lambda u: cf_merton(u, T_m, **M76)


def merton_serie(S, K, T, r, sigma, lam, mu, delta, n_max=60):
    kap = np.exp(mu + 0.5 * delta**2) - 1
    lq = lam * (1 + kap)
    tot = 0.0
    for n in range(n_max):
        sn = np.sqrt(sigma**2 + n * delta**2 / T)
        rn = r - lam * kap + n * np.log1p(kap) / T
        tot = tot + poisson.pmf(n, lq * T) * bsm(S, K, T, rn, sn)
    return tot


ref_m = merton_serie(S0, K_alvo, T_m, r_m, **M76)
Kg, Cg = cm_fft(S0, T_m, r_m, cf_m)
```

```
mask = (Kg > 40) & (Kg < 200) & (Cg > 1e-8)
lin = np.exp(np.interp(np.log(K_alvo), np.log(Kg[mask]), np.log(Cg[mask])))
spl = np.exp(CubicSpline(np.log(Kg[mask]), np.log(Cg[mask]))(np.log(K_alvo)))
lw_m = lewis(S0, K_alvo, T_m, r_m, cf_m, 300.0, 30001)
tabela(pd.DataFrame({"Strike": K_alvo, "Série": ref_m, "Lewis: erro (×10⁶)": np.abs(lw_m - ref_m)
 ↪  * 1e6, "FFT, interp. linear: erro (×10⁶)": np.abs(lin - ref_m) * 1e6, "FFT, spline: erro
 ↪  (×10⁶)": np.abs(spl - ref_m) * 1e6}))
ok(np.max(np.abs(spl - ref_m)) < np.max(np.abs(lin - ref_m)) / 2, "spline erra menos que
 ↪  interpolação linear")
ok(np.max(np.abs(lw_m - ref_m)) < np.max(np.abs(spl - ref_m)), "a integração direta não sofre com
 ↪  interpolação")

Km = np.linspace(70, 140, 2000)
t0 = perf_counter(); lewis(S0, Km, T_m, r_m, cf_m, 300.0, 6001); t_l = perf_counter() - t0
t0 = perf_counter(); Kg2, Cg2 = cm_fft(S0, T_m, r_m, cf_m); np.exp(CubicSpline(np.log(Kg2[mask]),
 ↪  np.log(Cg2[mask]))(np.log(Km))); t_f = perf_counter() - t0
K1 = np.array([100.0])
t0 = perf_counter()
for _ in range(50):
    lewis(S0, K1, T_m, r_m, cf_m, 300.0, 6001)
t_l1 = (perf_counter() - t0) / 50
t0 = perf_counter()
for _ in range(50):
    cm_fft(S0, T_m, r_m, cf_m)
t_f1 = (perf_counter() - t0) / 50
tabela(pd.DataFrame([
    {"Tarefa": "1 opção", "Lewis (ms)": t_l1 * 1e3, "FFT (ms)": t_f1 * 1e3},
    {"Tarefa": "2.000 opções", "Lewis (ms)": t_l * 1e3, "FFT (ms)": t_f * 1e3},
]))
ok(t_f < t_l, "para muitas opções, a FFT é mais rápida que a integração vetorial")
```

Strike	Série	Lewis: erro ($\times 10^6$)	FFT, interp. linear: erro ($\times 10^6$)	FFT, spline: erro ($\times 10^6$)
87,300	15,258	0,000	1.612,652	0,005
94,100	9,971	0,000	712,216	0,040
101,700	5,356	0,000	1.244,798	0,048
108,200	2,730	0,000	447,894	0,045
119,900	0,577	0,000	132,917	0,044

```
OK — spline erra menos que interpolação linear
OK — a integração direta não sofre com interpolação
```

Tarefa	Lewis (ms)	FFT (ms)
1 opção	0,570	0,568
2.000 opções	404,433	1,253

```
OK — para muitas opções, a FFT é mais rápida que a integração vetorial
```

---

5. A árvore binomial como convolução

Livro: §6.6.3

Para ver o que acontece “por dentro” do método, voltamos à árvore do Módulo 6.

5.1 O passo para trás é uma convolução

A **convolução circular** de a e b é $c_j = \sum_k a_{(j-k) \bmod n}\, b_k$. Definindo o vetor de probabilidades $qv = (q, 1-q, 0, \dots, 0)$ e seu reverso $\mathrm{rev}(qv) = (q, 0, \dots, 0, 1-q)$:

$$\left(C \circ \mathrm{rev}(qv)\right)_j = C_j\, q + C_{j+1}\,(1-q), \tag{8.22}$$

em que o índice crescente desce na árvore. É o passo CRR. O termo circular só contamina as posições da base, inválidas naquela data, e nunca chega à raiz em M passos.

5.2 A DFT diagonaliza a convolução

A propriedade é análoga à do caso contínuo: $\mathcal{F}(a \circ b) = \mathcal{F}(a) \cdot \mathcal{F}(b)$, produto termo a termo. M convoluções viram **uma potência**:

$$C_0 = e^{-rT}\, \mathcal{F}^{-1}\Big[\mathcal{F}(C_T) \cdot \mathcal{F}\big(\mathrm{rev}(qv)\big)^{M}\Big]. \tag{8.23}$$

> **Errata do livro:** o exemplo do método usa $qv = (0{,}5034;\ 0{,}4966;\ 0;\ 0;\ 0)$, embora a Tabela 5.1 e a própria conta na §5.4 tenham $q = 0{,}5378$. Com o vetor impresso, o valor não seria 9,97.

Laboratório 5.1 — O passo a passo e a fórmula fechada

```python
def params_crr(T, r, sig, M):
    dt = T / M
    u = np.exp(sig * np.sqrt(dt))
    d = 1 / u
    return dt, u, d, (np.exp(r * dt) - d) / (u - d)


def circular(a, b):
    n = a.size
    return np.array([sum(a[(j - k) % n] * b[k] for k in range(n)) for j in range(n)])


def rev(a):
    return np.r_[a[0], a[:0:-1]]


M = 4
dt, u, d, q = params_crr(T0, r0, s0, M)
C = np.maximum(S0 * u ** np.arange(M, -1, -1) * d ** np.arange(M + 1) - K0, 0.0)
qv = np.r_[q, 1 - q, np.zeros(M - 1)]
qv_livro = np.r_[0.5034, 0.4966, np.zeros(M - 1)]
Cl = C.copy()
for _ in range(M):
    C = np.exp(-r0 * dt) * circular(C, rev(qv))
    Cl = np.exp(-r0 * dt) * circular(Cl, rev(qv_livro))
print("q da árvore:", f3(float(q)), "| Valor pela convolução:", f3(float(C[0])), "| Com o vetor
↪  impresso no livro:", f3(float(Cl[0])))
ok(abs(C[0] - 9.97) < 0.005, "a convolução reproduz o valor da Tabela 5.1")
ok(abs(Cl[0] - 9.97) > 0.2, "o vetor impresso no livro não reproduz o valor")

a = np.array([1.0, 2.0, 3.0, 4.0])
b = np.array([0.5, -1.0, 2.0, 0.25])
```

```
ok(np.allclose(np.fft.fft(circular(a, b)), np.fft.fft(a) * np.fft.fft(b)), "a DFT transforma
↪  convolução em produto")


def crr_dft(S0, K, T, r, sig, M, cortar=None):
    """Árvore por DFT. 'cortar' zera os nós acima de S0·exp(cortar·σ√T)."""
    dt, u, d, q = params_crr(T, r, sig, M)
    ST = S0 * u ** np.arange(M, -1, -1) * d ** np.arange(M + 1)
    C = np.maximum(ST - K, 0.0)
    if cortar is not None:
        C[ST > S0 * np.exp(cortar * sig * np.sqrt(T))] = 0.0
    qv = np.zeros(M + 1)
    qv[:2] = q, 1 - q
    return float(np.exp(-r * T) * np.fft.ifft(np.fft.fft(C) * np.fft.fft(rev(qv)) ** M)[0].real)


def crr_dft2(S0, K, T, r, sig, M):
    """Forma alternativa do livro: C0 = F(exp(-rT) · F^{-1}(C_T) · F(qv)^M)."""
    dt, u, d, q = params_crr(T, r, sig, M)
    C = np.maximum(S0 * u ** np.arange(M, -1, -1) * d ** np.arange(M + 1) - K, 0.0)
    qv = np.zeros(M + 1)
    qv[:2] = q, 1 - q
    return float(np.fft.fft(np.exp(-r * T) * np.fft.ifft(C) * np.fft.fft(qv) ** M)[0].real)


def crr_vetor(S0, K, T, r, sig, M):
    dt, u, d, q = params_crr(T, r, sig, M)
    V = np.maximum(S0 * u ** np.arange(M, -1, -1) * d ** np.arange(M + 1) - K, 0.0)
    desc = np.exp(-r * dt)
    for _ in range(M):
        V = desc * (q * V[:-1] + (1 - q) * V[1:])
    return float(V[0])


linhas = []
for M in [500, 5000, 20000]:
    t0 = perf_counter(); v1 = crr_dft(S0, K0, T0, r0, s0, M); t1 = perf_counter() - t0
    t0 = perf_counter(); v2 = crr_dft2(S0, K0, T0, r0, s0, M); t2 = perf_counter() - t0
    t0 = perf_counter(); v3 = crr_vetor(S0, K0, T0, r0, s0, M); t3 = perf_counter() - t0
    v4 = crr_dft(S0, K0, T0, r0, s0, M, cortar=10.0)
    ST_max = S0 * params_crr(T0, r0, s0, M)[1] ** M
    linhas.append({"M": M, "Maior nó final": ST_max, "DFT": v1, "DFT, forma alternativa": v2,
↪  "Árvore vetorizada": v3, "DFT cortando extremos": v4, "Erro DFT": abs(v1 - v3), "Erro DFT
↪  cortando": abs(v4 - v3), "DFT (ms)": t1 * 1e3, "Árvore (ms)": t3 * 1e3})
d_ = pd.DataFrame(linhas)
tabela(d_)
ok(d_["Erro DFT"].iloc[0] < 1e-8 and abs(d_["DFT"].iloc[0] - d_["DFT, forma alternativa"].iloc[0])
↪  < 1e-8, "para M moderado, as duas formas com DFT coincidem com a árvore")
ok(d_["Erro DFT"].iloc[-1] > 1, "para M muito grande, a DFT direta perde toda a precisão")
ok((d_["Erro DFT cortando"] < 1e-7).all(), "cortando os nós extremos, a precisão volta")
ok(d_["DFT (ms)"].iloc[-1] < d_["Árvore (ms)"].iloc[-1], "para M grande, a DFT é mais rápida que
↪  a árvore vetorizada")
```

```
q da árvore: 0,538 | Valor pela convolução: 9,971 | Com o vetor impresso no livro: 8,341
OK — a convolução reproduz o valor da Tabela 5.1
OK — o vetor impresso no livro não reproduz o valor
OK — a DFT transforma convolução em produto
```

M	Maior nó final	DFT	DFT, forma alternativa	Árvore vetorizada	DFT cortando extremos	Erro DFT	Erro DFT cortando	DFT (ms)	Árvore (ms)
500	8.754,351	10,447	10,447	10,447	10,447	0,000	0,000	0,565	1,267
5000	138.628.161,529	10,450	10,450	10,450	10,450	0,000	0,000	1,625	34,901
20000	192.177.671.690.278,062	34,602	13,728	10,450	10,450	24,152	0,000	4,957	902,205

```
OK — para M moderado, as duas formas com DFT coincidem com a árvore
OK — para M muito grande, a DFT direta perde toda a precisão
OK — cortando os nós extremos, a precisão volta
OK — para M grande, a DFT é mais rápida que a árvore vetorizada
```

Cuidado numérico. Com $M = 20.000$, o maior nó da árvore está na casa de 10^{12}. A FFT mistura todas as entradas em cada coeficiente, e o erro de arredondamento é proporcional à **maior** delas. Ele contamina o valor que queremos. A árvore direta não sofre desse problema, porque os nós extremos só afetam os vizinhos. A correção é zerar nós irrelevantes: o que está acima de 10 desvios-padrão contribui com um valor desprezível.

---

6. Qual método usar?

Método	Vantagem	Limitação	Uso natural
Lewis (integração direta)	Precisão e simplicidade; strikes arbitrários	Caudas exigem cuidado	Calibração
Carr-Madan com FFT	Grade inteira ao custo de uma FFT	Interpolação; acoplamento entre grades	Densidades, grandes malhas
Carr-Madan com α por strike	Erro relativo pequeno nas caudas	Uma integral por strike	Opções muito fora do dinheiro
Transformada OTM	Sem subtrair S_0	Cauda lenta; singular em $k = 0$	Grades de FFT muito longas
FFT na árvore	Instrutivo	Não serve para americanas; a soma binomial já é $O(M)$	Ensino

Uma observação final, em nome da honestidade do método: no Módulo 6, a soma binomial fechada já custa $O(M)$. A FFT na árvore, de custo $O(M \log M)$, é mais lenta que ela. O interesse do exemplo é mostrar a estrutura, e não ganhar tempo.

---

7. Síntese

- Carr-Madan é a fórmula de Lewis antes de atravessar o polo, com $u_i = \alpha + 1$.
- α muito pequeno ou muito grande estraga a precisão.
- Nas caudas, escolher α por strike é melhor que a transformada do valor no tempo, que tem cauda lenta.

- A regularidade determina a velocidade de convergência, o que vale para séries e para árvores.
- A DFT se apoia na ortogonalidade das raízes da unidade, e a FFT reduz seu custo para $O(n \log n)$.
- A restrição $\varepsilon\eta = 2\pi/N$ é o preço da FFT.
- Na árvore, o passo para trás é uma convolução, e a DFT a transforma em potência; para muitos passos, é preciso controlar a amplitude dos dados.

8. Notas sobre o livro neste trecho

Local	Problema
§6.5.1	e^{iux} está expandida incorretamente: o fator é $e^{-u_i x}$ e a oscilação usa apenas u_r
Def. 43	as somas de potências devem começar em $j = 0$
Def. 43	potências positivas de z_n giram em sentido horário
Def. 44	a inversa da DFT precisa do fator $1/n$
§6.6.2, equação (6.13)	a lei $\ln(1+J) \sim N(\ln(1+\mu_J) - \delta^2/2, \delta^2)$ não é compatível com $r_J = \lambda(e^{\mu_J + \delta^2/2} - 1)$ e com (6.14), que correspondem a $\ln(1+J) \sim N(\mu_J, \delta^2)$
§6.6.3	o vetor $qv = (0{,}5034;\ 0{,}4966; \dots)$ é incompatível com $q = 0{,}5378$
Equação (6.6)	falta tomar a parte real
§6.6.1, precisão da FFT	os cinco dígitos relatados vêm do ramo OTM do script, de cauda lenta; não são um limite da FFT

9. Exercícios

Exercício 1. Com $N = 4096$ e $\varepsilon = 1/150$, qual é o maior limite de integração $v_{\max}$?

Solução

$\eta = 2\pi/(N\varepsilon) = 0{,}230$. Logo, $v_{\max} = \eta N = 2\pi/\varepsilon = 942{,}478$.

Exercício 2. Mostre que $\mathcal{F}(a \circ b) = \mathcal{F}(a)\mathcal{F}(b)$.

Solução

$\sum_j z^{kj} \sum_l a_{j-l} b_l = \sum_l b_l z^{kl} \sum_j a_{j-l} z^{k(j-l)} = \mathcal{F}(b)_k \, \mathcal{F}(a)_k$, usando a periodicidade de z^{kj} em j.

Exercício 3. Em Carr-Madan, por que α precisa ser positivo? O que acontece com a put se $\alpha < -1$?

Solução

Com $\alpha > 0$, a função $e^{\alpha k} C(k)$ vai a zero em $k \to -\infty$, onde $C \to S_0$. Com $\alpha < -1$, o amortecimento favorece $k \to +\infty$ e a integral fica associada à put, de modo análogo ao que vimos ao mudar de faixa na integral de Lewis.

Exercício 4. Por que a FFT não serve diretamente para a árvore de uma opção americana?

Solução

O `max` entre continuação e exercício é uma operação não linear, aplicada a cada data. Não se pode juntar todos os passos em uma potência, que só vale para operadores lineares.

```
print(f"Módulo 8 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 8 concluído: 26 verificações aprovadas.
```

Capítulo 9

Opções americanas por simulação: Longstaff-Schwartz e o problema dual

Introdução

Árvores e métodos de Fourier funcionam bem com um ou dois fatores de risco. Com mais fatores, como volatilidade estocástica e juros estocásticos ao mesmo tempo, a simulação de Monte Carlo passa a ser a ferramenta natural. Mas a simulação gera trajetórias para a frente, enquanto a decisão de exercer uma opção americana exige comparar, em cada data, o pagamento imediato com o **valor de continuar**, uma esperança condicional que depende do futuro.

O algoritmo de **Longstaff e Schwartz** (LSM, *least-squares Monte Carlo*) resolve esse impasse estimando o valor de continuação por regressão dos pagamentos futuros simulados sobre funções do estado atual. O resultado é uma regra de exercício aproximada e, com ela, um preço que tende a ficar ligeiramente abaixo do verdadeiro.

O **problema dual**, de Rogers e de Haugh e Kogan, dá o outro lado: um limite superior para o preço, construído a partir de um martingal. Juntos, LSM e dual formam um intervalo que mede a qualidade da aproximação. O capítulo implementa os dois e os confere contra valores de referência.

Fundamentos teóricos

Parada ótima

Seja $Z_t = e^{-rt} h(S_t)$ o payoff descontado de uma opção que pode ser exercida nas datas $0 = t_0 < t_1 < \cdots < t_M = T$ (bermudana; a americana é o limite de malha fina).

Teorema 9.1 (Programação dinâmica). *O valor da opção é $V_0 = \sup_\tau \mathbb{E}^{\mathbb{Q}}[Z_\tau]$, com o supremo sobre tempos de parada com valores em $\{t_0, \dots, t_M\}$. Ele é obtido pela recursão*

$$V_M = Z_M, \qquad V_m = \max\left\{Z_m,\ \mathbb{E}^{\mathbb{Q}}[V_{m+1} \mid \mathcal{F}_{t_m}]\right\}, \tag{9.1}$$

e $\tau^* = \min\{t_m : Z_m \geq C_m\}$, com $C_m = \mathbb{E}^{\mathbb{Q}}[V_{m+1} \mid \mathcal{F}_{t_m}]$, é ótimo.

O obstáculo numérico está no valor de continuação C_m: uma esperança condicional, função do estado, que a simulação não fornece diretamente.

Mínimos quadrados de Longstaff e Schwartz

Proposição 9.2 (Esperança condicional como projeção). *Se $Y \in L^2$ e X é o estado em t_m, então $\mathbb{E}[Y \mid X]$ é a projeção ortogonal de Y sobre o subespaço $L^2(\sigma(X))$: minimiza $\mathbb{E}[(Y - g(X))^2]$ entre as funções mensuráveis g.*

O algoritmo LSM (Longstaff and Schwartz, 2001) aproxima essa projeção por regressão sobre uma base finita $\{e_1, \dots, e_D\}$:

1. simule I trajetórias do estado e inicialize o fluxo de caixa com Z_M;
2. para $m = M-1, \dots, 1$: regrida o fluxo de caixa descontado até t_m sobre $e_d(X_{m,i})$, *apenas nas trajetórias dentro do dinheiro*, obtendo $\widehat{C}_m(x) = \sum_d \widehat{\alpha}_{d,m} e_d(x)$;
3. onde $Z_{m,i} \geq \widehat{C}_m(X_{m,i})$, substitua o fluxo de caixa pelo exercício em t_m;
4. estime V_0 pela média dos fluxos descontados.

Teorema 9.3 (Convergência). *Para base fixa, quando $I \to \infty$ o estimador LSM converge quase certamente para o valor da regra de exercício definida pelas projeções exatas sobre a base; quando, em seguida, $D \to \infty$, esse valor converge para V_0 (Clément et al., 2002).*

Observação 9.4 (Viés baixo). Se a regra de exercício é estimada num conjunto de trajetórias e o preço é calculado em outro independente, o estimador é *viesado para baixo*: qualquer regra de exercício é subótima, e $\mathbb{E}[Z_{\hat{\tau}}] \leq V_0$. Usar as mesmas trajetórias introduz um viés para cima (sobreajuste) que, na prática, é pequeno.

O problema dual

Teorema 9.5 (Dualidade de Rogers, Haugh e Kogan). *Para todo martingal M com $M_0 = 0$,*

$$V_0 \leq \mathbb{E}^{\mathbb{Q}}\Big[\max_{0 \leq m \leq M} (Z_m - M_{t_m})\Big], \tag{9.2}$$

com igualdade quando M é a parte martingal da decomposição de Doob do envelope de Snell (Rogers, 2002; Haugh and Kogan, 2004).

Demonstração. Para qualquer tempo de parada τ, pelo teorema da parada opcional $\mathbb{E}[M_\tau] = 0$, logo $\mathbb{E}[Z_\tau] = \mathbb{E}[Z_\tau - M_\tau] \leq \mathbb{E}[\max_m (Z_m - M_{t_m})]$. Tomando o supremo em τ obtém-se (9.2). Para a igualdade, a decomposição $V_m = V_0 + M_m^* - A_m$, com A previsível crescente, dá $Z_m - M_m^* \leq V_m - M_m^* = V_0 - A_m \leq V_0$. ∎

Na prática, M é construído a partir do valor de continuação estimado pelo LSM, com simulações aninhadas para as esperanças condicionais. O resultado é um *intervalo*: o LSM dá uma cota inferior, o dual uma cota superior, e a distância entre elas mede a qualidade da regra de exercício.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulo 7 do livro.

O que você vai conseguir fazer ao final

1. Formular o valor de uma americana como problema de parada ótima e como programação dinâmica.
2. Explicar por que a simulação, que anda para a frente, tem dificuldade com um problema que anda para trás.
3. Entender o algoritmo de Longstaff-Schwartz como uma projeção, e implementá-lo.
4. Distinguir os vieses para cima e para baixo do LSM, e evitá-los.
5. Deduzir a formulação dual e obter um limite superior.
6. Construir um intervalo de confiança para o valor americano com os dois limites.
7. Comparar os resultados com a referência certa: o valor bermudano.

Onde estamos

No Módulo 6, a árvore resolveu a put americana com um único `max` por nó. Mas árvores ficam inviáveis com várias variáveis de estado, como no modelo geral que vamos usar (índice, variância e juros). A simulação não sofre com a dimensão, mas anda no sentido errado. Este módulo resolve esse impasse.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
from time import perf_counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from numpy.polynomial import polynomial as P
from scipy.stats import norm
from IPython.display import display

# Os dois exemplos do livro
PUT = dict(S0=36.0, K=40.0, T=1.0, r=0.06, sig=0.20)
CONDOR = dict(S0=100.0, K=100.0, T=1.0, r=0.05, sig=0.50)
pay_put = lambda s: np.maximum(40.0 - s, 0.0)
pay_condor = lambda s: np.minimum(40.0, np.maximum(90.0 - s, 0.0) + np.maximum(s - 110.0, 0.0))
print("Ambiente pronto.")
```

```
Ambiente pronto.
```

1. O problema

Livro: §7.3.1

1.1 Parada ótima

O titular de uma americana escolhe **quando** exercer. Essa escolha só pode usar a informação disponível até o momento: é um **tempo de parada**, no sentido do Módulo 3. O valor é o melhor resultado possível entre todos os tempos de parada:

$$V_0 = \sup_{\tau} \ \mathbb{E}^Q\big[e^{-r\tau} h(S_\tau)\big]. \tag{9.3}$$

1.2 Programação dinâmica

Para calcular, discretizamos o tempo em M datas de exercício, $t_m = m\Delta t$. Na última data, $V_M = h(S_M)$. Nas anteriores, comparamos o exercício com o **valor de continuação**:

$$C_m(s) = \mathbb{E}^Q\big[e^{-r\Delta t} V_{m+1}(S_{m+1}) \,\big|\, S_m = s\big], \qquad V_m(s) = \max\big[h(s),\ C_m(s)\big]. \tag{9.4}$$

O processo V_m é a **envoltória de Snell**: o menor supermartingale que fica acima do payoff descontado.

1.3 Americana ou bermudana?

Com M datas, o algoritmo precifica uma **bermudana** de M datas, e não a americana com exercício contínuo. A diferença é sistemática e sempre para baixo. No Módulo 6 vimos que, para a put de Longstaff-Schwartz, a diferença com 50 datas é de cerca de 0,009.

Isso tem uma consequência para a leitura dos resultados: **um algoritmo com M datas deve ser comparado com a bermudana de M datas**. Comparar com a americana mistura dois erros diferentes.

Laboratório 1.1 — As referências corretas

Calculamos por árvore (Módulo 6) o valor bermudano para as datas usadas no livro e o valor americano.

```python
def arvore_bermudana(pay, S0, T, r, sig, datas, fator=200, guardar=False):
    """Árvore CRR com 'fator' passos entre datas de exercício. Opcionalmente guarda V nas
    ↪ datas."""
    N = datas * fator
    dt = T / N
    u = np.exp(sig * np.sqrt(dt))
    d = 1 / u
    q = (np.exp(r * dt) - d) / (u - d)
    desc = np.exp(-r * dt)
    j = np.arange(N + 1)
    ST = S0 * u**j * d ** (N - j)
    V = pay(ST)
    tab = {datas: (np.log(ST), V.copy())} if guardar else None
    for t in range(N - 1, -1, -1):
        V = desc * (q * V[1:] + (1 - q) * V[:-1])
```

```
        if t % fator == 0 and t > 0:
            j = np.arange(t + 1)
            St = S0 * u**j * d ** (t - j)
            V = np.maximum(V, pay(St))
            if guardar:
                tab[t // fator] = (np.log(St), V.copy())
    return (float(V[0]), tab) if guardar else float(V[0])


p_ = dict(S0=PUT["S0"], T=PUT["T"], r=PUT["r"], sig=PUT["sig"])
c_ = dict(S0=CONDOR["S0"], T=CONDOR["T"], r=CONDOR["r"], sig=CONDOR["sig"])
ref = {}
linhas = []
for M in [25, 50, 75]:
    ref[("put", M)] = arvore_bermudana(pay_put, datas=M, **p_)
    ref[("condor", M)] = arvore_bermudana(pay_condor, datas=M, **c_)
    linhas.append({"Datas de exercício M": str(M), "Put: bermudana": ref[("put", M)], "Condor:
↪  bermudana": ref[("condor", M)]})
am_put = arvore_bermudana(pay_put, datas=15000, fator=1, **p_)
am_con = arvore_bermudana(pay_condor, datas=15000, fator=1, **c_)
linhas.append({"Datas de exercício M": "Contínuo (americana)", "Put: bermudana": am_put, "Condor:
↪  bermudana": am_con})
linhas.append({"Datas de exercício M": "Livro: árvore de 500 passos", "Put: bermudana": 4.486,
↪  "Condor: bermudana": 26.97705})
refs = pd.DataFrame(linhas)
tabela(refs)

ok(abs(arvore_bermudana(pay_put, datas=500, fator=1, **p_) - 4.486) < 1e-3, "a árvore de 500
↪  passos reproduz a referência do livro para a put")
ok(abs(arvore_bermudana(pay_condor, datas=500, fator=1, **c_) - 26.97705) < 1e-4, "e para o
↪  condor")
ok(am_con - ref[("condor", 25)] > 0.9, "no condor, a diferença entre americana e bermudana de 25
↪  datas passa de 0,9")
```

Datas de exercício M	Put: bermudana	Condor: bermudana
25	4,469	26,015
50	4,478	26,332
75	4,481	26,469
Contínuo (americana)	4,487	27,008
Livro: árvore de 500 passos	4,486	26,977

```
OK — a árvore de 500 passos reproduz a referência do livro para a put
OK — e para o condor
OK — no condor, a diferença entre americana e bermudana de 25 datas passa de 0,9
```

Guarde esta tabela. O livro compara os resultados de simulação com 4,486 e 26,977, que são valores americanos. Veremos que boa parte do que ele chama de "diferença" do LSM não é erro do algoritmo, e sim a diferença entre americana e bermudana.

---

2. Por que simular é difícil

Livro: §7.3.2

A simulação gera trajetórias para a frente. O valor de continuação é uma esperança condicional: para cada nó de cada trajetória, precisaríamos de **outra** simulação a partir dali. Com I trajetórias, isso dá I^2 trajetórias por data, um custo inviável.

A tentação: usar o futuro realizado

Cada trajetória tem, de fato, um valor futuro realizado $Y_{m,i}$. Por que não decidir com ele?

$$V_{m,i} = \max\left[h(S_{m,i}),\ Y_{m,i}\right]. \tag{9.5}$$

Porque isso é **olhar o futuro**: decidir exercer sabendo que o índice vai subir. A decisão deixa de ser um tempo de parada. O valor obtido é sistematicamente **alto**.

Laboratório 2.1 — O preço da clarividência

```
def simular(S0, r, sig, T, M, I, rng, momentos=True, antiteticas=False):
    """Trajetórias do GBM. Linhas são datas, colunas são trajetórias."""
    dt = T / M
    if antiteticas:
        z = rng.standard_normal((M, I // 2))
        Z = np.concatenate([z, -z], axis=1)
    else:
        Z = rng.standard_normal((M, I))
    if momentos:
        Z = (Z - Z.mean(axis=1, keepdims=True)) / Z.std(axis=1, keepdims=True)
    inc = (r - 0.5 * sig**2) * dt + sig * np.sqrt(dt) * Z
    return S0 * np.exp(np.vstack([np.zeros(I), np.cumsum(inc, axis=0)]))


def clarividente(S, pay, r, T):
    M = S.shape[0] - 1
    desc = np.exp(-r * T / M)
    V = pay(S[-1])
    for m in range(M - 1, 0, -1):
        V = np.maximum(pay(S[m]), desc * V)
    return float(desc * V.mean())


rng = np.random.default_rng(901)
S = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 100_000, rng)
v_clar = clarividente(S, pay_put, PUT["r"], PUT["T"])
tabela(pd.DataFrame([{"Estimador": "Decidir com o futuro realizado", "Valor": v_clar},
 ↪ {"Estimador": "Bermudana de 25 datas (árvore)", "Valor": ref[("put", 25)]}]))
ok(v_clar > ref[("put", 25)] + 0.5, "usar o futuro realizado superestima muito o valor")
```

Estimador	Valor
Decidir com o futuro realizado	7,444
Bermudana de 25 datas (árvore)	4,469

```
OK — usar o futuro realizado superestima muito o valor
```

3. Longstaff-Schwartz: a esperança condicional como regressão

Livro: §7.3.2, Algoritmo 1

3.1 A ideia

Uma esperança condicional é a **melhor previsão** de Y a partir de S_m, no sentido dos mínimos quadrados:

$$C_m = \mathbb{E}[Y_m \mid S_m] = \arg\min_f \ \mathbb{E}[(Y_m - f(S_m))^2]. \tag{9.6}$$

Não podemos minimizar sobre todas as funções, mas podemos restringir a busca a combinações de D funções-base:

$$\hat{C}_m(s) = \sum_{d=1}^{D} \alpha_{d,m}\, b_d(s), \qquad \hat{\alpha} = \arg\min_{\alpha} \ \frac{1}{I} \sum_{i=1}^{I} \Big(Y_{m,i} - \sum_d \alpha_d\, b_d(S_{m,i}) \Big)^2. \tag{9.7}$$

É uma regressão comum. As trajetórias vizinhas fazem o papel da simulação aninhada. Em linguagem geométrica, é a **projeção ortogonal** de Y sobre o espaço gerado pelas funções-base.

3.2 O algoritmo, passo a passo

1. Simule I trajetórias.
2. Em T: $V_{M,i} = h(S_{M,i})$.
3. Para $m = M-1, \dots, 1$:
   - $Y_{m,i} = e^{-r\Delta t} V_{m+1,i}$ (o **realizado**);
   - regrida Y nas funções-base, apenas nas trajetórias dentro do dinheiro;
   - se $h(S_{m,i}) > \hat{C}_m(S_{m,i})$, exerça: $V_{m,i} = h(S_{m,i})$; senão, $V_{m,i} = Y_{m,i}$.
4. $\hat{V}_0 = e^{-r\Delta t} \frac{1}{I} \sum_i V_{1,i}$.

O detalhe que faz o método funcionar: a regressão só serve para **decidir**. O valor guardado é o realizado Y, nunca o estimado $\hat{C}$. Assim o erro da regressão só afeta a qualidade da decisão, e não se acumula.

Por que só as trajetórias dentro do dinheiro? Fora do dinheiro, não há decisão a tomar. Concentrar a regressão onde importa dá um ajuste melhor exatamente na região da fronteira.

Funções-base. Usamos monômios em $x = S/K$. Dividir por K deixa os números perto de 1 e evita problemas numéricos.

Laboratório 3.1 — Uma regressão vista de perto

Na data de meio prazo, colocamos em um gráfico os pares (S_m, Y_m), a curva ajustada e o valor de continuação exato, obtido pela árvore.

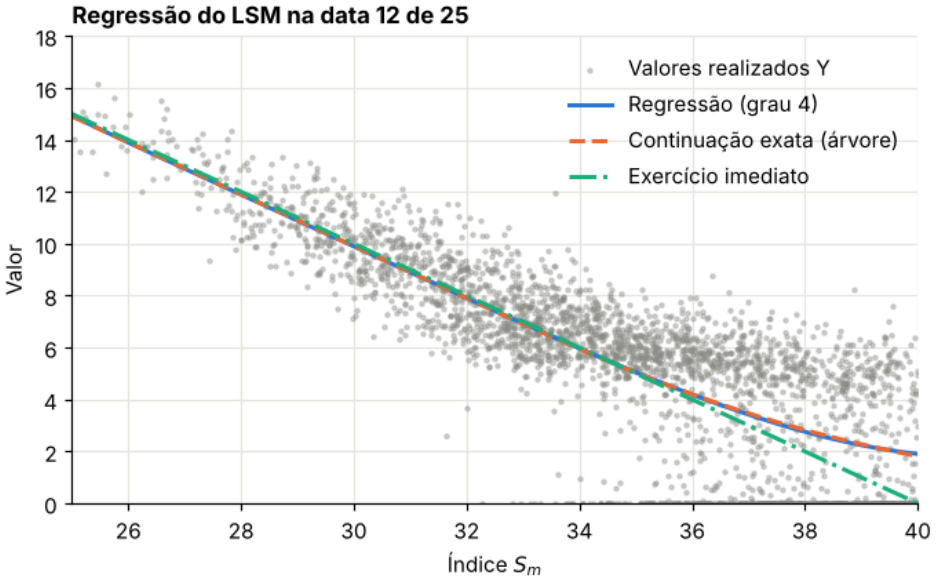
```
def base(s, K, grau):
    return P.polyvander(s / K, grau)
```

```
M, grau = 25, 4
rng = np.random.default_rng(902)
S = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], M, 20_000, rng)
desc = np.exp(-PUT["r"] * PUT["T"] / M)
V = pay_put(S[-1])
m_alvo = 12
for m in range(M - 1, 0, -1):
    Y = desc * V
    h = pay_put(S[m])
    itm = h > 0
    coef = np.linalg.lstsq(base(S[m][itm], 40.0, grau), Y[itm], rcond=None)[0]
    C = P.polyval(S[m] / 40.0, coef)
    if m == m_alvo:
        S_alvo, Y_alvo, coef_alvo, itm_alvo = S[m].copy(), Y.copy(), coef.copy(), itm.copy()
    V = np.where(itm & (h > C), h, Y)

# Valor de continuação exato na data m_alvo, pela árvore
_, tab = arvore_bermudana(pay_put, datas=M, guardar=True, **p_)
fator, N = 200, M * 200
dt_f = PUT["T"] / N
u_f = np.exp(PUT["sig"] * np.sqrt(dt_f))
d_f = 1 / u_f
q_f = (np.exp(PUT["r"] * dt_f) - d_f) / (u_f - d_f)
_, Vn = tab[m_alvo + 1]
Vc = Vn.copy()
for _ in range(fator):
    Vc = np.exp(-PUT["r"] * dt_f) * (q_f * Vc[1:] + (1 - q_f) * Vc[:-1])
t_no = (m_alvo + 1) * fator - fator
j = np.arange(t_no + 1)
S_no = PUT["S0"] * u_f**j * d_f ** (t_no - j)

grade = np.linspace(25, 40, 200)
fig, ax = plt.subplots()
idx = np.where(itm_alvo)[0][:3000]
ax.scatter(S_alvo[idx], Y_alvo[idx], s=4, color=CINZA, alpha=0.35, label="Valores realizados Y")
ax.plot(grade, P.polyval(grade / 40.0, coef_alvo), color=AZUL, label="Regressão (grau 4)")
ax.plot(grade, np.interp(grade, S_no, Vc), color=LARANJA, ls="--", label="Continuação exata
↪  (árvore)")
ax.plot(grade, pay_put(grade), color=VERDE, ls="-.", label="Exercício imediato")
ax.set_xlim(25, 40)
ax.set_ylim(0, 18)
ax.set_title(f"Regressão do LSM na data {m_alvo} de {M}")
ax.set_xlabel("Índice $S_m$")
ax.set_ylabel("Valor")
ax.legend()
plt.show()

erro_reg = np.abs(P.polyval(grade / 40.0, coef_alvo) - np.interp(grade, S_no, Vc))
print("Maior erro da regressão na faixa 28 a 40:", f3(float(erro_reg[grade >= 28].max())))
ok(erro_reg[grade >= 28].max() < 0.1, "a regressão aproxima bem o valor de continuação exato")
```




```
Maior erro da regressão na faixa 28 a 40: 0,090
OK — a regressão aproxima bem o valor de continuação exato
```

Leitura. Os pontos estão muito espalhados, porque cada Y é o resultado de uma única trajetória. Mesmo assim, a média condicional, que é o que a regressão recupera, coincide com a curva exata. A decisão de exercer está no cruzamento das curvas azul e verde.

3.3 Dois vieses de sinais opostos

Viés para baixo. A regra de exercício estimada é, no máximo, tão boa quanto a ótima. Qualquer regra de exercício avaliada em trajetórias **novas** é um tempo de parada legítimo, e seu valor esperado não pode superar o ótimo. É o **limite inferior**.

Viés para cima (dentro da amostra). Se a mesma amostra serve para ajustar a regressão e para avaliar, a regra se adapta ao ruído daquela amostra, uma pequena dose da clarividência da Seção 2. Com poucas trajetórias e muitas funções-base, o resultado pode até superar o valor verdadeiro.

A solução, como o livro sugere, é separar: I_1 trajetórias para estimar os coeficientes e I_2 trajetórias novas para avaliar.

```python
def lsm_regressoes(S, pay, K, r, T, grau, so_itm=True):
    """Passo para trás do LSM. Devolve os coeficientes (regra de exercício), os coeficientes em
    ↪ todas as trajetórias (função valor) e o valor dentro da amostra."""
    M = S.shape[0] - 1
    desc = np.exp(-r * T / M)
    V = pay(S[-1])
    regra, fval = [None] * (M + 1), [None] * (M + 1)
    for m in range(M - 1, 0, -1):
        Y = desc * V
        h = pay(S[m])
        sel = h > 0 if so_itm else np.ones_like(h, bool)
```

```
        x = S[m] / K
        faixa = (x.min(), x.max())
        c = np.linalg.lstsq(P.polyvander(x[sel], grau), Y[sel], rcond=None)[0]
        c_tudo = np.linalg.lstsq(P.polyvander(x, grau), Y, rcond=None)[0]
        regra[m], fval[m] = (c, faixa, so_itm), (c_tudo, faixa)
        C = P.polyval(x, c)
        V = np.where((h > 0) & (h > C) if so_itm else (h > C), h, Y)
    return regra, fval, float(desc * V.mean())


def continuacao(regra, m, s, K):
    c, faixa, _ = regra[m]
    return P.polyval(s / K, c)


def lsm_fora_amostra(S, pay, K, r, T, regra):
    """Aplica a regra de exercício a trajetórias novas: limite inferior."""
    M, I = S.shape[0] - 1, S.shape[1]
    dt = T / M
    valor = np.zeros(I)
    vivo = np.ones(I, bool)
    for m in range(1, M + 1):
        h = pay(S[m])
        if m < M:
            exerce = vivo & (h > 0) & (h > continuacao(regra, m, S[m], K))
        else:
            exerce = vivo & (h > 0)
        valor[exerce] = np.exp(-r * m * dt) * h[exerce]
        vivo &= ~exerce
    return float(valor.mean()), float(valor.std(ddof=1) / np.sqrt(I))


rng = np.random.default_rng(903)
linhas = []
for I1, grau in [(200, 8), (1_000, 8), (16_384, 8), (100_000, 4)]:
    dentro, fora = [], []
    for _ in range(20):
        S1 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, I1, rng)
        regra, _, v_in = lsm_regressoes(S1, pay_put, 40.0, PUT["r"], PUT["T"], grau)
        S2 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 50_000, rng)
        dentro.append(v_in)
        fora.append(lsm_fora_amostra(S2, pay_put, 40.0, PUT["r"], PUT["T"], regra)[0])
    linhas.append({"Trajetórias de regressão I1": str(I1), "Grau": grau, "Dentro da amostra:
↪  média": float(np.mean(dentro)), "Fora da amostra: média": float(np.mean(fora)), "Referência
↪  bermudana": ref[("put", 25)]})
vies = pd.DataFrame(linhas)
tabela(vies)
ok(vies["Dentro da amostra: média"].iloc[0] > vies["Fora da amostra: média"].iloc[0] + 0.03, "com
↪  poucas trajetórias, a estimativa dentro da amostra fica acima da de fora")
ok((vies["Fora da amostra: média"] < ref[("put", 25)] + 0.01).all(), "fora da amostra, a
↪  estimativa não ultrapassa a referência (é um limite inferior)")
```

Trajetórias de regressão I1	Grau	Dentro da amostra: média	Fora da amostra: média	Referência bermudana
200	8	4,854	4,259	4,469
1000	8	4,537	4,426	4,469
16384	8	4,470	4,465	4,469
100000	4	4,469	4,466	4,469

```
OK — com poucas trajetórias, a estimativa dentro da amostra fica acima da de fora
OK — fora da amostra, a estimativa não ultrapassa a referência (é um limite inferior)
```

Laboratório 3.2 — Funções-base e regressão só dentro do dinheiro

Usamos $I_1 = 16.384$ trajetórias para estimar a regra e $I_2 = 100.000$ para avaliá-la.

```
rng = np.random.default_rng(904)
S1 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 16_384, rng)
S2 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 100_000, rng)
linhas = []
for so_itm in [True, False]:
    for grau in [1, 2, 3, 5, 8]:
        regra, _, _ = lsm_regressoes(S1, pay_put, 40.0, PUT["r"], PUT["T"], grau, so_itm)
        v, ep = lsm_fora_amostra(S2, pay_put, 40.0, PUT["r"], PUT["T"], regra)
        linhas.append({"Regressão": "Só dentro do dinheiro" if so_itm else "Todas as trajetórias",
 ↪  "Grau": grau, "Valor (fora da amostra)": v, "Erro-padrão": ep, "Distância à bermudana": v -
 ↪  ref[("put", 25)]})
bas = pd.DataFrame(linhas)
tabela(bas)
ok(bas.query("Regressão == 'Só dentro do dinheiro' and Grau == 3")["Distância à
 ↪  bermudana"].abs().iloc[0] < 0.02, "com regressão só dentro do dinheiro e grau 3, o LSM fica a
 ↪  menos de 0,02 da bermudana")
ok(bas.query("Grau == 1")["Valor (fora da amostra)"].max() < bas.query("Grau == 3")["Valor (fora
 ↪  da amostra)"].min(), "uma base muito pobre (reta) produz uma regra de exercício pior")
```

Regressão	Grau	Valor (fora da amostra)	Erro-padrão	Distância à bermudana
Só dentro do dinheiro	1	4,417	0,010	-0,052
Só dentro do dinheiro	2	4,460	0,010	-0,009
Só dentro do dinheiro	3	4,464	0,009	-0,005
Só dentro do dinheiro	5	4,467	0,009	-0,003
Só dentro do dinheiro	8	4,465	0,009	-0,004
Todas as trajetórias	1	4,439	0,010	-0,030
Todas as trajetórias	2	4,458	0,009	-0,011
Todas as trajetórias	3	4,462	0,009	-0,007
Todas as trajetórias	5	4,466	0,009	-0,003
Todas as trajetórias	8	4,466	0,009	-0,003

```
OK — com regressão só dentro do dinheiro e grau 3, o LSM fica a menos de 0,02 da bermudana
OK — uma base muito pobre (reta) produz uma regra de exercício pior
```

---

4. A formulação dual: um limite superior

Livro: §7.3.1 e Algoritmo 2

O LSM dá um limite inferior, mas não diz **o quanto** está abaixo. Haugh e Kogan (2004) e Rogers (2002) mostraram como obter um limite superior.

4.1 Dedução

Trabalhe com valores descontados, $\tilde{h}_m = e^{-rt_m} h(S_m)$. Seja $\mathcal{M}$ qualquer martingale com $\mathcal{M}_0 = 0$.

Passo 1. Para qualquer tempo de parada τ, o teorema de parada opcional dá $\mathbb{E}[\mathcal{M}_\tau] = \mathcal{M}_0 = 0$. Logo,

$$\mathbb{E}[\tilde{h}_\tau] = \mathbb{E}[\tilde{h}_\tau - \mathcal{M}_\tau]. \tag{9.8}$$

Passo 2. O valor em τ não supera o maior valor da trajetória:

$$\mathbb{E}[\tilde{h}_\tau - \mathcal{M}_\tau] \le \mathbb{E}\Big[\max_m (\tilde{h}_m - \mathcal{M}_m)\Big]. \tag{9.9}$$

Passo 3. O lado direito não depende de τ. Tomando o supremo à esquerda:

$$\boxed{V_0 \le \mathbb{E}\Big[\max_m (\tilde{h}_m - \mathcal{M}_m)\Big] \quad \text{para todo martingale } \mathcal{M}.} \tag{9.10}$$

Passo 4: a igualdade. Tome o martingale da decomposição de Doob da envoltória de Snell, $\tilde{V}_m = V_0 + \mathcal{M}^*_m - A_m$, com A crescente. Como $\tilde{V}$ fica acima de $\tilde{h}$:

$$\tilde{h}_m - \mathcal{M}^*_m \le \tilde{V}_m - \mathcal{M}^*_m = V_0 - A_m \le V_0. \tag{9.11}$$

O máximo é, portanto, no máximo V_0, e a desigualdade vira igualdade. Mais do que isso: **com o martingale ótimo, o estimador tem variância zero**. Toda trajetória dá exatamente V_0.

4.2 Como construir o martingale na prática

Não conhecemos $\mathcal{M}^*$, mas temos uma aproximação da função valor pelo LSM: $\hat{V}_m(s) = \max[h(s), \hat{C}_m(s)]$. Construímos o martingale pelos seus incrementos:

$$\mathcal{M}_m = \mathcal{M}_{m-1} + e^{-rt_m}\Big(\hat{V}_m(S_m) - \mathbb{E}[\hat{V}_m(S_m) \,|\, S_{m-1}]\Big). \tag{9.12}$$

Subtrair a esperança condicional garante que é um martingale, **seja qual for a qualidade de $\hat{V}$**. Uma $\hat{V}$ ruim dá um limite superior folgado, mas válido.

Resta calcular a esperança condicional. O livro faz uma **simulação aninhada**: J sucessores de cada ponto. Num modelo de um fator, há uma alternativa sem ruído: a **quadratura de Gauss-Hermite**. Com n nós x_k e pesos w_k da normal padrão,

$$\mathbb{E}[\hat{V}(S_m) \,|\, S_{m-1}] \approx \sum_{k=1}^{n} w_k \, \hat{V}\Big(S_{m-1} e^{(r-\frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\, x_k}\Big). \tag{9.13}$$

Por que isso importa? O ruído da simulação aninhada entra dentro do máximo. Pela desigualdade de Jensen, $\mathbb{E}[\max(\cdot)]$ cresce com o ruído: o limite superior fica viesado para cima, e tanto mais quanto menor J.

Cuidado com a função valor fora do dinheiro. O limite superior usa $\hat{V}$ em todo lugar, inclusive em regiões em que a regressão só dentro do dinheiro não tem dados e um polinômio de grau 8 extrapola de forma descontrolada. Por isso usamos a regressão em todas as trajetórias para $\hat{V}$, limitamos x à faixa observada e truncamos $\hat{C}$ em zero.

Laboratório 4.1 — Com a função valor exata, o limite é exato

Primeiro, um teste de sanidade: usamos como $\hat{V}$ o valor exato da árvore.

```
def dual(S, pay, r, T, sig, Vf, J=None, rng=None, n_gh=40):
    """Limite superior dual. Com J, esperança condicional por simulação aninhada; sem J, por
    ↪ Gauss-Hermite."""
    M, I = S.shape[0] - 1, S.shape[1]
    dt = T / M
    if J is None:
        x, w = np.polynomial.hermite_e.hermegauss(n_gh)
```

```
        w = w / w.sum()
    Mt = np.zeros(I)
    U = np.full(I, -np.inf)
    for m in range(1, M + 1):
        d = np.exp(-r * m * dt)
        if J is None:
            suc = S[m - 1][:, None] * np.exp((r - 0.5 * sig**2) * dt + sig * np.sqrt(dt) * x)
            esp = Vf(m, suc.ravel()).reshape(I, -1) @ w
        else:
            z = rng.standard_normal((I, J))
            suc = S[m - 1][:, None] * np.exp((r - 0.5 * sig**2) * dt + sig * np.sqrt(dt) * z)
            esp = Vf(m, suc.ravel()).reshape(I, J).mean(axis=1)
        Mt = Mt + d * (Vf(m, S[m]) - esp)
        U = np.maximum(U, d * pay(S[m]) - Mt)
    return float(U.mean()), float(U.std(ddof=1) / np.sqrt(I)), float(U.std(ddof=1))


v_arv, tab25 = arvore_bermudana(pay_put, datas=25, fator=40, guardar=True, **p_)
V_exata = lambda m, s: np.interp(np.log(s), *tab25[m])

rng = np.random.default_rng(905)
S3 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 2_000, rng, momentos=False)
linhas = []
for J in [10, 50, 500, None]:
    v, ep, dp = dual(S3, pay_put, PUT["r"], PUT["T"], PUT["sig"], V_exata, J=J, rng=rng)
    linhas.append({"Esperança condicional": f"Aninhada, J = {J}" if J else "Gauss-Hermite (40
↪   nós)", "Limite superior": v, "Desvio-padrão por trajetória": dp, "Folga sobre a bermudana": v
↪   - v_arv})
ex = pd.DataFrame(linhas)
tabela(ex)
ok(ex["Folga sobre a bermudana"].iloc[0] > ex["Folga sobre a bermudana"].iloc[2] > ex["Folga sobre
↪   a bermudana"].iloc[3] - 1e-4, "a folga cai quando a esperança condicional fica mais precisa")
ok(abs(ex["Folga sobre a bermudana"].iloc[-1]) < 0.003 and ex["Desvio-padrão por
↪   trajetória"].iloc[-1] < 0.05, "com função valor exata e sem ruído, o limite é exato e quase
↪   sem variância")
```

Esperança condicional	Limite superior	Desvio-padrão por trajetória	Folga sobre a bermudana
Aninhada, J = 10	5,059	0,948	0,589
Aninhada, J = 50	4,637	0,394	0,167
Aninhada, J = 500	4,492	0,126	0,023
Gauss-Hermite (40 nós)	4,469	0,005	-0,000

```
OK — a folga cai quando a esperança condicional fica mais precisa
OK — com função valor exata e sem ruído, o limite é exato e quase sem variância
```

Leitura. A primeira linha mostra o tamanho do problema: com $J = 10$, o limite superior fica mais de meio real acima do valor verdadeiro, e isso **com a função valor perfeita**. A folga vem inteira do ruído aninhado. Com a quadratura, ela desaparece, e o desvio por trajetória vai a quase zero, como o Passo 4 previa.

Laboratório 4.2 — O intervalo completo para a put

Agora o caso real: função valor estimada pelo LSM.

```
def funcao_valor(fval, pay, K, M):
    def Vf(m, s):
        h = pay(s)
```

```
        if m == M:
            return h
        c, (a, b) = fval[m]
        return np.maximum(h, np.maximum(P.polyval(np.clip(s / K, a, b), c), 0.0))
    return Vf


rng = np.random.default_rng(906)
S1 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 16_384, rng)
S2 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 100_000, rng)
S3 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 2_000, rng, momentos=False)
linhas = []
for grau in [3, 5, 8]:
    regra, fval, _ = lsm_regressoes(S1, pay_put, 40.0, PUT["r"], PUT["T"], grau)
    lo, ep_lo = lsm_fora_amostra(S2, pay_put, 40.0, PUT["r"], PUT["T"], regra)
    Vf = funcao_valor(fval, pay_put, 40.0, 25)
    up_a, ep_a, _ = dual(S3, pay_put, PUT["r"], PUT["T"], PUT["sig"], Vf, J=50, rng=rng)
    up_q, ep_q, _ = dual(S3, pay_put, PUT["r"], PUT["T"], PUT["sig"], Vf)
    linhas.append({"Grau": grau, "LSM (inferior)": lo, "Dual aninhado J=50": up_a, "Dual
↪  Gauss-Hermite": up_q, "Largura do intervalo (quadratura)": up_q - lo})
du = pd.DataFrame(linhas)
tabela(du)
print("Referência bermudana de 25 datas:", f3(ref[("put", 25)]))
ok((du["LSM (inferior)"] < ref[("put", 25)] + 0.01).all() and (du["Dual Gauss-Hermite"] >
↪  ref[("put", 25)] - 0.01).all(), "a bermudana fica entre os dois limites")
ok(du["Largura do intervalo (quadratura)"].iloc[-1] < 0.05, "com grau 8 e quadratura, o intervalo
↪  tem largura menor que 0,05")
ok((du["Dual aninhado J=50"] > du["Dual Gauss-Hermite"] + 0.05).all(), "a simulação aninhada com
↪  J=50 infla o limite superior")
```

Grau	LSM (inferior)	Dual aninhado J=50	Dual Gauss-Hermite	Largura do intervalo (quadratura)
3	4,475	4,713	4,568	0,093
5	4,471	4,675	4,508	0,038
8	4,470	4,683	4,496	0,026

```
Referência bermudana de 25 datas: 4,469
OK — a bermudana fica entre os dois limites
OK — com grau 8 e quadratura, o intervalo tem largura menor que 0,05
OK — a simulação aninhada com J=50 infla o limite superior
```

5. Reproduzindo a Tabela 7.1, com a referência certa

O livro roda cada configuração 25 vezes. Fazemos o mesmo para $M = 25$ e $M = 50$, com os parâmetros do livro para o dual aninhado ($I_1 = 16.384$, $J = 50$), e acrescentamos o dual por quadratura.

```
rng = np.random.default_rng(907)
corridas = {}
t0 = perf_counter()
for M in [25, 50]:
    res = {"LSM": [], "Dual J=50": [], "Dual quadratura": []}
    for _ in range(15):
        S1 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], M, 16_384, rng)
        regra, fval, _ = lsm_regressoes(S1, pay_put, 40.0, PUT["r"], PUT["T"], 8)
        S2 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], M, 20_000, rng)
        res["LSM"].append(lsm_fora_amostra(S2, pay_put, 40.0, PUT["r"], PUT["T"], regra)[0])
```

```
        S3 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], M, 1_000, rng, momentos=False)
        Vf = funcao_valor(fval, pay_put, 40.0, M)
        res["Dual J=50"].append(dual(S3, pay_put, PUT["r"], PUT["T"], PUT["sig"], Vf, J=50,
↪   rng=rng)[0])
        res["Dual quadratura"].append(dual(S3, pay_put, PUT["r"], PUT["T"], PUT["sig"], Vf)[0])
    corridas[M] = res
print("Tempo total:", f3(perf_counter() - t0), "s")

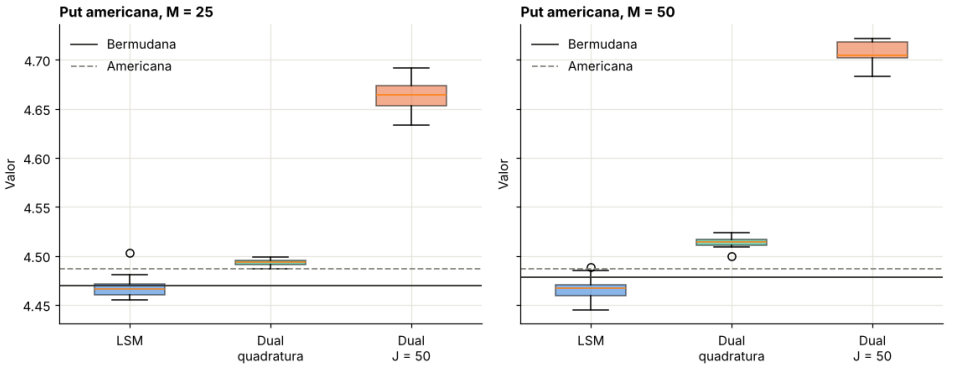
linhas = []
livro = {25: (4.452, 4.651), 50: (4.470, 4.595)}
for M, res in corridas.items():
    linhas.append({"M": M, "LSM: média": float(np.mean(res["LSM"])), "Livro LSM": livro[M][0],
↪   "Dual J=50: média": float(np.mean(res["Dual J=50"])), "Livro dual": livro[M][1], "Dual
↪   quadratura: média": float(np.mean(res["Dual quadratura"])), "Bermudana": ref[("put", M)],
↪   "Americana": am_put})
t71 = pd.DataFrame(linhas)
tabela(t71)

fig, eixos = plt.subplots(1, 2, figsize=(10.5, 4.2), sharey=True)
for ax, M in zip(eixos, [25, 50]):
    dados = [corridas[M]["LSM"], corridas[M]["Dual quadratura"], corridas[M]["Dual J=50"]]
    caixa = ax.boxplot(dados, tick_labels=["LSM", "Dual\nquadratura", "Dual\nJ = 50"],
↪   patch_artist=True, widths=0.5)
    for b, cor in zip(caixa["boxes"], [AZUL, VERDE, LARANJA]):
        b.set_facecolor(cor)
        b.set_alpha(0.55)
    ax.axhline(ref[("put", M)], color="#333330", lw=1.2, label="Bermudana")
    ax.axhline(am_put, color=CINZA, lw=1.2, ls="--", label="Americana")
    ax.set_title(f"Put americana, M = {M}")
    ax.set_ylabel("Valor")
    ax.legend(loc="upper left")
plt.tight_layout()
plt.show()

for M in [25, 50]:
    ok(abs(np.mean(corridas[M]["LSM"]) - ref[("put", M)]) < 0.02, f"M = {M}: o LSM fica a menos
↪   de 0,02 da bermudana")
    ok(np.mean(corridas[M]["Dual quadratura"]) - ref[("put", M)] < 0.5 *
↪   (np.mean(corridas[M]["Dual J=50"]) - ref[("put", M)]), f"M = {M}: a quadratura corta a folga
↪   do dual pela metade ou mais")
```

```
Tempo total: 14,141 s
```

M	LSM: média	Livro LSM	Dual J=50: média	Livro dual	Dual quadratura: média	Bermudana	Americana
25	4,468	4,452	4,662	4,651	4,493	4,469	4,487
50	4,466	4,470	4,708	4,595	4,514	4,478	4,487




```
OK — M = 25: o LSM fica a menos de 0,02 da bermudana
OK — M = 25: a quadratura corta a folga do dual pela metade ou mais
OK — M = 50: o LSM fica a menos de 0,02 da bermudana
OK — M = 50: a quadratura corta a folga do dual pela metade ou mais
```

Leitura. Duas correções à leitura do livro:

1. Parte da "diferença" de \$-\$0,034 atribuída ao LSM com $M = 25$ é a diferença entre americana e bermudana, de cerca de 0,017. Medido contra a referência correta, o LSM erra bem pouco.
2. A distância do dual, de 0,165, vem principalmente do ruído da simulação aninhada. Com quadratura, o limite superior fica perto do valor verdadeiro.

---

6. Um payoff difícil: o short condor

Livro: §7.4.2

$$h(s) = \min\left[40,\ \max(90 - s, 0) + \max(s - 110, 0)\right]. \tag{9.14}$$

O payoff é **não convexo**, com duas quinas e um platô. Polinômios simples têm dificuldade de aproximá-lo, e o livro conclui que a convergência do LSM é fraca. Como o payoff é positivo quase em todo lugar, regredimos em todas as trajetórias.

Laboratório 6.1 — O condor com a referência certa

```
rng = np.random.default_rng(908)
linhas = []
for M in [25, 50]:
    S1 = simular(CONDOR["S0"], CONDOR["r"], CONDOR["sig"], CONDOR["T"], M, 16_384, rng)
    S2 = simular(CONDOR["S0"], CONDOR["r"], CONDOR["sig"], CONDOR["T"], M, 50_000, rng)
    S3 = simular(CONDOR["S0"], CONDOR["r"], CONDOR["sig"], CONDOR["T"], M, 1_500, rng,
↪   momentos=False)
    for grau in [4, 8]:
        regra, fval, _ = lsm_regressoes(S1, pay_condor, 100.0, CONDOR["r"], CONDOR["T"], grau,
↪   so_itm=False)
        lo, _ = lsm_fora_amostra(S2, pay_condor, 100.0, CONDOR["r"], CONDOR["T"], regra)
        up, _, _ = dual(S3, pay_condor, CONDOR["r"], CONDOR["T"], CONDOR["sig"],
↪   funcao_valor(fval, pay_condor, 100.0, M))
```

```
        linhas.append({"M": M, "Grau": grau, "LSM (inferior)": lo, "Dual quadratura (superior)":
↪  up, "Bermudana": ref[("condor", M)], "LSM − bermudana": lo - ref[("condor", M)]})
con = pd.DataFrame(linhas)
tabela(con)
print("Livro: LSM médio de 25,977 (M=25) e 26,027 (M=50), comparado com 26,977.")
ok((con["LSM (inferior)"] <= con["Bermudana"] + 0.05).all() and (con["Dual quadratura
↪  (superior)"] >= con["Bermudana"] - 0.05).all(), "no condor, a bermudana também fica entre os
↪  limites")
ok(con.query("Grau == 8")["LSM − bermudana"].abs().max() < 0.3, "com grau 8, o LSM fica a menos
↪  de 0,3 da bermudana")
```

M	Grau	LSM (inferior)	Dual quadratura (superior)	Bermudana	LSM − bermudana
25	4	25,474	27,168	26,015	-0,541
25	8	25,878	26,288	26,015	-0,136
50	4	25,449	27,790	26,332	-0,884
50	8	26,128	26,593	26,332	-0,204

```
Livro: LSM médio de 25,977 (M=25) e 26,027 (M=50), comparado com 26,977.
OK — no condor, a bermudana também fica entre os limites
OK — com grau 8, o LSM fica a menos de 0,3 da bermudana
```

Leitura. O livro mede uma diferença de cerca de \$-\$1,000 com $M = 25$. Mas a bermudana de 25 datas vale 26,015, e a americana, 26,977: **quase toda a diferença é efeito das datas de exercício**, e não fraqueza do LSM. No condor, o efeito é grande porque a volatilidade é alta (50%) e o payoff tem um teto, o que dá muito valor a exercer exatamente no momento certo.

---

7. Fronteira de exercício

A regra do LSM define uma fronteira: em cada data, o maior nível do índice em que se exerce. Comparamos com a fronteira exata da árvore.

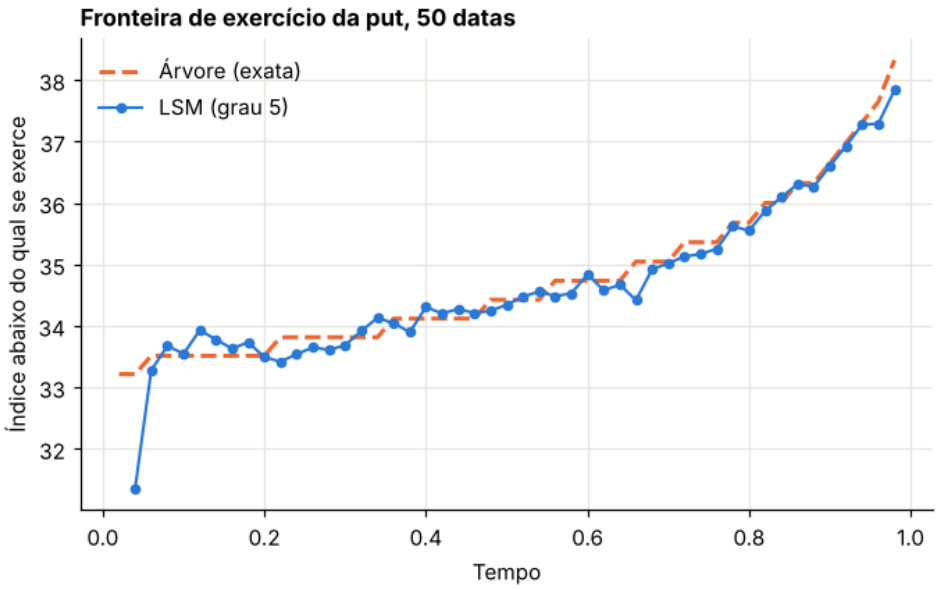
```
rng = np.random.default_rng(909)
M = 50
S1 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], M, 50_000, rng)
regra, _, _ = lsm_regressoes(S1, pay_put, 40.0, PUT["r"], PUT["T"], 5)
grade = np.linspace(20, 40, 4001)
fr_lsm = []
for m in range(1, M):
    exerce = (pay_put(grade) > 0) & (pay_put(grade) > continuacao(regra, m, grade, 40.0))
    fr_lsm.append(grade[exerce].max() if exerce.any() else np.nan)

_, tab50 = arvore_bermudana(pay_put, datas=M, fator=40, guardar=True, **p_)
fr_arv = []
N, fator = M * 40, 40
dt_f = PUT["T"] / N
u_f = np.exp(PUT["sig"] * np.sqrt(dt_f))
d_f = 1 / u_f
q_f = (np.exp(PUT["r"] * dt_f) - d_f) / (u_f - d_f)
for m in range(1, M):
    _, Vn = tab50[m + 1]
    Vc = Vn.copy()
    for _ in range(fator):
```

```
        Vc = np.exp(-PUT["r"] * dt_f) * (q_f * Vc[1:] + (1 - q_f) * Vc[:-1])
    S_no = PUT["S0"] * u_f ** np.arange(m * fator + 1) * d_f ** (m * fator - np.arange(m * fator +
↪   1))
    exerce = pay_put(S_no) > Vc + 1e-12
    fr_arv.append(S_no[exerce].max() if exerce.any() else np.nan)

ts = np.arange(1, M) / M
fig, ax = plt.subplots()
ax.plot(ts, fr_arv, color=LARANJA, ls="--", label="Árvore (exata)")
ax.plot(ts, fr_lsm, color=AZUL, marker="o", markersize=4, lw=1.4, label="LSM (grau 5)")
ax.set_title("Fronteira de exercício da put, 50 datas")
ax.set_xlabel("Tempo")
ax.set_ylabel("Índice abaixo do qual se exerce")
ax.legend()
plt.show()

dif = np.nanmean(np.abs(np.array(fr_lsm) - np.array(fr_arv)))
print("Diferença média entre as fronteiras:", f3(float(dif)))
ok(dif < 0.5, "a fronteira do LSM fica perto da exata")
```




```
Diferença média entre as fronteiras: 0,203
OK — a fronteira do LSM fica perto da exata
```

Pequenos desvios na fronteira custam pouco: o valor perto dela é quase indiferente entre exercer e continuar. Esse é o motivo de o LSM funcionar bem mesmo com regressões imperfeitas.

---

8. Redução de variância

Livro: §7.4.1

O livro usa duas técnicas, que estudaremos no Módulo 12:

- **Variáveis antitéticas:** para cada sorteio z, usa-se também $-z$.
- **Moment matching:** padronizam-se os sorteios de cada data para média 0 e variância 1 exatas.

Laboratório 8.1 — Efeito no desvio entre corridas

```
rng = np.random.default_rng(910)
linhas = []
for nome, mm, an in [("Sem redução", False, False), ("Antitéticas", False, True), ("Moment
↪  matching", True, False), ("As duas", True, True)]:
    vals = []
    for _ in range(30):
        S1 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 8_192, rng, momentos=mm,
↪  antiteticas=an)
        regra, _, _ = lsm_regressoes(S1, pay_put, 40.0, PUT["r"], PUT["T"], 5)
        S2 = simular(PUT["S0"], PUT["r"], PUT["sig"], PUT["T"], 25, 8_192, rng, momentos=mm,
↪  antiteticas=an)
        vals.append(lsm_fora_amostra(S2, pay_put, 40.0, PUT["r"], PUT["T"], regra)[0])
    linhas.append({"Técnica": nome, "Média": float(np.mean(vals)), "Desvio entre corridas":
↪  float(np.std(vals, ddof=1))})
vr = pd.DataFrame(linhas).set_index("Técnica")
tabela(vr.reset_index())
ok(vr.loc["Moment matching", "Desvio entre corridas"] < vr.loc["Sem redução", "Desvio entre
↪  corridas"], "o moment matching reduz o desvio entre corridas")
```

Técnica	Média	Desvio entre corridas
Sem redução	4,459	0,028
Antitéticas	4,461	0,023
Moment matching	4,460	0,021
As duas	4,460	0,025

```
OK — o moment matching reduz o desvio entre corridas
```

Atenção ao combinar as duas técnicas com a avaliação fora da amostra. Com moment matching, as trajetórias deixam de ser independentes entre si, e o erro-padrão calculado da forma usual deixa de ser exato. O desvio entre corridas, medido acima, é a forma segura de avaliar a precisão.

---

9. Síntese

- O valor americano é uma parada ótima; com M datas, uma bermudana.
- Comparar o LSM com o valor americano mistura dois erros; a referência certa é a bermudana com as mesmas datas.
- Decidir com o futuro realizado superestima; decidir com a regressão é um tempo de parada legítimo.
- A regressão só decide; o valor guardado é o realizado.
- Fora da amostra, o LSM é limite inferior; dentro da amostra, pode ser viesado para cima.

- O dual dá um limite superior para qualquer martingale, e exato para o de Doob.
- O ruído da simulação aninhada infla o limite superior; com quadratura, o intervalo fica estreito.

10. Notas sobre o livro neste capítulo

Local	Comentário
Tabelas 7.1 e 7.2	a referência (árvore de 500 passos) é americana; os algoritmos estimam bermudanas de M datas. Boa parte da "diferença" reportada, sobretudo no condor, é a diferença entre as duas
Tabela 7.2, nota	diz "American put option"; a tabela é do short condor
Tabela 7.1 e §7.4.1	o espaçamento entre LSM e dual é atribuído ao algoritmo; ele vem sobretudo do ruído da simulação aninhada ($J = 50$), que infla o limite superior
Equações (7.8) a (7.12)	o martingale deve ser expresso em unidades descontadas, como h; o texto mistura $B_0(t)[h_t - \mathcal{M}_t]$ com um $\mathcal{M}^*$ formado por valores não descontados
Equação (7.11)	o máximo passa de $t \in \{\Delta t, \dots, T\}$ para $t \in \{0, \dots, T\}$ sem comentário

11. Exercícios

Exercício 1. Por que a regra de exercício estimada, aplicada a trajetórias novas, define um tempo de parada?

Solução

A decisão em t_m depende apenas de S_m e de coeficientes calculados com outra amostra, independente das trajetórias avaliadas. Logo, $\{\tau \le t_m\}$ é determinado pela informação até t_m.

Exercício 2. No dual, o que acontece com o limite se usarmos $\mathcal{M} \equiv 0$?

Solução

O limite fica $\mathbb{E}[\max_m \tilde{h}_m]$: o valor de exercer no melhor momento de cada trajetória, com conhecimento perfeito do futuro. É o valor "clarividente" da Seção 2, um limite superior válido, mas muito folgado.

Exercício 3. Mostre que, com o martingale de Doob, $\tilde{h}_m - \mathcal{M}^*_m \le V_0$ em todas as datas e trajetórias.

Solução

$\tilde{V}_m = V_0 + \mathcal{M}^*_m - A_m$, com $A_m \geq 0$ e $\tilde{V}_m \geq \tilde{h}_m$. Logo, $\tilde{h}_m - \mathcal{M}^*_m \leq \tilde{V}_m - \mathcal{M}^*_m = V_0 - A_m \leq V_0$.

Exercício 4. Use a função `lsm_regressoes` numa call americana sem dividendos. O que o LSM deveria encontrar?

Solução

Que nunca vale a pena exercer antes do vencimento (Módulo 6). A regressão deve dar continuação sempre acima do exercício; o valor fica igual ao da europeia, a menos do ruído.

Exercício 5. Em modelos com várias variáveis de estado, como Heston, por que a quadratura de Gauss-Hermite fica mais difícil, e o que se pode fazer?

Solução

A esperança condicional passa a ser uma integral em várias dimensões, e o número de nós cresce exponencialmente. Pode-se usar quadratura esparsa, ou voltar à simulação aninhada com J maior e ciente do viés para cima. O LSM, por sua vez, se adapta bem: basta incluir a variância nas funções-base.

```
print(f"Módulo 9 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 9 concluído: 22 verificações aprovadas.
```

Parte III

Precificação calibrada ao mercado

Capítulo 10

Calibração ao mercado com o modelo de saltos de Merton

Introdução

Este capítulo junta pela primeira vez todas as etapas da precificação calibrada ao mercado, com um modelo simples o bastante para que cada etapa fique clara: o modelo de **Merton (1976)**, que acrescenta ao movimento browniano geométrico saltos de tamanho aleatório que chegam em instantes aleatórios.

Os saltos explicam o que Black-Scholes não explica em prazos curtos: quedas bruscas, caudas pesadas e um skew acentuado nos vencimentos próximos. Com saltos de tamanho aleatório, porém, o mercado deixa de ser completo, e a escolha da medida neutra ao risco passa a depender de uma hipótese. A precificação calibrada ao mercado contorna esse problema: em vez de supor os parâmetros dos saltos, ela os **lê** nos preços das opções.

O roteiro é o que se repete no restante do livro: precificar opções europeias no modelo, calibrar os parâmetros a um conjunto de opções negociadas e usar o modelo calibrado para precificar e analisar contratos novos. O capítulo também discute por que essa calibração é delicada: combinações diferentes de parâmetros podem ajustar o mercado quase igualmente bem.

Fundamentos teóricos

Processo de Poisson composto e o modelo de Merton

Definição 10.1 (Processo de Poisson). Um processo de contagem N_t é de Poisson com intensidade $\lambda > 0$ se tem incrementos independentes e estacionários e $\mathbb{P}(N_{t+h} - N_t = 1) = \lambda h + o(h)$, $\mathbb{P}(N_{t+h} - N_t \geq 2) = o(h)$. Então $N_t \sim \text{Poisson}(\lambda t)$ e $N_t - \lambda t$ é um martingal: λt é o *compensador* de N_t.

Definição 10.2 (Modelo de Merton, 1976). Sob $\mathbb{Q}$,

$$\frac{dS_t}{S_{t-}} = (r - r_J)\,dt + \sigma\,dW_t + J_t\,dN_t, \qquad r_J = \lambda(e^{\mu_J + \delta^2/2} - 1), \tag{10.1}$$

com $\ln(1+J) \sim N(\mu_J, \delta^2)$ independente de W e N. O termo $-r_J$ é a correção de deriva que compensa o salto médio $\mathbb{E}[J] = e^{\mu_J + \delta^2/2} - 1$.

Proposição 10.3 (Solução e martingalidade). *A solução de* (10.1) *é* $S_t = S_0 \exp\big((r - r_J - \tfrac{1}{2}\sigma^2)t + \sigma W_t\big)\prod_{k=1}^{N_t}(1+J_k)$*, e* $e^{-rt}S_t$ *é um* $\mathbb{Q}$*-martingal. A função característica de* $\ln(S_T/S_0)$ *é* (7.1)*.*

Demonstração. A parte contínua é tratada por Itô como no Capítulo 4. Para a parte de saltos, condicione em $N_T = n$: o produto tem esperança $(1+\mathbb{E}J)^n$; somando sobre a lei de Poisson, $\mathbb{E}[\prod_{k\le N_T}(1+J_k)] = e^{\lambda T\,\mathbb{E}J} = e^{r_J T}$, que cancela o fator $e^{-r_J T}$. ∎

Proposição 10.4 (Fórmula em série). *Condicionando no número de saltos,*

$$C^{\mathrm{M}} = \sum_{n=0}^{\infty} \frac{e^{-\lambda' T}(\lambda' T)^n}{n!}\, C^{\mathrm{BS}}(S_0, K, T, r_n, \sigma_n), \tag{10.2}$$

com $\lambda' = \lambda(1+\mathbb{E}J)$*,* $\sigma_n^2 = \sigma^2 + n\delta^2/T$ *e* $r_n = r - r_J + n\ln(1+\mathbb{E}J)/T$ *(Merton, 1976).*

Observação 10.5 (Mercado incompleto). Com saltos de tamanho aleatório, o ativo e a conta remunerada não replicam uma opção: o mercado é incompleto, e a MME não é única. A escolha de $\mathbb{Q}$ com saltos de mesma lei sob $\mathbb{P}$ e $\mathbb{Q}$ é uma hipótese de Merton (risco de salto diversificável). A precificação calibrada ao mercado contorna a questão: os parâmetros de $\mathbb{Q}$ são *lidos* dos preços de opções por calibração.

Calibração como problema inverso

Definição 10.6 (Problema de calibração). Dados preços C_j^{mkt} de opções com strikes e prazos (K_j, T_j), procure

$$\hat{\vartheta} = \arg\min_{\vartheta\in\Theta}\ \frac{1}{m}\sum_{j=1}^{m}\big(C_j^{\mathrm{mod}}(\vartheta) - C_j^{\mathrm{mkt}}\big)^2, \qquad \vartheta = (\sigma, \lambda, \mu_J, \delta). \tag{10.3}$$

Observação 10.7 (Boa colocação). Um problema é *bem posto* no sentido de Hadamard se a solução existe, é única e depende continuamente dos dados. Calibrações de modelos com saltos violam com frequência a unicidade e a estabilidade: combinações diferentes de (λ, μ_J, δ) geram superfícies quase iguais, e pequenas mudanças nos preços movem muito os parâmetros (Cont and Tankov, 2004). As consequências práticas são três: usar várias partidas no otimizador, combinar busca global e local, e acompanhar não só o erro de ajuste mas a estabilidade dos parâmetros. O Capítulo 14 trata a regularização.

Uma escolha de pesos que equilibra opções caras e baratas é minimizar erros em vol implícita, ou dividir cada termo de (10.3) pelo vega. Para erros pequenos, $\Delta C \approx \mathcal{V}\,\Delta\sigma$, e as duas abordagens coincidem em primeira ordem.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulo 8 do livro.

O que você vai conseguir fazer ao final

1. Escrever o modelo de Merton e interpretar cada parâmetro.

2. Deduzir a fórmula fechada em série de Poisson, prometida no Módulo 1.
3. Precificar por três métodos e diagnosticar quando um deles está errado.
4. Calibrar o modelo a uma superfície de preços, combinando busca global e local.
5. Reconhecer quando a calibração está mal posta.
6. Simular o modelo calibrado sem viés de discretização.
7. Usar o modelo calibrado para precificar um exótico e medir o risco de modelo.

O plano

É o processo de seis etapas do Módulo 1, agora com as ferramentas completas:

Etapa	Aqui
Mercado	superfície sintética no estilo do EURO STOXX 50
Modelo	Merton (1976)
Vanilla	Lewis, FFT e série
Calibração	força bruta + mínimos quadrados
Simulação	esquema exato
Exótico	barreira, comparando modelos

Sobre os dados. O livro usa cotações do EURO STOXX 50 de 30/09/2014, distribuídas na plataforma do autor. Aqui geramos um mercado com o mesmo nível de índice, os mesmos prazos e a mesma faixa de strikes, mas a partir de um modelo **mais rico que o de Merton**: volatilidade estocástica com saltos. Assim reproduzimos a situação do livro, em que o modelo calibrado não é o verdadeiro, com a vantagem de conhecer a verdade.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
from time import perf_counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import quad, simpson
from scipy.optimize import brentq, brute, least_squares
from scipy.stats import norm, poisson
from IPython.display import display

def bsm(S, K, T, r, sig, tipo="call"):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    d2 = d1 - sig * np.sqrt(T)
    if tipo == "call":
        return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)

def vol_impl(preco, S, K, T, r):
    piso = max(S - K * np.exp(-r * T), 0.0)
    if not (piso + 1e-10 < preco < S):
        return np.nan
```

```
    return brentq(lambda s: float(bsm(S, K, T, r, s)) - preco, 1e-4, 4.0)

S0, R = 3225.93, 0.005
print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. O modelo

Livro: §8.2

Sob Q, o índice segue

$$\frac{dS_t}{S_{t-}} = (r - r_J)\,dt + \sigma\,dZ_t + (e^J - 1)\,dN_t, \tag{10.4}$$

em que N_t é um processo de Poisson de intensidade λ e cada log-salto é $J \sim N(\mu_J, \delta^2)$, independente de tudo. O termo r_J compensa o crescimento médio dos saltos (Módulo 4):

$$r_J = \lambda\,\mathbb{E}[e^J - 1] = \lambda(e^{\mu_J + \delta^2/2} - 1). \tag{10.5}$$

Parâmetro	Significado	Efeito na superfície
σ	volatilidade da parte contínua	nível
λ	saltos por ano, em média	intensidade do skew e das caudas
μ_J	log-salto médio	direção do skew; negativo para índices
δ	dispersão dos saltos	curvatura do smile

> **Errata do livro.** A §8.2 especifica $\ln(1 + J_t) \sim N(\ln(1 + \mu_J) - \frac{1}{2}\delta^2, \delta^2)$. Com essa lei, o salto relativo médio seria μ_J e a compensação seria $r_J = \lambda\mu_J$. Mas a fórmula de r_J que o livro usa, a função característica (8.2) e os scripts correspondem a $\ln(1 + J) \sim N(\mu_J, \delta^2)$, que é a lei adotada aqui.

Laboratório 1.1 — Como é uma trajetória

Simulamos com os parâmetros do script de simulação do livro, que vêm da calibração ao prazo mais curto: $\sigma = 0{,}113$, $\lambda = 3{,}559$, $\mu_J = -0{,}075$, $\delta = 0{,}041$.

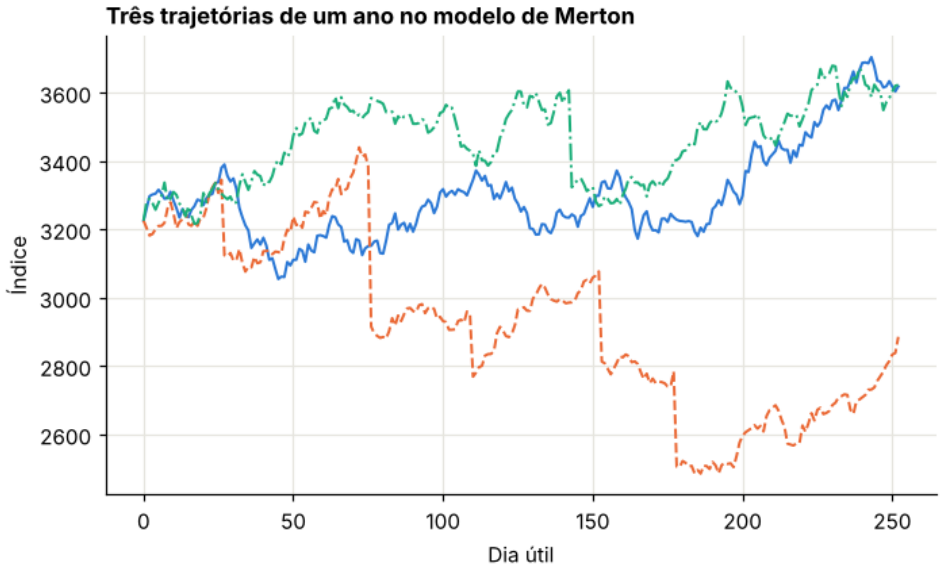
```
LIVRO = dict(sig=0.113, lam=3.559, mu=-0.075, delta=0.041)


def r_j(lam, mu, delta):
    return lam * (np.exp(mu + 0.5 * delta**2) - 1)


rng = np.random.default_rng(1001)
```

```
dias = 252
dt = 1 / dias
fig, ax = plt.subplots()
for cor, est in zip([AZUL, LARANJA, VERDE], ["-", "--", "-."]):
    n = rng.poisson(LIVRO["lam"] * dt, dias)
    J = n * LIVRO["mu"] + np.sqrt(n) * LIVRO["delta"] * rng.standard_normal(dias)
    x = (R - r_j(**{k: LIVRO[k] for k in ("lam", "mu", "delta")}) - 0.5 * LIVRO["sig"] ** 2) * dt
 ↪  + LIVRO["sig"] * np.sqrt(dt) * rng.standard_normal(dias) + J
    ax.plot(np.arange(dias + 1), S0 * np.exp(np.r_[0.0, np.cumsum(x)]), color=cor, ls=est, lw=1.3)
ax.set_title("Três trajetórias de um ano no modelo de Merton")
ax.set_xlabel("Dia útil")
ax.set_ylabel("Índice")
plt.show()

print("Compensação r_J:", f3(r_j(LIVRO["lam"], LIVRO["mu"], LIVRO["delta"])), "| Volatilidade
 ↪  total equivalente:", f3(float(np.sqrt(LIVRO["sig"] ** 2 + LIVRO["lam"] * (LIVRO["mu"] ** 2 +
 ↪  LIVRO["delta"] ** 2)))))
```




```
Compensação r_J: -0,254 | Volatilidade total equivalente: 0,197
```

---

2. Três formas de precificar

Livro: §8.3

2.1 A série de Poisson, deduzida

Passo 1: condicionar no número de saltos. Se $N_T = n$, o log-retorno é a soma de uma normal (difusão) com n normais (saltos), logo é normal:

$$\ln \frac{S_T}{S_0} \,\Big|\, N_T = n \;\sim\; N\Big((r - r_J - \tfrac{1}{2}\sigma^2)T + n\mu_J,\;\; \sigma^2 T + n\delta^2\Big). \tag{10.6}$$

Passo 2: reescrever como um BSM. Procuramos (r_n, σ_n) tais que essa normal seja a de um BSM. A variância dá $\sigma_n^2 = \sigma^2 + n\delta^2/T$. Para a média, exigimos $(r_n - \frac{1}{2}\sigma_n^2)T = (r - r_J - \frac{1}{2}\sigma^2)T + n\mu_J$. Resolvendo:

$$r_n = r - r_J + \frac{n(\mu_J + \frac{1}{2}\delta^2)}{T} = r - r_J + \frac{n\ln(1+\kappa)}{T}, \qquad \kappa = e^{\mu_J + \delta^2/2} - 1. \tag{10.7}$$

Passo 3: o desconto. O BSM com taxa r_n desconta a r_n, mas o desconto correto é a r. Logo, condicionado a n:

$$\mathbb{E}[e^{-rT}(S_T - K)^+ \mid n] = e^{(r_n - r)T}\, C^{BSM}(S_0, K, T, r_n, \sigma_n). \tag{10.8}$$

Passo 4: tirar a média nos n. O fator $e^{(r_n - r)T} = e^{-r_J T}(1+\kappa)^n$ se junta à probabilidade de Poisson:

$$e^{-\lambda T}\frac{(\lambda T)^n}{n!}\, e^{-\lambda\kappa T}(1+\kappa)^n = e^{-\lambda' T}\frac{(\lambda' T)^n}{n!}, \qquad \lambda' = \lambda(1+\kappa). \tag{10.9}$$

Resulta a fórmula de Merton:

$$\boxed{C_0 = \sum_{n=0}^{\infty} e^{-\lambda' T}\frac{(\lambda' T)^n}{n!}\, C^{BSM}(S_0, K, T, r_n, \sigma_n).} \tag{10.10}$$

É uma média de preços BSM, com pesos de Poisson sob uma intensidade ajustada. Foi a fórmula usada como caixa-preta no Módulo 1.

2.2 Lewis e FFT

A função característica de $\ln(S_T/S_0) - rT$ é a do Módulo 7, e as fórmulas de Lewis e Carr-Madan se aplicam diretamente.

Laboratório 2.1 — Os três métodos e um diagnóstico

Usamos os parâmetros e o prazo do script de simulação do livro ($T = 0{,}22$) e os mesmos strikes da sua tabela de resultados.

```
def cf_merton(u, T, sig, lam, mu, delta):
    om = -0.5 * sig**2 - r_j(lam, mu, delta)
    return np.exp(T * (1j * u * om - 0.5 * sig**2 * u**2 + lam * (np.exp(1j * u * mu - 0.5 * u**2
    ↪  * delta**2) - 1)))


def lewis(S, K, T, r, cf, U=300.0, n=3001):
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(S / K) + r * T
    z = np.linspace(0.0, U, n)
    h = np.exp(1j * z * k[:, None]) * cf(z - 0.5j)
    return S - np.sqrt(S * K) * np.exp(-0.5 * r * T) / np.pi * simpson(h.real / (z**2 + 0.25),
    ↪  x=z, axis=1)


def merton_serie(S, K, T, r, sig, lam, mu, delta, n_max=80):
    kap = np.exp(mu + 0.5 * delta**2) - 1
    lq = lam * (1 + kap)
    tot = 0.0
```

```
    for n in range(n_max):
        sn = np.sqrt(sig**2 + n * delta**2 / T)
        rn = r - r_j(lam, mu, delta) + n * np.log1p(kap) / T
        tot = tot + poisson.pmf(n, lq * T) * bsm(S, K, T, rn, sn)
    return tot


def cm_fft(S, T, r, cf, alpha=1.5, N=4096, eps=1 / 150, k_ref=0.0):
    """Grade de FFT com o log-strike ln(K/S) = k_ref exatamente no ponto N/2, como no script do
    ↪ livro."""
    eta = 2 * np.pi / (N * eps)
    b = 0.5 * N * eps - k_ref
    j = np.arange(N)
    v = eta * j
    w = v - (alpha + 1) * 1j
    psi = np.exp(-r * T) * np.exp(1j * w * r * T) * cf(w) / (alpha**2 + alpha - v**2 + 1j * (2 *
↪   alpha + 1) * v)
    peso = (3 + (-1.0) ** (j + 1) - (j == 0)) * eta / 3
    k = -b + eps * j
    return S * np.exp(k), S * np.exp(-alpha * k) / np.pi * np.fft.fft(np.exp(1j * b * v) * psi *
    ↪ peso).real


def lewis_quad(S, K, T, r, cf, U):
    """Como no script do livro: integração adaptativa de 0 a U."""
    k = np.log(S / K) + r * T
    f = lambda z: (np.exp(1j * z * k) * cf(z - 0.5j)).real / (z**2 + 0.25)
    return S - np.sqrt(S * K) * np.exp(-0.5 * r * T) / np.pi * quad(f, 0, U, limit=250)[0]


T_l = 0.22
cf_l = lambda u: cf_merton(u, T_l, **LIVRO)
Kl = np.arange(3000.0, 3601.0, 100.0)
serie = merton_serie(S0, Kl, T_l, R, **LIVRO)
lw = lewis(S0, Kl, T_l, R, cf_l, U=400.0, n=40001)
fft = np.array([cm_fft(S0, T_l, R, cf_l, k_ref=np.log(k / S0))[1][2048] for k in Kl])
q50 = np.array([lewis_quad(S0, k, T_l, R, cf_l, 50.0) for k in Kl])
livro_int = {3000: 269.749, 3100: 194.905, 3200: 130.761, 3300: 79.695, 3400: 43.104, 3500:
↪ 20.232, 3600: 8.104}
livro_fft = {3000: 269.733, 3100: 194.890, 3200: 130.785, 3300: 79.705, 3400: 43.076, 3500:
↪ 20.224, 3600: 8.132}
tm = pd.DataFrame({"Strike": Kl, "Série": serie, "Lewis (até 400)": lw, "FFT": fft, "Lewis até
↪ 50": q50, "Livro: Int": [livro_int[int(k)] for k in Kl], "Livro: FFT": [livro_fft[int(k)] for
↪ k in Kl]})
tabela(tm)

ok(np.max(np.abs(lw - serie)) < 1e-6, "Lewis com limite adequado coincide com a série")
ok(np.max(np.abs(q50 - tm["Livro: Int"])) < 1e-3, "integrar só até 50 reproduz exatamente os
↪ valores 'Int' do livro")
ok(np.max(np.abs(serie - tm["Livro: FFT"])) < 0.002 and np.max(np.abs(fft - serie)) < 0.002, "os
↪ valores FFT, do livro e daqui, é que estão corretos")
```

Strike	Série	Lewis (até 400)	FFT	Lewis até 50	Livro: Int	Livro: FFT
3.000,000	269,733	269,733	269,733	269,749	269,749	269,733
3.100,000	194,890	194,890	194,890	194,905	194,905	194,890
3.200,000	130,785	130,785	130,785	130,761	130,761	130,785
3.300,000	79,705	79,705	79,705	79,695	79,695	79,705
3.400,000	43,075	43,075	43,075	43,104	43,104	43,076
3.500,000	20,224	20,224	20,224	20,232	20,232	20,224
3.600,000	8,132	8,132	8,132	8,104	8,104	8,132

```
OK — Lewis com limite adequado coincide com a série
OK — integrar só até 50 reproduz exatamente os valores 'Int' do livro
OK — os valores FFT, do livro e daqui, é que estão corretos
```

Diagnóstico. O livro interpreta a coluna "Difference FFT/Int" como imprecisão da FFT. É o contrário: a FFT está certa, e a integração numérica do script para em $z = 50$. Com prazo curto e volatilidade baixa, o integrando ainda não decaiu nesse ponto (Módulo 7, Seção 4). Ter uma terceira fonte independente, a série, é o que permite saber qual dos dois está certo.

---

3. O mercado sintético

O livro calibra a calls do EURO STOXX 50 com strikes até 5% do spot, em três prazos (17, 80 e 171 dias). Geramos preços com os mesmos prazos e strikes a cada 25 pontos, a partir de um modelo de volatilidade estocástica com saltos (o modelo dos Módulos 2 e 7), acrescentando ruído do tamanho de meio spread.

```python
def cf_bates(u, T, v0, kappa, theta, nu, rho, lam, mu, delta):
    comp = lam * (np.exp(mu + 0.5 * delta**2) - 1)
    salto = np.exp(lam * T * (np.exp(1j * u * mu - 0.5 * u**2 * delta**2) - 1))
    xi = kappa - rho * nu * 1j * u
    d = np.sqrt(xi**2 + nu**2 * (1j * u + u**2))
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
    A = kappa * theta / nu**2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / nu**2 * (1 - e) / (1 - g * e)
    return np.exp(-1j * u * comp * T) * salto * np.exp(A + B * v0)


VERDADE = dict(v0=0.03, kappa=1.5, theta=0.05, nu=0.45, rho=-0.75, lam=0.4, mu=-0.10, delta=0.08)
PRAZOS = {"17 dias": 17 / 365, "80 dias": 80 / 365, "171 dias": 171 / 365}

rng = np.random.default_rng(1002)
linhas = []
for nome, T in PRAZOS.items():
    K = np.arange(np.ceil(0.95 * S0 / 25) * 25, 1.05 * S0, 25.0)
    preco = lewis(S0, K, T, R, lambda u: cf_bates(u, T, **VERDADE))
    ruido = rng.normal(0.0, np.maximum(0.25, 0.005 * preco))
    for k, p, e in zip(K, preco, ruido):
        linhas.append({"Prazo": nome, "T": T, "Strike": k, "Preço justo": p, "Cotação": p + e})
mercado = pd.DataFrame(linhas)
mercado["Vol. implícita"] = [vol_impl(l.Cotação, S0, l.Strike, l.T, R) for l in
 ↪  mercado.itertuples()]
print("Cotações:", len(mercado))
```

```
tabela(mercado.groupby("Prazo", sort=False).agg(Opções=("Strike", "size"), **{"Menor strike":
↪  ("Strike", "min"), "Maior strike": ("Strike", "max"), "Vol. implícita mínima": ("Vol.
↪  implícita", "min"), "Vol. implícita máxima": ("Vol. implícita", "max")}).reset_index())
ok(mercado["Vol. implícita"].notna().all(), "todas as cotações têm volatilidade implícita")
```

```
Cotações: 39
```

Prazo	Opções	Menor strike	Maior strike	Vol. implícita mínima	Vol. implícita máxima
17 dias	13	3.075,000	3.375,000	0,155	0,214
80 dias	13	3.075,000	3.375,000	0,166	0,206
171 dias	13	3.075,000	3.375,000	0,177	0,207

```
OK — todas as cotações têm volatilidade implícita
```

---

4. Calibração

Livro: §8.4

4.1 O problema

Escolhemos os parâmetros que minimizam a distância entre modelo e mercado. O livro usa a raiz do erro quadrático médio em preço:

$$\min_{\sigma,\lambda,\mu_J,\delta} \sqrt{\frac{1}{N}\sum_{n=1}^{N}\left(C_n^{*} - C_n^{M76}(\sigma,\lambda,\mu_J,\delta)\right)^2}. \tag{10.11}$$

4.2 Por que é difícil

O problema é **mal posto**: a solução pode não ser única, nem depender de forma estável dos dados. Na prática, aparecem três sintomas:

1. **Mínimos locais.** Um otimizador local fica preso no primeiro vale.
2. **Vales planos.** Combinações diferentes de parâmetros dão quase o mesmo erro. Em Merton, muitos saltos pequenos se parecem com volatilidade a mais.
3. **Restrições.** $\sigma, \delta > 0$ e $\lambda \geq 0$. O livro usa penalidades; aqui usamos limites diretos no otimizador.

4.3 A estratégia em duas etapas

Etapa global: avaliar o erro numa grade grosseira de parâmetros (força bruta) e escolher o melhor ponto. **Etapa local:** partir dele com um otimizador de mínimos quadrados.

Laboratório 4.1 — Calibração aos três prazos

```
def erros(x, dados):
    sig, lam, mu, delta = x
    saida = []
    for T, g in dados.groupby("T"):
        saida.append(lewis(S0, g["Strike"].values, T, R, lambda u: cf_merton(u, T, sig, lam, mu,
↪  delta)) - g["Cotação"].values)
    return np.concatenate(saida)


def rmse(x, dados):
    if x[0] <= 0 or x[1] < 0 or x[3] <= 0:
        return 1e6
    return float(np.sqrt(np.mean(erros(x, dados) ** 2)))


LIM = ([0.01, 0.0, -0.6, 0.001], [0.6, 5.0, 0.2, 0.5])
GRADE = ((0.075, 0.201, 0.025), (0.10, 0.401, 0.10), (-0.5, 0.01, 0.10), (0.10, 0.301, 0.10))


def calibrar(dados):
    t0 = perf_counter()
    x0 = brute(lambda x: rmse(x, dados), GRADE, finish=None)
    res = least_squares(lambda x: erros(x, dados), x0, bounds=LIM)
    return x0, res.x, perf_counter() - t0


x0_todos, x_todos, t_todos = calibrar(mercado)
tabela(pd.DataFrame({"Parâmetro": ["σ", "λ", "μ_J", "δ"], "Melhor ponto da grade": x0_todos,
↪  "Após otimização local": x_todos}))
print("Tempo:", f3(t_todos), "s | RMSE na grade:", f3(rmse(x0_todos, mercado)), "| RMSE final:",
↪  f3(rmse(x_todos, mercado)))

por_prazo = []
for nome, T in PRAZOS.items():
    g = mercado[mercado["T"] == T]
    por_prazo.append({"Prazo": nome, "RMSE em preço": rmse(x_todos, g)})
tabela(pd.DataFrame(por_prazo))
ok(rmse(x_todos, mercado) < rmse(x0_todos, mercado), "a etapa local melhora o ponto da grade")
ok(min(p["RMSE em preço"] for p in por_prazo) > 1.0, "com quatro parâmetros, Merton não ajusta
↪  bem nenhum dos três prazos ao mesmo tempo")
```

Parâmetro	Melhor ponto da grade	Após otimização local
σ	0,150	0,138
λ	0,200	0,698
μ_J	-0,400	-0,167
δ	0,200	0,081

```
Tempo: 2,625 s | RMSE na grade: 3,622 | RMSE final: 2,939
```

Prazo	RMSE em preço
17 dias	3,239
80 dias	2,023
171 dias	3,366

```
OK — a etapa local melhora o ponto da grade
OK — com quatro parâmetros, Merton não ajusta bem nenhum dos três prazos ao mesmo tempo
```

Laboratório 4.2 — Calibração só ao prazo mais curto

O livro observa que, restrito ao prazo mais curto, Merton ajusta muito bem. É o papel que os saltos terão no modelo geral: cuidar dos prazos curtos, enquanto a volatilidade estocástica cuida dos longos.

```
curto = mercado[mercado["Prazo"] == "17 dias"]
x0_curto, x_curto, t_curto = calibrar(curto)
tabela(pd.DataFrame({"Parâmetro": ["σ", "λ", "μ_J", "δ"], "Três prazos": x_todos, "Só 17 dias":
↪  x_curto, "Livro (só 17 dias)": [0.113, 3.559, -0.075, 0.041]}))

linhas = []
for nome, T in PRAZOS.items():
    g = mercado[mercado["T"] == T]
    linhas.append({"Prazo": nome, "RMSE: calibrado aos três": rmse(x_todos, g), "RMSE: calibrado
↪  a 17 dias": rmse(x_curto, g)})
fora = pd.DataFrame(linhas).set_index("Prazo")
tabela(fora.reset_index())
ok(fora.loc["17 dias", "RMSE: calibrado a 17 dias"] < 0.5, "no prazo curto, o ajuste fica dentro
↪  do tamanho do spread")
ok(fora.loc["171 dias", "RMSE: calibrado a 17 dias"] > 2 * fora.loc["171 dias", "RMSE: calibrado
↪  aos três"], "mas extrapola muito mal para prazos mais longos")
```

Parâmetro	Três prazos	Só 17 dias	Livro (só 17 dias)
σ	0,138	0,131	0,113
λ	0,698	5,000	3,559
μ_J	-0,167	-0,061	-0,075
δ	0,081	0,001	0,041

Prazo	RMSE: calibrado aos três	RMSE: calibrado a 17 dias
17 dias	3,239	0,496
80 dias	2,023	5,048
171 dias	3,366	8,126

```
OK — no prazo curto, o ajuste fica dentro do tamanho do spread
OK — mas extrapola muito mal para prazos mais longos
```

Leitura dos parâmetros. Repare na calibração ao prazo curto: muitos saltos e muito pequenos, como no livro ($\lambda = 3{,}559$, $\delta = 0{,}041$). Aqui o resultado é ainda mais extremo: λ para no limite superior imposto (5) e δ praticamente zera, ou seja, saltos quase determinísticos de tamanho μ_J. Com um único prazo, a calibração encontra uma combinação que imita a forma do smile, mas que não representa saltos raros e grandes, como os do modelo verdadeiro ($\lambda = 0{,}4$, $\mu_J = -0{,}10$). É a mesma identificabilidade fraca que vimos no Módulo 1.

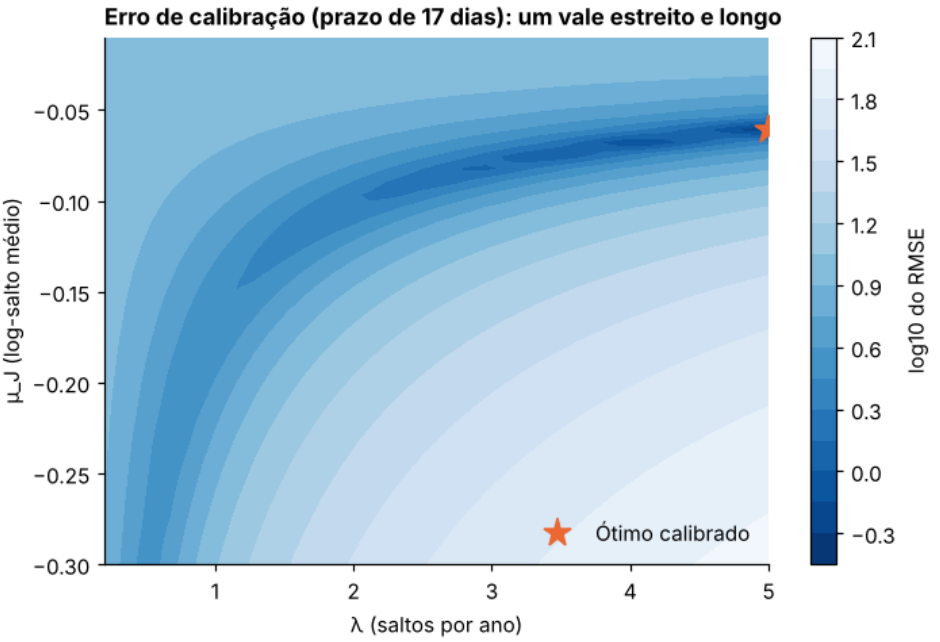
Laboratório 4.3 — A paisagem do erro

Fixamos σ e δ no ótimo do prazo curto e desenhamos o RMSE em função de λ e μ_J.

```
lam_g = np.linspace(0.2, 5.0, 41)
mu_g = np.linspace(-0.30, -0.01, 41)
Zg = np.array([[rmse([x_curto[0], l, m, x_curto[3]], curto) for l in lam_g] for m in mu_g])

fig, ax = plt.subplots(figsize=(7.2, 4.6))
cs = ax.contourf(lam_g, mu_g, np.log10(Zg), levels=20, cmap="Blues_r")
fig.colorbar(cs, ax=ax, label="log10 do RMSE")
ax.plot(x_curto[1], x_curto[2], marker="*", markersize=14, color=LARANJA, ls="none", label="Ótimo
↪  calibrado")
ax.set_title("Erro de calibração (prazo de 17 dias): um vale estreito e longo")
ax.set_xlabel("λ (saltos por ano)")
ax.set_ylabel("μ_J (log-salto médio)")
ax.legend(loc="lower right")
plt.show()

vale = Zg < 2 * Zg.min()
print("Fração da grade com RMSE até o dobro do mínimo:", f3(float(vale.mean())), "| Faixa de λ
↪  nessa região:", f3(float(lam_g[vale.any(axis=0)].min())), "a",
↪  f3(float(lam_g[vale.any(axis=0)].max())))
ok(np.ptp(lam_g[vale.any(axis=0)]) > 1.0, "há uma faixa larga de λ com erro próximo do mínimo")
```



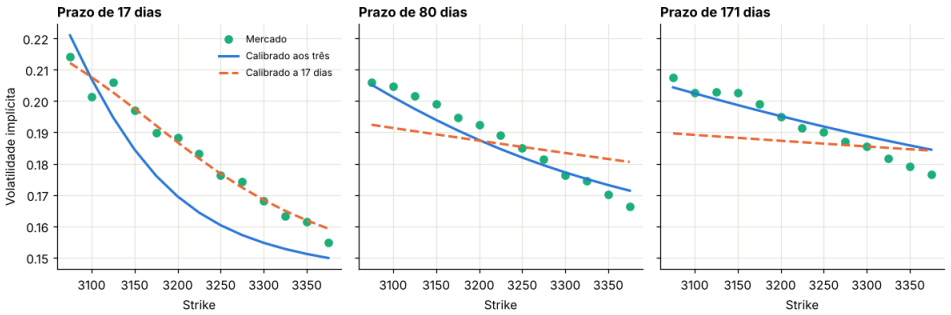

```
Fração da grade com RMSE até o dobro do mínimo: 0,007 | Faixa de λ nessa região: 3,320 a 5,000
OK — há uma faixa larga de λ com erro próximo do mínimo
```

O vale acompanha uma curva em que o produto $\lambda\mu_J$, aproximadamente o drift dos saltos, fica estável. Com os dados de um prazo, o mercado informa bem essa combinação, mas mal cada parâmetro separadamente.

Laboratório 4.4 — O ajuste em volatilidade implícita

Comparar em preço esconde problemas nos strikes altos, em que os preços são pequenos. A volatilidade implícita põe todos na mesma escala.

```
fig, eixos = plt.subplots(1, 3, figsize=(11, 3.8), sharey=True)
for ax, (nome, T) in zip(eixos, PRAZOS.items()):
    g = mercado[mercado["T"] == T]
    ax.plot(g["Strike"], g["Vol. implícita"], "o", color=VERDE, markersize=6, label="Mercado")
    for x, cor, est, rot in [(x_todos, AZUL, "-", "Calibrado aos três"), (x_curto, LARANJA, "--",
    ↪  "Calibrado a 17 dias")]:
        p = lewis(S0, g["Strike"].values, T, R, lambda u: cf_merton(u, T, *x))
        ax.plot(g["Strike"], [vol_impl(pp, S0, k, T, R) for pp, k in zip(p, g["Strike"])],
↪   color=cor, ls=est, label=rot)
    ax.set_title(f"Prazo de {nome}")
    ax.set_xlabel("Strike")
eixos[0].set_ylabel("Volatilidade implícita")
eixos[0].legend(fontsize=8)
plt.tight_layout()
plt.show()
```



---

5. Simulação

Livro: §8.5

5.1 Três esquemas

O livro apresenta duas discretizações. Na primeira, tudo é somado:

$$S_t = S_{t-\Delta t}\Big[1 + (r - r_J)\Delta t + \sigma\sqrt{\Delta t}\, z_t^1 + (e^{\mu_J + \delta z_t^2} - 1) y_t\Big]. \tag{10.12}$$

Na segunda, a difusão vira exponencial, mas o salto continua somado:

$$S_t = S_{t-\Delta t}\Big[e^{(r - r_J - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t}\, z_t^1} + (e^{\mu_J + \delta z_t^2} - 1) y_t\Big]. \tag{10.13}$$

Nas duas, se houver $y_t = 2$ saltos no mesmo passo, o tamanho aplicado é **o dobro de um único salto**, em vez da soma de dois saltos independentes, e o salto é somado à difusão, em vez de multiplicado. A média fica correta, mas a distribuição não.

O esquema **exato** usa o log-preço, em que tudo soma de verdade:

$$\ln S_t = \ln S_{t-\Delta t} + (r - r_J - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\, z_t^1 + y_t \mu_J + \sqrt{y_t}\,\delta\, z_t^2. \tag{10.14}$$

A soma de y_t normais independentes $N(\mu_J, \delta^2)$ é $N(y_t\mu_J, y_t\delta^2)$, daí o $\sqrt{y_t}$. Para uma opção europeia, nem é preciso discretizar: um único passo de tamanho T é exato.

Laboratório 5.1 — Há viés?

O livro mostra uma única simulação, com diferenças positivas e decrescentes em relação à fórmula (de +0,754 a +0,057). Com uma só semente, não dá para distinguir erro de amostragem de viés. Para medir o viés com precisão, usamos uma técnica do Módulo 12, as **variáveis comuns**: os três esquemas recebem exatamente os mesmos números aleatórios. O ruído quase se cancela na diferença, e o que sobra é o efeito do esquema. Repetimos 20 vezes, com 200 mil trajetórias e 50 passos, como no livro.

```
def tres_esquemas(rng, T, M, I, sig, lam, mu, delta):
    """Simula os três esquemas com os MESMOS números aleatórios (variáveis comuns)."""
    dt = T / M
    rj = r_j(lam, mu, delta)
    Sa, Sb, x = np.full(I, S0), np.full(I, S0), np.zeros(I)
    for _ in range(M):
        z1, z2, y = rng.standard_normal(I), rng.standard_normal(I), rng.poisson(lam * dt, I)
        salto_livro = (np.exp(mu + delta * z2) - 1) * y
        Sa = Sa * (1 + (R - rj) * dt + sig * np.sqrt(dt) * z1 + salto_livro)
        Sb = Sb * (np.exp((R - rj - 0.5 * sig**2) * dt + sig * np.sqrt(dt) * z1) + salto_livro)
        x += (R - rj - 0.5 * sig**2) * dt + sig * np.sqrt(dt) * z1 + y * mu + np.sqrt(y) * delta *
↪   z2
    return {"Livro, aditivo": Sa, "Livro, log da difusão": Sb, "Exato": S0 * np.exp(x)}


Kt = np.array([3000.0, 3200.0, 3400.0, 3600.0])
refe = merton_serie(S0, Kt, T_l, R, **LIVRO)
rng = np.random.default_rng(1003)
acum = {nome: [] for nome in ["Livro, aditivo", "Livro, log da difusão", "Exato"]}
for _ in range(20):
    S = tres_esquemas(rng, T_l, 50, 200_000, **LIVRO)
    pay = {n: np.exp(-R * T_l) * np.maximum(v[:, None] - Kt, 0.0) for n, v in S.items()}
    for n in acum:
        acum[n].append(pay[n].mean(axis=0) - (refe if n == "Exato" else
↪   pay["Exato"].mean(axis=0)))

linhas = []
for nome, rot in [("Exato", "Exato − fórmula"), ("Livro, aditivo", "Livro aditivo − exato"),
↪   ("Livro, log da difusão", "Livro log − exato")]:
    d = np.array(acum[nome])
    for k, mm, ee in zip(Kt, d.mean(axis=0), d.std(axis=0, ddof=1) / np.sqrt(20)):
        linhas.append({"Comparação": rot, "Strike": k, "Diferença média": mm, "Erro-padrão": ee,
↪   "Em erros-padrão": mm / ee})
vies = pd.DataFrame(linhas)
tabela(vies)
ok((vies.query("Comparação == 'Exato − fórmula'")["Em erros-padrão"].abs() < 3.5).all(), "o
↪   esquema exato não tem viés detectável")
ok((vies.query("Comparação == 'Livro log − exato'")["Em erros-padrão"] > 4).all(), "com as mesmas
↪   trajetórias, o viés do esquema do livro aparece com clareza")
```

Comparação	Strike	Diferença média	Erro-padrão	Em erros-padrão
Exato − fórmula	3.000,000	0,044	0,117	0,381
Exato − fórmula	3.200,000	0,045	0,082	0,545
Exato − fórmula	3.400,000	0,026	0,044	0,599
Exato − fórmula	3.600,000	0,011	0,016	0,657
Livro aditivo − exato	3.000,000	0,130	0,002	58,653
Livro aditivo − exato	3.200,000	0,032	0,002	19,135
Livro aditivo − exato	3.400,000	-0,045	0,001	-40,860
Livro aditivo − exato	3.600,000	-0,041	0,001	-68,273
Livro log − exato	3.000,000	0,234	0,002	106,062
Livro log − exato	3.200,000	0,132	0,002	82,714
Livro log − exato	3.400,000	0,045	0,001	40,932
Livro log − exato	3.600,000	0,009	0,001	15,605

```
OK — o esquema exato não tem viés detectável
OK — com as mesmas trajetórias, o viés do esquema do livro aparece com clareza
```

Leitura. O viés do esquema do livro é pequeno, mas real, positivo e decrescente com o strike, como as diferenças que ele relata. Contra a fórmula, sozinho, ele mal se distingue do ruído de 200 mil trajetórias; com variáveis comuns, fica evidente. O restante das diferenças do livro é ruído de uma única semente, com o mesmo sinal em todos os strikes porque as mesmas trajetórias servem a todos eles.

---

6. Usando o modelo: um exótico e o risco de modelo

O objetivo de calibrar é precificar o que o mercado não cota. Tomamos uma **down-and-in put** de 171 dias: uma put com strike no dinheiro que só passa a existir se o índice tocar 85% do valor atual, com monitoramento diário.

Comparamos quatro precificações:

1. **Verdade:** o modelo que gerou o mercado.
2. **Merton calibrado aos três prazos.**
3. **Merton calibrado a 17 dias.**
4. **BSM** com a volatilidade implícita ATM de 171 dias.

Laboratório 6.1 — Quanto custa errar o modelo

```
T_x, H = 171 / 365, 0.85 * S0
M_x, I_x = 117, 100_000


def dip(S):
    tocou = S.min(axis=0) <= H
    return np.exp(-R * T_x) * np.maximum(S0 - S[-1], 0.0) * tocou


def caminhos_merton(rng, sig, lam, mu, delta):
    dt = T_x / M_x
    y = rng.poisson(lam * dt, (M_x, I_x))
```

```
    x = (R - r_j(lam, mu, delta) - 0.5 * sig**2) * dt + sig * np.sqrt(dt) *
↪   rng.standard_normal((M_x, I_x)) + y * mu + np.sqrt(y) * delta * rng.standard_normal((M_x,
↪   I_x))
    return S0 * np.exp(np.vstack([np.zeros(I_x), np.cumsum(x, axis=0)]))


def caminhos_bates(rng, v0, kappa, theta, nu, rho, lam, mu, delta):
    dt = T_x / M_x
    v = np.full(I_x, v0)
    x = np.zeros(I_x)
    S = [np.full(I_x, S0)]
    comp = lam * (np.exp(mu + 0.5 * delta**2) - 1)
    for _ in range(M_x):
        z1 = rng.standard_normal(I_x)
        z2 = rho * z1 + np.sqrt(1 - rho**2) * rng.standard_normal(I_x)
        vp = np.maximum(v, 0.0)
        y = rng.poisson(lam * dt, I_x)
        x += (R - comp - 0.5 * vp) * dt + np.sqrt(vp * dt) * z1 + y * mu + np.sqrt(y) * delta *
↪   rng.standard_normal(I_x)
        v += kappa * (theta - vp) * dt + nu * np.sqrt(vp * dt) * z2
        S.append(S0 * np.exp(x))
    return np.array(S)


g171 = mercado[mercado["Prazo"] == "171 dias"]
vol_atm = float(np.interp(S0, g171["Strike"], g171["Vol. implícita"]))
rng = np.random.default_rng(1004)
modelos = {
    "Verdade (vol. estoc. + saltos)": caminhos_bates(rng, **VERDADE),
    "Merton, três prazos": caminhos_merton(rng, *x_todos),
    "Merton, 17 dias": caminhos_merton(rng, *x_curto),
    "BSM com vol. ATM": caminhos_merton(rng, vol_atm, 0.0, 0.0, 0.01),
}
linhas = []
for nome, S in modelos.items():
    a = dip(S)
    eur = np.exp(-R * T_x) * np.maximum(S0 - S[-1], 0.0)
    linhas.append({"Modelo": nome, "Put europeia ATM": float(eur.mean()), "Down-and-in put":
↪   float(a.mean()), "Erro-padrão": float(a.std(ddof=1) / np.sqrt(I_x)), "P(tocar a barreira)":
↪   float((S.min(axis=0) <= H).mean())})
ex = pd.DataFrame(linhas).set_index("Modelo")
tabela(ex.reset_index())

v = ex["Down-and-in put"]
ok(abs(ex["Put europeia ATM"].iloc[1] - ex["Put europeia ATM"].iloc[0]) < 0.05 * ex["Put europeia
↪   ATM"].iloc[0], "o Merton calibrado aos três prazos acerta a put europeia ATM com erro menor
↪   que 5%")
ok(abs(v.iloc[3] - v.iloc[0]) > abs(v.iloc[1] - v.iloc[0]), "o BSM erra mais o exótico do que o
↪   Merton calibrado aos três prazos")
ok(v.max() / v.min() > 1.2, "os quatro modelos discordam em mais de 20% no exótico")
```

Modelo	Put europeia ATM	Down-and-in put	Erro-padrão	P(tocar a barreira)
Verdade (vol. estoc. + saltos)	165,424	132,411	0,909	0,247
Merton, três prazos	165,730	119,214	0,872	0,191
Merton, 17 dias	161,353	104,713	0,733	0,200
BSM com vol. ATM	164,636	104,990	0,715	0,210

```
OK — o Merton calibrado aos três prazos acerta a put europeia ATM com erro menor que 5%
OK — o BSM erra mais o exótico do que o Merton calibrado aos três prazos
OK — os quatro modelos discordam em mais de 20% no exótico
```

Leitura. Todos os modelos foram calibrados aos mesmos dados e concordam razoavelmente na put europeia. No exótico, a dispersão é bem maior: o valor depende da probabilidade de tocar a barreira, que é uma propriedade da **dinâmica**, e não da distribuição terminal. Medir essa dispersão é uma forma direta de quantificar o risco de modelo, e é o que motiva o modelo geral dos próximos módulos.

---

7. Síntese

- Merton é BSM mais saltos lognormais compensados.
- A fórmula fechada é uma mistura de preços BSM com pesos de Poisson.
- Três métodos independentes permitem diagnosticar erros numéricos; um deles, sozinho, não.
- Calibrar é mal posto: há vales planos, e o mesmo ajuste é compatível com parâmetros muito diferentes.
- Merton ajusta bem um prazo, mas não vários ao mesmo tempo.
- Somar o salto à difusão, como no livro, introduz viés; o esquema em log é exato.
- Os modelos calibrados concordam nos vanilla e discordam nos exóticos: isso é risco de modelo.

8. Notas sobre o livro neste capítulo

Local	Comentário
§8.2	a lei $\ln(1+J) \sim N(\ln(1+\mu_J) - \delta^2/2, \delta^2)$ não é a usada em r_J, em (8.2) e nos scripts, que correspondem a $\ln(1+J) \sim N(\mu_J, \delta^2)$
§8.5, saída do script	os valores “Int” usam integração até $z = 50$, curta demais para $T = 0{,}22$; são os valores FFT que estão corretos
Equações (8.5) e (8.6)	o salto é somado, e y saltos no mesmo passo usam um único tamanho multiplicado por y; o esquema tem viés
§8.5, saída do script	a diferença entre simulação e fórmula combina esse viés com ruído de uma única semente
Script 8.7.5	comenta “T = 0.22 # shortest maturity”, mas o prazo mais curto é de 17 dias (0,047 ano)

9. Exercícios

Exercício 1. Mostre que, com $\lambda = 0$, a série de Merton se reduz ao BSM.

Solução

Com $\lambda = 0$, $\lambda' = 0$: só o termo $n = 0$ tem peso, e ele vale 1. Como $r_J = 0$, $r_0 = r$ e $\sigma_0 = \sigma$.

Exercício 2. Calcule a variância total por unidade de tempo dos parâmetros do livro e compare com σ.

Solução

$\sigma^2 + \lambda(\mu_J^2 + \delta^2) = 0{,}113^2 + 3{,}559(0{,}075^2 + 0{,}041^2) = 0{,}039$. A volatilidade total é 0,197, bem acima de $\sigma = 0{,}113$: a maior parte da variância vem dos saltos.

Exercício 3. Por que calibrar em volatilidade implícita, em vez de preço, muda o peso relativo das opções fora do dinheiro?

Solução

Um erro de preço ΔC equivale a um erro de volatilidade $\Delta C/\text{vega}$. Como o vega é pequeno longe do dinheiro, o mesmo erro em preço vira um erro grande em volatilidade, e essas opções passam a pesar mais. O Módulo 13 discute as alternativas.

Exercício 4. Recalibre o Merton aos três prazos, mas só com os strikes acima do spot. O que acontece com μ_J? Por quê?

Solução

As calls fora do dinheiro trazem pouca informação sobre a cauda esquerda, onde atuam os saltos negativos. μ_J fica mal determinado, e pode até mudar de sinal. A escolha dos dados de calibração é parte do modelo.

```
print(f"Módulo 10 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 10 concluído: 14 verificações aprovadas.
```

Capítulo 11

O modelo BCC97: volatilidade estocástica, saltos e juros estocásticos

Introdução

O modelo de **Bakshi, Cao e Chen (1997)**, o BCC97, é o modelo geral deste livro. Ele reúne três mecanismos, cada um ligado a um stylized fact do Capítulo 2: **volatilidade estocástica** à maneira de Heston, que produz agrupamento de volatilidade e, com correlação negativa entre preço e variância, o skew; **saltos** à maneira de Merton, que explicam o skew acentuado dos prazos curtos; e **juros estocásticos** à maneira de Cox, Ingersoll e Ross, relevantes para opções longas.

Apesar da riqueza, o modelo continua tratável. Os três blocos são processos **afins**, e para essa classe a função característica do log-preço tem forma fechada. Assim, as opções europeias podem ser precificadas pelos métodos de Fourier dos Capítulos 7 e 8.

O capítulo apresenta o modelo bloco a bloco, a dinâmica de cada fator e o seu efeito sobre o smile, deduz o preço de títulos no modelo CIR e monta a função característica completa.

Fundamentos teóricos

O modelo

Definição 11.1 (Modelo de Bakshi, Cao e Chen). Sob $\mathbb{Q}$, o índice S, sua variância instantânea v e a short rate r seguem

$$\frac{dS_t}{S_{t-}} = (r_t - r_J)\,dt + \sqrt{v_t}\,dZ_t^1 + J_t\,dN_t, \tag{11.1}$$

$$dv_t = \kappa_v(\theta_v - v_t)\,dt + \sigma_v\sqrt{v_t}\,dZ_t^2, \tag{11.2}$$

$$dr_t = \kappa_r(\theta_r - r_t)\,dt + \sigma_r\sqrt{r_t}\,dZ_t^3, \tag{11.3}$$

com $d\langle Z^1, Z^2\rangle_t = \rho\,dt$, Z^3 independente de (Z^1, Z^2), e saltos como no modelo de Merton (Bakshi et al., 1997).

O modelo reúne três blocos: volatilidade estocástica de Heston (agrupamento e efeito alavancagem, via $\rho < 0$), saltos de Merton (assimetria de curto prazo) e juros estocásticos de Cox, Ingersoll e Ross (Cox et al., 1985).

O processo de raiz quadrada

Proposição 11.2 (Propriedades do processo CIR). *Para $dx_t = \kappa(\theta - x_t)\,dt + \sigma\sqrt{x_t}\,dW_t$, $x_0 > 0$, com $\kappa, \theta, \sigma > 0$:*

1. *$x_t \geq 0$ para todo t, e $x_t > 0$ para todo t se e somente se $2\kappa\theta \geq \sigma^2$ (condição de Feller) (Feller, 1951);*
2. *$\mathbb{E}[x_t] = \theta + (x_0 - \theta)e^{-\kappa t}$ e $\operatorname{Var}(x_t) = x_0 \frac{\sigma^2}{\kappa}(e^{-\kappa t} - e^{-2\kappa t}) + \theta\frac{\sigma^2}{2\kappa}(1 - e^{-\kappa t})^2$;*
3. *condicionalmente a x_s, x_t/c tem distribuição qui-quadrado não central com $d = 4\kappa\theta/\sigma^2$ graus de liberdade e parâmetro de não centralidade $x_s e^{-\kappa(t-s)}/c$, onde $c = \sigma^2(1 - e^{-\kappa(t-s)})/(4\kappa)$.*

Proposição 11.3 (Preço do título no CIR). *O título de cupom zero de vencimento T vale $B(t,T) = \mathbb{E}^{\mathbb{Q}}\big[e^{-\int_t^T r_s\,ds} \mid r_t\big] = A(\tau)e^{-B(\tau)r_t}$, $\tau = T - t$, com $\gamma = \sqrt{\kappa_r^2 + 2\sigma_r^2}$ e*

$$A(\tau) = \left[\frac{2\gamma e^{(\kappa_r+\gamma)\tau/2}}{(\gamma+\kappa_r)(e^{\gamma\tau}-1)+2\gamma}\right]^{2\kappa_r\theta_r/\sigma_r^2}, \qquad B(\tau) = \frac{2(e^{\gamma\tau}-1)}{(\gamma+\kappa_r)(e^{\gamma\tau}-1)+2\gamma}. \tag{11.4}$$

Esboço. Pelo argumento de Feynman-Kac, $B(t,T) = F(t,r)$ resolve $\partial_t F + \kappa_r(\theta_r - r)\partial_r F + \frac{1}{2}\sigma_r^2 r\,\partial_{rr}F - rF = 0$, $F(T,r) = 1$. A tentativa $F = A(\tau)e^{-B(\tau)r}$ separa a EDP em duas EDOs: uma de Riccati para B, $B' = 1 - \kappa_r B - \frac{1}{2}\sigma_r^2 B^2$, e uma linear para $\ln A$, cujas soluções são as expressões acima. ∎

Estrutura afim e função característica

Teorema 11.4 (Modelos afins). *Se o vetor de estado X tem deriva, matriz de covariância instantânea e intensidade de saltos afins em X, então*

$$\mathbb{E}^{\mathbb{Q}}\big[e^{-\int_t^T r_s\,ds}\,e^{iu\cdot X_T} \,\big|\, X_t\big] = \exp\big(\alpha(t) + \beta(t)\cdot X_t\big), \tag{11.5}$$

com α e β soluções de EDOs de Riccati generalizadas (Duffie et al., 2000).

Corolário 11.5 (Função característica no BCC97). *Com juros independentes do par (S, v) e a aproximação usada no livro, que trata a taxa de desconto até T como a taxa constante equivalente do título CIR, $\bar r_T = -\ln B(0,T)/T$ (ignorando a volatilidade do título na dinâmica do forward), a função característica do log-retorno fatora-se em*

$$\varphi^{\mathrm{BCC}}(u) = \varphi^{\mathrm{H}}(u)\cdot\varphi^{\mathrm{J}}(u), \tag{11.6}$$

produto das contribuições de Heston e dos saltos, pois os dois blocos são independentes. A parte dos saltos é a de (7.1) sem o termo difusivo. A parte de Heston é $\varphi^{\mathrm{H}}(u) = \exp\big(H_1(u,T) + H_2(u,T)v_0\big)$, com

$$c(u) = \sqrt{(\rho\sigma_v ui - \kappa_v)^2 - \sigma_v^2(-ui - u^2)}, \qquad g(u) = \frac{\kappa_v - \rho\sigma_v ui - c}{\kappa_v - \rho\sigma_v ui + c}, \tag{11.7}$$

$$H_2 = \frac{\kappa_v - \rho\sigma_v ui - c}{\sigma_v^2}\,\frac{1 - e^{-cT}}{1 - ge^{-cT}}, \qquad H_1 = \bar r_T\,uiT + \frac{\kappa_v\theta_v}{\sigma_v^2}\Big[(\kappa_v - \rho\sigma_v ui - c)T - 2\ln\frac{1 - ge^{-cT}}{1-g}\Big], \tag{11.8}$$

nesta forma (com a raiz c de parte real positiva), que evita a descontinuidade do logaritmo complexo.

Com a função característica em mãos, a fórmula de Lewis do Capítulo 7 precifica opções europeias no BCC97 com uma integral unidimensional, o que torna a calibração viável.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulo 9 do livro.

O que você vai conseguir fazer ao final

1. Escrever o modelo de Bakshi, Cao e Chen (1997) e reconhecer seus casos particulares.
2. Deduzir o preço do título zero-cupom no CIR, resolvendo uma equação de Riccati.
3. Deduzir a função característica de Heston pelo mesmo caminho.
4. Evitar a armadilha numérica da fórmula original de Heston.
5. Deduzir a EDP do modelo e conferi-la numericamente.
6. Medir o que a aproximação do livro para os juros estocásticos deixa de fora.
7. Saber o que cada parâmetro faz com a superfície de volatilidade.

Onde estamos

O Módulo 2 mostrou, em mundos sintéticos, que cada stylized fact pede um mecanismo: volatilidade estocástica para o agrupamento e a curvatura, correlação negativa para o skew, saltos para o skew dos prazos curtos, e juros com reversão à média. O Módulo 10 mostrou que Merton sozinho não ajusta vários prazos. O BCC97 junta tudo.

A ferramenta que dá unidade ao módulo é a **estrutura afim**: nos três componentes, a função característica tem a forma $\exp(A + B \cdot \text{estado})$, e encontrar A e B se reduz a resolver equações diferenciais ordinárias.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import simpson, solve_ivp
from scipy.optimize import brentq
from scipy.stats import norm
from IPython.display import display

RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

def bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
```

```
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - sig * np.sqrt(T))

def vol_impl(preco, S, K, T, r):
    piso = max(S - K * np.exp(-r * T), 0.0)
    if not (piso + 1e-12 < preco < S):
        return np.nan
    return brentq(lambda s: float(bsm(S, K, T, r, s)) - preco, 1e-4, 4.0)

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. O modelo

Livro: §9.2 e §9.3

Sob Q, as três equações são:

$$\frac{dS_t}{S_{t-}} = (r_t - r_J)\,dt + \sqrt{v_t}\,dZ_t^1 + (e^J - 1)\,dN_t \qquad \text{(índice: Bates, 1996)} \tag{11.9}$$

$$dv_t = \kappa_v(\theta_v - v_t)\,dt + \sigma_v\sqrt{v_t}\,dZ_t^2 \qquad \text{(variância: Heston, 1993)} \tag{11.10}$$

$$dr_t = \kappa_r(\theta_r - r_t)\,dt + \sigma_r\sqrt{r_t}\,dZ_t^3 \qquad \text{(juros: CIR, 1985)} \tag{11.11}$$

com $dZ^1 dZ^2 = \rho\,dt$, e o terceiro browniano e os saltos independentes do resto. Os saltos são como no Módulo 10, com $J \sim N(\mu_J, \delta^2)$ e $r_J = \lambda(e^{\mu_J + \delta^2/2} - 1)$.

> **Errata repetida do capítulo 8.** A lei $\ln(1+J) \sim N(\ln(1+\mu_J) - \delta^2/2,\ \delta^2)$ escrita na §9.2 não é a que corresponde a r_J e à função característica (9.11). A lei correta é $J \sim N(\mu_J, \delta^2)$, para o log-salto.

1.1 Casos particulares

Modelo	Liga	Desliga
BSM	—	$\sigma_v = 0$, $\lambda = 0$, juros constantes
Merton (M76)	saltos	$\sigma_v = 0$
Heston (H93)	volatilidade estocástica	$\lambda = 0$
Bates (B96)	volatilidade estocástica e saltos	juros estocásticos
CIR (CIR85)	juros	—

São 3 parâmetros de juros e 9 de ações: $v_0, \kappa_v, \theta_v, \sigma_v, \rho, \lambda, \mu_J, \delta$, além de S_0 observado.

1.2 Um papel para cada componente

Os próprios autores resumem seus resultados empíricos assim (§9.3): a volatilidade estocástica é a melhoria de primeira ordem sobre o BSM; os saltos ajudam nos prazos curtos; os juros estocásticos, nos longos. Para hedge, a volatilidade estocástica sozinha já foi o melhor resultado.

---

2. Os juros: o título zero-cupom no CIR

Livro: §9.4

2.1 A estrutura afim, passo a passo

O preço de um título que paga 1 em T é $P(t,T) = \mathbb{E}_t^Q\big[e^{-\int_t^T r_s\,ds}\big]$. Pelo argumento de Feynman-Kac do Módulo 5, com $\tau = T - t$, esse valor satisfaz

$$-P_\tau + \kappa_r(\theta_r - r)P_r + \tfrac{1}{2}\sigma_r^2\, r\, P_{rr} - rP = 0, \qquad P(r,0) = 1. \tag{11.12}$$

Passo 1: o palpite afim. Tentamos $P = e^{a(\tau) - b(\tau)\,r}$. Então $P_\tau = (a' - b'r)P$, $P_r = -bP$ e $P_{rr} = b^2 P$.

Passo 2: substituir.

$$-(a' - b'r) - \kappa_r(\theta_r - r)b + \tfrac{1}{2}\sigma_r^2 r b^2 - r = 0. \tag{11.13}$$

Passo 3: separar. A equação vale para todo r, então o termo constante e o coeficiente de r se anulam separadamente:

$$b' = 1 - \kappa_r b - \tfrac{1}{2}\sigma_r^2 b^2, \qquad a' = -\kappa_r\theta_r\, b, \qquad a(0) = b(0) = 0. \tag{11.14}$$

Uma equação de derivada parcial virou duas equações **ordinárias**. A de b é uma equação de **Riccati**: derivada igual a um polinômio de segundo grau.

Passo 4: resolver. As raízes do lado direito são $b_\pm = (-\kappa_r \pm \gamma)/\sigma_r^2$, com $\gamma = \sqrt{\kappa_r^2 + 2\sigma_r^2}$. A integração por frações parciais leva às fórmulas (9.4) a (9.7) do livro:

$$b(\tau) = \frac{2(e^{\gamma\tau} - 1)}{2\gamma + (\kappa_r + \gamma)(e^{\gamma\tau} - 1)}, \qquad e^{a(\tau)} = \left[\frac{2\gamma\, e^{(\kappa_r + \gamma)\tau/2}}{2\gamma + (\kappa_r + \gamma)(e^{\gamma\tau} - 1)}\right]^{2\kappa_r\theta_r/\sigma_r^2}. \tag{11.15}$$

Laboratório 2.1 — Três caminhos para o mesmo número

Fórmula fechada, integração numérica das equações de Riccati e Monte Carlo pela transição exata (Módulo 2).

```
CIR = dict(r0=0.02, kappa=0.8, theta=0.035, sigma=0.12)


def zero_cir(tau, r0, kappa, theta, sigma):
    g = np.sqrt(kappa**2 + 2 * sigma**2)
    den = 2 * g + (kappa + g) * (np.exp(g * tau) - 1)
    b = 2 * (np.exp(g * tau) - 1) / den
    ea = (2 * g * np.exp((kappa + g) * tau / 2) / den) ** (2 * kappa * theta / sigma**2)
```

```
    return ea * np.exp(-b * r0)


def zero_riccati(tau, r0, kappa, theta, sigma):
    def eqs(t, y):
        a, b = y
        return [-kappa * theta * b, 1 - kappa * b - 0.5 * sigma**2 * b**2]
    sol = solve_ivp(eqs, (0, tau), [0.0, 0.0], rtol=1e-11, atol=1e-13)
    a, b = sol.y[:, -1]
    return np.exp(a - b * r0)


def simular_cir(r0, kappa, theta, sigma, T, M, I, rng):
    dt = T / M
    c = sigma**2 * (1 - np.exp(-kappa * dt)) / (4 * kappa)
    gl = 4 * kappa * theta / sigma**2
    r = np.full(I, r0)
    integ = np.zeros(I)
    for _ in range(M):
        novo = c * rng.noncentral_chisquare(gl, r * np.exp(-kappa * dt) / c, I)
        integ += 0.5 * (r + novo) * dt
        r = novo
    return integ


rng = np.random.default_rng(1101)
linhas = []
for tau in [0.5, 1.0, 2.0, 5.0]:
    integ = simular_cir(**CIR, T=tau, M=int(100 * tau), I=100_000, rng=rng)
    fd = np.exp(-integ)
    linhas.append({"Prazo": tau, "Fórmula": float(zero_cir(tau, **CIR)), "Riccati numérica":
↪   float(zero_riccati(tau, **CIR)), "Monte Carlo": fd.mean(), "Erro-padrão": fd.std(ddof=1) /
↪   np.sqrt(fd.size), "Taxa zero (%)": float(-np.log(zero_cir(tau, **CIR)) / tau * 100)})
zc = pd.DataFrame(linhas)
tabela(zc)
print("Condição de Feller 2κθ − σ²:", f3(2 * CIR["kappa"] * CIR["theta"] - CIR["sigma"] ** 2))
ok(np.allclose(zc["Fórmula"], zc["Riccati numérica"], atol=1e-10), "a fórmula fechada resolve as
↪   equações de Riccati")
ok(((zc["Fórmula"] - zc["Monte Carlo"]).abs() < 4 * zc["Erro-padrão"] + 2e-4).all(), "Monte Carlo
↪   confere com a fórmula")
```

Prazo	Fórmula	Riccati numérica	Monte Carlo	Erro-padrão	Taxa zero (%)
0,500	0,989	0,989	0,989	0,000	2,263
1,000	0,976	0,976	0,976	0,000	2,464
2,000	0,947	0,947	0,947	0,000	2,743
5,000	0,856	0,856	0,856	0,000	3,111

```
Condição de Feller 2κθ − σ²: 0,042
OK — a fórmula fechada resolve as equações de Riccati
OK — Monte Carlo confere com a fórmula
```

---

3. A volatilidade: a função característica de Heston

Livro: §9.5.2, equação (9.10)

Fixe, por ora, juros constantes. Seja $x_t = \ln S_t - rt$, o log do preço forward. Queremos $\phi(u) = \mathbb{E}\big[e^{iu(x_T - x_0)}\big]$.

3.1 Dedução pela mesma receita

Pelo lema de Itô, $dx = -\tfrac{1}{2}v\,dt + \sqrt{v}\,dZ^1$. A função $f(x, v, \tau) = \mathbb{E}[e^{iux_T} \mid x, v]$ satisfaz a EDP de Feynman-Kac:

$$f_\tau = \tfrac{1}{2}v\,f_{xx} - \tfrac{1}{2}v\,f_x + \rho\sigma_v v\,f_{xv} + \tfrac{1}{2}\sigma_v^2 v\,f_{vv} + \kappa_v(\theta_v - v)f_v. \tag{11.16}$$

Passo 1: palpite afim. $f = \exp\big(iux + A(\tau) + B(\tau)v\big)$. Então $f_x = iuf$, $f_{xx} = -u^2 f$, $f_v = Bf$, $f_{vv} = B^2 f$, $f_{xv} = iuBf$.

Passo 2: substituir e separar em termo constante e coeficiente de v:

$$A' = \kappa_v\theta_v\,B, \qquad B' = \tfrac{1}{2}\sigma_v^2 B^2 - (\kappa_v - \rho\sigma_v iu)B - \tfrac{1}{2}(u^2 + iu). \tag{11.17}$$

Outra Riccati, agora com coeficientes complexos.

Passo 3: resolver. Com $\xi = \kappa_v - \rho\sigma_v iu$ e $d = \sqrt{\xi^2 + \sigma_v^2(u^2 + iu)}$, as raízes são $(\xi \pm d)/\sigma_v^2$. Definindo $g = (\xi - d)/(\xi + d)$:

$$B(\tau) = \frac{\xi - d}{\sigma_v^2}\cdot\frac{1 - e^{-d\tau}}{1 - g\,e^{-d\tau}}, \qquad A(\tau) = \frac{\kappa_v\theta_v}{\sigma_v^2}\Big[(\xi - d)\tau - 2\ln\frac{1 - g\,e^{-d\tau}}{1 - g}\Big]. \tag{11.18}$$

E $\phi^{H93}(u) = e^{A + Bv_0}$.

3.2 A armadilha

Heston (1993) escreveu a solução com a **outra raiz**: $g' = 1/g$ e $e^{+d\tau}$. Matematicamente é a mesma função. Numericamente, não: o logaritmo complexo $\ln\frac{1 - g'e^{d\tau}}{1 - g'}$ cruza o corte do ramo principal à medida que u cresce, e o computador escolhe o ramo errado. O integrando fica descontínuo e o preço sai errado, sem nenhum aviso. Albrecher et al. (2007) chamaram isso de *little Heston trap*.

O livro usa a forma estável: seu c_2 é $-d$, de modo que $c_3 = g$ e $e^{c_2 T} = e^{-dT}$. Vale verificar, porque muitos códigos copiados da literatura usam a forma original.

Laboratório 3.1 — Riccati numérica e as duas fórmulas

```python
def cf_heston(u, T, v0, kappa, theta, sigma, rho):
    """Forma estável (Albrecher et al.): a usada pelo livro."""
    xi = kappa - rho * sigma * 1j * u
    d = np.sqrt(xi**2 + sigma**2 * (1j * u + u**2))
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
    A = kappa * theta / sigma**2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / sigma**2 * (1 - e) / (1 - g * e)
    return np.exp(A + B * v0)


def cf_heston_original(u, T, v0, kappa, theta, sigma, rho):
    """Forma de Heston (1993), com a outra raiz."""
    xi = kappa - rho * sigma * 1j * u
    d = np.sqrt(xi**2 + sigma**2 * (1j * u + u**2))
    g = (xi + d) / (xi - d)
```

```
    with np.errstate(all="ignore"):
        e = np.exp(d * T)
        A = kappa * theta / sigma**2 * ((xi + d) * T - 2 * np.log((1 - g * e) / (1 - g)))
        B = (xi + d) / sigma**2 * (1 - e) / (1 - g * e)
        return np.exp(A + B * v0)


def cf_heston_ode(u, T, v0, kappa, theta, sigma, rho):
    xi = kappa - rho * sigma * 1j * u

    def eqs(t, y):
        A, B = y[0] + 1j * y[1], y[2] + 1j * y[3]
        dA = kappa * theta * B
        dB = 0.5 * sigma**2 * B**2 - xi * B - 0.5 * (u**2 + 1j * u)
        return [dA.real, dA.imag, dB.real, dB.imag]

    s = solve_ivp(eqs, (0, T), [0, 0, 0, 0], rtol=1e-10, atol=1e-12, method="DOP853").y[:, -1]
    return np.exp(s[0] + 1j * s[1] + (s[2] + 1j * s[3]) * v0)


H = dict(v0=0.04, kappa=1.0, theta=0.04, sigma=0.9, rho=-0.7)
us = [0.5, 2.0, 7.0, 15.0]
dif = max(abs(cf_heston(u, 2.0, **H) - cf_heston_ode(u, 2.0, **H)) for u in us)
ok(dif < 1e-8, "a fórmula estável resolve as equações de Riccati")
ok(abs(cf_heston(-1j, 3.0, **H) - 1) < 1e-12, "martingale: φ(−i) = 1")

Z = np.linspace(0.0, 200.0, 20001)


def lewis(S, K, T, r, cf):
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(S / K) + r * T
    h = np.exp(1j * Z * k[:, None]) * cf(Z - 0.5j)
    return S - np.sqrt(S * K) * np.exp(-0.5 * r * T) / np.pi * simpson(h.real / (Z**2 + 0.25),
    ↪  x=Z, axis=1)


S0, r = 100.0, 0.02
linhas = []
for T in [1.0, 2.0, 5.0, 10.0]:
    a = float(lewis(S0, 100.0, T, r, lambda u: cf_heston(u, T, **H))[0])
    b = float(lewis(S0, 100.0, T, r, lambda u: cf_heston_original(u, T, **H))[0])
    linhas.append({"Prazo": T, "Forma estável": a, "Forma original": b})
arm = pd.DataFrame(linhas)
tabela(arm)

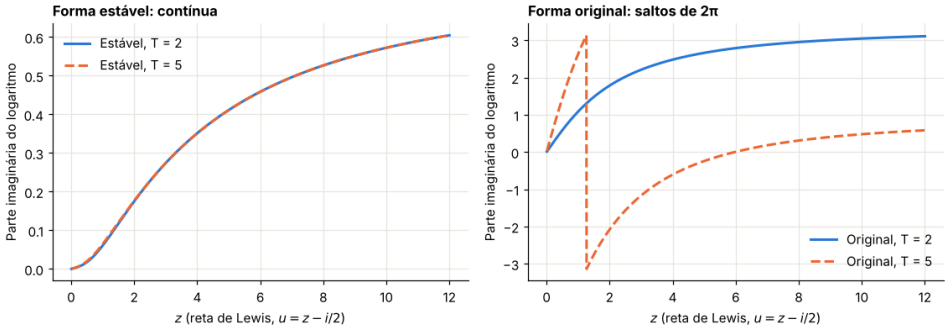
ur = np.linspace(0.01, 12, 2000)
xi = H["kappa"] - H["rho"] * H["sigma"] * 1j * (ur - 0.5j)
d = np.sqrt(xi**2 + H["sigma"] ** 2 * (1j * (ur - 0.5j) + (ur - 0.5j) ** 2))
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10.5, 3.8))
for T, cor, est in [(2.0, AZUL, "-"), (5.0, LARANJA, "--")]:
    g1, g2 = (xi - d) / (xi + d), (xi + d) / (xi - d)
    with np.errstate(all="ignore"):
        a1.plot(ur, np.imag(np.log((1 - g1 * np.exp(-d * T)) / (1 - g1))), color=cor, ls=est,
↪   label=f"Estável, T = {T:.0f}")
        a2.plot(ur, np.imag(np.log((1 - g2 * np.exp(d * T)) / (1 - g2))), color=cor, ls=est,
↪   label=f"Original, T = {T:.0f}")
for ax, t in [(a1, "Forma estável: contínua"), (a2, "Forma original: saltos de 2π")]:
    ax.set_title(t)
    ax.set_xlabel("$z$ (reta de Lewis, $u = z - i/2$)")
```

```
    ax.set_ylabel("Parte imaginária do logaritmo")
    ax.legend()
plt.tight_layout()
plt.show()

ok(abs(arm["Forma estável"].iloc[0] - arm["Forma original"].iloc[0]) < 1e-8, "em prazo curto, as
↪  duas formas coincidem")
ok(not np.isclose(arm["Forma estável"].iloc[2], arm["Forma original"].iloc[2], atol=0.01), "em
↪  prazo longo, a forma original dá um preço errado")
```

```
OK — a fórmula estável resolve as equações de Riccati
OK — martingale: φ(−i) = 1
```

Prazo	Forma estável	Forma original
1,000	6,924	6,924
2,000	10,335	10,422
5,000	18,727	24,244
10,000	29,626	n/d




```
OK — em prazo curto, as duas formas coincidem
OK — em prazo longo, a forma original dá um preço errado
```

Laboratório 3.2 — Conferência por simulação

Uma última conferência, independente das deduções: Monte Carlo com esquema de Euler e truncamento total, em passos finos (o Módulo 12 discute alternativas melhores).

```
def heston_mc(S0, r, T, v0, kappa, theta, sigma, rho, M, I, rng):
    dt = T / M
    v = np.full(I, v0)
    x = np.zeros(I)
    for _ in range(M):
        z1 = rng.standard_normal(I)
        z2 = rho * z1 + np.sqrt(1 - rho**2) * rng.standard_normal(I)
        vp = np.maximum(v, 0.0)
        x += (r - 0.5 * vp) * dt + np.sqrt(vp * dt) * z1
        v += kappa * (theta - vp) * dt + sigma * np.sqrt(vp * dt) * z2
    return S0 * np.exp(x)


rng = np.random.default_rng(1102)
```

```
ST = heston_mc(S0, r, 1.0, **H, M=500, I=200_000, rng=rng)
linhas = []
for K in [80.0, 100.0, 120.0]:
    a = np.exp(-r) * np.maximum(ST - K, 0.0)
    ref = float(lewis(S0, K, 1.0, r, lambda u: cf_heston(u, 1.0, **H))[0])
    linhas.append({"Strike": K, "Lewis": ref, "Monte Carlo": a.mean(), "Erro-padrão":
↪  a.std(ddof=1) / np.sqrt(a.size)})
mc = pd.DataFrame(linhas)
tabela(mc)
ok(((mc["Lewis"] - mc["Monte Carlo"]).abs() < 4 * mc["Erro-padrão"] + 0.02).all(), "Monte Carlo
↪  confere com a fórmula de Heston")
```

Strike	Lewis	Monte Carlo	Erro-padrão
80,000	23,286	23,324	0,027
100,000	6,924	6,949	0,018
120,000	0,518	0,521	0,008

```
OK — Monte Carlo confere com a fórmula de Heston
```

4. Juntando: a função característica do BCC97

Livro: §9.5.2, equações (9.11) a (9.13)

Como os saltos são independentes da parte de Heston, a função característica do log-retorno é o produto (Módulo 7):

$$\phi^{BCC97}(u) = \phi^{H93}(u)\cdot\phi^{M76J}(u), \qquad \phi^{M76J}(u) = \exp\left\{T\left[-iu\,r_J+\lambda(e^{iu\mu_J-u^2\delta^2/2}-1)\right]\right\}. \tag{11.19}$$

Os juros entram, no livro, por um atalho: a taxa constante é substituída pela taxa zero do CIR para o prazo da opção, $\bar{r}_T = -\ln P(0,T)/T$. A Seção 6 examina esse atalho.

Laboratório 4.1 — Os casos particulares

```
def cf_bcc(u, T, v0, kappa, theta, sigma, rho, lam, mu, delta):
    rj = lam * (np.exp(mu + 0.5 * delta**2) - 1)
    saltos = np.exp(T * (-1j * u * rj + lam * (np.exp(1j * u * mu - 0.5 * u**2 * delta**2) - 1)))
    return cf_heston(u, T, v0, kappa, theta, sigma, rho) * saltos


def merton_serie(S, K, T, r, sig, lam, mu, delta, n_max=60):
    from scipy.stats import poisson
    kap = np.exp(mu + 0.5 * delta**2) - 1
    tot = 0.0
    for n in range(n_max):
        sn = np.sqrt(sig**2 + n * delta**2 / T)
        rn = r - lam * kap + n * np.log1p(kap) / T
        tot = tot + poisson.pmf(n, lam * (1 + kap) * T) * bsm(S, K, T, rn, sn)
    return tot
```

```
K = np.array([85.0, 100.0, 115.0])
T = 0.75
eps = 1e-5
bsm_via = lewis(S0, K, T, r, lambda u: cf_bcc(u, T, 0.04, 1.0, 0.04, eps, 0.0, 0.0, 0.0, 0.1))
m76_via = lewis(S0, K, T, r, lambda u: cf_bcc(u, T, 0.03, 1.0, 0.03, eps, 0.0, 0.8, -0.1, 0.08))
h93_via = lewis(S0, K, T, r, lambda u: cf_bcc(u, T, **H, lam=0.0, mu=0.0, delta=0.1))
tabela(pd.DataFrame({"Strike": K, "BCC97 → BSM": bsm_via, "BSM": bsm(S0, K, T, r, 0.2), "BCC97 →
↪  M76": m76_via, "Série de Merton": merton_serie(S0, K, T, r, np.sqrt(0.03), 0.8, -0.1, 0.08),
↪  "BCC97 → H93": h93_via, "Heston": lewis(S0, K, T, r, lambda u: cf_heston(u, T, **H))}))
ok(np.max(np.abs(bsm_via - bsm(S0, K, T, r, 0.2))) < 1e-4, "com σ_v → 0 e sem saltos, o BCC97
↪  vira BSM")
ok(np.max(np.abs(m76_via - merton_serie(S0, K, T, r, np.sqrt(0.03), 0.8, -0.1, 0.08))) < 1e-4,
↪  "com σ_v → 0, vira Merton")
ok(np.max(np.abs(h93_via - lewis(S0, K, T, r, lambda u: cf_heston(u, T, **H)))) < 1e-12, "sem
↪  saltos, vira Heston")
```

Strike	BCC97 → BSM	BSM	BCC97 → M76	Série de Merton	BCC97 → H93	Heston
85,000	17,522	17,522	17,780	17,780	18,083	18,083
100,000	7,620	7,620	7,744	7,744	5,964	5,964
115,000	2,539	2,539	2,465	2,465	0,618	0,618

```
OK — com σ_v → 0 e sem saltos, o BCC97 vira BSM
OK — com σ_v → 0, vira Merton
OK — sem saltos, vira Heston
```

---

5. A EDP do modelo

Livro: §9.5.1 e §9.7.1

5.1 Itô em várias dimensões, com saltos

Para $f(S, v, r, t)$, a expansão de Taylor de segunda ordem com as regras $(dZ^i)^2 = dt$, $dZ^1 dZ^2 = \rho\, dt$ e demais produtos nulos dá a parte contínua. O salto exige outro tratamento: num salto, S passa de S_- a $S_- e^J$ de uma vez, e a variação de f é a diferença **exata**, e não uma derivada:

$$df = f_t\,dt + f_S\,dS^c + f_v\,dv + f_r\,dr + \tfrac{1}{2}(vS^2 f_{SS} + \sigma_v^2 v f_{vv} + \sigma_r^2 r f_{rr})dt + \rho\sigma_v v S f_{Sv}\,dt + [f(S_- e^J) - f(S_-)]\,dN, \tag{11.20}$$

em que dS^c é a parte contínua de dS.

5.2 A EDP

Sob Q, o preço descontado de qualquer derivativo é martingale, logo o drift de dC é rC. Tomando a esperança dos termos e usando $\mathbb{E}[dN] = \lambda\, dt$:

$$\tfrac{1}{2}vS^2 C_{SS} + (r - r_J)SC_S + \rho\sigma_v v S\, C_{Sv} + \tfrac{1}{2}\sigma_v^2 v\, C_{vv} + \kappa_v(\theta_v - v)C_v \tag{11.21}$$

$$+\ \tfrac{1}{2}\sigma_r^2 r\, C_{rr} + \kappa_r(\theta_r - r)C_r + C_t - rC + \lambda\,\mathbb{E}[C(Se^J) - C(S)] = 0. \tag{11.22}$$

Duas observações:

1. Ao contrário do BSM, essa EDP **não** vem de uma replicação. O mercado é incompleto (Módulo 4), e a equação vale sob a medida Q escolhida. Os parâmetros que aparecem nela são os parâmetros **sob** Q, que é o que a calibração estima.
2. Usamos $\mathbb{E}[dN] = \lambda\,dt$, mas o termo $f_S\,dS$ usa só a parte contínua. Se usássemos $f_S\,dS$ completo, o salto seria contado duas vezes.

Erratas do livro na equação (9.8). (i) O primeiro termo aparece como $\frac{1}{2}vS^2\,\partial^2 C/\partial S$; o denominador correto é ∂S^2. (ii) Faltam os fatores de variância nos termos de difusão de v e r: o correto é $\frac{1}{2}\sigma_v^2\,v\,\partial^2 C/\partial v^2$ e $\frac{1}{2}\sigma_r^2\,r\,\partial^2 C/\partial r^2$, já que os coeficientes de difusão são $\sigma_v\sqrt{v}$ e $\sigma_r\sqrt{r}$, cujos quadrados aparecem na própria prova. (iii) Na prova, o termo de salto entra como $\frac{\partial A}{\partial S}\,jdN$, uma aproximação de primeira ordem, enquanto a EDP usa a diferença exata, que é a forma correta.

Laboratório 5.1 — A fórmula de Fourier satisfaz a EDP?

Com juros constantes, calculamos as derivadas de $C(S, v, \tau)$ por diferenças finitas sobre preços de Lewis, e o termo de salto por quadratura de Gauss-Hermite. O resíduo da EDP correta deve ser próximo de zero. Testamos também a versão impressa, sem o fator v em C_{vv}.

```
B96 = dict(kappa=1.5, theta=0.04, sigma=0.5, rho=-0.7, lam=0.5, mu=-0.10, delta=0.10)


def preco(S, v, tau, K=100.0, r=0.02):
    return float(lewis(S, K, tau, r, lambda u: cf_bcc(u, tau, v, **B96))[0])


def residuos(S, v, tau, r=0.02):
    hS, hv, ht = 0.5, 1e-3, 1e-4
    C = preco(S, v, tau)
    CS = (preco(S + hS, v, tau) - preco(S - hS, v, tau)) / (2 * hS)
    CSS = (preco(S + hS, v, tau) - 2 * C + preco(S - hS, v, tau)) / hS**2
    Cv = (preco(S, v + hv, tau) - preco(S, v - hv, tau)) / (2 * hv)
    Cvv = (preco(S, v + hv, tau) - 2 * C + preco(S, v - hv, tau)) / hv**2
    CSv = (preco(S + hS, v + hv, tau) - preco(S + hS, v - hv, tau) - preco(S - hS, v + hv, tau) +
↪   preco(S - hS, v - hv, tau)) / (4 * hS * hv)
    Ct = -(preco(S, v, tau + ht) - preco(S, v, tau - ht)) / (2 * ht)
    x, w = np.polynomial.hermite_e.hermegauss(40)
    w = w / w.sum()
    Jn = B96["mu"] + B96["delta"] * x
    salto = sum(wi * preco(S * np.exp(ji), v, tau) for wi, ji in zip(w, Jn)) - C
    rj = B96["lam"] * (np.exp(B96["mu"] + 0.5 * B96["delta"] ** 2) - 1)
    comum = 0.5 * v * S**2 * CSS + (r - rj) * S * CS + B96["rho"] * B96["sigma"] * v * S * CSv +
↪   B96["kappa"] * (B96["theta"] - v) * Cv + Ct - r * C + B96["lam"] * salto
    return comum + 0.5 * B96["sigma"] ** 2 * v * Cvv, comum + 0.5 * B96["sigma"] ** 2 * Cvv, C


linhas = []
for S, v, tau in [(100.0, 0.04, 0.5), (90.0, 0.02, 1.0), (115.0, 0.06, 0.25)]:
    rc, ri, C = residuos(S, v, tau)
    linhas.append({"S": S, "v": v, "τ": tau, "Preço": C, "Resíduo: EDP correta": rc, "Resíduo:
↪   sem o fator v em C_vv": ri})
edp = pd.DataFrame(linhas)
tabela(edp)
ok((edp["Resíduo: EDP correta"].abs() < 2e-3).all(), "o preço de Fourier satisfaz a EDP correta")
```

```
ok((edp["Resíduo: sem o fator v em C_vv"].abs() > 0.05).all(), "a forma impressa, sem o fator v,
↪  não é satisfeita")
```

S	v	τ	Preço	Resíduo: EDP correta	Resíduo: sem o fator v em C_vv
100,000	0,040	0,500	6,399	-0,002	-35,060
90,000	0,020	1,000	2,822	-0,001	-16,669
115,000	0,060	0,250	16,830	0,000	0,104

```
OK — o preço de Fourier satisfaz a EDP correta
OK — a forma impressa, sem o fator v, não é satisfeita
```

---

6. Juros estocásticos: o que o atalho do livro deixa de fora

Livro: §9.5.2, definição de $r_{0,T}$ em H_1

6.1 O preço exato, sob independência

Seja $R = \int_0^T r_s\, ds$. Como os juros são independentes do resto, escrevemos $S_T = S_0\, e^R\, X$, em que X depende só da volatilidade e dos saltos. O preço é

$$C_0 = \mathbb{E}[e^{-R}(S_0 e^R X - K)^+] = \mathbb{E}_R\Big[\underbrace{\mathbb{E}_X[e^{-R}(S_0 e^R X - K)^+]}_{\text{preço com taxa constante } R/T}\Big]. \tag{11.23}$$

O preço exato é a **média**, sobre os cenários de juros, dos preços com taxa constante igual a R/T.

6.2 O atalho

O livro, seguindo uma prática comum, usa um único valor: $\bar r_T = -\ln P(0,T)/T$. Pela desigualdade de Jensen, $\mathbb{E}[e^{-R}] = P(0,T)$, ou seja, o atalho acerta o desconto médio. Mas o preço da call não é linear em R: ele é **convexo**, porque juros mais altos aumentam o forward e o valor da call, com curvatura. A diferença é

$$C^{\text{exato}} - C^{\text{atalho}} = \mathbb{E}_R[C(R/T)] - C(\bar r_T), \tag{11.24}$$

pequena quando a variância de R é pequena. O artigo original de Bakshi, Cao e Chen trata isso de forma exata, usando a função característica conjunta com os juros. O atalho é uma simplificação do livro.

Laboratório 6.1 — O tamanho do efeito

Calculamos o preço exato pela média sobre 400 quantis da distribuição de R, simulada pela transição exata do CIR. Variamos o prazo e a volatilidade dos juros.

```
B97 = dict(v0=0.04, kappa=1.5, theta=0.04, sigma=0.5, rho=-0.7, lam=0.5, mu=-0.10, delta=0.10)


def preco_taxa(K, T, rbar):
```

```
    """Preço com taxa constante rbar, vetorizado em rbar."""
    rbar = np.atleast_1d(rbar)
    fi = cf_bcc(Z - 0.5j, T, **B97)
    k = np.log(S0 / K) + rbar[:, None] * T
    integ = simpson((np.exp(1j * Z * k) * fi).real / (Z**2 + 0.25), x=Z, axis=1)
    return S0 - np.sqrt(S0 * K) * np.exp(-0.5 * rbar * T) / np.pi * integ


rng = np.random.default_rng(1103)
linhas = []
for sig_r in [0.05, 0.12, 0.25]:
    cir = dict(r0=0.02, kappa=0.8, theta=0.035, sigma=sig_r)
    for T in [0.5, 2.0, 5.0]:
        R_ = simular_cir(**cir, T=T, M=int(50 * T), I=100_000, rng=rng)
        q = np.quantile(R_, (np.arange(400) + 0.5) / 400)
        rbar = -np.log(zero_cir(T, **cir)) / T
        for K in [100.0]:
            exato = float(np.mean(preco_taxa(K, T, q / T)))
            atalho = float(preco_taxa(K, T, rbar)[0])
            linhas.append({"σ_r": sig_r, "Prazo": T, "Desvio de R/T (p.p.)": float(np.std(R_ / T)
↪  * 100), "Exato": exato, "Atalho do livro": atalho, "Diferença": exato - atalho, "Diferença (%
↪  do preço)": (exato - atalho) / exato * 100})
jur = pd.DataFrame(linhas)
tabela(jur)
ok((jur["Diferença"] > -1e-3).all(), "o atalho nunca supera o preço exato: efeito de convexidade")
ok(jur.query("σ_r == 0.12 and Prazo == 0.5")["Diferença (% do preço)"].iloc[0] < 0.1, "com
↪  volatilidade de juros moderada e prazo curto, o atalho é excelente")
ok(jur["Diferença (% do preço)"].max() > 5 * jur.query("σ_r == 0.12 and Prazo == 0.5")["Diferença
↪  (% do preço)"].iloc[0], "o erro do atalho cresce com o prazo e com a volatilidade dos juros")
```

σ_r	Prazo	Desvio de R/T (p.p.)	Exato	Atalho do livro	Diferença	Diferença (% do preço)
0,050	0,500	0,259	6,477	6,476	0,000	0,007
0,050	2,000	0,388	14,346	14,343	0,003	0,023
0,050	5,000	0,384	25,779	25,770	0,010	0,037
0,120	0,500	0,622	6,478	6,476	0,001	0,021
0,120	2,000	0,926	14,352	14,335	0,017	0,118
0,120	5,000	0,925	25,789	25,726	0,063	0,245
0,250	0,500	1,293	6,480	6,475	0,005	0,075
0,250	2,000	1,933	14,389	14,304	0,085	0,593
0,250	5,000	1,926	25,810	25,556	0,253	0,982

```
OK — o atalho nunca supera o preço exato: efeito de convexidade
OK — com volatilidade de juros moderada e prazo curto, o atalho é excelente
OK — o erro do atalho cresce com o prazo e com a volatilidade dos juros
```

Leitura. Para os prazos das opções listadas de índice, de dias a poucos meses, o atalho é praticamente exato. Para produtos longos ou híbridos, que dependem de juros e ações ao mesmo tempo, é preciso o tratamento completo, que também abriria espaço para correlação entre juros e índice, ausente no BCC97.

---

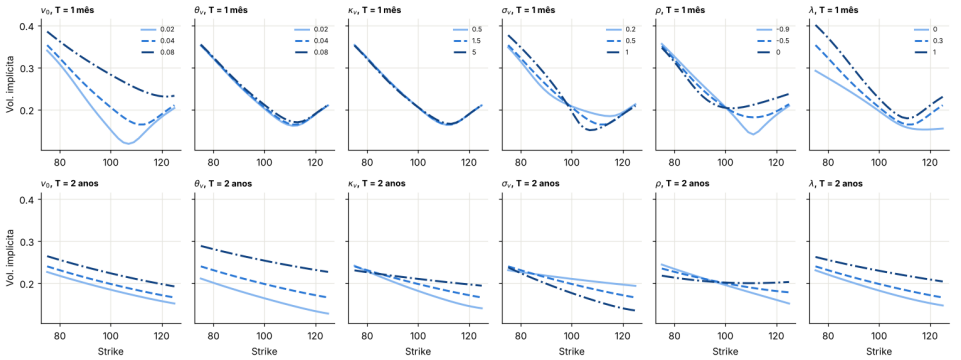
7. O que cada parâmetro faz com a superfície

Antes de calibrar (Módulos 13 e 14), vale ter intuição sobre o papel de cada parâmetro. Variamos um parâmetro de cada vez a partir de um caso-base e olhamos para a volatilidade implícita em dois prazos.

Laboratório 7.1 — Os parâmetros, um a um

```
base = dict(v0=0.04, kappa=1.5, theta=0.04, sigma=0.5, rho=-0.7, lam=0.3, mu=-0.10, delta=0.10)
variacoes = {
    "v0": [0.02, 0.04, 0.08],
    "theta": [0.02, 0.04, 0.08],
    "kappa": [0.5, 1.5, 5.0],
    "sigma": [0.2, 0.5, 1.0],
    "rho": [-0.9, -0.5, 0.0],
    "lam": [0.0, 0.3, 1.0],
}
nomes = {"v0": "$v_0$", "theta": "$\\theta_v$", "kappa": "$\\kappa_v$", "sigma": "$\\sigma_v$",
 ↪  "rho": "$\\rho$", "lam": "$\\lambda$"}
Ks = np.linspace(75, 125, 26)
fig, eixos = plt.subplots(2, 6, figsize=(14, 5.4), sharey=True)
efeito = {}
for j, (par, vals) in enumerate(variacoes.items()):
    for i, T in enumerate([1 / 12, 2.0]):
        ax = eixos[i, j]
        niveis = []
        for v_, cor, est in zip(vals, RAMPA, ["-", "--", "-."]):
            p = {**base, par: v_}
            c = lewis(S0, Ks, T, 0.02, lambda u: cf_bcc(u, T, **p))
            iv = np.array([vol_impl(cc, S0, k, T, 0.02) for cc, k in zip(c, Ks)])
            ax.plot(Ks, iv, color=cor, ls=est, label=f"{v_:g}")
            niveis.append(iv)
        efeito[(par, T)] = (np.nanmean(niveis[-1] - niveis[0]), np.nanmean((niveis[-1][:5] -
 ↪  niveis[-1][-5:]) - (niveis[0][:5] - niveis[0][-5:])))
        ax.set_title(f"{nomes[par]}, T = {'1 mês' if T < 1 else '2 anos'}", fontsize=9)
        if i == 1:
            ax.set_xlabel("Strike")
        if j == 0:
            ax.set_ylabel("Vol. implícita")
        if i == 0:
            ax.legend(fontsize=7)
plt.tight_layout()
plt.show()

ef = pd.DataFrame([{"Parâmetro": p, "Prazo": "1 mês" if T < 1 else "2 anos", "Variação do nível":
 ↪  n, "Variação do skew": s} for (p, T), (n, s) in efeito.items()])
tabela(ef)
e = {(r_["Parâmetro"], r_["Prazo"]): r_ for _, r_ in ef.iterrows()}
ok(abs(e[("v0", "1 mês")]["Variação do nível"]) > 2 * abs(e[("v0", "2 anos")]["Variação do
 ↪  nível"]), "v0 domina o nível dos prazos curtos")
ok(abs(e[("theta", "2 anos")]["Variação do nível"]) > 2 * abs(e[("theta", "1 mês")]["Variação do
 ↪  nível"]), "θ domina o nível dos prazos longos")
ok(e[("rho", "2 anos")]["Variação do skew"] < 0, "ρ mais alto (menos negativo) reduz o skew")
ok(e[("lam", "1 mês")]["Variação do skew"] > e[("lam", "2 anos")]["Variação do skew"], "os saltos
 ↪  pesam mais no skew dos prazos curtos")
```



Parâmetro	Prazo	Variação do nível	Variação do skew
v0	1 mês	0,088	0,007
v0	2 anos	0,039	-0,003
theta	1 mês	0,006	0,001
theta	2 anos	0,089	-0,019
kappa	1 mês	-0,000	-0,002
kappa	2 anos	0,026	-0,056
sigma	1 mês	0,004	0,038
sigma	2 anos	-0,031	0,056
rho	1 mês	0,012	-0,051
rho	2 anos	0,008	-0,065
lam	1 mês	0,054	0,038
lam	2 anos	0,046	-0,022

```
OK — v0 domina o nível dos prazos curtos
OK — θ domina o nível dos prazos longos
OK — ρ mais alto (menos negativo) reduz o skew
OK — os saltos pesam mais no skew dos prazos curtos
```

Resumo para a calibração:

Parâmetro	Efeito principal
v_0	nível dos prazos curtos
θ_v	nível dos prazos longos
κ_v	velocidade de passagem de v_0 para θ_v ao longo dos prazos
σ_v	curvatura do smile
ρ	inclinação (skew)
λ, μ_J, δ	skew e caudas dos prazos curtos

Vários parâmetros têm efeitos parecidos. Isso antecipa o que veremos na calibração: algumas combinações são bem determinadas pelo mercado, outras não.

8. Síntese

- O BCC97 junta Bates (índice) e CIR (juros), e contém BSM, Merton e Heston como casos particulares.
- A estrutura afim transforma o problema de preço em equações de Riccati, tanto nos juros quanto na volatilidade.
- A fórmula de Heston tem duas formas equivalentes na matemática; só a estável funciona no computador.
- A EDP vale sob Q, sem replicação, e o salto entra por diferença exata.
- O atalho do livro para juros estocásticos é excelente em prazos curtos e se degrada nos longos.
- Cada parâmetro tem uma assinatura na superfície, com sobreposições.

9. Notas sobre o livro neste capítulo

Local	Comentário
§9.2	a lei do salto não corresponde a r_J nem a (9.11); o correto é $J \sim N(\mu_J, \delta^2)$ para o log-salto
Eq. (9.8)	$\partial^2 C/\partial S$ deveria ser $\partial^2 C/\partial S^2$
Eq. (9.8)	faltam os fatores v e r nos termos $\frac{1}{2}\sigma_v^2\,\partial^2 C/\partial v^2$ e $\frac{1}{2}\sigma_r^2\,\partial^2 C/\partial r^2$
Prova da Prop. 8	o salto entra como $\partial A/\partial S \cdot jdN$, aproximação de primeira ordem; o certo é a diferença exata, como a própria (9.8) usa
Prova da Prop. 8	na primeira linha da expansão, $\partial A/\partial t$ aparece dentro do fator $\frac{1}{2}$
§9.5.2	o uso de $r_{0,T} = -\ln B_0(T)/T$ é uma aproximação, não o tratamento de juros estocásticos do artigo original
Eq. (9.10)	a forma usada é a estável, com $c_2 = -d$; ponto positivo do livro

10. Exercícios

Exercício 1. Verifique que $b(\tau)$ do CIR satisfaz $b'(0) = 1$ e interprete.

Solução

Pela equação, $b'(0) = 1 - 0 - 0 = 1$. Para prazos curtos, $b(\tau) \approx \tau$ e $P \approx e^{-r\tau}$: no curto prazo, o desconto é pela taxa atual.

Exercício 2. Use a Riccati de Heston para mostrar que, com $\sigma_v = 0$ e $v_0 = \theta_v$, a função característica é a do BSM.

Solução

Com $\sigma_v = 0$: $B' = -\kappa_v B - \frac{1}{2}(u^2 + iu)$, linear. Com $v_0 = \theta_v$, a variância fica constante, e $A + Bv_0 = -\frac{1}{2}\theta_v(u^2 + iu)\tau$, a função característica do BSM com $\sigma^2 = \theta_v$.

Exercício 3. Mostre que $\mathbb{E}[v_T] = \theta_v + (v_0 - \theta_v)e^{-\kappa_v T}$ e use o resultado para explicar a tabela da Seção 7.

Solução

Tomando esperança em dv: $\frac{d}{dt}\mathbb{E}[v] = \kappa_v(\theta_v - \mathbb{E}[v])$, cuja solução é a dada. A variância média até T é uma média ponderada entre v_0 e θ_v, com mais peso em v_0 para prazos curtos.

Exercício 4. Por que a convexidade da Seção 6 tem o sinal que tem? E para uma put?

Solução

Como função de $\bar{r}$, a call é $S_0 - ...$; seu preço cresce e é convexo em $\bar{r}$, e a média de uma função convexa supera a função da média. Para a put, que cai com $\bar{r}$, a convexidade também vale; por paridade, $P = C - S_0 + Ke^{-\bar{r}T}$, e $Ke^{-\bar{r}T}$ também é convexo.

```
print(f"Módulo 11 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 11 concluído: 19 verificações aprovadas.
```

Capítulo 12

Monte Carlo: discretização e redução de variância

Introdução

A simulação de Monte Carlo precifica uma opção pela média de seus pagamentos descontados ao longo de muitas trajetórias simuladas do ativo. Sua força está na generalidade: funciona com muitos fatores de risco e com contratos que dependem de toda a trajetória. Sua fraqueza é a lentidão: o erro cai só com a raiz quadrada do número de trajetórias.

Dois problemas aparecem na prática. O primeiro é a **discretização**: modelos como o de Heston são definidos em tempo contínuo e precisam ser aproximados em passos discretos, e a aproximação ingênua pode até gerar variâncias negativas. O capítulo compara esquemas de correção e a simulação exata da variância. O segundo é a **variância** do estimador: técnicas como variáveis antitéticas, variáveis de controle e moment matching reduzem o erro sem aumentar o número de trajetórias.

O capítulo mede cada efeito em laboratório e separa com cuidado o que é erro estatístico, que diminui com mais trajetórias, do que é viés, que não diminui.

Fundamentos teóricos

O estimador de Monte Carlo

Proposição 12.1 (Lei forte e teorema central do limite). *Sejam $Y_1, \dots, Y_I$ cópias i.i.d. de $Y = e^{-rT} h(S_T)$ com $\mathbb{E}[Y^2] < \infty$, e $\hat{Y}_I = \frac{1}{I}\sum_i Y_i$. Então $\hat{Y}_I \to \mathbb{E}[Y]$ quase certamente e*

$$\sqrt{I}\,(\hat{Y}_I - \mathbb{E}[Y]) \Rightarrow N(0, \mathrm{Var}(Y)). \tag{12.1}$$

O erro-padrão $\hat{s}/\sqrt{I}$*, com $\hat{s}$ o desvio-padrão amostral, dá intervalos de confiança assintóticos $\hat{Y}_I \pm 1{,}96\,\hat{s}/\sqrt{I}$.*

A taxa $I^{-1/2}$ não depende da dimensão do problema, o que torna o método competitivo para produtos dependentes de trajetória e modelos multifatoriais (Boyle et al., 1997; Glasserman, 2004).

Discretização de EDEs

Definição 12.2 (Esquema de Euler). Para $dX_t = a(X_t)\,dt + b(X_t)\,dW_t$ e passo Δt, o esquema de Euler-Maruyama é $\hat{X}_{k+1} = \hat{X}_k + a(\hat{X}_k)\Delta t + b(\hat{X}_k)\sqrt{\Delta t}\,z_{k+1}$, com $z_k \sim N(0,1)$ i.i.d.

Teorema 12.3 (Ordens de convergência). *Sob condições de Lipschitz e crescimento linear sobre a e b, o esquema de Euler tem ordem forte $\frac{1}{2}$, $\mathbb{E}|\hat{X}_T - X_T| = O(\Delta t^{1/2})$, e, para g suave, ordem fraca* 1, $|\mathbb{E}\,g(\hat{X}_T) - \mathbb{E}\,g(X_T)| = O(\Delta t)$ *(Kloeden and Platen, 1992).*

Para precificar opções importa a ordem *fraca*. O erro total de Monte Carlo combina o viés de discretização, $O(\Delta t)$, e o erro estatístico, $O(I^{-1/2})$; com custo proporcional a $I/\Delta t$, o equilíbrio ótimo é $I \propto \Delta t^{-2}$.

O processo de raiz quadrada em tempo discreto

O processo (11.3) não satisfaz a condição de Lipschitz em 0, e o esquema de Euler pode produzir valores negativos, para os quais $\sqrt{x}$ não está definido.

Definição 12.4 (Esquemas corrigidos). Com f_1, f_2, f_3 funções de correção, os esquemas de Euler corrigidos são

$$\tilde{x}_{k+1} = f_1(\tilde{x}_k) + \kappa(\theta - f_2(\tilde{x}_k))\Delta t + \sigma\sqrt{f_3(\tilde{x}_k)}\sqrt{\Delta t}\,z_{k+1}, \qquad x_{k+1} = f_3(\tilde{x}_{k+1}). \tag{12.2}$$

Full truncation usa $f_1 = \mathrm{id}$, $f_2 = f_3 = x^+$; *reflection* usa $f_1 = f_2 = f_3 = |x|$ (Lord et al., 2010).

Proposição 12.5 (Esquema exato). *Pela Proposição 11.2, x_{k+1} pode ser amostrado exatamente da lei qui-quadrado não central condicional a x_k. Não há viés de discretização em x; o custo é maior, e funções integradas como $\int x_t\,dt$ ainda precisam de aproximação (Broadie and Kaya, 2006; Andersen, 2008).*

Redução de variância

Proposição 12.6 (Variáveis antitéticas). *Se $Y = g(z)$ e $Y' = g(-z)$, então* $\mathrm{Var}(\frac{Y+Y'}{2}) = \frac{1}{2}\mathrm{Var}(Y)(1 + \mathrm{corr}(Y, Y'))$. *O ganho existe se g é monótona, pois então* $\mathrm{corr}(Y, Y') \le 0$.

Proposição 12.7 (Variável de controle). *Seja X com $\mathbb{E}[X]$ conhecido. Para $Y_\beta = Y - \beta(X - \mathbb{E}[X])$, $\mathbb{E}[Y_\beta] = \mathbb{E}[Y]$ e*

$$\mathrm{Var}(Y_\beta) = \mathrm{Var}(Y) - 2\beta\,\mathrm{Cov}(X, Y) + \beta^2\mathrm{Var}(X), \tag{12.3}$$

minimizada em $\beta^* = \mathrm{Cov}(X,Y)/\mathrm{Var}(X)$, onde vale $\mathrm{Var}(Y_{\beta^*}) = (1 - \rho_{XY}^2)\,\mathrm{Var}(Y)$.

Demonstração. A expressão da variância é direta; é uma parábola em β com mínimo em β^*. Substituindo, $\mathrm{Var}(Y) - \mathrm{Cov}(X,Y)^2/\mathrm{Var}(X) = (1 - \rho_{XY}^2)\mathrm{Var}(Y)$. ∎

Definição 12.8 (Moment matching e martingal empírica). O *moment matching* normaliza os choques simulados para média amostral 0 e variância 1. A *martingal empírica* ajusta as trajetórias para que a média amostral de $e^{-rt}S_t$ seja exatamente S_0 em cada data (Duan and Simonato, 1998). Ambos reduzem variância, mas introduzem dependência entre trajetórias, e o erro-padrão usual deixa de ser exato.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulo 10 do livro.

O que você vai conseguir fazer ao final

1. Distinguir erro forte de erro fraco e medir a ordem de convergência de um esquema de Euler.
2. Deduzir a transição exata do processo de raiz quadrada (CIR) e simulá-la com qui-quadrados.
3. Avaliar títulos zero-cupom no CIR por Monte Carlo e separar viés de discretização de ruído.
4. Conhecer os esquemas de Euler para a variância de Heston e saber quais deles são, de fato, diferentes.
5. Encontrar, medir e explicar um viés de "olhar para a frente" no código do livro.
6. Usar variáveis antitéticas, moment matching e variáveis de controle, sabendo o que cada uma corrige.
7. Avaliar puts americanas no modelo Heston com juros CIR por LSM com variável de controle.

Onde estamos

Até aqui, quase todo preço europeu veio de fórmulas: BSM no Módulo 5, Fourier nos Módulos 7, 8, 10 e 11. O Monte Carlo apareceu no Módulo 9, para as americanas, num mundo BSM em que a simulação de S era exata.

Agora o mundo é o do Módulo 11: variância e juros seguem processos de raiz quadrada. Simular exige **discretizar**, e discretizar introduz erros que não diminuem com mais trajetórias. Este módulo trata dessas duas fontes de erro, a de discretização (viés) e a estatística (ruído), e das técnicas para reduzir cada uma.

0. Preparação

Além do ambiente padrão, a célula traz três ferramentas dos módulos anteriores:

- a função característica de Heston na forma estável e o preço de Lewis (Módulos 7 e 11), que servem de referência "exata";
- o preço do título zero-cupom no CIR (Módulo 11);
- o critério de acurácia do livro: um preço está certo se o erro absoluto for menor que 2,5 centavos **ou** o erro relativo for menor que 1,5%.

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import simpson, solve_ivp
from scipy.stats import norm, ncx2, kstest
from IPython.display import display
```

```
RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

def bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - sig * np.sqrt(T))

def cf_heston(u, T, v0, kappa, theta, sigma, rho):
    """Função característica de ln(S_T/S_0) - rT no Heston, forma estável."""
    xi = kappa - rho * sigma * 1j * u
    d = np.sqrt(xi**2 + sigma**2 * (1j * u + u**2))
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
    A = kappa * theta / sigma**2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / sigma**2 * (1 - e) / (1 - g * e)
    return np.exp(A + B * v0)

_U = np.linspace(0, 300, 30001)

def call_heston(K, T, r, v0, kappa, theta, sigma, rho, S0=100.0):
    """Call europeia de Heston pela fórmula de Lewis (vetorizada em K)."""
    K = np.atleast_1d(np.asarray(K, float))
    k = np.log(S0 / K) + r * T
    h = np.exp(1j * _U * k[:, None]) * cf_heston(_U - 0.5j, T, v0, kappa, theta, sigma, rho)
    integral = simpson(h.real / (_U**2 + 0.25), x=_U, axis=1)
    return S0 - np.sqrt(S0 * K) * np.exp(-0.5 * r * T) / np.pi * integral

def zcb_cir(r0, kappa, theta, sigma, tau):
    """Preço do zero-cupom no CIR (Módulo 11)."""
    g = np.sqrt(kappa**2 + 2 * sigma**2)
    e = np.exp(g * tau) - 1
    den = 2 * g + (kappa + g) * e
    A = (2 * g * np.exp((kappa + g) * tau / 2) / den) ** (2 * kappa * theta / sigma**2)
    return A * np.exp(-2 * e / den * r0)

def acerto(erro, ref, abs_tol=0.025, rel_tol=0.015):
    """Critério do livro: certo se |erro| < 2,5 centavos ou |erro| < 1,5% do preço."""
    erro, ref = np.asarray(erro), np.asarray(ref)
    return (np.abs(erro) < abs_tol) | (np.abs(erro) < rel_tol * np.abs(ref))

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. Discretizar uma equação diferencial estocástica

Livro: §10.1 e §10.3 (equações 10.6 e 10.7)

1.1 O esquema de Euler, passo a passo

Considere $dX_t = a(X_t)\,dt + b(X_t)\,dW_t$. Dividimos $[0, T]$ em M passos de tamanho $\Delta t = T/M$. Num passo de s a $t = s + \Delta t$, a solução é

$$X_t = X_s + \int_s^t a(X_u)\,du + \int_s^t b(X_u)\,dW_u. \tag{12.4}$$

Euler congela os coeficientes no **início** do passo, $a(X_u) \approx a(X_s)$ e $b(X_u) \approx b(X_s)$:

$$X_t \approx X_s + a(X_s)\,\Delta t + b(X_s)\,(W_t - W_s) = X_s + a(X_s)\,\Delta t + b(X_s)\sqrt{\Delta t}\,z, \qquad z \sim N(0,1). \tag{12.5}$$

Repare no detalhe que vai importar na Seção 4: o coeficiente b é avaliado em X_s, um valor **conhecido** antes do choque z. É isso que faz a integral estocástica de Itô ter média zero: $\mathbb{E}[b(X_s)\,z] = \mathbb{E}[b(X_s)]\,\mathbb{E}[z] = 0$, porque z é independente do passado.

1.2 Dois tipos de erro

Seja $\hat{X}_T$ o valor simulado e X_T o verdadeiro, ambos com o mesmo browniano.

- **Erro forte**: $\mathbb{E}\,|\hat{X}_T - X_T|$. Mede se as *trajetórias* estão certas. Para Euler, cai como $\Delta t^{1/2}$.
- **Erro fraco**: $|\mathbb{E}[f(\hat{X}_T)] - \mathbb{E}[f(X_T)]|$. Mede se as *distribuições* estão certas. Para Euler, cai como Δt^{1}.

Para preço de opção europeia, só o erro fraco interessa: queremos a média do payoff. O erro fraco é um **viés**: ele não diminui com mais trajetórias, só com mais passos.

1.3 Três esquemas para o índice

Para $dS = rS\,dt + \sigma S\,dW$ há três escolhas naturais:

Esquema	Passo	Observação
Euler clássico	$S_t = S_s(1 + r\Delta t + \sigma\sqrt{\Delta t}\,z)$	média $(1 + r\Delta t)$, não $e^{r\Delta t}$
Ingênuo (10.7)	$S_t = S_s(e^{r\Delta t} + \sigma\sqrt{\Delta t}\,z)$	média certa, distribuição normal no passo
Log-Euler (10.6)	$S_t = S_s \exp((r - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\,z)$	exato para σ constante

O log-Euler vem de aplicar Itô a $\ln S$ (Módulo 4): $d\ln S = (r - \frac{1}{2}\sigma^2)dt + \sigma\,dW$, uma equação de coeficientes constantes, que Euler integra sem erro. Com volatilidade estocástica ele deixa de ser exato, mas continua sendo o melhor ponto de partida.

Laboratório 1 — A ordem do erro fraco

Usamos uma call fora do dinheiro ($K = 130$), em que o viés é mais visível. Para medir um viés de poucos centavos sem que o ruído o encubra, usamos **números aleatórios comuns**: geramos os incrementos brownianos numa grade fina e os somamos para as grades grossas. Assim, todos os esquemas veem o mesmo W_T, e o log-Euler (exato) dá o payoff de referência trajetória a trajetória. A diferença média entre um esquema e a referência é o viés, com pouquíssimo ruído.

```
S0, K, r, sig, T = 100.0, 130.0, 0.05, 0.3, 1.0   # call fora do dinheiro
I, Mfino = 200_000, 64
rng = np.random.default_rng(12)
dW = rng.standard_normal((Mfino, I)) * np.sqrt(T / Mfino)
payoff_exato = np.maximum(S0 * np.exp((r - 0.5 * sig**2) * T + sig * dW.sum(0)) - K, 0)
```

```
linhas = []
for M in [1, 2, 4, 8, 16, 32, 64]:
    dt = T / M
    inc = dW.reshape(M, Mfino // M, I).sum(1)          # incrementos da grade grossa
    S_cl = S0 * np.prod(1 + r * dt + sig * inc, axis=0)
    S_in = S0 * np.prod(np.exp(r * dt) + sig * inc, axis=0)
    vies_cl = np.exp(-r * T) * np.mean(np.maximum(S_cl - K, 0) - payoff_exato)
    vies_in = np.exp(-r * T) * np.mean(np.maximum(S_in - K, 0) - payoff_exato)
    linhas.append((M, dt, vies_cl, vies_in))
tab1 = pd.DataFrame(linhas, columns=["M", "Δt", "viés Euler clássico", "viés ingênuo (10.7)"])
tabela(tab1)
print("preço exato BSM:", f3(bsm(S0, K, T, r, sig)),
      "| média MC do log-Euler:", f3(np.exp(-r * T) * payoff_exato.mean()))
```

M	Δt	viés Euler clássico	viés ingênuo (10.7)
1	1,000	-1,425	-1,400
2	0,500	-0,681	-0,666
4	0,250	-0,318	-0,309
8	0,125	-0,156	-0,151
16	0,062	-0,079	-0,077
32	0,031	-0,039	-0,037
64	0,016	-0,019	-0,018

```
preço exato BSM: 4,673 | média MC do log-Euler: 4,633
```

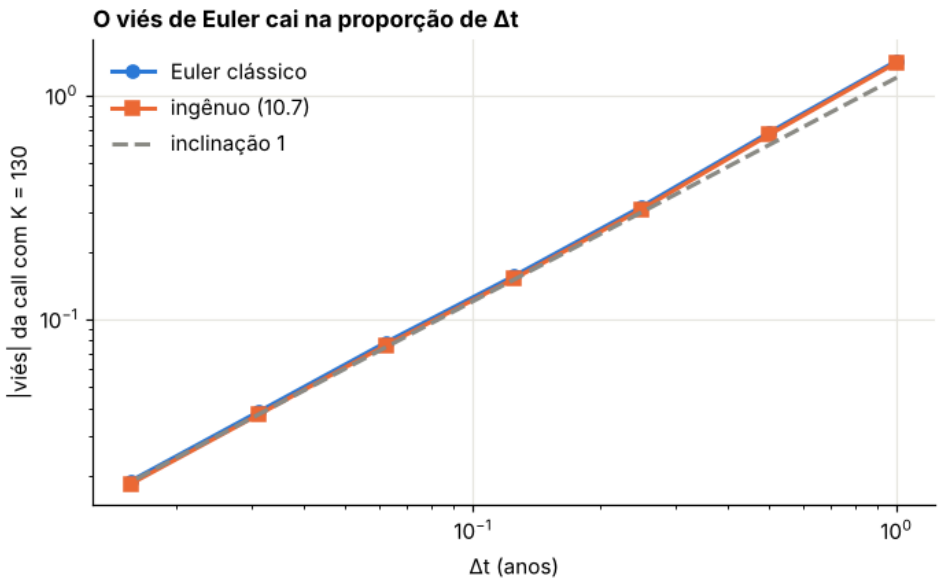
A tabela mostra o viés caindo pela metade quando M dobra: é a assinatura da ordem 1. Vamos estimá-la pela inclinação em escala log-log, usando $M \geq 4$ (para M muito pequeno, termos de ordem superior ainda pesam).

```
sel = tab1["M"] >= 4
p_cl = np.polyfit(np.log(tab1.loc[sel, "Δt"]), np.log(np.abs(tab1.loc[sel, "viés Euler
↪  clássico"])), 1)[0]
p_in = np.polyfit(np.log(tab1.loc[sel, "Δt"]), np.log(np.abs(tab1.loc[sel, "viés ingênuo
↪  (10.7)"])), 1)[0]
print("ordem estimada — clássico:", f3(p_cl), "| ingênuo:", f3(p_in))

fig, ax = plt.subplots()
ax.loglog(tab1["Δt"], np.abs(tab1["viés Euler clássico"]), "o-", color=AZUL, label="Euler
↪  clássico")
ax.loglog(tab1["Δt"], np.abs(tab1["viés ingênuo (10.7)"]), "s-", color=LARANJA, label="ingênuo
↪  (10.7)")
ax.loglog(tab1["Δt"], 1.2 * tab1["Δt"], "--", color=CINZA, label="inclinação 1")
ax.set_xlabel("Δt (anos)")
ax.set_ylabel("|viés| da call com K = 130")
ax.set_title("O viés de Euler cai na proporção de Δt")
ax.legend()
plt.show()

ok(0.8 < p_cl < 1.2, "o erro fraco do Euler clássico tem ordem 1")
ok(0.8 < p_in < 1.2, "o erro fraco do esquema ingênuo também tem ordem 1")
ok(abs(np.exp(-r * T) * payoff_exato.mean() - bsm(S0, K, T, r, sig)) < 0.1,
   "o log-Euler com σ constante reproduz o BSM, a menos do ruído")
```

```
ordem estimada — clássico: 1,015 | ingênuo: 1,016
```




```
OK — o erro fraco do Euler clássico tem ordem 1
OK — o erro fraco do esquema ingênuo também tem ordem 1
OK — o log-Euler com σ constante reproduz o BSM, a menos do ruído
```

O que aprendemos. Com 64 passos por ano o viés ainda é de cerca de 2 centavos numa call de uns 4,7, e sempre para baixo. O esquema ingênuo tem a média certa a cada passo, mas a distribuição errada; e o preço de uma opção depende da distribuição. A lição para o resto do módulo: **acertar a média de S_T não garante acertar o preço**, e errar a média garante errar.

---

2. O processo de raiz quadrada: a transição exata

Livro: §10.2

O processo

$$dx_t = \kappa(\theta - x_t)\,dt + \sigma\sqrt{x_t}\,dZ_t \tag{10.1}$$

é a dinâmica dos juros no CIR e da variância no Heston. Ao contrário do caso geral, aqui a lei de x_t dado x_s é conhecida, e podemos simular sem erro de discretização.

2.1 Dedução pela estrutura afim

No Módulo 11, deduzimos o zero-cupom tentando $\exp(A + B\,x)$. Fazemos o mesmo com a função geradora de momentos. Seja $\tau = t - s$ e w real com $1 - 2c(\tau)w > 0$. Procuramos

$$\mathbb{E}\big[e^{w x_t} \,\big|\, x_s\big] = \exp\big(A(\tau) + B(\tau)\,x_s\big). \tag{12.6}$$

Passo 1 — as equações. O lado esquerdo, como função de (s, x_s), é uma martingal; por Feynman-Kac, satisfaz $-\partial_\tau F + \kappa(\theta - x)F_x + \frac{1}{2}\sigma^2 x F_{xx} = 0$, com $F(0,x) = e^{wx}$. Substituindo a forma afim e separando os termos com x dos sem x:

$$B' = -\kappa B + \tfrac{1}{2}\sigma^2 B^2, \quad B(0) = w; \qquad A' = \kappa\theta B, \quad A(0) = 0. \tag{12.7}$$

Passo 2 — resolver B. É uma equação de Bernoulli. Com $u = 1/B$, temos $u' = -B'/B^2 = \kappa u - \frac{1}{2}\sigma^2$, que é linear:

$$u(\tau) = \Big(\frac{1}{w} - \frac{\sigma^2}{2\kappa}\Big)e^{\kappa\tau} + \frac{\sigma^2}{2\kappa} \quad\Longrightarrow\quad B(\tau) = \frac{w\,e^{-\kappa\tau}}{1 - 2c\,w}, \qquad c \equiv \frac{\sigma^2(1-e^{-\kappa\tau})}{4\kappa}. \tag{12.8}$$

Passo 3 — resolver A. Como $\frac{d}{d\tau}\ln(1-2cw) = -\frac{\sigma^2}{2}\,\frac{we^{-\kappa\tau}}{1-2cw} = -\frac{\sigma^2}{2}B$, temos

$$A(\tau) = \kappa\theta\int_0^\tau B = -\frac{2\kappa\theta}{\sigma^2}\ln(1-2cw) = -\frac{d}{2}\ln(1-2cw), \qquad d \equiv \frac{4\kappa\theta}{\sigma^2}. \tag{12.9}$$

Passo 4 — reconhecer a distribuição. Juntando,

$$\mathbb{E}\big[e^{wx_t}\,\big|\,x_s\big] = (1-2cw)^{-d/2}\exp\Big(\frac{\lambda\,c\,w}{1-2cw}\Big), \qquad \lambda \equiv \frac{x_s\,e^{-\kappa\tau}}{c}. \tag{12.10}$$

A geradora de momentos de uma qui-quadrado não central $Y \sim \chi'^2_d(\lambda)$ é $\mathbb{E}[e^{vY}] = (1-2v)^{-d/2}\exp\left(\lambda v/(1-2v)\right)$. Com $v = cw$, as duas coincidem. Logo

$$\boxed{\;x_t = c\,Y, \qquad Y \sim \chi'^2_d(\lambda), \qquad c = \frac{\sigma^2(1-e^{-\kappa\Delta t})}{4\kappa}, \quad d = \frac{4\kappa\theta}{\sigma^2}, \quad \lambda = \frac{4\kappa e^{-\kappa\Delta t}}{\sigma^2(1-e^{-\kappa\Delta t})}\,x_s.\;} \tag{12.11}$$

É a fórmula do livro. Note que d não depende de Δt, e que a condição de Feller $2\kappa\theta \geq \sigma^2$ é exatamente $d \geq 2$.

2.2 Como sortear uma qui-quadrado não central

O livro usa duas decomposições, que vêm direto da definição $\chi'^2_d(\lambda) = \sum_{j=1}^{d}(z_j + \mu_j)^2$ com $\sum \mu_j^2 = \lambda$:

- **Se $d > 1$:** ponha toda a não centralidade numa coordenada: $\chi'^2_d(\lambda) = (z + \sqrt{\lambda})^2 + \chi^2_{d-1}$.
- **Se $d \leq 1$** (aí $d-1 \leq 0$ não serve): use a mistura de Poisson $\chi'^2_d(\lambda) = \chi^2_{d+2N}$, com $N \sim \text{Poisson}(\lambda/2)$.

Os dois primeiros momentos condicionais, que vamos conferir, são

$$\mathbb{E}[x_t\,|\,x_s] = \theta + (x_s - \theta)e^{-\kappa\Delta t}, \qquad \text{Var}[x_t\,|\,x_s] = x_s\frac{\sigma^2 e^{-\kappa\Delta t}}{\kappa}(1-e^{-\kappa\Delta t}) + \theta\frac{\sigma^2}{2\kappa}(1-e^{-\kappa\Delta t})^2. \tag{12.12}$$

Laboratório 2 — Conferir a transição exata

Três conferências: (a) a solução fechada de B e A contra a integração numérica das equações; (b) as duas decomposições contra os momentos teóricos; (c) a distribuição inteira contra a do SciPy, por um teste de Kolmogorov-Smirnov.

```
def cir_constantes(kappa, theta, sigma, dt):
    c = sigma**2 * (1 - np.exp(-kappa * dt)) / (4 * kappa)
    d = 4 * kappa * theta / sigma**2
    return c, d


def cir_exato_passo(x, kappa, theta, sigma, dt, rng):
    """Um passo exato do processo de raiz quadrada, pelas decomposições do livro."""
    c, d = cir_constantes(kappa, theta, sigma, dt)
    lam = x * np.exp(-kappa * dt) / c
    if d > 1:
        z = rng.standard_normal(x.shape)
        return c * ((z + np.sqrt(lam)) ** 2 + rng.chisquare(d - 1, x.shape))
    N = rng.poisson(lam / 2)
    return c * rng.chisquare(d + 2 * N)


# (a) Riccati numérica contra a solução fechada
kappa, theta, sigma, tau, w = 0.3, 0.04, 0.1, 2.0, 3.0
sol = solve_ivp(lambda t, y: [kappa * theta * y[1], -kappa * y[1] + 0.5 * sigma**2 * y[1] ** 2],
                [0, tau], [0.0, w], rtol=1e-11, atol=1e-13)
A_num, B_num = sol.y[:, -1]
c, d = cir_constantes(kappa, theta, sigma, tau)
B_fec = w * np.exp(-kappa * tau) / (1 - 2 * c * w)
A_fec = -d / 2 * np.log(1 - 2 * c * w)
print("B: numérico", f3(B_num), "fechado", f3(B_fec), "| A: numérico", f3(A_num), "fechado",
↪  f3(A_fec))
ok(abs(B_num - B_fec) < 1e-8 and abs(A_num - A_fec) < 1e-8, "a solução fechada da Riccati confere
↪  com a integração numérica")

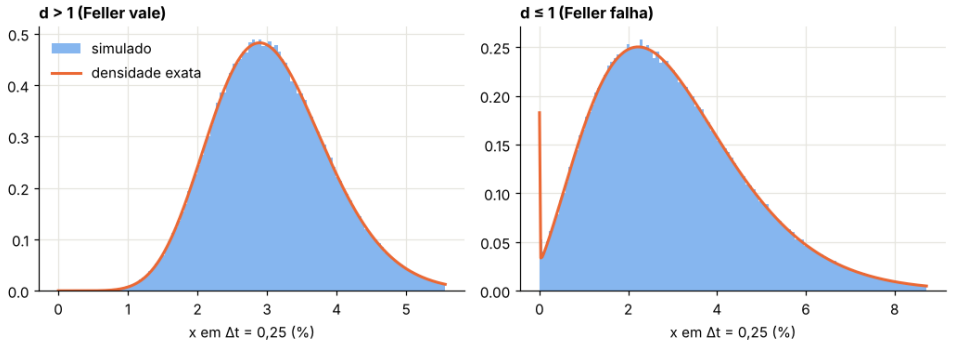
# (b) e (c): momentos e distribuição, nos dois regimes de d
rng = np.random.default_rng(2)
casos = {"d > 1 (Feller vale)": (0.3, 0.04, 0.1), "d ≤ 1 (Feller falha)": (0.1, 0.03, 0.2)}
x_s, dt, n = 0.03, 0.25, 400_000
linhas = []
amostras = {}
for nome, (kappa, theta, sigma) in casos.items():
    c, d = cir_constantes(kappa, theta, sigma, dt)
    xt = cir_exato_passo(np.full(n, x_s), kappa, theta, sigma, dt, rng)
    e = np.exp(-kappa * dt)
    media = theta + (x_s - theta) * e
    var = x_s * sigma**2 * e / kappa * (1 - e) + theta * sigma**2 / (2 * kappa) * (1 - e) ** 2
    ks = kstest(xt[:20000] / c, ncx2(d, x_s * e / c).cdf).pvalue
    linhas.append((nome, d, xt.mean() * 100, media * 100, xt.std() * 100, np.sqrt(var) * 100, ks))
    amostras[nome] = (xt, c, d, x_s * e / c)
tab2 = pd.DataFrame(linhas, columns=["caso", "d", "média MC (%)", "média teórica (%)",
                                     "desvio MC (%)", "desvio teórico (%)", "p-valor KS"])
tabela(tab2)
for _, l in tab2.iterrows():
    ok(abs(l["média MC (%)"] / l["média teórica (%)"] - 1) < 0.005, f"{l['caso']}: a média
↪  confere")
    ok(abs(l["desvio MC (%)"] / l["desvio teórico (%)"] - 1) < 0.01, f"{l['caso']}: o desvio
↪  confere")
    ok(l["p-valor KS"] > 0.001, f"{l['caso']}: o teste KS não rejeita a qui-quadrado não central")
```

```
B: numérico 1,684 fechado 1,684 | A: numérico 0,055 fechado 0,055
OK — a solução fechada da Riccati confere com a integração numérica
```

caso	d	média MC (%)	média teórica (%)	desvio MC (%)	desvio teórico (%)	p-valor KS
d > 1 (Feller vale)	4,800	3,074	3,072	0,840	0,840	0,243
d ≤ 1 (Feller falha)	0,300	3,001	3,000	1,712	1,711	0,999

```
OK — d > 1 (Feller vale): a média confere
OK — d > 1 (Feller vale): o desvio confere
OK — d > 1 (Feller vale): o teste KS não rejeita a qui-quadrado não central
OK — d ≤ 1 (Feller falha): a média confere
OK — d ≤ 1 (Feller falha): o desvio confere
OK — d ≤ 1 (Feller falha): o teste KS não rejeita a qui-quadrado não central
```

```python
fig, axs = plt.subplots(1, 2, figsize=(9.5, 3.6))
for ax, (nome, (xt, c, d, lam)) in zip(axs, amostras.items()):
    grade = np.linspace(1e-5, np.quantile(xt, 0.995), 300)
    ax.hist(xt * 100, bins=120, range=(0, grade[-1] * 100), density=True, color=RAMPA[0],
    ↪ label="simulado")
    ax.plot(grade * 100, ncx2(d, lam).pdf(grade / c) / c / 100, color=LARANJA, label="densidade
    ↪ exata")
    ax.set_title(nome)
    ax.set_xlabel("x em Δt = 0,25 (%)")
axs[0].legend()
plt.tight_layout()
plt.show()
```



Com $d \leq 1$ (direita), a densidade explode em zero: o processo encosta no zero com frequência. É justamente o regime em que os esquemas de Euler sofrem, porque o $\sqrt{x}$ de um valor discretizado pode ficar negativo.

---

3. Zero-cupons no CIR por Monte Carlo

Livro: §10.2 (Figuras 10.1 a 10.3)

3.1 O estimador

Numa trajetória i com grade $t_0 = 0 < t_1 < \cdots < t_M = T$, descontamos o valor 1 de trás para a frente, com a regra do trapézio para a integral de r:

$$B_{t_j,i} = B_{t_{j+1},i}\,\exp\!\Big(-\frac{r_{t_j,i}+r_{t_{j+1},i}}{2}\,\Delta t\Big), \qquad B_{T,i}=1. \tag{12.13}$$

Em $t=0$, a média $\frac{1}{I}\sum_i B_{0,i}$ estima $P(0,T)$, que conhecemos em forma fechada.

3.2 O que a média estima em $t>0$

Em $t>0$, cada $B_{t,i}$ estima o preço do zero-cupom **condicional** à taxa $r_{t,i}$ daquela trajetória. A média sobre trajetórias estima, portanto, $\mathbb{E}[P(r_t, T-t)]$.

O livro compara essa média com $P(\mathbb{E}[r_t], T-t)$. Como P é convexa em r ($P=e^{A-br}$), pela desigualdade de Jensen

$$\mathbb{E}[P(r_t,\tau)] \geq P(\mathbb{E}[r_t],\tau). \tag{12.14}$$

A referência certa pode ser calculada **exatamente** com o que deduzimos na Seção 2: $\mathbb{E}[e^{A-b\,r_t}] = e^{A}\,\mathbb{E}[e^{-b\,r_t}]$, e a esperança é a geradora de momentos da qui-quadrado não central em $w=-b$.

3.3 O esquema de Euler para comparação

O livro usa o **Full Truncation** (10.2)-(10.3): trunca em zero dentro do drift e da raiz, e trunca a saída:

$$\tilde{x}_t = \tilde{x}_s + \kappa(\theta-\tilde{x}_s^+)\Delta t + \sigma\sqrt{\tilde{x}_s^+}\sqrt{\Delta t}\,z_t, \qquad x_t=\tilde{x}_t^+. \tag{12.15}$$

Laboratório 3 — Exato contra Euler, e a referência certa

Usamos os parâmetros do livro: $r_0=1\%$, $\kappa_r=0{,}1$, $\theta_r=3\%$, $\sigma_r=0{,}2$, $T=2$, $M=50$, $I=50.000$. Aqui $d=0{,}3$: a condição de Feller falha com folga.

```python
def cir_euler_ft_passo(xh, kappa, theta, sigma, dt, z):
    """Full Truncation: devolve (x truncado, x~ auxiliar)."""
    p = np.maximum(xh, 0)
    xh = xh + kappa * (theta - p) * dt + sigma * np.sqrt(p * dt) * z
    return np.maximum(xh, 0), xh


def cir_trajetorias(r0, kappa, theta, sigma, T, M, I, rng, esquema="exato"):
    dt = T / M
    x = np.empty((M + 1, I))
    x[0] = r0
    xh = np.full(I, r0)
    for t in range(M):
        if esquema == "exato":
            x[t + 1] = cir_exato_passo(x[t], kappa, theta, sigma, dt, rng)
        else:
            x[t + 1], xh = cir_euler_ft_passo(xh, kappa, theta, sigma, dt, rng.standard_normal(I))
    return x


def zcb_mc_todas_datas(x, dt):
    """B_t estimado em cada data t_j, j = 0..M-1 (desconto trapezoidal de T até t_j)."""
    fatias = (x[1:] + x[:-1]) / 2 * dt
    return np.exp(-np.cumsum(fatias[::-1], axis=0))[::-1].mean(axis=1)
```

```
r0, kr, thr, sr, T, M, I = 0.01, 0.1, 0.03, 0.2, 2.0, 50, 50_000
dt = T / M
ts = np.arange(M) * dt


def ref_jensen(t):
    """E[P(r_t, T-t)] exato, pela geradora de momentos da qui-quadrado não central."""
    if t == 0:
        return zcb_cir(r0, kr, thr, sr, T)
    tau = T - t
    P_em_zero = zcb_cir(0.0, kr, thr, sr, tau)                  # e^A
    b = -np.log(zcb_cir(1.0, kr, thr, sr, tau) / P_em_zero)     # P = e^{A - b r}
    c, d = cir_constantes(kr, thr, sr, t)
    lam = r0 * np.exp(-kr * t) / c
    w = -b
    return P_em_zero * (1 - 2 * c * w) ** (-d / 2) * np.exp(lam * c * w / (1 - 2 * c * w))


ref_certa = np.array([ref_jensen(t) for t in ts])
ref_livro = np.array([zcb_cir(thr + np.exp(-kr * t) * (r0 - thr), kr, thr, sr, T - t) for t in ts])

rng = np.random.default_rng(10)
R = 5
est = {e: np.array([zcb_mc_todas_datas(cir_trajetorias(r0, kr, thr, sr, T, M, I, rng, e), dt)
                    for _ in range(R)]) for e in ["exato", "euler"]}

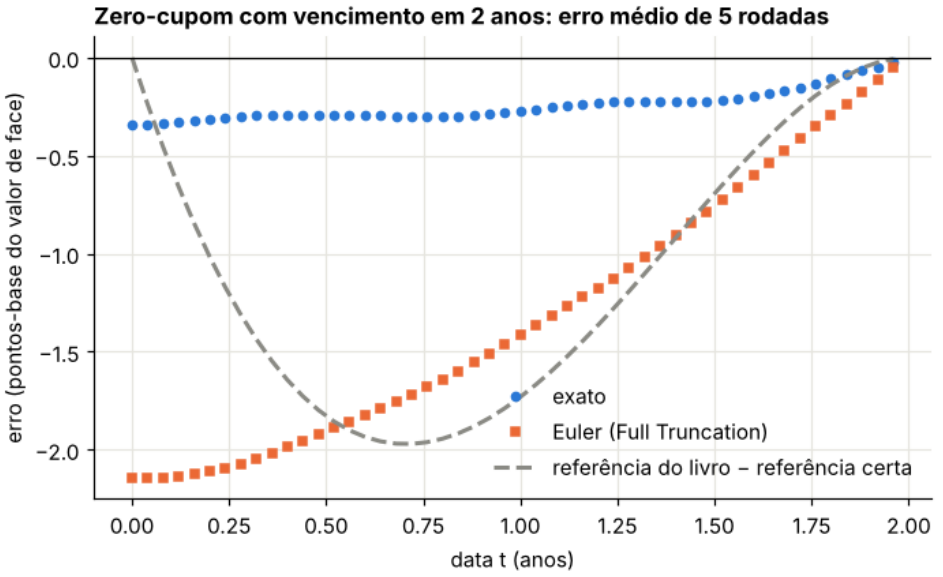
linhas = []
for e in est:
    erro = est[e] - ref_certa
    linhas.append((e, 1e4 * erro[:, 0].mean(), 1e4 * erro.mean(), 1e4 * erro.std() /
↪   np.sqrt(erro.size),
                   1e4 * (est[e] - ref_livro).mean()))
tab3 = pd.DataFrame(linhas, columns=["esquema", "erro em t=0 (pb)", "erro médio (pb)",
↪   "erro-padrão (pb)",
                                     "erro contra a referência do livro (pb)"])
tabela(tab3)
gap = ref_certa - ref_livro
print("diferença de Jensen entre as referências: máxima", f3(1e4 * gap.max()), "pb")
```

esquema	erro em t=0 (pb)	erro médio (pb)	erro-padrão (pb)	erro contra a referência do livro (pb)
exato	-0,340	-0,245	0,103	0,881
euler	-2,142	-1,325	0,097	-0,198

```
diferença de Jensen entre as referências: máxima 1,970 pb
```

```
fig, ax = plt.subplots()
ax.plot(ts, 1e4 * (est["exato"] - ref_certa).mean(0), "o", ms=4, color=AZUL, label="exato")
ax.plot(ts, 1e4 * (est["euler"] - ref_certa).mean(0), "s", ms=4, color=LARANJA, label="Euler
↪   (Full Truncation)")
ax.plot(ts, -1e4 * gap, "--", color=CINZA, label="referência do livro − referência certa")
ax.axhline(0, color="black", lw=0.8)
ax.set_xlabel("data t (anos)")
ax.set_ylabel("erro (pontos-base do valor de face)")
ax.set_title("Zero-cupom com vencimento em 2 anos: erro médio de 5 rodadas")
ax.legend()
plt.show()
```

```
ok(abs(tab3.loc[0, "erro médio (pb)"]) < 0.25 * abs(tab3.loc[1, "erro médio (pb)"]) + 3 *
↪  tab3.loc[0, "erro-padrão (pb)"],
   "o esquema exato tem erro muito menor que o do Euler")
ok(tab3.loc[1, "erro médio (pb)"] < -3 * tab3.loc[1, "erro-padrão (pb)"],
   "o Euler tem viés para baixo, como no livro")
ok(np.all(gap[1:] > 0), "a desigualdade de Jensen: E[P(r_t)] > P(E[r_t]) em todas as datas t > 0")
```




```
OK — o esquema exato tem erro muito menor que o do Euler
OK — o Euler tem viés para baixo, como no livro
OK — a desigualdade de Jensen: E[P(r_t)] > P(E[r_t]) em todas as datas t > 0
```

O que aprendemos.

- O viés do Euler existe e é para baixo, como diz o livro, mas é pequeno: um a dois pontos-base do valor de face.
- O "exato" ainda carrega o pequeno erro da regra do trapézio na integral de r: a transição de r é exata, a integral não.
- Ele é da **mesma ordem** que o erro da referência usada nas Figuras 10.2 e 10.3. Contra a referência do livro, o exato parece ter viés para cima e o Euler parece quase sem viés. Para comparar esquemas, a referência precisa ser mais exata que a diferença que se quer medir.

---

4. O modelo de Heston: discretizar a variância

Livro: §10.3

4.1 O índice

Com juros constantes, sob Q:

$$dS_t = rS_t\,dt + \sqrt{v_t}\,S_t\,dZ_t^1, \qquad dv_t = \kappa_v(\theta_v - v_t)\,dt + \sigma_v\sqrt{v_t}\,dZ_t^2, \qquad dZ^1 dZ^2 = \rho\,dt. \tag{12.16}$$

Pelo Itô, $d\ln S = (r - \frac{1}{2}v)\,dt + \sqrt{v}\,dZ^1$. O log-Euler (10.6) congela v no **início** do passo:

$$S_t = S_s \exp\Big((r - \tfrac{1}{2}v_s)\Delta t + \sqrt{v_s}\sqrt{\Delta t}\,z_t^1\Big). \tag{12.17}$$

Para correlacionar, usamos Cholesky: $z_t^2 = \rho\,z_t^1 + \sqrt{1-\rho^2}\,\varepsilon_t$.

4.2 Cinco esquemas com sete nomes

A transição exata da Seção 2 não se correlaciona facilmente com z^1. Por isso o livro usa esquemas de Euler, que só diferem no que fazem quando $\tilde{x}$ fica negativo. Na notação de Lord, Koekkoek e van Dijk (2006), o passo é

$$\tilde{x}_t = f_1(\tilde{x}_s) + \kappa(\theta - f_2(\tilde{x}_s))\Delta t + \sigma\sqrt{f_3(\tilde{x}_s)}\sqrt{\Delta t}\,z_t, \qquad x_t = f_3(\tilde{x}_t), \tag{12.18}$$

Esquema	f_1	f_2	f_3
Absorption (A)	x^+	x^+	x^+
Reflection (R)	$\lvert x\rvert$	$\lvert x\rvert$	$\lvert x\rvert$
Higham-Mao (HM)	x	x	$\lvert x\rvert$
Partial Truncation (PT)	x	x	x^+
Full Truncation (FT)	x	x^+	x^+

O livro lista mais dois: **Truncation** e **Simple Reflection**. Olhando o código (§10.6.3), Truncation recalcula a partir de $x_s = \max(0, \tilde{x}_s)$ com drift e raiz em x_s: é exatamente o Absorption. Simple Reflection faz o mesmo com $|\cdot|$: é exatamente o Reflection. O laboratório confere isso trajetória a trajetória. (No Reflection, diferenças de arredondamento da ordem de 10^{-17} podem crescer um pouco quando x passa muito perto de zero, porque a derivada de $\sqrt{x}$ explode ali; por isso a tolerância é maior nesse caso.)

> **Nota.** O código do livro para o Reflection usa `xh[t-1]` sem valor absoluto no primeiro termo, ao contrário da fórmula impressa e de Lord et al. Aqui seguimos a fórmula.

```python
ESQUEMAS = ["FT", "PT", "A", "R", "HM"]


def passo_v(esquema, xh, kappa, theta, sigma, dt, z):
    """Um passo dos esquemas de Lord et al. Recebe x˜_s, devolve (x_t, x˜_t)."""
    if esquema == "FT":
        p = np.maximum(xh, 0); xh = xh + kappa * (theta - p) * dt + sigma * np.sqrt(p * dt) * z
        return np.maximum(xh, 0), xh
    if esquema == "PT":
```

```
        p = np.maximum(xh, 0); xh = xh + kappa * (theta - xh) * dt + sigma * np.sqrt(p * dt) * z
        return np.maximum(xh, 0), xh
    if esquema == "A":
        p = np.maximum(xh, 0); xh = p + kappa * (theta - p) * dt + sigma * np.sqrt(p * dt) * z
        return np.maximum(xh, 0), xh
    if esquema == "R":
        p = np.abs(xh); xh = p + kappa * (theta - p) * dt + sigma * np.sqrt(p * dt) * z
        return np.abs(xh), xh
    if esquema == "HM":
        p = np.abs(xh); xh = xh + kappa * (theta - xh) * dt + sigma * np.sqrt(p * dt) * z
        return np.abs(xh), xh
    raise ValueError(esquema)


# Os dois esquemas "extras" do livro, transcritos do código da §10.6.3
def truncation_livro(x, kappa, theta, sigma, dt, z):
    return np.maximum(0, x + kappa * (theta - x) * dt + np.sqrt(x) * sigma * z * np.sqrt(dt))


def simple_reflection_livro(x, kappa, theta, sigma, dt, z):
    return np.abs(x + kappa * (theta - x) * dt + np.sqrt(x) * sigma * z * np.sqrt(dt))


rng = np.random.default_rng(4)
kappa, theta, sigma, dt = 0.75, 0.02, 0.3, 1 / 50
Z = rng.standard_normal((100, 20_000))
xa = xh_a = np.full(20_000, 0.04); xt_ = xt0 = np.full(20_000, 0.04)
xr = xh_r = np.full(20_000, 0.04); xs_ = np.full(20_000, 0.04)
for z in Z:
    xa, xh_a = passo_v("A", xh_a, kappa, theta, sigma, dt, z)
    xt_ = truncation_livro(xt_, kappa, theta, sigma, dt, z)
    xr, xh_r = passo_v("R", xh_r, kappa, theta, sigma, dt, z)
    xs_ = simple_reflection_livro(xs_, kappa, theta, sigma, dt, z)
print("fração de passos com x̃ < 0 no Absorption:", f3(np.mean(xa == 0)))
ok(np.allclose(xa, xt_, rtol=0, atol=1e-14), "Truncation do livro coincide com Absorption em
↪  todas as trajetórias")
print("maior diferença Reflection × Simple Reflection:", f"{np.abs(xr - xs_).max():.1e}")
ok(np.abs(xr - xs_).max() < 1e-8, "Simple Reflection do livro coincide com Reflection, a menos de
↪  arredondamento")
```

```
fração de passos com x̃ < 0 no Absorption: 0,052
OK — Truncation do livro coincide com Absorption em todas as trajetórias
maior diferença Reflection × Simple Reflection: 1.2e-14
OK — Simple Reflection do livro coincide com Reflection, a menos de arredondamento
```

4.3 Um viés escondido no código do livro: olhar para a frente

No código da §10.6.3, o passo do índice é

```
S[t] = S[t - 1] * np.exp((r - 0.5 * v[t]) * dt + np.sqrt(v[t]) * ran[row] * sdt - bias)
```

com `v[t]`, a variância do **fim** do passo, e não `v[t - 1]`. O problema é que `v[t]` foi gerada com o choque z_t^2, que é correlacionado com o mesmo z_t^1 que multiplica a raiz. A propriedade da Seção 1.1 se perde: $\sqrt{v_t}$ não é mais independente de z_t^1.

Quanto vale o viés, passo a passo. Expandindo a raiz em primeira ordem em torno de v_s:

$$\sqrt{v_t} \approx \sqrt{v_s} + \frac{v_t - v_s}{2\sqrt{v_s}} \approx \sqrt{v_s} + \frac{\sigma_v \sqrt{v_s}\sqrt{\Delta t}\, z_t^2}{2\sqrt{v_s}} = \sqrt{v_s} + \tfrac{1}{2}\sigma_v \sqrt{\Delta t}\, z_t^2. \tag{12.19}$$

Então

$$\mathbb{E}\big[\sqrt{v_t}\sqrt{\Delta t}\, z_t^1\big] \approx \tfrac{1}{2}\sigma_v\, \Delta t\, \mathbb{E}[z_t^2 z_t^1] = \tfrac{1}{2}\rho\, \sigma_v\, \Delta t. \tag{12.20}$$

Cada passo ganha um drift espúrio de $\frac{1}{2}\rho\sigma_v\Delta t$ em $\ln S$, que **não some** quando $\Delta t \to 0$: somando M passos, o drift extra é $\frac{1}{2}\rho\sigma_v T$. A previsão é

$$\mathbb{E}[S_T] \approx S_0 e^{rT}\, e^{\rho\sigma_v T/2}. \tag{12.21}$$

Isso não é um erro de discretização, que diminui com Δt: é um erro de **modelo**. O esquema converge para o processo errado (é a diferença entre as integrais de Itô e de Stratonovich, Módulo 4).

Com o `bias` do moment matching, que subtrai a média amostral de $\sqrt{v_t}\sqrt{\Delta t}\, z_t^1$ a cada passo, o drift espúrio é removido. Por isso a Tabela 10.1 do livro mostra um efeito tão dramático do moment matching (erro quadrático médio de cerca de 3,7 para cerca de 0,001): ele está consertando o bug.

Laboratório 4 — O viés previsto e o observado

Usamos os quatro painéis de Medvedev e Scaillet (2010) do livro, $\theta_v = 0{,}02$, $S_0 = 100$, $r = 5\%$, 50 passos por ano, e simulamos o índice com `v[t-1]` (certo) e com `v[t]` (livro).

```
PAINEIS = [(0.01, 1.50, 0.15, 0.10),     # (v0, κ_v, σ_v, ρ)
           (0.04, 0.75, 0.30, 0.10),
           (0.04, 1.50, 0.30, 0.10),
           (0.04, 1.50, 0.15, -0.50)]
THETA_V, S0, r = 0.02, 100.0, 0.05
STRIKES = np.array([90.0, 100.0, 110.0])
PRAZOS = [1 / 12, 1.0, 2.0]


def heston_mc(painel, T, M0, I, rng, esquema="FT", adiante=False, mm=False, anti=True):
    """Simula S_T no Heston. adiante=True reproduz o v[t] do livro; mm=True aplica
    o moment matching dos normais e a correção de drift do livro."""
    v0, kappa, sigma, rho = painel
    M = max(int(round(M0 * T)), 1)
    dt = T / M
    if anti:
        Z = rng.standard_normal((2, M, I // 2))
        Z = np.concatenate([Z, -Z], axis=2)
    else:
        Z = rng.standard_normal((2, M, I))
    if mm:
        Z = (Z - Z.mean()) / Z.std()
    v = np.full(I, v0); vh = v.copy(); x = np.zeros(I)
    for t in range(M):
        z1 = Z[0, t]
        z2 = rho * z1 + np.sqrt(1 - rho**2) * Z[1, t]
        v_novo, vh = passo_v(esquema, vh, kappa, THETA_V, sigma, dt, z2)
        vu = v_novo if adiante else v
        choque = np.sqrt(vu * dt) * z1
        if mm:
```

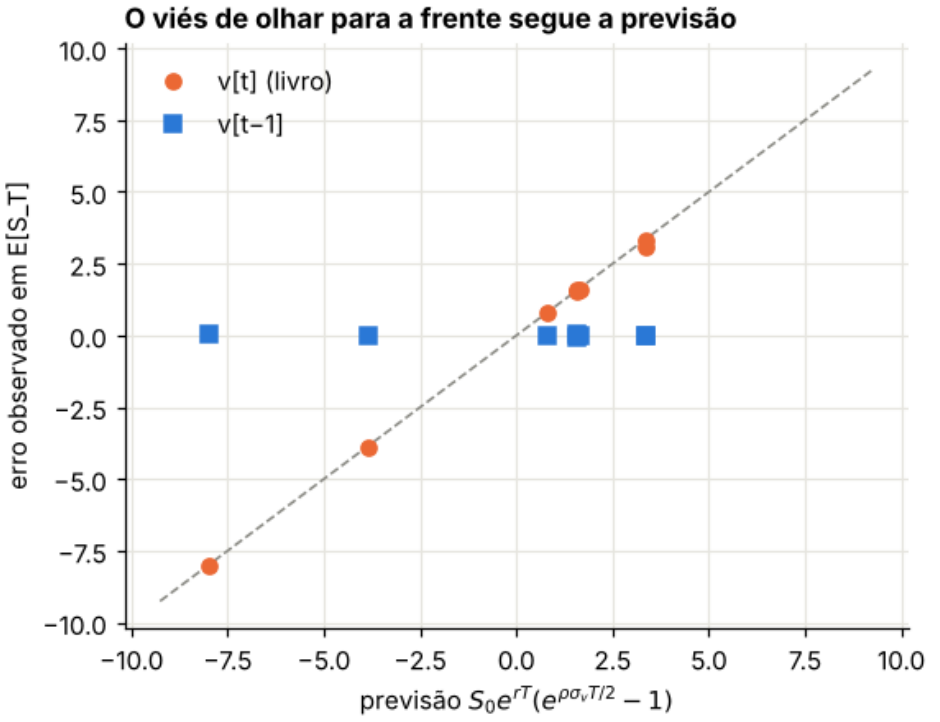
```
            choque = choque - choque.mean()
        x += (r - 0.5 * vu) * dt + choque
        v = v_novo
    return S0 * np.exp(x)


rng = np.random.default_rng(5)
linhas = []
for i, p in enumerate(PAINEIS):
    for T in (1.0, 2.0):
        fwd = S0 * np.exp(r * T)
        prev = fwd * (np.exp(p[3] * p[2] * T / 2) - 1)
        e_certo = heston_mc(p, T, 50, 100_000, rng).mean() - fwd
        e_livro = heston_mc(p, T, 50, 100_000, rng, adiante=True).mean() - fwd
        linhas.append((i + 1, T, p[3] * p[2], prev, e_livro, e_certo))
tab4 = pd.DataFrame(linhas, columns=["painel", "T", "ρσ_v", "erro previsto de E[S_T]",
                                     "erro com v[t] (livro)", "erro com v[t−1]"])
tabela(tab4)
```

painel	T	$\rho\sigma$_v	erro previsto de E[S_T]	erro com v[t] (livro)	erro com v[t−1]
1	1,000	0,015	0,791	0,793	0,008
1	2,000	0,015	1,670	1,579	-0,011
2	1,000	0,030	1,589	1,555	-0,068
2	2,000	0,030	3,366	3,101	-0,006
3	1,000	0,030	1,589	1,601	0,065
3	2,000	0,030	3,366	3,289	0,017
4	1,000	-0,075	-3,869	-3,919	0,014
4	2,000	-0,075	-7,986	-8,049	0,021

```
fig, ax = plt.subplots(figsize=(5.6, 4.2))
lim = 1.15 * np.abs(tab4[["erro previsto de E[S_T]", "erro com v[t] (livro)"]]).values.max()
ax.plot([-lim, lim], [-lim, lim], "--", color=CINZA, lw=1)
ax.plot(tab4["erro previsto de E[S_T]"], tab4["erro com v[t] (livro)"], "o", color=LARANJA,
 ↪  label="v[t] (livro)")
ax.plot(tab4["erro previsto de E[S_T]"], tab4["erro com v[t−1]"], "s", color=AZUL,
 ↪  label="v[t−1]")
ax.set_xlabel(r"previsão $S_0e^{rT}(e^{\rho\sigma_vT/2}-1)$")
ax.set_ylabel("erro observado em E[S_T]")
ax.set_title("O viés de olhar para a frente segue a previsão")
ax.legend()
plt.show()

rel = tab4["erro com v[t] (livro)"] / tab4["erro previsto de E[S_T]"]
ok(np.all((rel > 0.75) & (rel < 1.25)), "o erro com v[t] fica a até 25% da previsão ½ρσ_vT em
 ↪  todos os casos")
ok(np.all(np.abs(tab4["erro com v[t−1]"]) < 0.25), "com v[t−1], o erro de E[S_T] é pequeno (menos
 ↪  de 25 centavos)")
```




```
OK — o erro com v[t] fica a até 25% da previsão ½ρσ_vT em todos os casos
OK — com v[t−1], o erro de E[S_T] é pequeno (menos de 25 centavos)
```

O viés com `v[t]` chega a 8 unidades num índice de 100 (painel 4, $T = 2$), e a previsão de primeira ordem o explica bem. Com `v[t-1]` sobra algo de poucos centavos, vindo da truncação e do ruído.

Há uma razão estrutural para o esquema certo acertar a média: com v_s conhecido no início do passo e z_t^1 independente dele,

$$\mathbb{E}\big[S_t \,\big|\, \mathcal{F}_s\big] = S_s\, e^{(r-v_s/2)\Delta t}\, \mathbb{E}\big[e^{\sqrt{v_s \Delta t}\, z^1}\big] = S_s\, e^{(r-v_s/2)\Delta t}\, e^{v_s \Delta t/2} = S_s\, e^{r\Delta t}. \tag{12.22}$$

O preço descontado simulado é uma martingal **exata**, qualquer que seja Δt e qualquer que seja o esquema da variância.

---

5. O teste de Medvedev e Scaillet

Livro: §10.3 (Tabelas 10.1 e 10.2, Figura 10.4)

O livro avalia calls e puts europeias nos quatro painéis, com $T \in \{1/12, 1, 2\}$ e $K \in \{90, 100, 110\}$: 36 opções de cada tipo. A referência é o preço de Fourier (aqui, Lewis). Um preço "erra" se viola os dois critérios de acurácia ao mesmo tempo.

Laboratório 5 — Reproduzir a Tabela 10.1 e ver o que muda

Rodamos 3 vezes cada configuração, com $I = 50.000$ trajetórias (antitéticas) e 50 passos por ano, sobre as 72 opções (calls e puts das mesmas trajetórias): 216 avaliações por linha. As puts usam o payoff simulado, não a paridade, como no livro.

```
REFS = {}
for i, p in enumerate(PAINEIS):
    for T in PRAZOS:
        C = call_heston(STRIKES, T, r, p[0], p[1], THETA_V, p[2], p[3])
        REFS[i, T] = (C, C - S0 + STRIKES * np.exp(-r * T))


def teste_ms(rng, rodadas=3, I=50_000, M0=50, **kw):
    erros, refs = [], []
    for _ in range(rodadas):
        for i, p in enumerate(PAINEIS):
            for T in PRAZOS:
                S = heston_mc(p, T, M0, I, rng, **kw)
                df = np.exp(-r * T)
                C = df * np.maximum(S[:, None] - STRIKES, 0).mean(0)
                P = df * np.maximum(STRIKES - S[:, None], 0).mean(0)
                erros += list(C - REFS[i, T][0]) + list(P - REFS[i, T][1])
                refs += list(REFS[i, T][0]) + list(REFS[i, T][1])
    erros, refs = np.array(erros), np.array(refs)
    return erros, refs

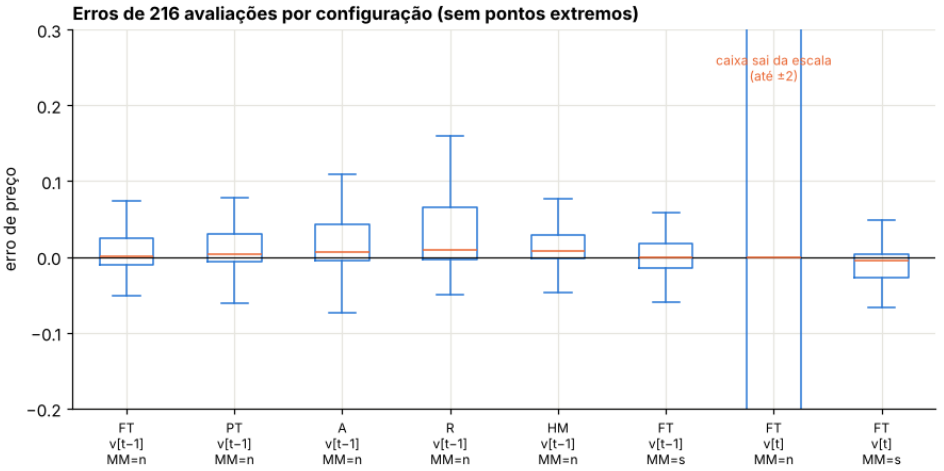

configs = [("FT", "v[t-1]", False, dict(esquema="FT")),
           ("PT", "v[t-1]", False, dict(esquema="PT")),
           ("A",  "v[t-1]", False, dict(esquema="A")),
           ("R",  "v[t-1]", False, dict(esquema="R")),
           ("HM", "v[t-1]", False, dict(esquema="HM")),
           ("FT", "v[t-1]", True,  dict(esquema="FT", mm=True)),
           ("FT", "v[t] (livro)", False, dict(esquema="FT", adiante=True)),
           ("FT", "v[t] (livro)", True,  dict(esquema="FT", adiante=True, mm=True))]
rng = np.random.default_rng(250000)
t0 = time.time()
linhas, caixas = [], {}
for esq, idx, mm, kw in configs:
    e, ref = teste_ms(rng, **kw)
    n_err = int((~acerto(e, ref)).sum())
    linhas.append((esq, idx, "sim" if mm else "não", n_err, len(e), e.mean(), np.mean(e**2)))
    caixas[f"{esq}\n{idx.split()[0]}\nMM={'s' if mm else 'n'}"] = e
tab5 = pd.DataFrame(linhas, columns=["variância", "índice usa", "moment matching",
                                     "erros", "opções", "erro médio", "EQM"])
tabela(tab5)
print(f"tempo: {time.time() - t0:.1f} s")
```

variância	índice usa	moment matching	erros	opções	erro médio	EQM
FT	v[t−1]	não	9	216	0,007	0,002
PT	v[t−1]	não	12	216	0,012	0,002
A	v[t−1]	não	23	216	0,022	0,003
R	v[t−1]	não	42	216	0,046	0,010
HM	v[t−1]	não	21	216	0,025	0,004
FT	v[t−1]	sim	4	216	0,003	0,001
FT	v[t] (livro)	não	174	216	0,036	2,286
FT	v[t] (livro)	sim	5	216	-0,011	0,001

```
tempo: 64.7 s
```

```
fig, ax = plt.subplots(figsize=(9.5, 4.2))
ax.boxplot(list(caixas.values()), tick_labels=list(caixas.keys()), showfliers=False,
           medianprops=dict(color=LARANJA), boxprops=dict(color=AZUL),
 ↪  whiskerprops=dict(color=AZUL),
           capprops=dict(color=AZUL))
ax.axhline(0, color="black", lw=0.8)
ax.set_ylim(-0.2, 0.3)
ax.annotate("caixa sai da escala\n(até ±2)", xy=(7, 0.27), ha="center", va="top", fontsize=8,
 ↪  color=LARANJA)
ax.set_ylabel("erro de preço")
ax.set_title("Erros de 216 avaliações por configuração (sem pontos extremos)")
ax.tick_params(axis="x", labelsize=8)
plt.show()

eqm = tab5.set_index(["variância", "índice usa", "moment matching"])["EQM"]
erros = tab5.set_index(["variância", "índice usa", "moment matching"])["erros"]
ok(eqm["FT", "v[t] (livro)", "não"] > 1.0, "o código do livro sem moment matching tem EQM acima
 ↪  de 1, como na Tabela 10.1")
ok(erros["FT", "v[t] (livro)", "não"] > 0.6 * 216, "e erra a maioria das opções")
ok(eqm["FT", "v[t-1]", "não"] < 0.01, "com v[t-1], o Full Truncation acerta sem nenhuma correção
 ↪  (EQM < 0,01)")
ok(erros["FT", "v[t-1]", "não"] <= 0.05 * 216, "e erra no máximo 5% das opções")
ok(eqm["FT", "v[t] (livro)", "sim"] < 0.01, "o moment matching conserta o código do livro")
ok(eqm["R", "v[t-1]", "não"] > eqm["FT", "v[t-1]", "não"], "o Reflection é pior que o Full
 ↪  Truncation")
```




```
OK — o código do livro sem moment matching tem EQM acima de 1, como na Tabela 10.1
OK — e erra a maioria das opções
OK — com v[t-1], o Full Truncation acerta sem nenhuma correção (EQM < 0,01)
OK — e erra no máximo 5% das opções
OK — o moment matching conserta o código do livro
OK — o Reflection é pior que o Full Truncation
```

O que aprendemos.

- A linha “v[t] (livro), sem moment matching” reproduz a Tabela 10.1: a grande maioria das opções erradas e EQM acima de 2, que é a média entre calls e puts

(o livro relata cerca de 3,7 para as calls e 0,9 para as puts).

- Com `v[t-1]`, o Full Truncation sem correção nenhuma já fica perto do nível que o livro só atinge com moment matching. O "efeito impressionante" da Figura 10.4 é, quase todo, o conserto do bug.
- Entre os esquemas, Full Truncation e Partial Truncation são os melhores; Higham-Mao e Absorption ficam atrás; Reflection é o pior. Todos erram para cima. Refletir ou absorver no zero muda a variância de forma sistemática: refletir transforma um valor negativo num positivo do mesmo tamanho, o que infla a variância média. É o resultado de Lord et al. (2006), que o livro cita: o Full Truncation é o vencedor.

---

6. Redução de variância

Livro: §10.3 e §10.4 (equação 10.8)

O viés se combate com esquemas melhores e mais passos. O **ruído** se combate com mais trajetórias, cujo custo é alto (o erro-padrão cai como $1/\sqrt{I}$), ou com técnicas de redução de variância.

6.1 Variáveis antitéticas

Para cada vetor de choques z, simulamos também $-z$. O estimador de um par é $\bar{Y} = (Y + Y')/2$, e

$$\mathrm{Var}(\bar{Y}) = \frac{\sigma_Y^2}{2}(1 + \mathrm{Corr}(Y, Y')). \tag{12.23}$$

Comparado com duas trajetórias independentes ($\sigma_Y^2/2$), o ganho depende de $\mathrm{Corr}(Y, Y') < 0$. Para payoffs **monótonos** no choque (uma call bem dentro do dinheiro) a correlação é muito negativa e o ganho é grande; para payoffs simétricos, como um straddle, pode não haver ganho nenhum.

6.2 Moment matching

Depois de gerar os normais, impomos média amostral 0 e desvio amostral 1: $z \leftarrow (z - \bar{z})/s_z$. O livro faz isso sobre o array inteiro, e ainda subtrai, a cada passo, a média amostral do choque do índice. Essa última correção força a média $\frac{1}{I}\sum_i S_{t,i}$ a ser igual à média de $S_s e^{r\Delta t}$, uma ideia conhecida como **martingal empírica** (Duan e Simonato, 1998).

Duas observações. As trajetórias deixam de ser independentes, e o erro-padrão usual deixa de valer. E, como vimos, essa correção também apaga erros de modelo, como o da Seção 4.3, o que é útil, mas esconde o problema.

6.3 Variáveis de controle

Seja Y o payoff descontado que queremos e X outro, simulado nas mesmas trajetórias, cuja média $\mathbb{E}[X]$ **conhecemos**. O estimador

$$\hat{Y}_{CV} = \frac{1}{I}\sum_{i}\left(Y_i - \lambda(X_i - \mathbb{E}[X])\right) \tag{12.24}$$

é não viesado para qualquer λ. Sua variância é $\sigma_Y^2 - 2\lambda\,\mathrm{Cov}(X,Y) + \lambda^2\sigma_X^2$, mínima em

$$\lambda^* = \frac{\mathrm{Cov}(X,Y)}{\mathrm{Var}(X)}, \qquad \mathrm{Var}(\hat{Y}_{CV}) = \frac{\sigma_Y^2}{I}(1 - \mathrm{Corr}(X,Y)^2). \tag{12.25}$$

Com correlação de 0,99, a variância cai 50 vezes: o mesmo que 50 vezes mais trajetórias. O livro usa a put europeia como controle da americana (10.8), com $\lambda = 1$.

Laboratório 6 — Quanto cada técnica reduz o ruído

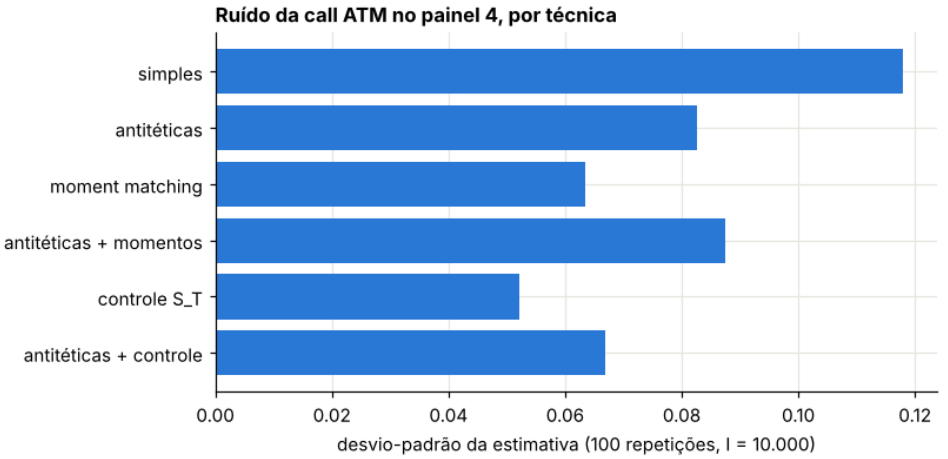
Para a call de $K = 100$, $T = 1$, painel 4, repetimos 100 vezes cada configuração com $I = 10.000$ e medimos o desvio-padrão das estimativas. Como controle, usamos o próprio $e^{-rT}S_T$, cuja média é S_0 exatamente (a martingal da Seção 4.3), com λ^* estimado na amostra.

```
p, T, Kc, I, rep = PAINEIS[3], 1.0, 100.0, 10_000, 100
ref_c = REFS[3, 1.0][0][1]
df = np.exp(-r * T)
rng = np.random.default_rng(6)
resultado = {}
for nome, kw, cv in [("simples", dict(anti=False), False),
                     ("antitéticas", dict(anti=True), False),
                     ("moment matching", dict(anti=False, mm=True), False),
                     ("antitéticas + momentos", dict(anti=True, mm=True), False),
                     ("controle S_T", dict(anti=False), True),
                     ("antitéticas + controle", dict(anti=True), True)]:
    est = []
    for _ in range(rep):
        S = heston_mc(p, T, 50, I, rng, **kw)
        Y = df * np.maximum(S - Kc, 0)
        if cv:
            X = df * S
            lam = np.cov(X, Y)[0, 1] / X.var()
            Y = Y - lam * (X - S0)
        est.append(Y.mean())
    est = np.array(est)
    resultado[nome] = (est.mean() - ref_c, est.std(ddof=1))
tab6 = pd.DataFrame([(k, v[0], v[1], (resultado["simples"][1] / v[1]) ** 2) for k, v in
↪  resultado.items()],
                    columns=["técnica", "viés médio", "desvio-padrão", "ganho equivalente em
↪  trajetórias"])
tabela(tab6)
```

técnica	viés médio	desvio-padrão	ganho equivalente em trajetórias
simples	0,002	0,118	1,000
antitéticas	0,003	0,083	2,042
moment matching	0,001	0,063	3,463
antitéticas + momentos	0,016	0,087	1,819
controle S_T	0,009	0,052	5,111
antitéticas + controle	0,005	0,067	3,108

```
fig, ax = plt.subplots(figsize=(7.2, 3.6))
ax.barh(tab6["técnica"][::-1], tab6["desvio-padrão"][::-1], color=AZUL)
ax.set_xlabel("desvio-padrão da estimativa (100 repetições, I = 10.000)")
ax.set_title("Ruído da call ATM no painel 4, por técnica")
plt.show()

g = tab6.set_index("técnica")["ganho equivalente em trajetórias"]
ok(g["antitéticas"] > 1.5, "as antitéticas reduzem a variância de uma call (payoff monótono no
↪  choque)")
ok(g["controle S_T"] > 2.5, "o controle S_T rende mais que 2,5 vezes mais trajetórias")
ok(g["controle S_T"] > g["antitéticas"], "para esta call, o controle supera as antitéticas")
ok(abs(resultado["antitéticas + controle"][0]) < 3 * resultado["antitéticas + controle"][1] /
↪  np.sqrt(rep) + 0.01,
   "o estimador com controle não tem viés detectável")
```




```
OK — as antitéticas reduzem a variância de uma call (payoff monótono no choque)
OK — o controle S_T rende mais que 2,5 vezes mais trajetórias
OK — para esta call, o controle supera as antitéticas
OK — o estimador com controle não tem viés detectável
```

Para esta call, o controle S_T é mais eficaz que as antitéticas, mas os dois **não se somam**: atacam a mesma coisa, a parte do payoff que é linear nos choques. As antitéticas cancelam essa parte por simetria; o controle S_T, por regressão. O moment matching rende algo parecido com as antitéticas, porque também corrige o primeiro momento dos choques. Combinadas, aqui renderam até menos que o controle sozinho. As técnicas só se complementam quando miram partes diferentes do payoff. (Os desvios-padrão de 100 repetições têm cerca de 7% de imprecisão; a ordem entre as linhas próximas pode mudar com outra semente.)

---

7. Americanas no Heston com juros CIR

Livro: §10.4 e §10.6.4 (Tabelas 10.3 e 10.4)

7.1 O modelo e o algoritmo

O modelo agora tem três fatores: S (Heston), v (raiz quadrada) e r (CIR, com $r_0 = 4\%$, $\kappa_r = 0{,}3$, $\theta_r = 4\%$, $\sigma_r = 0{,}1$), com Z^3 independente. O livro avalia puts americanas com $T \in \{1/12, 1/4, 1/2\}$ e $K \in \{90, 100, 110\}$ nos quatro painéis: 36 opções, contra as referências de Medvedev e Scaillet (LSM com 1 milhão de trajetórias e 500 passos).

O algoritmo é o LSM do Módulo 9, com quatro ingredientes:

1. **Índice**: log-Euler com a média trapezoidal dos juros, $\ln S_t = \ln S_s + (\frac{r_s + r_t}{2} - \frac{v}{2})\Delta t + \sqrt{v}\sqrt{\Delta t}\, z_t^1$.
2. **Regressão**: só trajetórias dentro do dinheiro, com 11 funções de base: $1, r, v, S, r^2, v^2, S^2, vr, Sr, Sv, Svr$.
3. **Desconto**: trajetória a trajetória, pela média trapezoidal de r.
4. **Controle**: a put europeia simulada nas mesmas trajetórias, com $\lambda = 1$ em (10.8). Sua média é tomada como o preço de Heston com a taxa constante $\bar{r} = -\ln P(0,T)/T$. Pelo Módulo 11, essa aproximação erra muito pouco em prazos de até seis meses.

Mais um detalhe do código do livro. Na §10.6.4, a matriz de correlação tem ρ na posição (0, 1). A linha 0 é a dos juros, a linha 1 a do índice, a linha 2 a da variância. Ou seja: a correlação ρ é posta entre **juros e índice**, e a variância fica independente do índice. Como o controle europeu vem da fórmula de Heston **com** ρ, o estimador (10.8) "contrabandeia" a correlação de volta: o resultado é aproximadamente americana($\rho = 0$) + [europeia(ρ) − europeia($\rho = 0$)]. Como o prêmio de exercício antecipado é pouco sensível a ρ, o erro passa despercebido no teste.

Laboratório 7 — 36 puts americanas contra Medvedev e Scaillet

```
BENCH_MS = np.array(((0.0001, 1.0438, 9.9950, 0.0346, 1.7379, 9.9823, 0.2040, 2.3951, 9.9726),
                     (0.0619, 2.1306, 10.0386, 0.5303, 3.4173, 10.4271, 1.1824, 4.4249, 11.0224),
                     (0.0592, 2.1138, 10.0372, 0.4950, 3.3478, 10.3825, 1.0752, 4.2732, 10.8964),
                    (0.0787, 2.1277, 10.0198, 0.6012, 3.4089, 10.2512, 1.2896, 4.4103, 10.6988)))
R0, KR, THR, SR = 0.04, 0.3, 0.04, 0.1
PRAZOS_AM = [1 / 12, 0.25, 0.5]


def trajetorias_ft(x0, kappa, theta, sigma, dt, Z):
    x = np.empty((Z.shape[0] + 1, Z.shape[1])); x[0] = x0; xh = np.full(Z.shape[1], x0)
    for t in range(Z.shape[0]):
        x[t + 1], xh = passo_v("FT", xh, kappa, theta, sigma, dt, Z[t])
    return x


def lsm_svsi(painel, T, M, I, rng, controle=True, como_livro=False):
    """Puts americanas (K = 90, 100, 110) no Heston + CIR.
    controle: False (sem), True (λ = 1, como no livro) ou "otimo" (λ* estimado).
    como_livro=True usa v[t] e a correlação entre juros e índice, como na §10.6.4."""
    v0, kv, sv, rho = painel
    dt = T / M
    Z = rng.standard_normal((3, M, I // 2)); Z = np.concatenate([Z, -Z], axis=2)
    zs = Z[0]
    if como_livro:
        zv, zr = Z[1], rho * Z[0] + np.sqrt(1 - rho**2) * Z[2]
    else:
```

```
        zv, zr = rho * Z[0] + np.sqrt(1 - rho**2) * Z[1], Z[2]
    rr = trajetorias_ft(R0, KR, THR, SR, dt, zr)
    v = trajetorias_ft(v0, kv, THETA_V, sv, dt, zv)
    x = np.zeros((M + 1, I))
    for t in range(M):
        vu = v[t + 1] if como_livro else v[t]
        x[t + 1] = x[t] + ((rr[t] + rr[t + 1]) / 2 - vu / 2) * dt + np.sqrt(vu * dt) * zs[t]
    S = S0 * np.exp(x)
    fd = np.exp(-(rr[1:] + rr[:-1]) / 2 * dt)        # fator de desconto de cada passo
    B0T = zcb_cir(R0, KR, THR, SR, T)
    rbar = -np.log(B0T) / T
    precos = []
    for K in STRIKES:
        h = np.maximum(K - S, 0)
        V = h[-1].copy()
        for t in range(M - 1, 0, -1):
            V = V * fd[t]
            itm = h[t] > 0
            if itm.sum() > 20:
                s_, v_, r_ = S[t, itm], v[t, itm], rr[t, itm]
                X = np.column_stack([np.ones_like(s_), r_, v_, s_, r_**2, v_**2, s_**2,
                                     v_ * r_, s_ * r_, s_ * v_, s_ * v_ * r_])
                cont = X @ np.linalg.lstsq(X, V[itm], rcond=None)[0]
                idx = np.where(itm)[0][h[t, itm] > cont]
                V[idx] = h[t, idx]
        Y = V * fd[0]
        if controle:
            P_eur = call_heston(K, T, rbar, v0, kv, THETA_V, sv, rho)[0] - S0 + K * B0T
            X_eur = np.prod(fd, axis=0) * h[-1]
            lam = 1.0 if controle is True else np.cov(X_eur, Y)[0, 1] / X_eur.var()
            Y = Y - lam * (X_eur - P_eur)
        precos.append(max(Y.mean(), K - S0))           # exercício em t = 0
    return np.array(precos)


def teste_americanas(rng, **kw):
    out = np.zeros((4, 9))
    for i, p in enumerate(PAINEIS):
        for j, T in enumerate(PRAZOS_AM):
            out[i, 3 * j:3 * j + 3] = lsm_svsi(p, T, 25, 35_000, rng, **kw)
    return out


rng = np.random.default_rng(250000)
t0 = time.time()
res_cv = teste_americanas(rng)
t_cv = time.time() - t0
res_sem = teste_americanas(rng, controle=False)
res_livro = teste_americanas(rng, como_livro=True)
res_livro_sem = teste_americanas(rng, como_livro=True, controle=False)

linhas = []
for nome, res in [("certo, com controle", res_cv), ("certo, sem controle", res_sem),
                  ("como no livro, com controle", res_livro), ("como no livro, sem controle",
 ↪  res_livro_sem)]:
    e = res - BENCH_MS
    linhas.append((nome, int((~acerto(e, BENCH_MS)).sum()), 36, e.mean(), np.abs(e).mean(),
 ↪  np.mean(e**2)))
tab7 = pd.DataFrame(linhas, columns=["configuração", "erros", "opções", "erro médio", "erro
 ↪  absoluto médio", "EQM"])
```

```
tabela(tab7)
print(f"tempo com controle: {t_cv:.1f} s para 36 opções ({t_cv / 36:.3f} s por opção)")

# ruído: 12 repetições das três puts do painel 4 com T = 1/2
dp = {}
for nome, c in [("sem controle", False), ("λ = 1 (livro)", True), ("λ* estimado", "otimo")]:
    a = np.array([lsm_svsi(PAINEIS[3], 0.5, 25, 35_000, rng, controle=c) for _ in range(12)])
    dp[nome] = a.std(axis=0, ddof=1)
tab7b = pd.DataFrame(dp, index=[f"K = {k:.0f}" for k in STRIKES]).T
print("desvio-padrão em 12 repetições (painel 4, T = 1/2):")
tabela(tab7b)
```

configuração	erros	opções	erro médio	erro absoluto médio	EQM
certo, com controle	0	36	-0,006	0,014	0,000
certo, sem controle	0	36	0,002	0,014	0,000
como no livro, com controle	0	36	0,004	0,013	0,000
como no livro, sem controle	9	36	-0,009	0,042	0,004

```
tempo com controle: 3.8 s para 36 opções (0.104 s por opção)
```

```
desvio-padrão em 12 repetições (painel 4, T = 1/2):
```

	K = 90	K = 100	K = 110
sem controle	0,013	0,023	0,029
$\lambda = 1$ (livro)	0,008	0,029	0,031
λ^* estimado	0,008	0,015	0,023

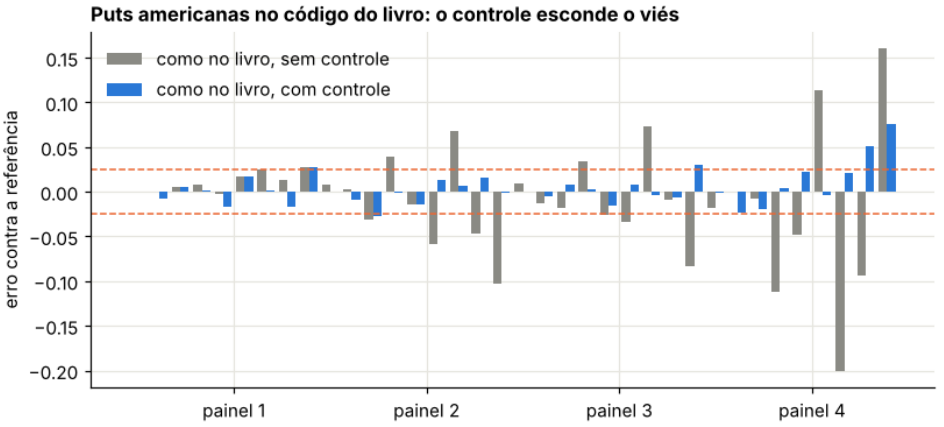
```
det = pd.DataFrame({"painel": np.repeat([1, 2, 3, 4], 9),
                    "T": np.tile(np.repeat(["1/12", "1/4", "1/2"], 3), 4),
                    "K": np.tile(STRIKES, 12),
                    "Medvedev-Scaillet": BENCH_MS.ravel(),
                    "LSM com controle": res_cv.ravel(),
                    "erro": (res_cv - BENCH_MS).ravel()})
tabela(det[det["painel"] == 4])

fig, ax = plt.subplots(figsize=(8.5, 3.6))
xs = np.arange(36)
ax.bar(xs - 0.2, (res_livro_sem - BENCH_MS).ravel(), 0.4, color=CINZA, label="como no livro, sem
↪ controle")
ax.bar(xs + 0.2, (res_livro - BENCH_MS).ravel(), 0.4, color=AZUL, label="como no livro, com
↪ controle")
ax.axhline(0.025, ls="--", color=LARANJA, lw=1); ax.axhline(-0.025, ls="--", color=LARANJA, lw=1)
ax.set_xticks(np.arange(4.5, 36, 9), [f"painel {i}" for i in range(1, 5)])
ax.set_ylabel("erro contra a referência")
ax.set_title("Puts americanas no código do livro: o controle esconde o viés")
ax.legend()
plt.show()

err = tab7.set_index("configuração")
ok(err.loc["certo, com controle", "erros"] <= 3, "com controle, no máximo 3 das 36 puts ficam
↪ fora do critério")
ok(err.loc["certo, sem controle", "erros"] <= 3, "com o índice certo, mesmo sem controle o teste
↪ passa")
```

```
ok(tab7b.loc["λ = 1 (livro)", "K = 90"] < 0.7 * tab7b.loc["sem controle", "K = 90"],
   "na put fora do dinheiro, o controle com λ = 1 reduz o ruído em mais de 30%")
ok(err.loc["como no livro, sem controle", "EQM"] > 10 * err.loc["certo, sem controle", "EQM"],
   "sem controle, o código do livro erra muito mais")
ok(err.loc["como no livro, com controle", "erros"] <= 6,
   "com controle, o código do livro passa no teste: o controle esconde os dois desvios")
```

	painel	T	K	Medvedev-Scaillet	LSM com controle	erro
27	4	1/12	90,000	0,079	0,079	0,000
28	4	1/12	100,000	2,128	2,132	0,004
29	4	1/12	110,000	10,020	10,000	-0,020
30	4	1/4	90,000	0,601	0,605	0,004
31	4	1/4	100,000	3,409	3,391	-0,018
32	4	1/4	110,000	10,251	10,234	-0,017
33	4	1/2	90,000	1,290	1,287	-0,002
34	4	1/2	100,000	4,410	4,369	-0,041
35	4	1/2	110,000	10,699	10,620	-0,079




```
OK — com controle, no máximo 3 das 36 puts ficam fora do critério
OK — com o índice certo, mesmo sem controle o teste passa
OK — na put fora do dinheiro, o controle com λ = 1 reduz o ruído em mais de 30%
OK — sem controle, o código do livro erra muito mais
OK — com controle, o código do livro passa no teste: o controle esconde os dois desvios
```

O que aprendemos.

- Com o índice certo, o LSM acerta as 36 puts com ou sem controle. Não é o controle que faz o teste passar.
- O ganho de ruído do controle depende da opção. Na put fora do dinheiro ele é claro. Dentro do dinheiro, muitas trajetórias são exercidas cedo, e o payoff americano fica pouco correlacionado com o europeu no vencimento; com $\lambda = 1$, o controle pode até **aumentar** o ruído, porque $\mathrm{Var}(Y - X) = \sigma_Y^2 + \sigma_X^2 - 2\,\mathrm{Cov}$ supera σ_Y^2 quando $\mathrm{Cov} < \sigma_X^2/2$. O λ^* estimado protege contra isso dentro de cada rodada (com 12 repetições, os desvios-padrão da tabela ainda têm uns 20% de imprecisão).
- No código do livro, sem controle, nove das 36 puts saem do critério e o EQM é dez vezes maior: é o viés de `v[t]` de novo, menor porque os prazos são curtos.

A Tabela 10.4 conclui que o controle é "de importância primordial". Em boa parte, isso acontece porque o controle cancela esse viés: americana e europeia são simuladas nas mesmas trajetórias e erram quase igual.

- Lição geral: um controle absorve qualquer erro que afete a americana e a europeia da mesma forma. Um teste que só compara preços com controle não diagnostica esses erros; o teste europeu da Seção 5 diagnostica.

---

8. Síntese

Fonte de erro	Como aparece	Como reduzir
Discretização (viés fraco)	cai como Δt	mais passos; esquema melhor; transição exata quando existe
Tratamento do zero na variância	viés que depende do esquema	Full Truncation (ou esquemas como o QE de Andersen)
Erro de modelo no código (`v[t]`)	viés que **não** cai com Δt	avaliar coeficientes no início do passo; conferir a martingal $e^{-rT}\mathbb{E}[S_T] = S_0$
Ruído estatístico	cai como $1/\sqrt{I}$	antitéticas; moment matching; variáveis de controle
Referência errada	erros aparentes	referência mais exata que a diferença medida

A conferência mais barata e mais útil de qualquer simulação de preços é a da martingal: o preço descontado do ativo simulado deve ter média S_0. Ela teria apontado o problema da Seção 4.3 em uma linha.

9. Erratas e notas sobre o capítulo 10

Onde	O quê
§10.6.3 e §10.6.4, processo do índice	usa `v[t]` (fim do passo) em vez de `v[t-1]`; cria o drift espúrio $\frac{1}{2}\rho\sigma_v$ por unidade de tempo em $\ln S$, que não diminui com Δt
Tabela 10.1 e Figura 10.4	o salto de precisão atribuído ao moment matching é, em sua maior parte, a correção de drift (`bias`) desfazendo o problema acima; com `v[t-1]` não há salto
§10.3, lista de esquemas	"Truncation" coincide com "Absorption" e "Simple Reflection" com "Reflection": são cinco esquemas, não sete

Onde	O quê
§10.2, Figuras 10.2 e 10.3	a referência em $t>0$ é $P(\mathbb{E}[r_t])$, mas a média de Monte Carlo estima $\mathbb{E}[P(r_t)]$; a diferença (Jensen) é da ordem do viés do Euler que se quer mostrar
§10.6.2, esquema exato	`np.ramdom.chisquare` (erro de digitação)
§10.6.4, correlação	ρ posto entre juros e índice, e não entre variância e índice; o controle europeu disfarça o efeito
§10.4, controle	a média do controle usa Heston com taxa constante $\bar{r}$, uma aproximação (Módulo 11), pequena para prazos de até seis meses
§10.2	a transição por qui-quadrado não central é clássica (Feller, 1951; Cox, Ingersoll e Ross, 1985); Broadie e Kaya (2006) a usam na simulação exata do Heston
Scripts	Python 2 (`xrange`, `print` sem parênteses; `I / 2` vira real no Python 3)

10. Exercícios

Exercício 1. Mostre que, no esquema ingênuo (10.7), $\mathbb{E}[S_t \mid S_s] = S_s e^{r\Delta t}$, mas S_t pode ser negativo. Qual a probabilidade de um passo ficar negativo com $\sigma = 0{,}3$ e $\Delta t = 1/50$?

Solução

A média sai de $\mathbb{E}[z]=0$. O passo fica negativo se $z < -e^{r\Delta t}/(\sigma\sqrt{\Delta t}) \approx -1/(0{,}3\cdot 0{,}141) \approx -23{,}6$: probabilidade da ordem de 10^{-123}, desprezível. O problema do esquema não é o sinal, é a forma da distribuição.

Exercício 2. Refaça o Laboratório 4 com 100 e 200 passos por ano. O que acontece com o erro de `v[t]`? E com o de `v[t-1]`?

Solução

O erro de `v[t]` praticamente não muda: é $\frac{1}{2}\rho\sigma_v T$, independente de Δt. O de `v[t-1]`, que vem da truncação e do ruído, diminui ou fica no nível do ruído. Essa é a forma prática de distinguir erro de modelo de erro de discretização.

Exercício 3. Prove a fórmula da variância das antitéticas e mostre que, para $Y=(S_T-K)^2$ num mundo BSM com $K = S_0 e^{rT}$ e $\sigma\sqrt{T}$ pequeno, quase não há ganho.

Solução

$\mathrm{Var}((Y+Y')/2) = \frac{1}{4}(\sigma^2+\sigma^2+2\mathrm{Cov}) = \frac{\sigma^2}{2}(1+\mathrm{Corr})$. Com $\sigma\sqrt{T}$ pequeno, $S_T - K \approx K\sigma\sqrt{T}z$, e $Y \approx K^2\sigma^2 T z^2$, função par de z: $Y' = Y$, correlação 1, ganho nenhum

(o par custa o dobro e rende o mesmo).

Exercício 4. No Laboratório 6, troque o controle S_T pela call europeia de BSM com $\sigma = \sqrt{\theta_v}$, simulada com os mesmos choques z^1 e volatilidade constante. O ganho é maior ou menor?

Solução

Depende do painel. A call BSM tem forma de payoff idêntica, o que aumenta a correlação nos estados em que a call paga; mas a variância simulada no Heston difere de θ_v, o que a reduz. Nos painéis com σ_v pequeno, o ganho tende a superar o do controle S_T. Meça e compare $\mathrm{Corr}(X, Y)^2$.

Exercício 5 (desafio). Implemente o esquema QE de Andersen (2008) para a variância e compare com o Full Truncation no teste da Seção 5 com 10 passos por ano.

Solução

O QE casa os dois primeiros momentos condicionais da Seção 2.2: se $\psi = \mathrm{Var}/\mathrm{média}^2 \leq 1{,}5$, usa $v_t = a(b + z)^2$; senão, uma mistura de massa em zero com uma exponencial. Com 10 passos por ano, o QE costuma errar bem menos que o Full Truncation, porque respeita a distribuição de v mesmo com passos longos.

```
print(f"Módulo 12 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 12 concluído: 32 verificações aprovadas.
```

Capítulo 13

Calibração I: a curva de juros no CIR e as opções no Heston

Introdução

Calibrar é escolher os parâmetros de um modelo para que seus preços reproduzam os preços observados no mercado. Este capítulo trata os dois blocos mais importantes do BCC97 separadamente.

O primeiro é a **curva de juros**. No modelo CIR, a curva de juros gerada pelo modelo depende de quatro parâmetros, e o primeiro exercício é ajustá-la a uma curva de mercado, entendendo o que o modelo consegue e não consegue reproduzir. O segundo bloco são as **opções**: o modelo de Heston é calibrado a uma superfície de preços de opções por mínimos quadrados, usando a fórmula de Lewis.

A calibração parece um simples problema de otimização, mas não é. A função objetivo tem vales longos, nos quais parâmetros bem diferentes produzem quase o mesmo ajuste; restrições como a condição de Feller ficam ativas; e o otimizador pode parar em mínimos locais. O capítulo mostra esses fenômenos com dados sintéticos, nos quais os parâmetros verdadeiros são conhecidos, e discute o que a calibração consegue e não consegue identificar.

Fundamentos teóricos

Curva forward e ajuste do CIR

Definição 13.1 (Taxas forward). Para preços de títulos $B(0,T)$, a *taxa spot* contínua é $y(T) = -\ln B(0,T)/T$ e a *taxa forward instantânea* é $f(0,T) = -\partial_T \ln B(0,T)$. Então $B(0,T) = \exp\big(-\int_0^T f(0,s)\,ds\big)$.

Proposição 13.2 (Curva forward do CIR). *Na Proposição 11.3,* $f(0,T) = -\partial_T \ln A(T) + r_0\,\partial_T B(T)$. *Com* $\gamma = \sqrt{\kappa^2 + 2\sigma^2}$ *e* $D(T) = (\kappa+\gamma)(e^{\gamma T}-1) + 2\gamma$,

$$f^{\mathrm{CIR}}(0,T) = \frac{2\kappa\theta\,(e^{\gamma T}-1)}{D(T)} + r_0\,\frac{4\gamma^2 e^{\gamma T}}{D(T)^2}. \tag{13.1}$$

Demonstração. Derive $\ln A = \frac{2\kappa\theta}{\sigma^2}\big[\ln(2\gamma) + \frac{(\kappa+\gamma)T}{2} - \ln D\big]$ e $B = 2(e^{\gamma T}-1)/D$. Usando

$D' = (\kappa+\gamma)\gamma e^{\gamma T}$ e $\gamma^2 - \kappa^2 = 2\sigma^2$, o primeiro termo simplifica para $2\kappa\theta(e^{\gamma T}-1)/D$; o segundo dá $B' = 4\gamma^2 e^{\gamma T}/D^2$. ■

Observação 13.3. No limite $T \to \infty$, $f^{\mathrm{CIR}}(0,T) \to 2\kappa\theta/(\kappa+\gamma)$, menor que θ: a convexidade dos juros puxa a taxa forward longa para baixo da média de longo prazo da short rate. O fator 2 em (13.1) é essencial; sem ele, a curva ajustada exige um θ duas vezes maior.

Calibração de Heston

Definição 13.4 (Problema de calibração). Para $\vartheta = (\kappa_v, \theta_v, \sigma_v, \rho, v_0)$, minimize

$$\mathrm{EQM}(\vartheta) = \frac{1}{m}\sum_{j=1}^{m}\left(C_j^{\mathrm{H}}(\vartheta) - C_j^{\mathrm{mkt}}\right)^2 \tag{13.2}$$

sobre $\Theta = \{\kappa_v, \theta_v, \sigma_v, v_0 > 0,\ -1 < \rho < 1\}$, possivelmente com a restrição de Feller $2\kappa_v\theta_v \geq \sigma_v^2$.

Observação 13.5 (Papel de cada parâmetro). Num prazo fixo, v_0 e θ_v controlam o nível das vols implícitas, ρ o skew e σ_v a convexidade. A velocidade κ_v controla como skew e convexidade se dissipam com o prazo. Por isso uma calibração a um único vencimento identifica mal κ_v e θ_v: só o produto e a combinação com v_0 são bem determinados, e a superfície de erro tem um vale longo.

Proposição 13.6 (Variância esperada). *Em Heston,* $\mathbb{E}\big[\frac{1}{T}\int_0^T v_t\,dt\big] = \theta_v + (v_0 - \theta_v)\frac{1-e^{-\kappa_v T}}{\kappa_v T}$.

A proposição explica o vale: diferentes pares (κ_v, θ_v) produzem a mesma variância média até o prazo calibrado, e portanto preços ATM muito próximos. Ela também mostra o risco: fora do prazo calibrado, as previsões divergem (Capítulo 14).

Otimização

Observação 13.7 (Estratégia em duas etapas). O EQM é não convexo e, em geral, tem mínimos locais. A abordagem usual combina uma busca global grosseira (grade ou força bruta) com um refinamento local sem derivadas (Nelder-Mead) ou com gradiente (quase-Newton, Levenberg-Marquardt). Restrições de sinal e de Feller entram como barreiras ou penalidades; uma restrição ativa na solução sinaliza que o mercado "pede" um processo fora do domínio permitido.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulo 11 do livro, §11.1 a §11.4.3 (a calibração dos saltos e do BCC97 completo fica para o Módulo 14).

O que você vai conseguir fazer ao final

1. Explicar por que, num mercado incompleto, o preço depende de uma escolha de medida, e por que a calibração faz essa escolha.

2. Comparar as funções objetivo usuais e saber o que cada uma pondera.
3. Transformar taxas de mercado monetário em taxas contínuas e a termo, e calibrar o CIR à curva.
4. Deduzir a taxa forward do CIR e encontrar um erro na fórmula do livro.
5. Calibrar o Heston a 15 opções com a estratégia do livro: busca em grade e depois busca local.
6. Diagnosticar quais parâmetros os dados de fato identificam, e o que a condição de Feller faz com o ajuste.

Onde estamos

Os Módulos 7 a 11 entregaram preços a partir de parâmetros. Calibrar é o caminho inverso: partir dos preços de mercado e chegar aos parâmetros. O livro usa dados reais (Euribor e opções do EURO STOXX 50 de 30/09/2014). Aqui, como em todo o curso, usamos **dados sintéticos** com a mesma estrutura: 9 taxas de 1 dia a 1 ano e 15 calls em 3 vencimentos e 5 strikes. A vantagem é poder comparar o calibrado com o verdadeiro, o que nenhum dado real permite.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.interpolate as sci
from scipy.integrate import simpson
from scipy.optimize import brentq, brute, fmin, linprog
from scipy.stats import norm
from IPython.display import display

RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

def bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - sig * np.sqrt(T))

def vega_bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.pdf(d1) * np.sqrt(T)

def vol_impl(preco, S, K, T, r):
    piso = max(S - K * np.exp(-r * T), 0.0)
    if not (piso + 1e-12 < preco < S):
        return np.nan
    return brentq(lambda s: float(bsm(S, K, T, r, s)) - preco, 1e-4, 4.0)

def cf_heston(u, T, v0, kappa, theta, sigma, rho):
    """Função característica de ln(S_T/S_0) - rT no Heston, forma estável (Módulo 11)."""
    xi = kappa - rho * sigma * 1j * u
    d = np.sqrt(xi**2 + sigma**2 * (1j * u + u**2))
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
```

```
    A = kappa * theta / sigma**2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / sigma**2 * (1 - e) / (1 - g * e)
    return np.exp(A + B * v0)

def cf_saltos(u, T, lam, mu, delta):
    """Parte de saltos de Merton, com o drift compensado (11.13)-(11.14)."""
    omega = -lam * (np.exp(mu + 0.5 * delta**2) - 1)
    return np.exp((1j * u * omega + lam * (np.exp(1j * u * mu - 0.5 * u**2 * delta**2) - 1)) * T)

U_FINO = np.linspace(0, 2000, 200_001)
U_CAL = np.linspace(0, 300, 3001)

def calls(S0, K, T, r, heston, saltos=None, u=U_CAL):
    """Calls europeias pela fórmula de Lewis (11.12), vetorizada em (K, T, r) do mesmo tamanho.
    heston = (v0, κ_v, θ_v, σ_v, ρ); saltos = (λ, μ_J, δ) ou None."""
    K, T, r = (np.atleast_1d(np.asarray(x, float)) for x in (K, T, r))
    k = np.log(S0 / K) + r * T
    phi = cf_heston(u[None, :] - 0.5j, T[:, None], *heston)
    if saltos is not None:
        phi = phi * cf_saltos(u[None, :] - 0.5j, T[:, None], *saltos)
    h = np.exp(1j * u[None, :] * k[:, None]) * phi
    integral = simpson(h.real / (u**2 + 0.25), x=u, axis=1)
    return S0 - np.sqrt(S0 * K) * np.exp(-0.5 * r * T) / np.pi * integral

def zcb_cir(r0, kappa, theta, sigma, T):
    """Zero-cupom no CIR, (11.7)-(11.9) com t = 0."""
    g = np.sqrt(kappa**2 + 2 * sigma**2)
    e = np.exp(g * T) - 1
    D = 2 * g + (kappa + g) * e
    return (2 * g * np.exp((kappa + g) * T / 2) / D) ** (2 * kappa * theta / sigma**2) * np.exp(-2
    ↪  * e / D * r0)

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. Por que calibrar?

Livro: §11.1 e §11.2.1

1.1 A ideia

Na visão antiga, os parâmetros vêm primeiro (estimados de séries históricas) e o preço vem depois. Na precificação calibrada ao mercado (Módulo 1), a ordem se inverte: os preços das opções líquidas são dados, e procuramos os parâmetros que os reproduzem. Só então o modelo é usado para o que não tem preço observável.

1.2 Mercados incompletos

O BSM é completo: toda opção é replicável com o índice e o título. O Heston é incompleto com o índice apenas, mas completo se uma segunda opção for negociada (há duas fontes de risco). O Merton é incompleto mesmo com todas as opções: o tamanho

do salto é aleatório, e seria preciso um continuum de ativos para cobri-lo. O BCC97 herda essa incompletude.

Num mercado incompleto há infinitas medidas Q sem arbitragem e, portanto, um **intervalo** de preços para cada derivativo (Módulo 3, Seção 9). A calibração escolhe, dentro dessa família, a medida que reprecifica as opções observadas. Como diz Björk, citado no livro: quem escolhe a medida martingal é o mercado.

1.3 O Exemplo 4 do livro

Um período, três estados equiprováveis, $r = 0$. O título custa 10 e paga 10. O índice custa 10 e paga $(20, 10, 0)$. O derivativo paga $A_1 = (16, 4, 0)$.

Passo 1 — as medidas. $Q = (q_1, q_2, q_3)$ precisa de $q_s > 0$, soma 1 e $\mathbb{E}^Q[S_1] = S_0$:

$$20q_1 + 10q_2 = 10 \;\Rightarrow\; q_2 = 1 - 2q_1, \qquad q_1 + q_2 + q_3 = 1 \;\Rightarrow\; q_3 = q_1. \tag{13.3}$$

Com $q_1 = \nu$, a família é $Q_\nu = (\nu,\, 1-2\nu,\, \nu)$, e a positividade exige $0 < \nu < 1/2$.

Passo 2 — os preços. $A_0(\nu) = 16\nu + 4(1-2\nu) = 4 + 8\nu$, que percorre o intervalo aberto $]4,\ 8[$.

> **Errata.** O livro escreve o limite de ν como $1/4$ e calcula o preço superior como "$1 + 8 = 9$". Os dois estão errados: a positividade de $q_2 = 1 - 2\nu$ só exige $\nu < 1/2$, e o preço em $\nu = 1/4$ seria $4 + 2 = 6$, não 9. O intervalo correto é $]4, 8[$.

Laboratório 1 — O intervalo por programação linear

Os extremos do intervalo são os preços de **super-replicação** e **sub-replicação**: o menor custo de uma carteira (título, índice) que paga pelo menos A_1 em todos os estados, e o maior custo de uma que paga no máximo A_1. São dois problemas de programação linear.

```
S1 = np.array([20.0, 10.0, 0.0])
B1 = np.array([10.0, 10.0, 10.0])
A1 = np.array([16.0, 4.0, 0.0])
X = np.column_stack([B1, S1])                # payoffs de (título, índice)
custo = np.array([10.0, 10.0])               # preços de hoje

sup = linprog(custo, A_ub=-X, b_ub=-A1, bounds=[(None, None)] * 2)    # X·h ≥ A1, custo mínimo
inf = linprog(-custo, A_ub=X, b_ub=A1, bounds=[(None, None)] * 2)     # X·h ≤ A1, custo máximo
print("super-replicação:", f3(sup.fun), "com (título, índice) =", [f3(x) for x in sup.x])
print("sub-replicação:  ", f3(-inf.fun), "com (título, índice) =", [f3(x) for x in inf.x])

nus = np.linspace(0.001, 0.499, 500)
precos_nu = 4 + 8 * nus
ok(abs(sup.fun - 8) < 1e-8 and abs(-inf.fun - 4) < 1e-8, "o intervalo sem arbitragem é ]4, 8[")
ok(np.all((nus > 0) & (1 - 2 * nus > 0)), "todas as medidas com 0 < ν < 1/2 são positivas")
ok(abs(16 * 0.25 + 4 * 0.5 - 6) < 1e-12, "em ν = 1/4, o preço é 6, não 9 como no livro")
```

```
super-replicação: 8,000 com (título, índice) = ['-0,000', '0,800']
sub-replicação:   4,000 com (título, índice) = ['-0,000', '0,400']
OK — o intervalo sem arbitragem é ]4, 8[
OK — todas as medidas com 0 < ν < 1/2 são positivas
OK — em ν = 1/4, o preço é 6, não 9 como no livro
```

O mercado tem um ativo arriscado e três estados: falta um ativo para completá-lo. Se a opção A fosse negociada a, digamos, 6, só a medida $\nu = 1/4$ seria compatível com esse preço. Essa é a calibração em miniatura: **usar preços observados para escolher** Q.

---

2. Qual função objetivo?

Livro: §11.2.3

Com preços de mercado C_n^* e de modelo $C_n(p)$, as três escolhas mais comuns (Christoffersen e Jacobs, 2004) são

$$\text{(11.1)}\ \frac{1}{N}\sum_n \left(C_n^* - C_n(p)\right)^2, \qquad \text{(11.2)}\ \frac{1}{N}\sum_n \left(\frac{C_n^* - C_n(p)}{C_n^*}\right)^2, \qquad \text{(11.3)}\ \frac{1}{N}\sum_n \left(\sigma_n^* - \sigma_n(p)\right)^2. \tag{13.4}$$

O livro acrescenta uma versão de (11.3) ponderada pelo vega:

$$\text{(11.4)}\ \frac{1}{N}\sum_n \Big(\left(\sigma_n^* - \sigma_n(p)\right)\mathcal{V}_n\Big)^2, \qquad \mathcal{V}_n = \frac{\partial C^{BSM}}{\partial \sigma}\Big|_{\sigma_n^*}. \tag{13.5}$$

2.1 Por que (11.4) é quase (11.1)

Pela definição de volatilidade implícita, $C_n(p) = C^{BSM}(\sigma_n(p))$ e $C_n^* = C^{BSM}(\sigma_n^*)$. Expandindo em torno de σ_n^*:

$$C_n^* - C_n(p) \approx \mathcal{V}_n\,(\sigma_n^* - \sigma_n(p)). \tag{13.6}$$

Para erros pequenos, (11.4) e (11.1) medem a mesma coisa. A diferença real está entre (11.1)/(11.4), que pesam mais as opções com vega alto (no dinheiro, prazos longos), e (11.3), que pesa igualmente cada ponto da superfície de volatilidade. (11.2) dá peso enorme às opções baratas, fora do dinheiro.

Laboratório 2 — O que cada função pondera

Montamos o mercado sintético do módulo e perturbamos o modelo verdadeiro (variância inicial 10% maior). Medimos quanto cada opção contribui para cada função objetivo.

O mercado sintético

- Índice: $S_0 = 3.225{,}93$ (o nível do EURO STOXX 50 no livro).
- Vencimentos: 17, 80 e 171 dias (os do livro, em base 365).
- Strikes: 3.000 a 3.400, de 100 em 100: 15 calls.
- Modelo "verdadeiro": **Bates** (Heston com saltos de Merton), $v_0 = 0{,}03$, $\kappa_v = 3$, $\theta_v = 0{,}04$, $\sigma_v = 0{,}5$, $\rho = -0{,}7$, $\lambda = 0{,}15$, $\mu_J = -0{,}15$, $\delta = 0{,}1$. O mercado real tem saltos; o Heston que vamos calibrar não tem.
- Juros: os da curva CIR sintética da Seção 3 (definida já aqui).
- Cotações arredondadas a 0,1 ponto, o passo de cotação da Eurex.

```
S0 = 3225.93
DIAS = np.array([17, 80, 171])
STRIKES = np.arange(3000, 3401, 100.0)
T_OP = np.repeat(DIAS / 365, 5)
K_OP = np.tile(STRIKES, 3)

CIR_VERDADE = dict(r0=0.02, kappa=0.5, theta=0.045, sigma=0.08)      # curva sintética
R_OP = -np.log(zcb_cir(T=T_OP, **CIR_VERDADE)) / T_OP                 # r̂(T) = -ln B0(T)/T

HESTON_VERDADE = (0.03, 3.0, 0.04, 0.5, -0.7)
SALTOS_VERDADE = (0.15, -0.15, 0.10)
C_JUSTO = calls(S0, K_OP, T_OP, R_OP, HESTON_VERDADE, SALTOS_VERDADE, U_FINO)
C_MERC = np.round(C_JUSTO, 1)

ok(np.abs(calls(S0, K_OP, T_OP, R_OP, HESTON_VERDADE, SALTOS_VERDADE) - C_JUSTO).max() < 1e-3,
   "a grade de integração da calibração (0 a 300) erra menos de 0,001 ponto")

IV_MERC = np.array([vol_impl(c, S0, k, t, r) for c, k, t, r in zip(C_MERC, K_OP, T_OP, R_OP)])
mercado = pd.DataFrame({"dias": np.repeat(DIAS, 5), "K": K_OP, "r̄ (%)": 100 * R_OP,
                        "call": C_MERC, "vol. implícita (%)": 100 * IV_MERC})
tabela(mercado)
```

```
OK — a grade de integração da calibração (0 a 300) erra menos de 0,001 ponto
```

dias	K	r̄ (%)	call	vol. implícita (%)
17	3.000,000	2,029	233,200	22,686
17	3.100,000	2,029	141,900	20,241
17	3.200,000	2,029	66,300	18,269
17	3.300,000	2,029	19,400	16,569
17	3.400,000	2,029	2,600	15,094
80	3.000,000	2,132	279,000	21,356
80	3.100,000	2,132	201,300	20,043
80	3.200,000	2,132	133,500	18,695
80	3.300,000	2,132	78,700	17,350
80	3.400,000	2,132	39,500	16,073
171	3.000,000	2,271	335,200	21,005
171	3.100,000	2,271	263,000	20,052
171	3.200,000	2,271	198,100	19,092
171	3.300,000	2,271	141,800	18,135
171	3.400,000	2,271	95,400	17,211

```
pert = (0.033,) + HESTON_VERDADE[1:]
C_pert = calls(S0, K_OP, T_OP, R_OP, pert, SALTOS_VERDADE)
IV_pert = np.array([vol_impl(c, S0, k, t, r) for c, k, t, r in zip(C_pert, K_OP, T_OP, R_OP)])
VEGA = vega_bsm(S0, K_OP, T_OP, R_OP, IV_MERC)

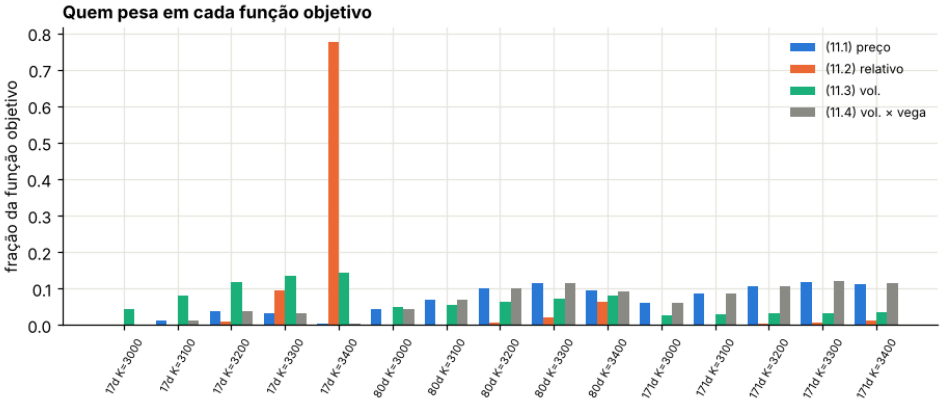
termos = pd.DataFrame({
    "(11.1) preço": (C_MERC - C_pert) ** 2,
    "(11.2) relativo": ((C_MERC - C_pert) / C_MERC) ** 2,
    "(11.3) vol.": (IV_MERC - IV_pert) ** 2,
    "(11.4) vol. × vega": ((IV_MERC - IV_pert) * VEGA) ** 2,
})
pesos = termos / termos.sum()
pesos.insert(0, "opção", [f"{d}d K={k:.0f}" for d, k in zip(np.repeat(DIAS, 5), K_OP)])
```

```
tabela(pesos)

fig, ax = plt.subplots(figsize=(8.5, 3.8))
x = np.arange(15)
for j, (col, cor) in enumerate(zip(termos.columns, [AZUL, LARANJA, VERDE, CINZA])):
    ax.bar(x + (j - 1.5) * 0.2, pesos[col], 0.2, color=cor, label=col)
ax.set_xticks(x, pesos["opção"], rotation=60, fontsize=7)
ax.set_ylabel("fração da função objetivo")
ax.set_title("Quem pesa em cada função objetivo")
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

razao = termos["(11.4) vol. × vega"].sum() / termos["(11.1) preço"].sum()
print("(11.4)/(11.1):", f3(razao))
ok(abs(razao - 1) < 0.05, "para erros pequenos, (11.4) e (11.1) coincidem a menos de 5%")
ok(pesos["(11.2) relativo"].iloc[4] > 0.5, "o erro relativo concentra mais da metade do peso na
↪  call de 17 dias mais fora do dinheiro")
```

opção	(11.1) preço	(11.2) relativo	(11.3) vol.	(11.4) vol. × vega
17d K=3000	0,002	0,000	0,044	0,002
17d K=3100	0,012	0,001	0,081	0,011
17d K=3200	0,039	0,010	0,119	0,039
17d K=3300	0,034	0,096	0,135	0,033
17d K=3400	0,005	0,776	0,143	0,004
80d K=3000	0,043	0,001	0,051	0,043
80d K=3100	0,070	0,002	0,056	0,070
80d K=3200	0,101	0,006	0,064	0,101
80d K=3300	0,115	0,020	0,072	0,116
80d K=3400	0,094	0,065	0,080	0,093
171d K=3000	0,062	0,001	0,027	0,062
171d K=3100	0,085	0,001	0,029	0,085
171d K=3200	0,106	0,003	0,031	0,106
171d K=3300	0,119	0,006	0,034	0,119
171d K=3400	0,113	0,013	0,035	0,114




```
(11.4)/(11.1): 0,995
OK — para erros pequenos, (11.4) e (11.1) coincidem a menos de 5%
OK — o erro relativo concentra mais da metade do peso na call de 17 dias mais fora do dinheiro
```

O erro relativo (11.2) é dominado por uma única opção, a call barata de 17 dias com $K = 3.400$, que vale poucos pontos. Já (11.1) e (11.4) quase coincidem e espalham o peso pelas opções de vega alto. O livro usa (11.1) para calibrar a preços e (11.4) para calibrar a volatilidades (Módulo 14).

---

3. A curva de juros: calibrar o CIR

Livro: §11.3 e §11.6.1

3.1 De taxas de mercado monetário a taxas forward

Taxas como a Euribor são **simples**, com contagem real/360: quem aplica 1 por t anos recebe $1 + tL$. O zero-cupom vale $B_0(t) = 1/(1 + tL)$, e a taxa contínua equivalente é

$$y(t) = -\frac{\ln B_0(t)}{t} = \frac{\ln(1 + tL)}{t}. \tag{13.7}$$

A taxa forward instantânea é $f(0,T) = -\partial_T \ln B_0(T)$. Como $\ln B_0(T) = -T\,y(T)$, derivando o produto:

$$f(0,T) = y(T) + T\,y'(T). \tag{11.10}$$

As taxas só existem em 9 prazos. O livro interpola y por spline cúbico e deriva o spline para obter f em 24 pontos de $[0,1]$.

3.2 A taxa forward no CIR, passo a passo

Do Módulo 11, com $\gamma = \sqrt{\kappa^2 + 2\sigma^2}$, $E = e^{\gamma T} - 1$ e $D = 2\gamma + (\kappa + \gamma)E$:

$$\ln B_0(T) = \frac{2\kappa\theta}{\sigma^2}\Big[\ln(2\gamma) + \frac{(\kappa+\gamma)T}{2} - \ln D\Big] - \frac{2E}{D}\,r_0. \tag{13.8}$$

Primeiro termo. Com $D' = (\kappa + \gamma)\gamma e^{\gamma T}$:

$$\frac{\partial}{\partial T}\Big[\frac{(\kappa+\gamma)T}{2} - \ln D\Big] = \frac{\kappa+\gamma}{2D}(D - 2\gamma e^{\gamma T}) = \frac{\kappa+\gamma}{2D}(\kappa - \gamma)E = -\frac{\sigma^2 E}{D}, \tag{13.9}$$

usando $D - 2\gamma e^{\gamma T} = (\kappa - \gamma)E$ e $\kappa^2 - \gamma^2 = -2\sigma^2$. Multiplicando por $2\kappa\theta/\sigma^2$: $-2\kappa\theta E/D$.

Segundo termo. $\dfrac{\partial}{\partial T}\dfrac{2E}{D} = \dfrac{2\gamma e^{\gamma T} D - 2E(\kappa+\gamma)\gamma e^{\gamma T}}{D^2} = \dfrac{2\gamma e^{\gamma T}\cdot 2\gamma}{D^2} = \dfrac{4\gamma^2 e^{\gamma T}}{D^2}$.

Resultado. Com o sinal de $f = -\partial_T \ln B_0$:

$$\boxed{f^{CIR}(0,T) = \frac{2\kappa\theta\,(e^{\gamma T} - 1)}{2\gamma + (\kappa+\gamma)(e^{\gamma T} - 1)} + r_0\,\frac{4\gamma^2 e^{\gamma T}}{\big(2\gamma + (\kappa+\gamma)(e^{\gamma T} - 1)\big)^2}} \tag{13.10}$$

Errata. A equação (11.6) do livro, e o código da §11.6.1, trazem $\kappa\theta$ no primeiro termo, sem o fator 2. Uma conferência rápida: para $T \to \infty$, a fórmula correta dá $f \to 2\kappa\theta/(\kappa+\gamma)$, o limite conhecido do CIR, que tende a θ quando $\sigma \to 0$. A do livro tenderia a $\theta/2$: com volatilidade zero, a taxa forward de longo prazo seria metade da média de longo prazo, o que não faz sentido.

Laboratório 3a — Conferir a fórmula pela derivada numérica

```
def fwd_cir(T, r0, kappa, theta, sigma, fator=2.0):
    """Taxa forward do CIR. fator=2 é a fórmula correta; fator=1 é a (11.6) do livro."""
    g = np.sqrt(kappa**2 + 2 * sigma**2)
    E = np.exp(g * T) - 1
    D = 2 * g + (kappa + g) * E
    return fator * kappa * theta * E / D + r0 * 4 * g**2 * np.exp(g * T) / D**2


Tt = np.array([0.25, 1.0, 5.0, 30.0])
h = 1e-6
f_num = -(np.log(zcb_cir(T=Tt + h, **CIR_VERDADE)) - np.log(zcb_cir(T=Tt - h, **CIR_VERDADE))) /
↪  (2 * h)
tab3a = pd.DataFrame({"T": Tt, "−∂ ln B/∂T (%)": 100 * f_num,
                      "fórmula correta (%)": 100 * fwd_cir(Tt, **CIR_VERDADE),
                      "(11.6) do livro (%)": 100 * fwd_cir(Tt, **CIR_VERDADE, fator=1)})
tabela(tab3a)
c = CIR_VERDADE
lim = 2 * c["kappa"] * c["theta"] / (c["kappa"] + np.sqrt(c["kappa"] ** 2 + 2 * c["sigma"] ** 2))
print("limite 2κθ/(κ+γ):", f3(100 * lim), "%")
ok(np.allclose(f_num, fwd_cir(Tt, **CIR_VERDADE), atol=1e-8), "a fórmula com o fator 2 confere
↪  com a derivada numérica")
ok(abs(fwd_cir(Tt, **CIR_VERDADE, fator=1)[-1] - f_num[-1]) > 0.01, "a (11.6) do livro erra mais
↪  de 1 ponto percentual em 30 anos")
ok(abs(fwd_cir(200.0, **CIR_VERDADE) - lim) < 1e-6, "a taxa forward correta converge para
↪  2κθ/(κ+γ)")
```

T	$-\partial$ ln B/∂T (%)	fórmula correta (%)	(11.6) do livro (%)
0,250	2,293	2,293	2,029
1,000	2,979	2,979	2,094
5,000	4,255	4,255	2,206
30,000	4,444	4,444	2,222

```
limite 2κθ/(κ+γ): 4,444 %
OK — a fórmula com o fator 2 confere com a derivada numérica
OK — a (11.6) do livro erra mais de 1 ponto percentual em 30 anos
OK — a taxa forward correta converge para 2κθ/(κ+γ)
```

Laboratório 3b — Calibrar à curva, com as duas fórmulas

A curva sintética vem do CIR "verdadeiro" ($r_0 = 2\%$, $\kappa = 0{,}5$, $\theta = 4{,}5\%$, $\sigma = 0{,}08$): calculamos as taxas simples nos 9 prazos do livro e as arredondamos a 3 casas em porcentagem, como são publicadas. Depois seguimos o roteiro do livro: taxas contínuas, spline, taxas forward em 24 pontos, r_0 fixado na taxa de 1 dia, e o erro quadrático médio (11.11) minimizado por Nelder-Mead (`fmin`) a partir de $(\kappa, \theta, \sigma) = (1;\ 0{,}02;\ 0{,}1)$, com a condição de Feller como restrição.

```
t_merc = np.array([1, 7, 14, 30, 60, 90, 180, 270, 360]) / 360
L_merc = np.round((1 / zcb_cir(T=t_merc, **CIR_VERDADE) - 1) / t_merc * 100, 3) / 100
y_merc = np.log(1 + t_merc * L_merc) / t_merc
r0_merc = L_merc[0]

tck = sci.splrep(t_merc, y_merc, k=3)
tn = np.linspace(0.0, 1.0, 24)
f_merc = sci.splev(tn, tck) + sci.splev(tn, tck, der=1) * tn

print("taxas simples (%):", [f3(100 * x) for x in L_merc])
print("erro máximo da taxa forward do spline contra a verdadeira:",
      f3(1e4 * np.abs(f_merc - fwd_cir(tn, **CIR_VERDADE)).max()), "pb")


def calibra_cir(fator, inicio=(1.0, 0.02, 0.1)):
    def erro(p):
        kappa, theta, sigma = p
        if 2 * kappa * theta < sigma**2 or kappa < 0 or theta < 0 or sigma < 0.001:
            return 100.0
        return np.mean((f_merc - fwd_cir(tn, r0_merc, kappa, theta, sigma, fator)) ** 2)
    p = fmin(erro, inicio, xtol=1e-7, ftol=1e-12, maxiter=3000, maxfun=5000, disp=False)
    return p, erro(p)


linhas = []
for nome, fator, inicio in [("(11.6) do livro, início do livro", 1, (1.0, 0.02, 0.1)),
                            ("(11.6) do livro, outro início", 1, (0.5, 0.09, 0.08)),
                            ("fórmula correta, início do livro", 2, (1.0, 0.02, 0.1))]:
    p, e = calibra_cir(fator, inicio)
    linhas.append((nome, *p, 1e4 * np.sqrt(e)))
linhas.append(("verdade", CIR_VERDADE["kappa"], CIR_VERDADE["theta"], CIR_VERDADE["sigma"],
↪  np.nan))
tab3b = pd.DataFrame(linhas, columns=["calibração", "κ_r", "θ_r", "σ_r", "RMSE (pb)"])
tabela(tab3b)
```

```
taxas simples (%): ['2,002', '2,013', '2,025', '2,053', '2,105', '2,156', '2,301', '2,436', '2,563']
erro máximo da taxa forward do spline contra a verdadeira: 0,179 pb
```

calibração	κ__r	θ__r	σ__r	RMSE (pb)
(11.6) do livro, início do livro	0,081	0,029	0,068	63,138
(11.6) do livro, outro início	0,496	0,090	0,001	0,098
fórmula correta, início do livro	0,496	0,045	0,001	0,098
verdade	0,500	0,045	0,080	n/d

Três lições saem da tabela:

1. **Com a fórmula do livro, o ajuste pode ser perfeito, mas θ_r sai dobrado.** Como o θ só aparece no produto $\kappa\theta$ do primeiro termo, a fórmula sem o fator 2 compensa com θ duas vezes maior. Quem usar esse θ_r na fórmula correta do zero-cupom (como faz o próprio livro ao montar $\bar{r}$ nas §11.4 e seguintes) terá descontos errados.
2. **O ponto de partida importa.** Do início do livro, o Nelder-Mead com a fórmula do livro para num ponto ruim.
3. **σ_r não é identificado pela curva.** Ele só entra pela convexidade, que é minúscula em prazos de até um ano; a calibração o empurra para qualquer valor, inclusive o limite inferior. Para fixar σ_r são necessários derivativos de juros (caps,

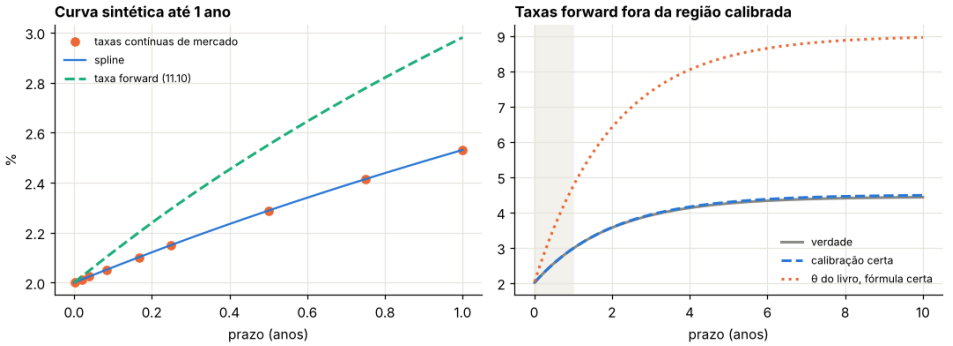
swaptions).

```
lin = tab3b.set_index("calibração")
p_livro = lin.loc["(11.6) do livro, outro início", ["κ_r", "θ_r", "σ_r"]].values
p_certo = lin.loc["fórmula correta, início do livro", ["κ_r", "θ_r", "σ_r"]].values
Tz = np.array([1.0, 2.0, 5.0, 10.0])
B_v = zcb_cir(T=Tz, **CIR_VERDADE)
B_livro = zcb_cir(r0_merc, *p_livro, Tz)
B_certo = zcb_cir(r0_merc, *p_certo, Tz)
tab3c = pd.DataFrame({"T": Tz, "B verdadeiro": B_v,
                      "θ do livro na fórmula certa": B_livro, "calibração certa": B_certo,
                      "erro do livro (pb de taxa)": 1e4 * (np.log(B_v) - np.log(B_livro)) / Tz})
tabela(tab3c)

fig, axs = plt.subplots(1, 2, figsize=(10, 3.8))
axs[0].plot(t_merc, 100 * y_merc, "o", color=LARANJA, label="taxas contínuas de mercado")
axs[0].plot(tn, 100 * sci.splev(tn, tck), color=AZUL, lw=1.5, label="spline")
axs[0].plot(tn, 100 * f_merc, "--", color=VERDE, label="taxa forward (11.10)")
axs[0].set_xlabel("prazo (anos)"); axs[0].set_ylabel("%")
axs[0].set_title("Curva sintética até 1 ano"); axs[0].legend(fontsize=8)
Tl = np.linspace(0.01, 10, 200)
axs[1].plot(Tl, 100 * fwd_cir(Tl, **CIR_VERDADE), color=CINZA, label="verdade")
axs[1].plot(Tl, 100 * fwd_cir(Tl, r0_merc, *p_certo), "--", color=AZUL, label="calibração certa")
axs[1].plot(Tl, 100 * fwd_cir(Tl, r0_merc, *p_livro), ":", color=LARANJA, label="θ do livro,
↪  fórmula certa")
axs[1].axvspan(0, 1, color="#f2f1ec", zorder=0)
axs[1].set_xlabel("prazo (anos)"); axs[1].set_title("Taxas forward fora da região calibrada")
axs[1].legend(fontsize=8)
plt.tight_layout(); plt.show()

ok(lin.loc["(11.6) do livro, outro início", "RMSE (pb)"] < 0.5, "com a fórmula do livro, o ajuste
↪  à curva até 1 ano é perfeito")
ok(abs(p_livro[1] / CIR_VERDADE["theta"] - 2) < 0.05, "mas θ_r sai praticamente dobrado")
ok(lin.loc["(11.6) do livro, início do livro", "RMSE (pb)"] > 10, "do ponto de partida do livro,
↪  o otimizador para longe do ajuste")
ok(abs(p_certo[1] / CIR_VERDADE["theta"] - 1) < 0.05, "com a fórmula correta, θ_r é recuperado a
↪  menos de 5%")
ok(abs(tab3c["erro do livro (pb de taxa)"].iloc[-1]) > 50, "o θ dobrado erra a taxa de 10 anos em
↪  mais de 50 pb")
```

T	B verdadeiro	θ do livro na fórmula certa	calibração certa	erro do livro (pb de taxa)
1,000	0,975	0,966	0,975	95,219
2,000	0,943	0,913	0,943	164,799
5,000	0,837	0,726	0,836	285,301
10,000	0,672	0,468	0,670	363,193



```
OK — com a fórmula do livro, o ajuste à curva até 1 ano é perfeito
OK — mas θ_r sai praticamente dobrado
OK — do ponto de partida do livro, o otimizador para longe do ajuste
OK — com a fórmula correta, θ_r é recuperado a menos de 5%
OK — o θ dobrado erra a taxa de 10 anos em mais de 50 pb
```

Nota sobre os dados do livro. A taxa Eonia de 01/10/2014, usada como r_0, é **negativa** ($-0{,}032\%$). O CIR não admite taxas negativas: o termo $\sigma\sqrt{r}$ nem está definido. A fórmula fechada ainda devolve um número, mas ele deixa de ser um preço do modelo. Em ambientes de juros negativos usa-se um CIR deslocado (a extensão CIR++ de Brigo e Mercurio, citada na nota 13 do livro) ou outro modelo.

---

4. A função característica do Heston no livro

Livro: §11.4.1 e §9.7.3

O livro escreve a função característica com $c_1 = \kappa_v\theta_v$, $c_2 = -\sqrt{(\rho\sigma_v ui - \kappa_v)^2 - \sigma_v^2(-ui - u^2)}$ e $c_3 = (\kappa_v - \rho\sigma_v ui + c_2)/(\kappa_v - \rho\sigma_v ui - c_2)$. Comparando com a nossa: $c_2 = -d$ e $c_3 = g$. Como $c_2 = -d$, os termos $e^{c_2 T}$ são os e^{-dT} que decaem: é a forma estável do Módulo 11.

Errata. Na equação de H_1 impressa na §11.4.1, o logaritmo traz $1 - c_1 e^{c_3 T}$. O código (§9.7.3) usa $1 - c_3 e^{c_2 T}$, que é o correto.

Laboratório 4 — O código do livro e a nossa função

```python
def cf_heston_livro(u, T, r, kappa, theta, sigma, rho, v0, versao="código"):
    """Transcrição da §9.7.3 (com o r·iuT incluído); versao="impressa" usa o 1 - c1·e^{c3 T}."""
    c1 = kappa * theta
    c2 = -np.sqrt((rho * sigma * u * 1j - kappa) ** 2 - sigma**2 * (-u * 1j - u**2))
    c3 = (kappa - rho * sigma * u * 1j + c2) / (kappa - rho * sigma * u * 1j - c2)
    termo = (1 - c3 * np.exp(c2 * T)) if versao == "código" else (1 - c1 * np.exp(c3 * T))
    H1 = r * u * 1j * T + c1 / sigma**2 * ((kappa - rho * sigma * u * 1j + c2) * T
                                           - 2 * np.log(termo / (1 - c3)))
    H2 = (kappa - rho * sigma * u * 1j + c2) / sigma**2 * (1 - np.exp(c2 * T)) / (1 - c3 *
↪   np.exp(c2 * T))
    return np.exp(H1 + H2 * v0)


u = np.linspace(0, 50, 501) - 0.5j
v0, kv, tv, sv, rho = HESTON_VERDADE
r_, T_ = 0.03, 0.5
nossa = np.exp(1j * u * r_ * T_) * cf_heston(u, T_, v0, kv, tv, sv, rho)
livro = cf_heston_livro(u, T_, r_, kv, tv, sv, rho, v0)
impressa = cf_heston_livro(u, T_, r_, kv, tv, sv, rho, v0, "impressa")
print("maior diferença, código do livro × nossa:", f"{np.abs(livro - nossa).max():.1e}")
print("maior diferença, fórmula impressa × nossa:", f3(np.abs(impressa - nossa).max()))
ok(np.abs(livro - nossa).max() < 1e-12, "o código do livro é a forma estável, idêntica à nossa")
ok(np.abs(impressa - nossa).max() > 0.01, "a fórmula impressa de H1 dá outra função")
```

```
maior diferença, código do livro × nossa: 2.3e-16
maior diferença, fórmula impressa × nossa: 0,132
OK — o código do livro é a forma estável, idêntica à nossa
OK — a fórmula impressa de H1 dá outra função
```

---

5. Calibrar o Heston às 15 opções

Livro: §11.4.2, §11.4.3, §11.6.2 e §11.6.3

5.1 A estratégia do livro

O problema é minimizar (11.1) em $p = (\kappa_v, \theta_v, \sigma_v, \rho, v_0)$. A função objetivo não é convexa e pode ter vários mínimos locais. O livro usa dois passos:

1. **Busca em grade** (`brute`): avalia a função num reticulado grosso ($\kappa_v \in \{2{,}5;\ 7{,}5\}$, $\theta_v \in \{0{,}01; \ldots; 0{,}04\}$, $\sigma_v \in \{0{,}05;\ 0{,}15;\ 0{,}25\}$, $\rho \in \{-0{,}75; \ldots; 0\}$, $v_0 \in \{0{,}01;\ 0{,}02;\ 0{,}03\}$): 288 pontos.
2. **Busca local** (`fmin`, Nelder-Mead) a partir do melhor ponto da grade.

Restrições são impostas devolvendo um valor alto (500) fora da região admissível: $\kappa_v \geq 0$, $\theta_v \geq 0{,}005$, $\sigma_v \geq 0$, $|\rho| \leq 1$ e a condição de Feller $2\kappa_v\theta_v \geq \sigma_v^2$.

Os juros entram como uma taxa constante por vencimento, $\bar{r}(T) = -\ln B_0(T)/T$, com B_0 do CIR calibrado. Pelo Módulo 11, isso é uma aproximação; em prazos de até seis meses ela é muito boa.

Laboratório 5 — A calibração e o que sobra

```python
GRADE_LIVRO = ((2.5, 10.6, 5.0), (0.01, 0.041, 0.01), (0.05, 0.251, 0.1), (-0.75, 0.01, 0.25),
 ↪  (0.01, 0.031, 0.01))
R_CAL = -np.log(zcb_cir(r0_merc, *p_certo, T_OP)) / T_OP      # juros da curva calibrada


def erro_h93(p, alvo, feller=True):
    kappa, theta, sigma, rho, v0 = p
    if kappa < 0 or theta < 0.005 or sigma < 0 or abs(rho) > 1 or v0 <= 0:
        return 500.0
    if feller and 2 * kappa * theta < sigma**2:
        return 500.0
    return np.mean((calls(S0, K_OP, T_OP, R_CAL, (v0, kappa, theta, sigma, rho)) - alvo) ** 2)


def calibra_h93(alvo, feller=True):
    p0 = brute(erro_h93, GRADE_LIVRO, args=(alvo, feller), finish=None)
    p = fmin(erro_h93, p0, args=(alvo, feller), xtol=1e-6, ftol=1e-6, maxiter=750, maxfun=900,
 ↪  disp=False)
    return p0, p, erro_h93(p, alvo, feller)


t0 = time.time()
p0_bates, p_bates, mse_bates = calibra_h93(C_MERC)
print(f"tempo: {time.time() - t0:.1f} s")
nomes = ["κ_v", "θ_v", "σ_v", "ρ", "v0"]
verd = [HESTON_VERDADE[i] for i in (1, 2, 3, 4, 0)]
```

```
tab5 = pd.DataFrame({"parâmetro": nomes, "melhor ponto da grade": p0_bates,
                     "calibrado": p_bates, "verdadeiro (Bates)": verd})
tabela(tab5)
print("EQM (11.1):", f3(mse_bates), "| folga de Feller 2κθ − σ²:", f3(2 * p_bates[0] * p_bates[1]
↪  - p_bates[2] ** 2))
```

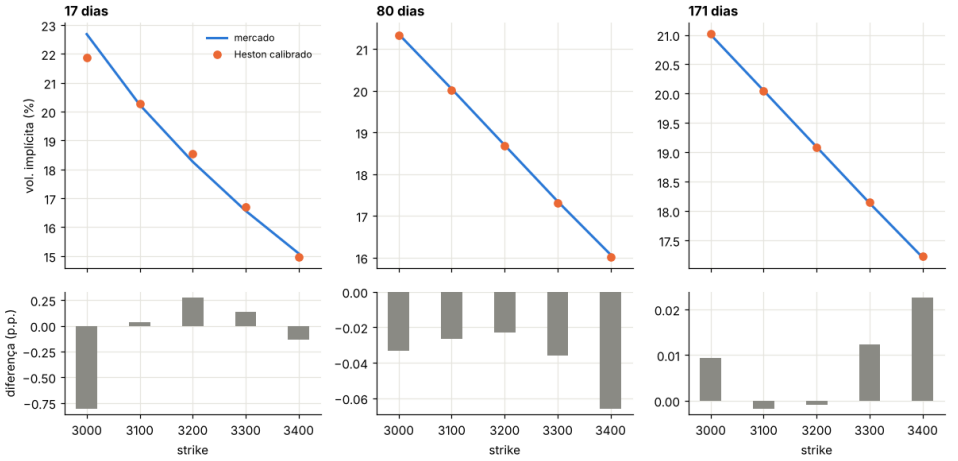
```
tempo: 36.8 s
```

parâmetro	melhor ponto da grade	calibrado	verdadeiro (Bates)
κ_v	2,500	4,116	3,000
θ_v	0,040	0,045	0,040
σ_v	0,250	0,609	0,500
ρ	-0,750	-0,744	-0,700
v0	0,030	0,033	0,030

```
EQM (11.1): 0,092 | folga de Feller 2κθ − σ²: 0,001
```

```
C_cal = calls(S0, K_OP, T_OP, R_CAL, (p_bates[4], *p_bates[:4]))
IV_cal = np.array([vol_impl(c, S0, k, t, r) for c, k, t, r in zip(C_cal, K_OP, T_OP, R_OP)])
fig, axs = plt.subplots(2, 3, figsize=(11, 5.5), sharex=True, gridspec_kw={"height_ratios": [2,
↪  1]})
for j, d in enumerate(DIAS):
    m = slice(5 * j, 5 * j + 5)
    axs[0, j].plot(STRIKES, 100 * IV_MERC[m], "-", color=AZUL, label="mercado")
    axs[0, j].plot(STRIKES, 100 * IV_cal[m], "o", color=LARANJA, label="Heston calibrado")
    axs[0, j].set_title(f"{d} dias")
    axs[1, j].bar(STRIKES, 100 * (IV_cal[m] - IV_MERC[m]), width=40, color=CINZA)
    axs[1, j].set_xlabel("strike")
axs[0, 0].set_ylabel("vol. implícita (%)"); axs[1, 0].set_ylabel("diferença (p.p.)")
axs[0, 0].legend(fontsize=8)
plt.tight_layout(); plt.show()

res = pd.DataFrame({"dias": np.repeat(DIAS, 5), "erro (pontos)": C_cal - C_MERC,
                    "erro de vol. (p.p.)": 100 * (IV_cal - IV_MERC)})
eqm_prazo = res.groupby("dias")["erro (pontos)"].apply(lambda e: np.mean(e**2))
print("EQM por vencimento:", {k: f3(v) for k, v in eqm_prazo.items()})
ok(mse_bates < 0.5, "o Heston ajusta as 15 calls com EQM menor que 0,5 ponto²")
ok(np.abs(res["erro de vol. (p.p.)"]).max() < 1.5, "o maior erro de volatilidade fica abaixo de
↪  1,5 p.p.")
ok(np.argmax(np.abs(res["erro de vol. (p.p.)"].values)) < 5, "o pior erro de volatilidade está no
↪  vencimento de 17 dias")
```




```
EQM por vencimento: {17: '0,223', 80: '0,043', 171: '0,011'}
OK — o Heston ajusta as 15 calls com EQM menor que 0,5 ponto²
OK — o maior erro de volatilidade fica abaixo de 1,5 p.p.
OK — o pior erro de volatilidade está no vencimento de 17 dias
```

O que aprendemos. O ajuste é bom, como no livro (que obtém EQM de 0,307 com dados reais). Os resíduos não são aleatórios: o maior erro está no prazo mais curto, onde o skew do mercado é mais íngreme do que a volatilidade estocástica consegue gerar em 17 dias. É o papel dos saltos, que entram no Módulo 14. Repare também que os parâmetros calibrados **não** são os do Heston do modelo verdadeiro: o σ_v e o ρ absorvem parte do efeito dos saltos. E a folga de Feller impressa acima é praticamente zero: aqui também a solução parou na fronteira da restrição, como no livro (Seção 6).

---

6. O que os dados identificam

Livro: §11.2.3 e §11.4.4 (degenerescência e indeterminação)

Uma calibração com EQM baixo não garante parâmetros "certos". Duas perguntas:

- **Recuperação:** se o mercado fosse exatamente Heston, a calibração devolveria os parâmetros verdadeiros?
- **Perfil:** fixando um parâmetro em vários valores e recalibrando os outros, quanto o erro piora? Se piorar menos que o ruído das cotações, os dados não distinguem esses valores.

O modelo verdadeiro escolhido **viola** a condição de Feller: $2\kappa_v\theta_v = 0{,}24 < \sigma_v^2 = 0{,}25$. Isso é comum em calibrações a mercados de ações. A restrição de Feller do livro, então, proíbe justamente a verdade.

> **No livro.** Os parâmetros da §11.4.3 dão $2\kappa_v\theta_v - \sigma_v^2 = 2 \cdot 18{,}447 \cdot 0{,}026 - 0{,}978^2 \approx 0{,}003$: a solução está colada na fronteira de Feller. É o sinal de que a restrição está ativa e distorce o ajuste.

Laboratório 6 — Recuperação, Feller e o perfil de κ_v

```
C_HESTON = np.round(calls(S0, K_OP, T_OP, R_CAL, HESTON_VERDADE, None, U_FINO), 1)
linhas = []
for feller in (True, False):
    _, p, e = calibra_h93(C_HESTON, feller)
    linhas.append(("com Feller" if feller else "sem Feller", *p, e, 2 * p[0] * p[1] - p[2] ** 2))
linhas.append(("verdadeiro", *verd, np.nan, 2 * 3.0 * 0.04 - 0.25))
tab6 = pd.DataFrame(linhas, columns=["calibração", *nomes, "EQM", "2κθ − σ²"])
tabela(tab6)
```

calibração	κ_v	θ_v	σ_v	ρ	v0	EQM	$2\kappa\theta - \sigma^2$
com Feller	2,492	0,041	0,453	-0,727	0,030	0,023	0,000
sem Feller	2,992	0,040	0,498	-0,702	0,030	0,001	-0,009
verdadeiro	3,000	0,040	0,500	-0,700	0,030	n/d	-0,010

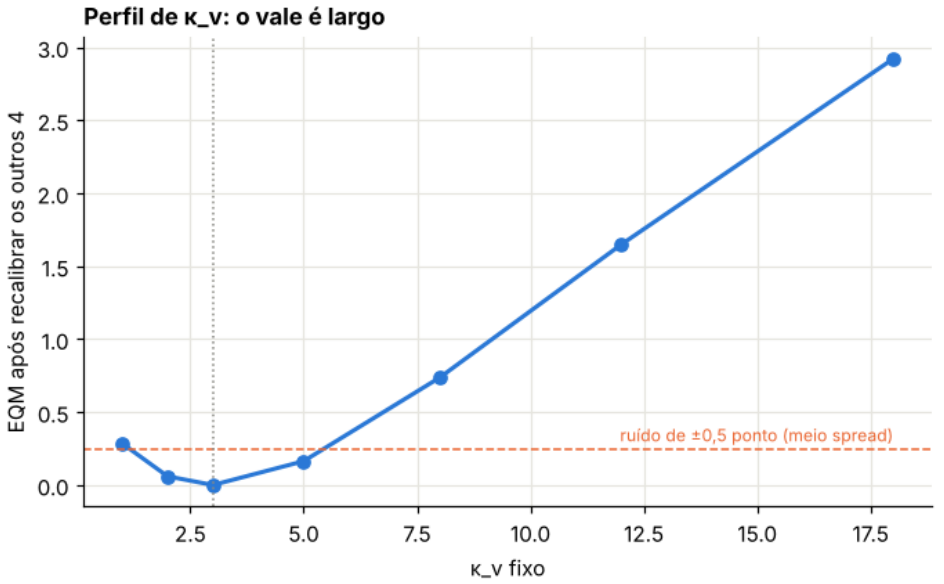
```
t0 = time.time()
perfil = []
for kappa in [1, 2, 3, 5, 8, 12, 18]:
    def erro_fixo(q):
        theta, sigma, rho, v0 = q
        if theta < 0.001 or sigma < 0.01 or abs(rho) > 1 or v0 <= 0:
            return 500.0
        return np.mean((calls(S0, K_OP, T_OP, R_CAL, (v0, kappa, theta, sigma, rho)) - C_HESTON)
        ↪  ** 2)
    q = fmin(erro_fixo, [0.04, 0.5, -0.7, 0.03], xtol=1e-6, ftol=1e-8, maxiter=1500, maxfun=2000,
↪  disp=False)
    perfil.append((kappa, *q, erro_fixo(q)))
tab6b = pd.DataFrame(perfil, columns=["κ_v fixo", "θ_v", "σ_v", "ρ", "v0", "EQM"])
tabela(tab6b)
print(f"tempo do perfil: {time.time() - t0:.1f} s")

fig, ax = plt.subplots()
ax.plot(tab6b["κ_v fixo"], tab6b["EQM"], "o-", color=AZUL)
ax.axhline(0.25, ls="--", color=LARANJA, lw=1)
ax.text(18, 0.3, "ruído de ±0,5 ponto (meio spread)", ha="right", color=LARANJA, fontsize=8)
ax.axvline(3, color=CINZA, lw=1, ls=":")
ax.set_xlabel("κ_v fixo"); ax.set_ylabel("EQM após recalibrar os outros 4")
ax.set_title("Perfil de κ_v: o vale é largo")
plt.show()

t6 = tab6.set_index("calibração")
ok(abs(t6.loc["sem Feller", "κ_v"] - 3) < 0.3 and abs(t6.loc["sem Feller", "σ_v"] - 0.5) < 0.03,
   "sem Feller, a calibração recupera κ_v e σ_v verdadeiros")
ok(t6.loc["com Feller", "EQM"] > 3 * t6.loc["sem Feller", "EQM"], "com Feller, o EQM fica mais de
↪  3 vezes maior")
ok(abs(t6.loc["com Feller", "2κθ − σ²"]) < 0.005, "e a solução com Feller fica na fronteira da
↪  restrição")
dentro = tab6b[tab6b["EQM"] < 0.25]["κ_v fixo"]
print("valores de κ_v com EQM abaixo do ruído de meio spread:", list(dentro))
ok(dentro.min() <= 2 and dentro.max() >= 5, "κ_v entre 2 e 5 é compatível com cotações de spread
↪  de 1 ponto")
ok(tab6b["v0"].std() < 0.002, "já v0 quase não se move ao longo do perfil: é bem identificado")
```

κ_v fixo	θ_v	σ_v	ρ	v0	EQM
1	0,058	0,434	-0,639	0,030	0,285
2	0,044	0,468	-0,672	0,030	0,060
3	0,040	0,499	-0,702	0,030	0,001
5	0,037	0,564	-0,747	0,030	0,164
8	0,035	0,696	-0,761	0,031	0,737
12	0,034	0,915	-0,739	0,031	1,649
18	0,034	1,306	-0,689	0,032	2,922

```
tempo do perfil: 20.7 s
```




```
OK — sem Feller, a calibração recupera κ_v e σ_v verdadeiros
OK — com Feller, o EQM fica mais de 3 vezes maior
OK — e a solução com Feller fica na fronteira da restrição
valores de κ_v com EQM abaixo do ruído de meio spread: [2, 3, 5]
OK — κ_v entre 2 e 5 é compatível com cotações de spread de 1 ponto
OK — já v0 quase não se move ao longo do perfil: é bem identificado
```

O que aprendemos.

- Sem Feller, com dados exatamente Heston, a calibração devolve os parâmetros verdadeiros: o método funciona.
- Com Feller, a calibração para na fronteira e piora o ajuste. A condição de Feller garante que v não toca o zero, o que é conveniente para simulação (Módulo 12), mas não é uma exigência de ausência de arbitragem. Impô-la deve ser uma decisão consciente.
- Com vencimentos de até seis meses, κ_v é **mal identificado**: entre 2 e 5, a diferença no ajuste fica abaixo do ruído de um spread típico. θ_v e σ_v se ajustam para compensar. Já v_0, que domina o nível das volatilidades curtas, é bem identificado. O $\kappa_v = 18{,}4$ do livro é típico desse problema: um vale largo e uma solução

empurrada para longe.
- Esse é o problema de **degenerescência** que o livro cita na §11.4.4. O Módulo 14 trata do remédio, a regularização.

---

7. Síntese

Etapa	O que aprendemos
Por que calibrar	em mercado incompleto há um intervalo de preços; a calibração escolhe a medida que reprecifica o que é negociado
Função objetivo	(11.1) e (11.4) quase coincidem; (11.2) concentra peso nas opções baratas; a escolha faz parte do modelo
Curva de juros	taxas simples → contínuas → spline → taxas forward por (11.10); a fórmula do CIR precisa do fator 2
Identificação dos juros	uma curva até 1 ano não fixa σ_r e mal fixa κ_r; o θ_r vale para fora da região calibrada só se o modelo estiver certo
Heston	grade + busca local funciona; os resíduos no prazo curto pedem saltos
Identificação do Heston	v_0 bem identificado, κ_v mal; a restrição de Feller pode distorcer o ajuste

8. Erratas e notas sobre o capítulo 11 (primeira parte)

Onde	O quê
Exemplo 4 (§11.2.1)	o limite é $\nu < 1/2$, não $1/4$; o intervalo de preços é $]4, 8[$, e não $]4, 9[$; e em $\nu = 1/4$ o preço seria 6, não "$1 + 8 = 9$"
Eq. (11.6) e código da §11.6.1	falta o fator 2 no primeiro termo: o correto é $2\kappa_r\theta_r(e^{\gamma t} - 1)/D$; o θ_r calibrado sai dobrado
§11.6.1, ponto de partida	com a fórmula do livro, o Nelder-Mead a partir de $(1;\ 0{,}02;\ 0{,}1)$ pode parar longe do ajuste
§11.3.2, dados	a Eonia de 01/10/2014 ($-0{,}032\%$) é usada como r_0 do CIR, que não admite taxas negativas

Onde	O quê
§11.3.2, contagem	o texto chama a Euribor de "30/360", mas o código usa dias corridos/360 (real/360, que é a convenção da Euribor); os prazos das opções usam /365
Eq. (11.7)	$B_t(T) = a\,e^{-b\,\mathbb{E}[r_t]}$ não é o preço esperado do título em t (desigualdade de Jensen; ver Módulo 12)
§11.4.1, H_1	o logaritmo impresso traz $1 - c_1 e^{c_3 T}$; o correto (e o código) é $1 - c_3 e^{c_2 T}$
§11.4.3	a solução ($\kappa_v = 18{,}447$) está na fronteira de Feller: a restrição está ativa
§11.6.2	`options.ix` foi removido do pandas; use `.loc`

9. Exercícios

Exercício 1. No Exemplo 4, suponha que a opção A seja negociada a 6. Que medida Q fica determinada? Qual o preço, por ela, de um derivativo que paga $(0, 0, 10)$?

Solução

$4 + 8\nu = 6 \Rightarrow \nu = 1/4$, logo $Q = (1/4, 1/2, 1/4)$. O derivativo vale $10 \cdot 1/4 = 2{,}5$. Com três ativos linearmente independentes, o mercado ficou completo e o preço é único.

Exercício 2. Mostre que, no CIR, $f(0,0) = r_0$ e $f'(0,0) = \kappa(\theta - r_0)$, e interprete.

Solução

Em $T = 0$: $E = 0$, $D = 2\gamma$, então $f = r_0 \cdot 4\gamma^2/(2\gamma)^2 = r_0$. Derivando em $T = 0$, o primeiro termo dá $2\kappa\theta\gamma/(2\gamma) = \kappa\theta$, e o segundo dá $r_0(\gamma - (\kappa + \gamma)) = -\kappa r_0$. A inclinação inicial da curva forward é o drift esperado da short rate: $\kappa(\theta - r_0)$. Isso também mostra por que a curva curta identifica bem o produto $\kappa\theta$, mas não κ e θ separados.

Exercício 3. Calibre o CIR às **taxas contínuas** $y(t)$ nos 9 prazos, em vez das taxas forward do spline. Compare os parâmetros.

Solução

Minimize $\sum (y_i + \ln B^{CIR}(t_i)/t_i)^2$. Não há spline nem derivada, então não há ruído de interpolação; a derivada do spline amplifica o arredondamento das taxas, sobretudo nas pontas. Os parâmetros ficam próximos, e σ_r continua não identificado.

Exercício 4. Refaça o Laboratório 5 com a função objetivo (11.2). O que acontece com o ajuste das opções de 171 dias?

Solução

O peso migra para as calls baratas de 17 dias e K alto. O ajuste nelas melhora, e o das opções caras, de vega alto, piora em pontos. Se o uso do modelo for precificar algo parecido com as opções no dinheiro de prazo mais longo, (11.2) é a escolha errada.

Exercício 5. Acrescente ao mercado do Laboratório 6 o vencimento de 2 anos (5 strikes) e refaça o perfil de κ_v. O vale fica mais estreito?

Solução

Sim. A reversão à média só se manifesta em prazos comparáveis a $1/\kappa_v$ (um terço de ano para $\kappa_v = 3$). Com 2 anos, a estrutura a termo das volatilidades no dinheiro passa a revelar a velocidade de reversão, e κ_v e θ_v se separam.

```
print(f"Módulo 13 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 13 concluído: 24 verificações aprovadas.
```

Capítulo 14

Calibração II: saltos, regularização e o BCC97 completo

Introdução

O capítulo anterior mostrou que a calibração pode ser instável: pequenas mudanças nos preços de mercado podem mover muito os parâmetros. Na terminologia matemática, ela é um **problema mal posto**. Este capítulo trata do remédio clássico, a **regularização**: acrescentar ao erro de ajuste uma penalidade que prende os parâmetros a uma referência, como a calibração da véspera, trocando um pouco de ajuste por muita estabilidade.

Em seguida, os saltos entram na calibração. Saltos e volatilidade estocástica produzem skews parecidos, mas com assinaturas diferentes ao longo dos prazos: os saltos dominam os vencimentos curtos e a volatilidade estocástica os longos. O capítulo compara a calibração sequencial (um bloco de cada vez) com a calibração conjunta do BCC97 completo.

Por fim, o capítulo mede o **risco de modelo** que sobra depois de uma boa calibração: conjuntos de parâmetros que ajustam o mercado igualmente bem podem dar preços diferentes para contratos que dependem de aspectos que as opções calibradas não revelam.

Fundamentos teóricos

Problemas mal postos e regularização

Definição 14.1 (Problema bem posto). O problema de encontrar ϑ com $F(\vartheta) = y$ é *bem posto* (Hadamard) se, para todo dado y admissível, existe solução, ela é única e depende continuamente de y. Caso contrário, é *mal posto*.

A calibração do Capítulo 13 é mal posta por falta de unicidade (vales de parâmetros quase equivalentes) e de estabilidade (ruído de alguns centavos nos preços move muito $\hat{\vartheta}$). A regularização troca o problema original por um próximo e bem posto (Engl et al., 1996).

Definição 14.2 (Regularização de Tikhonov). Dado um parâmetro de referência ϑ^0 (por exemplo, a calibração da véspera) e $\alpha > 0$,

$$\hat{\vartheta}_\alpha = \arg\min_{\vartheta \in \Theta} \left\{ \mathrm{EQM}(\vartheta) + \alpha \, \|\vartheta - \vartheta^0\|_W^2 \right\}, \tag{14.1}$$

com $\|x\|_W^2 = x^\top W x$ e W diagonal positiva para pôr os parâmetros na mesma escala.

Proposição 14.3 (Efeito da penalidade). *Se* EQM *é duas vezes diferenciável e* $\nabla^2 \mathrm{EQM}(\vartheta) \succeq -\mu I$ *numa vizinhança convexa de* ϑ^0*, então para* $\alpha > \mu / (2\lambda_{\min}(W))$ *o objetivo* (14.1) *é fortemente convexo nessa vizinhança e tem, ali, um único minimizador, que depende continuamente dos dados.*

Demonstração. A Hessiana do objetivo é $\nabla^2 \mathrm{EQM} + 2\alpha W \succeq (2\alpha\lambda_{\min}(W) - \mu) I \succ 0$. Funções fortemente convexas têm minimizador único, e a estabilidade decorre do teorema da função implícita aplicado à condição de primeira ordem. ∎

Observação 14.4 (Escolha de α). α grande demais prende o modelo em ϑ^0 (viés); pequeno demais não estabiliza (variância). O *princípio da discrepância* de Morozov escolhe o maior α cujo erro de ajuste ainda é comparável ao ruído dos dados, por exemplo à metade do spread de compra e venda.

Identificação de saltos

Proposição 14.5 (Cumulantes). *Para o log-retorno* X_T *de um processo de Lévy, os cumulantes são lineares no prazo,* $c_n(X_T) = T\, c_n(X_1)$*. Em particular, a assimetria decai como* $T^{-1/2}$ *e o excesso de curtose como* T^{-1}*.*

A proposição explica por que o smile de prazos curtos é a principal fonte de informação sobre os saltos: com o prazo, a lei do log-retorno se aproxima da normal e o efeito dos saltos sobre as vols implícitas se dilui. Em Heston, ao contrário, a assimetria cresce com o prazo nos primeiros meses, porque a correlação precisa de tempo para agir (Gatheral, 2006). Calibrar BCC97 a prazos curtos e longos ao mesmo tempo usa essa diferença para separar os dois mecanismos.

Calibração sequencial e conjunta

Definição 14.6 (Estratégias). Na *calibração sequencial*, calibra-se primeiro um submodelo (por exemplo, Heston aos prazos longos) e, com seus parâmetros fixos, o bloco restante (saltos aos prazos curtos). Na *calibração conjunta*, todos os parâmetros são ajustados juntos, usando o resultado sequencial como ponto de partida.

Observação 14.7 (Risco de modelo após a calibração). Mesmo bem calibrados, conjuntos de parâmetros diferentes dentro do vale de soluções quase ótimas dão preços diferentes para produtos que dependem de outros aspectos da lei conjunta, como a variância futura ou o smile a termo. Medir essa dispersão (recalibrando com restrições, como fixar κ_v em valores alternativos) é uma forma direta de quantificar risco de modelo.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulo 11 do livro, §11.4.4 a §11.6.6.

O que você vai conseguir fazer ao final

1. Explicar por que calibrar saltos é um problema mal posto e o que a regularização de Tikhonov faz.
2. Distinguir a penalidade quadrática da penalidade em norma do livro, e prever quando a segunda congela os parâmetros.
3. Reproduzir o roteiro sequencial do livro (Heston, saltos, BCC97 completo) e ver onde ele falha.
4. Mapear a degenerescência entre intensidade e tamanho dos saltos.
5. Medir o efeito da regularização na estabilidade de recalibrações.
6. Calibrar a volatilidades implícitas e entender por que (11.4) quase não muda nada.
7. Medir o risco de modelo que sobra depois de uma calibração bem-sucedida.

Onde estamos

O Módulo 13 calibrou o CIR e o Heston, e deixou dois recados: o resíduo do Heston se concentra no prazo curto, onde os saltos deveriam ajudar, e nem todo parâmetro é identificado pelos dados. Este módulo completa o capítulo 11 com os saltos e o modelo geral, no mesmo mercado sintético.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.interpolate as sci
from scipy.integrate import simpson
from scipy.optimize import brentq, brute, fmin, linprog
from scipy.stats import norm
from IPython.display import display

RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

def bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - sig * np.sqrt(T))

def vega_bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.pdf(d1) * np.sqrt(T)

def vol_impl(preco, S, K, T, r):
    piso = max(S - K * np.exp(-r * T), 0.0)
    if not (piso + 1e-12 < preco < S):
        return np.nan
    return brentq(lambda s: float(bsm(S, K, T, r, s)) - preco, 1e-4, 4.0)

def cf_heston(u, T, v0, kappa, theta, sigma, rho):
    """Função característica de ln(S_T/S_0) - rT no Heston, forma estável (Módulo 11)."""
    xi = kappa - rho * sigma * 1j * u
    d = np.sqrt(xi**2 + sigma**2 * (1j * u + u**2))
```

```
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
    A = kappa * theta / sigma**2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / sigma**2 * (1 - e) / (1 - g * e)
    return np.exp(A + B * v0)

def cf_saltos(u, T, lam, mu, delta):
    """Parte de saltos de Merton, com o drift compensado (11.13)-(11.14)."""
    omega = -lam * (np.exp(mu + 0.5 * delta**2) - 1)
    return np.exp((1j * u * omega + lam * (np.exp(1j * u * mu - 0.5 * u**2 * delta**2) - 1)) * T)

U_FINO = np.linspace(0, 2000, 200_001)
U_CAL = np.linspace(0, 300, 3001)

def calls(S0, K, T, r, heston, saltos=None, u=U_CAL):
    """Calls europeias pela fórmula de Lewis (11.12), vetorizada em (K, T, r) do mesmo tamanho.
    heston = (v0, κ_v, θ_v, σ_v, ρ); saltos = (λ, μ_J, δ) ou None."""
    K, T, r = (np.atleast_1d(np.asarray(x, float)) for x in (K, T, r))
    k = np.log(S0 / K) + r * T
    phi = cf_heston(u[None, :] - 0.5j, T[:, None], *heston)
    if saltos is not None:
        phi = phi * cf_saltos(u[None, :] - 0.5j, T[:, None], *saltos)
    h = np.exp(1j * u[None, :] * k[:, None]) * phi
    integral = simpson(h.real / (u**2 + 0.25), x=u, axis=1)
    return S0 - np.sqrt(S0 * K) * np.exp(-0.5 * r * T) / np.pi * integral

def zcb_cir(r0, kappa, theta, sigma, T):
    """Zero-cupom no CIR, (11.7)-(11.9) com t = 0."""
    g = np.sqrt(kappa**2 + 2 * sigma**2)
    e = np.exp(g * T) - 1
    D = 2 * g + (kappa + g) * e
    return (2 * g * np.exp((kappa + g) * T / 2) / D) ** (2 * kappa * theta / sigma**2) * np.exp(-2
    ↪  * e / D * r0)

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

O mercado sintético do Módulo 13

As mesmas 15 calls: $S_0 = 3.225{,}93$; 17, 80 e 171 dias; strikes de 3.000 a 3.400. O modelo verdadeiro é o Bates ($v_0 = 0{,}03$, $\kappa_v = 3$, $\theta_v = 0{,}04$, $\sigma_v = 0{,}5$, $\rho = -0{,}7$, $\lambda = 0{,}15$, $\mu_J = -0{,}15$, $\delta = 0{,}1$), e as cotações são arredondadas a 0,1 ponto. Os juros vêm da curva CIR; no Módulo 13, a calibração com a fórmula correta reproduziu a curva com erro abaixo de 0,1 pb, então usamos a curva verdadeira.

Para não depender do Módulo 13, refazemos aqui a calibração do Heston (grade do livro + Nelder-Mead), que é o ponto de partida do livro para os saltos.

Um detalhe prático aparece já aqui. Com os juros da curva verdadeira em vez dos da curva calibrada (diferença abaixo de 0,1 pb), o Nelder-Mead do mesmo ponto da grade **para** num EQM de 0,8, e não de 0,09 como no Módulo 13. O simplex encosta na parede de Feller (onde a função vale 500), se achata e declara convergência. A correção usual é **reiniciar** a busca a partir do ponto encontrado, com um simplex novo. Fazemos isso abaixo.

```
S0 = 3225.93
DIAS = np.array([17, 80, 171])
STRIKES = np.arange(3000, 3401, 100.0)
T_OP = np.repeat(DIAS / 365, 5)
K_OP = np.tile(STRIKES, 3)
CURTO = T_OP == T_OP.min()
CIR_VERDADE = dict(r0=0.02, kappa=0.5, theta=0.045, sigma=0.08)
R_OP = -np.log(zcb_cir(T=T_OP, **CIR_VERDADE)) / T_OP

HESTON_VERDADE = (0.03, 3.0, 0.04, 0.5, -0.7)          # (v0, κ_v, θ_v, σ_v, ρ)
SALTOS_VERDADE = (0.15, -0.15, 0.10)                   # (λ, μ_J, δ)
C_JUSTO = calls(S0, K_OP, T_OP, R_OP, HESTON_VERDADE, SALTOS_VERDADE, U_FINO)
C_MERC = np.round(C_JUSTO, 1)
IV_MERC = np.array([vol_impl(c, S0, k, t, r) for c, k, t, r in zip(C_MERC, K_OP, T_OP, R_OP)])
VEGA = vega_bsm(S0, K_OP, T_OP, R_OP, IV_MERC)

# Vetor completo do BCC97 (com juros por vencimento): p = (κ_v, θ_v, σ_v, ρ, v0, λ, μ_J, δ)
NOMES = ["κ_v", "θ_v", "σ_v", "ρ", "v0", "λ", "μ_J", "δ"]
VERDADE = np.array([3.0, 0.04, 0.5, -0.7, 0.03, 0.15, -0.15, 0.10])


def admissivel(p, feller):
    kappa, theta, sigma, rho, v0, lam, mu, delta = p
    if (kappa < 0 or theta < 0.005 or sigma < 0 or abs(rho) > 1 or v0 < 0
            or lam < 0 or mu < -0.6 or mu > 0 or delta < 0):
        return False
    return not (feller and 2 * kappa * theta < sigma**2)


def precos_bcc(p, mascara=slice(None), u=U_CAL):
    kappa, theta, sigma, rho, v0, lam, mu, delta = p
    return calls(S0, K_OP[mascara], T_OP[mascara], R_OP[mascara], (v0, kappa, theta, sigma, rho),
     ↪  (lam, mu, delta), u)


def eqm_bcc(p, feller=False, alvo=C_MERC):
    """Função objetivo (11.1) do BCC97 completo; 5000 fora da região admissível, como no livro."""
    if not admissivel(p, feller):
        return 5000.0
    return np.mean((precos_bcc(p) - alvo) ** 2)


# Heston calibrado como no Módulo 13 (com Feller, como no livro)
def eqm_h93(q):
    return eqm_bcc(np.r_[q, 0.0, -0.1, 0.0], feller=True)


t0 = time.time()
q0 = brute(eqm_h93, ((2.5, 10.6, 5.0), (0.01, 0.041, 0.01), (0.05, 0.251, 0.1),
                     (-0.75, 0.01, 0.25), (0.01, 0.031, 0.01)), finish=None)
H93_1 = fmin(eqm_h93, q0, xtol=1e-6, ftol=1e-6, maxiter=750, maxfun=900, disp=False)
H93_CAL = fmin(eqm_h93, H93_1, xtol=1e-6, ftol=1e-6, maxiter=750, maxfun=900, disp=False)   #
 ↪  reinício
print("primeira busca local:", [f3(x) for x in H93_1], "| EQM", f3(eqm_h93(H93_1)))
print(f"Heston calibrado em {time.time() - t0:.1f} s:", dict(zip(NOMES[:5], [f3(x) for x in
 ↪  H93_CAL])),
      "| EQM", f3(eqm_h93(H93_CAL)))
ok(eqm_h93(H93_1) > 0.5, "a primeira busca local para cedo, presa na parede de Feller")
ok(eqm_h93(H93_CAL) < 0.15, "depois do reinício, o Heston reproduz o resultado do Módulo 13 (EQM
 ↪  perto de 0,09)")
```

```
primeira busca local: ['1,203', '0,061', '0,382', '-0,848', '0,032'] | EQM 0,808
Heston calibrado em 19.0 s: {'κ_v': '4,118', 'θ_v': '0,045', 'σ_v': '0,609', 'ρ': '-0,744', 'v0': '0,033'} |
↪  EQM 0,093
OK — a primeira busca local para cedo, presa na parede de Feller
OK — depois do reinício, o Heston reproduz o resultado do Módulo 13 (EQM perto de 0,09)
```

---

1. Problemas mal postos e regularização

Livro: §11.4.4

1.1 Os dois problemas

O livro, citando Galluccio e Le Cam (2008), aponta dois defeitos da calibração de modelos com saltos:

- **Degenerescência**: combinações diferentes de parâmetros dão o mesmo erro. Exemplo típico: muitos saltos pequenos ou poucos saltos grandes produzem skews parecidos no prazo curto.
- **Indeterminação**: a função objetivo não é convexa e tem vários mínimos locais.

Juntos, fazem o problema **mal posto** no sentido de Hadamard: a solução pode não ser única e pode não depender continuamente dos dados. Na prática, recalibrar no dia seguinte, com cotações quase iguais, pode fazer os parâmetros saltarem, e com eles as posições de hedge.

1.2 Tikhonov

A regularização de Tikhonov acrescenta à função objetivo um termo que puxa a solução para uma referência p_0 (a calibração anterior, ou o resultado da busca global):

$$\min_{p} \; F(p) + w\,\|p - p_0\|^2. \tag{14.2}$$

O termo quadrático é suave e estritamente convexo. Para w grande o bastante, torna o problema convexo perto de p_0, e a solução passa a depender continuamente dos dados.

1.3 O que o livro faz de fato

A penalidade (11.17) é $\sqrt{\sum (p_0 - p)^2} = \|p - p_0\|$: a **norma**, não o quadrado. A diferença não é cosmética.

Passo 1. Em $p = p_0$, a norma tem um bico: não é diferenciável. Seu subgradiente é a bola $\{g : \|g\| \le 1\}$.

Passo 2. p_0 é mínimo de $F(p) + w\|p - p_0\|$ se e só se $0 \in \nabla F(p_0) + w\,\{g : \|g\| \le 1\}$, isto é, se

$$\|\nabla F(p_0)\| \le w. \tag{14.3}$$

Passo 3. Interpretação: a norma é uma **penalidade exata**. Enquanto a inclinação da função objetivo em p_0 for menor que w, os parâmetros **não saem do lugar**. Com o EQM em pontos^2 da ordem de 10^{-2} e $w = 1$, essa é a situação típica: a "calibração local" devolve o ponto de partida.

Laboratório 1 — As duas penalidades num exemplo de uma dimensão

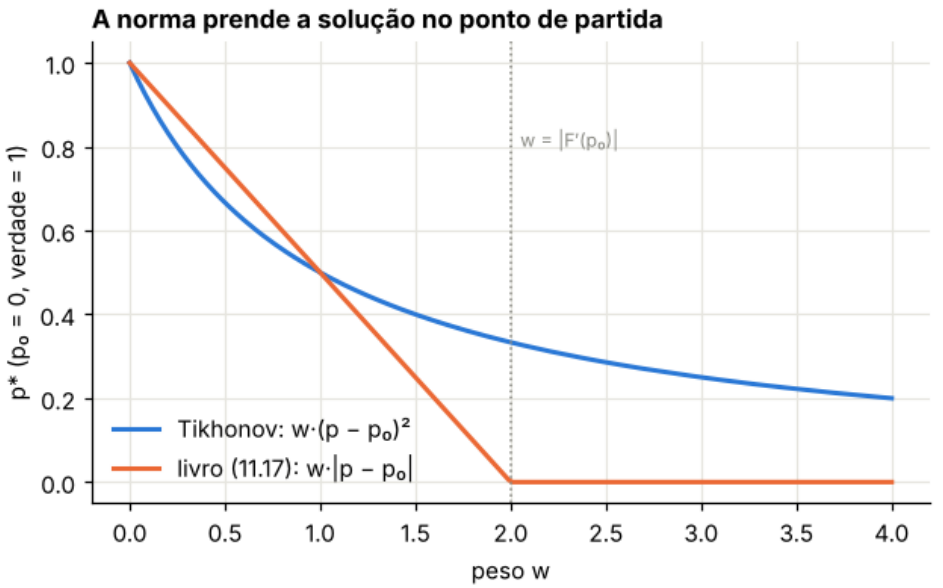
Com $F(p) = (p-1)^2$ e $p_0 = 0$:

- quadrática: $\min (p-1)^2 + wp^2 \Rightarrow p^* = 1/(1+w)$, que se aproxima de zero devagar;
- norma: $\min (p-1)^2 + w|p| \Rightarrow p^* = \max(0,\ 1 - w/2)$, que chega a zero e para lá a partir de $w = 2 = |F'(0)|$.

```
ws = np.linspace(0, 4, 200)
grade_p = np.linspace(-0.5, 1.5, 20001)
p_quad = [grade_p[np.argmin((grade_p - 1) ** 2 + w * grade_p**2)] for w in ws]
p_norm = [grade_p[np.argmin((grade_p - 1) ** 2 + w * np.abs(grade_p))] for w in ws]

fig, ax = plt.subplots(figsize=(6.5, 3.6))
ax.plot(ws, p_quad, color=AZUL, label="Tikhonov: w·(p − p₀)²")
ax.plot(ws, p_norm, color=LARANJA, label="livro (11.17): w·|p − p₀|")
ax.axvline(2, color=CINZA, ls=":", lw=1)
ax.text(2.05, 0.8, "w = |F′(p₀)|", color=CINZA, fontsize=8)
ax.set_xlabel("peso w"); ax.set_ylabel("p* (p₀ = 0, verdade = 1)")
ax.set_title("A norma prende a solução no ponto de partida")
ax.legend(); plt.show()

ok(np.allclose(p_quad, 1 / (1 + ws), atol=2e-4), "Tikhonov: p* = 1/(1+w)")
ok(np.allclose(p_norm, np.maximum(0, 1 - ws / 2), atol=2e-4), "norma: p* = max(0, 1 − w/2)")
ok(np.all(np.array(p_norm)[ws >= 2] == 0), "com w ≥ |F'(p₀)| a solução com a norma fica
↪  exatamente em p₀")
```




```
OK — Tikhonov: p* = 1/(1+w)
OK — norma: p* = max(0, 1 − w/2)
OK — com w ≥ |F'(p₀)| a solução com a norma fica exatamente em p₀
```

2. Calibrar só os saltos

Livro: §11.4.4 e §11.6.4

2.1 O roteiro do livro

1. Fixe os parâmetros de volatilidade estocástica no resultado do Heston.
2. Calibre (λ, μ_J, δ) às 5 opções de 17 dias: grade ($\lambda \in \{0; 0{,}1; \ldots; 0{,}5\}$, $\mu_J \in \{-0{,}5; \ldots; -0{,}2\}$, $\delta \in \{0;\ 0{,}25;\ 0{,}5\}$), depois Nelder-Mead **com a penalidade** (11.17) em torno do ponto da grade.

Restrições: $\lambda \geq 0$, $-0{,}6 \leq \mu_J \leq 0$, $\delta \geq 0$.

2.2 Um detalhe de Python: a penalidade nunca é aplicada

No script da §11.6.4, a função objetivo lê duas variáveis globais, `local_opt` (começa `False`) e `opt1`. A função de calibração faz `opt1 = brute(...)` e depois `local_opt = True`, **sem** declarar `global`. Em Python, atribuir a um nome dentro de uma função cria uma variável **local**. A global `local_opt` continua `False`, e o ramo da penalidade nunca roda. (Se rodasse, `opt1` global nem existiria.)

Laboratório 2a — O escopo das variáveis, em miniatura

```
local_opt = False      # global, como no script do livro
chamadas = []


def erro_livro(p):
    chamadas.append(local_opt)          # o que a função objetivo enxerga
    return p**2


def calibracao_livro():
    opt1 = 0.5                          # local
    local_opt = True                    # local! não muda a global
    return erro_livro(opt1)


calibracao_livro()
print("local_opt visto pela função objetivo:", chamadas)
ok(chamadas == [False], "a atribuição sem 'global' não liga a penalidade: o livro calibra sem
↪  regularização")
```

```
local_opt visto pela função objetivo: [False]
OK — a atribuição sem 'global' não liga a penalidade: o livro calibra sem regularização
```

Laboratório 2b — Os saltos sobre o Heston calibrado

Seguimos o roteiro do livro: volatilidade estocástica fixa no Heston calibrado, grade e Nelder-Mead nas 5 opções de 17 dias. Como a penalidade do livro não roda, rodamos sem ela.

```python
def eqm_saltos(j, sv, alvo, pen=None, w=0.0, ref=None):
    lam, mu, delta = j
    if lam < 0 or mu < -0.6 or mu > 0 or delta < 0:
        return 5000.0
    e = np.mean((precos_bcc(np.r_[sv, j], CURTO) - alvo) ** 2)
    if pen is not None:
        d = np.asarray(j) - ref
        e += w * (np.sqrt(d @ d) if pen == "norma" else d @ d)
    return e


GRADE_SALTOS = ((0.0, 0.51, 0.1), (-0.5, -0.11, 0.1), (0.0, 0.51, 0.25))
j0 = brute(eqm_saltos, GRADE_SALTOS, args=(H93_CAL, C_MERC[CURTO]), finish=None)
j_seq = fmin(eqm_saltos, j0, args=(H93_CAL, C_MERC[CURTO]), xtol=1e-7, ftol=1e-7, maxiter=550,
 ↪  maxfun=750, disp=False)
sem = eqm_saltos([0.0, -0.1, 0.0], H93_CAL, C_MERC[CURTO])
print("melhor ponto da grade:", [f3(x) for x in j0], "| após Nelder-Mead:", [f3(x) for x in
 ↪  j_seq])
print("EQM dos 17 dias — sem saltos:", f3(sem), "| com os saltos calibrados:",
 ↪  f3(eqm_saltos(j_seq, H93_CAL, C_MERC[CURTO])))
ok(j_seq[0] * abs(j_seq[1]) < 0.01, "os saltos calibrados são desprezíveis: λ·|μ_J| < 0,01")
ok(eqm_saltos(j_seq, H93_CAL, C_MERC[CURTO]) > 0.9 * sem, "e quase não melhoram o ajuste dos 17
 ↪  dias")
```

```
melhor ponto da grade: ['0,000', '-0,500', '0,000'] | após Nelder-Mead: ['0,000', '-0,506', '0,000']
EQM dos 17 dias — sem saltos: 0,224 | com os saltos calibrados: 0,224
OK — os saltos calibrados são desprezíveis: λ·|μ_J| < 0,01
OK — e quase não melhoram o ajuste dos 17 dias
```

O mercado tem saltos ($\lambda = 0{,}15$, $\mu_J = -0{,}15$), mas a calibração sequencial não os encontra. O motivo: ao calibrar o Heston sozinho, σ_v e ρ já foram esticados para produzir o skew curto. Sobre esse Heston, qualquer salto adicional piora o ajuste.

O livro chega ao mesmo lugar com dados reais: $\lambda = 0{,}008$, $\mu_J = -0{,}600$ (o limite inferior) e $\delta = 0{,}001$, ou seja, um salto de -45% a cada 125 anos. Na prática, nenhum salto.

---

3. O BCC97 completo

Livro: §11.4.5 e §11.6.5

O último passo do livro é uma busca local nos 8 parâmetros, partindo de (Heston calibrado, saltos calibrados), com a condição de Feller. Como a partida já é um mínimo local razoável, o Nelder-Mead tende a ficar perto dela.

Testamos três pontos de partida, com e sem Feller:

- **sequencial (livro)**: Heston calibrado e os saltos da Seção 2;
- **chute com saltos**: o mesmo Heston, com saltos moderados $(0{,}2;\ -0{,}1;\ 0{,}1)$;
- **verdade**: só para referência.

Laboratório 3 — Partidas, Feller e mínimos locais

```
partidas = {"sequencial (livro)": np.r_[H93_CAL, j_seq],
            "chute com saltos": np.r_[H93_CAL, 0.2, -0.1, 0.1],
            "verdade": VERDADE}
t0 = time.time()
resultados = {}
for feller in (True, False):
    for nome, p_ini in partidas.items():
        if nome == "verdade" and feller:
            continue
        p = fmin(eqm_bcc, p_ini, args=(feller,), xtol=1e-6, ftol=1e-6, maxiter=2000, maxfun=3000,
 ↪  disp=False)
        resultados[(nome, "com Feller" if feller else "sem Feller")] = p
print(f"tempo: {time.time() - t0:.1f} s")
tab3 = pd.DataFrame({f"{a} / {b}": p for (a, b), p in resultados.items()}, index=NOMES).T
tab3["EQM"] = [eqm_bcc(p) for p in resultados.values()]
tab3.loc["verdade (EQM só do arredondamento)"] = list(VERDADE) + [eqm_bcc(VERDADE)]
tabela(tab3)
```

```
tempo: 103.2 s
```

	κ_v	θ_v	σ_v	ρ	v0	λ	μ_J	δ	EQM
sequencial (livro) / com Feller	4,119	0,045	0,609	-0,744	0,033	0,000	-0,592	0,000	0,093
chute com saltos / com Feller	4,087	0,036	0,463	-0,781	0,030	0,090	-0,206	0,183	0,004
sequencial (livro) / sem Feller	4,119	0,045	0,609	-0,744	0,033	0,000	-0,518	0,000	0,093
chute com saltos / sem Feller	2,845	0,040	0,500	-0,702	0,030	0,223	-0,105	0,109	0,001
verdade / sem Feller	2,888	0,040	0,498	-0,699	0,030	0,191	-0,124	0,105	0,000
verdade (EQM só do arredondamento)	3,000	0,040	0,500	-0,700	0,030	0,150	-0,150	0,100	0,001

```
P_SEQ = resultados[("sequencial (livro)", "com Feller")]
P_JOINT = resultados[("chute com saltos", "sem Feller")]
eqm_por_prazo = pd.DataFrame({
    "Heston (Módulo 13)": [np.mean((precos_bcc(np.r_[H93_CAL, 0, -0.1, 0])[T_OP == d / 365] -
    ↪  C_MERC[T_OP == d / 365]) ** 2) for d in DIAS],
    "BCC97 sequencial, com Feller": [np.mean((precos_bcc(P_SEQ)[T_OP == d / 365] - C_MERC[T_OP ==
    ↪  d / 365]) ** 2) for d in DIAS],
    "BCC97 com saltos, sem Feller": [np.mean((precos_bcc(P_JOINT)[T_OP == d / 365] - C_MERC[T_OP
    ↪  == d / 365]) ** 2) for d in DIAS],
}, index=[f"{d} dias" for d in DIAS])
tabela(eqm_por_prazo)

e = tab3["EQM"]
ok(e["chute com saltos / sem Feller"] < e["sequencial (livro) / com Feller"] / 10,
   "partindo com saltos e sem Feller, o EQM cai mais de 10 vezes em relação ao roteiro do livro")
ok(e["chute com saltos / sem Feller"] < 2 * e["verdade (EQM só do arredondamento)"],
   "e chega ao nível do erro de arredondamento das cotações")
ok(e["sequencial (livro) / sem Feller"] > 5 * e["chute com saltos / sem Feller"],
   "mesmo sem Feller, a partida sequencial para num mínimo local pior")
ok(abs(P_JOINT[0] - 3) < 0.5 and abs(P_JOINT[2] - 0.5) < 0.05, "o ajuste conjunto recupera κ_v e
 ↪  σ_v aproximadamente")
```

	Heston (Módulo 13)	BCC97 sequencial, com Feller	BCC97 com saltos, sem Feller
17 dias	0,224	0,224	0,001
80 dias	0,043	0,043	0,000
171 dias	0,011	0,011	0,000

```
OK — partindo com saltos e sem Feller, o EQM cai mais de 10 vezes em relação ao roteiro do livro
OK — e chega ao nível do erro de arredondamento das cotações
OK — mesmo sem Feller, a partida sequencial para num mínimo local pior
OK — o ajuste conjunto recupera κ_v e σ_v aproximadamente
```

O que aprendemos.

- O roteiro sequencial do livro (Heston primeiro, saltos depois, busca local no fim) prende a calibração num mínimo local: os saltos nunca entram de verdade. O resultado ($\lambda = 0$, μ_J encostado no limite de $-0{,}6$, $\delta = 0$) tem a mesma cara do resultado do livro com dados reais.
- Com uma partida que já tem saltos, e sem a restrição de Feller, o ajuste chega ao nível do ruído de arredondamento das cotações e recupera os parâmetros de volatilidade.
- O melhor antídoto contra a indeterminação é barato: **várias partidas** e comparar os mínimos. Com 8 parâmetros, uma única busca local não basta.
- Os resultados do livro mostram os sinais desse problema: no BCC97 completo, $\rho = -0{,}999$ e $\delta = 0{,}000$ (dois parâmetros na fronteira) e $\lambda = 0{,}008$.

3.1 A degenerescência dos saltos

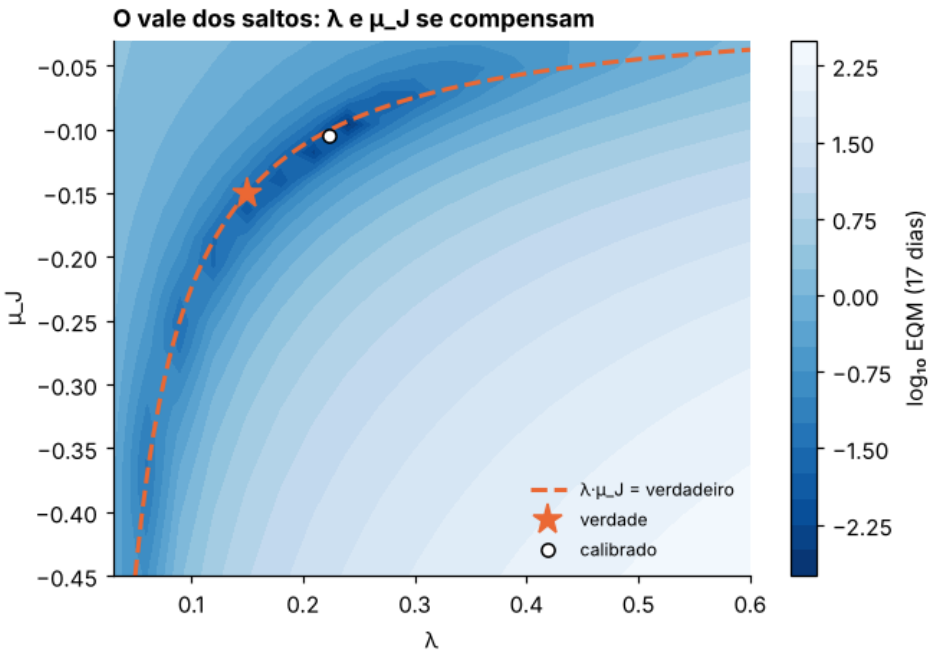
Mesmo no ajuste conjunto, λ e μ_J não são recuperados separadamente com precisão. No prazo curto, o skew depende sobretudo do **produto**: a média dos saltos por ano, $\lambda\,(e^{\mu_J+\delta^2/2}-1)$, que é a correção de drift $-\omega$ de (11.14). O mapa do erro (com δ fixo no valor calibrado) mostra um vale estreito ao longo de $\lambda\mu_J \approx$ constante. Chamamos de "fundo do vale" os pontos com EQM abaixo de 0,05, um erro típico de 0,2 ponto (dois ticks), bem menor que o 0,22 que o Heston sem saltos deixa nesse prazo.

```
SV_J = P_JOINT[:5]
lams = np.linspace(0.03, 0.6, 20)
mus = np.linspace(-0.45, -0.03, 20)
Z = np.array([[eqm_saltos([l, m, P_JOINT[7]], SV_J, C_MERC[CURTO]) for l in lams] for m in mus])

fig, ax = plt.subplots(figsize=(6.5, 4.4))
cs = ax.contourf(lams, mus, np.log10(Z), levels=20, cmap="Blues_r")
plt.colorbar(cs, ax=ax, label="log₁₀ EQM (17 dias)")
lin = np.linspace(0.05, 0.6, 100)
ax.plot(lin, VERDADE[5] * VERDADE[6] / lin, "--", color=LARANJA, label="λ·μ_J = verdadeiro")
ax.plot(*VERDADE[5:7], "*", color=LARANJA, ms=14, label="verdade")
ax.plot(*P_JOINT[5:7], "o", color="white", mec="black", label="calibrado")
ax.set_ylim(mus[0], mus[-1]); ax.set_xlabel("λ"); ax.set_ylabel("μ_J")
ax.set_title("O vale dos saltos: λ e μ_J se compensam")
ax.legend(fontsize=8, loc="lower right"); plt.show()

i_min = np.unravel_index(Z.argmin(), Z.shape)
fundo = Z < 0.05                        # erro quadrático médio de ~0,2 ponto (dois ticks)
prod = (lams[None, :] * mus[:, None])[fundo]
print("pontos no fundo do vale (EQM < 0,05):", int(fundo.sum()),
      "| λ entre", f3(np.broadcast_to(lams, Z.shape)[fundo].min()), "e", f3(np.broadcast_to(lams,
      ↪  Z.shape)[fundo].max()),
      "| λ·μ_J entre", f3(prod.min()), "e", f3(prod.max()))
ok(np.broadcast_to(lams, Z.shape)[fundo].max() > 2 * np.broadcast_to(lams, Z.shape)[fundo].min(),
   "no fundo do vale, λ varia por um fator maior que 2")
lam_f = np.broadcast_to(lams, Z.shape)[fundo]
disp_lam = (lam_f.max() - lam_f.min()) / lam_f.mean()
```

```
disp_prod = (prod.max() - prod.min()) / abs(prod.mean())
print("amplitude relativa — λ:", f3(disp_lam), "| λ·μ_J:", f3(disp_prod))
ok(disp_prod < disp_lam / 3, "enquanto o produto λ·μ_J varia, em termos relativos, menos de um
↪  terço")
```




```
pontos no fundo do vale (EQM < 0,05): 15 | λ entre 0,060 e 0,300 | λ·μ_J entre -0,026 e -0,020
OK — no fundo do vale, λ varia por um fator maior que 2
amplitude relativa — λ: 1,364 | λ·μ_J: 0,260
OK — enquanto o produto λ·μ_J varia, em termos relativos, menos de um terço
```

---

4. Estabilidade: recalibrar com cotações que mudaram pouco

É aqui que a regularização mostra seu valor. Simulamos 8 “dias seguintes”: às cotações justas de 17 dias somamos um ruído normal de desvio 0,1 ponto (um tick) e arredondamos. Recalibramos os saltos, com a volatilidade estocástica fixa no ajuste conjunto, partindo da calibração de “hoje”:

- sem penalidade;
- com Tikhonov, $w \in \{0{,}01;\ 0{,}1;\ 1\}$;
- com a norma do livro, $w = 1$.

Medimos a dispersão dos parâmetros entre os 8 dias e o EQM médio.

Laboratório 4 — O preço da estabilidade

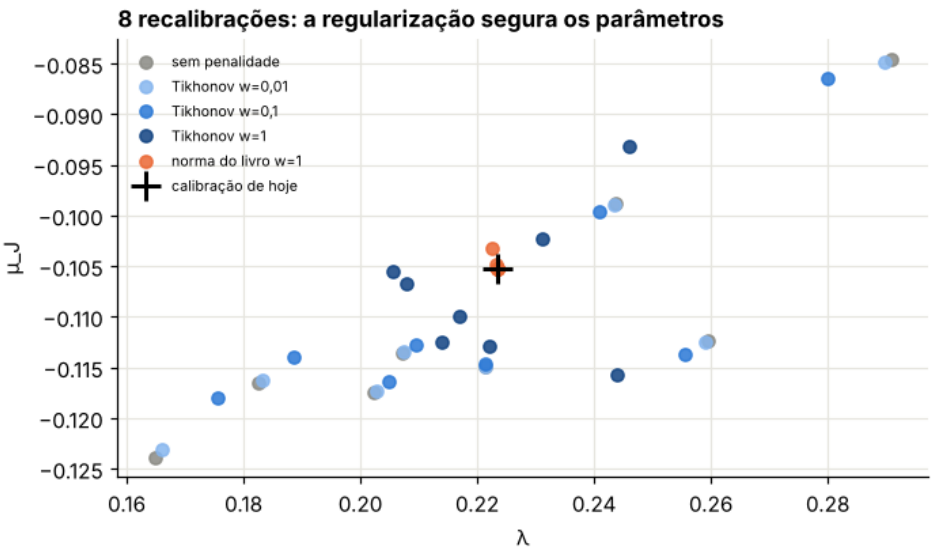
```
rng = np.random.default_rng(1)
J_HOJE = P_JOINT[5:]
alvos = [np.round(C_JUSTO[CURTO] + rng.normal(0, 0.1, 5), 1) for _ in range(8)]
t0 = time.time()
linhas, dispersao = [], {}
for nome, pen, w in [("sem penalidade", None, 0.0), ("Tikhonov w=0,01", "quad", 0.01),
                     ("Tikhonov w=0,1", "quad", 0.1), ("Tikhonov w=1", "quad", 1.0),
                     ("norma do livro w=1", "norma", 1.0)]:
    sols, erros = [], []
    for a in alvos:
        j = fmin(eqm_saltos, J_HOJE, args=(SV_J, a, pen, w, J_HOJE), xtol=1e-7, ftol=1e-9,
                 maxiter=1500, maxfun=2000, disp=False)
        sols.append(j)
        erros.append(eqm_saltos(j, SV_J, a))
    sols = np.array(sols)
    dispersao[nome] = sols
    linhas.append((nome, *sols.std(axis=0), np.mean(erros)))
tab4 = pd.DataFrame(linhas, columns=["regularização", "desvio de λ", "desvio de μ_J", "desvio de
↪  δ", "EQM médio"])
tabela(tab4)
print(f"tempo: {time.time() - t0:.1f} s")

fig, ax = plt.subplots(figsize=(7, 3.8))
for (nome, sols), cor in zip(dispersao.items(), [CINZA, RAMPA[0], RAMPA[1], RAMPA[2], LARANJA]):
    ax.plot(sols[:, 0], sols[:, 1], "o", color=cor, label=nome, alpha=0.85)
ax.plot(*J_HOJE[:2], "k+", ms=14, mew=2, label="calibração de hoje")
ax.set_xlabel("λ"); ax.set_ylabel("μ_J")
ax.set_title("8 recalibrações: a regularização segura os parâmetros")
ax.legend(fontsize=7); plt.show()

t4 = tab4.set_index("regularização")
ok(t4.loc["Tikhonov w=1", "desvio de λ"] < t4.loc["sem penalidade", "desvio de λ"] / 2,
   "Tikhonov com w = 1 reduz a dispersão de λ a menos da metade")
ok(t4.loc["Tikhonov w=1", "EQM médio"] < 1.5 * t4.loc["sem penalidade", "EQM médio"],
   "pagando menos de 50% a mais de EQM")
ok(t4.loc["norma do livro w=1", "desvio de λ"] < 0.005, "a norma do livro, com w = 1,
↪  praticamente não deixa os parâmetros se moverem")
ok(t4.loc["norma do livro w=1", "EQM médio"] > 1.2 * t4.loc["Tikhonov w=1", "EQM médio"],
   "e o ajuste piora mais de 20% em relação a Tikhonov: a calibração vira 'repetir ontem'")
```

regularização	desvio de λ	desvio de μ_J	desvio de δ	EQM médio
sem penalidade	0,039	0,012	0,026	0,008
Tikhonov w=0,01	0,038	0,012	0,026	0,008
Tikhonov w=0,1	0,033	0,010	0,024	0,008
Tikhonov w=1	0,015	0,007	0,016	0,008
norma do livro w=1	0,000	0,001	0,000	0,011

```
tempo: 28.3 s
```




```
OK — Tikhonov com w = 1 reduz a dispersão de λ a menos da metade
OK — pagando menos de 50% a mais de EQM
OK — a norma do livro, com w = 1, praticamente não deixa os parâmetros se moverem
OK — e o ajuste piora mais de 20% em relação a Tikhonov: a calibração vira 'repetir ontem'
```

O que aprendemos. A regularização é uma troca explícita entre estabilidade e ajuste, controlada por w. Sem ela, um tick de ruído move λ e μ_J ao longo do vale da Seção 3.1. Com Tikhonov e w moderado, os parâmetros ficam perto dos de ontem, e o ajuste piora pouco. Com a norma do livro e $w = 1$, a penalidade exata da Seção 1.3 entra em ação: a calibração devolve o ponto de partida. A escolha de w deve ser documentada e testada, como qualquer outro parâmetro do modelo.

---

5. Calibrar a volatilidades implícitas

Livro: §11.4.6 e §11.6.6

O livro recalibra o BCC97 com a função (11.4), o erro de volatilidade ponderado pelo vega, partindo do ajuste em preços. Cada avaliação exige inverter o BSM 15 vezes.

Pelo Módulo 13, Seção 2.1, (11.4) é (11.1) a menos de termos de segunda ordem. Calibrar a (11.4) deveria, portanto, dar quase o mesmo resultado. Para ver uma diferença real, comparamos também com (11.3), o erro de volatilidade **sem** peso, que trata igualmente todos os pontos da superfície.

Laboratório 5 — (11.1), (11.4) e (11.3)

```python
def ivs_modelo(p):
    C = precos_bcc(p)
    return np.array([vol_impl(c, S0, k, t, r) for c, k, t, r in zip(C, K_OP, T_OP, R_OP)])
```

```
def erro_vol(p, ponderado):
    if not admissivel(p, False):
        return 5000.0
    iv = ivs_modelo(p)
    if np.isnan(iv).any():
        return 5000.0
    d = iv - IV_MERC
    return np.mean((d * VEGA) ** 2) if ponderado else np.mean((100 * d) ** 2)


t0 = time.time()
P_114 = fmin(erro_vol, P_JOINT, args=(True,), xtol=1e-6, ftol=1e-8, maxiter=450, maxfun=650,
 ↪  disp=False)
P_113 = fmin(erro_vol, P_JOINT, args=(False,), xtol=1e-6, ftol=1e-8, maxiter=450, maxfun=650,
 ↪  disp=False)
print(f"tempo: {time.time() - t0:.1f} s")

linhas = []
for nome, p in [("preços (11.1)", P_JOINT), ("vol. × vega (11.4)", P_114), ("vol. sem peso
 ↪  (11.3)", P_113)]:
    d = ivs_modelo(p) - IV_MERC
    linhas.append((nome, *p, eqm_bcc(p), erro_vol(p, True), 100 * np.abs(d[CURTO]).max(), 100 *
 ↪  np.abs(d[~CURTO]).max()))
tab5 = pd.DataFrame(linhas, columns=["função objetivo", *NOMES, "(11.1)", "(11.4)",
                                     "maior erro vol. 17d (p.p.)", "maior erro vol. 80/171d (p.p.)"])
tabela(tab5)

t5 = tab5.set_index("função objetivo")
ok(np.abs(P_114 - P_JOINT).max() < 0.2, "(11.4) parte do ajuste em preços e quase não o altera")
ok(abs(t5.loc["vol. × vega (11.4)", "(11.4)"] / t5.loc["preços (11.1)", "(11.4)"]) > 0.5,
   "o ganho em (11.4) é pequeno: as duas funções medem quase a mesma coisa")
ok(t5.loc["vol. sem peso (11.3)", "maior erro vol. 17d (p.p.)"] <= t5.loc["preços (11.1)", "maior
 ↪  erro vol. 17d (p.p.)"] + 1e-6,
   "(11.3) dá mais peso ao prazo curto e não piora o maior erro de volatilidade de 17 dias")
```

```
tempo: 30.6 s
```

função objetivo	κ_v	θ_v	σ_v	ρ	v0	λ	μ_J	δ	(11.1)	(11.4)	maior erro vol. 17d (p.p.)	maior erro vol. 80/171d (p.p.)
preços (11.1)	2,845	0,040	0,500	-0,702	0,030	0,223	-0,105	0,109	0,001	0,001	0,042	0,005
vol. × vega (11.4)	2,829	0,040	0,500	-0,695	0,030	0,196	-0,123	0,102	0,000	0,000	0,017	0,007
vol. sem peso (11.3)	2,988	0,039	0,490	-0,711	0,030	0,191	-0,123	0,112	0,001	0,001	0,015	0,009

```
OK — (11.4) parte do ajuste em preços e quase não o altera
OK — o ganho em (11.4) é pequeno: as duas funções medem quase a mesma coisa
OK — (11.3) dá mais peso ao prazo curto e não piora o maior erro de volatilidade de 17 dias
```

O livro relata que, ao passar de (11.1) para (11.4), κ_v vai de 22,2 para 28,5 e σ_v de 0,95 para 1,18, com EQM ponderado de 0,083. Se as duas funções são quase iguais, essa mudança de parâmetros é mais um sinal do vale largo do Módulo 13: os parâmetros andam sem que o ajuste mude. Aqui, com dados que de fato vêm do modelo, os três ajustes ficam todos no nível do ruído.

6. O que a calibração não fixa: risco de modelo

Conjuntos de parâmetros que ajustam igualmente bem as 15 opções podem discordar sobre produtos **fora** da amostra de calibração. Além dos ajustes das seções anteriores, recalibramos o BCC97 com κ_v fixo em 1,5 e em 6 (como no perfil do Módulo 13). Medimos três produtos:

- call no dinheiro de 1 ano e de 5 anos (fora dos vencimentos calibrados), em volatilidade implícita;
- **swap de variância** de 5 anos: paga a variância realizada anualizada contra um preço fixo. O preço justo é a variação quadrática esperada. Num modelo com volatilidade de Heston e saltos de Merton:

$$K_{var}(T) = \frac{1}{T}\,\mathbb{E}\Big[\int_0^T v_t\,dt\Big] + \lambda\,\mathbb{E}[J^2] = \theta_v + (v_0-\theta_v)\frac{1-e^{-\kappa_v T}}{\kappa_v T} + \lambda(\mu_J^2+\delta^2). \quad (14.4)$$

O primeiro termo usa $\mathbb{E}[v_t] = \theta_v + (v_0-\theta_v)e^{-\kappa_v t}$ (Módulo 11); o segundo soma o quadrado de cada salto, que entra integralmente na variância realizada.

Laboratório 6 — Mesmo ajuste, preços diferentes

```
def k_var(p, T):
    kappa, theta, sigma, rho, v0, lam, mu, delta = p
    return theta + (v0 - theta) * (1 - np.exp(-kappa * T)) / (kappa * T) + lam * (mu**2 + delta**2)


# dois ajustes alternativos: κ_v fixo e os outros 7 parâmetros recalibrados (como o perfil do
↪ Módulo 13)
alternativos = {}
for kf in (1.5, 6.0):
    q = fmin(lambda q: eqm_bcc(np.r_[kf, q]), P_JOINT[1:], xtol=1e-6, ftol=1e-6, maxiter=2000,
↪ maxfun=3000, disp=False)
    alternativos[kf] = np.r_[kf, q]

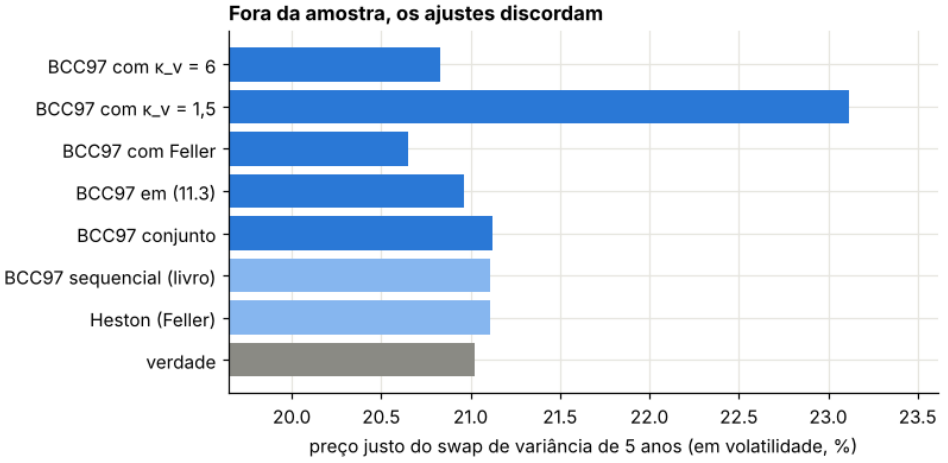
conjuntos = {"verdade": VERDADE,
             "Heston (Feller)": np.r_[H93_CAL, 0.0, -0.1, 0.0],
             "BCC97 sequencial (livro)": P_SEQ,
             "BCC97 conjunto": P_JOINT,
             "BCC97 em (11.3)": P_113,
             "BCC97 com Feller": resultados[("chute com saltos", "com Feller")],
             "BCC97 com κ_v = 1,5": alternativos[1.5],
             "BCC97 com κ_v = 6": alternativos[6.0]}
r_long = 0.04
linhas = []
for nome, p in conjuntos.items():
    kappa, theta, sigma, rho, v0, lam, mu, delta = p
    ivs_fora = []
    for T in (1.0, 5.0):
        c = calls(S0, [S0], [T], [r_long], (v0, kappa, theta, sigma, rho), (lam, mu, delta),
↪ U_FINO)[0]
        ivs_fora.append(100 * vol_impl(c, S0, S0, T, r_long))
    linhas.append((nome, eqm_bcc(p), *ivs_fora, 100 * np.sqrt(k_var(p, 5.0))))
tab6 = pd.DataFrame(linhas, columns=["parâmetros", "EQM nas 15 calls", "vol. impl. ATM 1 ano (%)",
                                     "vol. impl. ATM 5 anos (%)", "swap de variância 5 anos (vol.
                                     ↪ %)"])
```

```
tabela(tab6)

fig, ax = plt.subplots(figsize=(7, 3.6))
y = np.arange(len(tab6))
ax.barh(y, tab6["swap de variância 5 anos (vol. %)"], color=[CINZA] + [AZUL if e < 0.05 else
↪  RAMPA[0] for e in tab6["EQM nas 15 calls"][1:]])
ax.set_yticks(y, tab6["parâmetros"])
ax.set_xlim(tab6["swap de variância 5 anos (vol. %)"].min() - 1, tab6["swap de variância 5 anos
↪  (vol. %)"].max() + 0.5)
ax.set_xlabel("preço justo do swap de variância de 5 anos (em volatilidade, %)")
ax.set_title("Fora da amostra, os ajustes discordam")
plt.show()

t6 = tab6.set_index("parâmetros")
bons = t6[t6["EQM nas 15 calls"] < 0.05]
amp5 = bons["swap de variância 5 anos (vol. %)"].max() - bons["swap de variância 5 anos (vol.
↪  %)"].min()
amp1 = bons["vol. impl. ATM 1 ano (%)"].max() - bons["vol. impl. ATM 1 ano (%)"].min()
print("conjuntos com EQM < 0,05:", list(bons.index))
print("amplitude entre eles — ATM 1 ano:", f3(amp1), "p.p. | swap de variância 5 anos:", f3(amp5),
↪  "p.p.")
ok(t6.loc["BCC97 com κ_v = 1,5", "EQM nas 15 calls"] < 0.02, "com κ_v = 1,5 o ajuste fica em
↪  torno de um tick²: indistinguível da verdade")
ok(amp5 > 1.5, "entre ajustes com EQM < 0,05, o swap de variância de 5 anos varia mais de 1,5
↪  ponto de volatilidade")
ok(amp5 > 3 * amp1, "mais de três vezes a variação da call ATM de 1 ano")
```

parâmetros	EQM nas 15 calls	vol. impl. ATM 1 ano (%)	vol. impl. ATM 5 anos (%)	swap de variância 5 anos (vol. %)
verdade	0,001	19,816	21,015	21,026
Heston (Feller)	0,093	19,916	21,192	21,112
BCC97 sequencial (livro)	0,093	19,916	21,192	21,112
BCC97 conjunto	0,001	19,820	21,090	21,123
BCC97 em (11.3)	0,001	19,792	20,956	20,963
BCC97 com Feller	0,004	19,767	20,648	20,647
BCC97 com $\kappa_v = 1{,}5$	0,012	19,994	22,494	23,114
BCC97 com $\kappa_v = 6$	0,038	19,788	20,681	20,827




```
conjuntos com EQM < 0,05: ['verdade', 'BCC97 conjunto', 'BCC97 em (11.3)', 'BCC97 com Feller', 'BCC97 com κ_v =
↪  1,5', 'BCC97 com κ_v = 6']
amplitude entre eles — ATM 1 ano: 0,228 p.p. | swap de variância 5 anos: 2,466 p.p.
OK — com κ_v = 1,5 o ajuste fica em torno de um tick²: indistinguível da verdade
OK — entre ajustes com EQM < 0,05, o swap de variância de 5 anos varia mais de 1,5 ponto de volatilidade
OK — mais de três vezes a variação da call ATM de 1 ano
```

Barras escuras: conjuntos que ajustam as 15 calls com EQM abaixo de 0,05; claras: os que ajustam pior (os dois que pararam no Heston).

O que aprendemos. As 15 opções, todas com até seis meses, fixam bem o que acontece até seis meses e pouco além. O ajuste com $\kappa_v = 1{,}5$ tem EQM de cerca de um tick ao quadrado, indistinguível da verdade com essas cotações, e ainda assim põe o swap de variância de 5 anos uns 2 pontos de volatilidade acima. O swap de variância de 5 anos depende de θ_v, de κ_v e do termo dos saltos, justamente o que a calibração identifica mal. A distância entre os conjuntos é uma medida direta do **risco de modelo de calibração**: para validar um modelo usado em prazos longos, não basta olhar o ajuste aos dados de calibração.

---

7. Síntese

Etapa	Lição
Regularização	Tikhonov (quadrado) troca ajuste por estabilidade de forma suave; a norma (11.17) é uma penalidade exata que pode congelar os parâmetros
Saltos sequenciais	sobre um Heston já calibrado, os saltos não entram: o Heston absorveu o skew curto
BCC97 completo	uma única busca local herda o mínimo local da partida; várias partidas e a remoção de Feller levam ao ajuste no nível do ruído
Degenerescência	λ e μ_J se compensam; o produto é bem identificado, cada um não
Volatilidades implícitas	(11.4) $\approx$ (11.1); a diferença real é para (11.3)
Risco de modelo	ajustes equivalentes discordam fora da amostra, mais em prazos longos e em produtos de variância

8. Erratas e notas sobre o capítulo 11 (segunda parte)

Onde	O quê
§11.6.4, `BCC_calibration_short`	`opt1` e `local_opt` são atribuídos sem `global`: viram variáveis locais, e a penalidade da função objetivo nunca é aplicada

Onde	O quê
Eq. (11.17)	a penalidade é a norma $\|p - p_0\|$, não o quadrado: não é Tikhonov; com $w = 1$ e EQM em pontos^2, congela os parâmetros no ponto de partida
§11.4.4, resultados	$\lambda = 0{,}008$, $\mu_J = -0{,}600$ (no limite) e $\delta = 0{,}001$: na prática, nenhum salto; sintoma da calibração sequencial
§11.4.5, resultados	$\rho = -0{,}999$ e $\delta = 0{,}000$ na fronteira, $\lambda = 0{,}008$: o BCC97 calibrado é praticamente um Heston
§11.4.6, resultados	κ_v vai de 22,2 a 28,5 entre (11.1) e (11.4), funções quase iguais: parâmetros mal identificados; $\mu_J = -0{,}600$ de novo no limite
§11.6.5 e §11.6.6	a condição de Feller continua imposta (ver Módulo 13)
§11.6.2 e seguintes	Nelder-Mead com restrições por "valor alto" pode parar cedo encostado numa parede; sem reinício, o resultado depende de detalhes mínimos dos dados
§11.6.6	`options.ix` foi removido do pandas; use `.loc`

9. Exercícios

Exercício 1. Mostre que, para F diferenciável e $w > 0$, a solução de Tikhonov $p^*(w)$ satisfaz $\nabla F(p^*) = -2w(p^* - p_0)$. O que isso diz sobre a distância $\|p^* - p_0\|$ quando $w \to \infty$?

Solução

É a condição de primeira ordem. Tomando normas, $\|p^* - p_0\| = \|\nabla F(p^*)\|/(2w)$, que vai a zero como $1/w$: a solução se aproxima de p_0 suavemente, sem nunca "travar", ao contrário da norma.

Exercício 2. Corrija o script do livro para que a penalidade funcione (declare `global opt1, local_opt` em `BCC_calibration_short`) e troque a norma pelo quadrado. Qual w você escolheria para EQM medido em pontos^2?

Solução

Um critério prático: escolha w para que a penalidade, numa variação típica dos parâmetros (digamos 0,05 em cada um), seja comparável ao ruído do EQM (da ordem de tick^2). Com tick de 0,1, $w \cdot 3 \cdot 0{,}05^2 \approx 0{,}01$ dá $w \approx 1$. O Laboratório 4 mostra que esse valor reduz a dispersão a menos da metade, com perda pequena de ajuste.

Exercício 3. Repita o Laboratório 3 com 10 partidas aleatórias (uniformes nas caixas da grade do livro, com $\lambda \in [0,0{,}5]$). Quantas chegam ao nível do ruído?

Solução

Depende da semente; em geral, só uma parte delas. Esse "multi-start" é a forma mais simples de busca global; alternativas são `differential_evolution` e `basinhopping` do SciPy. Guarde todos os mínimos encontrados: se vários têm o mesmo erro e parâmetros diferentes, você tem uma medida da degenerescência.

Exercício 4. Deduza a fórmula do swap de variância. Por que a correlação ρ não aparece?

Solução

A variação quadrática de $\ln S$ é $\int_0^T v_t\,dt + \sum J_i^2$. A esperança do primeiro termo vem de $\mathbb{E}[v_t]$; a do segundo é $\lambda T\,\mathbb{E}[J^2] = \lambda T(\mu_J^2 + \delta^2)$. A correlação afeta a distribuição conjunta de S e v, mas não a média de v, e o swap só depende dessa média.

```
print(f"Módulo 14 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 14 concluído: 24 verificações aprovadas.
```

Capítulo 15

Simulação e precificação no modelo BCC97

Introdução

Com o modelo BCC97 calibrado, opções europeias podem ser precificadas por Fourier. Para contratos sem fórmula, como opções americanas ou contratos que dependem da trajetória, é preciso **simular** o modelo completo: preço do ativo, variância estocástica, juros estocásticos e saltos, com as correlações corretas entre os choques.

Cada bloco traz cuidados próprios. A variância e os juros não podem ficar negativos; os saltos precisam de um termo de compensação para que o preço descontado continue sendo um martingal; e a ordem em que as variáveis são atualizadas em cada passo importa, porque usar informação do fim do passo para calcular o seu início introduz um viés que não desaparece com passos menores.

O capítulo constrói o simulador, confere seus preços europeus contra a fórmula de Fourier e o usa, com o algoritmo LSM do Capítulo 9, para precificar opções americanas no modelo geral.

Fundamentos teóricos

Discretização do sistema BCC97

Simulamos $(r, v, \ln S)$ numa malha $t_k = k\Delta t$. Os choques são construídos a partir de normais independentes $\varepsilon^1, \varepsilon^2, \varepsilon^3$ pela decomposição de Cholesky da matriz de correlação:

$$z^1 = \varepsilon^1, \qquad z^2 = \rho\,\varepsilon^1 + \sqrt{1-\rho^2}\,\varepsilon^2, \qquad z^3 = \varepsilon^3. \tag{15.1}$$

Definição 15.1 (Esquema). Com $\tilde{x}^+ = \max(\tilde{x}, 0)$ (full truncation) para v e r,

$$\tilde{v}_{k+1} = \tilde{v}_k + \kappa_v(\theta_v - \tilde{v}_k^+)\Delta t + \sigma_v\sqrt{\tilde{v}_k^+}\sqrt{\Delta t}\, z^2_{k+1}, \tag{15.2}$$

$$\ln S_{k+1} = \ln S_k + \Big(\bar{r}_k - r_J - \tfrac{1}{2}v_k\Big)\Delta t + \sqrt{v_k}\sqrt{\Delta t}\, z^1_{k+1} + \sum_{\ell=1}^{\Delta N_{k+1}} \ln(1+J_\ell), \tag{15.3}$$

onde $v_k = \tilde{v}_k^+$ é a variância *no início* do passo e $\bar{r}_k = \tfrac{1}{2}(r_k + r_{k+1})$ é a média trapezoidal da short rate.

Observação 15.2 (Não antecipação). Em (15.3), usar v_{k+1} no lugar de v_k faria a variância do passo depender do choque z^2_{k+1}, correlacionado com z^1_{k+1}. Como $\sqrt{v_{k+1}} \approx \sqrt{v_k} + \frac{1}{2}\sigma_v\sqrt{\Delta t}\, z^2_{k+1}$, o incremento $\sqrt{v_{k+1}}\sqrt{\Delta t}\, z^1_{k+1}$ teria média $\approx \frac{1}{2}\rho\sigma_v\Delta t$. Somados os $T/\Delta t$ passos, surge uma deriva espúria de $\frac{1}{2}\rho\sigma_v T$ no log-preço, que não desaparece quando $\Delta t \to 0$; o preço descontado deixa de ser martingal. A integral de Itô exige integrandos avaliados no início do intervalo.

Saltos exatos

Proposição 15.3 (Passo exato dos saltos). *Num passo,* $\Delta N \sim \text{Poisson}(\lambda\Delta t)$ *e, dado* $\Delta N = n$, $\sum_{\ell\le n} \ln(1+J_\ell) \sim N(n\mu_J, n\delta^2)$. *Amostrar* $(\Delta N, \sum \ln(1+J))$ *dessa forma não tem erro de discretização, e a compensação* $-r_J\Delta t$ *garante* $\mathbb{E}[\prod_\ell (1+J_\ell)]e^{-r_J\Delta t} = 1$ *em cada passo.*

Demonstração. $\mathbb{E}[\prod_{\ell\le\Delta N}(1+J_\ell)] = \sum_n \frac{e^{-\lambda\Delta t}(\lambda\Delta t)^n}{n!}(1+\mathbb{E}J)^n = e^{\lambda\Delta t\,\mathbb{E}J} = e^{r_J\Delta t}$. ∎

Omitir o compensador soma uma deriva espúria r_J ao índice: com saltos médios negativos, o preço descontado simulado decai, e as calls ficam baratas e as puts caras na mesma proporção.

Avaliação e desconto

Proposição 15.4 (Desconto com juros estocásticos). *O preço de um payoff* H *em* T *é* $\mathbb{E}^{\mathbb{Q}}[e^{-\int_0^T r_s\,ds}H]$. *Na simulação, o fator de desconto de cada trajetória é* $\exp(-\sum_k \bar r_k\Delta t)$, *e a média dos fatores de desconto estima* $B(0,T)$, *conhecido em forma fechada (Proposição 11.3): uma variável de controle natural.*

Observação 15.5 (Consistência com a fórmula fechada). Sem juros estocásticos ou com a aproximação do Capítulo 11, o preço europeu por simulação e o preço pela fórmula de Lewis devem coincidir dentro do erro de Monte Carlo mais o viés de discretização. Essa comparação é o teste básico de qualquer simulador antes de usá-lo em produtos sem fórmula, como as opções americanas, avaliadas aqui pelo LSM do Capítulo 9.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulo 12 do livro.

O que você vai conseguir fazer ao final

1. Discretizar o BCC97 completo: juros CIR, variância de Heston e índice com saltos.
2. Simular os saltos de Merton de forma exata e conferir a simulação pelo teste da martingal.
3. Reproduzir o viés de Monte Carlo da §12.3.1 do livro e identificar sua causa real.
4. Conferir a distribuição simulada inteira pelos cumulantes da função característica.
5. Calcular o prêmio de exercício antecipado de puts americanas com LSM e variável de controle.

Onde estamos

O capítulo 12 é o ponto em que as peças se juntam: o modelo do Módulo 11, os esquemas do Módulo 12 e os parâmetros calibrados dos Módulos 13 e 14. O livro diz que o capítulo "apenas cola" as peças. Mas é na colagem que aparecem os problemas: um compensador esquecido, uma variância lida no instante errado. Com parâmetros extremos, os dois produzem vieses de dezenas de pontos.

Usamos dois conjuntos de parâmetros: o **Bates sintético** dos Módulos 13 e 14 (saltos relevantes) e os **parâmetros calibrados publicados pelo livro** na §11.4.5 (correlação quase $-$1 e volatilidade da variância perto de 1), que permitem reproduzir as tabelas do capítulo.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.interpolate as sci
from scipy.integrate import simpson
from scipy.optimize import brentq, brute, fmin, linprog
from scipy.stats import norm
from IPython.display import display

RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

def bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - sig * np.sqrt(T))

def vega_bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.pdf(d1) * np.sqrt(T)

def vol_impl(preco, S, K, T, r):
    piso = max(S - K * np.exp(-r * T), 0.0)
    if not (piso + 1e-12 < preco < S):
        return np.nan
    return brentq(lambda s: float(bsm(S, K, T, r, s)) - preco, 1e-4, 4.0)

def cf_heston(u, T, v0, kappa, theta, sigma, rho):
    """Função característica de ln(S_T/S_0) - rT no Heston, forma estável (Módulo 11)."""
    xi = kappa - rho * sigma * 1j * u
    d = np.sqrt(xi**2 + sigma**2 * (1j * u + u**2))
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
    A = kappa * theta / sigma**2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / sigma**2 * (1 - e) / (1 - g * e)
    return np.exp(A + B * v0)

def cf_saltos(u, T, lam, mu, delta):
    """Parte de saltos de Merton, com o drift compensado (11.13)-(11.14)."""
    omega = -lam * (np.exp(mu + 0.5 * delta**2) - 1)
    return np.exp((1j * u * omega + lam * (np.exp(1j * u * mu - 0.5 * u**2 * delta**2) - 1)) * T)
```

```python
U_FINO = np.linspace(0, 2000, 200_001)
U_CAL = np.linspace(0, 300, 3001)

def calls(S0, K, T, r, heston, saltos=None, u=U_CAL):
    """Calls europeias pela fórmula de Lewis (11.12), vetorizada em (K, T, r) do mesmo tamanho.
    heston = (v0, κ_v, θ_v, σ_v, ρ); saltos = (λ, μ_J, δ) ou None."""
    K, T, r = (np.atleast_1d(np.asarray(x, float)) for x in (K, T, r))
    k = np.log(S0 / K) + r * T
    phi = cf_heston(u[None, :] - 0.5j, T[:, None], *heston)
    if saltos is not None:
        phi = phi * cf_saltos(u[None, :] - 0.5j, T[:, None], *saltos)
    h = np.exp(1j * u[None, :] * k[:, None]) * phi
    integral = simpson(h.real / (u**2 + 0.25), x=u, axis=1)
    return S0 - np.sqrt(S0 * K) * np.exp(-0.5 * r * T) / np.pi * integral

def zcb_cir(r0, kappa, theta, sigma, T):
    """Zero-cupom no CIR, (11.7)-(11.9) com t = 0."""
    g = np.sqrt(kappa**2 + 2 * sigma**2)
    e = np.exp(g * T) - 1
    D = 2 * g + (kappa + g) * e
    return (2 * g * np.exp((kappa + g) * T / 2) / D) ** (2 * kappa * theta / sigma**2) * np.exp(-2
    ↪  * e / D * r0)

def fator_ft(xh, kappa, theta, sigma, dt, z):
    """Um passo Full Truncation (Módulo 12): devolve (x truncado, x˜ auxiliar)."""
    p = np.maximum(xh, 0)
    xh = xh + kappa * (theta - p) * dt + sigma * np.sqrt(p * dt) * z
    return np.maximum(xh, 0), xh

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. O esquema de simulação do BCC97

Livro: §12.2 (equações 12.1 a 12.3)

1.1 As três equações discretizadas

Com $s = t - \Delta t$, o livro propõe:

$$r_t: \quad \tilde{r}_t = \tilde{r}_s + \kappa_r(\theta_r - \tilde{r}_s^+)\Delta t + \sigma_r\sqrt{\tilde{r}_s^+}\sqrt{\Delta t}\, z_t^3, \qquad r_t = \tilde{r}_t^+ \tag{12.3}$$

$$v_t: \quad \tilde{v}_t = \tilde{v}_s + \kappa_v(\theta_v - \tilde{v}_s^+)\Delta t + \sigma_v\sqrt{\tilde{v}_s^+}\sqrt{\Delta t}\, z_t^2, \qquad v_t = \tilde{v}_t^+ \tag{12.2}$$

Os dois são o Full Truncation do Módulo 12. Para o índice, o livro escreve

$$S_t = S_s\left(e^{(\bar{r}_t - r_J - v_t/2)\Delta t + \sqrt{v_t}\sqrt{\Delta t}\, z_t^1} + (e^{\mu_J + \delta z_t^4} - 1)\, y_t\right), \qquad \bar{r}_t = \frac{r_s + r_t}{2}, \tag{12.1}$$

com $y_t \sim \text{Poisson}(\lambda\Delta t)$ e $r_J = \lambda(e^{\mu_J + \delta^2/2} - 1)$, a correção de drift dos saltos.

1.2 O esquema exato para os saltos, passo a passo

Para o índice, aplicamos Itô a $\ln S$ como no Módulo 10, agora com v e r estocásticos. Num passo, a parte contínua é integrada por log-Euler, e a de saltos pode ser integrada **exatamente**.

Passo 1. Em $[s,t]$ chegam $N \sim \text{Poisson}(\lambda\Delta t)$ saltos, cada um multiplicando S por e^{J_i}, com $J_i \sim N(\mu_J, \delta^2)$ independentes.

Passo 2. A soma dos log-saltos, dado N, é normal: $\sum_{i=1}^{N} J_i \sim N(N\mu_J,\ N\delta^2)$. Basta um normal z^4 por passo: $\sum J_i = N\mu_J + \sqrt{N}\,\delta\, z^4$.

Passo 3. O passo completo é

$$\ln S_t = \ln S_s + (\bar{r}_t - r_J - \tfrac{1}{2}v_s)\Delta t + \sqrt{v_s}\sqrt{\Delta t}\, z_t^1 + N\mu_J + \sqrt{N}\,\delta\, z_t^4. \tag{15.4}$$

Passo 4 (conferência). Dado o passado, com v_s conhecido e r independente do resto,

$$\mathbb{E}\big[e^{N\mu_J+\sqrt{N}\delta z^4}\big] = \mathbb{E}\big[(e^{\mu_J+\delta^2/2})^N\big] = e^{\lambda\Delta t(e^{\mu_J+\delta^2/2}-1)} = e^{r_J\Delta t}, \tag{15.5}$$

que cancela exatamente o $-r_J\Delta t$. Com o $e^{-v_s\Delta t/2}$ cancelando a convexidade do choque browniano (Módulo 12), S descontado por $\bar{r}$ é uma martingal passo a passo.

1.3 O que o código do livro faz de diferente

O script da §12.5.1 se afasta de (12.1) e do esquema exato em três pontos:

	Esquema exato (acima)	Código da §12.5.1
Variância no passo do índice	v_s (início do passo)	`v[t]` (fim do passo), como no Módulo 12
Compensador dos saltos	$-r_J\Delta t$ no expoente	`rj` é calculado e **nunca usado**
Saltos	multiplicativos, $e^{\sum J_i}$	aditivos, $(e^{\mu_J+\delta z}-1)\,y_t$, com um único tamanho para todos os y_t saltos do passo

Além disso, o script liga por padrão o moment matching com a correção de drift `bias`, que, como no Módulo 12, esconde parte desses efeitos.

```
S0 = 3225.93
CIR = dict(r0=0.02, kappa=0.5, theta=0.045, sigma=0.08)      # curva sintética dos Módulos 13 e 14
NOMES = ["κ_v", "θ_v", "σ_v", "ρ", "v0", "λ", "μ_J", "δ"]
BATES = np.array([3.0, 0.04, 0.5, -0.7, 0.03, 0.15, -0.15, 0.10])              # modelo
↪ verdadeiro dos Módulos 13 e 14
LIVRO = np.array([22.212, 0.025, 0.952, -0.999, 0.036, 0.008, -0.501, 0.000])   # BCC97 calibrado
↪ no livro (§11.4.5)

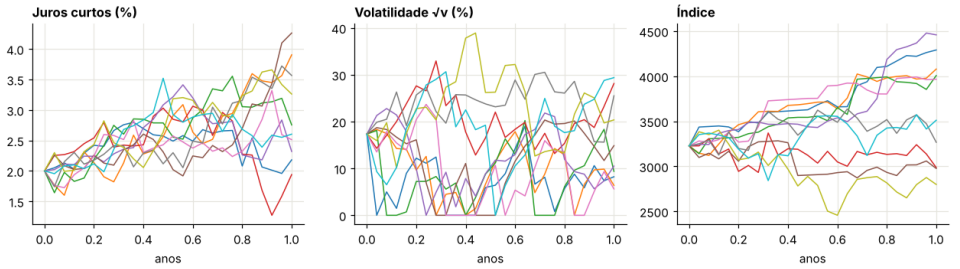

def simular_bcc(p, T, M0, I, rng, olhar_frente=False, compensador=True, aditivo=False,
                mm=False, trajetorias=False):
    """Simula o BCC97 com Full Truncation para r e v.
    Padrão = esquema exato da Seção 1.2. As opções reproduzem os desvios do código do livro:
```

```
      olhar_frente: usa v[t] no passo do índice; compensador=False: omite −r_J;
      aditivo: saltos aditivos de (12.1); mm: moment matching e correção 'bias'.
    Devolve S_T e o desconto exp(−∫r), ou as trajetórias completas (S, v, r)."""
    kappa, theta, sigma, rho, v0, lam, mu, delta = p
    M = max(int(round(M0 * T)), 1)
    dt = T / M
    Z = rng.standard_normal((4, M, I // 2))
    Z = np.concatenate([Z, -Z], axis=2)                      # antitéticas, como no livro
    if mm:
        Z = (Z - Z.mean(axis=(1, 2), keepdims=True)) / Z.std(axis=(1, 2), keepdims=True)
    N = rng.poisson(lam * dt, (M, I))
    rJ = lam * (np.exp(mu + 0.5 * delta**2) - 1)
    r = np.full(I, CIR["r0"]); rh = r.copy()
    v = np.full(I, v0); vh = v.copy()
    S = np.full(I, S0); int_r = np.zeros(I)
    if trajetorias:
        Sp, vp, rp = [S.copy()], [v.copy()], [r.copy()]
    for t in range(M):
        z1 = Z[1, t]
        z2 = rho * z1 + np.sqrt(1 - rho**2) * Z[2, t]
        r_novo, rh = fator_ft(rh, CIR["kappa"], CIR["theta"], CIR["sigma"], dt, Z[0, t])
        v_novo, vh = fator_ft(vh, kappa, theta, sigma, dt, z2)
        rbar = 0.5 * (r + r_novo)
        int_r += rbar * dt
        vu = v_novo if olhar_frente else v
        choque = np.sqrt(vu * dt) * z1
        if mm:
            choque = choque - choque.mean()
        deriva = (rbar - (rJ if compensador else 0.0) - 0.5 * vu) * dt
        if aditivo:
            S = S * (np.exp(deriva + choque) + (np.exp(mu + delta * Z[3, t]) - 1) * N[t])
        else:
            S = S * np.exp(deriva + choque + N[t] * mu + np.sqrt(N[t]) * delta * Z[3, t])
        r, v = r_novo, v_novo
        if trajetorias:
            Sp.append(S.copy()); vp.append(v.copy()); rp.append(r.copy())
    if trajetorias:
        return np.array(Sp), np.array(vp), np.array(rp)
    return S, np.exp(-int_r)


def calls_bcc(p, K, T):
    """Calls europeias pela fórmula de Lewis com r̄ = −ln B0(T)/T, como no livro."""
    kappa, theta, sigma, rho, v0, lam, mu, delta = p
    K = np.atleast_1d(np.asarray(K, float))
    rbar = -np.log(zcb_cir(T=T, **CIR)) / T
    return calls(S0, K, np.full(len(K), T), np.full(len(K), rbar), (v0, kappa, theta, sigma, rho),
    ↪  (lam, mu, delta), U_FINO)


rng = np.random.default_rng(100000)
Sp, vp, rp = simular_bcc(BATES, 1.0, 25, 10_000, rng, trajetorias=True)
fig, axs = plt.subplots(1, 3, figsize=(11.5, 3.4))
tt = np.linspace(0, 1, 26)
for i in range(10):
    axs[0].plot(tt, 100 * rp[:, i], lw=1)
    axs[1].plot(tt, 100 * np.sqrt(vp[:, i]), lw=1)
    axs[2].plot(tt, Sp[:, i], lw=1)
axs[0].set_title("Juros curtos (%)"); axs[1].set_title("Volatilidade √v (%)");
↪  axs[2].set_title("Índice")
```

```
for ax in axs:
    ax.set_xlabel("anos")
plt.tight_layout(); plt.show()
ok(np.all(vp >= 0) and np.all(rp >= 0), "Full Truncation: variância e juros nunca ficam
↪  negativos")
```




```
OK — Full Truncation: variância e juros nunca ficam negativos
```

As figuras correspondem às 12.1 a 12.3 do livro, para o modelo verdadeiro dos Módulos 13 e 14 (Bates com juros CIR), 25 passos em 1 ano. Os saltos aparecem como quedas verticais no índice.

---

2. O teste da martingal e os saltos

Como visto no Módulo 12, a conferência mais barata de uma simulação de preços é $\mathbb{E}[e^{-\int_0^T r\,dt} S_T] = S_0$. Aqui também conferimos calls de 1 ano contra a fórmula de Lewis.

Laboratório 1 — Separar os efeitos do código dos saltos

Usamos o modelo de Bates verdadeiro ($\lambda = 0{,}15$, $\mu_J = -0{,}15$), em que os saltos pesam. $r_J = 0{,}15\,(e^{-0{,}15+0{,}005} - 1) \approx -2{,}0\%$ ao ano. Variamos um defeito de cada vez, sem olhar para a frente, para isolar o efeito dos saltos.

```
T1, K1 = 1.0, np.array([3050.0, 3225.0, 3400.0])
ref1 = calls_bcc(BATES, K1, T1)
rJ_bates = BATES[5] * (np.exp(BATES[6] + 0.5 * BATES[7] ** 2) - 1)
print("r_J =", f3(100 * rJ_bates), "% ao ano | previsão do erro de E[e^{-∫r}S_T] sem
↪  compensador:",
      f3(S0 * (np.exp(rJ_bates * T1) - 1)))

variantes = [("esquema exato", dict()),
             ("sem compensador −r_J", dict(compensador=False)),
             ("saltos aditivos", dict(aditivo=True)),
            ("aditivos e sem compensador (como o livro)", dict(compensador=False, aditivo=True))]
rng = np.random.default_rng(1)
linhas = []
for nome, kw in variantes:
    ST, D = simular_bcc(BATES, T1, 50, 200_000, rng, **kw)
    mart = np.mean(D * ST) - S0
    ep = np.std(D * ST) / np.sqrt(len(ST))
    c = (D[:, None] * np.maximum(ST[:, None] - K1, 0)).mean(axis=0)
```

```
    linhas.append((nome, mart, ep, *(c - ref1)))
tab1 = pd.DataFrame(linhas, columns=["variante", "erro de E[e^{-∫r}S_T]", "erro-padrão",
                                     "erro call 3.050", "erro call 3.225", "erro call 3.400"])
tabela(tab1)

t1 = tab1.set_index("variante")
ok(abs(t1.loc["esquema exato", "erro de E[e^{-∫r}S_T]"]) < 3 * t1.loc["esquema exato",
↪  "erro-padrão"] + 0.5,
   "o esquema exato passa no teste da martingal")
prev = S0 * (np.exp(rJ_bates * T1) - 1)
obs = t1.loc["sem compensador −r_J", "erro de E[e^{-∫r}S_T]"]
ok(abs(obs / prev - 1) < 0.1, "sem o compensador, o erro é o previsto, S0·(e^{r_J T} − 1), a
↪  menos de 10%")
ok(abs(t1.loc["saltos aditivos", "erro de E[e^{-∫r}S_T]"]) < 0.15 * abs(prev),
   "os saltos aditivos, sozinhos, quase não afetam a média")
ok(t1.loc["aditivos e sem compensador (como o livro)", "erro call 3.225"] < -30,
   "no código do livro, a call de 1 ano no dinheiro sai mais de 30 pontos abaixo da fórmula")
```

```
r_J = -2,025 % ao ano | previsão do erro de E[e^{-∫r}S_T] sem compensador: -64,658
```

variante	erro de E[e^{−∫r}S_T]	erro-padrão	erro call 3.050	erro call 3.225	erro call 3.400
esquema exato	0,011	1,379	0,188	0,138	0,075
sem compensador −r_J	-64,756	1,349	-48,152	-41,917	-34,536
saltos aditivos	0,294	1,378	0,449	0,388	0,257
aditivos e sem compensador (como o livro)	-65,211	1,355	-47,141	-40,945	-33,663

```
OK — o esquema exato passa no teste da martingal
OK — sem o compensador, o erro é o previsto, S0·(e^{r_J T} − 1), a menos de 10%
OK — os saltos aditivos, sozinhos, quase não afetam a média
OK — no código do livro, a call de 1 ano no dinheiro sai mais de 30 pontos abaixo da fórmula
```

O que aprendemos. O defeito que pesa é o **compensador ausente**: a variável `rj` é calculada no script e nunca entra na conta. Sem o $-r_J\Delta t$ (que, com $r_J < 0$, somaria 2% ao ano), o índice descontado cai em média como $e^{r_J T}$: perde $|r_J| \approx 2\%$ ao ano, exatamente o previsto. Os saltos aditivos, embora não sejam o modelo (um único tamanho de salto vale para todos os saltos do passo, e o fator não é exponencial), quase não mudam a média com $\lambda\Delta t$ pequeno.

Com os parâmetros calibrados do livro ($\lambda = 0{,}008$, $\mu_J = -0{,}501$), $r_J \approx -0{,}3\%$ ao ano: o defeito existe, mas é pequeno. O viés grande que o livro mostra na §12.3.1 vem de outro lugar.

---

3. O viés da Seção 12.3.1

Livro: §12.3.1 e §12.5.2

O livro compara Monte Carlo e Fourier para calls europeias e relata desvios **enormes**: com 50 passos por ano, o Monte Carlo fica 22 a 28 pontos acima da fórmula em 1 ano, e 45 a 52 pontos acima em 3 anos. Com 200 passos por ano o desvio cai, mas não some. O livro conclui que esquemas de Euler exigem muitos passos em modelos complexos.

Os parâmetros calibrados do livro são extremos: $\sigma_v = 0{,}952$, $\kappa_v = 22{,}2$ e, sobretudo, $\rho = -0{,}999$. Com $\rho \approx -1$, o viés de olhar para a frente do Módulo 12 vale $\frac{1}{2}\rho\sigma_v \approx$

$-0{,}475$ por ano no drift de $\ln S$. A correção `bias` do moment matching remove a média desse drift espúrio, mas não os efeitos de segunda ordem, que dependem de Δt.

Laboratório 2 — Reproduzir a tabela do livro e encontrar a causa

Usamos os parâmetros publicados do livro (§11.4.5) com a nossa curva CIR, os vencimentos e strikes da §12.3.1, 50.000 trajetórias, e três versões do simulador:

- **exato**: o esquema da Seção 1.2;
- **livro**: `v[t]`, sem compensador, saltos aditivos, moment matching ligado (o padrão do script);
- **livro sem moment matching**: o mesmo, com `mm` desligado.

```
PRAZOS2 = [1 / 12, 0.5, 1.0, 3.0]
K2 = np.array([3050.0, 3225.0, 3400.0])
LIVRO_CFG = dict(olhar_frente=True, compensador=False, aditivo=True, mm=True)
rng = np.random.default_rng(2)
t0 = time.time()
linhas = []
for T in PRAZOS2:
    ref = calls_bcc(LIVRO, K2, T)
    for M0 in (50, 200):
        for nome, kw in [("exato", dict()), ("livro", LIVRO_CFG),
                         ("livro sem moment matching", dict(LIVRO_CFG, mm=False))]:
            ST, D = simular_bcc(LIVRO, T, M0, 50_000, rng, **kw)
            pay = D[:, None] * np.maximum(ST[:, None] - K2, 0)
            c = pay.mean(axis=0)
            ep = (pay[: len(ST) // 2] + pay[len(ST) // 2:]).std(axis=0) / 2 / np.sqrt(len(ST) //
↪ 2)   # pares antitéticos
            for K, ci, fi, ei in zip(K2, c, ref, ep):
                linhas.append((T, M0, nome, K, fi, ci - fi, ei))
tab2 = pd.DataFrame(linhas, columns=["T", "passos por ano", "simulador", "K", "Fourier", "MC -
↪ Fourier", "erro-padrão"])
print(f"tempo: {time.time() - t0:.1f} s")
piv = tab2.pivot_table(index=["T", "K"], columns=["simulador", "passos por ano"], values="MC -
↪ Fourier")
tabela(piv[["exato", "livro"]])
```

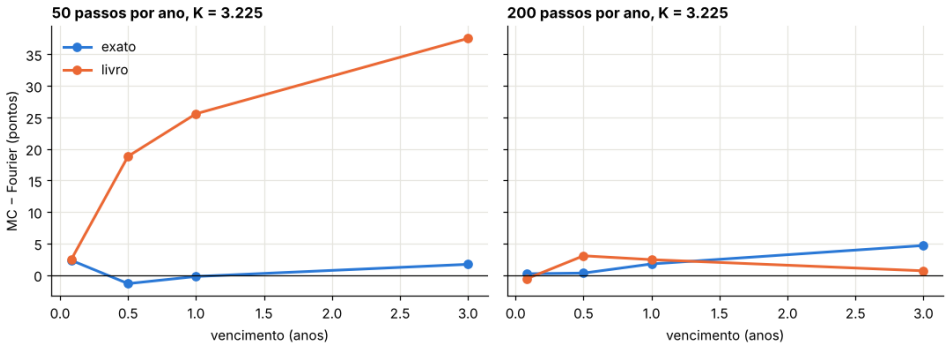
```
tempo: 54.6 s
```

T	simulador passos por ano K	exato 50	livro 200	50	200
0.083333	3050.0	0,633	0,148	2,071	-0,351
	3225.0	2,314	0,191	2,490	-0,588
	3400.0	1,192	0,090	0,672	0,625
0.500000	3050.0	-1,477	0,409	14,763	0,853
	3225.0	-1,369	0,298	18,808	3,039
	3400.0	-0,807	0,097	20,045	4,751
1.000000	3050.0	-0,517	1,806	21,510	0,247
	3225.0	-0,238	1,746	25,559	2,429
	3400.0	-0,305	1,433	27,690	4,216
3.000000	3050.0	1,683	4,405	32,921	-2,395
	3225.0	1,687	4,675	37,502	0,658
	3400.0	1,707	4,837	41,043	3,410

```
fig, axs = plt.subplots(1, 2, figsize=(10, 3.8), sharey=True)
for ax, M0 in zip(axs, (50, 200)):
    for nome, cor in [("exato", AZUL), ("livro", LARANJA)]:
        d = tab2[(tab2["passos por ano"] == M0) & (tab2["simulador"] == nome) & (tab2["K"] ==
↪  3225.0)]
        ax.plot(d["T"], d["MC − Fourier"], "o-", color=cor, label=nome)
    ax.axhline(0, color="black", lw=0.8)
    ax.set_title(f"{M0} passos por ano, K = 3.225"); ax.set_xlabel("vencimento (anos)")
axs[0].set_ylabel("MC − Fourier (pontos)"); axs[0].legend()
plt.tight_layout(); plt.show()

sem_mm = tab2[(tab2["simulador"] == "livro sem moment matching") & (tab2["T"] == 1.0) &
↪  (tab2["passos por ano"] == 50)]
print("livro sem moment matching, T = 1, 50 passos — MC − Fourier:", [f3(x) for x in sem_mm["MC −
↪  Fourier"]])

ex = tab2[tab2["simulador"] == "exato"]
lv = tab2[tab2["simulador"] == "livro"]
z_ex = (ex["MC − Fourier"] / ex["erro-padrão"]).abs()
print("esquema exato: maior |erro|/erro-padrão =", f3(z_ex.max()), "| erro-padrão em 3 anos ≈",
↪  f3(ex[ex["T"] == 3.0]["erro-padrão"].mean()))
longos = ex["T"] >= 0.5
ok(z_ex[longos].max() < 4, "a partir de 6 meses, o esquema exato fica dentro de 4 erros-padrão,
↪  já com 50 passos por ano")
curto = ex[ex["T"] < 0.1]
ok(curto[curto["passos por ano"] == 200]["MC − Fourier"].abs().max() < 0.5 < curto[curto["passos
↪  por ano"] == 50]["MC − Fourier"].abs().max(),
   "em 1 mês (só 4 passos com 50 por ano) há um viés de Euler real, que some com 200 passos por
   ↪  ano")
d1 = lv[(lv["T"] == 1.0) & (lv["passos por ano"] == 50)]["MC − Fourier"]
ok(d1.between(15, 35).all(), "o simulador do livro reproduz o viés da §12.3.1: 15 a 35 pontos
↪  acima em 1 ano")
d3_50 = lv[(lv["T"] == 3.0) & (lv["passos por ano"] == 50)]["MC − Fourier"].abs().mean()
d3_200 = lv[(lv["T"] == 3.0) & (lv["passos por ano"] == 200)]["MC − Fourier"].abs().mean()
ok(d3_200 < d3_50 / 2, "com 200 passos por ano o viés do livro cai, mas continua acima do esquema
↪  exato")
ok((sem_mm["MC − Fourier"] < -100).all(), "sem o moment matching, o código do livro erra por
↪  centenas de pontos")
```




```
livro sem moment matching, T = 1, 50 passos — MC − Fourier: ['-358,571', '-249,357', '-160,905']
esquema exato: maior |erro|/erro-padrão = 19,257 | erro-padrão em 3 anos ≈ 2,204
OK — a partir de 6 meses, o esquema exato fica dentro de 4 erros-padrão, já com 50 passos por ano
OK — em 1 mês (só 4 passos com 50 por ano) há um viés de Euler real, que some com 200 passos por ano
OK — o simulador do livro reproduz o viés da §12.3.1: 15 a 35 pontos acima em 1 ano
OK — com 200 passos por ano o viés do livro cai, mas continua acima do esquema exato
OK — sem o moment matching, o código do livro erra por centenas de pontos
```

O que aprendemos.

- O simulador do livro reproduz a tabela da §12.3.1 com boa precisão: o viés não é ruído, é sistemático e cresce com o vencimento.
- O esquema exato, **com os mesmos 50 passos por ano**, acerta dentro do ruído estatístico a partir de 6 meses (o erro-padrão de uma call de 3 anos com 50.000 trajetórias é de cerca de 2 pontos). No vencimento de 1 mês, 50 passos por ano são só 4 passos, e aparece um viés de Euler real de 1 a 2 pontos, que some com 200 passos. Ele é do mesmo tamanho nos dois simuladores: essa parte da tabela do livro é, de fato, discretização. A conclusão do livro ("esquemas de Euler exigem muitos passos") não se sustenta: o problema é o `v[t]`, não o Euler.
- Sem o moment matching, o código do livro erra por centenas de pontos: é o drift espúrio $\frac{1}{2}\rho\sigma_v$ com $\rho \approx -1$. A correção `bias` esconde a maior parte do erro, e o que sobra (de segunda ordem, por isso dependente de Δt) é o que o livro viu e atribuiu ao Euler.

---

4. A distribuição do índice

Livro: §12.2 (Figura 12.4)

O livro observa que os saltos afastam a distribuição de S_T da lognormal. Podemos medir isso com precisão: os **cumulantes** de $X = \ln(S_T/S_0)$ saem da função característica, $\kappa_n = i^{-n}\frac{d^n}{du^n}\ln\varphi(u)\big|_{u=0}$. A assimetria é $\kappa_3/\kappa_2^{3/2}$ e o excesso de curtose é κ_4/κ_2^2; ambos são zero na normal.

Laboratório 3 — Assimetria e curtose: simulação contra a função característica

Para juros constantes ($\bar{r}$ da curva) a função característica do BCC97 é o produto Heston × saltos (11.16). Derivamos $\ln\varphi$ numericamente por diferenças finitas em $u = 0$.

```
T3 = 1.0
rbar3 = -np.log(zcb_cir(T=T3, **CIR)) / T3


def log_cf(u):
    kappa, theta, sigma, rho, v0, lam, mu, delta = BATES
    return np.log(cf_heston(u, T3, v0, kappa, theta, sigma, rho) * cf_saltos(u, T3, lam, mu,
    ↪  delta)) + 1j * u * rbar3 * T3

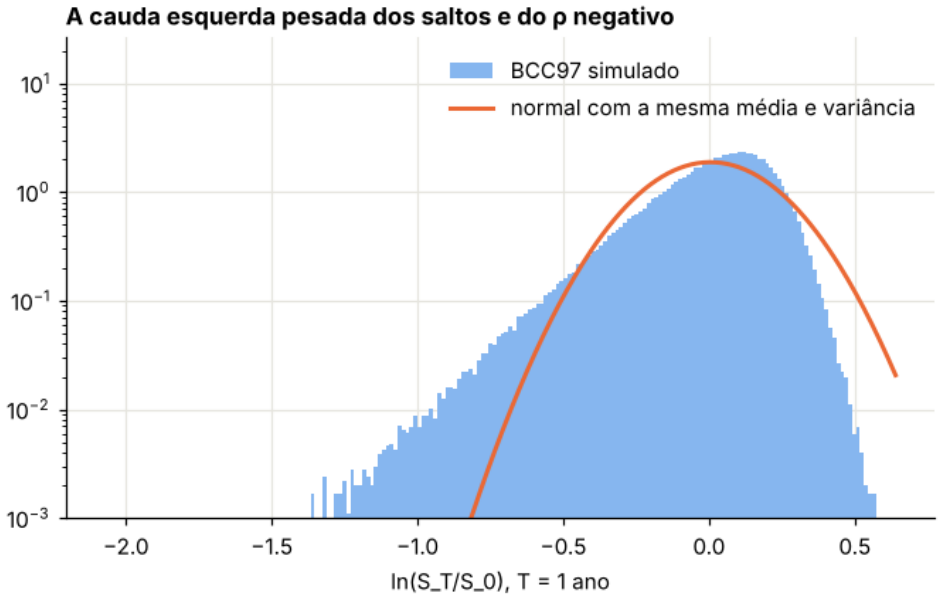

h = 0.02
us = h * np.arange(-3, 4)
L = np.array([log_cf(u) for u in us])
d2 = (L[2] - 2 * L[3] + L[4]) / h**2
d3 = (L[5] - 2 * L[4] + 2 * L[2] - L[1]) / (2 * h**3)
d4 = (L[5] - 4 * L[4] + 6 * L[3] - 4 * L[2] + L[1]) / h**4
k2, k3, k4 = (-d2).real, (1j * d3).real, d4.real
assim_cf, curt_cf = k3 / k2**1.5, k4 / k2**2

rng = np.random.default_rng(3)
ST, D = simular_bcc(BATES, T3, 50, 400_000, rng)
X = np.log(ST / S0)
c = X - X.mean()
assim_mc = np.mean(c**3) / np.mean(c**2) ** 1.5
curt_mc = np.mean(c**4) / np.mean(c**2) ** 2 - 3
tab3 = pd.DataFrame({"momento": ["desvio-padrão de ln S_T", "assimetria", "excesso de curtose"],
                     "função característica": [np.sqrt(k2), assim_cf, curt_cf],
                     "simulação": [c.std(), assim_mc, curt_mc]})
tabela(tab3)

fig, ax = plt.subplots()
ax.hist(X, bins=200, density=True, color=RAMPA[0], label="BCC97 simulado")
gx = np.linspace(X.min(), X.max(), 400)
ax.plot(gx, norm.pdf(gx, X.mean(), X.std()), color=LARANJA, label="normal com a mesma média e
↪  variância")
ax.set_yscale("log"); ax.set_ylim(1e-3, None)
ax.set_xlabel("ln(S_T/S_0), T = 1 ano"); ax.set_title("A cauda esquerda pesada dos saltos e do ρ
↪  negativo")
ax.legend(); plt.show()

ok(abs(np.sqrt(k2) / c.std() - 1) < 0.01, "o desvio-padrão simulado confere com o da função
↪  característica a menos de 1%")
ok(abs(assim_mc - assim_cf) < 0.05, "a assimetria simulada confere com a da função
↪  característica")
ok(abs(curt_mc - curt_cf) < 0.3, "o excesso de curtose simulado confere com o da função
↪  característica")
ok(assim_cf < -0.5 and curt_cf > 1, "a distribuição é bem assimétrica à esquerda e de cauda
↪  pesada")
```

momento	função característica	simulação
desvio-padrão de ln S_T	0,211	0,211
assimetria	-1,194	-1,185
excesso de curtose	2,609	2,587




```
OK — o desvio-padrão simulado confere com o da função característica a menos de 1%
OK — a assimetria simulada confere com a da função característica
OK — o excesso de curtose simulado confere com o da função característica
OK — a distribuição é bem assimétrica à esquerda e de cauda pesada
```

Em escala logarítmica, a diferença para a normal fica nítida: a cauda esquerda do modelo é muito mais grossa. É ela que dá o skew das volatilidades implícitas (Módulo 2).

---

5. Puts americanas no BCC97

Livro: §12.3.2 e §12.5.3

5.1 O que o livro calcula

O livro aplica o LSM com 11 funções de base (como no Módulo 12) às trajetórias do seu simulador e reporta a diferença entre a put americana (LSM) e a europeia (Fourier) como **prêmio de exercício antecipado**. Dois problemas:

1. As trajetórias carregam o viés da Seção 3. A diferença mistura prêmio e viés.
2. O título da §12.5.3 fala de "American Call Options", mas o script avalia puts. É bom lembrar que, **sem dividendos**, uma call americana não deve ser exercida antes (Módulo 9): o prêmio da call é zero.

5.2 A forma limpa: LSM com controle

Seguimos o Módulo 12: trajetórias do esquema exato, LSM com as 11 funções de base, e a put europeia simulada nas mesmas trajetórias como variável de controle, com o λ^*

estimado. O prêmio passa a ser **americana com controle − europeia de Fourier**, e o viés comum (se houver) se cancela.

Laboratório 4 — Prêmio de exercício antecipado

```
def lsm(S, v, r, K, T, tipo="put"):
    """LSM com as 11 funções de base do livro. Devolve os fluxos descontados até 0 e o payoff
    ↪ europeu descontado."""
    M = S.shape[0] - 1
    dt = T / M
    h = np.maximum(K - S, 0) if tipo == "put" else np.maximum(S - K, 0)
    fd = np.exp(-(r[1:] + r[:-1]) / 2 * dt)
    V = h[-1].copy()
    for t in range(M - 1, 0, -1):
        V = V * fd[t]
        itm = h[t] > 0
        if itm.sum() > 20:
            s_, v_, r_ = S[t, itm] / S0, v[t, itm], r[t, itm]
            X = np.column_stack([np.ones_like(s_), r_, v_, s_, r_**2, v_**2, s_**2,
                                 v_ * r_, s_ * r_, s_ * v_, s_ * v_ * r_])
            cont = X @ np.linalg.lstsq(X, V[itm], rcond=None)[0]
            idx = np.where(itm)[0][h[t, itm] > cont]
            V[idx] = h[t, idx]
    return V * fd[0], np.prod(fd, axis=0) * h[-1]

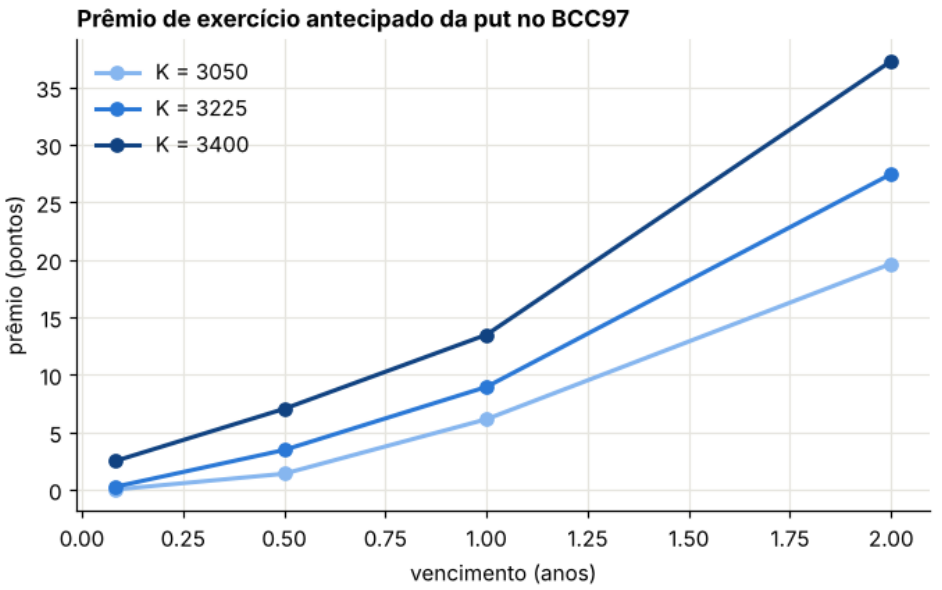

PRAZOS4 = [1 / 12, 0.5, 1.0, 2.0]
rng = np.random.default_rng(4)
t0 = time.time()
linhas = []
for T in PRAZOS4:
    Sp, vp, rp = simular_bcc(BATES, T, 50, 50_000, rng, trajetorias=True)
    C_f = calls_bcc(BATES, K2, T)
    B0T = zcb_cir(T=T, **CIR)
    for K, cf_ in zip(K2, C_f):
        P_f = cf_ - S0 + K * B0T                    # paridade put-call
        Y, Xe = lsm(Sp, vp, rp, K, T)
        lam_cv = np.cov(Xe, Y)[0, 1] / Xe.var()
        am = max((Y - lam_cv * (Xe - P_f)).mean(), K - S0)
        Yc, Xc = lsm(Sp, vp, rp, K, T, "call")
        linhas.append((T, K, P_f, am, am - P_f, Yc.mean() - Xc.mean()))
print(f"tempo: {time.time() - t0:.1f} s")
tab4 = pd.DataFrame(linhas, columns=["T", "K", "put europeia (Fourier)", "put americana (LSM +
↪ controle)",
                                     "prêmio de exercício", "call americana − europeia (mesmas
                                     ↪ trajetórias)"])
tabela(tab4)
```

```
tempo: 15.3 s
```

T	K	put europeia (Fourier)	put americana (LSM + controle)	prêmio de exercício	call americana − europeia (mesmas trajetórias)
0,083	3.050,000	17,325	17,342	0,017	-0,095
0,083	3.225,000	63,567	63,827	0,260	0,056
0,083	3.400,000	177,292	179,795	2,503	0,021
0,500	3.050,000	94,347	95,734	1,386	-0,227
0,500	3.225,000	152,791	156,242	3,451	-0,228
0,500	3.400,000	237,523	244,552	7,029	-0,124
1,000	3.050,000	147,377	153,506	6,129	-1,128
1,000	3.225,000	209,386	218,324	8,938	-0,061
1,000	3.400,000	288,972	302,445	13,473	-0,155
2,000	3.050,000	208,871	228,512	19,642	0,178
2,000	3.225,000	271,733	299,177	27,444	0,474
2,000	3.400,000	346,186	383,448	37,262	0,005

```
fig, ax = plt.subplots()
for K, cor in zip(K2, RAMPA):
    d = tab4[tab4["K"] == K]
    ax.plot(d["T"], d["prêmio de exercício"], "o-", color=cor, label=f"K = {K:.0f}")
ax.set_xlabel("vencimento (anos)"); ax.set_ylabel("prêmio (pontos)")
ax.set_title("Prêmio de exercício antecipado da put no BCC97")
ax.legend(); plt.show()

ok((tab4["prêmio de exercício"] > -0.5).all(), "a put americana nunca vale menos que a europeia
↪  (a menos do ruído)")
for K in K2:
    d = tab4[tab4["K"] == K]["prêmio de exercício"].values
    ok(np.all(np.diff(d) > -0.5), f"K = {K:.0f}: o prêmio cresce com o vencimento")
dc = tab4["call americana − europeia (mesmas trajetórias)"]
ok(dc.max() < 0.6, "sem dividendos, a call americana não tem prêmio sobre a europeia (diferença
↪  positiva só de ruído)")
ok(dc.min() > -1.5, "as diferenças negativas são pequenas: custo de exercícios por erro de
↪  regressão do LSM")
```




```
OK — a put americana nunca vale menos que a europeia (a menos do ruído)
OK — K = 3050: o prêmio cresce com o vencimento
OK — K = 3225: o prêmio cresce com o vencimento
OK — K = 3400: o prêmio cresce com o vencimento
OK — sem dividendos, a call americana não tem prêmio sobre a europeia (diferença positiva só de ruído)
OK — as diferenças negativas são pequenas: custo de exercícios por erro de regressão do LSM
```

O que aprendemos. O prêmio cresce com o vencimento e com o strike, como o livro mostra. Mas os números do livro para 3 anos (49 a 75 pontos) incluem o viés da Seção 3, que em 3 anos passa de 40 pontos. Com trajetórias corretas e a europeia como controle, o prêmio é uma diferença limpa. E a call americana, como manda a teoria sem dividendos, não tem prêmio. As pequenas diferenças negativas vêm de exercícios por engano: a regressão às vezes estima o valor de continuação abaixo do valor intrínseco, e todo exercício antecipado de uma call sem dividendos destrói valor. É o viés para baixo do LSM (Módulo 9), visível aqui porque o valor certo é conhecido.

---

6. Síntese

Tema	Lição
Saltos na simulação	integram-se exatamente: $N \sim$ Poisson, soma de log-saltos normal dado N, compensador $-r_J \Delta t$
Teste da martingal	detecta em uma linha a falta do compensador ($\approx S_0 \lvert r_J \rvert T$)
Viés da §12.3.1	não é do Euler: é o `v[t]` com $\rho \approx -1$, parcialmente mascarado pelo moment matching
Distribuição	os cumulantes da função característica conferem a simulação inteira, não só a média
Americanas	prêmio = americana com controle − europeia de Fourier; call sem dividendos não tem prêmio

7. Erratas e notas sobre o capítulo 12

Onde	O quê
§12.5.1, `B96_generate_paths`	`rj` é calculado e nunca usado: falta o compensador $-r_J \Delta t$ de (12.1); o índice perde $\lvert r_J \rvert$ de drift por ano
Eq. (12.1) e código	saltos aditivos com um único tamanho para todos os y_t saltos do passo; o exato é $e^{N\mu_J + \sqrt{N}\delta z}$
Eq. (12.1) e código	a variância do passo é `v[t]`, do fim do passo (Módulo 12); com $\rho = -0{,}999$, o drift espúrio é $-0{,}475$ por ano

Onde	O quê
§12.3.1	o viés de 22 a 52 pontos atribuído ao número de passos vem do `v[t]`; o esquema exato acerta com 50 passos por ano a partir de 6 meses (só no prazo de 1 mês, com 4 passos, há viés de Euler de 1 a 2 pontos)
§12.3.2	a "diferença" americana − europeia mistura prêmio e viés de simulação
§12.5.3, título	"American Call Options", mas o script avalia puts
Parâmetros do livro	$\rho = -0{,}999$, $\delta = 0$ e $\lambda = 0{,}008$ vêm da calibração degenerada do Módulo 14
Scripts	Python 2 (`xrange`, `print`, `I / 2`)

8. Exercícios

Exercício 1. Mostre que, no esquema aditivo do livro, $\mathbb{E}[S_t \,|\, S_s] = S_s(e^{(\bar{r}-r_J)\Delta t} + \lambda\Delta t\,(e^{\mu_J+\delta^2/2}-1))$ quando se usa v_s, e que isso difere de $S_s e^{\bar{r}\Delta t}$ por um termo de ordem Δt^2.

Solução

O primeiro termo tem média $e^{(\bar{r}-r_J)\Delta t}$; o segundo, $\mathbb{E}[e^{\mu_J+\delta z}-1]\,\mathbb{E}[y] = (e^{\mu_J+\delta^2/2}-1)\lambda\Delta t = r_J\Delta t$. Logo $e^{\bar{r}\Delta t}e^{-r_J\Delta t} + r_J\Delta t \approx e^{\bar{r}\Delta t}(1 - r_J\Delta t + r_J\Delta t) + O(\Delta t^2)$. Com o compensador, a forma aditiva acerta a média em primeira ordem; sem ele, erra por $r_J\Delta t$ a cada passo, como no Laboratório 1.

Exercício 2. No Laboratório 2, troque apenas `olhar_frente=True` por `False`, mantendo o resto do código do livro. O viés desaparece?

Solução

O viés positivo desaparece. Sobra um viés **negativo**, o do compensador ausente: com $r_J \approx -0{,}3\%$ ao ano, o índice descontado fica cerca de 11 pontos abaixo de S_0 em 1 ano e 30 em 3 anos, e as calls de 1 ano saem 5 a 7 pontos abaixo da fórmula. A correção `bias` não o remove, porque só zera a média do choque browniano, não o drift ausente.

Exercício 3. Calcule por Monte Carlo o prêmio de exercício da put com $K = 3.400$ e $T = 1$ usando o simulador do livro, como no livro. Compare com o Laboratório 4.

Solução

Use `simular_bcc(LIVRO, ..., **LIVRO_CFG, trajetorias=True)` e o `lsm` sem controle, e subtraia a put europeia de Fourier. A diferença sai inflada pelo viés das trajetórias. Com o controle europeu nas mesmas trajetórias, boa parte do viés se cancela, e o

prêmio volta para perto do valor limpo.

Exercício 4. Use os cumulantes do Laboratório 3 para estimar a vol. implícita no dinheiro e o skew de 1 ano pela expansão de Gram-Charlier (Backus, Foresi e Wu). Compare com as vol. implícitas de Lewis.

Solução

Na expansão, $\sigma_{impl}(k) \approx \sigma\big(1 - \frac{\gamma_1}{3!}\, d + \frac{\gamma_2}{4!}(d^2 - 1)\big)$, com d a variável padronizada do strike, γ_1 a assimetria e γ_2 o excesso de curtose. A aproximação acerta o sinal e a ordem de grandeza do skew, mas perde precisão longe do dinheiro.

```
print(f"Módulo 15 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 15 concluído: 20 verificações aprovadas.
```

Capítulo 16

Hedge dinâmico e síntese do curso

Introdução

A precificação responde quanto vale um derivativo; o **hedge** responde como neutralizar o risco de tê-lo vendido. Na teoria de Black-Scholes, as duas perguntas têm a mesma resposta: a carteira que replica a opção é também o seu hedge perfeito. Na prática, o hedge é rebalanceado em datas discretas, a volatilidade verdadeira não é a usada no modelo e há riscos, como saltos e mudanças de volatilidade, que o ativo sozinho não cobre.

O capítulo estuda o **hedge dinâmico** nessas condições: mede o erro de hedge quando o rebalanceamento é discreto, mostra de onde vem o resultado do hedge quando a volatilidade realizada difere da implícita e avalia o hedge de uma opção americana no modelo BCC97, inclusive com uma segunda opção para neutralizar o vega.

O capítulo termina com uma síntese do curso: como as etapas de modelagem, precificação, calibração, simulação e hedge se encaixam num processo único de precificação calibrada ao mercado.

Fundamentos teóricos

O P&L do hedge delta

Considere quem vende uma opção por um preço calculado com volatilidade σ_h e faz hedge contínuo com o delta de Black-Scholes nessa mesma volatilidade, enquanto o ativo segue $dS_t = \mu_t S_t\,dt + \sigma_t S_t\,dW_t$ com volatilidade realizada σ_t.

Proposição 16.1 (Fórmula do P&L de hedge). *O valor no vencimento da carteira (opção vendida, hedge e caixa), descontado, é*

$$\mathrm{P\&L}_T = \int_0^T e^{-rt}\,\tfrac{1}{2}\,\Gamma_t\,S_t^2\,(\sigma_h^2 - \sigma_t^2)\,dt, \tag{16.1}$$

com Γ_t a gama de Black-Scholes calculada com σ_h (El Karoui et al., 1998).

Demonstração. Seja V o preço de Black-Scholes com σ_h. Por Itô com a dinâmica verdadeira, $dV = (\partial_t V + \frac{1}{2}\sigma_t^2 S^2 \Gamma)dt + \Delta\, dS$. A EDP com σ_h dá $\partial_t V = rV - rS\Delta - \frac{1}{2}\sigma_h^2 S^2\Gamma$. A carteira $-V + \Delta S + (V - \Delta S)$ em caixa tem variação descontada $e^{-rt}\frac{1}{2}S^2\Gamma(\sigma_h^2 - \sigma_t^2)\,dt$; integrando obtém-se (16.1). ■

Corolário 16.2. *Se $\sigma_t = \sigma_h$, o P&L é zero em todas as trajetórias: a replicação é perfeita. Se $\sigma_h > \sigma_t$ em todo instante, o vendedor ganha com certeza; mas o tamanho do ganho depende da trajetória, através de $\Gamma_t S_t^2$, que é máximo perto do strike e do vencimento.*

Hedge em tempo discreto

Proposição 16.3 (Erro de hedge discreto). *Com rebalanceamento em N datas igualmente espaçadas e $\sigma_t = \sigma_h = \sigma$, o P&L tem média aproximadamente zero e desvio-padrão*

$$\mathrm{dp(P\&L)} \approx \sqrt{\frac{\pi}{4}}\,\frac{\mathcal{V}\,\sigma}{\sqrt{N}}, \tag{16.2}$$

onde $\mathcal{V}$ é o vega da opção na data inicial.

Esboço. Em cada intervalo, o erro local é $\frac{1}{2}\Gamma S^2(\sigma^2\Delta t - (\Delta S/S)^2)$, de média zero e variância $\frac{1}{2}\Gamma^2 S^4\sigma^4\Delta t^2$. Somando sobre os intervalos e aproximando $\mathbb{E}\int \Gamma^2 S^4\,dt$ com a densidade lognormal, chega-se à expressão, que relaciona o resultado ao vega pela identidade $\mathcal{V} = \Gamma S^2\sigma T$ (Wilmott, 2006). ■

O erro cai com $N^{-1/2}$: dobrar a precisão exige quadruplicar a frequência de rebalanceamento, e os custos de transação crescem na mesma proporção. Esse é o conflito estudado por Leland (1985).

Mercados incompletos

Observação 16.4 (Volatilidade estocástica e saltos). No BCC97, o ativo sozinho não replica uma opção: há riscos de variância e de salto não negociados. O hedge delta neutraliza apenas a exposição a dW^1; os resíduos vêm dos termos em dv e dN. Acrescentar uma segunda opção como instrumento de hedge permite neutralizar também o vega:

$$\Pi = V - \Delta S - \eta\, U, \qquad \eta = \frac{\partial_v V}{\partial_v U}, \quad \Delta = \partial_S V - \eta\,\partial_S U. \tag{16.3}$$

Observação 16.5 (Média do P&L). Sob $\mathbb{Q}$, qualquer estratégia autofinanciada descontada é um martingal e tem P&L esperado zero. Um P&L médio diferente de zero numa simulação sob $\mathbb{Q}$ indica viés do simulador (deriva incorreta, saltos sem compensador) ou do estimador do delta, não uma oportunidade de lucro.

Roteiro do capítulo

Curso: Derivativos com Python, a partir de *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015). **Este módulo cobre:** capítulos 13 e 14 do livro.

O que você vai conseguir fazer ao final

1. Montar uma carteira replicante autofinanciável e provar que o P&L médio do hedge é zero sob Q.
2. Medir como o erro de hedge diminui com a frequência de rebalanceamento.
3. Estimar deltas a partir de simulações sem que o ruído domine o resultado.
4. Fazer o hedge de uma put americana com os deltas da regressão do LSM, fora da amostra.
5. Explicar de onde vem o "lucro médio" do hedge no BCC97 do livro.
6. Separar risco de volatilidade e risco de salto, e medir o que uma opção adicional na carteira resolve.

Onde estamos

O preço de um derivativo é o custo da sua replicação. Os Módulos 1 a 15 trataram do preço; este trata da replicação. É o teste final do modelo: um modelo pode precificar bem as opções do mercado e ainda assim deixar o vendedor exposto, se os riscos que ele carrega não podem ser cobertos pelos instrumentos disponíveis.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.interpolate as sci
from scipy.integrate import simpson
from scipy.optimize import brentq
from scipy.interpolate import RegularGridInterpolator
import warnings
warnings.simplefilter("ignore", np.exceptions.RankWarning)
from scipy.stats import norm
from IPython.display import display

RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

def bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d1 - sig * np.sqrt(T))

def vega_bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return S * norm.pdf(d1) * np.sqrt(T)

def vol_impl(preco, S, K, T, r):
    piso = max(S - K * np.exp(-r * T), 0.0)
    if not (piso + 1e-12 < preco < S):
        return np.nan
    return brentq(lambda s: float(bsm(S, K, T, r, s)) - preco, 1e-4, 4.0)

def cf_heston(u, T, v0, kappa, theta, sigma, rho):
    """Função característica de ln(S_T/S_0) - rT no Heston, forma estável (Módulo 11)."""
```

```
    xi = kappa - rho * sigma * 1j * u
    d = np.sqrt(xi**2 + sigma**2 * (1j * u + u**2))
    g = (xi - d) / (xi + d)
    e = np.exp(-d * T)
    A = kappa * theta / sigma**2 * ((xi - d) * T - 2 * np.log((1 - g * e) / (1 - g)))
    B = (xi - d) / sigma**2 * (1 - e) / (1 - g * e)
    return np.exp(A + B * v0)

def cf_saltos(u, T, lam, mu, delta):
    """Parte de saltos de Merton, com o drift compensado (11.13)-(11.14)."""
    omega = -lam * (np.exp(mu + 0.5 * delta**2) - 1)
    return np.exp((1j * u * omega + lam * (np.exp(1j * u * mu - 0.5 * u**2 * delta**2) - 1)) * T)

U_FINO = np.linspace(0, 2000, 200_001)
U_CAL = np.linspace(0, 300, 3001)

def calls(S0, K, T, r, heston, saltos=None, u=U_CAL):
    """Calls europeias pela fórmula de Lewis (11.12), vetorizada em (K, T, r) do mesmo tamanho.
    heston = (v0, κ_v, θ_v, σ_v, ρ); saltos = (λ, μ_J, δ) ou None."""
    K, T, r = (np.atleast_1d(np.asarray(x, float)) for x in (K, T, r))
    k = np.log(S0 / K) + r * T
    phi = cf_heston(u[None, :] - 0.5j, T[:, None], *heston)
    if saltos is not None:
        phi = phi * cf_saltos(u[None, :] - 0.5j, T[:, None], *saltos)
    h = np.exp(1j * u[None, :] * k[:, None]) * phi
    integral = simpson(h.real / (u**2 + 0.25), x=u, axis=1)
    return S0 - np.sqrt(S0 * K) * np.exp(-0.5 * r * T) / np.pi * integral

def zcb_cir(r0, kappa, theta, sigma, T):
    """Zero-cupom no CIR, (11.7)-(11.9) com t = 0."""
    g = np.sqrt(kappa**2 + 2 * sigma**2)
    e = np.exp(g * T) - 1
    D = 2 * g + (kappa + g) * e
    return (2 * g * np.exp((kappa + g) * T / 2) / D) ** (2 * kappa * theta / sigma**2) * np.exp(-2
    ↪  * e / D * r0)

def fator_ft(xh, kappa, theta, sigma, dt, z):
    """Um passo Full Truncation (Módulo 12): devolve (x truncado, x˜ auxiliar)."""
    p = np.maximum(xh, 0)
    xh = xh + kappa * (theta - p) * dt + sigma * np.sqrt(p * dt) * z
    return np.maximum(xh, 0), xh

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. Hedge dinâmico: a teoria em quatro passos

Livro: §13.1

Passo 1 — o delta. Para uma put $P(t,S)$, $\Delta_t = \partial P/\partial S$. Carregar $-\Delta_t$ unidades do índice junto com a put anula a variação de primeira ordem: $dP - \Delta\, dS = 0$ para movimentos pequenos de S.

Passo 2 — a carteira replicante. Com Δ_t ações e $\gamma_t = P_t - \Delta_t S_t$ no título sem risco, a carteira vale P_t. Ela é **autofinanciável** se, a cada rebalanceamento, a compra de ações é paga pela venda de título e vice-versa:

$$\Delta_t S_t + \gamma_t B_t = \Delta_{t-\Delta t} S_t + \gamma_{t-\Delta t} B_t. \tag{16.4}$$

Passo 3 — a média do erro. Seja Π_t o valor da carteira autofinanciável, iniciada em $\Pi_0 = P_0$. Descontada, ela é uma soma de termos $\Delta_s \, d\tilde{S}_s$, onde $\tilde{S}$ é o índice descontado. Se $\tilde{S}$ é martingal sob Q, a carteira descontada também é, qualquer que seja a regra de delta. Então

$$\mathbb{E}^Q[\tilde{\Pi}_\tau - \tilde{h}_\tau] = P_0 - \mathbb{E}^Q[\tilde{h}_\tau]. \tag{16.5}$$

Se P_0 é o preço justo para a regra de exercício τ usada, a **média do P&L descontado é zero**, quer o hedge seja bom ou ruim. Numa simulação sob Q, um P&L médio diferente de zero só pode vir de preço errado ou de simulação errada. Hedge reduz **variância**, não cria média.

Passo 4 — o tamanho do erro. Com rebalanceamento a cada $\Delta t = T/N$ no BSM, o erro de cada intervalo vem do gama: $\frac{1}{2}\Gamma S^2(\sigma^2 \Delta t - (\Delta S/S)^2)$. Somando N termos de média zero e variância proporcional a Δt^2, o desvio-padrão do P&L cai como $1/\sqrt{N}$. Derman e Kamal (1999) dão a aproximação

$$\mathrm{dp(P\&L)} \approx \sqrt{\frac{\pi}{4}} \, \frac{\sigma \, \mathcal{V}}{\sqrt{N}}, \qquad \mathcal{V} = \text{vega da opção}. \tag{16.6}$$

Laboratório 1 — Hedge de uma put europeia no BSM

Put no dinheiro, $S_0 = K = 100$, $T = 1$, $r = 5\%$, $\sigma = 20\%$, hedge com o delta exato do BSM, 20.000 trajetórias.

```
S0b, Kb, Tb, rb, sigb = 100.0, 100.0, 1.0, 0.05, 0.2


def put_bsm(S, K, T, r, sig):
    return bsm(S, K, T, r, sig) - S + K * np.exp(-r * T)


def delta_put_bsm(S, K, T, r, sig):
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    return norm.cdf(d1) - 1


def hedge_bsm_europeia(N, I, rng):
    dt = Tb / N
    S = np.full(I, S0b)
    d = delta_put_bsm(S, Kb, Tb, rb, sigb)
    B = put_bsm(S0b, Kb, Tb, rb, sigb) - d * S
    for j in range(1, N + 1):
        S = S * np.exp((rb - 0.5 * sigb**2) * dt + sigb * np.sqrt(dt) * rng.standard_normal(I))
        B = B * np.exp(rb * dt)
        port = d * S + B
        if j < N:
            d_novo = delta_put_bsm(S, Kb, Tb - j * dt, rb, sigb)
            B = port - d_novo * S                       # autofinanciável
            d = d_novo
    return (port - np.maximum(Kb - S, 0)) * np.exp(-rb * Tb)


rng = np.random.default_rng(13)
```

```
vega0 = vega_bsm(S0b, Kb, Tb, rb, sigb)
linhas = []
for N in [12, 50, 200, 800]:
    pl = hedge_bsm_europeia(N, 20_000, rng)
    linhas.append((N, pl.mean(), pl.std() / np.sqrt(len(pl)), pl.std(), np.sqrt(np.pi / 4) * sigb
↪  * vega0 / np.sqrt(N)))
tab1 = pd.DataFrame(linhas, columns=["rebalanceamentos N", "P&L médio", "erro-padrão da média",
↪  "dp do P&L", "Derman-Kamal"])
tabela(tab1)
incl = np.polyfit(np.log(tab1["rebalanceamentos N"]), np.log(tab1["dp do P&L"]), 1)[0]
print("put BSM:", f3(put_bsm(S0b, Kb, Tb, rb, sigb)), "| inclinação log-log do dp:", f3(incl))

ok(np.all(np.abs(tab1["P&L médio"]) < 3 * tab1["erro-padrão da média"] + 0.01), "o P&L médio é
↪  zero, para qualquer N")
ok(abs(incl + 0.5) < 0.05, "o desvio-padrão do P&L cai como 1/√N")
ok(np.all(np.abs(tab1["dp do P&L"] / tab1["Derman-Kamal"] - 1) < 0.15), "e a fórmula de
↪  Derman-Kamal acerta a menos de 15%")
```

rebalanceamentos N	P&L médio	erro-padrão da média	dp do P&L	Derman-Kamal
12	-0,006	0,013	1,852	1,920
50	-0,003	0,007	0,923	0,941
200	0,001	0,003	0,462	0,470
800	0,000	0,002	0,237	0,235

```
put BSM: 5,574 | inclinação log-log do dp: -0,491
OK — o P&L médio é zero, para qualquer N
OK — o desvio-padrão do P&L cai como 1/√N
OK — e a fórmula de Derman-Kamal acerta a menos de 15%
```

Num mundo BSM, o hedge funciona como a teoria promete: a média é zero e o erro some com rebalanceamentos mais frequentes. Quadruplicar a frequência corta o erro pela metade.

---

2. Deltas tirados de uma simulação

Livro: §13.2, equações (13.1) e (13.2)

Para uma put americana não há fórmula. O livro estima o delta inicial pela diferença de dois preços LSM:

$$\Delta_0 \approx \frac{P^{LSM}(S_0 + 0{,}01) - P^{LSM}(S_0)}{0{,}01}. \tag{16.7}$$

No script da §13.5.1, cada um dos dois preços vem de uma simulação **com números aleatórios diferentes** (a semente é fixada uma vez, antes das duas chamadas). Isso é um problema sério: o erro de Monte Carlo de cada preço, da ordem de 0,02, é dividido por 0,01. O ruído do delta fica da ordem de 2, quando o delta verdadeiro está entre -1 e 0.

O remédio é usar **números aleatórios comuns**: as duas simulações com os mesmos choques. Então os erros de Monte Carlo dos dois preços são quase iguais e se cancelam na diferença. Uma alternativa sem simulação extra é derivar a própria regressão do LSM em S, que é o que o livro faz nos instantes seguintes (13.2).

Laboratório 2 — Delta inicial da put americana de Longstaff e Schwartz

$S_0 = 36$, $K = 40$, $T = 1$, $r = 6\%$, $\sigma = 20\%$, 50 datas de exercício, 50.000 trajetórias, regressão polinomial de grau 9 em todas as trajetórias (como no script). A referência é a árvore CRR com 2.000 passos (Módulo 6).

```
S0a, Ka, Ta, ra, siga, Ma = 36.0, 40.0, 1.0, 0.06, 0.2, 50


def crr_put_americana(S0, K, T, r, sig, M):
    dt = T / M
    u = np.exp(sig * np.sqrt(dt)); d = 1 / u
    q = (np.exp(r * dt) - d) / (u - d); desc = np.exp(-r * dt)
    j = np.arange(M + 1)
    V = np.maximum(K - S0 * u**j * d ** (M - j), 0.0)
    for t in range(M - 1, -1, -1):
        V = desc * (q * V[1:] + (1 - q) * V[:-1])
        j = np.arange(t + 1)
        V = np.maximum(V, K - S0 * u**j * d ** (t - j))
        if t == 1:
            V1 = V.copy()
    return V[0], (V1[1] - V1[0]) / (S0 * (u - d))


def lsm_bsm(S0, Z, regra_livro=True, D=9):
    """LSM do script da §13.5.1. Z: normais (M, I). Devolve preço, trajetórias, coeficientes e
    ↪ matriz de exercício."""
    M, I = Z.shape
    dt = Ta / M
    df = np.exp(-ra * dt)
    S = S0 * np.exp(np.vstack([np.zeros(I), np.cumsum((ra - 0.5 * siga**2) * dt + siga *
↪ np.sqrt(dt) * Z, axis=0)]))
    h = np.maximum(Ka - S, 0)
    V = h[-1].copy()
    rg = np.zeros((M + 1, D + 1))
    ex = np.zeros((M + 1, I), dtype=bool)
    for t in range(M - 1, 0, -1):
        rg[t] = np.polyfit(S[t], V * df, D)
        C = np.maximum(np.polyval(rg[t], S[t]), 0.0)
        ex[t] = (h[t] >= C) if regra_livro else ((h[t] > C) & (h[t] > 0))
        V = np.where(ex[t], h[t], V * df)
    return V.mean() * df, S, rg, ex


P_crr, D_crr = crr_put_americana(S0a, Ka, Ta, ra, siga, 2000)
rng = np.random.default_rng(50000)
ds = 0.01
d_indep, d_crn, d_reg, precos = [], [], [], []
for _ in range(12):
    Z1 = rng.standard_normal((Ma, 50_000))
    Z2 = rng.standard_normal((Ma, 50_000))
    V_up, _, rg, _ = lsm_bsm(S0a + ds, Z1)
    V_0, _, _, _ = lsm_bsm(S0a, Z2)
    V_0crn, _, _, _ = lsm_bsm(S0a, Z1)
    d_indep.append((V_up - V_0) / ds)
    d_crn.append((V_up - V_0crn) / ds)
    d_reg.append(np.polyval(np.polyder(rg[1]), S0a))
    precos.append(V_0)
```

```
tab2 = pd.DataFrame({"método": ["dois LSM independentes (livro)", "números aleatórios comuns",
↪  "derivada da regressão em t = Δt"],
                     "delta médio": [np.mean(d_indep), np.mean(d_crn), np.mean(d_reg)],
                     "desvio-padrão (12 repetições)": [np.std(d_indep), np.std(d_crn),
                      ↪  np.std(d_reg)]})
tabela(tab2)
print("CRR (2.000 passos): preço", f3(P_crr), "| delta", f3(D_crr), "| LSM médio:",
↪  f3(np.mean(precos)))

t2 = tab2.set_index("método")["desvio-padrão (12 repetições)"]
ok(t2["dois LSM independentes (livro)"] > 0.5, "com simulações independentes, o delta tem
↪  desvio-padrão acima de 0,5: é ruído")
ok(t2["números aleatórios comuns"] < 0.1, "com números aleatórios comuns, o desvio-padrão cai
↪  abaixo de 0,1")
ok(t2["dois LSM independentes (livro)"] > 10 * t2["números aleatórios comuns"], "uma redução de
↪  mais de 10 vezes")
ok(abs(np.mean(d_crn) - D_crr) < 0.05 and abs(np.mean(d_reg) - D_crr) < 0.05, "os dois métodos
↪  corretos acertam o delta da árvore a menos de 0,05")
```

método	delta médio	desvio-padrão (12 repetições)
dois LSM independentes (livro)	0,908	1,631
números aleatórios comuns	-0,714	0,081
derivada da regressão em t = Δt	-0,705	0,044

```
CRR (2.000 passos): preço 4,487 | delta -0,697 | LSM médio: 4,462
OK — com simulações independentes, o delta tem desvio-padrão acima de 0,5: é ruído
OK — com números aleatórios comuns, o desvio-padrão cai abaixo de 0,1
OK — uma redução de mais de 10 vezes
OK — os dois métodos corretos acertam o delta da árvore a menos de 0,05
```

O delta inicial do livro, $-0{,}587$, é um sorteio de uma distribuição muito larga. Com números aleatórios comuns, ou derivando a regressão, o delta fica estável e perto da árvore. Os números comuns não zeram o ruído: com S_0 deslocado, algumas decisões de exercício mudam, e a diferença deixa de ser suave. A derivada da regressão evita esse problema e não custa nenhuma simulação extra.

---

3. Hedge da put americana no BSM com os deltas da regressão

Livro: §13.2 e §13.5.2

O livro reaproveita os coeficientes da regressão do LSM: em cada data, o delta é a derivada do polinômio de continuação em S_t, e a decisão de exercício é a da regra do LSM. O P&L é calculado nas **mesmas** trajetórias usadas na regressão.

Dois cuidados que fazemos aqui:

1. **Fora da amostra.** Treinamos a regressão em 50.000 trajetórias e fazemos o hedge em outras 10.000. Nas mesmas trajetórias, a regra de exercício "conhece o futuro" e o P&L sai otimista (Módulo 9).
2. **A regra de exercício do script.** O script exerce quando $h_t \geq \hat{C}_t$, com $\hat{C}_t$ truncado em zero. Fora do dinheiro, $h_t = 0$; onde o polinômio fica negativo,

$\hat{C}_t = 0$; e $0 \geq 0$ marca um **exercício** de uma put sem valor intrínseco. Na contagem do hedge, isso encerra a operação antes da hora. O correto é exigir $h_t > \hat{C}_t$ e $h_t > 0$.

Laboratório 3 — P&L do hedge, regra do livro e regra corrigida

```
def hedge_americana_bsm(Z_trein, Z_teste, regra_livro):
    V0, _, rg, _ = lsm_bsm(S0a, Z_trein, regra_livro)
    M, I = Z_teste.shape
    dt = Ta / M
    S = S0a * np.exp(np.vstack([np.zeros(I), np.cumsum((ra - 0.5 * siga**2) * dt + siga *
↪   np.sqrt(dt) * Z_teste, axis=0)]))
    h = np.maximum(Ka - S, 0)
    d = np.full(I, np.polyval(np.polyder(rg[1]), S0a))
    B = V0 - d * S0a
    vivo = np.ones(I, dtype=bool)
    pl = np.zeros(I)
    falso = np.zeros(I, dtype=bool)
    for t in range(1, M + 1):
        B = B * np.exp(ra * dt)
        port = d * S[t] + B
        if t == M:
            ex = vivo.copy()
        else:
            C = np.maximum(np.polyval(rg[t], S[t]), 0.0)
            regra = (h[t] >= C) if regra_livro else ((h[t] > C) & (h[t] > 0))
            ex = vivo & regra
            falso |= ex & (h[t] == 0)
        pl[ex] = (port[ex] - h[t, ex]) * np.exp(-ra * t * dt)
        vivo &= ~ex
        if t < M:
            d_novo = np.clip(np.polyval(np.polyder(rg[t]), S[t]), -1, 0)
            B = port - d_novo * S[t]
            d = d_novo
    return V0, pl, falso.mean()


rng = np.random.default_rng(7)
Z_trein = rng.standard_normal((Ma, 50_000))
Z_teste = rng.standard_normal((Ma, 10_000))
linhas = []
for nome, regra in [("regra do livro (h ≥ Ĉ)", True), ("regra corrigida (h > Ĉ e h > 0)", False)]:
    V0, pl, frac = hedge_americana_bsm(Z_trein, Z_teste, regra)
    linhas.append((nome, V0, pl.mean(), pl.std() / np.sqrt(len(pl)), pl.std(), np.percentile(pl,
↪   1), pl.min(), pl.max(), 100 * frac))
tab3 = pd.DataFrame(linhas, columns=["exercício", "preço LSM", "P&L médio", "erro-padrão", "dp do
↪   P&L",
                                     "percentil 1%", "mínimo", "máximo", "% 'exercícios' sem valor"])
tabela(tab3)

t3 = tab3.set_index("exercício")
ok(t3.loc["regra do livro (h ≥ Ĉ)", "% 'exercícios' sem valor"] > 5, "a regra do livro 'exerce'
↪   sem valor intrínseco em mais de 5% das trajetórias")
ok(t3.loc["regra corrigida (h > Ĉ e h > 0)", "% 'exercícios' sem valor"] == 0, "a regra corrigida
↪   nunca exerce fora do dinheiro")
ok(abs(t3.loc["regra corrigida (h > Ĉ e h > 0)", "P&L médio"]) < 3 * t3.loc["regra corrigida (h >
↪   Ĉ e h > 0)", "erro-padrão"] + 0.02,
   "com a regra corrigida e fora da amostra, o P&L médio é zero")
```

exercício	preço LSM	P&L médio	erro-padrão	dp do P&L	percentil 1%	mínimo	máximo	% 'exercícios' sem valor
regra do livro (h ≥ Ĉ)	4,470	0,014	0,004	0,371	-1,010	-2,066	2,109	22,100
regra corrigida (h > Ĉ e h > 0)	4,483	0,011	0,004	0,401	-1,134	-2,529	2,114	0,000

```
OK — a regra do livro 'exerce' sem valor intrínseco em mais de 5% das trajetórias
OK — a regra corrigida nunca exerce fora do dinheiro
OK — com a regra corrigida e fora da amostra, o P&L médio é zero
```

Com a regra corrigida, o hedge com deltas da regressão funciona bem no BSM: média zero e desvio-padrão comparável ao da put europeia do Laboratório 1 com 50 rebalanceamentos. Com a regra do script, cerca de um quinto das trajetórias "termina" quando a put ainda está viva, fora do dinheiro; o P&L dessas trajetórias é medido na hora errada. Neste exemplo o efeito nas estatísticas é pequeno, porque essas puts valem quase nada quando a regra dispara; mas a contagem de exercícios, e qualquer análise da fronteira de exercício feita com essa regra, sai errada.

---

4. Hedge da put americana no BCC97

Livro: §13.3 e §13.5.3 a §13.5.4

O livro repete o estudo no BCC97 calibrado (put no dinheiro, $T = 1$, 50 passos) com 11 funções de base em (S, v, r) e diferenças finitas de segunda ordem para o delta. Resultado: P&L médio de +4,34, mediana de 15 e mínimo de −1.014. O livro conclui que o lucro médio "justificaria" aplicar a estratégia a uma carteira grande, e que o risco de salto seria "diversificável".

Pelo Passo 3 da Seção 1, um P&L médio positivo numa simulação sob Q não é lucro: é inconsistência entre o preço e a simulação. E a simulação do livro carrega os vieses do Módulo 15 (`v[t]` e compensador ausente), que fazem o índice descontado **não** ser martingal. Uma posição vendida em delta (put → delta negativo) ganha quando o índice cai mais do que deveria.

Laboratório 4 — De onde vem o lucro médio

Mesmo procedimento do livro (LSM com 11 bases treinado em 50.000 trajetórias; delta pela derivada analítica da regressão em S), hedge **fora da amostra** em 20.000 trajetórias, para:

- parâmetros do livro com o simulador do livro;
- parâmetros do livro com o esquema exato do Módulo 15;
- o Bates sintético dos Módulos 13 a 15 com o esquema exato.

```python
S0 = 3225.93
CIR = dict(r0=0.02, kappa=0.5, theta=0.045, sigma=0.08)
BATES = np.array([3.0, 0.04, 0.5, -0.7, 0.03, 0.15, -0.15, 0.10])
LIVRO = np.array([22.212, 0.025, 0.952, -0.999, 0.036, 0.008, -0.501, 0.000])
LIVRO_CFG = dict(olhar_frente=True, compensador=False, aditivo=True, mm=True)


def simular(p, T, M, I, rng, cir=None, olhar_frente=False, compensador=True, aditivo=False,
↪  mm=False):
    """Trajetórias (S, v, r) do BCC97 e o número de saltos por trajetória (simulador do Módulo
    ↪  15)."""
```

```
    cir = CIR if cir is None else cir
    kappa, theta, sigma, rho, v0, lam, mu, delta = p
    dt = T / M
    Z = rng.standard_normal((4, M, I // 2)); Z = np.concatenate([Z, -Z], axis=2)
    if mm:
        Z = (Z - Z.mean(axis=(1, 2), keepdims=True)) / Z.std(axis=(1, 2), keepdims=True)
    N = rng.poisson(lam * dt, (M, I))
    rJ = lam * (np.exp(mu + 0.5 * delta**2) - 1)
    r = np.full(I, cir["r0"]); rh = r.copy(); v = np.full(I, v0); vh = v.copy(); S = np.full(I, S0)
    Sp, vp, rp = [S], [v], [r]
    for t in range(M):
        z1 = Z[1, t]; z2 = rho * z1 + np.sqrt(1 - rho**2) * Z[2, t]
        if cir["sigma"] > 0:
            r_novo, rh = fator_ft(rh, cir["kappa"], cir["theta"], cir["sigma"], dt, Z[0, t])
        else:
            r_novo = r
        v_novo, vh = fator_ft(vh, kappa, theta, sigma, dt, z2)
        vu = v_novo if olhar_frente else v
        choque = np.sqrt(vu * dt) * z1
        if mm:
            choque = choque - choque.mean()
        deriva = (0.5 * (r + r_novo) - (rJ if compensador else 0.0) - 0.5 * vu) * dt
        if aditivo:
            S = S * (np.exp(deriva + choque) + (np.exp(mu + delta * Z[3, t]) - 1) * N[t])
        else:
            S = S * np.exp(deriva + choque + N[t] * mu + np.sqrt(N[t]) * delta * Z[3, t])
        r, v = r_novo, v_novo
        Sp.append(S); vp.append(v); rp.append(r)
    return np.array(Sp), np.array(vp), np.array(rp), N.sum(axis=0)


def bases(S, v, r):
    """As 11 bases do livro (com S normalizado por S0) e suas derivadas em S."""
    s = S / S0
    z, um = np.zeros_like(s), np.ones_like(s)
    X = np.stack([um, r, v, s, r * r, v * v, s * s, v * r, s * r, s * v, s * v * r], axis=-1)
    dX = np.stack([z, z, z, um, z, z, 2 * s, z, r, v, v * r], axis=-1) / S0
    return X, dX


def treinar_lsm(p, K, T, M, I, rng, **kw):
    S, v, r, _ = simular(p, T, M, I, rng, **kw)
    dt = T / M
    h = np.maximum(K - S, 0)
    fd = np.exp(-(r[1:] + r[:-1]) / 2 * dt)
    V = h[-1].copy()
    coef = [None] * (M + 1)
    for t in range(M - 1, 0, -1):
        V = V * fd[t]
        X, _ = bases(S[t], v[t], r[t])
        coef[t] = np.linalg.lstsq(X, V, rcond=None)[0]
        ex = (h[t] > X @ coef[t]) & (h[t] > 0)
        V[ex] = h[t, ex]
    return (V * fd[0]).mean(), coef


def hedge_bcc(p, K, T, M, I, rng, V0, coef, **kw):
    S, v, r, Nsaltos = simular(p, T, M, I, rng, **kw)
    dt = T / M
    h = np.maximum(K - S, 0)
```

```
    _, dX = bases(S[0], v[0], r[0])
    d = np.clip(dX @ coef[1], -1, 0)
    B = V0 - d * S[0]
    vivo = np.ones(I, dtype=bool); pl = np.zeros(I); desc = np.ones(I)
    for t in range(1, M + 1):
        cresc = np.exp((r[t - 1] + r[t]) / 2 * dt)
        B = B * cresc; desc = desc / cresc
        port = d * S[t] + B
        if t == M:
            ex = vivo.copy()
        else:
            X, dX = bases(S[t], v[t], r[t])
            ex = vivo & (h[t] > X @ coef[t]) & (h[t] > 0)
        pl[ex] = (port[ex] - h[t, ex]) * desc[ex]
        vivo &= ~ex
        if t < M:
            d_novo = np.clip(dX @ coef[t], -1, 0)
            B = port - d_novo * S[t]
            d = d_novo
    return pl, Nsaltos


t0 = time.time()
linhas, pls = [], {}
for nome, p, kw in [("livro, simulador do livro", LIVRO, LIVRO_CFG),
                    ("livro, esquema exato", LIVRO, {}),
                    ("Bates, esquema exato", BATES, {})]:
    rng = np.random.default_rng(1)
    V0, coef = treinar_lsm(p, S0, 1.0, 50, 50_000, rng, **kw)
    pl, Ns = hedge_bcc(p, S0, 1.0, 50, 20_000, rng, V0, coef, **kw)
    pls[nome] = (pl, Ns)
    linhas.append((nome, V0, pl.mean(), pl.std() / np.sqrt(len(pl)), np.median(pl), pl.min(),
↪   pl.max(), pl.std()))
tab4 = pd.DataFrame(linhas, columns=["caso", "preço LSM", "P&L médio", "erro-padrão", "mediana",
↪   "mínimo", "máximo", "dp"])
tabela(tab4)
print(f"tempo: {time.time() - t0:.1f} s")
```

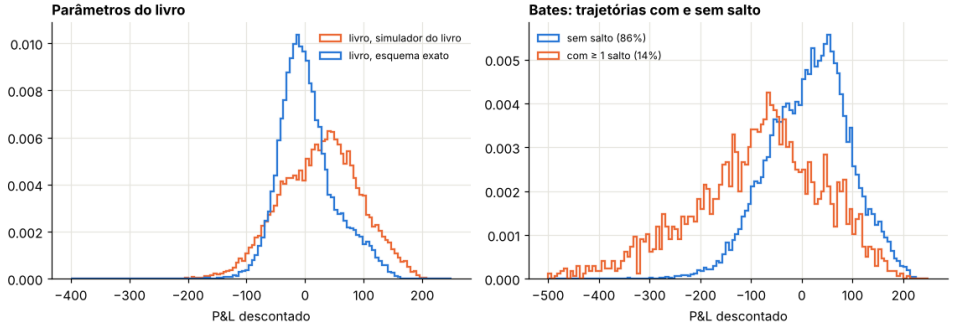
caso	preço LSM	P&L médio	erro-padrão	mediana	mínimo	máximo	dp
livro, simulador do livro	199,921	22,725	0,592	29,128	-917,299	230,498	83,662
livro, esquema exato	179,823	-0,186	0,489	-3,148	-913,265	171,527	69,093
Bates, esquema exato	218,830	0,427	0,673	11,248	-696,700	235,292	95,130

```
tempo: 3.5 s
```

```
fig, axs = plt.subplots(1, 2, figsize=(10.5, 3.8))
for nome, cor in [("livro, simulador do livro", LARANJA), ("livro, esquema exato", AZUL)]:
    axs[0].hist(pls[nome][0], bins=120, range=(-400, 250), histtype="step", lw=1.5, color=cor,
↪   label=nome, density=True)
axs[0].set_xlabel("P&L descontado"); axs[0].set_title("Parâmetros do livro");
↪   axs[0].legend(fontsize=8)
pl, Ns = pls["Bates, esquema exato"]
for n, cor in [(0, AZUL), (1, LARANJA)]:
    sel = Ns == 0 if n == 0 else Ns > 0
    axs[1].hist(pl[sel], bins=120, range=(-500, 250), histtype="step", lw=1.5, color=cor,
↪   density=True,
                label=("sem salto" if n == 0 else "com ≥ 1 salto") + f" ({100 * sel.mean():.0f}%)")
```

```
axs[1].set_xlabel("P&L descontado"); axs[1].set_title("Bates: trajetórias com e sem salto");
↪  axs[1].legend(fontsize=8)
plt.tight_layout(); plt.show()

t4 = tab4.set_index("caso")
ok(t4.loc["livro, simulador do livro", "P&L médio"] > 5 * t4.loc["livro, simulador do livro",
↪  "erro-padrão"],
   "com o simulador do livro, o P&L médio é positivo e significativo, como no livro")
for c in ["livro, esquema exato", "Bates, esquema exato"]:
    ok(abs(t4.loc[c, "P&L médio"]) < 3 * t4.loc[c, "erro-padrão"] + 0.5, f"{c}: o P&L médio é
↪  zero, como manda a teoria")
ok(t4.loc["livro, simulador do livro", "preço LSM"] > t4.loc["livro, esquema exato", "preço LSM"]
↪  + 5,
   "o simulador do livro também infla o preço da put")
dp_salto = pl[Ns > 0].std(); dp_sem = pl[Ns == 0].std()
print("Bates — dp do P&L com salto:", f3(dp_salto), "| sem salto:", f3(dp_sem))
ok(dp_salto > 1.5 * dp_sem, "no Bates, o P&L das trajetórias com salto tem dispersão pelo menos
↪  50% maior")
```




```
OK — com o simulador do livro, o P&L médio é positivo e significativo, como no livro
OK — livro, esquema exato: o P&L médio é zero, como manda a teoria
OK — Bates, esquema exato: o P&L médio é zero, como manda a teoria
OK — o simulador do livro também infla o preço da put
Bates — dp do P&L com salto: 129,772 | sem salto: 79,194
OK — no Bates, o P&L das trajetórias com salto tem dispersão pelo menos 50% maior
```

O que aprendemos.

- Com a simulação certa, o P&L médio é zero: não há lucro sistemático a colher. O "lucro" do livro mede o quanto o índice simulado deixou de ser martingal.
- A assimetria (mediana positiva, cauda esquerda longa) é real e é, em boa parte, o retrato do risco de salto (o restante da dispersão, também nas trajetórias sem salto, é risco de volatilidade, que a Seção 5 separa): na maioria das trajetórias o hedge ganha um pouco; nas que têm salto, perde muito. Merton (1976) supõe que o risco de salto é **diversificável entre ativos** e, por isso, não precificado; isso não quer dizer que um livro de opções vendidas o elimine. Somar muitas cópias da mesma put sobre o mesmo índice não diversifica nada: os saltos são os mesmos.

5. Que risco o delta deixa: vol, saltos e o hedge com opções

Livro: §13.3 (parte final) e §13.4

O livro conclui que, para hedgear de verdade, é preciso acrescentar outras opções à carteira. Medimos isso para uma put **europeia** no dinheiro de 1 ano (para separar o efeito do hedge do efeito do exercício), com juros constantes de 3%, 50 rebalanceamentos e 10.000 trajetórias, em três mundos:

- **BSM** com $\sigma = \sqrt{\theta_v} = 20\%$;
- **Heston**: o Bates sem saltos;
- **Bates**: o modelo completo dos Módulos 13 a 15.

E duas estratégias:

- **delta**: índice + título;
- **delta-vega**: índice + título + uma put de 18 meses no dinheiro, na quantidade que anula a sensibilidade à variância v (a "vega" do modelo), e índice para anular o delta da carteira. A quantidade da put de hedge é limitada a 3 por put vendida, para que trajetórias extremas (variância perto de zero, em que as duas sensibilidades somem) não gerem posições absurdas.

Preços e gregas no Heston e no Bates vêm da fórmula de Lewis. Para não recalcular a integral para cada trajetória e cada data, montamos em cada data uma **tabela** de preços numa grade de $(v, \ln S/K)$, usando que a função característica não depende de S, e interpolamos por splines cúbicos. A tabela é conferida contra a fórmula direta.

Laboratório 5 — Decompor o erro de hedge

```
R5, T5, T_H = 0.03, 1.0, 1.5
U5 = np.linspace(0, 400, 4001)
W5 = np.full(len(U5), U5[1] - U5[0]); W5[[0, -1]] /= 2
XG = np.linspace(-1.5, 1.0, 201)
VG = np.linspace(0.0, 0.5, 101)


def tabela_put(tau, p):
    """Put/K na grade (v, x = ln S/K) pela fórmula de Lewis; a função característica é calculada
    ↪ uma vez por v."""
    kappa, theta, sigma, rho, v0, lam, mu, delta = p
    phi = cf_heston(U5[None, :] - 0.5j, tau, VG[:, None], kappa, theta, sigma, rho)
    if lam > 0:
        phi = phi * cf_saltos(U5[None, :] - 0.5j, tau, lam, mu, delta)
    E = np.exp(1j * U5[None, :] * (XG + R5 * tau)[:, None])
    integral = ((phi * W5 / (U5**2 + 0.25)) @ E.T).real
    s = np.exp(XG)[None, :]
    C = s - np.sqrt(s) * np.exp(-0.5 * R5 * tau) / np.pi * integral
    return RegularGridInterpolator((VG, XG), C - s + np.exp(-R5 * tau), method="cubic",
    ↪ bounds_error=False, fill_value=None)


def gregas(f, S, v, K):
    """Preço, delta e sensibilidade a v pela tabela interpolada."""
    x = np.clip(np.log(S / K), XG[0] + 0.01, XG[-1] - 0.01); hx, hv = 1e-3, 1e-4   # nunca
↪ extrapola a grade
```

```
    v = np.clip(v, 0.0, VG[-1] - hv)
    vv = np.clip(v, hv, VG[-1] - hv)
    P = f(np.column_stack([v, x]))
    dx = (f(np.column_stack([v, x + hx])) - f(np.column_stack([v, x - hx]))) / (2 * hx)
    dv = (f(np.column_stack([vv + hv, x])) - f(np.column_stack([vv - hv, x]))) / (2 * hv)
    return K * P, K * dx / S, K * dv


# conferência da tabela contra a fórmula direta
p_t = BATES
f_t = tabela_put(0.5, p_t)
S_t = np.array([2900.0, S0, 3500.0]); v_t = np.array([0.02, 0.03, 0.05])
tab_vals = gregas(f_t, S_t, v_t, S0)[0]
dir_vals = np.array([calls(Si, [S0], [0.5], [R5], (vi, *p_t[:4]), tuple(p_t[5:]), U_FINO)[0] - Si
↪  + S0 * np.exp(-R5 * 0.5)
                     for Si, vi in zip(S_t, v_t)])
print("tabela:", [f3(x) for x in tab_vals], "| fórmula:", [f3(x) for x in dir_vals])
ok(np.abs(tab_vals - dir_vals).max() < 0.05, "a tabela interpolada reproduz a fórmula de Lewis a
↪  menos de 0,05 ponto")
```

```
tabela: ['309,383', '148,593', '96,785'] | fórmula: ['309,363', '148,590', '96,783']
OK — a tabela interpolada reproduz a fórmula de Lewis a menos de 0,05 ponto
```

```
CIR_CONST = dict(r0=R5, kappa=0.0, theta=R5, sigma=0.0)
M5, I5 = 50, 10_000
dt5 = T5 / M5


def hedge_europeia(p, estrategia, rng):
    S, v, _, Ns = simular(p, T5, M5, I5, rng, cir=CIR_CONST)
    alvo = [tabela_put(T5 - j * dt5, p) for j in range(M5)]
    inst = [tabela_put(T_H - j * dt5, p) for j in range(M5 + 1)]

    def posicao(j):
        P, dP, vP = gregas(alvo[j], S[j], v[j], S0)
        H, dH, vH = gregas(inst[j], S[j], v[j], S0)
        with np.errstate(divide="ignore", invalid="ignore"):
            n = np.zeros(I5) if estrategia == "delta" else np.nan_to_num(np.clip(vP / vH, 0.0,
↪  3.0))   # posição limitada
        return P, H, dP - n * dH, n

    P, H, d, n = posicao(0)
    B = P - d * S[0] - n * H
    for j in range(1, M5 + 1):
        B = B * np.exp(R5 * dt5)
        H = gregas(inst[j], S[j], v[j], S0)[0]
        port = d * S[j] + n * H + B
        if j < M5:
            _, _, d_novo, n_novo = posicao(j)
            B = port - d_novo * S[j] - n_novo * H
            d, n = d_novo, n_novo
    return (port - np.maximum(S0 - S[-1], 0)) * np.exp(-R5 * T5), Ns


def hedge_bsm_mundo(rng):
    sig = np.sqrt(BATES[1])
    S = np.full(I5, S0)
    d = delta_put_bsm(S, S0, T5, R5, sig)
```

```
    B = put_bsm(S0, S0, T5, R5, sig) - d * S
    for j in range(1, M5 + 1):
        S = S * np.exp((R5 - 0.5 * sig**2) * dt5 + sig * np.sqrt(dt5) * rng.standard_normal(I5))
        B = B * np.exp(R5 * dt5)
        port = d * S + B
        if j < M5:
            d_novo = delta_put_bsm(S, S0, T5 - j * dt5, R5, sig)
            B = port - d_novo * S
            d = d_novo
    return (port - np.maximum(S0 - S, 0)) * np.exp(-R5 * T5)

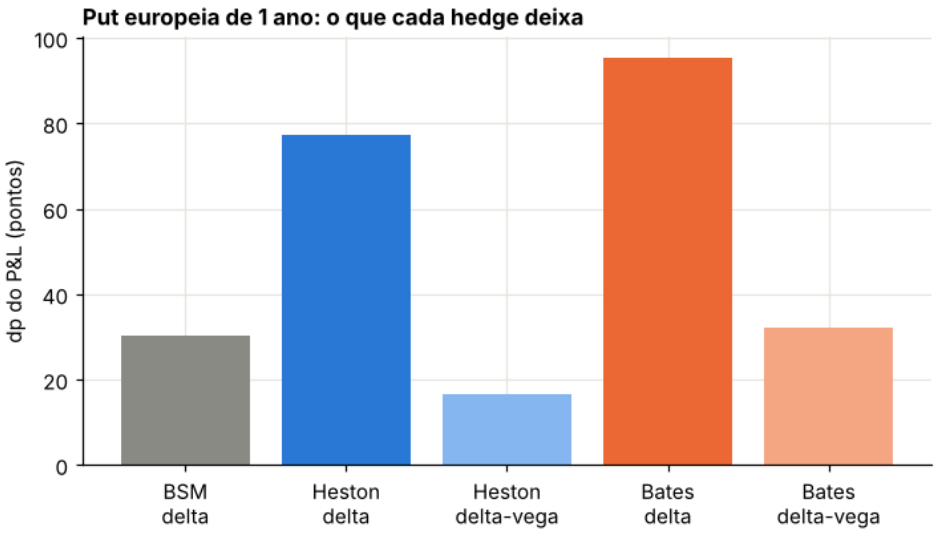

HESTON_SO = BATES.copy(); HESTON_SO[5] = 0.0
t0 = time.time()
resultados5 = {("BSM", "delta"): (hedge_bsm_mundo(np.random.default_rng(5)), None)}
for mundo, p in [("Heston", HESTON_SO), ("Bates", BATES)]:
    for estr in ("delta", "delta-vega"):
        resultados5[(mundo, estr)] = hedge_europeia(p, estr, np.random.default_rng(5))
print(f"tempo: {time.time() - t0:.1f} s")
linhas = []
for (mundo, estr), (pl, Ns) in resultados5.items():
    com = pl[Ns > 0].std() if Ns is not None and (Ns > 0).sum() > 1 else np.nan
    sem = pl[Ns == 0].std() if Ns is not None else np.nan
    linhas.append((mundo, estr, pl.mean(), pl.std(), np.percentile(pl, 1), pl.min(), com, sem))
tab5 = pd.DataFrame(linhas, columns=["mundo", "estratégia", "P&L médio", "dp", "percentil 1%",
↪  "mínimo",
                                     "dp com salto", "dp sem salto"])
tabela(tab5)
```

```
tempo: 31.1 s
```

mundo	estratégia	P&L médio	dp	percentil 1%	mínimo	dp com salto	dp sem salto
BSM	delta	0,094	30,408	-83,352	-217,601	n/d	n/d
Heston	delta	0,984	77,303	-214,426	-367,200	n/d	77,303
Heston	delta-vega	0,195	16,476	-51,655	-183,150	n/d	16,476
Bates	delta	1,194	95,423	-320,003	-724,297	136,450	77,108
Bates	delta-vega	0,251	32,221	-103,448	-597,051	70,676	16,804

```
fig, ax = plt.subplots(figsize=(7.5, 3.8))
rot = [f"{m}\n{e}" for m, e in zip(tab5["mundo"], tab5["estratégia"])]
ax.bar(rot, tab5["dp"], color=[CINZA, AZUL, RAMPA[0], LARANJA, "#f4a582"])
ax.set_ylabel("dp do P&L (pontos)")
ax.set_title("Put europeia de 1 ano: o que cada hedge deixa")
plt.show()

t5 = tab5.set_index(["mundo", "estratégia"])
ok(t5.loc[("Heston", "delta"), "dp"] > 2 * t5.loc[("BSM", "delta"), "dp"], "no Heston, o delta
↪  sozinho deixa mais que o dobro do erro do BSM: risco de volatilidade")
ok(t5.loc[("Heston", "delta-vega"), "dp"] < 0.5 * t5.loc[("Heston", "delta"), "dp"], "a opção de
↪  18 meses corta o erro do Heston pela metade ou mais")
ok(t5.loc[("Bates", "delta-vega"), "dp com salto"] > 2 * t5.loc[("Bates", "delta-vega"), "dp sem
↪  salto"],
   "no Bates, o que sobra depois do delta-vega está concentrado nas trajetórias com salto")
ok(t5.loc[("Bates", "delta-vega"), "dp"] < t5.loc[("Bates", "delta"), "dp"], "mesmo assim, a
↪  opção também reduz o erro total no Bates")
```




```
OK — no Heston, o delta sozinho deixa mais que o dobro do erro do BSM: risco de volatilidade
OK — a opção de 18 meses corta o erro do Heston pela metade ou mais
OK — no Bates, o que sobra depois do delta-vega está concentrado nas trajetórias com salto
OK — mesmo assim, a opção também reduz o erro total no Bates
```

O que aprendemos.

- No BSM, o delta resolve. No Heston, o delta deixa o **risco de volatilidade**: a variância muda, e o índice sozinho não protege contra isso. Uma segunda opção, dimensionada para anular a sensibilidade a v, leva o erro para **abaixo** até do BSM só com delta (Módulo 11: o Heston fica completo com uma opção a mais). A opção de hedge também reduz o gama da carteira, e com ele o erro de rebalanceamento discreto.
- No Bates sobra o **risco de salto**. A opção de prazo maior ajuda, porque é convexa e ganha quando o índice salta, mas não cobre o salto: o tamanho é aleatório, e nenhuma quantidade finita de instrumentos replica todos os tamanhos possíveis (§11.2.1). É a incompletude "no sentido amplo" do livro, agora medida.

---

6. Síntese do curso

6.1 O que os 16 módulos construíram

Módulo	Ferramenta	Pergunta que responde
1	precificação calibrada ao mercado	por que preços de opções e não modelos?
2	stylized facts, vol. implícita	o que um modelo precisa reproduzir?

Módulo	Ferramenta	Pergunta que responde
3-4	neutralidade ao risco, Itô, Girsanov	por que descontar a esperança sob Q?
5-6	BSM, árvore CRR	o caso de referência e sua aproximação discreta
7-8	Lewis, Carr-Madan, FFT	como precificar com a função característica
9	LSM e dual	como tratar o exercício antecipado
10	Merton	saltos e o primeiro ajuste a mercado
11	BCC97	o modelo geral e a estrutura afim
12	Monte Carlo	discretização, viés e ruído
13-14	calibração	escolher Q com os preços observados, e o que fica indeterminado
15	simulação no BCC97	juntar tudo e conferir pela martingal
16	hedge dinâmico	o que o modelo protege e o que não protege

6.2 Os testes que se mostraram mais úteis

Ao longo do curso, as mesmas poucas conferências pegaram quase todos os problemas do livro:

1. **Martingal**: $\mathbb{E}^Q[e^{-\int r} S_T] = S_0$. Pegou o `v[t]` (Módulos 12 e 15), o compensador ausente (Módulo 15) e o "lucro" do hedge (Módulo 16).
2. **Duas formas de calcular a mesma coisa**: Fourier contra Monte Carlo, árvore contra LSM, derivada numérica contra fórmula. Pegou a taxa forward do CIR (Módulo 13), a função característica impressa (Módulo 13) e as referências bermudanas (Módulo 9).
3. **Dados sintéticos com verdade conhecida**: calibrar onde se sabe a resposta. Mostrou a armadilha de Feller, a calibração sequencial que perde os saltos e a degenerescência (Módulos 13 e 14).
4. **Erro-padrão em todo número de Monte Carlo**: separa viés de ruído e evita conclusões sobre o "número de passos" (Módulo 15).

6.3 Para continuar

- **Esquemas melhores** para a variância: o QE de Andersen (Módulo 12, exercício 5) e a simulação exata de Broadie e Kaya.
- **Calibração global**: multi-start, `differential_evolution`, e regularização com w escolhido por validação (Módulo 14).
- **Hedge com custos de transação** e frequência ótima de rebalanceamento (Whalley e Wilmott).
- **Volatilidade local e estocástica-local**, que reproduzem a superfície inteira por construção.

7. Erratas e notas sobre os capítulos 13 e 14

Onde	O quê
§13.5.1 e §13.5.3, delta inicial	os dois preços LSM da diferença finita usam números aleatórios diferentes: o delta inicial é dominado pelo ruído (desvio-padrão acima de 0,5)
§13.5.1, regra de exercício	`h >= C` com `C` truncado em zero marca como exercidas puts fora do dinheiro, encerrando o hedge antes da hora
§13.5.2	P&L calculado nas mesmas trajetórias da regressão (dentro da amostra), apesar de a nota 3 defender a avaliação fora da amostra
§13.3, P&L médio de +4,34	numa simulação sob Q, o P&L médio de um hedge autofinanciável iniciado no preço justo é zero; o valor positivo vem do simulador (Módulo 15)
§13.3, "risco de salto diversificável"	a hipótese de Merton é de diversificação entre ativos; não vale para muitas opções sobre o mesmo índice, cujos saltos são comuns
§13.5.3	herda o simulador do cap. 12 (`v[t]`, compensador ausente, saltos aditivos, moment matching)
Cap. 14	o resumo é fiel ao livro; os módulos 12 a 16 mostram que as etapas de simulação, calibração e hedge precisam das conferências da Seção 6.2

8. Exercícios

Exercício 1. Prove o Passo 3 da Seção 1 num modelo de um período: com $\Pi_1 = \Delta S_1 + (\Pi_0 - \Delta S_0)(1+r)$, mostre que $\mathbb{E}^Q[\Pi_1/(1+r)] = \Pi_0$ para qualquer Δ.

Solução

$\mathbb{E}^Q[\Pi_1]/(1+r) = \Delta\,\mathbb{E}^Q[S_1]/(1+r) + \Pi_0 - \Delta S_0 = \Delta S_0 + \Pi_0 - \Delta S_0 = \Pi_0$, porque $\mathbb{E}^Q[S_1] = S_0(1+r)$. O delta não entra na média; só na variância.

Exercício 2. No Laboratório 1, troque o delta exato por um delta calculado com a volatilidade errada (25% em vez de 20%). O que acontece com a média e com o desvio-padrão do P&L?

Solução

Se o preço inicial também for o de 25%, a média do P&L fica positiva: vendeu-se a put cara num mundo de 20%. O ganho esperado é a diferença de preço, $P(25\%) - P(20\%)$, acumulada ao longo do caminho em proporção ao gama. O desvio-padrão aumenta pouco: o delta é pouco sensível à volatilidade perto do dinheiro.

Exercício 3. No Laboratório 5, troque a estratégia delta-vega por delta-gama (mesma put de 18 meses, quantidade que anula o gama). Por que o resultado pode ficar **pior** perto do vencimento?

Solução

O gama da put de 1 ano no dinheiro explode perto do vencimento, enquanto o da put de 18 meses continua moderado. A razão de gamas, e portanto a posição na opção de hedge, cresce sem limite. Pequenos erros de modelo viram perdas grandes. Na prática, limita-se a posição ou troca-se o instrumento por um de vencimento mais próximo.

Exercício 4. Repita o Laboratório 4 com o hedge **dentro da amostra** (as mesmas trajetórias da regressão). O desvio-padrão do P&L muda?

Solução

Dentro da amostra, a regra de exercício usa informação das próprias trajetórias, e a regressão foi ajustada a elas: o P&L sai um pouco mais favorável e menos disperso. A diferença é pequena com 50.000 trajetórias, mas cresce com menos trajetórias ou mais funções de base.

```
print(f"Módulo 16 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 16 concluído: 20 verificações aprovadas.
```

Parte IV

Complemento — modelos de juros

Capítulo 17

Hull-White I: medidas, o modelo e as fórmulas fechadas

Introdução

Os capítulos anteriores tratavam os juros como coadjuvantes. Nos derivativos de renda fixa, eles são o próprio ativo subjacente: o pagamento de um cap, de um título com opção de resgate ou de uma swaption depende da evolução de toda a curva de juros.

O modelo de **Hull-White** descreve a curva a partir de uma única variável, a **short rate** (a taxa de juros instantânea), que oscila em torno de uma média que depende do tempo e é atraída de volta para ela. Essa dependência temporal permite que o modelo reproduza exatamente a curva de juros observada hoje, condição mínima para precificar derivativos de juros de forma consistente com o mercado.

Para tirar proveito do modelo, o capítulo introduz uma ferramenta nova: a **mudança de numerário**. Medindo preços em unidades de um título em vez de unidades da conta remunerada, o desconto sai de dentro da esperança, e opções sobre títulos e caplets ganham fórmulas fechadas parecidas com a de Black. O capítulo deduz essas fórmulas, calibra o modelo a preços de caplets e estima parâmetros com dados de T-bills.

Fundamentos teóricos

Numerários e medidas

Teorema 17.1 (Mudança de numerário). *Sejam N e M dois numerários (processos de preço positivos) e $\mathbb{Q}^N$ uma medida sob a qual todo preço dividido por N é martingal. A medida definida por*

$$\left.\frac{d\mathbb{Q}^M}{d\mathbb{Q}^N}\right|_{\mathcal{F}_T} = \frac{M_T/M_0}{N_T/N_0} \tag{17.1}$$

torna martingal todo preço dividido por M. Em particular, para qualquer payoff H em T,

$$V_0 = N_0\,\mathbb{E}^{\mathbb{Q}^N}\left[\frac{H}{N_T}\right] = M_0\,\mathbb{E}^{\mathbb{Q}^M}\left[\frac{H}{M_T}\right] \tag{17.2}$$

(Geman et al., 1995).

Definição 17.2 (Medida forward). Tomando como numerário o título de cupom zero $P(t,T)$, obtém-se a *medida T-forward* $\mathbb{Q}^T$. Como $P(T,T)=1$,

$$V_0 = P(0,T)\,\mathbb{E}^{\mathbb{Q}^T}[H], \tag{17.3}$$

e o desconto sai da esperança mesmo com juros estocásticos. Sob $\mathbb{Q}^T$, a taxa forward $f(t,T)$ e o preço forward de qualquer ativo para entrega em T são martingais.

O modelo de Hull-White

Definição 17.3 (Hull-White de um fator). Sob a medida neutra ao risco,

$$dr_t = (\vartheta(t) - a r_t)\,dt + \sigma\,dW_t, \tag{17.4}$$

com $a,\sigma>0$ constantes e $\vartheta(t)$ determinística (Hull and White, 1990). Com ϑ constante, é o modelo de Vasicek; com $a=0$, o de Ho-Lee (Vasicek, 1977; Ho and Lee, 1986).

Proposição 17.4 (Ajuste à curva). *O modelo reproduz a curva inicial $P^{\text{mkt}}(0,T)$ para todo T se, e somente se,*

$$\vartheta(t) = \partial_t f^{\text{mkt}}(0,t) + a f^{\text{mkt}}(0,t) + \frac{\sigma^2}{2a}(1-e^{-2at}). \tag{17.5}$$

Equivalentemente, $r_t = x_t + \alpha(t)$, com $dx_t = -a x_t\,dt + \sigma\,dW_t$, $x_0=0$, e $\alpha(t) = f^{\text{mkt}}(0,t) + \frac{\sigma^2}{2a^2}(1-e^{-at})^2$.

Proposição 17.5 (Preço do título). *Em Hull-White, $P(t,T) = A(t,T)\,e^{-B(t,T)r_t}$, com*

$$B(t,T) = \frac{1-e^{-a(T-t)}}{a}, \qquad \ln A(t,T) = \ln\frac{P^{\text{mkt}}(0,T)}{P^{\text{mkt}}(0,t)} + B(t,T) f^{\text{mkt}}(0,t) - \frac{\sigma^2}{4a}(1-e^{-2at})B(t,T)^2. \tag{17.6}$$

Esboço. $\int_t^T r_s\,ds$ é gaussiana condicionalmente a $\mathcal{F}_t$, pois x é um processo de Ornstein-Uhlenbeck. Logo $P(t,T) = \exp(-m + \tfrac12 s^2)$, com m e s^2 a média e a variância condicionais da integral; ambas são afins em r_t, e o coeficiente de r_t é $B(t,T)$. A parte determinística é fixada pela condição de reproduzir $P^{\text{mkt}}(0,\cdot)$. ∎

Opções sobre títulos e caps

Teorema 17.6 (Opção sobre título de cupom zero). *Uma call europeia de vencimento T e strike X sobre o título que vence em $S>T$ vale*

$$\text{ZBC} = P(0,S)\,N(h) - X\,P(0,T)\,N(h-\sigma_P), \tag{17.7}$$

$$\sigma_P = \sigma\sqrt{\frac{1-e^{-2aT}}{2a}}\,B(T,S), \qquad h = \frac{1}{\sigma_P}\ln\frac{P(0,S)}{X\,P(0,T)} + \frac{\sigma_P}{2}. \tag{17.8}$$

Demonstração. Sob $\mathbb{Q}^T$, $P(T,S) = P(t,S)/P(t,T)$ avaliado em T é o preço forward, um martingal lognormal com variância σ_P^2. A fórmula é a de Black para esse preço forward (Black, 1976; Brigo and Mercurio, 2006). ∎

Proposição 17.7 (Caplet como put sobre título). *Um caplet de taxa simples $L(T,S)$ com strike K, período $\delta = S-T$ e nominal 1, pago em S, vale o mesmo que $(1+K\delta)$ puts europeias de vencimento T e strike $1/(1+K\delta)$ sobre o título $P(\cdot,S)$.*

Demonstração. O payoff $\delta\,(L-K)^+$ em S vale em T $P(T,S)\,\delta(L-K)^+$. Com $1+\delta L = 1/P(T,S)$, isso é $(1-(1+K\delta)P(T,S))^+ = (1+K\delta)(\frac{1}{1+K\delta} - P(T,S))^+$. ■

Teorema 17.8 (Jamshidian). *Num modelo de um fator em que $P(T,S_i)$ é decrescente em r_T, uma opção sobre um título com cupons c_i é uma carteira de opções sobre títulos de cupom zero, com strikes $X_i = P(T,S_i)|_{r_T=r^*}$, onde r^* resolve $\sum_i c_i P(T,S_i)|_{r^*} = X$ (Jamshidian, 1989).*

Roteiro do capítulo

Curso: Derivativos com Python. Módulo complementar ao livro de Hilpisch, construído a partir do material de renda fixa (Wilmott, *Paul Wilmott Introduces Quantitative Finance*, caps. sobre taxas; Clewlow e Strickland, *Implementing Derivatives Models*, caps. 7 a 9; Hull e White, 1990, 1993, 1994).

O que você vai conseguir fazer ao final

1. Trocar de numerário (conta remunerada, título zero-cupom) e saber qual esperança calcula qual preço.
2. Deduzir a solução do modelo de Hull-White e a fórmula afim do título zero-cupom.
3. Escolher $\theta(t)$ para que o modelo reproduza exatamente a curva de juros de hoje.
4. Simular o modelo **sem erro de discretização**, inclusive a integral da short rate.
5. Deduzir e aplicar a fórmula fechada da opção sobre título (Jamshidian) e precificar caps e floors.
6. Calibrar a e σ a volatilidades de caps, e estimar σ a partir de séries históricas sem cair em armadilhas de anualização.

Onde estamos

O livro de Hilpisch trata os juros como coadjuvantes: o CIR entra no BCC97 só para descontar (Módulos 11 e 13). Este módulo e o próximo colocam a taxa de juros no centro. O Hull-White é o modelo de short rate mais usado em mesas e em validação de modelos: é gaussiano, tem fórmulas fechadas, ajusta a curva de hoje por construção e tem uma árvore trinomial eficiente, que é o tema do Módulo 18.

Notação aqui (Hull)	Wilmott	Clewlow-Strickland e planilha
$dr = (\theta(t) - a\,r)\,dt + \sigma\,dW$	$dr = (\eta(t) - \gamma r)\,dt + c\,dX$	$dr = (\theta(t) - \alpha r)\,dt + \sigma\,dz$
a: velocidade de reversão	γ	α (célula “alpha”)
σ: volatilidade normal da taxa	c	σ (célula “vol”)

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import brentq, least_squares
from scipy.stats import norm
from IPython.display import display

RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

def carregar_treasury():
    """Taxas diárias dos T-bills (U.S. Treasury, domínio público), as mesmas da planilha
    ↪ HW_arvore_final."""
    local = Path("dados/treasury_bills_2023_2024.csv")
    fonte = local if local.exists() else
↪ "https://raw.githubusercontent.com/walterCNeto/XRiscs/main/dados/treasury_bills_2023_2024.csv"
    return pd.read_csv(fonte, parse_dates=["data"])

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. A curva de juros em uma página

Material: Aula 1 (Fixed Income Products and Analysis)

Três objetos descrevem a mesma informação:

- o preço do título zero-cupom $P(0,T)$, que paga 1 em T;
- a taxa (yield) contínua $y(T) = -\ln P(0,T)/T$;
- a taxa forward instantânea $f(0,T) = -\partial_T \ln P(0,T)$, que satisfaz $P(0,T) = e^{-\int_0^T f(0,u)\,du}$ e $f = y + T\,y'$ (Módulo 13).

Para os laboratórios, a curva de mercado sintética é uma **Nelson-Siegel** para a taxa forward:

$$f(0,t) = \beta_0 + \beta_1 e^{-t/\tau} + \beta_2 \frac{t}{\tau} e^{-t/\tau}. \tag{17.9}$$

A integral e a derivada são fechadas, o que permite conferir cada fórmula do Hull-White sem erro de interpolação. Escolhemos uma curva positivamente inclinada com uma corcova: $\beta_0 = 4{,}5\%$, $\beta_1 = -1\%$, $\beta_2 = 2\%$, $\tau = 1{,}5$.

Laboratório 1 — A curva sintética

```
NS = dict(b0=0.045, b1=-0.010, b2=0.020, tau=1.5)


def f_merc(t, b0=NS["b0"], b1=NS["b1"], b2=NS["b2"], tau=NS["tau"]):
    e = np.exp(-np.asarray(t, float) / tau)
```

```
    return b0 + b1 * e + b2 * (np.asarray(t, float) / tau) * e

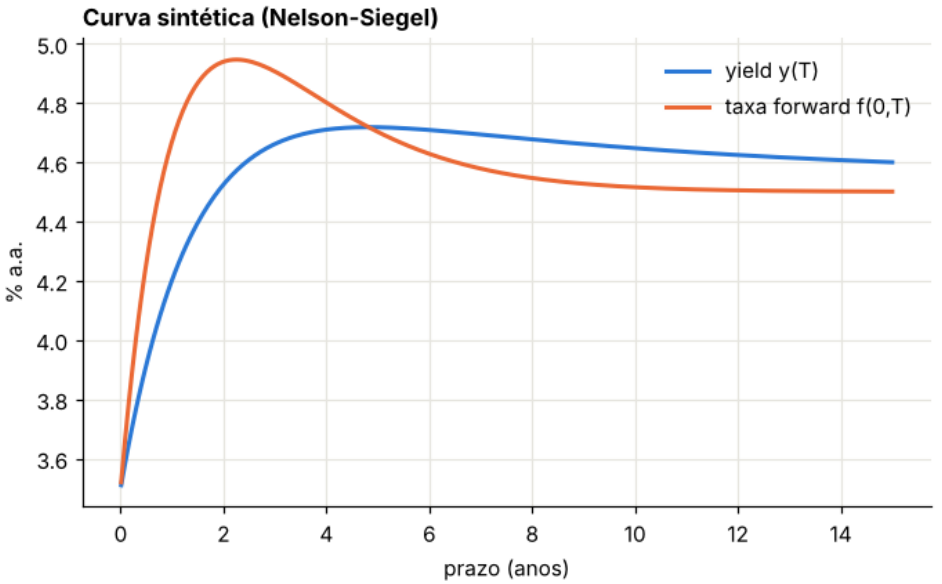

def df_merc(t, b0=NS["b0"], b1=NS["b1"], b2=NS["b2"], tau=NS["tau"]):
    """Derivada ∂f(0,t)/∂t."""
    t = np.asarray(t, float); e = np.exp(-t / tau)
    return -b1 / tau * e + b2 / tau * e * (1 - t / tau)


def P_merc(T, b0=NS["b0"], b1=NS["b1"], b2=NS["b2"], tau=NS["tau"]):
    T = np.asarray(T, float); e = np.exp(-T / tau)
    integral = b0 * T + b1 * tau * (1 - e) + b2 * tau * (1 - (1 + T / tau) * e)
    return np.exp(-integral)


Tg = np.linspace(0.01, 15, 400)
y = -np.log(P_merc(Tg)) / Tg
h = 1e-5
y_der = (-np.log(P_merc(Tg + h)) / (Tg + h) + np.log(P_merc(Tg - h)) / (Tg - h)) / (2 * h)

fig, ax = plt.subplots()
ax.plot(Tg, 100 * y, color=AZUL, label="yield y(T)")
ax.plot(Tg, 100 * f_merc(Tg), color=LARANJA, label="taxa forward f(0,T)")
ax.set_xlabel("prazo (anos)"); ax.set_ylabel("% a.a."); ax.set_title("Curva sintética
↪  (Nelson-Siegel)")
ax.legend(); plt.show()

ok(np.allclose(f_merc(Tg), y + Tg * y_der, atol=1e-8), "f = y + T·y' (a relação da Seção 1)")
num = (np.log(P_merc(Tg - h)) - np.log(P_merc(Tg + h))) / (2 * h)
ok(np.allclose(num, f_merc(Tg), atol=1e-8), "a integral fechada é coerente com a taxa forward")
```




```
OK — f = y + T·y' (a relação da Seção 1)
OK — a integral fechada é coerente com a taxa forward
```

2. Numerários e medidas

Material: Aula 2 (Martingales & Measures) e Aula 3 (Standard Model)

2.1 O preço de mercado do risco

Seja θ uma variável de estado com $d\theta/\theta = m\,dt + s\,dZ$, e f_1, f_2 dois derivativos que dependem só de θ: $df_i/f_i = \mu_i\,dt + \sigma_i\,dZ$. A carteira $\Pi = (\sigma_2 f_2)f_1 - (\sigma_1 f_1)f_2$ elimina dZ, logo rende r:

$$(\sigma_2\mu_1 - \sigma_1\mu_2)\,\Delta t = r(\sigma_2 - \sigma_1)\,\Delta t \quad\Longrightarrow\quad \frac{\mu_1 - r}{\sigma_1} = \frac{\mu_2 - r}{\sigma_2} \equiv \lambda. \tag{17.10}$$

λ é o **preço de mercado do risco** de θ: o mesmo para todos os derivativos dessa fonte de incerteza. Escolher um valor de λ é escolher um "mundo": com $\lambda = 0$, todos rendem r (o mundo neutro ao risco tradicional); com $\lambda = \sigma_g$, a volatilidade de um ativo g, todos os preços **relativos** f/g viram martingais.

2.2 O resultado da medida martingal equivalente

Com $\lambda = \sigma_g$, Itô em $\ln(f/g)$ dá $d(f/g) = (\sigma_f - \sigma_g)(f/g)\,dZ$: sem drift. Logo

$$\frac{f_0}{g_0} = \mathbb{E}^g\Big[\frac{f_T}{g_T}\Big] \quad\Longrightarrow\quad f_0 = g_0\,\mathbb{E}^g\Big[\frac{f_T}{g_T}\Big]. \tag{17.11}$$

Duas escolhas de numerário g dominam a renda fixa:

Numerário g	g_0, g_T	Fórmula de preço	Medida
conta remunerada	1, $e^{\int_0^T r\,du}$	$f_0 = \mathbb{E}^Q\big[e^{-\int_0^T r\,du} f_T\big]$	neutra ao risco Q
zero-cupom $P(t,T)$	$P(0,T)$, 1	$f_0 = P(0,T)\,\mathbb{E}^T[f_T]$	a termo em T

A segunda é a base do modelo de Black (Aula 3): sob a medida forward, o **preço forward** de qualquer ativo é martingal, e o desconto sai da esperança, mesmo com juros estocásticos.

2.3 Como fica o Hull-White sob cada medida

Sob Q: $dr = (\theta(t) - a\,r)\,dt + \sigma\,dW^Q$. Pelo teorema de Girsanov (Módulo 4), passar à medida forward em T acrescenta ao drift o produto da volatilidade de r pela volatilidade do numerário $P(t,T)$, que no Hull-White é $-\sigma B(t,T)$ (Seção 3):

$$dr = (\theta(t) - a\,r - \sigma^2 B(t,T))\,dt + \sigma\,dW^T, \qquad B(t,T) = \frac{1 - e^{-a(T-t)}}{a}. \tag{17.12}$$

Sob a medida forward, a taxa tem drift **menor**: os estados de juros altos, em que o numerário $P(t,T)$ vale pouco, ganham menos peso. O Laboratório 3 confere que as duas medidas dão o mesmo preço.

3. O modelo de Hull-White

Material: Aula 4 (Stochastic Interest Rate Modeling) e Aula 5 (Calibration and Data Analysis)

3.1 De Vasicek a Hull-White

- **Vasicek** (1977): $dr = (\eta - a\,r)\,dt + \sigma\,dW$. Reversão à média, taxa normal, fórmulas fechadas. Mas com três parâmetros constantes não reproduz a curva de hoje.
- **Ho-Lee** (1986): $dr = \eta(t)\,dt + \sigma\,dW$. O drift dependente do tempo ajusta a curva, mas não há reversão: a variância de r cresce sem limite.
- **Hull-White** (1990), o "Vasicek estendido": $dr = (\theta(t) - a\,r)\,dt + \sigma\,dW$. Junta os dois: reversão e ajuste exato da curva.

3.2 A solução, passo a passo

Passo 1. Multiplique por e^{at}: $d(e^{at}r) = e^{at}\theta(t)\,dt + \sigma e^{at}\,dW$.

Passo 2. Integre de s a t:

$$r(t) = r(s)\,e^{-a(t-s)} + \int_s^t e^{-a(t-u)}\theta(u)\,du + \sigma\int_s^t e^{-a(t-u)}\,dW(u). \tag{17.13}$$

Passo 3. A integral estocástica de um integrando determinístico é normal, com média zero e variância (isometria de Itô) $\sigma^2\int_s^t e^{-2a(t-u)}du = \frac{\sigma^2}{2a}(1 - e^{-2a(t-s)})$. Logo $r(t)$ é **normal**: pode ficar negativa, com probabilidade calculável.

3.3 O título zero-cupom: a forma afim

Pelo argumento de hedge da Seção 2.1, $V(r,t)$ satisfaz a EDP de precificação

$$\frac{\partial V}{\partial t} + \tfrac{1}{2}\sigma^2\frac{\partial^2 V}{\partial r^2} + (\theta(t) - a\,r)\frac{\partial V}{\partial r} - rV = 0, \qquad V(r,T) = 1. \tag{17.14}$$

Tente $P = e^{A(t) - rB(t)}$, como no CIR (Módulo 11). Substituindo e dividindo por P:

$$\dot{A} - r\dot{B} + \tfrac{1}{2}\sigma^2 B^2 - (\theta - a\,r)B - r = 0. \tag{17.15}$$

O termo em r e o termo sem r se anulam separadamente:

$$\dot{B} = aB - 1,\ B(T) = 0 \ \Rightarrow\ B(t,T) = \frac{1 - e^{-a(T-t)}}{a}; \qquad \dot{A} = \theta B - \tfrac{1}{2}\sigma^2 B^2,\ A(T) = 0. \tag{17.16}$$

3.4 Ajustar a curva: a escolha de $\theta(t)$

Queremos que o modelo devolva $P(0,T) = P^{mkt}(0,T)$ para todo T. Em termos de taxas forward, a condição é $f^{modelo}(0,T) = f^{mkt}(0,T)$. Escrevendo $r(t) = x(t) + \varphi(t)$, com x o OU que parte de zero, $dx = -a\,x\,dt + \sigma\,dW$, e φ determinística, a condição de ajuste é (Brigo e Mercurio, cap. 3)

$$\varphi(t) = f^{mkt}(0,t) + \frac{\sigma^2}{2a^2}\left(1 - e^{-at}\right)^2. \tag{17.17}$$

Derivando $r = x + \varphi$ e comparando com a EDS, $\theta(t) = \varphi'(t) + a\,\varphi(t)$:

$$\boxed{\theta(t) = \frac{\partial f^{mkt}(0,t)}{\partial t} + a\, f^{mkt}(0,t) + \frac{\sigma^2}{2a}(1 - e^{-2at}).} \tag{17.18}$$

Com θ assim, o preço do título em $t > 0$ fica escrito só com a curva de hoje:

$$P(t,T) = \frac{P^{mkt}(0,T)}{P^{mkt}(0,t)} \exp\Big(B(t,T)\, f^{mkt}(0,t) - \frac{\sigma^2}{4a}(1 - e^{-2at}) B(t,T)^2 - B(t,T)\, r(t) \Big). \tag{17.19}$$

3.5 Simular sem erro de discretização

Como tudo é gaussiano, o par (x no fim do passo, integral de x no passo) tem lei conhecida. Num passo de tamanho Δ:

$$x_{t+\Delta} = x_t e^{-a\Delta} + \varepsilon_1, \qquad \int_t^{t+\Delta} x\, du = x_t\, B(\Delta) + \varepsilon_2, \tag{17.20}$$

com $(\varepsilon_1, \varepsilon_2)$ normal bivariada de média zero,

$$\operatorname{Var}\varepsilon_1 = \frac{\sigma^2}{2a}(1 - e^{-2a\Delta}), \quad \operatorname{Var}\varepsilon_2 = \frac{\sigma^2}{a^2}\Big(\Delta - 2B(\Delta) + \frac{1 - e^{-2a\Delta}}{2a}\Big), \quad \operatorname{Cov} = \frac{\sigma^2}{2a^2}(1 - e^{-a\Delta})^2. \tag{17.21}$$

A integral de r é a de x mais $\int \varphi$, que é fechada para a curva Nelson-Siegel. Nenhuma aproximação de Euler: o único erro do Monte Carlo é estatístico.

Laboratório 2 — O modelo reproduz a curva

Parâmetros: $a = 0{,}1$ e $\sigma = 1\%$ ao ano (volatilidade normal, em pontos de taxa).

```
A_HW, S_HW = 0.10, 0.010


def B_hw(tau, a=A_HW):
    return (1 - np.exp(-a * np.asarray(tau, float))) / a


def theta_hw(t, a=A_HW, s=S_HW):
    return df_merc(t) + a * f_merc(t) + s**2 / (2 * a) * (1 - np.exp(-2 * a * np.asarray(t,
    ↪  float)))


def phi_hw(t, a=A_HW, s=S_HW):
    return f_merc(t) + s**2 / (2 * a**2) * (1 - np.exp(-a * np.asarray(t, float))) ** 2


def int_phi(t0, t1, a=A_HW, s=S_HW):
    """∫ φ(u) du de t0 a t1, fechada."""
    def prim(t):
        t = np.asarray(t, float)
        return -np.log(P_merc(t)) + s**2 / (2 * a**2) * (t - 2 * (1 - np.exp(-a * t)) / a + (1 -
        ↪  np.exp(-2 * a * t)) / (2 * a))
    return prim(t1) - prim(t0)
```

```
def P_hw(t, T, r, a=A_HW, s=S_HW):
    """Zero-cupom em t, dado r(t), com θ ajustado à curva (Seção 3.4)."""
    B = B_hw(T - t, a)
    return P_merc(T) / P_merc(t) * np.exp(B * f_merc(t) - s**2 / (4 * a) * (1 - np.exp(-2 * a *
    ↪  t)) * B**2 - B * r)

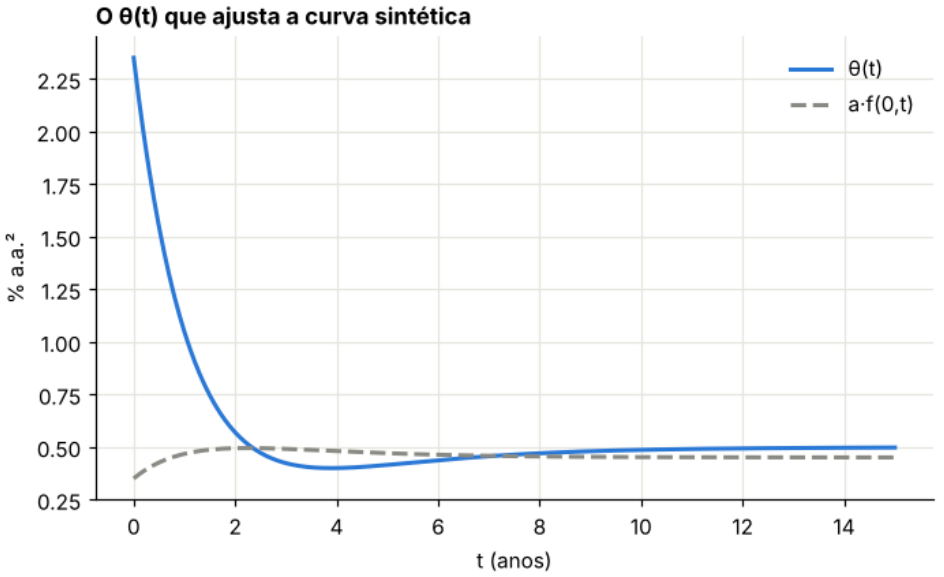

def simular_hw(datas, I, rng, a=A_HW, s=S_HW, medida_T=None):
    """Simulação exata de (r, ∫r) nas datas pedidas. Com medida_T, simula sob a medida forward em
    ↪  medida_T."""
    datas = np.asarray(datas, float)
    x = np.zeros(I); integ = np.zeros(I); t = 0.0
    rs, Is = [], []
    for t1 in datas:
        d = t1 - t
        e1 = np.exp(-a * d)
        v1 = s**2 / (2 * a) * (1 - e1**2)
        v2 = s**2 / a**2 * (d - 2 * (1 - e1) / a + (1 - e1**2) / (2 * a))
        c12 = s**2 / (2 * a**2) * (1 - e1) ** 2
        L = np.linalg.cholesky(np.array([[v1, c12], [c12, v2]]))
        z = L @ rng.standard_normal((2, I))
        m1, m2 = 0.0, 0.0
        if medida_T is not None:
            # drift extra −σ²B(u,T) integrado exatamente no passo
            k = lambda u: -(s**2 / a) * (1 - np.exp(-a * (medida_T - u)))
            us = np.linspace(t, t1, 201)
            m1 = np.trapezoid(k(us) * np.exp(-a * (t1 - us)), us)
            m2 = np.trapezoid(k(us) * (1 - np.exp(-a * (t1 - us))) / a, us)
        integ = integ + x * (1 - e1) / a + z[1] + m2
        x = x * e1 + z[0] + m1
        rs.append(x + phi_hw(t1, a, s)); Is.append(integ + int_phi(0, t1, a, s))
        t = t1
    return np.array(rs), np.array(Is)


rng = np.random.default_rng(17)
Ts = np.array([1.0, 2.0, 5.0, 10.0])
r_sim, I_sim = simular_hw(Ts, 200_000, rng)
desc = np.exp(-I_sim)
linhas = []
for k, T in enumerate(Ts):
    est = desc[k].mean(); ep = desc[k].std() / np.sqrt(desc.shape[1])
    linhas.append((T, P_merc(T), est, (est - P_merc(T)) / ep, r_sim[k].mean(), phi_hw(T),
↪  r_sim[k].std(),
                   S_HW * np.sqrt((1 - np.exp(-2 * A_HW * T)) / (2 * A_HW)), 100 *
↪  np.mean(r_sim[k] < 0)))
tab2 = pd.DataFrame(linhas, columns=["T", "P mercado", "E[e^{−∫r}] (MC)", "desvio em
↪  erros-padrão",
                                     "média de r(T) MC", "φ(T)", "dp de r(T) MC", "dp teórico", "%
                                      ↪  r(T) < 0"])
tabela(tab2)

fig, ax = plt.subplots()
tt = np.linspace(0, 15, 300)
ax.plot(tt, 100 * theta_hw(tt), color=AZUL, label="θ(t)")
ax.plot(tt, 100 * A_HW * f_merc(tt), "--", color=CINZA, label="a·f(0,t)")
ax.set_xlabel("t (anos)"); ax.set_ylabel("% a.a.²"); ax.set_title("O θ(t) que ajusta a curva
↪  sintética")
ax.legend(); plt.show()
```

```
ok(np.all(np.abs(tab2["desvio em erros-padrão"]) < 4), "o Monte Carlo exato reprecifica a curva
↪  de mercado em todos os prazos")
ok(np.allclose(tab2["média de r(T) MC"], tab2["φ(T)"], atol=4e-4), "a média de r(T) é φ(T)")
ok(np.allclose(tab2["dp de r(T) MC"], tab2["dp teórico"], rtol=0.01), "e o desvio-padrão é o da
↪  isometria de Itô")
ok(abs(np.log(P_hw(0.0, 7.0, f_merc(0.0)) / P_merc(7.0))) < 1e-12, "a fórmula de P(t,T) em t = 0
↪  devolve a curva de mercado")
```

T	P mercado	E[e^{−∫r}] (MC)	desvio em erros-padrão	média de r(T) MC	φ(T)	dp de r(T) MC	dp teórico	% r(T) < 0
1,000	0,959	0,959	-0,226	0,047	0,047	0,010	0,010	0,000
2,000	0,913	0,913	-1,007	0,050	0,050	0,013	0,013	0,002
5,000	0,790	0,790	-1,424	0,048	0,048	0,018	0,018	0,376
10,000	0,628	0,628	-2,897	0,047	0,047	0,021	0,021	1,142




```
OK — o Monte Carlo exato reprecifica a curva de mercado em todos os prazos
OK — a média de r(T) é φ(T)
OK — e o desvio-padrão é o da isometria de Itô
OK — a fórmula de P(t,T) em t = 0 devolve a curva de mercado
```

O modelo reprecifica a curva com erro dentro do ruído estatístico, como manda a construção. Repare também na última coluna: com $\sigma = 1\%$ e a curva em torno de 4,5%, cerca de 1 em cada 100 cenários de 10 anos tem taxa negativa; com a curva perto de zero, como na Europa de 2015 a 2021, seriam quase metade. Em ambientes de juros baixos isso é um defeito; em outros, uma propriedade aceitável.

Laboratório 3 — A mesma opção sob duas medidas

Uma call europeia com vencimento $T = 1$ sobre o zero-cupom que vence em $S = 5$, com strike igual ao preço forward, $K = P(0,S)/P(0,T)$. Pela Seção 2.2, devemos ter

$$\mathbb{E}^{Q}\big[e^{-\int_0^T r}\,(P(T,S)-K)^{+}\big] = P(0,T)\,\mathbb{E}^{T}\big[(P(T,S)-K)^{+}\big]. \tag{17.22}$$

Também conferimos a fórmula fechada de $P(t,T)$ por um Monte Carlo **condicional**:

partindo de três valores de $r(1)$, simulamos de 1 a 5 anos e comparamos $\mathbb{E}[e^{-\int_1^5 r} \,|\, r(1)]$ com a fórmula.

```
T_op, S_op = 1.0, 5.0
K_atm = P_merc(S_op) / P_merc(T_op)

rng = np.random.default_rng(3)
rQ, IQ = simular_hw([T_op], 400_000, rng)
pay_Q = np.exp(-IQ[0]) * np.maximum(P_hw(T_op, S_op, rQ[0]) - K_atm, 0)
rT, _ = simular_hw([T_op], 400_000, rng, medida_T=T_op)
pay_T = P_merc(T_op) * np.maximum(P_hw(T_op, S_op, rT[0]) - K_atm, 0)
print("call sob Q:", f3(1e4 * pay_Q.mean()), "pb | sob a medida forward:", f3(1e4 * pay_T.mean()),
↪  "pb (por unidade de face)")
print("E^T[P(T,S)] =", f"{P_hw(T_op, S_op, rT[0]).mean():.6f}", "| preço forward P(0,S)/P(0,T)
↪  =", f"{K_atm:.6f}")

ep_Q = pay_Q.std() / np.sqrt(len(pay_Q)); ep_T = pay_T.std() / np.sqrt(len(pay_T))
ok(abs(pay_Q.mean() - pay_T.mean()) < 4 * np.hypot(ep_Q, ep_T), "as duas medidas dão o mesmo
↪  preço, a menos do ruído")
ok(abs(P_hw(T_op, S_op, rT[0]).mean() / K_atm - 1) < 2e-4, "sob a medida forward, o preço forward
↪  do título é martingal")
ok(rT[0].mean() < rQ[0].mean(), "e a short rate tem drift menor sob a medida forward")

# P(t,T) por Monte Carlo condicional
linhas = []
rng = np.random.default_rng(5)
for r1 in [0.0, phi_hw(1.0), 0.08]:
    x1 = r1 - phi_hw(1.0)
    # simula x de 1 a 5 a partir de x1, com passos exatos
    x = np.full(100_000, x1); integ = np.zeros_like(x); t = 1.0
    for t1 in np.linspace(1.0, 5.0, 41)[1:]:
        d = t1 - t; e1 = np.exp(-A_HW * d)
        v1 = S_HW**2 / (2 * A_HW) * (1 - e1**2)
        v2 = S_HW**2 / A_HW**2 * (d - 2 * (1 - e1) / A_HW + (1 - e1**2) / (2 * A_HW))
        c12 = S_HW**2 / (2 * A_HW**2) * (1 - e1) ** 2
        z = np.linalg.cholesky([[v1, c12], [c12, v2]]) @ rng.standard_normal((2, len(x)))
        integ += x * (1 - e1) / A_HW + z[1]; x = x * e1 + z[0]; t = t1
    est = np.exp(-(integ + int_phi(1.0, 5.0))).mean()
    linhas.append((100 * r1, P_hw(1.0, 5.0, r1), est))
tab3 = pd.DataFrame(linhas, columns=["r(1) (%)", "P(1,5) fórmula", "P(1,5) Monte Carlo
↪  condicional"])
tabela(tab3)
ok(np.allclose(tab3["P(1,5) fórmula"], tab3["P(1,5) Monte Carlo condicional"], rtol=5e-4), "a
↪  fórmula afim de P(t,T) confere com a esperança condicional")
```

```
call sob Q: 98,788 pb | sob a medida forward: 98,735 pb (por unidade de face)
E^T[P(T,S)] = 0.823765 | preço forward P(0,S)/P(0,T) = 0.823767
OK — as duas medidas dão o mesmo preço, a menos do ruído
OK — sob a medida forward, o preço forward do título é martingal
OK — e a short rate tem drift menor sob a medida forward
```

r(1) (%)	P(1,5) fórmula	P(1,5) Monte Carlo condicional
0,000	0,960	0,961
4,676	0,823	0,823
8,000	0,738	0,738

```
OK — a fórmula afim de P(t,T) confere com a esperança condicional
```

---

4. Opções sobre títulos: a fórmula de Jamshidian

Material: Aula 3 (modelo de Black para opções sobre títulos) e planilha, aba "opcao sobre bond"

4.1 Dedução pela medida forward

Passo 1. Pela Seção 2.2 com numerário $P(t,T)$: $c_0 = P(0,T)\,\mathbb{E}^T[(P(T,S)-K)^+]$.

Passo 2. O preço forward $F(t) = P(t,S)/P(t,T)$ é martingal sob a medida forward. Pela forma afim, $\ln F$ é afim em r, e sua volatilidade é determinística:

$$\frac{dF}{F} = -\sigma\big[B(t,S) - B(t,T)\big]\,dW^T. \tag{17.23}$$

Passo 3. Então $\ln F(T)$ é normal, e $F(T) = P(T,S)$ (porque $P(T,T)=1$). A variância acumulada é

$$\sigma_P^2 = \sigma^2 \int_0^T \big[B(t,S) - B(t,T)\big]^2 dt \quad\Longrightarrow\quad \sigma_P = \frac{\sigma}{a}\big(1-e^{-a(S-T)}\big)\sqrt{\frac{1-e^{-2aT}}{2a}}. \tag{17.24}$$

Passo 4. É o modelo de Black (Aula 3) com volatilidade integrada σ_P:

$$\boxed{c_0 = P(0,S)\,N(h) - K\,P(0,T)\,N(h-\sigma_P), \qquad h = \frac{1}{\sigma_P}\ln\frac{P(0,S)}{K\,P(0,T)} + \frac{\sigma_P}{2},} \tag{17.25}$$

e a put, por paridade: $p_0 = K\,P(0,T)\,N(-h+\sigma_P) - P(0,S)\,N(-h)$.

Laboratório 4 — Fórmula, Monte Carlo e a aba da planilha

```python
def sigma_P(T, S, a=A_HW, s=S_HW):
    return s / a * (1 - np.exp(-a * (S - T))) * np.sqrt((1 - np.exp(-2 * a * T)) / (2 * a))


def zbc(T, S, K, PT, PS, a=A_HW, s=S_HW):
    sp = sigma_P(T, S, a, s)
    h_ = np.log(PS / (K * PT)) / sp + sp / 2
    return PS * norm.cdf(h_) - K * PT * norm.cdf(h_ - sp)


def zbp(T, S, K, PT, PS, a=A_HW, s=S_HW):
    sp = sigma_P(T, S, a, s)
    h_ = np.log(PS / (K * PT)) / sp + sp / 2
    return K * PT * norm.cdf(-h_ + sp) - PS * norm.cdf(-h_)


linhas = []
for K in [0.95 * K_atm, K_atm, 1.05 * K_atm]:
```

```
    formula = zbc(T_op, S_op, K, P_merc(T_op), P_merc(S_op))
    mc = (np.exp(-IQ[0]) * np.maximum(P_hw(T_op, S_op, rQ[0]) - K, 0))
    put = zbp(T_op, S_op, K, P_merc(T_op), P_merc(S_op))
    linhas.append((K, 1e4 * formula, 1e4 * mc.mean(), 1e4 * mc.std() / np.sqrt(len(mc)),
                   1e4 * (formula - put - (P_merc(S_op) - K * P_merc(T_op)))))
tab4 = pd.DataFrame(linhas, columns=["strike", "call fórmula (pb)", "call MC (pb)", "erro-padrão
↪  (pb)", "paridade (pb)"])
tabela(tab4)
print("σ_P =", f3(100 * sigma_P(T_op, S_op)), "% (volatilidade integrada do preço forward)")
ok(np.all(np.abs(tab4["call fórmula (pb)"] - tab4["call MC (pb)"]) < 4 * tab4["erro-padrão
↪  (pb)"]), "Jamshidian confere com o Monte Carlo exato")
ok(np.all(np.abs(tab4["paridade (pb)"]) < 1e-9), "paridade put-call vale exatamente")
```

strike	call fórmula (pb)	call MC (pb)	erro-padrão (pb)	paridade (pb)
0,783	400,108	399,790	0,379	-0,000
0,824	98,897	98,788	0,234	0,000
0,865	6,567	6,546	0,057	-0,000

```
σ_P = 3,139 % (volatilidade integrada do preço forward)
OK — Jamshidian confere com o Monte Carlo exato
OK — paridade put-call vale exatamente
```

A aba "opcao sobre bond" da planilha

A planilha implementa a mesma fórmula, com $T = 0{,}25$ (3 meses), $S = 1/3$ (4 meses), $\sigma = 0{,}02464$, $a = 0{,}0527$ e $K = 0{,}9999$. Os descontos usam uma única taxa, $r = 5{,}24\%$ (a de 3 meses), para os dois prazos. Reproduzimos o resultado e mostramos o efeito de três escolhas de entrada.

```
a_pl, s_pl, T_pl, S_pl, r_pl, K_pl = 0.0527, 0.02463782739818595, 0.25, 1 / 3, 0.0524, 0.9999
PT_pl, PS_pl = np.exp(-r_pl * T_pl), np.exp(-r_pl * S_pl)
sp_pl = sigma_P(T_pl, S_pl, a_pl, s_pl)
premio_pl = zbc(T_pl, S_pl, K_pl, PT_pl, PS_pl, a_pl, s_pl)
print("planilha — σ_P:", f"{sp_pl:.7f}", "(planilha: 0,0010176) | prêmio:", f"{premio_pl:.3e}",
↪  "(planilha: 2,995e-09)")
ok(abs(sp_pl - 0.0010176144014825044) < 1e-12 and abs(premio_pl - 2.994811264221483e-09) < 1e-15,
↪  "a fórmula reproduz a planilha")

# curva da própria planilha: P(0,0,25) com 5,24% e P(0,1/3) com 5,21% (os yields das linhas 32 e
↪  33)
PT_c, PS_c = np.exp(-0.0524 * 0.25), np.exp(-0.0521 / 3)
F_c = PS_c / PT_c
casos = [("planilha (taxa única, K = 0,9999)", PT_pl, PS_pl, K_pl),
         ("curva da planilha, K = 0,9999", PT_c, PS_c, K_pl),
         ("curva da planilha, K no dinheiro a termo", PT_c, PS_c, F_c)]
tab4b = pd.DataFrame([(n, k, ps / pt, 1e4 * zbc(T_pl, S_pl, k, pt, ps, a_pl, s_pl)) for n, pt, ps,
↪  k in casos],
                     columns=["caso", "strike", "preço forward P(0,S)/P(0,T)", "call (pb da face)"])
tabela(tab4b)
ok(tab4b["call (pb da face)"].iloc[2] > 1e3 * tab4b["call (pb da face)"].iloc[0], "no dinheiro, a
↪  opção vale milhares de vezes mais que com K = 0,9999")
```

```
planilha — σ_P: 0.0010176 (planilha: 0,0010176) | prêmio: 2.995e-09 (planilha: 2,995e-09)
OK — a fórmula reproduz a planilha
```

caso	strike	preço forward P(0,S)/P(0,T)	call (pb da face)
planilha (taxa única, K = 0,9999)	1,000	0,996	0,000
curva da planilha, K = 0,9999	1,000	0,996	0,000
curva da planilha, K no dinheiro a termo	0,996	0,996	3,990

```
OK — no dinheiro, a opção vale milhares de vezes mais que com K = 0,9999
```

O strike de 0,9999 está muito acima do preço forward do título (cerca de 0,9957): a call está muito fora do dinheiro, e o prêmio de 3×10^{-9} é praticamente zero. Com o strike no dinheiro a termo, a mesma fórmula dá um prêmio da ordem de alguns pontos-base da face, que é o que se espera de uma opção de 3 meses sobre um título de 4 meses.

---

5. Caps e floors como opções sobre títulos

Material: Aula 3 (caps e floors)

5.1 Um caplet é uma put sobre um zero-cupom

Um caplet com datas $t_k < t_{k+1}$, prazo $\delta = t_{k+1} - t_k$, principal L e taxa-teto R_K paga em t_{k+1} o valor $L\delta\max(R_k - R_K, 0)$, em que R_k é a taxa simples observada em t_k. Trazendo para t_k (dividindo por $1+\delta R_k$):

$$\frac{L\delta(R_k - R_K)^+}{1+\delta R_k} = \Big(L - \frac{L(1+\delta R_K)}{1+\delta R_k}\Big)^+ = L(1+\delta R_K)\Big(\frac{1}{1+\delta R_K} - P(t_k, t_{k+1})\Big)^+. \tag{17.26}$$

É uma **put** sobre o zero-cupom de t_{k+1}, com vencimento t_k, strike $1/(1+\delta R_K)$ e quantidade $L(1+\delta R_K)$. No Hull-White, a fórmula da Seção 4 dá o caplet em forma fechada; o cap é a soma dos caplets, e o floor é a soma das calls correspondentes.

5.2 Volatilidades implícitas de Black

O mercado cota caplets pelo modelo de Black (Aula 3): caplet $= L\delta P(0,t_{k+1})\,[F_k N(d_1) - R_K N(d_2)]$, com F_k a taxa forward simples e $d_{1,2} = [\ln(F_k/R_K) \pm \sigma_k^2 t_k/2]/(\sigma_k\sqrt{t_k})$. Como o Hull-White é um modelo **normal**, as vols de Black que ele implica caem com o strike: o modelo tem um "skew" embutido.

Laboratório 5 — Caplets: fórmula, Monte Carlo e o skew implícito

```python
def caplet_hw(tk, delta, RK, a=A_HW, s=S_HW, L=1.0):
    Kb = 1 / (1 + delta * RK)
    return L * (1 + delta * RK) * zbp(tk, tk + delta, Kb, P_merc(tk), P_merc(tk + delta), a, s)


def caplet_black(tk, delta, RK, vol, L=1.0):
    F = (P_merc(tk) / P_merc(tk + delta) - 1) / delta
    d1 = (np.log(F / RK) + 0.5 * vol**2 * tk) / (vol * np.sqrt(tk))
```

```
    return L * delta * P_merc(tk + delta) * (F * norm.cdf(d1) - RK * norm.cdf(d1 - vol *
    ↪  np.sqrt(tk)))


def vol_black(preco, tk, delta, RK):
    return brentq(lambda v: caplet_black(tk, delta, RK, v) - preco, 1e-4, 3.0)

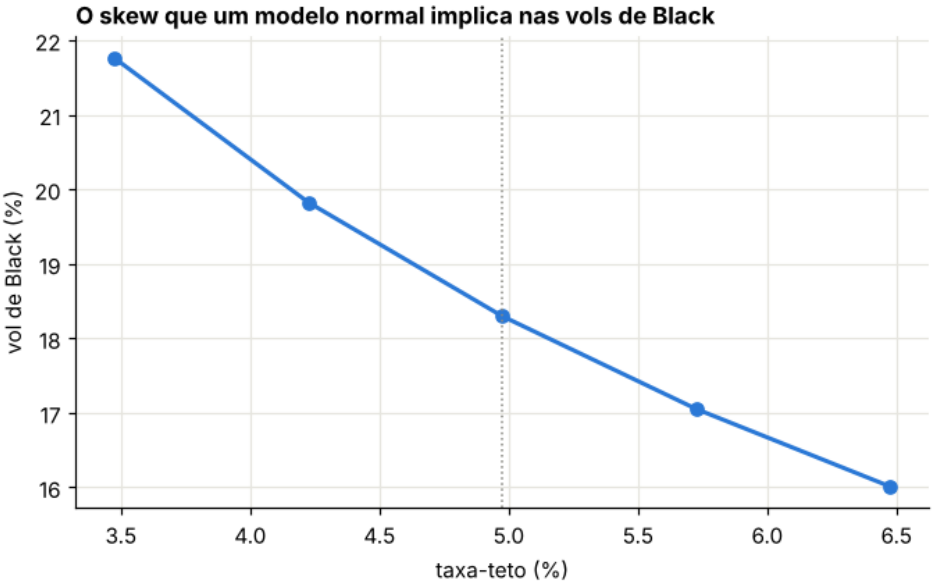

tk, delta = 2.0, 0.25
F2 = (P_merc(tk) / P_merc(tk + delta) - 1) / delta
rng = np.random.default_rng(8)
r_k, I_k = simular_hw([tk], 400_000, rng)
linhas = []
for RK in F2 + np.array([-0.015, -0.0075, 0.0, 0.0075, 0.015]):
    formula = caplet_hw(tk, delta, RK)
    Rk_sim = (1 / P_hw(tk, tk + delta, r_k[0]) - 1) / delta
    pay = np.exp(-I_k[0]) * P_hw(tk, tk + delta, r_k[0]) * delta * np.maximum(Rk_sim - RK, 0)
    linhas.append((100 * RK, 1e4 * formula, 1e4 * pay.mean(), 1e4 * pay.std() / np.sqrt(len(pay)),
↪  100 * vol_black(formula, tk, delta, RK)))
tab5 = pd.DataFrame(linhas, columns=["taxa-teto (%)", "caplet HW (pb)", "caplet MC (pb)",
↪  "erro-padrão (pb)", "vol de Black implícita (%)"])
tabela(tab5)
print("taxa forward F(2; 2,25) =", f3(100 * F2), "%")

fig, ax = plt.subplots()
ax.plot(tab5["taxa-teto (%)"], tab5["vol de Black implícita (%)"], "o-", color=AZUL)
ax.axvline(100 * F2, color=CINZA, ls=":", lw=1)
ax.set_xlabel("taxa-teto (%)"); ax.set_ylabel("vol de Black (%)")
ax.set_title("O skew que um modelo normal implica nas vols de Black")
plt.show()

ok(np.all(np.abs(tab5["caplet HW (pb)"] - tab5["caplet MC (pb)"]) < 4 * tab5["erro-padrão
↪  (pb)"]), "o caplet como put sobre título confere com o Monte Carlo do payoff original")
ok(np.all(np.diff(tab5["vol de Black implícita (%)"]) < 0), "as vols de Black implícitas pelo
↪  Hull-White caem com a taxa-teto")
```

taxa-teto (%)	caplet HW (pb)	caplet MC (pb)	erro-padrão (pb)	vol de Black implícita (%)
3,475	35,556	35,568	0,040	21,762
4,225	21,919	21,932	0,034	19,825
4,975	11,552	11,560	0,026	18,298
5,725	5,020	5,020	0,017	17,053
6,475	1,743	1,743	0,010	16,012

```
taxa forward F(2; 2,25) = 4,975 %
```




```
OK — o caplet como put sobre título confere com o Monte Carlo do payoff original
OK — as vols de Black implícitas pelo Hull-White caem com a taxa-teto
```

5.3 Calibrar a e σ a um cap

Material: Aula 5

Com $\theta(t)$ fixado pela curva, sobram a e σ. Eles são calibrados a opções: tipicamente, a vols de Black de caplets no dinheiro em vários vencimentos. σ dá o nível; a dá a forma da estrutura a termo das vols. Com reversão, a volatilidade da taxa de prazo longo é menor que a da curta, e as vols de caplets longos caem.

Geramos um "mercado" de vols ATM a partir de $a = 0{,}08$ e $\sigma = 0{,}9\%$, com ruído de 0,2 ponto de vol, e recuperamos os parâmetros por mínimos quadrados.

```
venc = np.arange(1.0, 10.01, 1.0)
A_V, S_V = 0.08, 0.009
rng = np.random.default_rng(11)


def vols_atm(a, s):
    out = []
    for t in venc:
        F = (P_merc(t) / P_merc(t + delta) - 1) / delta
        out.append(vol_black(caplet_hw(t, delta, F, a, s), t, delta, F))
    return np.array(out)

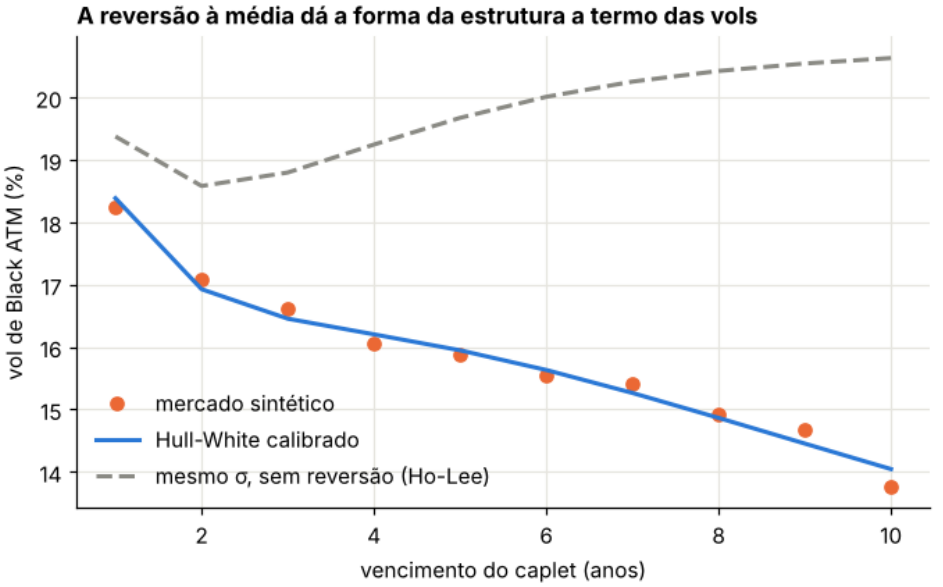

vols_merc = vols_atm(A_V, S_V) + rng.normal(0, 0.002, len(venc))
res = least_squares(lambda p: vols_atm(*p) - vols_merc, x0=[0.2, 0.012], bounds=([0.001, 0.001],
↪  [1.0, 0.05]))
a_cal, s_cal = res.x
print("calibrado: a =", f3(a_cal), "| σ =", f3(100 * s_cal), "% | verdade: a = 0,080, σ = 0,900 %")

fig, ax = plt.subplots()
```

```
ax.plot(venc, 100 * vols_merc, "o", color=LARANJA, label="mercado sintético")
ax.plot(venc, 100 * vols_atm(a_cal, s_cal), color=AZUL, label="Hull-White calibrado")
ax.plot(venc, 100 * vols_atm(0.0001, s_cal), "--", color=CINZA, label="mesmo σ, sem reversão
↪  (Ho-Lee)")
ax.set_xlabel("vencimento do caplet (anos)"); ax.set_ylabel("vol de Black ATM (%)")
ax.set_title("A reversão à média dá a forma da estrutura a termo das vols"); ax.legend();
↪  plt.show()

ok(abs(a_cal - A_V) < 0.03 and abs(s_cal - S_V) < 0.0007, "a calibração recupera a e σ dentro do
↪  ruído das cotações")
```

```
calibrado: a = 0,085 | σ = 0,911 % | verdade: a = 0,080, σ = 0,900 %
```




```
OK — a calibração recupera a e σ dentro do ruído das cotações
```

---

6. Estimar σ pela série histórica

Material: Aula 5 (Data Analysis) e planilha, aba "daily-treasury-rates"

6.1 A conta certa

No Hull-White, σ é a volatilidade **normal** da short rate: num dia, $\Delta r \approx \sigma\sqrt{\Delta t}\, z$. Com variações diárias e 252 dias úteis por ano,

$$\hat{\sigma} = \mathrm{dp}(\Delta r_{\text{diário}}) \times \sqrt{252}. \tag{17.27}$$

A variância cresce com o tempo; o desvio-padrão, com a **raiz** do tempo.

6.2 O que a planilha faz

A aba "daily-treasury-rates" calcula, para o prazo de m meses, a variação diária do fator $1 + R \cdot 30m/360$ e anualiza o desvio-padrão multiplicando por 252. Dois efeitos se somam:

1. **Anualização.** Multiplicar o desvio-padrão por 252, em vez de $\sqrt{252}$, aumenta o resultado em $\sqrt{252} \approx 15{,}9$ vezes.
2. **Escala do prazo.** A variação do fator é $\approx \Delta R \cdot m/12$. Para o prazo de 1 mês isso divide por 12; para 4 meses, por 3. A conta não volta para unidades de taxa.

Os dois se compensam parcialmente no prazo de 1 mês ($252/12 = 21$ contra 15,9: 32% acima) e se acumulam nos prazos maiores. Por isso a "estrutura a termo de vols" da planilha **cresce** com o prazo a partir de 2 meses, de 2,1% a 3,1%, quando a vol dos T-bills, medida corretamente, **cai** com o prazo, de 1,9% a 0,6%.

Laboratório 6 — A série dos T-bills e um teste com verdade conhecida

```
dados = carregar_treasury()
janela = dados[dados["data"] <= "2024-01-03"]
linhas = []
for col, m in [("bd_4w", 1), ("bd_8w", 2), ("bd_13w", 3), ("bd_17w", 4)]:
    Lt = janela[col].to_numpy() / 100
    fator = (1 + Lt[:-1] * 30 * m / 360) / (1 + Lt[1:] * 30 * m / 360) - 1
    vol_planilha = fator.std(ddof=1) * 252
    vol_certa = np.diff(Lt).std(ddof=1) * np.sqrt(252)
    linhas.append((f"{m} {'mês' if m == 1 else 'meses'}", 100 * vol_planilha, 100 * vol_certa,
↪   vol_planilha / vol_certa))
tab6 = pd.DataFrame(linhas, columns=["prazo", "vol da planilha (%)", "vol normal anual (%)",
↪   "razão"])
tabela(tab6)
print(f"janela: {janela['data'].iloc[0].date()} a {janela['data'].iloc[-1].date()} ({len(janela)}
↪   dias úteis)")

ok(abs(tab6["vol da planilha (%)"].iloc[0] - 2.4638) < 0.01, "a conta da planilha reproduz o vol
↪   de 2,464% do prazo de 1 mês")
ok(abs(tab6["razão"].iloc[0] - 21 / np.sqrt(252)) < 0.03, "no prazo de 1 mês, a razão é 21/√252 ≈
↪   1,32, como previsto")
ok(np.all(np.diff(tab6["vol da planilha (%)"].iloc[1:]) > 0) and np.all(np.diff(tab6["vol normal
↪   anual (%)"]) < 0),
   "a vol da planilha cresce com o prazo a partir de 2 meses; a vol normal cai sempre")
```

prazo	vol da planilha (%)	vol normal anual (%)	razão
1 mês	2,469	1,874	1,318
2 meses	2,140	0,815	2,625
3 meses	2,723	0,694	3,921
4 meses	3,068	0,589	5,205

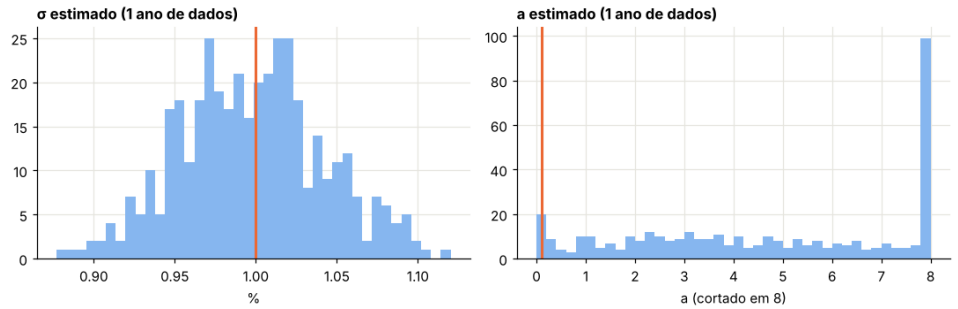
```
janela: 2023-01-03 a 2024-01-03 (252 dias úteis)
OK — a conta da planilha reproduz o vol de 2,464% do prazo de 1 mês
OK — no prazo de 1 mês, a razão é 21/√252 ≈ 1,32, como previsto
OK — a vol da planilha cresce com o prazo a partir de 2 meses; a vol normal cai sempre
```

```
# teste com verdade conhecida: um ano de taxas diárias simuladas pelo próprio HW (a = 0,1; σ = 1%)
rng = np.random.default_rng(21)
dt = 1 / 252
n_anos = 400
est_s, est_a = [], []
for _ in range(n_anos):
    r = np.empty(253); r[0] = 0.05
    e1 = np.exp(-A_HW * dt); sd = S_HW * np.sqrt((1 - e1**2) / (2 * A_HW))
    z = rng.standard_normal(252)
    for i in range(252):
        r[i + 1] = 0.05 + (r[i] - 0.05) * e1 + sd * z[i]
    est_s.append(np.diff(r).std(ddof=1) * np.sqrt(252))
    b = np.polyfit(r[:-1], r[1:], 1)[0]                       # AR(1): r_{t+1} = c + b r_t
    est_a.append(-np.log(max(b, 1e-6)) / dt)
est_s, est_a = np.array(est_s), np.array(est_a)
print("σ estimado em 1 ano: média", f3(100 * est_s.mean()), "% | dp", f3(100 * est_s.std()), "%
↪  (verdade 1,000%)")
print("a estimado em 1 ano: mediana", f3(np.median(est_a)), "| percentis 10%-90%:",
↪  f3(np.percentile(est_a, 10)), "a", f3(np.percentile(est_a, 90)), "(verdade 0,100)")

fig, axs = plt.subplots(1, 2, figsize=(10, 3.4))
axs[0].hist(100 * est_s, bins=40, color=RAMPA[0]); axs[0].axvline(100 * S_HW, color=LARANJA)
axs[0].set_title("σ estimado (1 ano de dados)"); axs[0].set_xlabel("%")
axs[1].hist(np.clip(est_a, 0, 8), bins=40, color=RAMPA[0]); axs[1].axvline(A_HW, color=LARANJA)
axs[1].set_title("a estimado (1 ano de dados)"); axs[1].set_xlabel("a (cortado em 8)")
plt.tight_layout(); plt.show()

ok(abs(est_s.mean() / S_HW - 1) < 0.01, "a conta com √252 recupera σ sem viés")
ok(np.percentile(est_a, 90) > 10 * A_HW, "com 1 ano de dados, a estimativa de a é inútil: o
↪  percentil 90% passa de 10 vezes a verdade")
```

```
σ estimado em 1 ano: média 0,998 % | dp 0,044 % (verdade 1,000%)
a estimado em 1 ano: mediana 4,656 | percentis 10%-90%: 0,916 a 12,790 (verdade 0,100)
```




```
OK — a conta com √252 recupera σ sem viés
OK — com 1 ano de dados, a estimativa de a é inútil: o percentil 90% passa de 10 vezes a verdade
```

O que aprendemos.

- σ se estima bem com um ano de dados diários, desde que a anualização use $\sqrt{252}$ e a variação seja medida em unidades de taxa. Mesmo assim, num ano como 2023 (ciclo de alta do Fed), boa parte da variância das short rates vem de saltos em reuniões de política monetária, não do movimento difuso que o modelo supõe.
- a não se estima com um ano de dados: a reversão atua em escalas de anos ($1/a = 10$

anos aqui). Na prática, a vem da calibração a opções (Seção 5.3), não da série histórica.

- Na planilha, o a ("alpha") é a célula da taxa de 1 mês (0,0527). Se a intenção era um valor de referência, convém documentá-lo; se era outra célula, vale corrigir. Os dois parâmetros entram em tudo que se segue (θ, árvore e opções).

---

7. Ajustar a curva: a favor e contra

Material: Aula 5 (Yield-curve fitting: for and against)

A favor. O argumento de hedge exige negociar a preços de mercado. Um modelo que não reprecifica os títulos de hoje dá arbitragem contra o próprio hedge. Ajustar $\theta(t)$ é o mínimo para usar o modelo numa mesa.

Contra. Se o modelo estivesse certo, o $\theta(t)$ ajustado hoje continuaria valendo amanhã (deslocado no tempo). Na prática, ao reajustar uma semana depois, o θ muda muito. É a evidência de que o modelo de um fator não descreve a dinâmica da curva: ele é uma **interpolação consistente** dos preços de hoje, e não uma previsão de como a curva vai se mover.

Para validação de modelos, a conclusão prática é a do Módulo 14: o ajuste perfeito à curva e às opções de calibração não diz nada sobre o desempenho fora da amostra. É preciso acompanhar a estabilidade dos parâmetros recalibrados e o P&L do hedge.

---

8. Síntese

Resultado	Fórmula	Laboratório
Troca de numerário	$f_0 = g_0\,\mathbb{E}^g[f_T/g_T]$	3
Solução do HW	$r(t)$ normal, média $\varphi(t)$, variância $\frac{\sigma^2}{2a}(1-e^{-2at})$	2
Zero-cupom	$P = e^{A-Br}$, $B = (1-e^{-a\tau})/a$	2 e 3
Ajuste à curva	$\theta = f' + af + \frac{\sigma^2}{2a}(1-e^{-2at})$	2
Opção sobre título	Black com $\sigma_P = \frac{\sigma}{a}(1-e^{-a(S-T)})\sqrt{\frac{1-e^{-2aT}}{2a}}$	4
Caplet	put sobre zero-cupom, strike $1/(1+\delta R_K)$	5
Estimação	$\hat\sigma = \mathrm{dp}(\Delta r)\sqrt{252}$; a vem de opções	6

9. Notas sobre a planilha HW_arvore_final

Onde	Observação
“HW yield”, célula B23 (alpha)	a aponta para a taxa de 1 mês (0,0527), não para uma estimativa de reversão; o mesmo valor alimenta a aba de opções
“daily-treasury-rates”, linha 3	desvio-padrão anualizado por 252, e não $\sqrt{252}$; variação do fator $1 + R \cdot 30m/360$ sem dividir por $m/12$: vol 32% acima no prazo de 1 mês e até 5 vezes acima no de 4 meses; a “estrutura a termo de vols” sobe com o prazo por construção
Curvas	as taxas usadas são as de *bank discount*, tratadas como contínuas; o *yield* contínuo equivalente do T-bill de 1 mês é cerca de 5,36% (base 365) contra 5,27%
“opcao sobre bond”	fórmula correta (Jamshidian); os dois descontos usam a mesma taxa (5,24%), e o strike de 0,9999 deixa a call muito fora do dinheiro (prêmio de 3×10^{-9})

10. Exercícios

Exercício 1. Mostre que, no Ho-Lee ($a \to 0$), $\theta(t) = \partial_t f^{mkt}(0,t) + \sigma^2 t$, e que a variância de $r(t)$ é $\sigma^2 t$.

Solução

Com $a \to 0$: $af \to 0$ e $\frac{\sigma^2}{2a}(1 - e^{-2at}) \to \sigma^2 t$. A variância $\frac{\sigma^2}{2a}(1 - e^{-2at}) \to \sigma^2 t$, sem o limite $\sigma^2/(2a)$ que a reversão impõe.

Exercício 2. Deduza a volatilidade do preço do zero-cupom $P(t,T)$ no Hull-White e explique por que ela vai a zero no vencimento.

Solução

$\ln P = A - Br$, então $d \ln P$ tem parte aleatória $-B\,\sigma\,dW$: a volatilidade é $\sigma B(t, T) = \frac{\sigma}{a}(1 - e^{-a(T-t)})$. Em $t = T$, $B = 0$: o título vale 1 com certeza (*pull to par*).

Exercício 3. Use a fórmula do caplet para precificar um cap de 5 anos com pagamentos trimestrais e taxa-teto de 5%. Qual a fração do preço que vem dos caplets dos dois últimos anos?

Solução

Some `caplet_hw(t, 0.25, 0.05)` para $t = 0{,}25, 0{,}5, \ldots, 4{,}75$. Os caplets longos pesam mais, porque a incerteza acumulada sobre a taxa é maior e a curva sintética sobe; a reversão à média limita esse crescimento.

Exercício 4. Refaça a calibração da Seção 5.3 usando só os vencimentos de 1 a 3 anos. O que acontece com o erro de a?

Solução

Com prazos curtos, $B(t, t+\delta) \approx \delta$ e a vol quase não depende de a: o parâmetro fica mal identificado, como o κ_v do Módulo 13. Os prazos longos é que revelam a reversão.

```
print(f"Módulo 17 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 17 concluído: 22 verificações aprovadas.
```

Capítulo 18

Hull-White II: a árvore trinomial

Introdução

Fórmulas fechadas cobrem opções europeias simples. Para títulos com opções embutidas, como os que podem ser resgatados antecipadamente pelo emissor (*callable*) ou devolvidos pelo investidor (*puttable*), e para modelos sem fórmulas, como o de Black-Karasinski, precisa-se de um método numérico. No mundo dos juros, o padrão é a **árvore trinomial de Hull e White**.

A árvore é construída em duas etapas. Primeiro, uma árvore para a parte aleatória da short rate, com probabilidades escolhidas para reproduzir sua média e variância. Depois, um deslocamento em cada data, escolhido para que a árvore reproduza exatamente a curva de juros de hoje. Os **preços de Arrow-Debreu**, valores de receber 1 em cada nó, fazem esse ajuste em uma única passagem para a frente.

Com a árvore pronta, o capítulo precifica títulos, opções europeias e americanas, caps e títulos callable e puttable, estuda a convergência e estende a construção ao modelo lognormal de Black-Karasinski.

Fundamentos teóricos

Árvores trinomiais por moment matching

Hull e White constroem a árvore em duas etapas (Hull and White, 1994): primeiro uma árvore para o processo auxiliar x, com $dx_t = -a x_t\,dt + \sigma\,dW_t$ e $x_0 = 0$; depois um deslocamento determinístico α_m em cada data, $r = x + \alpha$, escolhido para reproduzir a curva.

Definição 18.1 (Malha). Com passo Δt, os nós são $x_{m,j} = j\,\Delta x$, com $\Delta x = \sigma\sqrt{3\Delta t}$. Condicionalmente a $x_{m,j}$, o próximo valor tem média $x_{m,j} + M\,x_{m,j}$ e variância V, com

$$M = e^{-a\Delta t} - 1 \approx -a\Delta t, \qquad V = \frac{\sigma^2}{2a}(1 - e^{-2a\Delta t}) \approx \sigma^2 \Delta t. \tag{18.1}$$

Proposição 18.2 (Probabilidades de transição). *Para o ramo padrão $(j+1, j, j-1)$ e $\eta = jM$, as probabilidades que casam média e variância são*

$$p_u = \frac{1}{6} + \frac{\eta^2 + \eta}{2}, \qquad p_m = \frac{2}{3} - \eta^2, \qquad p_d = \frac{1}{6} + \frac{\eta^2 - \eta}{2}. \tag{18.2}$$

Elas são positivas se e somente se $|jM| < \sqrt{2/3}$. Para $j > j_{\max}$, com $j_{\max} = \lceil 0{,}184/(a\Delta t)\rceil$, usa-se o ramo $(j, j-1, j-2)$, e simetricamente embaixo.

Demonstração. Em unidades de Δx, as condições são $p_u - p_d = \eta$, $p_u + p_d = \frac{1}{3} + \eta^2$ (variância $V/\Delta x^2 = \frac{1}{3}$ mais o quadrado da média) e $p_u + p_m + p_d = 1$. A positividade de p_m exige $\eta^2 < \frac{2}{3}$. O valor 0,184 é o menor que garante probabilidades positivas nos dois ramos na transição; é a escolha de Hull e White. ∎

Preços de Arrow-Debreu e o deslocamento

Definição 18.3 (Preços de Arrow-Debreu). $Q_{m,j}$ é o valor hoje de um título que paga 1 se o nó (m, j) for atingido. Começa com $Q_{0,0} = 1$ e avança por

$$Q_{m+1,k} = \sum_j Q_{m,j}\, \pi_{j\to k}\, e^{-(\alpha_m + j\Delta x)\Delta t}. \tag{18.3}$$

Proposição 18.4 (Ajuste exato da curva). *O deslocamento que faz a árvore reproduzir $P^{\text{mkt}}(0, t_{m+1})$ é*

$$\alpha_m = \frac{1}{\Delta t} \ln \frac{\sum_j Q_{m,j}\, e^{-j\Delta x\, \Delta t}}{P^{\text{mkt}}(0, t_{m+1})}. \tag{18.4}$$

Demonstração. O título que vence em t_{m+1} vale $\sum_j Q_{m,j}\, e^{-(\alpha_m + j\Delta x)\Delta t}$, pois paga 1 em t_{m+1} a partir de qualquer nó de t_m. Igualando a $P^{\text{mkt}}(0, t_{m+1})$ e isolando α_m obtém-se a fórmula. ∎

Alternando a Proposição 18.4 e (18.3), a árvore inteira é calibrada à curva em uma passagem para a frente. A formulação de Clewlow and Strickland (1998) escreve o mesmo passo em termos de θ_m, o análogo discreto de $\vartheta(t)$ na Proposição 17.4.

Black-Karasinski

Definição 18.5 (Modelo de Black-Karasinski). $d\ln r_t = (\vartheta(t) - a \ln r_t)\, dt + \sigma\, dW_t$ (Black and Karasinski, 1991). As taxas são lognormais e positivas, mas não há fórmulas fechadas para títulos.

Proposição 18.6 (Calibração na árvore). *Com a mesma árvore para $x = \ln r - \alpha$, o deslocamento α_m resolve a equação não linear*

$$\sum_j Q_{m,j} \exp\big(-e^{\alpha_m + j\Delta x}\, \Delta t\big) = P^{\text{mkt}}(0, t_{m+1}), \tag{18.5}$$

cujo lado esquerdo é estritamente decrescente em α_m; o método de Newton converge em poucas iterações a partir de α_{m-1}.

Derivativos na árvore

Calibrada a árvore, títulos, opções europeias e americanas, caps e títulos com opções embutidas (*callable* e *puttable*) são precificados por indução para trás, com o desconto do nó $e^{-r_{m,j}\Delta t}$ e o máximo (ou mínimo) entre continuação e exercício onde houver direito de exercício. O erro de discretização é $O(\Delta t)$, com oscilações quando os vencimentos e as datas de exercício não caem na malha.

Roteiro do capítulo

Curso: Derivativos com Python. Módulo complementar, construído sobre Clewlow e Strickland, *Implementing Derivatives Models*, caps. 8 e 9; Hull e White (1993, 1994a, 1994b); e a planilha HW_arvore_final.

O que você vai conseguir fazer ao final

1. Explicar o papel dos preços de Arrow-Debreu ("pure securities") na indução para a frente.
2. Deduzir as probabilidades de uma árvore trinomial e o critério de troca de ramificação.
3. Construir a árvore de Hull-White pelos dois métodos clássicos: o de Clewlow-Strickland (§9.3), usado na planilha, e o de dois estágios de Hull-White (1994).
4. Reproduzir a aba "HW yield" da planilha e localizar um erro que nenhuma conferência de soma detecta.
5. Precificar na árvore opções europeias e americanas sobre títulos, títulos com opção de recompra e caps, e medir a convergência contra as fórmulas do Módulo 17.
6. Entender por que ajustar também a estrutura a termo das volatilidades (aba "HW yield and vol") é delicado.
7. Trocar o Hull-White pelo Black-Karasinski na mesma árvore e ver o que muda.

Onde estamos

O Módulo 17 deu fórmulas fechadas para títulos, opções europeias e caps. Elas não bastam para opções americanas, títulos com opções embutidas (*callable*, *puttable*) e payoffs dependentes do caminho. A árvore trinomial resolve esses casos e serve de laboratório para modelos sem fórmula fechada, como o Black-Karasinski. É o equivalente, para juros, da árvore CRR do Módulo 6.

0. Preparação

As funções da curva sintética e as fórmulas fechadas do Módulo 17 são repetidas aqui, para servir de referência.

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import time
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
from scipy.optimize import brentq
from scipy.stats import norm
from IPython.display import display

RAMPA = ["#86b6ef", "#2a78d6", "#104281"]

# curva sintética Nelson-Siegel do Módulo 17
def P_merc(T, b0=0.045, b1=-0.010, b2=0.020, tau=1.5):
    T = np.asarray(T, float); e = np.exp(-T / tau)
    return np.exp(-(b0 * T + b1 * tau * (1 - e) + b2 * tau * (1 - (1 + T / tau) * e)))

A_HW, S_HW = 0.10, 0.010

def sigma_P(T, S, a=A_HW, s=S_HW):
    return s / a * (1 - np.exp(-a * (S - T))) * np.sqrt((1 - np.exp(-2 * a * T)) / (2 * a))

def zbc(T, S, K, a=A_HW, s=S_HW):
    """Call europeia sobre zero-cupom (Jamshidian, Módulo 17)."""
    PT, PS = P_merc(T), P_merc(S); sp = sigma_P(T, S, a, s)
    h = np.log(PS / (K * PT)) / sp + sp / 2
    return PS * norm.cdf(h) - K * PT * norm.cdf(h - sp)

def zbp(T, S, K, a=A_HW, s=S_HW):
    PT, PS = P_merc(T), P_merc(S); sp = sigma_P(T, S, a, s)
    h = np.log(PS / (K * PT)) / sp + sp / 2
    return K * PT * norm.cdf(-h + sp) - PS * norm.cdf(-h)

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

---

1. Preços de Arrow-Debreu

Material: Ireland, *Financial Economics*, cap. 10; Clewlow e Strickland, §8.2

Um título de Arrow-Debreu ("pure security") paga 1 se, e somente se, um estado específico ocorre numa data específica. Numa árvore, o estado é o nó (i, j). Seu preço hoje, $Q_{i,j}$, é tudo de que precisamos para avaliar qualquer fluxo que dependa só do nó: $V_0 = \sum_j Q_{i,j}\, h_{i,j}$.

Duas propriedades fazem dos Q a ferramenta da calibração:

1. **Somam ao zero-cupom.** Um título que paga 1 em todos os estados da data i é o zero-cupom de prazo $i\Delta t$: $\sum_j Q_{i,j} = P(0, i\Delta t)$.
2. **Se propagam para a frente.** Para chegar ao nó $(i+1, k)$, é preciso estar em algum (i, j), descontar um período e ir de j para k com probabilidade $q(j, k)$:

$$Q_{i+1,k} = \sum_j Q_{i,j}\, q(j,k)\, e^{-r_{i,j}\Delta t}. \tag{18.6}$$

É a versão discreta da equação de Kolmogorov para a frente. A indução para a frente permite escolher o parâmetro de cada data (θ_i ou α_i) de modo que $\sum_k Q_{i+1,k}$ seja exatamente o preço de mercado, uma data por vez.

2. A geometria de uma árvore trinomial

Material: Clewlow e Strickland, §9.2

2.1 Espaçamento

O tempo avança em passos Δt. A taxa (ou a variável auxiliar x) fica numa grade fixa com espaçamento Δx. A escolha padrão,

$$\Delta x = \sigma\sqrt{3\Delta t}, \tag{18.7}$$

vem da analogia com diferenças finitas explícitas (Hull e White, 1993): ela dá a melhor relação entre estabilidade e precisão, e faz a probabilidade do ramo central valer $2/3$ quando não há drift.

2.2 As probabilidades, passo a passo

De um nó, o processo vai para três nós vizinhos, $k+1$, k, $k-1$. Seja η a distância (em unidades de taxa) entre a média condicional e o nó central k. Impomos três condições:

$$p_u + p_m + p_d = 1, \qquad (p_u - p_d)\,\Delta x = \eta, \qquad (p_u + p_d)\,\Delta x^2 = \sigma^2\Delta t + \eta^2. \tag{18.8}$$

A segunda casa a média; a terceira, o segundo momento (variância + média^2). Resolvendo:

$$p_u = \frac{\sigma^2\Delta t + \eta^2}{2\Delta x^2} + \frac{\eta}{2\Delta x}, \qquad p_m = 1 - \frac{\sigma^2\Delta t + \eta^2}{\Delta x^2}, \qquad p_d = \frac{\sigma^2\Delta t + \eta^2}{2\Delta x^2} - \frac{\eta}{2\Delta x}. \tag{18.9}$$

São as fórmulas da planilha (células “pu”, “pm”, “pd”).

2.3 Quando trocar de ramificação

Com reversão à média, a média condicional de x a partir do nó j é $j\Delta x(1+M)$, com $M = -a\Delta t$. Na ramificação normal (para $j{+}1$, j, $j{-}1$), $\eta = jM\Delta x$, e com $\Delta x^2 = 3\sigma^2\Delta t$ as probabilidades viram

$$p_u = \tfrac{1}{6} + \tfrac{1}{2}(j^2M^2 + jM), \qquad p_m = \tfrac{2}{3} - j^2M^2, \qquad p_d = \tfrac{1}{6} + \tfrac{1}{2}(j^2M^2 - jM). \tag{18.10}$$

$p_m > 0$ exige $|jM| < 0{,}816$. Longe do centro, a reversão empurra a média para baixo (ou para cima) mais que um Δx, e p_d (ou p_u) ficaria negativa. A solução de Hull e White é trocar a ramificação: no topo, ir para j, $j-1$, $j-2$; na base, para $j+2$, $j+1$, j. As probabilidades nesses casos são válidas para $0{,}184 < |jM| < 0{,}816$. Hull e White escolhem trocar no primeiro j com $|jM| > 0{,}184$:

$$j_{max} = \left\lceil \frac{0{,}184}{a\,\Delta t} \right\rceil. \tag{18.11}$$

Isso dá à árvore uma largura máxima: ela para de crescer, o que também a deixa eficiente.

Laboratório 1 — As probabilidades em toda a grade

```python
def probabilidades(j, M, jmax):
    """Destino do nó central k e probabilidades (p_u, p_m, p_d) da árvore de Hull-White."""
    jm = j * M
    if j == jmax:
        return j - 1, (7 / 6 + (jm**2 + 3 * jm) / 2, -1 / 3 - jm**2 - 2 * jm, 1 / 6 + (jm**2 + jm)
        ↪  / 2)
    if j == -jmax:
        return j + 1, (1 / 6 + (jm**2 - jm) / 2, -1 / 3 - jm**2 + 2 * jm, 7 / 6 + (jm**2 - 3 * jm)
        ↪  / 2)
    return j, (1 / 6 + (jm**2 + jm) / 2, 2 / 3 - jm**2, 1 / 6 + (jm**2 - jm) / 2)


dt1 = 0.25
M1 = -A_HW * dt1
jmax1 = int(np.ceil(0.184 / (A_HW * dt1)))
dx1 = S_HW * np.sqrt(3 * dt1)
linhas = []
for j in range(-jmax1, jmax1 + 1):
    k, (pu, pm, pdn) = probabilidades(j, M1, jmax1)
    media = (pu * (k + 1) + pm * k + pdn * (k - 1)) * dx1
    var = (pu * (k + 1) ** 2 + pm * k**2 + pdn * (k - 1) ** 2) * dx1**2 - media**2
    linhas.append((j, k, pu, pm, pdn, media / dx1, j * (1 + M1), var / (S_HW**2 * dt1)))
tab1 = pd.DataFrame(linhas, columns=["j", "nó central k", "p_u", "p_m", "p_d", "média/Δx", "alvo
↪  j(1+M)", "variância/(σ²Δt)"])
print(f"Δt = {dt1}, a = {A_HW}: j_max = {jmax1}")
tabela(tab1)
ok((tab1[["p_u", "p_m", "p_d"]] > 0).all().all(), "todas as probabilidades são positivas,
↪  inclusive nas bordas")
ok(np.allclose(tab1["média/Δx"], tab1["alvo j(1+M)"]), "a média condicional é a do processo
↪  discretizado em todos os nós")
ok(np.allclose(tab1["variância/(σ²Δt)"], 1), "e a variância também")
```

```
Δt = 0.25, a = 0.1: j_max = 8
```

j	nó central k	p_u	p_m	p_d	média/Δx	alvo j(1+M)	variância/($\sigma^2\Delta t$)
-8	-7	0,087	0,027	0,887	-7,800	-7,800	1,000
-7	-7	0,269	0,636	0,094	-6,825	-6,825	1,000
-6	-6	0,253	0,644	0,103	-5,850	-5,850	1,000
-5	-5	0,237	0,651	0,112	-4,875	-4,875	1,000
-4	-4	0,222	0,657	0,122	-3,900	-3,900	1,000
-3	-3	0,207	0,661	0,132	-2,925	-2,925	1,000
-2	-2	0,193	0,664	0,143	-1,950	-1,950	1,000
-1	-1	0,179	0,666	0,154	-0,975	-0,975	1,000
0	0	0,167	0,667	0,167	0,000	0,000	1,000
1	1	0,154	0,666	0,179	0,975	0,975	1,000
2	2	0,143	0,664	0,193	1,950	1,950	1,000
3	3	0,132	0,661	0,207	2,925	2,925	1,000
4	4	0,122	0,657	0,222	3,900	3,900	1,000
5	5	0,112	0,651	0,237	4,875	4,875	1,000
6	6	0,103	0,644	0,253	5,850	5,850	1,000
7	7	0,094	0,636	0,269	6,825	6,825	1,000
8	7	0,887	0,027	0,087	7,800	7,800	1,000

```
OK — todas as probabilidades são positivas, inclusive nas bordas
OK — a média condicional é a do processo discretizado em todos os nós
OK — e a variância também
```

---

3. O método de Clewlow e Strickland e a planilha

Material: Clewlow e Strickland, §9.3; planilha, aba "HW yield"

3.1 A construção

A taxa nos nós é $r_{i,j} = r_{0,0} + j\Delta r$, com $r_{0,0} = R(1)$, o yield do prazo Δt. O drift no nó é $\mu_{i,j} = (\theta_i - a\, r_{i,j})\,\Delta t$, e as probabilidades são as da Seção 2.2 com $\eta = \mu_{i,j}$ (ramificação normal) ou $\eta = \mu_{i,j} + (j-k)\Delta r$ (ramificação deslocada).

Como achar θ_i. Com a árvore construída até a data i, escolhe-se θ_i para acertar o título de prazo $(i+2)\Delta t$:

$$P(0,(i+2)\Delta t) = \sum_j Q_{i,j}\, e^{-r_{i,j}\Delta t}\, \mathbb{E}\big[e^{-r_{i+1}\Delta t} \,\big|\, r_{i,j}\big]. \tag{18.12}$$

Aproximando $\mathbb{E}[e^{-r_{i+1}\Delta t}] \approx \exp(-\mathbb{E}[r_{i+1}]\Delta t + \tfrac{1}{2}\sigma^2\Delta t^3)$, com $\mathbb{E}[r_{i+1}] = r_{i,j} + (\theta_i - a\, r_{i,j})\Delta t$, e isolando θ_i:

$$\theta_i = \frac{(i+2)R(i+2)}{\Delta t} + \frac{\sigma^2\Delta t}{2} + \frac{1}{\Delta t^2}\ln\sum_j Q_{i,j}\, e^{-2r_{i,j}\Delta t + a\, r_{i,j}\Delta t^2}. \tag{18.13}$$

É a fórmula das células B39 a D39 da planilha.

Laboratório 2 — Reproduzir a aba "HW yield"

Entradas da planilha: data 03/01/2024; yields de 1 a 4 meses: 5,27%, 5,28%, 5,24% e 5,21%; $\Delta t = 30/360$; $\sigma = 0{,}024638$ e $a = 0{,}0527$ (as mesmas células do Módulo 17).

```python
R_pl = np.array([0.0527, 0.0528, 0.0524, 0.0521])
dt_pl, s_pl, a_pl = 30 / 360, 0.02463782739818595, 0.0527
dr_pl = s_pl * np.sqrt(3 * dt_pl)
P_pl = np.exp(-R_pl * np.arange(1, 5) * dt_pl)


def arvore_cs(R, dt, s, a, n_passos, deslocar_no_menos1=False):
    """Árvore de Clewlow-Strickland §9.3 com ramificação normal, como na planilha.
    deslocar_no_menos1=True reproduz as fórmulas das células D75, D80 e D86 da planilha."""
    dr = s * np.sqrt(3 * dt)
    Q = {0: np.array([1.0])}
    thetas, probs = [], {}
    for i in range(n_passos):
        js = np.arange(-i, i + 1)
        r = R[0] + js * dr
        soma = np.sum(Q[i] * np.exp(-2 * r * dt + a * r * dt**2))
        th = (i + 2) * R[i + 1] / dt + s**2 * dt / 2 + np.log(soma) / dt**2
        thetas.append(th)
        mu = (th - a * r) * dt
        pu = (s**2 * dt + mu**2) / (2 * dr**2) + mu / (2 * dr)
        pm = 1 - (s**2 * dt + mu**2) / dr**2
```

```
        pd_ = 1 - pu - pm
        probs[i] = (pu, pm, pd_)
        Qn = np.zeros(2 * i + 3)
        for idx, j in enumerate(js):
            c = j + i + 1                                   # posição de j na data i+1
            if deslocar_no_menos1 and i == 1 and j == -1:
                c += 1                                      # o erro: os ramos do nó (1,-1) sobem
↪  um nível
            q = Q[i][idx] * np.exp(-r[idx] * dt)
            Qn[c + 1] += q * pu[idx]; Qn[c] += q * pm[idx]; Qn[c - 1] += q * pd_[idx]
        Q[i + 1] = Qn
    return np.array(thetas), Q, probs


th_ok, Q_ok, pr_ok = arvore_cs(R_pl, dt_pl, s_pl, a_pl, 3)
th_pl, Q_pl, _ = arvore_cs(R_pl, dt_pl, s_pl, a_pl, 3, deslocar_no_menos1=True)

planilha_theta = [0.0052025826057877644, -0.012736624924031048, -0.04884807241485989]
planilha_Q1 = [0.15790312019740224, 0.6634773214221333, 0.1742375209805194]          # j = -1, 0,
↪  1
planilha_Q2 = [0.18333158340384287, 0.5758209294059917, 0.2116414931613954, 0.02044460069953802]
↪  # j = -1..2
tab2 = pd.DataFrame({"i": [0, 1, 2], "θ_i planilha (% a.a.)": 100 * np.array(planilha_theta),
                     "θ_i reproduzido, como na planilha (% a.a.)": 100 * th_pl,
                     "θ_i com a ramificação correta (% a.a.)": 100 * th_ok})
tabela(tab2)
tab2b = pd.DataFrame({"j": [-2, -1, 0, 1, 2], "Q_2,j planilha": [0.0] + planilha_Q2,
                      "Q_2,j correto": Q_ok[2]})
tabela(tab2b)
print("Σ Q_2 planilha:", f"{sum(planilha_Q2):.12f}", "| correto:", f"{Q_ok[2].sum():.12f}", "|
↪  P(0, 2Δt):", f"{P_pl[1]:.12f}")
print("Σ Q_3 correto:", f"{Q_ok[3].sum():.12f}", "| P(0, 3Δt):", f"{P_pl[2]:.12f}")

ok(np.allclose(th_pl, planilha_theta, rtol=1e-10), "os três θ_i da planilha são reproduzidos
↪  exatamente")
ok(np.allclose(Q_ok[1], planilha_Q1, rtol=1e-12), "os preços de Arrow-Debreu da data 1 conferem")
ok(np.allclose(Q_pl[2][1:], planilha_Q2, rtol=1e-12), "os da data 2 conferem com a versão
↪  deslocada")
ok(abs(Q_ok[2].sum() - P_pl[1]) < 1e-14 and abs(sum(planilha_Q2) - P_pl[1]) < 1e-14,
   "as duas versões somam exatamente o zero-cupom de 2 meses: a conferência de soma não vê o erro")
ok(abs(th_pl[2] - th_ok[2]) > 0.04, "mas θ_2 muda mais de 4 pontos percentuais por ano")
ok(abs(Q_ok[3].sum() - P_pl[2]) < 1e-11, "com a ramificação correta, a aproximação de θ_i acerta
↪  também o título de 3 meses")
```

i	θ_i planilha (% a.a.)	θ_i reproduzido, como na planilha (% a.a.)	θ_i com a ramificação correta (% a.a.)
0	0,520	0,520	0,520
1	-1,274	-1,274	-1,274
2	-4,885	-4,885	-0,196

j	Q_2,j planilha	Q_2,j correto
-2	0,000	0,035
-1	0,183	0,252
0	0,576	0,492
1	0,212	0,193
2	0,020	0,020

```
Σ Q_2 planilha: 0.991238606671 | correto: 0.991238606671 | P(0, 2Δt): 0.991238606671
Σ Q_3 correto: 0.986985431542 | P(0, 3Δt): 0.986985431542
OK — os três θ_i da planilha são reproduzidos exatamente
```

```
OK — os preços de Arrow-Debreu da data 1 conferem
OK — os da data 2 conferem com a versão deslocada
OK — as duas versões somam exatamente o zero-cupom de 2 meses: a conferência de soma não vê o erro
OK — mas θ_2 muda mais de 4 pontos percentuais por ano
OK — com a ramificação correta, a aproximação de θ_i acerta também o título de 3 meses
```

O que aconteceu. Nas células D75, D80 e D86, os três ramos que saem do nó $(1,-1)$ são somados aos nós $(2,1)$, $(2,0)$ e $(2,-1)$, quando deveriam ir para $(2,0)$, $(2,-1)$ e $(2,-2)$. A massa total não muda, e por isso $\sum_j Q_{2,j}$ continua igual ao preço do zero-cupom: a conferência que a planilha faz (linha "sum(Q)") passa. Mas a distribuição das taxas fica deslocada para cima, e θ_2, que depende de **onde** a massa está, sai $-4{,}9\%$ em vez de $-0{,}2\%$ ao ano.

Lição de validação: uma conferência que só olha totais não detecta erros de posição. Aqui, o teste que pega o erro é conferir a média e a variância condicionais em cada nó (Laboratório 1) ou reprecificar um instrumento que dependa da distribuição, como uma opção.

---

4. O método de dois estágios de Hull e White (1994)

Material: Clewlow e Strickland, §9.5; Hull e White, 1994a

O método que se tornou padrão separa a forma da árvore do ajuste à curva.

Estágio 1 — a árvore de x. x segue o OU sem drift de longo prazo, $dx = -a\,x\,dt + \sigma\,dW$, com $x_0 = 0$. A árvore de x é simétrica, com a geometria e as probabilidades da Seção 2, e não depende da curva.

Estágio 2 — o deslocamento. A taxa do período no nó (m,j) é $r_{m,j} = \alpha_m + j\Delta x$. Com os $Q_{m,j}$ conhecidos, α_m sai do zero-cupom de prazo $(m+1)\Delta t$:

$$P(0,(m+1)\Delta t) = \sum_j Q_{m,j}\, e^{-(\alpha_m + j\Delta x)\Delta t} \quad\Longrightarrow\quad \alpha_m = \frac{1}{\Delta t}\ln\frac{\sum_j Q_{m,j}\, e^{-j\Delta x\Delta t}}{P(0,(m+1)\Delta t)}. \tag{18.14}$$

Sem aproximação: a árvore reproduz a curva **exatamente** em todas as datas da grade. O α_m é a versão discreta do $\varphi(t)$ do Módulo 17.

Laboratório 3 — A árvore de dois estágios

```python
class ArvoreTaxaCurta:
    """Árvore trinomial de Hull-White (1994). modelo='HW': r = α_m + x; modelo='BK': ln r = α_m +
    ↪ x."""

    def __init__(self, a, s, dt, N, P, modelo="HW"):
        self.a, self.s, self.dt, self.N, self.modelo = a, s, dt, N, modelo
        self.dx = s * np.sqrt(3 * dt)
        M = -a * dt
        self.jmax = J = int(np.ceil(0.184 / (a * dt)))
        self.js = np.arange(-J, J + 1)
        self.k = np.zeros(2 * J + 1, dtype=int)
        self.p = np.zeros((2 * J + 1, 3))
        for idx, j in enumerate(self.js):
```

```
            self.k[idx], self.p[idx] = probabilidades(j, M, J)
        self.alpha = np.zeros(N)
        self.Q = []
        Q = np.zeros(2 * J + 1); Q[J] = 1.0
        for m in range(N):
            nm = min(m, J)
            vivos = slice(J - nm, J + nm + 1)
            x = self.js[vivos] * self.dx
            alvo = P((m + 1) * dt)
            if modelo == "HW":
                self.alpha[m] = np.log(np.sum(Q[vivos] * np.exp(-x * dt)) / alvo) / dt
            else:                                              # BK: Newton-Raphson em α_m
                al = np.log(-np.log(alvo) / ((m + 1) * dt)) if m == 0 else self.alpha[m - 1]
                for _ in range(60):
                    rr = np.exp(al + x)
                    g = np.sum(Q[vivos] * np.exp(-rr * dt)) - alvo
                    al -= g / np.sum(-Q[vivos] * np.exp(-rr * dt) * rr * dt)
                    if abs(g) < 1e-15:
                        break
                self.alpha[m] = al
            self.Q.append(Q.copy())
            d = Q * np.exp(-self.taxa(m) * dt)
            Qn = np.zeros_like(Q)
            for idx in range(J - nm, J + nm + 1):
                c = self.k[idx] + J
                Qn[c + 1] += d[idx] * self.p[idx, 0]; Qn[c] += d[idx] * self.p[idx, 1]; Qn[c - 1]
↪  += d[idx] * self.p[idx, 2]
            Q = Qn
        self.Q.append(Q)

    def taxa(self, m):
        x = self.alpha[m] + self.js * self.dx
        return x if self.modelo == "HW" else np.exp(x)

    def voltar(self, V, m):
        """Valor em m (todos os nós) de um vetor de valores V na data m+1."""
        c = self.k + self.jmax
        cont = self.p[:, 0] * V[c + 1] + self.p[:, 1] * V[c] + self.p[:, 2] * V[c - 1]
        return np.exp(-self.taxa(m) * self.dt) * cont

    def zero_cupom(self, m_ini, m_fim):
        V = np.ones(2 * self.jmax + 1)
        for m in range(m_fim - 1, m_ini - 1, -1):
            V = self.voltar(V, m)
        return V

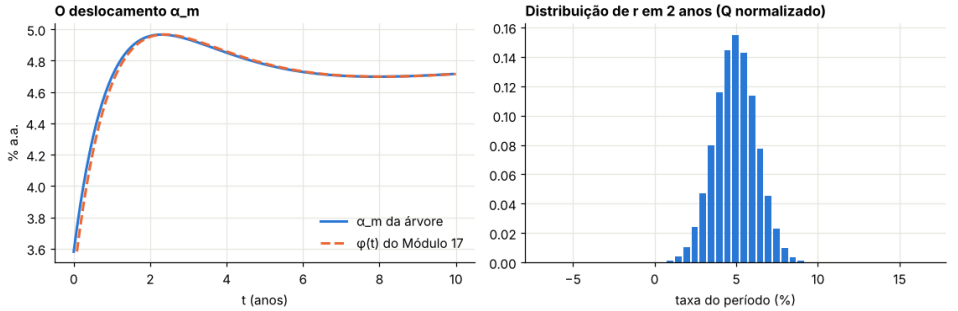

dt3, N3 = 1 / 12, 120
t0 = time.time()
arv = ArvoreTaxaCurta(A_HW, S_HW, dt3, N3, P_merc)
print(f"10 anos, passo mensal: {N3} passos, j_max = {arv.jmax}, construída em {time.time() -
↪  t0:.2f} s")
datas = np.arange(1, N3 + 1) * dt3
erro_curva = np.array([abs(arv.Q[m].sum() - P_merc(m * dt3)) for m in range(1, N3 + 1)])
phi = lambda t: -np.log(P_merc(t + 1e-6) / P_merc(t - 1e-6)) / 2e-6 + S_HW**2 / (2 * A_HW**2) *
↪  (1 - np.exp(-A_HW * t)) ** 2

fig, axs = plt.subplots(1, 2, figsize=(10.5, 3.6))
axs[0].plot(np.arange(N3) * dt3, 100 * arv.alpha, color=AZUL, label="α_m da árvore")
axs[0].plot(datas, 100 * phi(datas - dt3 / 2), "--", color=LARANJA, label="φ(t) do Módulo 17")
```

```
axs[0].set_xlabel("t (anos)"); axs[0].set_ylabel("% a.a."); axs[0].set_title("O deslocamento
↪  α_m"); axs[0].legend()
mm = 24
axs[1].bar(arv.taxa(mm) * 100, arv.Q[mm] / arv.Q[mm].sum(), width=arv.dx * 80, color=RAMPA[1])
axs[1].set_xlabel("taxa do período (%)"); axs[1].set_title("Distribuição de r em 2 anos (Q
↪  normalizado)")
plt.tight_layout(); plt.show()

ok(erro_curva.max() < 1e-14, "a árvore reprecifica todos os 120 zero-cupons exatamente")
ok(np.abs(arv.alpha[1:] - phi(np.arange(1, N3) * dt3 + dt3 / 2)).max() < 5e-4, "α_m acompanha o
↪  φ(t) contínuo a menos de 5 pb")
ok(arv.jmax == int(np.ceil(0.184 / (A_HW * dt3))), "a largura máxima é a de Hull-White")
```

```
10 anos, passo mensal: 120 passos, j_max = 23, construída em 0.04 s
```




```
OK — a árvore reprecifica todos os 120 zero-cupons exatamente
OK — α_m acompanha o φ(t) contínuo a menos de 5 pb
OK — a largura máxima é a de Hull-White
```

---

5. Avaliar na árvore

Material: Clewlow e Strickland, §9.9; Aula 3 (bond options, callable e puttable bonds)

Com a árvore pronta, avaliar é indução para trás: o valor num nó é o valor esperado nos três sucessores, descontado pela taxa do nó. Para exercício antecipado, compara-se o valor de continuação com o de exercício em cada data permitida.

5.1 Convergência para a fórmula fechada

Call europeia com vencimento em 1 ano sobre o zero-cupom de 5 anos, strike no dinheiro a termo (como no Laboratório 4 do Módulo 17).

```
T5, S5 = 1.0, 5.0
K5 = P_merc(S5) / P_merc(T5)
ref5 = zbc(T5, S5, K5)
linhas = []
for por_ano in [4, 12, 24, 52, 100, 200]:
    dt = 1 / por_ano
    N = int(round(S5 / dt)); nT = int(round(T5 / dt))
```

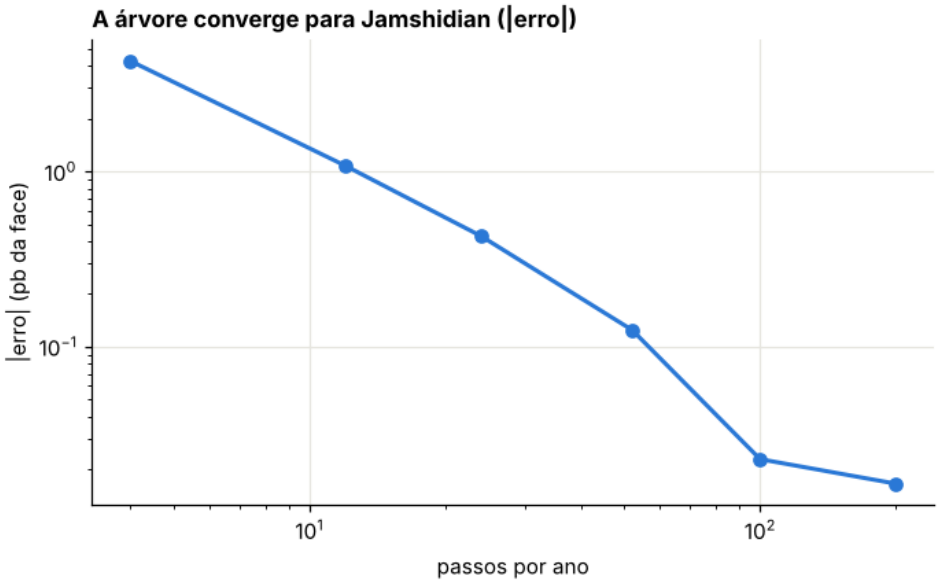
```
    t0 = time.time()
    arv5 = ArvoreTaxaCurta(A_HW, S_HW, dt, N, P_merc)
    PS = arv5.zero_cupom(nT, N)
    payoff = np.maximum(PS - K5, 0)
    V = payoff.copy()
    for m in range(nT - 1, -1, -1):
        V = arv5.voltar(V, m)
    via_AD = np.sum(arv5.Q[nT] * payoff)
    linhas.append((por_ano, arv5.jmax, 1e4 * V[arv5.jmax], 1e4 * via_AD, 1e4 * (V[arv5.jmax] -
↪   ref5), time.time() - t0))
tab5 = pd.DataFrame(linhas, columns=["passos por ano", "j_max", "árvore (pb)", "Σ Q·payoff (pb)",
↪   "erro (pb)", "tempo (s)"])
tabela(tab5)
print("Jamshidian:", f3(1e4 * ref5), "pb")
sel = tab5["passos por ano"] <= 100
incl = np.polyfit(np.log(tab5.loc[sel, "passos por ano"]), np.log(np.abs(tab5.loc[sel, "erro
↪   (pb)"])), 1)[0]
print("inclinação log-log do erro:", f3(incl))

fig, ax = plt.subplots()
ax.loglog(tab5["passos por ano"], np.abs(tab5["erro (pb)"]), "o-", color=AZUL)
ax.set_xlabel("passos por ano"); ax.set_ylabel("|erro| (pb da face)"); ax.set_title("A árvore
↪   converge para Jamshidian (|erro|)")
plt.show()

ok(np.allclose(tab5["árvore (pb)"], tab5["Σ Q·payoff (pb)"], rtol=1e-10), "indução para trás e
↪   soma dos preços de Arrow-Debreu dão o mesmo valor")
ok(abs(tab5["erro (pb)"].iloc[-1]) < 0.05, "com 200 passos por ano, o erro fica abaixo de 0,05
↪   pb")
ok(incl < -0.9, "até 100 passos por ano, o erro cai pelo menos na proporção de 1/N")
ok(tab5["erro (pb)"].iloc[-1] * tab5["erro (pb)"].iloc[-2] < 0, "e perto de 200 passos muda de
↪   sinal: a convergência não é monótona")
```

passos por ano	j_max	árvore (pb)	Σ Q·payoff (pb)	erro (pb)	tempo (s)
4	8	94,609	94,609	-4,288	0,001
12	23	97,816	97,816	-1,081	0,024
24	45	98,468	98,468	-0,429	0,058
52	96	98,772	98,772	-0,125	0,265
100	184	98,874	98,874	-0,023	0,849
200	368	98,913	98,913	0,017	3,678

```
Jamshidian: 98,897 pb
inclinação log-log do erro: -1,579
```




```
OK — indução para trás e soma dos preços de Arrow-Debreu dão o mesmo valor
OK — com 200 passos por ano, o erro fica abaixo de 0,05 pb
OK — até 100 passos por ano, o erro cai pelo menos na proporção de 1/N
OK — e perto de 200 passos muda de sinal: a convergência não é monótona
```

O erro cai depressa (mais rápido que $1/N$ no começo) e troca de sinal perto de 200 passos por ano: como na árvore CRR (Módulo 6), a convergência não é monótona, porque a posição do strike em relação aos nós muda com N. Na prática, compara-se o resultado de duas ou três malhas, e não de uma só.

5.2 Opções americanas e títulos com opções embutidas

Três produtos sobre a mesma árvore (passo semanal):

- **put americana** sobre o zero-cupom de 5 anos, exercível até 1 ano, com strike igual ao preço à vista do título, $P(0,5)$ (um strike no dinheiro *a termo* ficaria acima do preço à vista, e o exercício imediato seria ótimo);
- **título com opção de recompra** (*callable*): cupom anual de 5%, vencimento em 5 anos, o emissor pode resgatar ao par em cada data de cupom a partir do ano 2;
- **título com opção de venda** (*puttable*): o mesmo, mas quem pode devolver ao par é o investidor.

Pela Aula 3: a opção de recompra é do emissor, por isso o *callable* vale **menos** que o título simples (o investidor vende a call); o *puttable* vale **mais** (o investidor compra a put).

```
dt6 = 1 / 52
N6 = int(round(5 / dt6))
arv6 = ArvoreTaxaCurta(A_HW, S_HW, dt6, N6, P_merc)
J6 = arv6.jmax
idx = lambda t: int(round(t / dt6))
```

```
# put americana sobre o zero-cupom
nT = idx(1.0)
PS = [None] * (N6 + 1)
V = np.ones(2 * J6 + 1)
for m in range(N6 - 1, -1, -1):
    V = arv6.voltar(V, m)
    PS[m] = V
K_am = P_merc(5.0)                        # strike igual ao preço à vista do título
put_eur = np.maximum(K_am - PS[nT], 0); put_am = put_eur.copy()
for m in range(nT - 1, -1, -1):
    put_eur = arv6.voltar(put_eur, m)
    put_am = np.maximum(arv6.voltar(put_am, m), K_am - PS[m])
premio_am = put_am[J6] - put_eur[J6]
print("put europeia:", f3(1e4 * put_eur[J6]), "pb | americana:", f3(1e4 * put_am[J6]), "pb |
↪  Jamshidian:", f3(1e4 * zbp(1.0, 5.0, K_am)), "pb")


# títulos com cupom
def titulo(opcao=None, cupom=0.05, datas_opcao=(2, 3, 4)):
    datas_cupom = {idx(t): t for t in (1, 2, 3, 4, 5)}
    V = np.full(2 * J6 + 1, 1.0 + cupom)
    for m in range(N6 - 1, -1, -1):
        V = arv6.voltar(V, m)
        if m in datas_cupom and m > 0:
            if opcao == "call" and datas_cupom[m] in datas_opcao:
                V = np.minimum(V, 1.0)          # o emissor resgata ao par (ex-cupom)
            if opcao == "put" and datas_cupom[m] in datas_opcao:
                V = np.maximum(V, 1.0)          # o investidor devolve ao par
            V = V + cupom
    return V[J6]


simples = titulo()
fechado = sum(0.05 * P_merc(t) for t in (1, 2, 3, 4, 5)) + P_merc(5.0)
callable_, puttable = titulo("call"), titulo("put")
tab6 = pd.DataFrame({"título": ["simples (fórmula: Σ c·P + P)", "simples (árvore)", "callable",
↪  "puttable"],
                     "preço (% da face)": [100 * fechado, 100 * simples, 100 * callable_, 100 *
                     ↪  puttable]})
tabela(tab6)
print("call do emissor:", f3(100 * (simples - callable_)), "% | put do investidor:", f3(100 *
↪  (puttable - simples)), "%")

ok(abs(simples - fechado) < 1e-12, "o título simples na árvore é exatamente o da curva")
ok(premio_am > 0 and abs(put_eur[J6] - zbp(1.0, 5.0, K_am)) < 2e-5, "a put europeia confere com
↪  Jamshidian e a americana vale mais")
ok(callable_ < simples < puttable, "callable < simples < puttable, como na Aula 3")
```

```
put europeia: 10,225 pb | americana: 46,305 pb | Jamshidian: 10,163 pb
```

título	preço (% da face)
simples (fórmula: Σ c·P + P)	100,786
simples (árvore)	100,786
callable	99,385
puttable	102,080

```
call do emissor: 1,401 % | put do investidor: 1,294 %
OK — o título simples na árvore é exatamente o da curva
OK — a put europeia confere com Jamshidian e a americana vale mais
OK — callable < simples < puttable, como na Aula 3
```

5.3 Caps na árvore

O caplet como put sobre zero-cupom (Módulo 17, Seção 5.1) é avaliado igual: na data de fixação, o payoff é $(1+\delta R_K)\max\left(\frac{1}{1+\delta R_K} - P(t_k,t_{k+1}),0\right)$ em cada nó.

```
dt7, tk, delta = 1 / 48, 2.0, 0.25
N7 = int(round((tk + delta) / dt7)); nk = int(round(tk / dt7))
arv7 = ArvoreTaxaCurta(A_HW, S_HW, dt7, N7, P_merc)
PB7 = arv7.zero_cupom(nk, N7)
F7 = (P_merc(tk) / P_merc(tk + delta) - 1) / delta


def caplet_arvore(arv, PB, RK):
    Kb = 1 / (1 + delta * RK)
    V = (1 + delta * RK) * np.maximum(Kb - PB, 0)
    for m in range(nk - 1, -1, -1):
        V = arv.voltar(V, m)
    return V[arv.jmax]


def caplet_black(RK, vol):
    d1 = (np.log(F7 / RK) + 0.5 * vol**2 * tk) / (vol * np.sqrt(tk))
    return delta * P_merc(tk + delta) * (F7 * norm.cdf(d1) - RK * norm.cdf(d1 - vol * np.sqrt(tk)))


taxas_teto = F7 + np.array([-0.015, -0.0075, 0.0, 0.0075, 0.015])
c_arv = np.array([caplet_arvore(arv7, PB7, k) for k in taxas_teto])
c_fec = np.array([(1 + delta * k) * zbp(tk, tk + delta, 1 / (1 + delta * k)) for k in taxas_teto])
tab7 = pd.DataFrame({"taxa-teto (%)": 100 * taxas_teto, "árvore (pb)": 1e4 * c_arv, "fórmula
↪  (pb)": 1e4 * c_fec})
tabela(tab7)
ok(np.abs(c_arv - c_fec).max() < 0.05e-4, "os caplets da árvore conferem com a fórmula a menos de
↪  0,05 pb")
```

taxa-teto (%)	árvore (pb)	fórmula (pb)
3,475	35,557	35,556
4,225	21,919	21,919
4,975	11,553	11,552
5,725	5,022	5,020
6,475	1,746	1,743

```
OK — os caplets da árvore conferem com a fórmula a menos de 0,05 pb
```

---

6. Ajustar também as volatilidades

Material: Clewlow e Strickland, §9.8; planilha, aba “HW yield and vol”; Aula 5

6.1 A ideia

Com a e σ constantes, o Hull-White ajusta a curva de juros, mas a estrutura a termo das volatilidades das taxas sai do modelo. Na data zero, a volatilidade normal do yield de prazo T é

$$\sigma_R(T) = \sigma \frac{B(0,T)}{T} = \sigma \frac{1 - e^{-aT}}{aT}, \tag{18.15}$$

que **cai** com o prazo quando $a > 0$. Clewlow e Strickland (§9.8) permitem a e θ dependentes do tempo, a_i e θ_i, escolhidos a cada passo para acertar o yield **e** a vol do yield de cada prazo. É o que faz a segunda aba da planilha.

6.2 O que a planilha obtém e por quê

A aba encontra $a_1 = -6{,}31$ e $a_2 = -1{,}60$: reversão **negativa**, ou seja, taxas que se afastam da média. A razão está na fórmula acima: para $\sigma_R(T)$ **crescer** com o prazo, é preciso $a < 0$. E as vols que entram na aba crescem com o prazo (de 2,1% a 3,1%, de 2 a 4 meses) por causa da anualização vista no Módulo 17 (Seção 6.2). Com as vols medidas em unidades de taxa e $\sqrt{252}$, a estrutura cai com o prazo, e o sinal de a se inverte.

Laboratório 4 — O sinal de a depende da conta da volatilidade

Ajustamos $\sigma\,(1 - e^{-aT})/(aT)$, com a e σ constantes, às duas estruturas de vols do Módulo 17.

```
prazos = np.array([1, 2, 3, 4]) / 12
vols_planilha = np.array([0.02463782739818595, 0.021359591580382732, 0.02717743310261939,
↪  0.03061581517918258])
vols_corretas = np.array([0.018739, 0.008153, 0.006940, 0.005888])    # Módulo 17, Laboratório 6


def sigma_R(T, a, s):
    return s * np.where(abs(a) < 1e-9, 1.0, (1 - np.exp(-a * T)) / (a * T))


def ajusta(vols):
    from scipy.optimize import least_squares
    res = least_squares(lambda p: sigma_R(prazos, *p) - vols, x0=[0.1, vols[0]], bounds=([-60,
↪  1e-5], [60, 1.0]))
    return res.x

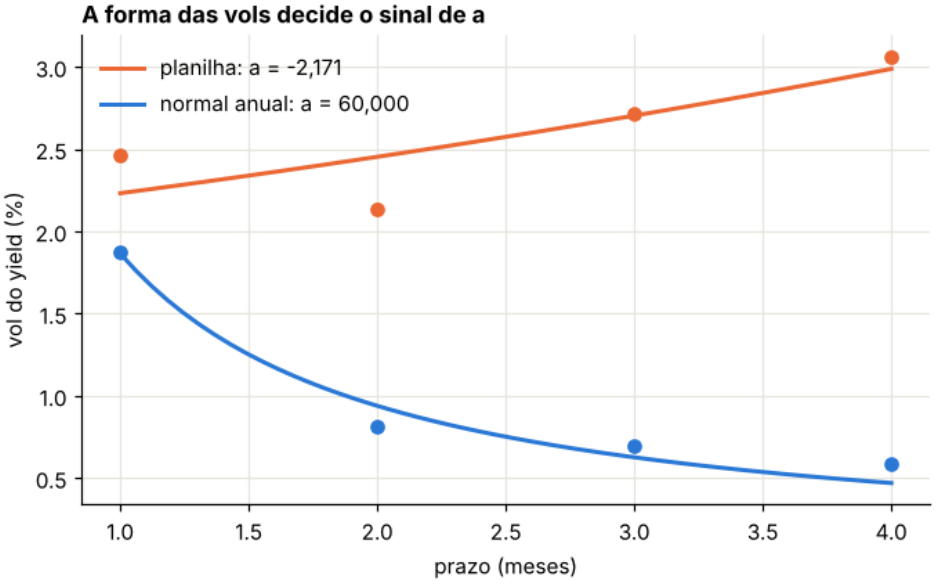

a_pl_fit, s_pl_fit = ajusta(vols_planilha)
a_ok_fit, s_ok_fit = ajusta(vols_corretas)
tab8 = pd.DataFrame({"estrutura de vols": ["planilha (×252, sem escala de prazo)", "normal anual
↪  (×√252)"],
                     "a ajustado": [a_pl_fit, a_ok_fit], "σ ajustado (%)": [100 * s_pl_fit, 100 *
                     ↪  s_ok_fit],
                     "1/a (meses)": [12 / a_pl_fit, 12 / a_ok_fit]})
tabela(tab8)

fig, ax = plt.subplots()
Tf = np.linspace(1 / 12, 1 / 3, 50)
for v, (a_, s_), cor, nome in [(vols_planilha, (a_pl_fit, s_pl_fit), LARANJA, "planilha"),
↪  (vols_corretas, (a_ok_fit, s_ok_fit), AZUL, "normal anual")]:
```

```
    ax.plot(12 * prazos, 100 * v, "o", color=cor)
    ax.plot(12 * Tf, 100 * sigma_R(Tf, a_, s_), color=cor, label=f"{nome}: a = {f3(a_)}")
ax.set_xlabel("prazo (meses)"); ax.set_ylabel("vol do yield (%)"); ax.set_title("A forma das vols
↪  decide o sinal de a"); ax.legend()
plt.show()

ok(a_pl_fit < 0, "com as vols da planilha, o ajuste pede reversão negativa (como a aba encontra)")
ok(a_ok_fit > 0, "com as vols em unidades de taxa, a reversão é positiva")
```

estrutura de vols	a ajustado	σ ajustado (%)	1/a (meses)
planilha ($\times 252$, sem escala de prazo)	-2,171	2,036	-5,527
normal anual ($\times \sqrt{252}$)	60,000	9,387	0,200




```
OK — com as vols da planilha, o ajuste pede reversão negativa (como a aba encontra)
OK — com as vols em unidades de taxa, a reversão é positiva
```

Mesmo com as vols corretas, o ajuste vai para o limite superior permitido de a ($1/a$ de poucos dias): a vol de 1 mês é mais que o dobro da de 2 meses, e um modelo de um fator com reversão constante só reproduz isso esquecendo os choques quase de imediato. Em 2023, a vol do T-bill de 1 mês carrega movimentos próprios desse prazo, que um fator não descreve.

Cautela de Hull e White (1994). Ajustar a_i e σ_i dependentes do tempo à vol de hoje faz a estrutura de vols **futura** ser muito diferente da atual: o modelo fica não estacionário, e o preço de opções que começam no futuro (opções sobre caps, swaptions *forward*) fica distorcido. A recomendação prática é manter a e σ constantes (ou σ por trechos), calibrados a opções.

7. Black-Karasinski na mesma árvore

Material: Black e Karasinski (1991); Itkin, Lipton e Muravey (2021); Silva (2010, UFRGS)

7.1 O modelo

$$d\ln r = (\theta(t) - a\ln r)\,dt + \sigma\,dW. \tag{18.16}$$

A taxa é lognormal: sempre positiva, com volatilidade proporcional ao nível. Não há fórmula fechada para títulos nem para opções, e por isso a árvore é o método natural. A construção é a mesma: árvore de x no estágio 1, e $\ln r_{m,j} = \alpha_m + j\Delta x$ no estágio 2. A diferença é que α_m não sai mais em forma fechada: resolvemos

$$\sum_j Q_{m,j}\exp\big(-e^{\alpha_m + j\Delta x}\Delta t\big) = P(0,(m+1)\Delta t) \tag{18.17}$$

por Newton-Raphson, partindo do α_{m-1} (o método está na classe `ArvoreTaxaCurta`, com `modelo="BK"`).

Itkin, Lipton e Muravey (2021) lembram o outro lado: num ambiente de juros muito baixos ou negativos, o BK não consegue representar taxas negativas, e o pequeno deslocamento necessário muda a dinâmica; eles propõem uma modificação do tipo Verhulst. A dissertação de Silva (UFRGS, 2010) calibra uma versão generalizada do BK a títulos de desconto brasileiros.

Laboratório 5 — HW e BK com o mesmo caplet no dinheiro

Calibramos o σ do BK para que o caplet no dinheiro de 2 anos tenha o mesmo preço que no HW, e comparamos as vols de Black ao longo dos strikes e o piso das taxas.

```
def caplets_modelo(arv):
    PB = arv.zero_cupom(nk, N7)
    return np.array([caplet_arvore(arv, PB, k) for k in taxas_teto])


def vol_black(preco, k):
    return brentq(lambda v: caplet_black(k, v) - preco, 1e-4, 3.0)


atm_hw = c_arv[2]
s_bk = brentq(lambda s: caplets_modelo(ArvoreTaxaCurta(A_HW, s, dt7, N7, P_merc, "BK"))[2] -
 ↪  atm_hw, 0.05, 0.6)
arv_bk = ArvoreTaxaCurta(A_HW, s_bk, dt7, N7, P_merc, "BK")
c_bk = caplets_modelo(arv_bk)
tab9 = pd.DataFrame({"taxa-teto (%)": 100 * taxas_teto,
                     "vol Black HW (%)": [100 * vol_black(c, k) for c, k in zip(c_arv, taxas_teto)],
                     "vol Black BK (%)": [100 * vol_black(c, k) for c, k in zip(c_bk, taxas_teto)]})
tabela(tab9)
print("σ do BK (lognormal):", f3(100 * s_bk), "% | erro de ajuste da curva no BK:",
 ↪  f"{abs(arv_bk.Q[N7].sum() - P_merc(N7 * dt7)):.1e}")
print("menor taxa nos nós da data 2 anos — HW:", f3(100 * arv7.taxa(nk).min()), "% | BK:", f3(100
 ↪  * arv_bk.taxa(nk).min()), "%")

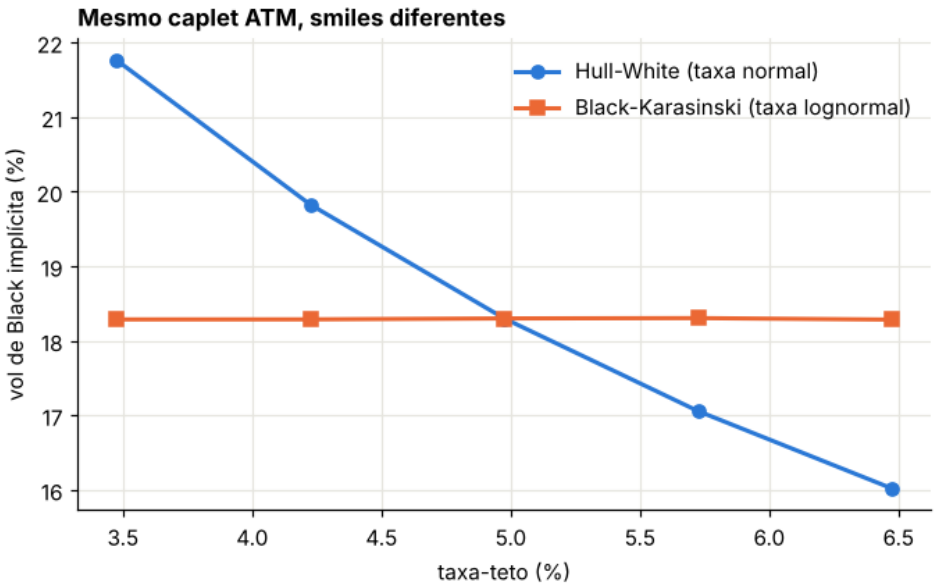
fig, ax = plt.subplots()
ax.plot(tab9["taxa-teto (%)"], tab9["vol Black HW (%)"], "o-", color=AZUL, label="Hull-White
 ↪  (taxa normal)")
```

```
ax.plot(tab9["taxa-teto (%)"], tab9["vol Black BK (%)"], "s-", color=LARANJA,
↪  label="Black-Karasinski (taxa lognormal)")
ax.set_xlabel("taxa-teto (%)"); ax.set_ylabel("vol de Black implícita (%)"); ax.set_title("Mesmo
↪  caplet ATM, smiles diferentes")
ax.legend(); plt.show()

ok(abs(arv_bk.Q[N7].sum() - P_merc(N7 * dt7)) < 1e-14, "o BK, com Newton em cada data, também
↪  reprecifica a curva exatamente")
ok(np.ptp(tab9["vol Black BK (%)"]) < 0.2 * np.ptp(tab9["vol Black HW (%)"]), "o BK produz vols
↪  de Black quase planas; o HW, um skew")
ok(arv_bk.taxa(nk).min() > 0 > arv7.taxa(nk).min(), "o BK não tem taxas negativas; o HW tem")
```

taxa-teto (%)	vol Black HW (%)	vol Black BK (%)
3,475	21,765	18,287
4,225	19,827	18,287
4,975	18,300	18,300
5,725	17,057	18,304
6,475	16,020	18,284

```
σ do BK (lognormal): 20,242 % | erro de ajuste da curva no BK: 1.1e-16
menor taxa nos nós da data 2 anos — HW: -17,294 % | BK: 0,053 %
```




```
OK — o BK, com Newton em cada data, também reprecifica a curva exatamente
OK — o BK produz vols de Black quase planas; o HW, um skew
OK — o BK não tem taxas negativas; o HW tem
```

O que aprendemos. A escolha entre HW e BK é uma escolha sobre o smile de vol. O HW, normal, dá vols de Black decrescentes no strike e admite juros negativos; o BK, lognormal, dá vols de Black quase planas e juros sempre positivos. Qual ajusta melhor depende do mercado e da época: com juros altos, o skew observado costuma ficar entre os dois.

8. Síntese

Etapa	O que fazer	Laboratório
Geometria	$\Delta x = \sigma\sqrt{3\Delta t}$, probabilidades por momentos, troca em $j_{max} = \lceil 0{,}184/(a\Delta t)\rceil$	1
Ajuste à curva	indução para a frente com Arrow-Debreu: θ_i aproximado (C&S) ou α_m exato (HW 1994)	2 e 3
Conferência	soma dos Q confere preços, não posições; conferir momentos nó a nó e reprecificar opções	2
Avaliação	indução para trás; exercício antecipado por comparação nó a nó	5.1 a 5.3
Volatilidades	ajustar vols com a_i variável é instável; a forma das vols decide o sinal de a	4
Outro modelo	BK na mesma árvore, α_m por Newton	5

9. Notas sobre a planilha HW_arvore_final

Onde	Observação
"HW yield", células D75, D80 e D86	os três ramos do nó $(1,-1)$ são somados um nível acima; a soma dos Q não muda (a conferência "sum(Q)" passa), mas θ_2 sai $-4{,}88\%$ em vez de $-0{,}20\%$ ao ano
"HW yield"	ramificação normal em todos os nós; adequado para poucos passos, mas para árvores longas é preciso a troca de ramificação da Seção 2.3
"HW yield and vol"	as vols de entrada crescem com o prazo por causa da anualização (Módulo 17), o que leva a reversão negativa ($a_1 = -6{,}31$; $a_2 = -1{,}60$)
Ambas	a igual à taxa de 1 mês (Módulo 17)

10. Exercícios

Exercício 1. Mostre que, com $j = 0$ e $\Delta x = \sigma\sqrt{3\Delta t}$, as probabilidades são $(1/6, 2/3, 1/6)$, e que a curtose da distribuição de um passo é 3, como a da normal.

Solução

Com $\eta = 0$: $p_u = p_d = \sigma^2\Delta t/(2\Delta x^2) = 1/6$, $p_m = 2/3$. O quarto momento é $2 \cdot \frac{1}{6}\Delta x^4 = \frac{1}{3}\Delta x^4 = 3\sigma^4\Delta t^2$, e o quadrado da variância é $\sigma^4\Delta t^2$: curtose 3. É por isso que $\sqrt{3}$ é a escolha natural.

Exercício 2. Construa a árvore de dois estágios com os dados da planilha (yields de 1 a 4 meses, $\Delta t = 1/12$). Compare os α_m com os $r_{0,0}$ e θ_i da Seção 3.

Solução

Use `ArvoreTaxaCurta(a, s, 1/12, 4, P)`, com `P` interpolando $e^{-R(i)\,i\Delta t}$ nas datas da grade. O α_0 é exatamente $R(1)$. Os demais diferem dos de C&S porque lá a árvore é centrada em $r_{0,0}$ e o drift vai para as probabilidades; aqui o drift vai para o deslocamento α_m.

Exercício 3. Calcule o delta do título com opção de recompra em relação a um deslocamento paralelo da curva (reconstrua a árvore com $P(0,t)e^{-\epsilon t}$) e compare com o do título simples. Por que o *callable* tem duração menor?

Solução

Quando os juros caem, o título simples sobe sem limite, mas o *callable* fica preso perto do par, porque o emissor resgata. A sensibilidade a juros (duração efetiva) do *callable* é menor, e pode ficar muito pequena quando o título está perto do preço de recompra: é a convexidade negativa.

Exercício 4. No Laboratório 5, encontre um σ do HW e um do BK que reproduzam ao mesmo tempo os caplets de 2 anos com taxa-teto 1,5 ponto abaixo e 1,5 ponto acima do dinheiro. É possível? O que isso sugere?

Solução

Com um único parâmetro de vol, cada modelo tem um skew de smile fixo: não dá para acertar dois strikes com skew diferente do do modelo. Isso motiva modelos com parâmetro de forma, como o CEV na short rate ($dr = \cdots + \sigma r^{\beta} dW$, §9.4 de Clewlow e Strickland), em que β interpola entre o normal ($\beta = 0$) e o lognormal ($\beta = 1$).

```
print(f"Módulo 18 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 18 concluído: 25 verificações aprovadas.
```

Capítulo 19

Simulação de trajetórias de juros: do Vasicek ao HJM

Introdução

Árvores e fórmulas fechadas precificam; a **simulação de trajetórias de juros** é indispensável quando o contrato depende de toda a trajetória da curva, ou quando se quer cenários para gestão de risco e ALM. O problema é que modelos diferentes, todos ajustados à mesma curva de hoje, geram trajetórias futuras muito diferentes.

O capítulo percorre uma escada de modelos, do mais simples ao mais rico: Vasicek, Ho-Lee, Hull-White, CIR com extensão determinística (CIR++), Black-Karasinski, o modelo de dois fatores G2++ e o arcabouço **HJM**, que modela diretamente a curva forward inteira. Todos são calibrados à mesma curva e ao mesmo caplet, de modo que as diferenças que restam vêm apenas da estrutura de cada modelo.

Uma técnica central é a **calibração por trajetórias**: ajustar a deriva data a data para que as próprias trajetórias simuladas reproduzam a curva, sem erro de Monte Carlo nos títulos. O capítulo compara o que cada modelo implica para taxas negativas, para a distribuição das taxas futuras e para a correlação entre taxas curtas e longas.

Fundamentos teóricos

Uma escada de modelos de short rate

Definição 19.1 (Modelos de um fator). Sob $\mathbb{Q}$:

$$\text{Vasicek:}\quad dr = a(b-r)\,dt + \sigma\,dW, \qquad \text{Ho-Lee:}\quad dr = \vartheta(t)\,dt + \sigma\,dW,$$

$$\text{Hull-White:}\quad dr = (\vartheta(t) - ar)\,dt + \sigma\,dW, \qquad \text{CIR:}\quad dr = a(b-r)\,dt + \sigma\sqrt{r}\,dW,$$

$$\text{Black-Karasinski:}\quad d\ln r = (\vartheta(t) - a\ln r)\,dt + \sigma\,dW.$$

Os modelos com um parâmetro dependente do tempo reproduzem exatamente a curva inicial; os de parâmetros constantes (Vasicek, CIR) não. A extensão determinística resolve isso para qualquer modelo.

Proposição 19.2 (Extensão determinística, CIR++). *Se x segue um modelo de short*

rate com preços de título $P^x(0,T)$, então $r_t = x_t + \varphi(t)$, com

$$\varphi(t) = f^{\mathrm{mkt}}(0,t) - f^x(0,t), \tag{19.1}$$

reproduz $P^{\mathrm{mkt}}(0,T)$ para todo T e preserva a tratabilidade de x (Brigo and Mercurio, 2006).

Demonstração. $P(0,T) = \mathbb{E}\big[e^{-\int_0^T (x_s+\varphi(s))ds}\big] = e^{-\int_0^T \varphi} P^x(0,T)$. Com a escolha de φ, $e^{-\int_0^T \varphi} = P^{\mathrm{mkt}}(0,T)/P^x(0,T)$. ∎

Calibração por trajetórias

Proposição 19.3 (Bootstrapping da deriva). *Simule I trajetórias de x e escreva $r = g(x+\alpha)$, com g crescente ($g(y) = y$ nos modelos gaussianos, $g(y) = e^y$ em Black-Karasinski). Data a data, o deslocamento α_{k+1} é a raiz de*

$$\frac{1}{I}\sum_{i=1}^{I} D_{k,i} \exp\Big(-\tfrac{\Delta t}{2}\big[r_{k,i} + g(\alpha_{k+1} + x_{k+1,i})\big]\Big) = P^{\mathrm{mkt}}(0, t_{k+1}), \tag{19.2}$$

onde $D_{k,i}$ é o fator de desconto acumulado da trajetória até t_k. O lado esquerdo é monótono em α_{k+1}, e a raiz é única.

O procedimento reproduz a curva *nas trajetórias simuladas*, sem erro de Monte Carlo nos títulos. Ele é o análogo, por simulação, da indução de Arrow-Debreu do Capítulo 18, e funciona para qualquer modelo que se possa simular.

O arcabouço HJM

Definição 19.4 (Modelo HJM). Heath, Jarrow e Morton modelam diretamente a curva forward (Heath et al., 1992):

$$df(t,T) = \mu(t,T)\,dt + \sum_{\ell=1}^{d} \sigma_\ell(t,T)\,dW_t^\ell, \qquad f(0,T) = f^{\mathrm{mkt}}(0,T). \tag{19.3}$$

Teorema 19.5 (Condição de deriva de HJM). *Os preços $P(t,T) = \exp\big(-\int_t^T f(t,u)\,du\big)$ descontados pela conta remunerada são $\mathbb{Q}$-martingais se, e somente se,*

$$\mu(t,T) = \sum_{\ell=1}^{d} \sigma_\ell(t,T) \int_t^T \sigma_\ell(t,u)\,du. \tag{19.4}$$

Esboço. Por Itô, $d\ln P(t,T) = \big(r_t - \int_t^T \mu(t,u)du\big)dt - \sum_\ell \Sigma_\ell(t,T)\,dW_t^\ell$, com $\Sigma_\ell(t,T) = \int_t^T \sigma_\ell(t,u)\,du$. O preço descontado é martingal se e somente se sua deriva se anula: $\int_t^T \mu(t,u)du = \frac{1}{2}\sum_\ell \Sigma_\ell(t,T)^2$. Derivando em T obtém-se (19.4). ∎

A consequência é notável: sob $\mathbb{Q}$, a deriva da curva forward é determinada pelas volatilidades. Calibrar um modelo HJM é escolher volatilidades; a curva inicial entra como condição inicial, e o ajuste a ela é automático. Hull-White é o caso $d = 1$, $\sigma(t,T) = \sigma e^{-a(T-t)}$.

Observação 19.6 (Parametrização de Musiela e componentes principais). Na prática simula-se a curva em prazos fixos, $\tilde{f}(t,\tau) = f(t,t+\tau)$, cuja dinâmica ganha o termo $\partial_\tau \tilde{f}\,dt$. As volatilidades podem ser estimadas por componentes principais das variações históricas da curva: os três primeiros fatores (nível, inclinação e curvatura) explicam quase toda a variância.

Dois fatores

Definição 19.7 (G2++). $r_t = x_t + y_t + \varphi(t)$, com $dx = -ax\,dt + \sigma\,dW^1$, $dy = -by\,dt + \eta\,dW^2$, $d\langle W^1, W^2\rangle = \rho\,dt$ (Brigo and Mercurio, 2006).

Em modelos de um fator, as variações de todas as taxas são perfeitamente correlacionadas. Com dois fatores e $\rho < 0$, a correlação entre short rates e longas fica abaixo de 1, o que é observado empiricamente e importa para produtos que dependem de inclinação da curva.

Roteiro do capítulo

Curso: Derivativos com Python. Terceiro módulo do complemento de juros, depois dos Módulos 17 e 18. Material de base: Wilmott, *Paul Wilmott Introduces Quantitative Finance* (Aulas 4, 5 e 8 do seu curso); Brigo e Mercurio, *Interest Rate Models — Theory and Practice*, caps. 3 e 4; Heath, Jarrow e Morton (1992); Black e Karasinski (1991).

O que você vai conseguir fazer ao final

1. Dizer o que significa "calibrar trajetórias": a curva de hoje, uma volatilidade de referência e a medida certa.
2. Simular seis modelos em ordem de complexidade: Vasicek, Ho-Lee, Hull-White, CIR++, Black-Karasinski, G2++ e HJM multifatorial.
3. Calibrar o drift **nas próprias trajetórias**, passo a passo, para qualquer modelo, inclusive os sem fórmula fechada.
4. Deduzir a condição de drift do HJM e estimar suas volatilidades por componentes principais.
5. Comparar os modelos calibrados ao mesmo mercado naquilo que a calibração não fixa: smile, prazos longos, juros negativos e correlação entre prazos.

Onde estamos

Os Módulos 17 e 18 trataram de um modelo, o Hull-White, por fórmulas e por árvore. Árvores são ótimas para um ou dois fatores e para exercício antecipado (os diagramas de árvores trinomiais, padrão e implícitas, de Derman, Kani e Chriss, 1996, ilustram bem a ideia de espaço de estados). Mas, com dois fatores ou mais, com dependência de trajetória, ou quando o modelo é a curva inteira (HJM), o caminho é a simulação. Este módulo constrói, sobre a mesma curva sintética, uma escada de modelos e calibra todos ao mesmo mercado.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import brentq, least_squares
from IPython.display import display

CORES = {"Ho-Lee": CINZA, "Hull-White": AZUL, "CIR++": VERDE, "Black-Karasinski": LARANJA,
 ↪  "G2++": "#7b4bc4", "HJM 3 fatores": "#c43b6b"}

# curva sintética Nelson-Siegel dos Módulos 17 e 18
def f_merc(t, b0=0.045, b1=-0.010, b2=0.020, tau=1.5):
    t = np.asarray(t, float); e = np.exp(-t / tau)
    return b0 + b1 * e + b2 * (t / tau) * e

def P_merc(T, b0=0.045, b1=-0.010, b2=0.020, tau=1.5):
    T = np.asarray(T, float); e = np.exp(-T / tau)
    return np.exp(-(b0 * T + b1 * tau * (1 - e) + b2 * tau * (1 - (1 + T / tau) * e)))

# grade comum: passo semanal até 10 anos
DT = 1 / 52
N10 = 520
TEMPOS = np.arange(N10 + 1) * DT
I_PATHS = 20_000

def desconto(r):
    """exp(−∫r) em cada data da grade, regra do trapézio. r tem forma (passos+1, trajetórias)."""
    integ = np.cumsum((r[1:] + r[:-1]) / 2 * DT, axis=0)
    return np.vstack([np.ones((1, r.shape[1])), np.exp(-integ)])

def teste_curva(r, prazos=(1, 2, 5, 10)):
    """Erro de E[exp(−∫r)] contra P(0,T), em pb, e o erro-padrão."""
    D = desconto(r)
    linhas = []
    for T in prazos:
        k = int(round(T / DT))
        if k < D.shape[0]:
            d = D[k]
            linhas.append((T, 1e4 * (d.mean() - P_merc(T)), 1e4 * d.std() / np.sqrt(len(d))))
    return pd.DataFrame(linhas, columns=["prazo", "erro (pb)", "erro-padrão (pb)"])

def ou_exato(a, s, Z):
    """Ornstein-Uhlenbeck dx = −a x dt + s dW, x0 = 0, transição exata."""
    e1 = np.exp(-a * DT); sd = s * np.sqrt((1 - e1**2) / (2 * a))
    x = np.zeros((Z.shape[0] + 1, Z.shape[1]))
    for k in range(Z.shape[0]):
        x[k + 1] = x[k] * e1 + sd * Z[k]
    return x

print("Ambiente pronto.")
```

```
Ambiente pronto.
```

1. O que é "calibrar trajetórias"

Uma simulação de juros para precificação precisa acertar três coisas.

1. A medida. Para preços, as trajetórias são simuladas sob a medida neutra ao risco Q, com a conta remunerada como numerário (Módulo 17, Seção 2). Para risco (VaR, estresse, ALM), simula-se sob a medida real P, que difere de Q pelo preço de mercado do risco λ. Este módulo trabalha sob Q.

2. A curva de hoje. Para todo prazo T,

$$\mathbb{E}^{Q}\left[e^{-\int_0^T r(u)\,du}\right] = P^{mkt}(0,T). \tag{19.5}$$

É o teste da martingal do Módulo 15 aplicado ao zero-cupom: o título descontado deve ter média igual ao seu preço de hoje. Um modelo que não passa nesse teste dá preços errados até para títulos sem opção.

3. Uma volatilidade. A curva fixa o drift; a dispersão das trajetórias tem de vir de preços de opções. Como "mercado" de volatilidade usamos um único instrumento: o **caplet no dinheiro de 2 anos com prazo de 3 meses, com taxa composta no período** (*in arrears*, no estilo do SOFR e do CDI, que capitalizam a taxa diária). Ele paga em $t = 2{,}25$

$$\delta\,\max(R-K,0), \qquad R=\frac{1}{\delta}\Big(e^{\int_2^{2,25} r\,du}-1\Big), \quad \delta = 0{,}25, \tag{19.6}$$

e tem a vantagem de depender só da trajetória de r, de modo que qualquer modelo o precifica com as mesmas linhas de código. O preço de "mercado" é o do Hull-White de referência ($a = 0{,}1$, $\sigma = 1\%$), calculado com 100 mil trajetórias.

1.1 A escada de modelos

Modelo	Short rate	Como ajusta a curva	Taxa negativa	Fatores
Vasicek	$dr = a(b-r)dt + \sigma dW$	não ajusta (3 parâmetros)	sim	1
Ho-Lee	$dr = \theta(t)dt + \sigma dW$	$\theta(t)$ fechado	sim	1
Hull-White	$dr = (\theta(t) - ar)dt + \sigma dW$	$\theta(t)$ fechado	sim	1
CIR++	$r = x + \varphi(t)$, x CIR	deslocamento $\varphi(t)$ fechado	só se $\varphi < 0$	1
Black-Karasinski	$\ln r = x + \alpha(t)$, x OU	$\alpha(t)$ numérico	não	1
G2++	$r = x+y+\varphi(t)$, dois OU	$\varphi(t)$ fechado	sim	2
HJM	a curva forward inteira	automático (começa na curva)	depende das vols	n

```
F_CAP = (P_merc(2.0) / P_merc(2.25) - 1) / 0.25                    # taxa forward simples de 2 a
↪ 2,25
K_CAP_IDX = (int(round(2.0 / DT)), int(round(2.25 / DT)))


def caplet_composto(r, K=F_CAP, delta=0.25, k0=K_CAP_IDX[0], k1=K_CAP_IDX[1]):
    """Caplet com taxa composta no período [t_k0, t_k1], pago em t_k1. Devolve preço e
    ↪ erro-padrão."""
    integ = np.sum((r[k0 + 1:k1 + 1] + r[k0:k1]) / 2 * DT, axis=0)
    R = (np.exp(integ) - 1) / delta
    pago = desconto(r[:k1 + 1])[-1] * delta * np.maximum(R - K, 0)
    return pago.mean(), pago.std() / np.sqrt(len(pago))


# Hull-White de referência, θ(t) fechado (Módulo 17): r = x + φ(t)
A_REF, S_REF = 0.10, 0.010


def phi_hw(t, a=A_REF, s=S_REF):
    return f_merc(t) + s**2 / (2 * a**2) * (1 - np.exp(-a * np.asarray(t, float))) ** 2


rng = np.random.default_rng(1900)
N_CAP = K_CAP_IDX[1]
Zref = rng.standard_normal((N_CAP, 100_000))
r_ref = ou_exato(A_REF, S_REF, Zref) + phi_hw(TEMPOS[:N_CAP + 1])[:, None]
CAP_MERC, ep_cap = caplet_composto(r_ref)
print("caplet de referência (mercado):", f3(1e4 * CAP_MERC), "pb da face | erro-padrão", f3(1e4 *
↪ ep_cap), "pb | strike", f3(100 * F_CAP), "%")
del Zref, r_ref
ok(ep_cap / CAP_MERC < 0.01, "o preço de referência tem erro-padrão abaixo de 1%")
```

```
caplet de referência (mercado): 11,873 pb da face | erro-padrão 0,054 pb | strike 4,975 %
OK — o preço de referência tem erro-padrão abaixo de 1%
```

---

2. Vasicek: o modelo que não cabe na curva

Material: Aula 4

No Vasicek, $dr = a(b - r)\,dt + \sigma\,dW$, o zero-cupom é afim, $P = e^{A - Br}$, com $B = (1 - e^{-a\tau})/a$ e

$$A(\tau) = \left(b - \frac{\sigma^2}{2a^2}\right)(B(\tau) - \tau) - \frac{\sigma^2}{4a} B(\tau)^2. \tag{19.7}$$

Com r_0, a e b livres (e σ fixo em 1%), procuramos os parâmetros que melhor ajustam a curva até 10 anos. A curva do Vasicek é sempre monótona ou com uma única corcova suave em formato restrito; a curva Nelson-Siegel, com corcova em 2 anos, não cabe.

Laboratório 1 — O melhor Vasicek possível

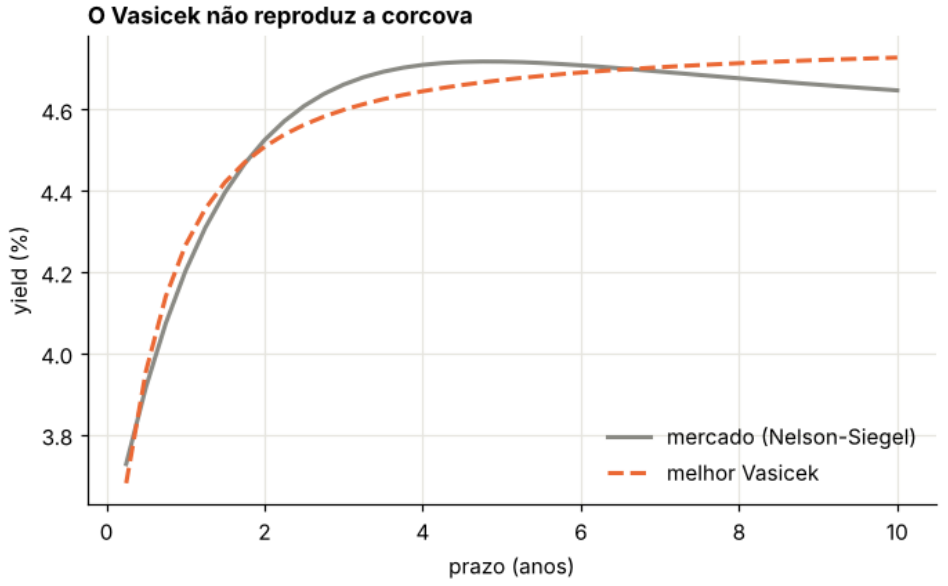
```
def P_vasicek(T, r0, a, b, s=0.01):
    B = (1 - np.exp(-a * T)) / a
    A = (b - s**2 / (2 * a**2)) * (B - T) - s**2 / (4 * a) * B**2
    return np.exp(A - B * r0)


Tc = np.linspace(0.25, 10, 40)
y_merc = -np.log(P_merc(Tc)) / Tc
res = least_squares(lambda p: -np.log(P_vasicek(Tc, *p)) / Tc - y_merc, x0=[0.035, 0.5, 0.047],
↪  bounds=([-0.05, 0.001, -0.1], [0.2, 5, 0.2]))
r0_v, a_v, b_v = res.x
erro_v = 1e4 * (-np.log(P_vasicek(Tc, *res.x)) / Tc - y_merc)
print(f"Vasicek ajustado: r0 = {f3(100 * r0_v)}%, a = {f3(a_v)}, b = {f3(100 * b_v)}%")
print("maior erro de yield:", f3(np.abs(erro_v).max()), "pb | erro médio absoluto:",
↪  f3(np.abs(erro_v).mean()), "pb")

fig, ax = plt.subplots()
ax.plot(Tc, 100 * y_merc, color=CINZA, label="mercado (Nelson-Siegel)")
ax.plot(Tc, -100 * np.log(P_vasicek(Tc, *res.x)) / Tc, "--", color=LARANJA, label="melhor
↪  Vasicek")
ax.set_xlabel("prazo (anos)"); ax.set_ylabel("yield (%)"); ax.set_title("O Vasicek não reproduz a
↪  corcova"); ax.legend()
plt.show()

ok(np.abs(erro_v).max() > 3, "o melhor Vasicek erra a curva em mais de 3 pb em algum prazo")
```

```
Vasicek ajustado: r0 = 3,255%, a = 2,772, b = 4,784%
maior erro de yield: 8,057 pb | erro médio absoluto: 4,341 pb
```




```
OK — o melhor Vasicek erra a curva em mais de 3 pb em algum prazo
```

Erros de alguns pontos-base parecem pequenos, mas são preço: num título de 10 anos, 5 pb são cerca de 0,4% do valor, muito mais que um spread de compra e venda. Daí a ideia de Ho-Lee e de Hull-White: deixar um parâmetro depender do tempo para que a curva saia **exata**.

---

3. Ho-Lee e Hull-White: ajuste fechado

Material: Aula 5; Módulo 17

Pelo Módulo 17, com $r = x + \varphi(t)$ e x um OU que parte de zero,

$$\varphi(t) = f^{mkt}(0,t) + \frac{\sigma^2}{2a^2}\left(1 - e^{-at}\right)^2 \quad \text{(Hull-White)}, \qquad \varphi(t) = f^{mkt}(0,t) + \frac{\sigma^2 t^2}{2} \quad \text{(Ho-Lee, } a \to 0\text{)}. \tag{19.8}$$

O termo extra é a **correção de convexidade**: como $e^{-\int r}$ é convexa, a média das taxas precisa ficar um pouco acima da taxa forward para que a média dos descontos acerte o preço.

Laboratório 2 — Curva exata e variâncias diferentes

```
rng = np.random.default_rng(19)
Z = rng.standard_normal((N10, I_PATHS))
x_hw = ou_exato(A_REF, S_REF, Z)
r_hw = x_hw + phi_hw(TEMPOS)[:, None]
x_hl = np.vstack([np.zeros((1, I_PATHS)), np.cumsum(S_REF * np.sqrt(DT) * Z, axis=0)])
r_hl = x_hl + (f_merc(TEMPOS) + S_REF**2 * TEMPOS**2 / 2)[:, None]

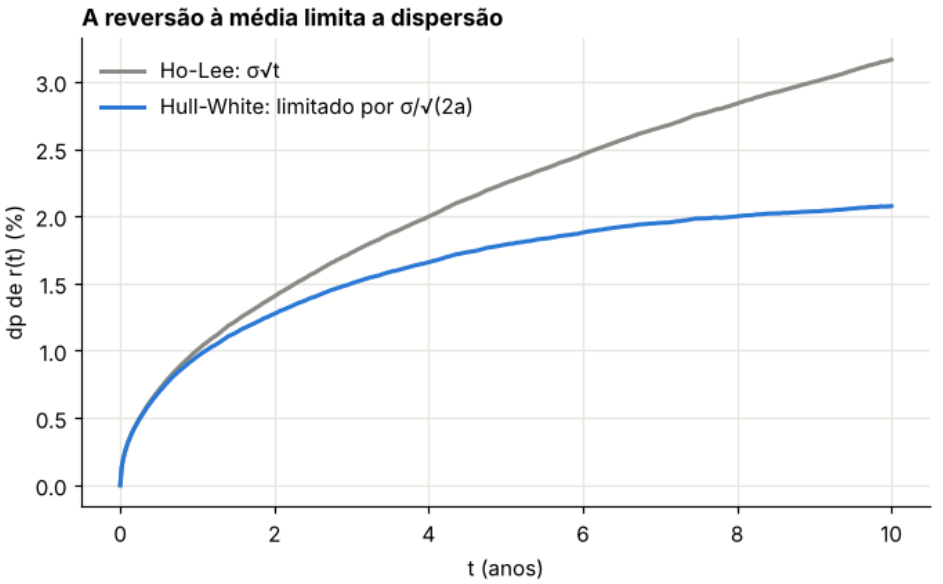
t_hw, t_hl = teste_curva(r_hw), teste_curva(r_hl)
tab2 = pd.DataFrame({"prazo": t_hw["prazo"], "Hull-White: erro (pb)": t_hw["erro (pb)"], "Ho-Lee:
↪  erro (pb)": t_hl["erro (pb)"],
                     "erro-padrão HW (pb)": t_hw["erro-padrão (pb)"]})
tabela(tab2)
dp_hw, dp_hl = r_hw.std(axis=1), r_hl.std(axis=1)
print("dp de r(10): Hull-White", f3(100 * dp_hw[-1]), "% | Ho-Lee", f3(100 * dp_hl[-1]), "% |
↪  teórico HW", f3(100 * S_REF * np.sqrt((1 - np.exp(-2 * A_REF * 10)) / (2 * A_REF))), "%")

fig, ax = plt.subplots()
ax.plot(TEMPOS, 100 * dp_hl, color=CORES["Ho-Lee"], label="Ho-Lee: σ√t")
ax.plot(TEMPOS, 100 * dp_hw, color=CORES["Hull-White"], label="Hull-White: limitado por σ/√(2a)")
ax.set_xlabel("t (anos)"); ax.set_ylabel("dp de r(t) (%)"); ax.set_title("A reversão à média
↪  limita a dispersão"); ax.legend()
plt.show()

del Z, x_hl
ok((np.abs(t_hw["erro (pb)"]) < 4 * t_hw["erro-padrão (pb)"] + 0.5).all(), "Hull-White: a curva é
↪  reproduzida em todos os prazos")
ok((np.abs(t_hl["erro (pb)"]) < 4 * t_hl["erro-padrão (pb)"] + 0.5).all(), "Ho-Lee: idem")
ok(abs(dp_hl[-1] / (S_REF * np.sqrt(10)) - 1) < 0.02, "no Ho-Lee, o desvio-padrão cresce como
↪  σ√t")
```

prazo	Hull-White: erro (pb)	Ho-Lee: erro (pb)	erro-padrão HW (pb)
1	0,213	0,220	0,379
2	0,339	0,383	0,979
5	0,590	0,724	3,010
10	4,847	6,059	5,798

```
dp de r(10): Hull-White 2,075 % | Ho-Lee 3,165 % | teórico HW 2,079 %
```




```
OK — Hull-White: a curva é reproduzida em todos os prazos
OK — Ho-Lee: idem
OK — no Ho-Lee, o desvio-padrão cresce como σ√t
```

4. Calibrar o drift nas próprias trajetórias

Para modelos sem $\varphi(t)$ fechado, como o Black-Karasinski, há um método geral que usa as trajetórias já simuladas. Escreva a taxa como $r_k = g(\alpha_k + x_k)$, com x simulado (o "ruído" do modelo) e α_k um deslocamento a determinar em cada data t_k. É a indução para a frente do Módulo 18, com as trajetórias no lugar dos nós da árvore.

Passo 1. Conhecidos $\alpha_0, \dots, \alpha_{k-1}$, cada trajetória i tem o desconto acumulado até t_k: $D_{k,i} = \exp(-\int_0^{t_k} r_i)$.

Passo 2. Escolha α_{k+1} para que o desconto médio até t_{k+1} seja o de mercado:

$$\frac{1}{I}\sum_i D_{k,i} \exp\Big(-\tfrac{\Delta t}{2}\big[r_{k,i} + g(\alpha_{k+1} + x_{k+1,i})\big]\Big) = P^{mkt}(0, t_{k+1}). \tag{19.9}$$

O lado esquerdo é monótono em α_{k+1}: uma busca de raiz resolve.

Passo 3. Atualize D_{k+1} e repita.

Por construção, a curva sai exata **nas trajetórias usadas na calibração**. O teste honesto é com trajetórias novas, sorteadas com outra semente, nas quais o erro deve ficar no nível do ruído de Monte Carlo. Para o Hull-White (g = identidade), os α_k devem reproduzir o $\varphi(t)$ fechado.

Laboratório 3 — O método reproduz o Hull-White

```
def calibrar_drift(x, g, r0, intervalo):
    """α_k nas trajetórias x (forma (passos+1, I)), com r_k = g(α_k + x_k) e r_0 dado. Regra do
    ↪ trapézio."""
    n = x.shape[0] - 1
    alfa = np.zeros(n + 1)
    r_ant = np.full(x.shape[1], r0)
    D = np.ones(x.shape[1])
    for k in range(n):
        alvo = P_merc((k + 1) * DT)
        func = lambda A: np.mean(D * np.exp(-0.5 * DT * (r_ant + g(A + x[k + 1])))) - alvo
        alfa[k + 1] = brentq(func, *intervalo, xtol=1e-14)
        r_novo = g(alfa[k + 1] + x[k + 1])
        D = D * np.exp(-0.5 * DT * (r_ant + r_novo))
        r_ant = r_novo
    return alfa


identidade = lambda v: v
t0 = time.time()
alfa_hw = calibrar_drift(x_hw, identidade, f_merc(0.0), (-0.5, 0.5))
print(f"calibração em {time.time() - t0:.1f} s para {N10} datas e {I_PATHS:,}
↪ trajetórias".replace(",", "."))
dif = 1e4 * (alfa_hw[1:] - phi_hw(TEMPOS[1:]))

rng_novo = np.random.default_rng(1919)
x_novo = ou_exato(A_REF, S_REF, rng_novo.standard_normal((N10, I_PATHS)))
r_dentro = alfa_hw[:, None] + x_hw; r_dentro[0] = f_merc(0.0)
r_fora = alfa_hw[:, None] + x_novo; r_fora[0] = f_merc(0.0)
t_dentro, t_fora = teste_curva(r_dentro), teste_curva(r_fora)
tab3 = pd.DataFrame({"prazo": t_dentro["prazo"], "erro nas trajetórias da calibração (pb)":
↪ t_dentro["erro (pb)"],
                     "erro em trajetórias novas (pb)": t_fora["erro (pb)"], "erro-padrão (pb)":
                     ↪ t_fora["erro-padrão (pb)"]})
tabela(tab3)
print("α_k − φ(t_k): média", f3(dif.mean()), "pb | maior", f3(np.abs(dif).max()), "pb")

del x_novo, r_dentro, r_fora
ok(np.abs(t_dentro["erro (pb)"]).max() < 1e-6, "nas trajetórias da calibração, a curva é exata")
ok((np.abs(t_fora["erro (pb)"]) < 4 * t_fora["erro-padrão (pb)"]).all(), "em trajetórias novas, o
↪ erro fica no nível do ruído")
ok(np.abs(dif).max() < 10, "o α_k numérico reproduz o φ(t) fechado a menos de 10 pb (ruído de
↪ Monte Carlo)")
```

```
calibração em 1.1 s para 520 datas e 20.000 trajetórias
```

prazo	erro nas trajetórias da calibração (pb)	erro em trajetórias novas (pb)	erro-padrão (pb)
1	0,000	-0,351	0,377
2	0,000	-0,973	0,981
5	0,000	-0,166	3,023
10	0,000	-4,591	5,766

```
α_k − φ(t_k): média 0,774 pb | maior 2,874 pb
OK — nas trajetórias da calibração, a curva é exata
OK — em trajetórias novas, o erro fica no nível do ruído
OK — o α_k numérico reproduz o φ(t) fechado a menos de 10 pb (ruído de Monte Carlo)
```

O desvio de alguns pontos-base entre α_k e $\varphi(t)$ é o ruído de Monte Carlo absorvido pela calibração: o método ajusta o drift **à amostra**. Com mais trajetórias, a diferença cai como $1/\sqrt{I}$.

---

5. CIR++: raiz quadrada com deslocamento

Material: Módulos 11, 12 e 13; Brigo e Mercurio, §3.9

O CIR puro não reproduz a curva. Brigo e Mercurio propõem somar um deslocamento determinístico a um CIR:

$$r(t) = x(t) + \varphi(t), \qquad dx = \kappa(\theta - x)\,dt + \sigma\sqrt{x}\,dW. \tag{19.10}$$

A curva sai exata se $\varphi(t) = f^{mkt}(0,t) - f^{CIR}(0,t)$, em que f^{CIR} é a taxa forward do CIR com a fórmula **com o fator 2** (Módulo 13):

$$f^{CIR}(0,t) = \frac{2\kappa\theta(e^{\gamma t}-1)}{2\gamma + (\kappa+\gamma)(e^{\gamma t}-1)} + x_0\frac{4\gamma^2 e^{\gamma t}}{(2\gamma+(\kappa+\gamma)(e^{\gamma t}-1))^2}, \qquad \gamma = \sqrt{\kappa^2+2\sigma^2}. \tag{19.11}$$

x é simulado pelo Full Truncation do Módulo 12: com a condição de Feller folgada, como aqui, o viés com passo semanal é desprezível, e os mesmos normais servem para todas as tentativas de σ (a transição exata por qui-quadrado não central também funciona, mas não varia suavemente com σ quando os sorteios ficam fixos). A taxa só fica negativa se $\varphi(t)$ for negativo; escolhendo x_0 e θ abaixo da curva, $\varphi > 0$ e as taxas ficam positivas.

Laboratório 4 — CIR++ calibrado à curva e ao caplet

$\kappa = 0{,}3$, $\theta = 3\%$, $x_0 = 2{,}5\%$. O σ é calibrado para que o caplet de referência tenha o preço de mercado. Usamos os mesmos sorteios em todas as tentativas (números aleatórios comuns), para que a busca de raiz não seja atrapalhada pelo ruído.

```python
K_C, TH_C, X0_C = 0.3, 0.03, 0.025


def f_cir(t, s, k=K_C, th=TH_C, x0=X0_C):
    g = np.sqrt(k**2 + 2 * s**2); E = np.exp(g * t) - 1; D = 2 * g + (k + g) * E
    return 2 * k * th * E / D + x0 * 4 * g**2 * np.exp(g * t) / D**2


def simular_cirpp(s, Z, passos):
    """CIR por Full Truncation (Módulo 12) com normais Z fixas; r = x + φ(t)."""
    x = np.empty((passos + 1, Z.shape[1])); x[0] = X0_C
    xh = x[0].copy()
    for k in range(passos):
        p = np.maximum(xh, 0)
```

```
        xh = xh + K_C * (TH_C - p) * DT + s * np.sqrt(p * DT) * Z[k]
        x[k + 1] = np.maximum(xh, 0)
    return x + (f_merc(TEMPOS[:passos + 1]) - f_cir(TEMPOS[:passos + 1], s))[:, None]


rng = np.random.default_rng(2024)
Z_cal = rng.standard_normal((N_CAP, 8000))
t0 = time.time()
s_cir = brentq(lambda s: caplet_composto(simular_cirpp(s, Z_cal, N_CAP))[0] - CAP_MERC, 0.02,
↪  0.15, xtol=1e-4)
print(f"σ do CIR++ calibrado: {f3(100 * s_cir)}% (√x0·σ = {f3(100 * s_cir * np.sqrt(X0_C))}% de
↪  vol normal inicial) em {time.time() - t0:.1f} s")
print("Feller 2κθ − σ²:", f3(2 * K_C * TH_C - s_cir**2))

r_cir = simular_cirpp(s_cir, rng.standard_normal((N10, I_PATHS)), N10)
t_cir = teste_curva(r_cir)
tabela(t_cir)
phi_cir = f_merc(TEMPOS) - f_cir(TEMPOS, s_cir)
print("menor φ(t):", f3(100 * phi_cir.min()), "% | fração de taxas negativas em todo o
↪  horizonte:", f3(100 * np.mean(r_cir < 0)), "%")

ok((np.abs(t_cir["erro (pb)"]) < 4 * t_cir["erro-padrão (pb)"] + 0.5).all(), "o CIR++ reproduz a
↪  curva em todos os prazos")
ok(phi_cir.min() > 0 and np.all(r_cir >= 0), "com φ positivo, nenhuma taxa simulada é negativa")
```

```
σ do CIR++ calibrado: 7,579% (√x0·σ = 1,198% de vol normal inicial) em 0.4 s
Feller 2κθ − σ²: 0,012
```

prazo	erro (pb)	erro-padrão (pb)
1	-0,235	0,420
2	-0,935	1,035
5	-4,055	2,699
10	-0,441	4,149

```
menor φ(t): 1,000 % | fração de taxas negativas em todo o horizonte: 0,000 %
OK — o CIR++ reproduz a curva em todos os prazos
OK — com φ positivo, nenhuma taxa simulada é negativa
```

---

6. Black-Karasinski: lognormal, drift numérico

Material: Black e Karasinski (1991); Itkin, Lipton e Muravey (2021); Módulo 18

$\ln r = \alpha(t) + x(t)$, com x um OU de reversão a e volatilidade σ. Não há fórmula para o zero-cupom, e $\alpha(t)$ é encontrado pelo método da Seção 4, com $g = \exp$. Para calibrar σ ao caplet, cada tentativa de σ exige recalibrar todo o α: é uma calibração aninhada, possível porque a calibração do drift é rápida.

Laboratório 5 — BK calibrado nas trajetórias

```
A_BK = 0.10
rng = np.random.default_rng(77)
x_unit_cal = ou_exato(A_BK, 1.0, rng.standard_normal((N_CAP, 8000)))


def r_bk(s, x_unit):
    alfa = calibrar_drift(s * x_unit, np.exp, f_merc(0.0), (-15, 3))
    r = np.exp(alfa[:, None] + s * x_unit); r[0] = f_merc(0.0)
    return r


t0 = time.time()
s_bk = brentq(lambda s: caplet_composto(r_bk(s, x_unit_cal))[0] - CAP_MERC, 0.05, 0.6, xtol=1e-4)
print(f"σ do BK calibrado: {f3(100 * s_bk)}% (lognormal) em {time.time() - t0:.1f} s; no Módulo
↪  18, pela árvore e caplet clássico: 20,242%")

x_unit = ou_exato(A_BK, 1.0, np.random.default_rng(78).standard_normal((N10, I_PATHS)))
r_bk10 = r_bk(s_bk, x_unit)
x_teste = ou_exato(A_BK, 1.0, np.random.default_rng(79).standard_normal((N10, I_PATHS)))
alfa_bk = np.log(r_bk10[1:, 0]) - s_bk * x_unit[1:, 0]
r_bk_fora = np.exp(np.r_[np.log(f_merc(0.0)), alfa_bk][:, None] + s_bk * x_teste); r_bk_fora[0] =
↪  f_merc(0.0)
t_bk = teste_curva(r_bk_fora)
tabela(t_bk)
del x_teste, r_bk_fora

ok((np.abs(t_bk["erro (pb)"]) < 4 * t_bk["erro-padrão (pb)"] + 0.5).all(), "em trajetórias novas,
↪  o BK calibrado reproduz a curva")
ok(np.all(r_bk10 > 0), "todas as taxas do BK são positivas")
ok(0.15 < s_bk < 0.25, "a vol lognormal calibrada fica perto dos 20% do Módulo 18")
```

```
σ do BK calibrado: 20,605% (lognormal) em 1.5 s; no Módulo 18, pela árvore e caplet clássico: 20,242%
```

prazo	erro (pb)	erro-padrão (pb)
1	-0,443	0,338
2	-2,110	0,952
5	-10,376	2,992
10	-9,155	5,476

```
OK — em trajetórias novas, o BK calibrado reproduz a curva
OK — todas as taxas do BK são positivas
OK — a vol lognormal calibrada fica perto dos 20% do Módulo 18
```

---

7. G2++: dois fatores

Material: Aula 4 (modelos de dois fatores); Brigo e Mercurio, cap. 4

Todos os modelos até aqui têm **um** fator: um único choque move a curva inteira, e as variações de taxas de prazos diferentes têm correlação **perfeita**. Na prática, a taxa de 1 ano e a de 10 anos se movem juntas, mas não em sincronia. O G2++ soma dois OU correlacionados:

$$r(t) = x(t)+y(t)+\varphi(t), \qquad dx = -a\,x\,dt+\sigma\,dW_1, \quad dy = -b\,y\,dt+\eta\,dW_2, \quad dW_1 dW_2 = \rho\,dt. \tag{19.12}$$

Tipicamente, um fator tem reversão rápida (move a parte curta) e o outro lenta (move a curva toda). O ajuste exato à curva é

$$\varphi(t) = f^{mkt}(0,t) + \frac{\sigma^2}{2a^2}(1-e^{-at})^2 + \frac{\eta^2}{2b^2}(1-e^{-bt})^2 + \rho\frac{\sigma\eta}{ab}(1-e^{-at})(1-e^{-bt}). \tag{19.13}$$

A correlação entre prazos. A variação instantânea do yield de prazo τ é $\Delta y(\tau) = \frac{B_a(\tau)}{\tau}\Delta x + \frac{B_b(\tau)}{\tau}\Delta y$, com $B_c(\tau) = (1-e^{-c\tau})/c$. A correlação entre dois prazos sai das cargas e da covariância dos choques:

$$\operatorname{corr}(\Delta y_{\tau_1}, \Delta y_{\tau_2}) = \frac{\ell(\tau_1)^\top \Sigma\, \ell(\tau_2)}{\sqrt{\ell(\tau_1)^\top \Sigma\, \ell(\tau_1)}\sqrt{\ell(\tau_2)^\top \Sigma\, \ell(\tau_2)}}, \qquad \Sigma = \begin{pmatrix} \sigma^2 & \rho\sigma\eta \\ \rho\sigma\eta & \eta^2 \end{pmatrix}. \tag{19.14}$$

Laboratório 6 — G2++ calibrado e a correlação entre prazos

Forma fixa: $a = 0{,}5$, $b = 0{,}05$, $\rho = -0{,}7$ e razão $\eta/\sigma = 0{,}875$; a escala comum é calibrada ao caplet.

```
A_G, B_G, RHO_G, RAZAO = 0.5, 0.05, -0.7, 0.875


def phi_g2(t, s, e):
    t = np.asarray(t, float)
    return (f_merc(t) + s**2 / (2 * A_G**2) * (1 - np.exp(-A_G * t)) ** 2 + e**2 / (2 * B_G**2) *
    ↪  (1 - np.exp(-B_G * t)) ** 2
           + RHO_G * s * e / (A_G * B_G) * (1 - np.exp(-A_G * t)) * (1 - np.exp(-B_G * t)))


def simular_g2(s, Z1, Z2):
    e = RAZAO * s
    ea, eb = np.exp(-A_G * DT), np.exp(-B_G * DT)
    sx, sy = s * np.sqrt((1 - ea**2) / (2 * A_G)), e * np.sqrt((1 - eb**2) / (2 * B_G))
    c = RHO_G * s * e * (1 - np.exp(-(A_G + B_G) * DT)) / (A_G + B_G) / (sx * sy)   # correlação
↪   exata dos incrementos
    passos = Z1.shape[0]
    x = np.zeros((passos + 1, Z1.shape[1])); y = np.zeros_like(x)
    for k in range(passos):
        x[k + 1] = x[k] * ea + sx * Z1[k]
        y[k + 1] = y[k] * eb + sy * (c * Z1[k] + np.sqrt(1 - c**2) * Z2[k])
    return x + y + phi_g2(TEMPOS[:passos + 1], s, e)[:, None], x, y


rng = np.random.default_rng(22)
Z1c, Z2c = rng.standard_normal((N_CAP, 8000)), rng.standard_normal((N_CAP, 8000))
s_g2 = brentq(lambda s: caplet_composto(simular_g2(s, Z1c, Z2c)[0])[0] - CAP_MERC, 0.002, 0.03,
↪  xtol=1e-6)
e_g2 = RAZAO * s_g2
print(f"G2++ calibrado: σ = {f3(100 * s_g2)}%, η = {f3(100 * e_g2)}% (a = {A_G}, b = {B_G}, ρ =
↪  {RHO_G})")

rng = np.random.default_rng(23)
```

```
r_g2, xg, yg = simular_g2(s_g2, rng.standard_normal((N10, I_PATHS)), rng.standard_normal((N10,
↪  I_PATHS)))
t_g2 = teste_curva(r_g2)
tabela(t_g2)


def corr_g2(t1, t2, s=s_g2, e=e_g2):
    Sig = np.array([[s**2, RHO_G * s * e], [RHO_G * s * e, e**2]])
    l = lambda t: np.array([(1 - np.exp(-A_G * t)) / (A_G * t), (1 - np.exp(-B_G * t)) / (B_G *
↪  t)])
    l1, l2 = l(t1), l(t2)
    return l1 @ Sig @ l2 / np.sqrt(l1 @ Sig @ l1 * (l2 @ Sig @ l2))


# conferência por simulação: variações semanais dos yields de 1 e 10 anos
la = lambda t, c: (1 - np.exp(-c * t)) / (c * t)
dy1 = la(1, A_G) * np.diff(xg[:, :2000], axis=0) + la(1, B_G) * np.diff(yg[:, :2000], axis=0)
dy10 = la(10, A_G) * np.diff(xg[:, :2000], axis=0) + la(10, B_G) * np.diff(yg[:, :2000], axis=0)
corr_sim = np.corrcoef(dy1.ravel(), dy10.ravel())[0, 1]
prazos_c = [0.25, 1, 2, 5, 10]
M = np.array([[corr_g2(u, v) for v in prazos_c] for u in prazos_c])
tabela(pd.DataFrame(M, index=[f"{p} ano(s)" for p in prazos_c], columns=[f"{p}" for p in
↪  prazos_c]))
print("corr(Δy 1 ano, Δy 10 anos): fórmula", f3(corr_g2(1, 10)), "| simulação", f3(corr_sim), "|
↪  qualquer modelo de 1 fator: 1,000")
del xg, yg

ok((np.abs(t_g2["erro (pb)"]) < 4 * t_g2["erro-padrão (pb)"] + 0.5).all(), "o G2++ reproduz a
↪  curva em todos os prazos")
ok(abs(corr_sim - corr_g2(1, 10)) < 0.03, "a correlação simulada confere com a fórmula")
ok(corr_g2(1, 10) < 0.9, "com dois fatores, os prazos de 1 e 10 anos deixam de ser perfeitamente
↪  correlacionados")
```

```
G2++ calibrado: σ = 1,467%, η = 1,284% (a = 0.5, b = 0.05, ρ = -0.7)
```

prazo	erro (pb)	erro-padrão (pb)
1	-0,077	0,377
2	0,115	0,957
5	2,864	3,119
10	-0,634	6,978

	0.25	1	2	5	10
0.25 ano(s)	1,000	0,982	0,918	0,699	0,529
1 ano(s)	0,982	1,000	0,976	0,821	0,679
2 ano(s)	0,918	0,976	1,000	0,925	0,822
5 ano(s)	0,699	0,821	0,925	1,000	0,977
10 ano(s)	0,529	0,679	0,822	0,977	1,000

```
corr(Δy 1 ano, Δy 10 anos): fórmula 0,679 | simulação 0,681 | qualquer modelo de 1 fator: 1,000
OK — o G2++ reproduz a curva em todos os prazos
OK — a correlação simulada confere com a fórmula
OK — com dois fatores, os prazos de 1 e 10 anos deixam de ser perfeitamente correlacionados
```

8. HJM: simular a curva inteira

Material: Aula 8 (The Heath, Jarrow and Morton Model)

8.1 A ideia

Em vez de modelar a short rate e deduzir a curva, Heath, Jarrow e Morton (1992) modelam diretamente **todas** as taxas forward:

$$df(t,T) = \mu(t,T)\,dt + \sum_{k=1}^{n} \sigma_k(t,T)\,dW_k(t). \tag{19.15}$$

A curva inicial é a de mercado por construção: $f(0,T) = f^{mkt}(0,T)$. A pergunta é que drift μ evita arbitragem.

8.2 A condição de drift, passo a passo

Passo 1. $P(t,T) = \exp(-\int_t^T f(t,u)\,du)$. Seja $X = -\int_t^T f(t,u)\,du$. Diferenciando (o limite inferior gera $+f(t,t)\,dt = r\,dt$):

$$dX = r\,dt - \int_t^T \mu(t,u)\,du\,dt - \sum_k \Sigma_k(t,T)\,dW_k, \qquad \Sigma_k(t,T) = \int_t^T \sigma_k(t,u)\,du. \tag{19.16}$$

Passo 2. Pelo Itô, $dP/P = dX + \frac{1}{2}(dX)^2 = (r - \int_t^T \mu\,du + \frac{1}{2}\sum_k \Sigma_k^2)dt - \sum_k \Sigma_k\,dW_k$.

Passo 3. Sob Q, todo título rende r: o drift deve ser r para todo T, isto é, $\int_t^T \mu(t,u)\,du = \frac{1}{2}\sum_k \Sigma_k(t,T)^2$.

Passo 4. Derivando em T:

$$\boxed{\mu(t,T) = \sum_{k=1}^{n} \sigma_k(t,T) \int_t^T \sigma_k(t,u)\,du.} \tag{19.17}$$

Sob Q, o drift das taxas forward é **determinado pelas volatilidades**. Escolhidas as vols, o modelo está completo.

Casos particulares. Com um fator e $\sigma(t,T) = \sigma e^{-a(T-t)}$, o HJM é o Hull-White. Com σ constante, é o Ho-Lee.

8.3 Parametrização de Musiela e a simulação

Com $x = T - t$ (prazo restante) e $g(t,x) = f(t,t+x)$, a dinâmica vira $dg = [\partial_x g + \mu(x)]dt + \sum_k \sigma_k(x)\,dW_k$. Na simulação, o termo $\partial_x g\,dt$ é só o **deslocamento** da curva de um passo ao longo da grade de prazos: a curva envelhece. A short rate é $r(t) = g(t,0)$.

8.4 De onde vêm as volatilidades: componentes principais

A Aula 8 decompõe os movimentos da curva em componentes principais. Com variações diárias da curva forward em vários prazos, a matriz de covariância C tem autovalores λ_k e autovetores v_k. As funções de volatilidade são $\sigma_k(x) = \sqrt{\lambda_k \cdot 252}\; v_k(x)$. Os três primeiros componentes costumam ser **nível**, **inclinação** e **curvatura**, e explicam quase toda a variância.

Laboratório 7 — Componentes principais de uma história sintética

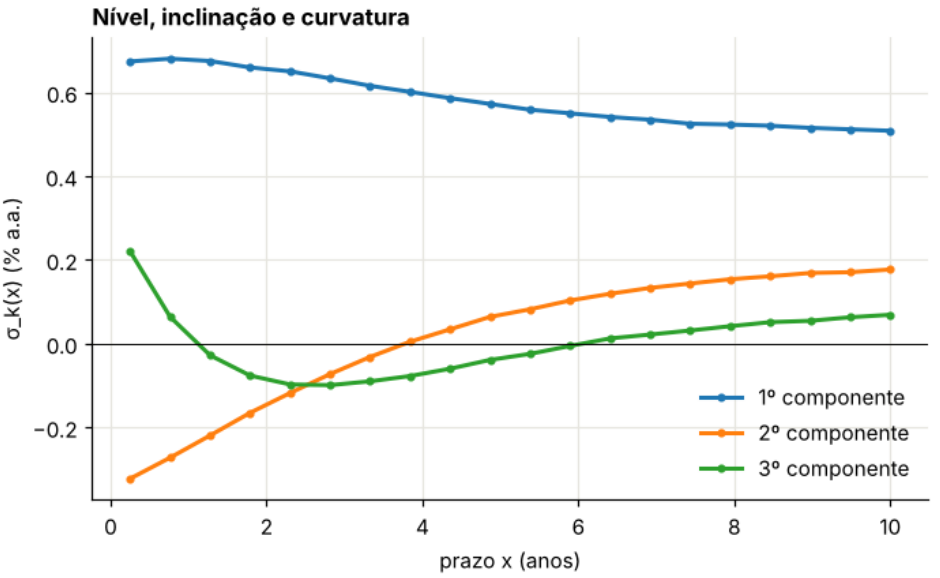
Geramos 3 anos de variações diárias da curva forward, em 20 prazos de 3 meses a 10 anos, a partir de três fatores verdadeiros (nível, inclinação, curvatura) e um ruído pequeno de cada prazo. Depois estimamos os fatores por componentes principais.

```
prazos_h = np.linspace(0.25, 10, 20)
fatores_verdade = np.array([0.0055 * np.ones_like(prazos_h),                    # nível
                            -0.0060 * np.exp(-0.4 * prazos_h),                   # inclinação
                            0.0040 * prazos_h * np.exp(-0.5 * prazos_h)])        # curvatura
rng = np.random.default_rng(8)
dias = 3 * 252
choques = rng.standard_normal((dias, 3)) @ fatores_verdade / np.sqrt(252) + rng.normal(0, 0.0004
↪  / np.sqrt(252), (dias, 20))
C = np.cov(choques.T)
autoval, autovet = np.linalg.eigh(C)
ordem = np.argsort(autoval)[::-1]; autoval, autovet = autoval[ordem], autovet[:, ordem]
explicado = np.cumsum(autoval) / autoval.sum()
sig_pca = (np.sqrt(autoval[:3] * 252) * autovet[:, :3]).T
sig_pca *= np.sign(sig_pca[:, -1:])                                              # sinal: positivo
↪  no prazo longo
print("variância explicada acumulada pelos 3 primeiros componentes:", [f3(100 * e) + "%" for e in
↪  explicado[:3]])
C_verdade = fatores_verdade.T @ fatores_verdade
C_pca = sig_pca.T @ sig_pca
print("erro relativo da covariância reconstruída (3 fatores):", f3(100 * np.linalg.norm(C_pca -
↪  C_verdade) / np.linalg.norm(C_verdade)), "%")

fig, ax = plt.subplots()
for k, nome in enumerate(["1º componente", "2º componente", "3º componente"]):
    ax.plot(prazos_h, 100 * sig_pca[k], "o-", ms=3, label=nome)
ax.axhline(0, color="black", lw=0.6)
ax.set_xlabel("prazo x (anos)"); ax.set_ylabel("σ_k(x) (% a.a.)"); ax.set_title("Nível,
↪  inclinação e curvatura"); ax.legend()
plt.show()

ok(explicado[2] > 0.97, "três componentes explicam mais de 97% da variância")
ok(np.linalg.norm(C_pca - C_verdade) / np.linalg.norm(C_verdade) < 0.15, "e reconstroem a
↪  covariância verdadeira a menos de 15%")
```

```
variância explicada acumulada pelos 3 primeiros componentes: ['91,570%', '98,090%', '99,642%']
erro relativo da covariância reconstruída (3 fatores): 2,110 %
```




```
OK — três componentes explicam mais de 97% da variância
OK — e reconstroem a covariância verdadeira a menos de 15%
```

Laboratório 8 — HJM de 3 fatores: sem o drift certo, há arbitragem

Simulamos a curva forward com as vols estimadas, interpoladas na grade de prazos, multiplicadas por uma escala comum calibrada ao caplet. Comparamos o drift de HJM com o drift zero ("as taxas forward são martingais"), que é o erro mais comum. Para conferir a implementação, rodamos também o HJM de um fator com σe^{-ax}, que deve reproduzir o Hull-White.

```python
DT_H = 2 * DT                                                    # passo quinzenal no HJM
xs = np.arange(0, 10 + DT_H / 2, DT_H)                           # prazos de 0 a 10 anos
sig_grade = np.array([np.interp(xs, prazos_h, s_) for s_ in sig_pca])   # constante fora de
↪ [0,25; 10]


def simular_hjm(sigs, Zs, com_drift=True, escala=1.0):
    """Musiela: g(t+Δ, x) = g(t, x+Δ) + μ(x)Δ + Σ σ_k(x)√Δ Z_k. Devolve r(t) na grade quinzenal."""
    s_ = escala * sigs
    cum = np.cumsum(s_, axis=1) * DT_H - s_ * DT_H / 2                    # ∫_0^x σ_k
    mu = (s_ * cum).sum(axis=0) if com_drift else np.zeros(len(xs))
    passos = Zs.shape[1]
    g = np.tile(f_merc(xs)[:, None], (1, Zs.shape[2]))
    r = np.empty((passos + 1, Zs.shape[2])); r[0] = g[0]
    for k in range(passos):
        g = np.vstack([g[1:], 2 * g[-1:] - g[-2:-1]])                # envelhecer a curva um
↪ passo
        g += mu[:, None] * DT_H + np.einsum("kx,ki->xi", s_, Zs[:, k]) * np.sqrt(DT_H)
        r[k + 1] = g[0]
    return r
```

```
def para_semanal(r_q):
    """Interpola linearmente a grade quinzenal na semanal, para reaproveitar os testes."""
    rs = np.empty((2 * (r_q.shape[0] - 1) + 1, r_q.shape[1]))
    rs[::2] = r_q; rs[1::2] = 0.5 * (r_q[1:] + r_q[:-1])
    return rs


rng = np.random.default_rng(31)
n_cap_h = N_CAP // 2 + 1
Zc = rng.standard_normal((3, n_cap_h, 8000))
esc_hjm = brentq(lambda e: caplet_composto(para_semanal(simular_hjm(sig_grade, Zc, escala=e)))[0]
 ↪  - CAP_MERC, 0.3, 3.0, xtol=1e-4)
print("escala das vols do HJM calibrada ao caplet:", f3(esc_hjm))

I_H = 10_000
Zh = np.random.default_rng(32).standard_normal((3, N10 // 2, I_H))
t0 = time.time()
r_hjm = para_semanal(simular_hjm(sig_grade, Zh, escala=esc_hjm))
r_hjm_sem = para_semanal(simular_hjm(sig_grade, Zh, com_drift=False, escala=esc_hjm))
sig_hw_hjm = np.array([S_REF * np.exp(-A_REF * xs)])
r_hjm_hw = para_semanal(simular_hjm(sig_hw_hjm, Zh[:1]))
print(f"três simulações HJM em {time.time() - t0:.1f} s")

t_hjm, t_sem = teste_curva(r_hjm), teste_curva(r_hjm_sem)
tab8 = pd.DataFrame({"prazo": t_hjm["prazo"], "HJM com drift (pb)": t_hjm["erro (pb)"],
 ↪  "erro-padrão (pb)": t_hjm["erro-padrão (pb)"],
                     "drift zero (pb)": t_sem["erro (pb)"]})
tabela(tab8)
k5 = int(round(5 / DT))
print("r(5 anos), HJM com σe^{−ax} × Hull-White: média", f3(100 * r_hjm_hw[k5].mean()), "% vs",
 ↪  f3(100 * r_hw[k5].mean()),
      "% | dp", f3(100 * r_hjm_hw[k5].std()), "% vs", f3(100 * r_hw[k5].std()), "%")

ok((np.abs(t_hjm["erro (pb)"]) < 4 * t_hjm["erro-padrão (pb)"] + 1).all(), "com o drift de HJM, a
 ↪  curva é reproduzida")
ok(abs(t_sem["erro (pb)"].iloc[-1]) > 4 * t_hjm["erro-padrão (pb)"].iloc[-1] + 1, "com drift
 ↪  zero, o título de 10 anos sai errado: há arbitragem")
ok(abs(r_hjm_hw[k5].std() / r_hw[k5].std() - 1) < 0.03 and abs(r_hjm_hw[k5].mean() -
 ↪  r_hw[k5].mean()) < 5e-4,
   "o HJM de um fator com σe^{−ax} reproduz a distribuição do Hull-White")
```

```
escala das vols do HJM calibrada ao caplet: 1,259
```

```
três simulações HJM em 32.3 s
```

prazo	HJM com drift (pb)	erro-padrão (pb)	drift zero (pb)
1	0,098	0,535	0,246
2	1,513	1,385	2,557
5	6,417	4,327	18,369
10	11,737	8,855	74,730

```
r(5 anos), HJM com σe^{−ax} × Hull-White: média 4,764 % vs 4,776 % | dp 1,769 % vs 1,789 %
OK — com o drift de HJM, a curva é reproduzida
OK — com drift zero, o título de 10 anos sai errado: há arbitragem
OK — o HJM de um fator com σe^{−ax} reproduz a distribuição do Hull-White
```

Com drift zero, o título longo sai **caro**: sem a correção de convexidade, as taxas forward não sobem o suficiente, e os descontos ficam grandes demais. O erro cresce com o prazo, como $\int \mu$.

---

9. Mesmo mercado, modelos diferentes

Todos os modelos de 1 a 3 fatores foram calibrados à **mesma curva** e ao **mesmo caplet** de 2 anos. Agora olhamos o que a calibração não fixa:

- o caplet no dinheiro de **5 anos** (outro vencimento);
- o caplet de 2 anos com strike **1,5 ponto acima** (o smile);
- a dispersão da short rate em 5 anos e a chance de ficar negativa;
- a correlação entre as variações dos yields de 1 e 10 anos.

Laboratório 9 — O risco de modelo de uma simulação calibrada

```
# o Ho-Lee do Laboratório 2 usava σ = 1%; calibramos agora ao caplet, como os demais
def simular_hl(s, Z):
    x = np.vstack([np.zeros((1, Z.shape[1])), np.cumsum(s * np.sqrt(DT) * Z, axis=0)])
    t_ = TEMPOS[:Z.shape[0] + 1]
    return x + (f_merc(t_) + s**2 * t_**2 / 2)[:, None]


Zhl = np.random.default_rng(41).standard_normal((N_CAP, 8000))
s_hl = brentq(lambda s: caplet_composto(simular_hl(s, Zhl))[0] - CAP_MERC, 0.002, 0.03, xtol=1e-6)
r_hl = simular_hl(s_hl, np.random.default_rng(42).standard_normal((N10, I_PATHS)))
print("σ do Ho-Lee calibrado ao caplet:", f3(100 * s_hl), "%")

k50, k525 = int(round(5 / DT)), int(round(5.25 / DT))
F5 = (P_merc(5.0) / P_merc(5.25) - 1) / 0.25
modelos = {"Ho-Lee": r_hl, "Hull-White": r_hw, "CIR++": r_cir, "Black-Karasinski": r_bk10,
 ↪  "G2++": r_g2, "HJM 3 fatores": r_hjm}
corr_1f = 1.0
L = lambda t: np.array([np.interp(np.linspace(0, t, 200), xs, s_).mean() for s_ in sig_grade])
corr_hjm = L(1) @ L(10) / np.sqrt(L(1) @ L(1) * (L(10) @ L(10)))
correlacoes = {"Ho-Lee": 1.0, "Hull-White": 1.0, "CIR++": 1.0, "Black-Karasinski": 1.0, "G2++":
 ↪  corr_g2(1, 10), "HJM 3 fatores": corr_hjm}
linhas = []
for nome, r in modelos.items():
    c2, _ = caplet_composto(r)
    c2_otm, _ = caplet_composto(r, K=F_CAP + 0.015)
    c5, e5 = caplet_composto(r, K=F5, k0=k50, k1=k525)
    tc = t_bk if nome == "Black-Karasinski" else teste_curva(r)          # BK: trajetórias novas
 ↪  (fora da amostra)
    linhas.append((nome, 1e4 * c2, 1e4 * c2_otm, 1e4 * c5, 100 * r[k50].std(), 100 *
 ↪  np.mean(r[k50] < 0), correlacoes[nome],
                   (np.abs(tc["erro (pb)"]) / tc["erro-padrão (pb)"]).max()))
tab9 = pd.DataFrame(linhas, columns=["modelo", "caplet 2a ATM (pb)", "caplet 2a +1,5 p.p. (pb)",
 ↪  "caplet 5a ATM (pb)",
                                     "dp r(5a) (%)", "% r(5a) < 0", "corr Δy1a × Δy10a", "maior
                                      ↪  erro de curva (em erros-padrão)"])
tabela(tab9)
print("mercado (caplet 2a ATM):", f3(1e4 * CAP_MERC), "pb")
```

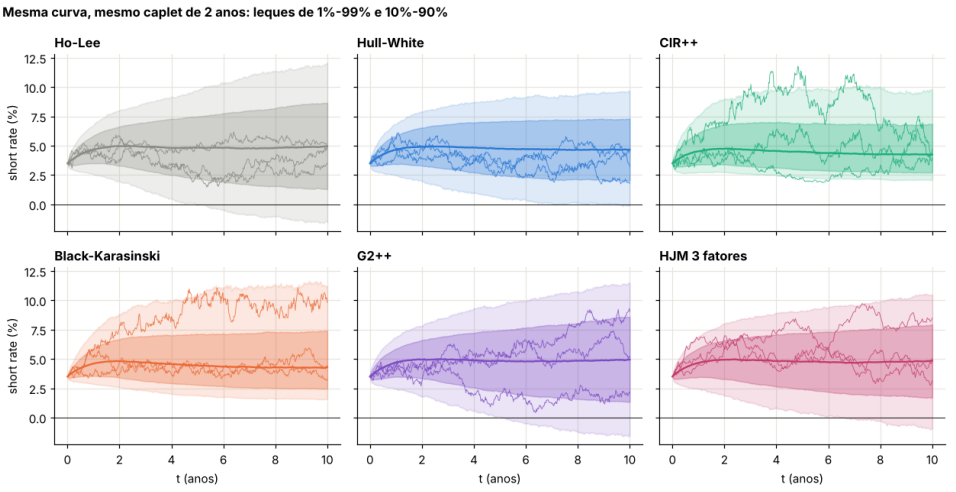
```
fig, axs = plt.subplots(2, 3, figsize=(11.5, 6), sharex=True, sharey=True)
for ax, (nome, r) in zip(axs.ravel(), modelos.items()):
    q = np.percentile(r[:, :5000], [1, 10, 50, 90, 99], axis=1)
    ax.fill_between(TEMPOS, 100 * q[0], 100 * q[4], color=CORES[nome], alpha=0.15)
    ax.fill_between(TEMPOS, 100 * q[1], 100 * q[3], color=CORES[nome], alpha=0.30)
    ax.plot(TEMPOS, 100 * q[2], color=CORES[nome], lw=1.5)
    ax.plot(TEMPOS, 100 * r[:, :3], color=CORES[nome], lw=0.6, alpha=0.8)
    ax.axhline(0, color="black", lw=0.6); ax.set_title(nome)
for ax in axs[1]:
    ax.set_xlabel("t (anos)")
for ax in axs[:, 0]:
    ax.set_ylabel("short rate (%)")
fig.suptitle("Mesma curva, mesmo caplet de 2 anos: leques de 1%-99% e 10%-90%", x=0.01, ha="left",
↪  fontweight="bold", fontsize=11)
plt.tight_layout(); plt.show()

t9 = tab9.set_index("modelo")
ok((np.abs(t9["caplet 2a ATM (pb)"] / (1e4 * CAP_MERC) - 1) < 0.05).all(), "todos os modelos
↪  acertam o caplet de calibração a menos de 5% (em trajetórias novas)")
ok((t9["maior erro de curva (em erros-padrão)"] < 4).all(), "e todos passam no teste da curva")
ok(t9["caplet 5a ATM (pb)"].max() > 1.3 * t9["caplet 5a ATM (pb)"].min(), "no caplet de 5 anos, o
↪  mais caro passa de 1,3 vez o mais barato")
ok(t9.loc["Black-Karasinski", "caplet 2a +1,5 p.p. (pb)"] > t9.loc["Hull-White", "caplet 2a +1,5
↪  p.p. (pb)"],
   "fora do dinheiro, o BK (lognormal) dá mais valor à cauda alta que o HW (normal)")
ok(t9.loc["Black-Karasinski", "% r(5a) < 0"] == 0 and t9.loc["CIR++", "% r(5a) < 0"] == 0 and
↪  t9.loc["Ho-Lee", "% r(5a) < 0"] > 0,
   "BK e CIR++ nunca ficam negativos; Ho-Lee e HW sim")
```

```
σ do Ho-Lee calibrado ao caplet: 0,919 %
```

modelo	caplet 2a ATM (pb)	caplet 2a +1,5 p.p. (pb)	caplet 5a ATM (pb)	dp r(5a) (%)	% r(5a) < 0	corr Δy1a × Δy10a	maior erro de curva (em erros-padrão)
Ho-Lee	12,189	2,080	16,561	2,069	1,005	1,000	1,286
Hull-White	11,795	1,894	14,095	1,789	0,330	1,000	0,836
CIR++	12,238	3,085	12,038	1,613	0,000	1,000	1,502
Black-Karasinski	11,951	2,960	13,423	1,822	0,000	1,000	3,468
G2++	11,746	1,872	16,279	2,046	0,895	0,679	0,918
HJM 3 fatores	11,572	1,879	14,613	1,865	0,560	0,894	1,483

```
mercado (caplet 2a ATM): 11,873 pb
```



```
OK — todos os modelos acertam o caplet de calibração a menos de 5% (em trajetórias novas)
OK — e todos passam no teste da curva
OK — no caplet de 5 anos, o mais caro passa de 1,3 vez o mais barato
OK — fora do dinheiro, o BK (lognormal) dá mais valor à cauda alta que o HW (normal)
OK — BK e CIR++ nunca ficam negativos; Ho-Lee e HW sim
```

O que aprendemos.

- Todos passam no teste da curva e acertam o caplet de calibração. Fora disso, discordam.
- O **Ho-Lee**, sem reversão, espalha as taxas sem limite: caplets longos caros, muitas taxas negativas.
- **HW**, **CIR++** e **BK** têm a mesma reversão, mas formas diferentes de distribuição: normal, raiz quadrada e lognormal. Isso aparece no caplet fora do dinheiro, o smile do Módulo 18.
- **G2++** e **HJM** são os únicos que descorrelacionam prazos. Para um caplet isso pouco importa; para produtos que dependem de dois pontos da curva (*spread options*, swaps de inclinação, ALM com descasamento de prazos), é decisivo.
- Calibrar a uma curva e a um instrumento de vol não identifica o modelo. A escolha deve vir do uso: os instrumentos que o modelo vai precificar ou os riscos que vai medir precisam estar, ou ter parentes próximos, na calibração.

---

10. Síntese

Modelo	Ajuste da curva	Vol calibrada	Forte em	Fraco em
Vasicek	aproximado	σ	intuição, fórmulas	não reprecifica a curva
Ho-Lee	exato, fechado	σ	simplicidade	dispersão sem limite
Hull-White	exato, fechado	σ (e a)	fórmulas, árvore, padrão de mercado	juros negativos, 1 fator
CIR++	exato, fechado	σ	taxas positivas, fórmulas	só um fator; $\varphi < 0$ reabre negativos
Black-Karasinski	exato, numérico	σ	positividade, smile lognormal	sem fórmulas; não representa juros negativos
G2++	exato, fechado	σ, η (ρ, a, b)	descorrelação entre prazos	mais parâmetros para identificar
HJM	exato por construção	escala das vols (PCA)	curva inteira, n fatores	short rate não markoviana; simulação mais cara

O método da Seção 4, calibrar o drift nas próprias trajetórias, serve para qualquer modelo da forma $r = g(\alpha(t) + \text{ruído})$. É o que permite ajustar a curva em modelos sem fórmula fechada, mas a curva sai exata só na amostra usada. A conferência fora da amostra é obrigatória.

11. Exercícios

Exercício 1. Refaça o Laboratório 9 com o G2++ com $\rho = 0$ e com $\rho = -0{,}95$. O que acontece com a correlação entre 1 e 10 anos e com o caplet de 5 anos?

Solução

Com ρ mais negativo, os dois fatores se compensam na parte curta, e a correlação entre prazos cai. Recalibrada a escala ao caplet de 2 anos, a vol dos prazos longos (que dependem mais do fator lento) muda, e o caplet de 5 anos acompanha. É por isso que, na prática, o G2++ é calibrado a uma matriz de swaptions, que contém informação sobre correlação.

Exercício 2. Mostre que, no HJM com $\sigma(t,T) = \sigma e^{-a(T-t)}$, o drift é $\mu(t,T) = \frac{\sigma^2}{a} e^{-a(T-t)}(1 - e^{-a(T-t)})$, e que, para $a \to 0$, $\mu = \sigma^2(T-t)$, o drift do Ho-Lee.

Solução

$\int_t^T \sigma e^{-a(u-t)} du = \frac{\sigma}{a}(1 - e^{-a(T-t)})$; multiplicando por $\sigma e^{-a(T-t)}$ sai a fórmula. Para $a \to 0$, $\frac{1-e^{-a\tau}}{a} \to \tau$ e $e^{-a\tau} \to 1$.

Exercício 3. No Laboratório 5, calibre o BK com 2.000 trajetórias em vez de 8.000. Quanto varia o σ calibrado entre sementes diferentes?

Solução

O caplet de Monte Carlo tem erro-padrão de alguns por cento com 2.000 trajetórias; como o σ entra quase linearmente no preço, ele herda essa incerteza. Com números aleatórios comuns, o σ é uma função suave dos sorteios, mas muda de uma semente para outra: é preciso reportar a incerteza da calibração junto com o parâmetro.

Exercício 4. Simule o Hull-White sob a medida real, com preço de mercado do risco constante λ: $dr = (\theta(t) - ar + \lambda\sigma)\,dt + \sigma\,dW^P$. Com $\lambda = -0{,}3$, quanto muda a short rate esperada em 5 anos? Por que esse cenário não serve para precificar?

Solução

O drift extra $\lambda\sigma$ desloca a média de $r(t)$ em $\frac{\lambda\sigma}{a}(1 - e^{-at})$: cerca de $-1{,}2$ ponto percentual em 5 anos com esses números. Sob P, $\mathbb{E}[e^{-\int r}]$ não é mais o preço do título; o prêmio de risco está justamente nessa diferença. Cenários sob P servem para risco e ALM; preços exigem Q.

```
print(f"Módulo 19 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 19 concluído: 26 verificações aprovadas.
```

Parte V

Complemento — sorriso e volatilidade local

Capítulo 20

Árvores implícitas de Derman-Kani-Chriss: construção e visualização interativa

Introdução

O Capítulo 2 mostrou que o mercado não precifica opções com uma única volatilidade: cada strike e cada vencimento têm a sua, formando o smile. Os modelos com saltos e volatilidade estocástica explicam o smile acrescentando fatores de risco. Há uma alternativa mais direta: manter um único fator, mas deixar a volatilidade depender do preço do ativo e do tempo. É o modelo de **volatilidade local**.

Dupire mostrou que existe exatamente uma função de volatilidade local compatível com todos os preços de opções europeias observados, e deu a fórmula para obtê-la. A **árvore implícita** de Derman, Kani e Chriss é a versão discreta e construtiva dessa ideia: uma árvore trinomial cujas probabilidades são escolhidas, nó a nó, para reproduzir os preços de mercado. O resultado é uma árvore que se curva e se deforma de acordo com o smile.

O capítulo deduz as equações da árvore, discute as escolhas de espaço de estados que evitam probabilidades negativas, lê a volatilidade local nos nós e usa a árvore para precificar opções americanas e com barreira. Na versão on-line, um widget interativo permite construir e desenhar a árvore.

Fundamentos teóricos

Preços de opções e densidade neutra ao risco

Teorema 20.1 (Breeden-Litzenberger). *Se $C(K,T) = e^{-rT}\mathbb{E}^{\mathbb{Q}}[(S_T-K)^+]$ é duas vezes diferenciável em K, então*

$$\partial_K C(K,T) = -e^{-rT}\,\mathbb{Q}(S_T > K), \qquad \partial_{KK} C(K,T) = e^{-rT}\,q_T(K), \tag{20.1}$$

onde q_T é a densidade de S_T sob $\mathbb{Q}$ (Breeden and Litzenberger, 1978).

Demonstração. $C = e^{-rT}\int_K^\infty (s-K)q_T(s)\,ds$. Derivando sob o sinal de integral, $\partial_K C = -e^{-rT}\int_K^\infty q_T(s)\,ds$, e derivando de novo, $\partial_{KK}C = e^{-rT}q_T(K)$. ■

Corolário 20.2 (Ausência de arbitragem estática). *Uma superfície de preços é livre de arbitragem de borboleta se $\partial_{KK}C \geq 0$ e livre de arbitragem de calendário (sem dividendos) se $C(Ke^{r(T_2-T_1)}, T_2) \geq C(K, T_1)$ para $T_2 > T_1$.*

Os preços de mercado de um vencimento, portanto, determinam a lei marginal de S_T. A volatilidade local escolhe, entre todas as dinâmicas compatíveis com essas leis marginais, a única que é uma difusão markoviana em S.

Volatilidade local

Teorema 20.3 (Dupire). *Se $dS_t = rS_t\,dt + \sigma(S_t,t)S_t\,dW_t$ sob $\mathbb{Q}$ e a superfície de calls é suficientemente regular, então*

$$\sigma^2(K,T) = \frac{\partial_T C + rK\,\partial_K C}{\frac{1}{2}K^2\,\partial_{KK}C}. \tag{20.2}$$

Reciprocamente, dada uma superfície livre de arbitragem, (20.2) *define a única volatilidade local que a reproduz (Dupire, 1994).*

Esboço. A densidade q_T satisfaz a equação de Fokker-Planck, $\partial_T q = -\partial_K(rKq) + \frac{1}{2}\partial_{KK}(\sigma^2K^2q)$. Substitua $q = e^{rT}\partial_{KK}C$, integre duas vezes em K entre K e ∞ e use as condições de decaimento: obtém-se $\partial_T C = -rK\partial_K C + \frac{1}{2}\sigma^2K^2\partial_{KK}C$, que é (20.2). ■

Proposição 20.4 (Regra do dobro do skew). *Perto do dinheiro e para prazos curtos, a vol implícita é aproximadamente a média da vol local entre o preço corrente e o strike:*

$$\sigma_{\text{imp}}(K,T) \approx \frac{1}{\ln(K/S_0)}\int_{\ln S_0}^{\ln K} \sigma_{\text{loc}}(e^x)\,dx. \tag{20.3}$$

Logo, se σ_{loc} é linear em $\ln S$ com inclinação β, a vol implícita é linear em $\ln K$ com inclinação $\beta/2$ (Derman et al., 1996b; Gatheral, 2006).

Árvores implícitas

Definição 20.5 (Árvore trinomial implícita). Dado um espaço de estados com $2n+1$ nós no nível n, ordenados de forma decrescente, uma árvore implícita é a escolha de probabilidades $(p_i, 1-p_i-q_i, q_i)$ tal que (i) cada nó respeita o forward, $\mathbb{E}[S_{n+1} \mid S_n = S_i] = S_i e^{r\Delta t}$, e (ii) as calls e puts de vencimento t_{n+1} com strikes nos nós do nível $n+1$ são reproduzidas (Derman and Kani, 1994; Derman et al., 1996a).

Proposição 20.6 (Recursão de Derman-Kani-Chriss). *Sejam λ_i os preços de Arrow-Debreu do nível n, $F_i = S_i e^{r\Delta t}$ e $s_i > s_{i+1} > s_{i+2}$ os sucessores do nó i. Para os nós da metade superior,*

$$p_i = \frac{e^{r\Delta t}C(s_{i+1},t_{n+1}) - \sum_{j<i}\lambda_j\,(F_j - s_{i+1})}{\lambda_i\,(s_i - s_{i+1})}, \qquad q_i = \frac{F_i - p_i(s_i - s_{i+1}) - s_{i+1}}{s_{i+2} - s_{i+1}}, \tag{20.4}$$

e, para os da metade inferior, a fórmula simétrica com puts. Com as probabilidades, os preços de Arrow-Debreu do nível seguinte são $\Lambda_k = e^{-r\Delta t}\sum_i \lambda_i\,\pi_{i\to k}$.

A recursão é a versão discreta de (20.1): impor o preço da call com strike no nó s_{i+1} é impor a massa de probabilidade que a árvore coloca acima desse nó. Quando o espaço de estados não comporta a variância local pedida pelo smile, as fórmulas dão probabilidades fora de $[0,1]$, e o nó recebe um *override* que só respeita o forward.

Observação 20.7 (Limitação dinâmica). O modelo de volatilidade local reproduz por construção os preços de hoje, mas prevê uma dinâmica do smile em que, quando o preço cai, o smile se desloca na direção oposta à observada (Hagan et al., 2002). Produtos que dependem do smile futuro pedem modelos de volatilidade estocástica ou modelos mistos.

Roteiro do capítulo

Curso: Derivativos com Python. Último módulo do complemento, depois das árvores de juros do Módulo 18. Material de base: Derman, Kani e Chriss, "Implied Trinomial Trees of the Volatility Smile", *Journal of Derivatives* (1996); Derman e Kani, "Riding on a Smile", *Risk* (1994); Dupire, "Pricing with a Smile", *Risk* (1994); Clewlow e Strickland, *Implementing Derivatives Models* (1998), sobre árvores implícitas.

O que você vai conseguir fazer ao final

1. Explicar por que uma árvore com volatilidade constante não reproduz o smile, e o que é volatilidade local.
2. Montar o espaço de estados de uma árvore trinomial, inclusive com passos de tempo desiguais e com nós que acompanham o forward (a árvore "que curva").
3. Deduzir e implementar as equações de Derman-Kani-Chriss: probabilidades de transição a partir de calls e puts de mercado e preços de Arrow-Debreu por indução para a frente.
4. Diagnosticar probabilidades negativas, aplicar o *override* e escolher um espaço de estados que o evite.
5. Ler a volatilidade local na árvore, conferir a regra do "dobro do skew" e precificar americanas e barreiras.
6. Usar o widget interativo do final para construir e desenhar árvores, nó a nó.

Onde estamos

No Módulo 6 a árvore CRR tinha uma única volatilidade; no Módulo 18 a árvore de Hull-White foi ajustada à curva de juros pelos preços de Arrow-Debreu. Aqui juntamos as duas ideias para ações e índices: em vez de ajustar uma curva, a árvore é ajustada a uma **superfície de volatilidade implícita**. O resultado é um modelo de volatilidade local, $dS = rS\,dt + \sigma_{loc}(S,t)S\,dW$, em forma discreta. Todos os dados são sintéticos.

0. Preparação

```
# Preparação padrão (cores, gráficos, f3, tabela e ok): ver Apêndice A.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection
from scipy.stats import norm
from scipy.optimize import brentq
from IPython.display import display

def bs(S, K, T, r, sig, tipo="call"):
    """Black-Scholes sem dividendos."""
    d1 = (np.log(S / K) + (r + 0.5 * sig**2) * T) / (sig * np.sqrt(T))
    d2 = d1 - sig * np.sqrt(T)
    if tipo == "call":
        return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    return K * np.exp(-r * T) * norm.cdf(-d2) - S * norm.cdf(-d1)

def vol_impl(preco, S, K, T, r):
    """Volatilidade implícita de uma call por bissecção robusta (brentq)."""
    return brentq(lambda s: bs(S, K, T, r, s) - preco, 1e-4, 3.0)
```

1. O smile e a volatilidade local

Se o mercado usasse Black-Scholes, todas as opções de mesmo vencimento teriam a mesma volatilidade implícita. Não têm: em ações e índices, strikes baixos negociam com volatilidade maior (o skew negativo do Módulo 2), e a curva costuma ser convexa. Uma árvore CRR com um único σ acerta, no máximo, um strike por vencimento.

Vamos usar um smile sintético, função da *log-moneyness* em relação ao forward, $m = \ln(K/F_t)$ com $F_t = S_0 e^{rt}$:

$$\sigma_{imp}(K,t) = \max\left(\sigma_{ATM} + b\,m + c\,m^2,\ 1\%\right). \tag{20.5}$$

A pergunta de Dupire (1994) e de Derman e Kani (1994) é: existe uma difusão com volatilidade dependente só do preço e do tempo, $\sigma_{loc}(S,t)$, que reproduza **todos** esses preços ao mesmo tempo? A resposta é sim, e ela é única. Em tempo contínuo, sem dividendos,

$$\sigma^2_{loc}(K,T) = \frac{\partial_T C + rK\,\partial_K C}{\frac{1}{2}K^2\,\partial_{KK} C}, \tag{20.6}$$

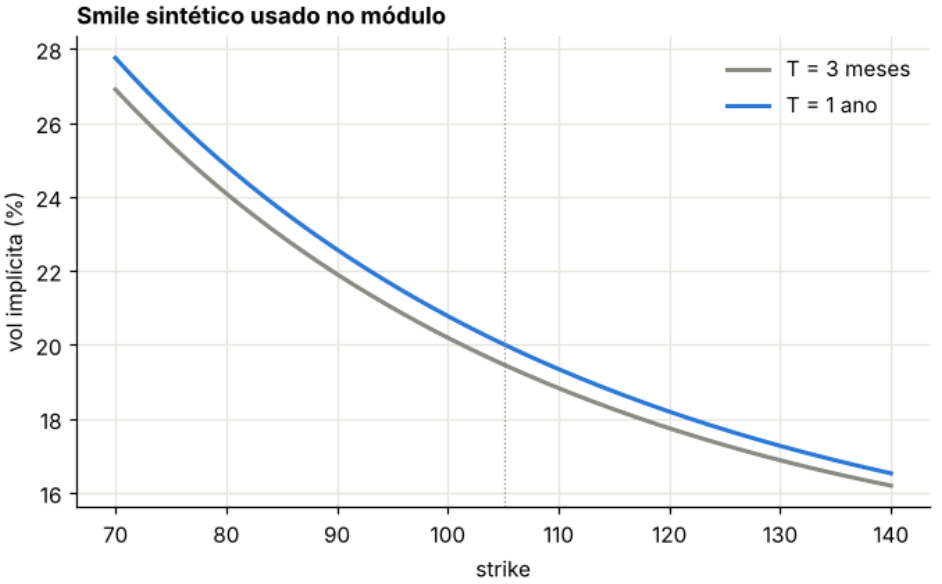
em que $C(K,T)$ é a superfície de preços de calls. A fórmula exige derivadas segundas de preços observados, o que é instável. A árvore implícita é a versão discreta e construtiva dessa mesma ideia: ela extrai a volatilidade local nó a nó, sem derivar numericamente a superfície.

```
S0, R = 100.0, 0.05


def fazer_smile(S0, r, s_atm=0.20, inclin=0.0, curv=0.0):
    """Smile quadrático em m = ln(K/F_t), com piso de 1%."""
    def smile(K, t):
        m = np.log(K / (S0 * np.exp(r * t)))
```

```
        return np.maximum(s_atm + inclin * m + curv * m**2, 0.01)
    return smile


SMILE = fazer_smile(S0, R, 0.20, -0.15, 0.10)
K = np.linspace(70, 140, 141)
fig, ax = plt.subplots()
for T, cor, rot in [(0.25, CINZA, "T = 3 meses"), (1.0, AZUL, "T = 1 ano")]:
    ax.plot(K, 100 * SMILE(K, T), color=cor, label=rot)
ax.axvline(S0 * np.exp(R), color=CINZA, lw=0.8, ls=":")
ax.set(title="Smile sintético usado no módulo", xlabel="strike", ylabel="vol implícita (%)")
ax.legend()
plt.show()
print("vol implícita em 1 ano: K = 80 →", f3(100 * SMILE(80, 1)), "%; K = 120 →", f3(100 *
↪  SMILE(120, 1)), "%")
ok(SMILE(80, 1) > SMILE(105.127, 1) > SMILE(130, 1) - 0.02, "skew negativo: strikes baixos mais
↪  caros em vol")
```




```
vol implícita em 1 ano: K = 80 → 24,843 %; K = 120 → 18,190 %
OK — skew negativo: strikes baixos mais caros em vol
```

2. Ingredientes de uma árvore implícita

Uma árvore trinomial recombinante tem três ingredientes, e a ordem importa:

1. **Espaço de estados.** Os preços dos nós em cada data t_n. No nível n há $2n+1$ nós, que numeramos **de cima para baixo**: $S_1 > S_2 > \cdots > S_{2n+1}$. O nó i liga-se aos nós i, $i+1$ e $i+2$ do nível seguinte, que chamaremos $s_i > s_{i+1} > s_{i+2}$.
2. **Probabilidades de transição.** Do nó i: p_i para cima, $1-p_i-q_i$ para o meio, q_i para baixo.
3. **Preços de Arrow-Debreu.** λ_i é o valor hoje de receber 1 se o nó i for atingido. Começa em $\lambda = 1$ na raiz e avança por indução para a frente, como no Módulo

18:

$$\Lambda_k = e^{-r\Delta t_n} \sum_i \lambda_i \, \pi_{i\to k}. \tag{20.7}$$

No método de Derman, Kani e Chriss (DKC), o espaço de estados é **escolhido livremente** e só depois as probabilidades são calculadas para reproduzir os preços de mercado. É essa liberdade que permite árvores com passos desiguais e árvores cujos nós "curvam".

Três espaços de estados

Seja $\Delta t_n = t_n - t_{n-1}$ o passo que chega ao nível n. Em log-preço, os nós do nível n são $x_{n,j} = c_n + j\,h_n$, $j = n, \dots, -n$, com espaçamento $h_n = f\,\sigma_{ATM}\sqrt{3\Delta t_n}$. O fator $\sqrt{3}$ é o da árvore trinomial padrão, em que as probabilidades com volatilidade constante seriam 1/6, 2/3 e 1/6; f é um fator de espaçamento. O centro c_n define o tipo:

- **centrado em S_0:** $c_n = \ln S_0$ (a árvore horizontal da figura de DKC com passos 0, 1, 3, 6);
- **centrado no forward:** $c_n = \ln S_0 + r t_n$. Em escala de preço, os ramos de cima se afastam e os de baixo se comprimem: é a árvore "que curva";
- **espaçamento pelo smile:** o centro segue o forward e cada degrau usa a vol implícita no próprio nível, $x_{j+1} = x_j + f\,\sigma_{imp}(e^{x_j}, t_n)\sqrt{3\Delta t_n}$. Onde o smile é alto, os nós se abrem.

```
def grade(T, N, tipo="uniforme"):
    """Datas da árvore. 'crescente' usa números triangulares: 0, 1, 3, 6, 10, ...
    ↪ (reescalados)."""
    k = np.arange(N + 1)
    return T * k * (k + 1) / (N * (N + 1)) if tipo == "crescente" else T * k / N


def espaco_estados(S0, r, tempos, modo, smile, fator=1.0):
    """Lista de arrays: nível n com 2n+1 preços em ordem decrescente."""
    niveis = [np.array([S0])]
    for n in range(1, len(tempos)):
        dt, t = tempos[n] - tempos[n - 1], tempos[n]
        Fc = S0 * np.exp(r * t)
        c = np.log(S0) + (0.0 if modo == "spot" else r * t)
        if modo != "smile":
            h = fator * smile(Fc, t) * np.sqrt(3 * dt)
            x = c + np.arange(n, -n - 1, -1) * h
        else:
            hmax = 4 * smile(Fc, t) * np.sqrt(3 * dt)    # trava contra degraus explosivos
            x = np.empty(2 * n + 1)
            x[n] = c
            for k in range(n - 1, -1, -1):
                x[k] = x[k + 1] + min(fator * smile(np.exp(x[k + 1]), t) * np.sqrt(3 * dt), hmax)
            for k in range(n + 1, 2 * n + 1):
                x[k] = x[k - 1] - min(fator * smile(np.exp(x[k - 1]), t) * np.sqrt(3 * dt), hmax)
        niveis.append(np.exp(x))
    return niveis


def desenhar_arvore(ax, tempos, niveis, arv=None, rotulos=False, cor="vl", escala_t=1.0,
↪ s_ref=None):
```

```
    """Desenha a árvore. Com arv, a espessura da aresta segue a probabilidade e a cor do nó, a vol
    ↪  local."""
    segs, larg = [], []
    for n in range(len(tempos) - 1):
        for i, s in enumerate(niveis[n]):
            pr = [1 / 3] * 3 if arv is None else [arv["p"][n][i], 1 - arv["p"][n][i] -
↪   arv["q"][n][i], arv["q"][n][i]]
            for k in range(3):
                if pr[k] > 1e-12:
                    segs.append([(tempos[n] * escala_t, s), (tempos[n + 1] * escala_t, niveis[n +
↪   1][i + k])])
                    larg.append(0.4 + 2.2 * pr[k])
    ax.add_collection(LineCollection(segs, linewidths=larg, colors=CINZA, alpha=0.55, zorder=1))
    for n, nivel in enumerate(niveis):
        tt = np.full(len(nivel), tempos[n] * escala_t)
        if arv is not None and n < len(tempos) - 1 and cor == "vl":
            v = (arv["vl"][n] - s_ref) / (0.5 * s_ref)
            cores = plt.cm.coolwarm(0.5 + 0.5 * np.clip(v, -1, 1))
            borda = np.where(arv["ovr"][n], MAGENTA, "#555555")
            ax.scatter(tt, nivel, s=26, c=cores, edgecolors=borda,
↪   linewidths=np.where(arv["ovr"][n], 1.6, 0.5), zorder=3)
        else:
            ax.scatter(tt, nivel, s=22, color="#d9d8d2" if arv is not None else AZUL,
↪   edgecolors="#555555", linewidths=0.5, zorder=3)
        if rotulos:
            for s in nivel:
                ax.annotate(f"{s:.2f}".replace(".", ","), (tempos[n] * escala_t, s),
↪   textcoords="offset points",
                            xytext=(0, 6), ha="center", fontsize=8)
    ax.set_xticks(np.asarray(tempos) * escala_t)
    ax.grid(axis="x", visible=False)
```

Laboratório 1 — as duas figuras de DKC

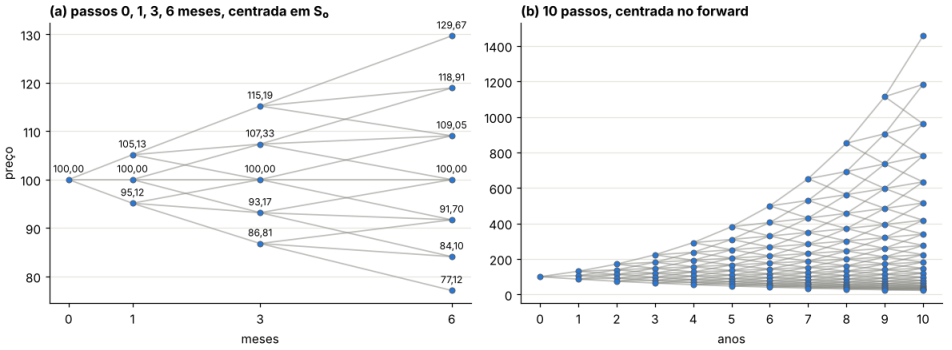
A figura (a) de Derman, Kani e Chriss mostra uma árvore com datas 0, 1, 3 e 6, centrada no preço inicial, com volatilidade ATM de 10% e juros de 3%. A figura (b) mostra uma árvore de 10 passos cujos nós sobem com o forward e se abrem em escala de preço. Reproduzimos as duas estruturas com o nosso gerador (os números exatos do artigo dependem de detalhes de espaçamento que não seguimos à risca).

```
TEMPOS_A = np.array([0, 1, 3, 6]) / 12
NIV_A = espaco_estados(100.0, 0.03, TEMPOS_A, "spot", fazer_smile(100.0, 0.03, 0.10), 1.0)
TEMPOS_B = grade(10.0, 10)
NIV_B = espaco_estados(100.0, 0.06, TEMPOS_B, "forward", fazer_smile(100.0, 0.06, 0.12), 1.0)

fig, axs = plt.subplots(1, 2, figsize=(11, 4.2))
desenhar_arvore(axs[0], TEMPOS_A, NIV_A, rotulos=True, escala_t=12)
axs[0].set(title="(a) passos 0, 1, 3, 6 meses, centrada em S₀", xlabel="meses", ylabel="preço")
desenhar_arvore(axs[1], TEMPOS_B, NIV_B)
axs[1].set(title="(b) 10 passos, centrada no forward", xlabel="anos")
plt.tight_layout()
plt.show()

for n, nivel in enumerate(NIV_A):
    print(f"t = {12 * TEMPOS_A[n]:.0f} mês(es):", "  ".join(f3(s) for s in nivel))
ok(all(len(nv) == 2 * n + 1 for niv in (NIV_A, NIV_B) for n, nv in enumerate(niv)), "cada nível n
↪   tem 2n + 1 nós")
```

```
ok(all(np.all(np.diff(nivel) < 0) for nivel in NIV_A + NIV_B), "nós em ordem estritamente
↪  decrescente")
ok(np.isclose(NIV_B[10][10], 100 * np.exp(0.06 * 10)), "na figura (b), o nó central segue o
↪  forward S₀e^{rt}")
ok((NIV_B[10][0] - NIV_B[10][10]) > 3 * (NIV_B[10][10] - NIV_B[10][-1]), "em escala de preço, os
↪  ramos de cima se abrem bem mais que os de baixo")
```




```
t = 0 mês(es): 100,000
t = 1 mês(es): 105,127  100,000  95,123
t = 3 mês(es): 115,191  107,327  100,000  93,173  86,812
t = 6 mês(es): 129,668  118,911  109,046  100,000  91,704  84,097  77,120
OK — cada nível n tem 2n + 1 nós
OK — nós em ordem estritamente decrescente
OK — na figura (b), o nó central segue o forward S₀e^{rt}
OK — em escala de preço, os ramos de cima se abrem bem mais que os de baixo
```

Repare na figura (a): os degraus crescem com o passo de tempo. O primeiro, de 1 mês, dá nós em torno de $\pm 5\%$; o de 3 para 6 meses é maior. Como o centro não acompanha o forward, toda a deriva de juros terá de ser absorvida pelas probabilidades. Na figura (b), o centro sobe com e^{rt} e o espaçamento constante em log vira, em preço, um leque assimétrico.

3. As equações de Derman-Kani-Chriss

Fixe o nível n, com nós S_i, preços de Arrow-Debreu λ_i conhecidos e forwards $F_i = S_i e^{r\Delta t}$, em que $\Delta t = t_{n+1} - t_n$. Queremos p_i e q_i para todos os nós. São $2(2n+1)$ incógnitas.

Condição do forward. Sob a medida neutra ao risco, o preço esperado no passo seguinte é o forward:

$$p_i\, s_i + (1 - p_i - q_i)\, s_{i+1} + q_i\, s_{i+2} = F_i \quad \Longleftrightarrow \quad p_i(s_i - s_{i+1}) + q_i(s_{i+2} - s_{i+1}) = F_i - s_{i+1}. \tag{20.8}$$

São $2n+1$ equações. As outras $2n+1$ vêm de opções de mercado.

Condição das calls. Tome a call com vencimento t_{n+1} e strike exatamente no nó s_{i+1}. Seu preço é a soma, sobre os nós do nível n, de λ_j vezes o valor descontado do payoff no passo seguinte. Para os nós acima de i ($j < i$), os três sucessores estão acima do strike, então o payoff é linear e sua esperança é $F_j - s_{i+1}$. Para o nó i, só o ramo de cima paga. Nós abaixo de i não pagam nada. Logo

$$e^{r\Delta t}\,C(s_{i+1},t_{n+1}) = \sum_{j<i}\lambda_j\,(F_j - s_{i+1}) + \lambda_i\,p_i\,(s_i - s_{i+1}), \tag{20.9}$$

$$p_i = \frac{e^{r\Delta t}C(s_{i+1},t_{n+1}) - \sum_{j<i}\lambda_j(F_j - s_{i+1})}{\lambda_i\,(s_i - s_{i+1})}. \tag{20.10}$$

Com p_i, a condição do forward dá q_i. A recursão começa no topo ($i = 1$, soma vazia) e desce.

Condição das puts. Para a metade de baixo, DKC usam puts com strike s_{i+1}, começando de baixo:

$$e^{r\Delta t}\,P(s_{i+1},t_{n+1}) = \sum_{j>i}\lambda_j\,(s_{i+1} - F_j) + \lambda_i\,q_i\,(s_{i+1} - s_{i+2}). \tag{20.11}$$

Por que trocar? Uma call com strike muito baixo está profundamente dentro do dinheiro: o seu preço é quase todo valor intrínseco, a soma $\sum_{j<i}\lambda_j(F_j - K)$ é enorme, e a diferença que sobra para determinar p_i seria pequena e cheia de erro de cancelamento. Usar sempre a opção fora do dinheiro evita isso. O nó central fica com as calls.

O preço das opções. Os preços C e P vêm do mercado, aqui do smile pela fórmula de Black-Scholes com a vol implícita do strike. A árvore não usa Black-Scholes como modelo; usa Black-Scholes só como tradutor de vol implícita em preço.

Volatilidade local no nó. Com as probabilidades, a variância do passo seguinte define a volatilidade local:

$$\sigma^2_{loc}(S_i,t_n)\,F_i^2\,\Delta t = p_i(s_i - F_i)^2 + (1 - p_i - q_i)(s_{i+1} - F_i)^2 + q_i(s_{i+2} - F_i)^2. \tag{20.12}$$

O *override* de DKC

Nada garante que as fórmulas acima deem probabilidades em $[0,1]$. Se o smile pede, num nó, uma variância maior do que os três sucessores conseguem oferecer, ou se o forward cai fora do intervalo entre eles, aparece $p_i < 0$, $q_i < 0$ ou $p_i + q_i > 1$. DKC então descartam a informação da opção naquele nó e escolhem probabilidades que só respeitam o forward. Se $s_{i+1} < F_i < s_i$,

$$p_i = \frac{1}{2}\left(\frac{F_i - s_{i+1}}{s_i - s_{i+1}} + \frac{F_i - s_{i+2}}{s_i - s_{i+2}}\right), \qquad q_i = \frac{1}{2}\,\frac{s_i - F_i}{s_i - s_{i+2}}, \tag{20.13}$$

e, se $s_{i+2} < F_i < s_{i+1}$,

$$p_i = \frac{1}{2}\,\frac{F_i - s_{i+2}}{s_i - s_{i+2}}, \qquad q_i = \frac{1}{2}\left(\frac{s_{i+1} - F_i}{s_{i+1} - s_{i+2}} + \frac{s_i - F_i}{s_i - s_{i+2}}\right). \tag{20.14}$$

A leitura é simples: é a média de duas árvores binomiais, cada uma ligando o nó a dois sucessores e respeitando o forward. O preço é perder o ajuste à opção daquele strike. Um override ocasional nas pontas, onde λ é desprezível, não incomoda; muitos overrides no miolo indicam um espaço de estados mal escolhido.

```
def override(F, a, b, c):
    """Probabilidades (p, q) de DKC que só respeitam o forward. a > b > c são os sucessores."""
    if F >= a:
        return 1.0, 0.0
    if F <= c:
```

```
        return 0.0, 1.0
    if F > b:
        return 0.5 * ((F - b) / (a - b) + (F - c) / (a - c)), 0.5 * (a - F) / (a - c)
    return 0.5 * (F - c) / (a - c), 0.5 * ((b - F) / (b - c) + (a - F) / (a - c))


def arvore_implicita(S0, r, tempos, smile, niveis):
    """Árvore trinomial implícita de Derman-Kani-Chriss (1996)."""
    N = len(tempos) - 1
    lam, P, Q, VL, OV = [np.array([1.0])], [], [], [], []
    for n in range(N):
        S, s, L = niveis[n], niveis[n + 1], lam[n]
        dt, t1 = tempos[n + 1] - tempos[n], tempos[n + 1]
        cr = np.exp(r * dt)
        F = S * cr
        m = len(S)
        p, q = np.zeros(m), np.zeros(m)
        with np.errstate(all="ignore"):
            soma = 0.0                                      # Σ_{j<i} λ_j (F_j − s_{i+1})
            for i in range(0, (m - 1) // 2 + 1):            # metade de cima e centro: calls
                K = s[i + 1]
                C = bs(S0, K, t1, r, smile(K, t1), "call")
                p[i] = (cr * C - soma) / (L[i] * (s[i] - K))
                q[i] = (F[i] - p[i] * (s[i] - K) - K) / (s[i + 2] - K)
                soma = np.sum(L[:i + 1] * (F[:i + 1] - s[i + 2]))   # já com o strike do próximo i
            soma = 0.0                                      # Σ_{j>i} λ_j (s_{i+1} − F_j)
            for i in range(m - 1, (m - 1) // 2, -1):        # metade de baixo: puts
                K = s[i + 1]
                Pp = bs(S0, K, t1, r, smile(K, t1), "put")
                q[i] = (cr * Pp - soma) / (L[i] * (K - s[i + 2]))
                p[i] = (F[i] + q[i] * (K - s[i + 2]) - K) / (s[i] - K)
                soma = np.sum(L[i:] * (s[i] - F[i:]))
        ov = ~(np.isfinite(p) & np.isfinite(q)) | (p < 0) | (q < 0) | (p + q > 1)
        for i in np.where(ov)[0]:
            p[i], q[i] = override(F[i], s[i], s[i + 1], s[i + 2])
        pm = 1 - p - q
        var = p * (s[:-2] - F)**2 + pm * (s[1:-1] - F)**2 + q * (s[2:] - F)**2
        L1 = np.zeros(m + 2)
        L1[:-2] += L * p
        L1[1:-1] += L * pm
        L1[2:] += L * q
        lam.append(L1 / cr)
        P.append(p); Q.append(q); VL.append(np.sqrt(var / (F**2 * dt))); OV.append(ov)
    return dict(S=niveis, lam=lam, p=P, q=Q, vl=VL, ovr=OV, t=np.asarray(tempos, float), r=r)


def preco_arvore(arv, payoff, americana=False, barreira=None):
    """Indução para trás. payoff recebe um array de preços; barreira = nível 'down-and-out'."""
    S, t, r = arv["S"], arv["t"], arv["r"]
    V = payoff(S[-1])
    if barreira is not None:
        V = np.where(S[-1] <= barreira, 0.0, V)
    for n in range(len(t) - 2, -1, -1):
        d = np.exp(-r * (t[n + 1] - t[n]))
        p, q = arv["p"][n], arv["q"][n]
        V = d * (p * V[:-2] + (1 - p - q) * V[1:-1] + q * V[2:])
        if americana:
            V = np.maximum(V, payoff(S[n]))
        if barreira is not None:
            V = np.where(S[n] <= barreira, 0.0, V)
```

```
    return V[0]


def massa_override(arv):
    """Maior fração de Arrow-Debreu, num mesmo nível, em nós com override."""
    return max((L * o).sum() / L.sum() for L, o in zip(arv["lam"], arv["ovr"]))
```

Laboratório 2 — smile plano: a árvore implícita vira uma árvore de volatilidade constante

Primeiro o caso em que sabemos a resposta. Com smile plano em 20%, a volatilidade local tem de ser 20% em todo nó, os preços de Arrow-Debreu do último nível têm de somar o fator de desconto, e o preço descontado tem de ser martingal: $\sum_k \Lambda_k S_k = S_0$.

```
T, N = 1.0, 40
t40 = grade(T, N)
plano = fazer_smile(S0, R, 0.20)
arv_p = arvore_implicita(S0, R, t40, plano, espaco_estados(S0, R, t40, "forward", plano, 1.0))

def vl_relevante(a, n0, n1):
    """Vol locais dos níveis n0..n1-1 em nós com peso de Arrow-Debreu relevante."""
    return np.concatenate([a["vl"][n][a["lam"][n] > 1e-4 * a["lam"][n].sum()] for n in range(n0,
    ↪  n1)])


print("vol local no nível 0:", f3(100 * arv_p["vl"][0][0]), "%; nível 2:", ", ".join(f3(100 * v)
↪  for v in arv_p["vl"][2]), "%")
vl_miolo = vl_relevante(arv_p, 10, N)
print("vol local dos níveis 10 a 39: mín", f3(100 * vl_miolo.min()), "% · máx", f3(100 *
↪  vl_miolo.max()), "%")
print("ΣΛ(T) =", f"{arv_p['lam'][-1].sum():.10f}", " e^{-rT} =", f"{np.exp(-R * T):.10f}")
print("ΣΛ·S(T) =", f"{(arv_p['lam'][-1] * arv_p['S'][-1]).sum():.10f}")
ok(sum(o.sum() for o in arv_p["ovr"]) == 0, "smile plano: nenhum override")
ok(np.allclose(vl_miolo, 0.20, atol=3e-3), "smile plano: a partir do nível 10, vol local de 20%
↪  (± 0,3 p.p.) nos nós com peso relevante")
ok(arv_p["vl"][0][0] > 0.22, "no primeiro passo, a vol local sai acima de 20%: três nós imitando
↪  uma lognormal")
ok(np.isclose(arv_p["lam"][-1].sum(), np.exp(-R * T), atol=1e-12), "ΣΛ no vencimento = fator de
↪  desconto")
ok(np.isclose((arv_p["lam"][-1] * arv_p["S"][-1]).sum(), S0, atol=1e-9), "preço descontado é
↪  martingal: ΣΛS = S₀")

err = [preco_arvore(arv_p, lambda x: np.maximum(x - k, 0)) - bs(S0, k, T, R, 0.20) for k in (80,
↪  100, 120)]
print("erro da call (árvore − BS) em K = 80, 100, 120:", ", ".join(f3(e) for e in err))
ok(max(abs(e) for e in err) < 0.1, "com 40 passos, calls fora dos nós a menos de 10 centavos de
↪  Black-Scholes")
```

```
vol local no nível 0: 23,517 %; nível 2: 21,642, 20,006, 20,089, 20,006, 21,642 %
vol local dos níveis 10 a 39: mín 19,996 % · máx 20,178 %
ΣΛ(T) = 0.9512294245  e^{-rT} = 0.9512294245
ΣΛ·S(T) = 100.0000000000
OK — smile plano: nenhum override
OK — smile plano: a partir do nível 10, vol local de 20% (± 0,3 p.p.) nos nós com peso relevante
OK — no primeiro passo, a vol local sai acima de 20%: três nós imitando uma lognormal
OK — ΣΛ no vencimento = fator de desconto
OK — preço descontado é martingal: ΣΛS = S₀
erro da call (árvore − BS) em K = 80, 100, 120: 0,001, 0,023, 0,061
OK — com 40 passos, calls fora dos nós a menos de 10 centavos de Black-Scholes
```

Os primeiros passos merecem atenção. Com um só nó na raiz e três sucessores, a árvore tem de reproduzir o preço contínuo de Black-Scholes da call com strike no nó central; uma distribuição de três pontos só consegue isso com variância maior, e a vol local do primeiro passo sai em torno de 23,5%. Com mais nós, a distribuição discreta se aproxima da lognormal e a vol local converge para 20%; o mesmo efeito, mais fraco, aparece nos nós da borda de cada nível, que têm peso desprezível. É um efeito de discretização, não de mercado: cai quando o primeiro passo encolhe, por exemplo com a grade crescente.

A conferência do martingal não é um acaso numérico. A soma $\sum_k \Lambda_k s_k$ é igual a $e^{-r\Delta t}\sum_i \lambda_i\, \mathbb{E}_i[s]$, e a condição do forward faz $\mathbb{E}_i[s] = F_i = S_i e^{r\Delta t}$. Logo $\sum_k \Lambda_k s_k = \sum_i \lambda_i S_i$, e assim por diante até a raiz. Vale **mesmo com overrides**, porque o override também respeita o forward.

4. Smile inclinado: calibração exata nos nós

Agora o smile da Seção 1, com skew $-0{,}15$ e convexidade $0{,}10$. Usamos o espaço de estados pelo smile com fator 1,3.

Há uma propriedade forte: **no vencimento, as calls com strike num nó são reproduzidas exatamente**, qualquer que seja o que aconteceu antes. O último passo da construção impõe essas equações diretamente, a partir dos λ do penúltimo nível. Pela paridade put-call, que vale na árvore porque o forward é respeitado, o mesmo vale para as puts. Só um override no último passo quebra a igualdade naquele strike.

```
FATOR = 1.3
arv = arvore_implicita(S0, R, t40, SMILE, espaco_estados(S0, R, t40, "smile", SMILE, FATOR))
SN = arv["S"][-1]
ov_ult = arv["ovr"][-1]

linhas, erros_nos = [], []
for k in range(1, 2 * N):
    Kk = SN[k]
    i = k - 1                          # o strike s_{i+1} é imposto pelo nó i do penúltimo nível
    if abs(np.log(Kk / (S0 * np.exp(R)))) > 0.35:
        continue
    pc = preco_arvore(arv, lambda x: np.maximum(x - Kk, 0))
    pm = bs(S0, Kk, T, R, SMILE(Kk, T))
    if not ov_ult[i]:
        erros_nos.append(abs(pc - pm))
    linhas.append({"strike (nó)": Kk, "vol mercado (%)": 100 * SMILE(Kk, T),
                   "vol árvore (%)": 100 * vol_impl(pc, S0, Kk, T, R), "override no último passo":
                    ↪  "sim" if ov_ult[i] else "não"})
tabela(pd.DataFrame(linhas)[::3])
print("maior erro de preço nos nós sem override:", f"{max(erros_nos):.2e}")
print("overrides:", sum(o.sum() for o in arv["ovr"]), "de", N * N, "nós · maior massa num nível:",
 ↪  f3(100 * massa_override(arv)), "%")
ok(max(erros_nos) < 1e-8, "calls com strike nos nós reproduzidas exatamente (erro < 1e-8)")
ok(abs((arv["lam"][-1] * SN).sum() - S0) < 1e-9, "martingal preservado com smile e overrides")
ok(massa_override(arv) < 0.05, "overrides concentrados nas pontas: menos de 5% da massa de
 ↪  Arrow-Debreu em qualquer nível")
```

	strike (nó)	vol mercado (%)	vol árvore (%)	override no último passo
0	145,446	16,184	16,184	não
3	120,778	18,111	18,111	não
6	97,902	21,119	21,119	não
9	77,007	25,638	25,638	não

```
maior erro de preço nos nós sem override: 2.52e-13
overrides: 774 de 1600 nós · maior massa num nível: 3,093 %
OK — calls com strike nos nós reproduzidas exatamente (erro < 1e-8)
OK — martingal preservado com smile e overrides
OK — overrides concentrados nas pontas: menos de 5% da massa de Arrow-Debreu em qualquer nível
```

Fora dos nós, o preço da árvore é uma interpolação linear do payoff entre dois nós, e aparece erro de discretização. Ele cai com N, mas não de forma monótona: depende de onde o strike cai em relação aos nós, o mesmo serrilhado da CRR no Módulo 6.

```
linhas = []
for n_ in (10, 20, 40, 80):
    tt = grade(T, n_)
    a_ = arvore_implicita(S0, R, tt, SMILE, espaco_estados(S0, R, tt, "smile", SMILE, FATOR))
    e = [abs(vol_impl(preco_arvore(a_, lambda x: np.maximum(x - k, 0)), S0, k, T, R) - SMILE(k,
↪   T)) for k in np.arange(85, 116, 2.5)]
    linhas.append({"N": n_, "erro máx. de vol (p.p.)": 100 * max(e), "erro médio (p.p.)": 100 *
↪   np.mean(e)})
conv = pd.DataFrame(linhas)
tabela(conv)
ok(conv["erro médio (p.p.)"].iloc[-1] < conv["erro médio (p.p.)"].iloc[0] / 3, "erro médio fora
↪   dos nós cai mais de 3 vezes de N = 10 para N = 80")
```

N	erro máx. de vol (p.p.)	erro médio (p.p.)
10	1,481	0,845
20	0,713	0,388
40	0,332	0,200
80	0,171	0,089

```
OK — erro médio fora dos nós cai mais de 3 vezes de N = 10 para N = 80
```

5. A escolha do espaço de estados

O espaço de estados não é neutro. Com a grade centrada em S_0, os nós do meio não acompanham o forward e o espaçamento é o mesmo em toda a altura. Onde o smile pede muita variância, nos strikes baixos, os três sucessores ficam perto demais e não há probabilidade positiva que produza a variância pedida. Onde pede pouca, nos strikes altos, a deriva de juros força uma variância mínima. Resultado: overrides no miolo da árvore e preços errados.

O laboratório compara, para um smile forte (skew −0,25, convexidade 0,20, 15 passos), os três espaços e alguns fatores.

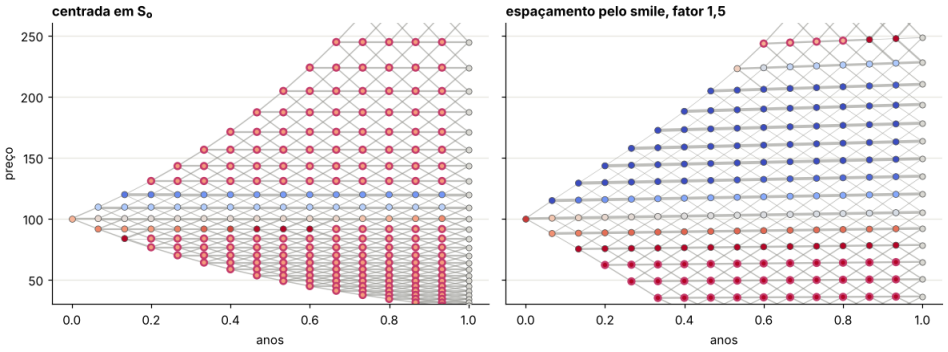
```
FORTE = fazer_smile(S0, R, 0.20, -0.25, 0.20)
t15 = grade(1.0, 15)
linhas = []
```

```
for modo, fator in [("spot", 1.0), ("forward", 1.0), ("forward", 1.5), ("smile", 1.0), ("smile",
↪  1.5)]:
    a_ = arvore_implicita(S0, R, t15, FORTE, espaco_estados(S0, R, t15, modo, FORTE, fator))
    e = [abs(vol_impl(preco_arvore(a_, lambda x: np.maximum(x - k, 0)), S0, k, 1.0, R) - FORTE(k,
↪  1.0)) for k in (85, 90, 95)]
    linhas.append({"espaço": modo, "fator": fator, "nós com override": int(sum(o.sum() for o in
↪  a_["ovr"])),
                   "massa máx. em override (%)": 100 * massa_override(a_), "erro de vol K≤95
                    ↪  (p.p.)": 100 * max(e)})
comp = pd.DataFrame(linhas)
tabela(comp)
ok(comp["massa máx. em override (%)"].iloc[-1] < comp["massa máx. em override (%)"].iloc[0] / 4,
   "espaço pelo smile com fator 1,5 reduz a massa em override a menos de 1/4 da grade centrada em
    ↪  S₀")
ok(comp["erro de vol K≤95 (p.p.)"].iloc[-1] < comp["erro de vol K≤95 (p.p.)"].iloc[0] / 3,
   "e corta o erro nos strikes baixos a menos de 1/3")

fig, axs = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
for ax, (modo, fator, tit) in zip(axs, [("spot", 1.0, "centrada em S₀"), ("smile", 1.5,
↪  "espaçamento pelo smile, fator 1,5")]):
    a_ = arvore_implicita(S0, R, t15, FORTE, espaco_estados(S0, R, t15, modo, FORTE, fator))
    desenhar_arvore(ax, t15, a_["S"], a_, s_ref=0.20)
    ax.set(title=tit, xlabel="anos", ylim=(30, 260))
    ax.set_xticks(t15[::3])
axs[0].set_ylabel("preço")
plt.tight_layout()
plt.show()
print("cor: vol local (azul abaixo, vermelho acima de 20%); borda magenta = override")
```

espaço	fator	nós com override	massa máx. em override (%)	erro de vol K≤95 (p.p.)
spot	1,000	173	42,193	2,821
forward	1,000	103	28,659	2,704
forward	1,500	115	16,693	0,702
smile	1,000	84	21,140	0,486
smile	1,500	97	7,574	0,768

```
OK — espaço pelo smile com fator 1,5 reduz a massa em override a menos de 1/4 da grade centrada em S₀
OK — e corta o erro nos strikes baixos a menos de 1/3
```




```
cor: vol local (azul abaixo, vermelho acima de 20%); borda magenta = override
```

À esquerda, a grade horizontal acumula overrides (bordas magenta) logo acima e

logo abaixo do miolo, onde ainda há probabilidade relevante. À direita, os nós se abrem para baixo, onde a vol implícita é maior, e os overrides recuam para regiões de pouca probabilidade (a tabela mostra a massa máxima caindo de 42% para menos de 8%). Essa é a recomendação prática de DKC: escolher o espaço de estados de modo que a variância pedida pelo smile caiba entre os sucessores.

6. Lendo a volatilidade local

Uma regra prática, devida a Derman, Kani e Zou, diz que a vol implícita de um strike K é aproximadamente a **média** da vol local no caminho entre S_0 e K:

$$\sigma_{imp}(K) \approx \frac{1}{\ln K - \ln S_0} \int_{\ln S_0}^{\ln K} \sigma_{loc}(x)\, dx. \tag{20.15}$$

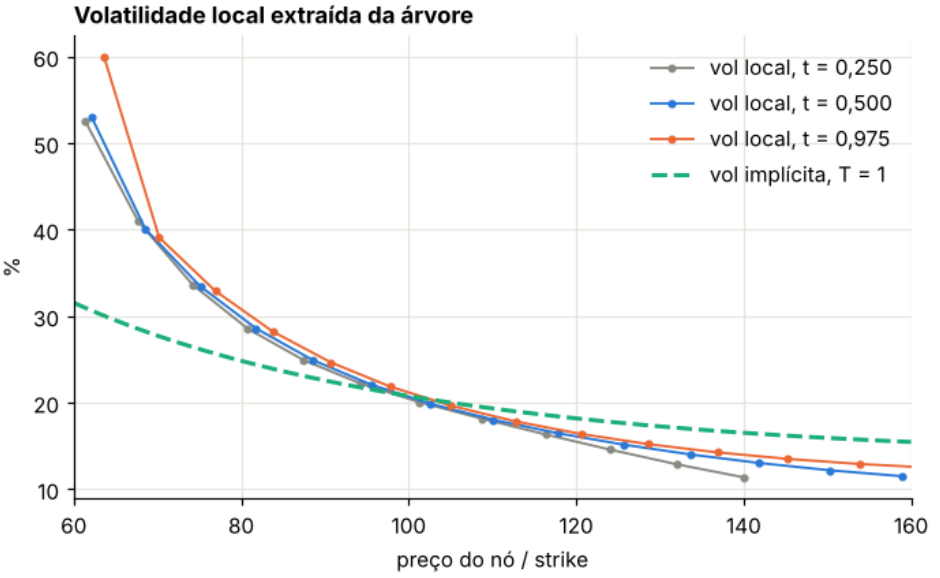
Se a vol local é linear em $x = \ln S$, com inclinação β, a média até K cresce com inclinação $\beta/2$. Logo a vol local é inclinada **duas vezes** mais que a vol implícita. A regra vale perto do dinheiro e para prazos curtos.

```
LINEAR = fazer_smile(S0, R, 0.20, -0.15, 0.0)
linhas = []
for Tq, Nq in [(0.25, 80), (1.0, 80)]:
    tq = grade(Tq, Nq)
    a_ = arvore_implicita(S0, R, tq, LINEAR, espaco_estados(S0, R, tq, "forward", LINEAR, 1.3))
    n = Nq // 4
    x = np.log(a_["S"][n] / (S0 * np.exp(R * tq[n])))
    sel = (np.abs(x) < 0.1) & ~a_["ovr"][n]
    beta = np.polyfit(x[sel], a_["vl"][n][sel], 1)[0]
    linhas.append({"prazo T": Tq, "data do nível": tq[n], "inclinação vol local": beta, "razão
↪  sobre −0,15": beta / -0.15})
regra = pd.DataFrame(linhas)
tabela(regra)
ok(regra["razão sobre −0,15"].between(1.6, 2.2).all(), "vol local cerca de duas vezes mais
↪  inclinada que a implícita (razão entre 1,6 e 2,2)")

fig, ax = plt.subplots()
for n, cor in [(10, CINZA), (20, AZUL), (39, LARANJA)]:
    sel = (arv["lam"][n] > 1e-4 * arv["lam"][n].max()) & ~arv["ovr"][n]
    ax.plot(arv["S"][n][sel], 100 * arv["vl"][n][sel], "o-", ms=3, lw=1.2, color=cor, label=f"vol
↪  local, t = {f3(t40[n])}")
KK = np.linspace(60, 160, 101)
ax.plot(KK, 100 * SMILE(KK, 1.0), color=VERDE, ls="--", label="vol implícita, T = 1")
ax.set(title="Volatilidade local extraída da árvore", xlabel="preço do nó / strike", ylabel="%",
↪  xlim=(60, 160))
ax.legend()
plt.show()
```

prazo T	data do nível	inclinação vol local	razão sobre −0,15
0,250	0,062	-0,294	1,957
1,000	0,250	-0,285	1,897

```
OK — vol local cerca de duas vezes mais inclinada que a implícita (razão entre 1,6 e 2,2)
```



A vol local é mais inclinada e, nas pontas, bem mais alta que a implícita. É por isso que as pontas da árvore são o lugar natural dos overrides: a variância pedida cresce mais depressa que o espaçamento.

7. Precificar na árvore implícita

A árvore calibrada serve para o que Black-Scholes com uma vol só não consegue: precificar, de forma consistente com todo o smile, contratos cujo valor depende da trajetória ou do exercício antecipado. Comparamos com uma árvore trinomial de vol constante igual à vol ATM. As duas concordam na call ATM e discordam no resto.

```
plano_atm = fazer_smile(S0, R, float(SMILE(S0 * np.exp(R), 1.0)))
arv_c = arvore_implicita(S0, R, t40, plano_atm, espaco_estados(S0, R, t40, "smile", SMILE, FATOR))

contratos = {
    "put europeia K = 90": dict(payoff=lambda x: np.maximum(90 - x, 0)),
    "put americana K = 90": dict(payoff=lambda x: np.maximum(90 - x, 0), americana=True),
    "put americana K = 100": dict(payoff=lambda x: np.maximum(100 - x, 0), americana=True),
    "call europeia K = 100": dict(payoff=lambda x: np.maximum(x - 100, 0)),
    "call down-and-out K = 100, B = 85": dict(payoff=lambda x: np.maximum(x - 100, 0),
    ↪  barreira=85.0),
    "call europeia K = 110": dict(payoff=lambda x: np.maximum(x - 110, 0)),
}
linhas = []
for nome, kw in contratos.items():
    linhas.append({"contrato": nome, "vol constante (ATM)": preco_arvore(arv_c, **kw), "árvore
↪  implícita": preco_arvore(arv, **kw)})
prec = pd.DataFrame(linhas)
prec["diferença"] = prec["árvore implícita"] - prec["vol constante (ATM)"]
tabela(prec)

pe = prec.set_index("contrato")["árvore implícita"]
```

```
ok(pe["put americana K = 90"] >= pe["put europeia K = 90"], "americana ≥ europeia na árvore
↪  implícita")
ok(prec.loc[0, "diferença"] > 0 and prec.loc[5, "diferença"] < 0,
   "com skew negativo, a put fora do dinheiro fica mais cara e a call fora do dinheiro mais
   ↪  barata")
ok(abs(pe["put europeia K = 90"] - bs(S0, 90, 1.0, R, SMILE(90, 1.0), "put")) < 0.05,
   "a put europeia de K = 90 sai da árvore implícita a menos de 5 centavos do preço de mercado")
```

contrato	vol constante (ATM)	árvore implícita	diferença
put europeia K = 90	2,350	3,062	0,713
put americana K = 90	2,512	3,176	0,664
put americana K = 100	6,181	6,337	0,156
call europeia K = 100	10,551	10,837	0,286
call down-and-out K = 100, B = 85	10,370	10,416	0,046
call europeia K = 110	6,157	5,911	-0,246

```
OK — americana ≥ europeia na árvore implícita
OK — com skew negativo, a put fora do dinheiro fica mais cara e a call fora do dinheiro mais barata
OK — a put europeia de K = 90 sai da árvore implícita a menos de 5 centavos do preço de mercado
```

A call com barreira baixa pede uma leitura cuidadosa. Em valor absoluto ela quase não muda, mas isso esconde dois efeitos opostos: a call europeia de strike 100 já é mais cara na árvore implícita (a vol de mercado em $K = 100$ é maior que a ATM-forward), e a barreira tira dela uma fatia maior. A medida limpa é a fração do valor da call que a barreira destrói.

```
fracao = 1 - prec.set_index("contrato").loc["call down-and-out K = 100, B = 85"] /
↪  prec.set_index("contrato").loc["call europeia K = 100"]
print("fração do valor da call europeia perdida com a barreira:")
print("  vol constante:", f3(100 * fracao["vol constante (ATM)"]), "%   árvore implícita:",
↪  f3(100 * fracao["árvore implícita"]), "%")
ok(fracao["árvore implícita"] > fracao["vol constante (ATM)"], "a barreira destrói uma fração
↪  maior do valor na árvore implícita")
```

```
fração do valor da call europeia perdida com a barreira:
  vol constante: 1,714 %   árvore implícita: 3,884 %
OK — a barreira destrói uma fração maior do valor na árvore implícita
```

A vol local alta perto de 85 torna mais provável tocar a barreira do que numa árvore de vol constante. Duas árvores que concordam na call ATM dão preços diferentes para o contrato com barreira: é **risco de modelo**, e só a árvore implícita é consistente com as puts de strike baixo que o mercado está cotando.

8. Widget interativo: construa e desenhe a sua árvore

O widget abaixo usa **o mesmo algoritmo** das Seções 2 a 5, reescrito em JavaScript para rodar direto no navegador: funciona na página do livro, sem kernel, e também no Jupyter e no Colab. Mexa nos controles e a árvore é reconstruída na hora.

- **Botões de cenário.** “Figura (a)” monta a árvore de passos 0, 1, 3 e 6 meses com rótulos de preço; “Figura (b)” monta a árvore de 10 passos que curva com o forward; “Smile forte” mostra os overrides de uma grade mal escolhida, e “espaço

ajustado" mostra a correção.

- **Passe o mouse num nó** para ver o preço, o preço de Arrow-Debreu, o forward, as três probabilidades e a vol local.
- A espessura das arestas é proporcional à probabilidade; a cor do nó pode mostrar a vol local ou o preço de Arrow-Debreu; bordas magenta marcam overrides.
- Abaixo da árvore: as conferências da Seção 3 ($\Sigma\lambda$ e martingal), a tabela de reprecificação nos nós e o gráfico do smile contra a vol local.

Experimentos sugeridos: (1) com "Volatilidade constante", aumente o skew e veja a vol local se abrir com o dobro do skew; (2) em "Smile forte", troque o espaço de estados e o fator até os overrides sumirem do miolo; (3) mude a grade para "crescente" e veja os degraus crescerem com o passo.

Widget interativo. Na versão on-line deste capítulo, esta célula mostra um construtor interativo da árvore implícita, que roda no próprio navegador. Acesse `https://waltercneto.github.io/XRiscs/20_arvores_implicitas.html`.

Conferência: o JavaScript do widget contra o Python

Se o Node.js estiver instalado, a célula abaixo roda o núcleo numérico do widget e compara os preços de Arrow-Debreu e uma put americana com os desta página. Sem Node.js, ela só avisa e segue.

```
import json, shutil, subprocess

if shutil.which("node"):
    nucleo = _WIDGET.split("/*NUCLEO-INICIO*/")[1].split("/*NUCLEO-FIM*/")[0]
    js = nucleo + """
var sm=fazerSmile(100,0.05,0.2,-
↪  0.15,0.1),t=grade(1,40,"uniforme"),niv=espacoEstados(100,0.05,t,1.3,"smile",sm),a=arvoreImplicita(100,0.05,t,sm,n
console.log(JSON.stringify({lam:a.lam[40],put:precoArvore(a,function(s){return
↪  Math.max(90-s,0)},true)}));"""
    saida = json.loads(subprocess.run(["node", "-e", js], capture_output=True, text=True,
↪  check=True).stdout)
    dif_l = np.max(np.abs(np.array(saida["lam"]) - arv["lam"][-1]))
    dif_p = abs(saida["put"] - pe["put americana K = 90"])
    print(f"maior diferença em λ: {dif_l:.1e} · diferença na put americana: {dif_p:.1e}")
    print("JavaScript e Python coincidem." if max(dif_l, dif_p) < 1e-10 else "ATENÇÃO: JavaScript
    ↪  e Python divergem.")
else:
    print("Node.js não encontrado: conferência pulada (o widget continua funcionando no
    ↪  navegador).")
```

```
maior diferença em λ: 4.1e-15 · diferença na put americana: 2.2e-15
JavaScript e Python coincidem.
```

9. Cuidados de implementação

- **Ordem dos nós.** As fórmulas de DKC supõem os nós em ordem decrescente e o nó i ligado a i, $i+1$, $i+2$. Inverter a ordem troca o papel de calls e puts e é fonte comum de erro.

- **A soma acumulada muda de strike.** Em $\sum_{j<i} \lambda_j (F_j - s_{i+1})$, o strike é o do nó atual; não dá para acumular a soma do passo anterior sem corrigir o strike.
- **Preços de opções consistentes.** Se o smile de entrada tiver arbitragem (borboleta ou calendário), a árvore acusa com probabilidades negativas em série. O override esconde o sintoma, mas não corrige a superfície.
- **Overrides não são de graça.** Cada override abandona uma opção de mercado. Reporte sempre a massa de Arrow-Debreu em override, como fizemos, e não só a contagem.
- **Passos desiguais.** A grade crescente economiza nós em prazos longos, mas degraus que crescem muito de um nível para o outro afastam o nó do seu sucessor do meio e geram overrides.

10. Síntese

etapa	o que se escolhe	o que se impõe
espaço de estados	datas, centro e espaçamento	2n + 1 nós por nível, recombinação
probabilidades	—	forward e calls (em cima) ou puts (embaixo) com strike nos nós
Arrow-Debreu	—	indução para a frente, como no Módulo 18
override	regra de DKC	só o forward, quando a opção não cabe
saída	—	vol local em cada nó, preços consistentes com o smile

A árvore implícita é o modelo de volatilidade local em forma discreta. Ela reproduz por construção os preços de hoje, mas descreve mal a **dinâmica** do smile: quando o preço cai, a vol local prevê que o smile se desloque de um jeito que o mercado em geral não confirma. Por isso, para produtos sensíveis a vol futura (cliquets, opções sobre variância), entram os modelos de volatilidade estocástica dos Módulos 11 a 16.

11. Exercícios

Exercício 1. Mostre que, se em todos os nós vale a condição do forward, então $\sum_k \Lambda_k^{(n)} S_k^{(n)} = S_0$ em qualquer nível, com ou sem overrides.

Solução

Por indução. $\sum_k \Lambda_k s_k = e^{-r\Delta t} \sum_k \sum_i \lambda_i \pi_{i\to k} s_k = e^{-r\Delta t} \sum_i \lambda_i F_i = \sum_i \lambda_i S_i$. Na raiz, $\lambda = 1$ e $S = S_0$. O override também satisfaz o forward, por isso a identidade não depende dele.

Exercício 2. Troque, no código, as puts da metade de baixo por calls (use calls em todos os nós). Com o smile da Seção 4 e $N = 40$, o que acontece com o número de overrides? Por quê?

Solução

Os overrides aumentam na parte de baixo. Para strikes baixos, a call está muito dentro do dinheiro e $e^{r\Delta t}C - \sum_{j<i} \lambda_j (F_j - K)$ é a diferença de dois números grandes e quase iguais. O pequeno resto, dividido por λ_i (também pequeno nas pontas), amplifica o erro numérico e o erro de discretização acumulado, e as probabilidades saem fora de $[0, 1]$.

Exercício 3. No widget, escolha "Figura (a)" e aumente o skew para $-0{,}5$. Em que nós surgem os overrides e por quê?

Solução

Com skew forte e só 3 passos, a variância pedida nos nós de baixo supera a que três sucessores tão próximos conseguem dar. No topo, a vol local cai muito e a deriva de juros, que a grade centrada em S_0 não absorve, impõe uma variância mínima. Trocar o espaço de estados para "centrado no forward" ou "pelo smile" alivia os dois lados.

Exercício 4. Precifique a call down-and-out da Seção 7 com $N = 20, 40, 80$. O preço converge de forma suave? Como melhorar?

Solução

Não: o preço oscila porque a barreira efetiva é o primeiro nível de nós abaixo de 85, que muda com N. A correção clássica é posicionar um nível de nós exatamente na barreira (o espaço de estados é livre em DKC, então basta ajustar o centro ou o espaçamento) ou interpolar entre duas árvores com barreiras efetivas acima e abaixo de B.

```
print(f"Módulo 20 concluído: {_checks['n']} verificações aprovadas.")
```

```
Módulo 20 concluído: 22 verificações aprovadas.
```

Apêndice A

Preparação padrão dos laboratórios

Todos os capítulos começam com a mesma célula de preparação: cores, estilo dos gráficos, a função `f3` (três casas decimais no padrão brasileiro), a função `tabela` e a função `ok`, que interrompe o notebook se uma verificação falhar. Ela aparece aqui uma vez; nos capítulos, só as partes específicas de cada um.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, poisson
from scipy.optimize import brentq, least_squares
from IPython.display import display

AZUL, LARANJA, VERDE, CINZA = "#2a78d6", "#eb6834", "#1baf7a", "#8a8a84"

plt.rcParams.update({
    "figure.figsize": (7.2, 4.0), "figure.dpi": 110,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.color": "#e6e5df", "grid.linewidth": 0.8,
    "axes.axisbelow": True, "axes.titlesize": 11, "axes.titleweight": "bold",
    "axes.titlelocation": "left", "lines.linewidth": 2.0, "legend.frameon": False,
    "font.size": 10,
})


def f3(x):
    """Formata com três casas decimais no padrão brasileiro."""
    return f"{x:,.3f}".replace(",", "§").replace(".", ",").replace("§", ".")


def tabela(df):
    """Exibe um DataFrame com os números reais em três casas decimais."""
    display(df.map(lambda v: f3(v) if isinstance(v, (float, np.floating)) else v))


_checks = {"n": 0}


def ok(condicao, mensagem):
    """Verificação do laboratório: interrompe o notebook se a conta não fechar."""
```

```
    assert bool(condicao), f"FALHOU: {mensagem}"
    _checks["n"] += 1
    print(f"OK — {mensagem}")


print("Ambiente pronto. Exemplo de formato:", f3(1234.5678))
```

Apêndice B

Erratas e notas sobre o livro de Hilpisch

Esta página junta, módulo a módulo, as erratas e observações registradas nos notebooks. Cada item foi conferido numericamente no laboratório correspondente. O livro de referência é *Derivatives Analytics with Python* (Y. Hilpisch, Wiley, 2015); as seções e equações citadas são as do livro.

Página gerada automaticamente por `scripts/gerar_erratas.py`.

Módulo 2 — Stylized facts e volatilidade implícita

Notebook: 02_fatos_estilizados

Local	Problema
Comentário 4 após a saída da GBM	cita p-valor conjunto de 0,426, enquanto a saída impressa mostra 0,374
§3.4.2, contagem de saltos	escreve $P(r_n > -0{,}05)$; deveria ser $+0{,}05$
§3.4.2, contagem de saltos	as probabilidades informadas (2,911 e 3,402 por 10.000 dias) não fecham com $\hat{\mu}$ e $\hat{\sigma}$ impressos, que dão 1,267 e 1,542 por 10.000 dias
§3.5.2, primeira bolha	diz que calls OTM têm implícita mais alta; a Figura 3.11 mostra isso para calls ITM (puts OTM)

Módulo 3 — A medida neutra ao risco em tempo discreto

Notebook: 03_neutralidade_discreto

Local	Problema
Defs. 19 e 25	admissibilidade com $V_t \geq -\alpha$ torna "arbitragem" qualquer estratégia alavancada com ganho esperado positivo; o correto (Harrison-Pliska) é $V_t \geq 0$
Prova do Lema 1	"$u < t$" na definição de ψ deveria ser "$u > t$"
Prova da Prop. 1	a soma usa $\beta_{t+1}S_{t+1}$; o correto é $\beta_{t-1}S_{t-1}$
Prova da Prop. 1	"trading strategy $\zeta \in T$" deveria ser ϕ
Nota 13	refere "condição 3 da definição 3"; o certo é definição 1 (álgebra)

Módulo 4 — Cálculo estocástico: movimento browniano, Itô e Girsanov

Notebook: 04_tempo_continuo

Local	Comentário
Justificativa de (4.1)	a esperança da integral é zero só se V for martingale verdadeiro; admissibilidade garante apenas $\mathbb{E}^Q[V_T] \leq V_0$
Teorema 3	a medida é martingale local (sigma-martingale, no caso mais geral), e não martingale, para S não limitado
Def. 37 (arbitragem contínua)	exige $V_T \geq 0$, corrigindo a Def. 25 do caso discreto
Geral	o teorema de Girsanov não é tratado; o livro passa direto à dinâmica sob Q

Módulo 5 — Black-Scholes-Merton: a EDP, a fórmula e as gregas

Notebook: 05_bsm

Local	Problema
Passagem para (5.6)	$r(V - \partial V/\partial S)$ deveria ser $r(V - S\,\partial V/\partial S)$
Equação (5.6)	falta o fator rS no primeiro termo: o correto é $rS\,\partial V/\partial S$
§5.2.2	a carteira $\Pi = V - \Delta S$ não é autofinanciada; a dedução usa Δ fixo no intervalo
§5.2.3, item 1	"$K > 100$ for the put, OTM" deveria ser $K < 100$

Módulo 6 — A árvore binomial de Cox-Ross-Rubinstein

Notebook: 06_crr

Local	Comentário
Nota da Tabela 5.1	diz “put”; a tabela é de call
§5.4, após a Figura 5.8	atribui a oscilação a “parametrização diferente”; o motivo é a paridade de M em relação à posição de K

Módulo 7 — Métodos de Fourier I: da função característica à fórmula de Lewis

Notebook: 07_fourier_lewis

Local	Comentário
Prova da Prop. 6	a expressão $(z-i/2)^2-(z-i/2)i=z^2-2zi+1/4$ está errada; o denominador correto, com a reta em $u_i=1/2$, é $z^2+1/4$. A fórmula (6.5) está certa
Geral	a existência de ϕ em argumentos complexos exige uma faixa de analiticidade, que o texto não discute

Módulo 8 — Métodos de Fourier II: Carr-Madan, FFT e convolução

Notebook: 08_fourier_fft

Local	Problema
§6.5.1	e^{iux} está expandida incorretamente: o fator é $e^{-u_i x}$ e a oscilação usa apenas u_r
Def. 43	as somas de potências devem começar em $j=0$
Def. 43	potências positivas de z_n giram em sentido horário
Def. 44	a inversa da DFT precisa do fator $1/n$
§6.6.2, equação (6.13)	a lei $\ln(1+J)\sim N(\ln(1+\mu_J)-\delta^2/2,\delta^2)$ não é compatível com $r_J=\lambda(e^{\mu_J+\delta^2/2}-1)$ e com (6.14), que correspondem a $\ln(1+J)\sim N(\mu_J,\delta^2)$

Local	Problema
§6.6.3	o vetor $qv = (0{,}5034;\ 0{,}4966; \dots)$ é incompatível com $q = 0{,}5378$
Equação (6.6)	falta tomar a parte real
§6.6.1, precisão da FFT	os cinco dígitos relatados vêm do ramo OTM do script, de cauda lenta; não são um limite da FFT

Módulo 9 — Opções americanas por simulação: Longstaff-Schwartz e o problema dual

Notebook: 09_americanas_lsm

Local	Comentário
Tabelas 7.1 e 7.2	a referência (árvore de 500 passos) é americana; os algoritmos estimam bermudanas de M datas. Boa parte da "diferença" reportada, sobretudo no condor, é a diferença entre as duas
Tabela 7.2, nota	diz "American put option"; a tabela é do short condor
Tabela 7.1 e §7.4.1	o espaçamento entre LSM e dual é atribuído ao algoritmo; ele vem sobretudo do ruído da simulação aninhada ($J = 50$), que infla o limite superior
Equações (7.8) a (7.12)	o martingale deve ser expresso em unidades descontadas, como h; o texto mistura $B_0(t)[h_t - \mathcal{M}_t]$ com um $\mathcal{M}^*$ formado por valores não descontados
Equação (7.11)	o máximo passa de $t \in \{\Delta t, \dots, T\}$ para $t \in \{0, \dots, T\}$ sem comentário

Módulo 10 — Calibração ao mercado com o modelo de saltos de Merton

Notebook: 10_merton_mercado

Local	Comentário
§8.2	a lei $\ln(1+J) \sim N(\ln(1+\mu_J) - \delta^2/2, \delta^2)$ não é a usada em r_J, em (8.2) e nos scripts, que correspondem a $\ln(1+J) \sim N(\mu_J, \delta^2)$

Local	Comentário
§8.5, saída do script	os valores “Int” usam integração até $z = 50$, curta demais para $T = 0{,}22$; são os valores FFT que estão corretos
Equações (8.5) e (8.6)	o salto é somado, e y saltos no mesmo passo usam um único tamanho multiplicado por y; o esquema tem viés
§8.5, saída do script	a diferença entre simulação e fórmula combina esse viés com ruído de uma única semente
Script 8.7.5	comenta “T = 0.22 # shortest maturity”, mas o prazo mais curto é de 17 dias (0,047 ano)

Módulo 11 — O modelo BCC97: volatilidade estocástica, saltos e juros estocásticos

Notebook: 11_bcc97

Local	Comentário
§9.2	a lei do salto não corresponde a r_J nem a (9.11); o correto é $J \sim N(\mu_J, \delta^2)$ para o log-salto
Eq. (9.8)	$\partial^2 C/\partial S$ deveria ser $\partial^2 C/\partial S^2$
Eq. (9.8)	faltam os fatores v e r nos termos $\frac{1}{2}\sigma_v^2\,\partial^2 C/\partial v^2$ e $\frac{1}{2}\sigma_r^2\,\partial^2 C/\partial r^2$
Prova da Prop. 8	o salto entra como $\partial A/\partial S \cdot jdN$, aproximação de primeira ordem; o certo é a diferença exata, como a própria (9.8) usa
Prova da Prop. 8	na primeira linha da expansão, $\partial A/\partial t$ aparece dentro do fator $\frac{1}{2}$
§9.5.2	o uso de $r_{0,T} = -\ln B_0(T)/T$ é uma aproximação, não o tratamento de juros estocásticos do artigo original
Eq. (9.10)	a forma usada é a estável, com $c_2 = -d$; ponto positivo do livro

Módulo 12 — Monte Carlo: discretização e redução de variância

Notebook: 12_monte_carlo

Onde	O quê
§10.6.3 e §10.6.4, processo do índice	usa `v[t]` (fim do passo) em vez de `v[t-1]`; cria o drift espúrio $\frac{1}{2}\rho\sigma_v$ por unidade de tempo em $\ln S$, que não diminui com Δt
Tabela 10.1 e Figura 10.4	o salto de precisão atribuído ao moment matching é, em sua maior parte, a correção de drift (`bias`) desfazendo o problema acima; com `v[t-1]` não há salto
§10.3, lista de esquemas	"Truncation" coincide com "Absorption" e "Simple Reflection" com "Reflection": são cinco esquemas, não sete
§10.2, Figuras 10.2 e 10.3	a referência em $t > 0$ é $P(\mathbb{E}[r_t])$, mas a média de Monte Carlo estima $\mathbb{E}[P(r_t)]$; a diferença (Jensen) é da ordem do viés do Euler que se quer mostrar
§10.6.2, esquema exato	`np.ramdom.chisquare` (erro de digitação)
§10.6.4, correlação	ρ posto entre juros e índice, e não entre variância e índice; o controle europeu disfarça o efeito
§10.4, controle	a média do controle usa Heston com taxa constante $\bar{r}$, uma aproximação (Módulo 11), pequena para prazos de até seis meses
§10.2	a transição por qui-quadrado não central é clássica (Feller, 1951; Cox, Ingersoll e Ross, 1985); Broadie e Kaya (2006) a usam na simulação exata do Heston
Scripts	Python 2 (`xrange`, `print` sem parênteses; `I / 2` vira real no Python 3)

Módulo 13 — Calibração I: a curva de juros no CIR e as opções no Heston

Notebook: 13_calibracao_cir_heston

Onde	O quê
Exemplo 4 (§11.2.1)	o limite é $\nu < 1/2$, não $1/4$; o intervalo de preços é $]4, 8[$, e não $]4, 9[$; e em $\nu = 1/4$ o preço seria 6, não "$1 + 8 = 9$"
Eq. (11.6) e código da §11.6.1	falta o fator 2 no primeiro termo: o correto é $2\kappa_r\theta_r(e^{\gamma t} - 1)/D$; o θ_r calibrado sai dobrado
§11.6.1, ponto de partida	com a fórmula do livro, o Nelder-Mead a partir de (1; 0,02; 0,1) pode parar longe do ajuste

Onde	O quê
§11.3.2, dados	a Eonia de 01/10/2014 (−0,032%) é usada como r_0 do CIR, que não admite taxas negativas
§11.3.2, contagem	o texto chama a Euribor de “30/360”, mas o código usa dias corridos/360 (real/360, que é a convenção da Euribor); os prazos das opções usam /365
Eq. (11.7)	$B_t(T) = a\, e^{-b\,\mathbb{E}[r_t]}$ não é o preço esperado do título em t (desigualdade de Jensen; ver Módulo 12)
§11.4.1, H_1	o logaritmo impresso traz $1 - c_1 e^{c_3 T}$; o correto (e o código) é $1 - c_3 e^{c_2 T}$
§11.4.3	a solução ($\kappa_v = 18{,}447$) está na fronteira de Feller: a restrição está ativa
§11.6.2	`options.ix` foi removido do pandas; use `.loc`

Módulo 14 — Calibração II: saltos, regularização e o BCC97 completo

Notebook: 14_calibracao_bcc97

Onde	O quê
§11.6.4, `BCC_calibration_short`	`opt1` e `local_opt` são atribuídos sem `global`: viram variáveis locais, e a penalidade da função objetivo nunca é aplicada
Eq. (11.17)	a penalidade é a norma $\|p - p_0\|$, não o quadrado: não é Tikhonov; com $w = 1$ e EQM em pontos^2, congela os parâmetros no ponto de partida
§11.4.4, resultados	$\lambda = 0{,}008$, $\mu_J = -0{,}600$ (no limite) e $\delta = 0{,}001$: na prática, nenhum salto; sintoma da calibração sequencial
§11.4.5, resultados	$\rho = -0{,}999$ e $\delta = 0{,}000$ na fronteira, $\lambda = 0{,}008$: o BCC97 calibrado é praticamente um Heston
§11.4.6, resultados	κ_v vai de 22,2 a 28,5 entre (11.1) e (11.4), funções quase iguais: parâmetros mal identificados; $\mu_J = -0{,}600$ de novo no limite
§11.6.5 e §11.6.6	a condição de Feller continua imposta (ver Módulo 13)

Onde	O quê
§11.6.2 e seguintes	Nelder-Mead com restrições por "valor alto" pode parar cedo encostado numa parede; sem reinício, o resultado depende de detalhes mínimos dos dados
§11.6.6	`options.ix` foi removido do pandas; use `.loc`

Módulo 15 — Simulação e precificação no modelo BCC97

Notebook: 15_simulacao_bcc97

Onde	O quê
§12.5.1, `B96_generate_paths`	`rj` é calculado e nunca usado: falta o compensador $-r_J \Delta t$ de (12.1); o índice perde $\lvert r_J \rvert$ de drift por ano
Eq. (12.1) e código	saltos aditivos com um único tamanho para todos os y_t saltos do passo; o exato é $e^{N\mu_J + \sqrt{N}\delta z}$
Eq. (12.1) e código	a variância do passo é `v[t]`, do fim do passo (Módulo 12); com $\rho = -0{,}999$, o drift espúrio é $-0{,}475$ por ano
§12.3.1	o viés de 22 a 52 pontos atribuído ao número de passos vem do `v[t]`; o esquema exato acerta com 50 passos por ano a partir de 6 meses (só no prazo de 1 mês, com 4 passos, há viés de Euler de 1 a 2 pontos)
§12.3.2	a "diferença" americana − europeia mistura prêmio e viés de simulação
§12.5.3, título	"American Call Options", mas o script avalia puts
Parâmetros do livro	$\rho = -0{,}999$, $\delta = 0$ e $\lambda = 0{,}008$ vêm da calibração degenerada do Módulo 14
Scripts	Python 2 (`xrange`, `print`, `I / 2`)

Módulo 16 — Hedge dinâmico e síntese do curso

Notebook: 16_hedge_dinamico

Onde	O quê
§13.5.1 e §13.5.3, delta inicial	os dois preços LSM da diferença finita usam números aleatórios diferentes: o delta inicial é dominado pelo ruído (desvio-padrão acima de 0,5)
§13.5.1, regra de exercício	`h >= C` com `C` truncado em zero marca como exercidas puts fora do dinheiro, encerrando o hedge antes da hora
§13.5.2	P&L calculado nas mesmas trajetórias da regressão (dentro da amostra), apesar de a nota 3 defender a avaliação fora da amostra
§13.3, P&L médio de +4,34	numa simulação sob Q, o P&L médio de um hedge autofinanciável iniciado no preço justo é zero; o valor positivo vem do simulador (Módulo 15)
§13.3, "risco de salto diversificável"	a hipótese de Merton é de diversificação entre ativos; não vale para muitas opções sobre o mesmo índice, cujos saltos são comuns
§13.5.3	herda o simulador do cap. 12 (`v[t]`, compensador ausente, saltos aditivos, moment matching)
Cap. 14	o resumo é fiel ao livro; os módulos 12 a 16 mostram que as etapas de simulação, calibração e hedge precisam das conferências da Seção 6.2

Referências

Leif Andersen. Simple and efficient simulation of the Heston stochastic volatility model. *Journal of Computational Finance*, 11(3):1–42, 2008.

Kenneth J. Arrow. The role of securities in the optimal allocation of risk-bearing. *Review of Economic Studies*, 31(2):91–96, 1964.

Gurdip Bakshi, Charles Cao, and Zhiwu Chen. Empirical performance of alternative option pricing models. *Journal of Finance*, 52(5):2003–2049, 1997.

Tomas Björk. *Arbitrage Theory in Continuous Time*. Oxford University Press, Oxford, 3 edition, 2009.

Fischer Black. The pricing of commodity contracts. *Journal of Financial Economics*, 3 (1–2):167–179, 1976.

Fischer Black and Piotr Karasinski. Bond and option pricing when short rates are lognormal. *Financial Analysts Journal*, 47(4):52–59, 1991.

Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

Phelim Boyle, Mark Broadie, and Paul Glasserman. Monte Carlo methods for security pricing. *Journal of Economic Dynamics and Control*, 21(8–9):1267–1321, 1997.

Douglas T. Breeden and Robert H. Litzenberger. Prices of state-contingent claims implicit in option prices. *Journal of Business*, 51(4):621–651, 1978.

Damiano Brigo and Fabio Mercurio. *Interest Rate Models – Theory and Practice*. Springer, Berlin, 2 edition, 2006.

Mark Broadie and Özgür Kaya. Exact simulation of stochastic volatility and other affine jump diffusion processes. *Operations Research*, 54(2):217–231, 2006.

Peter Carr and Dilip B. Madan. Option valuation using the fast Fourier transform. *Journal of Computational Finance*, 2(4):61–73, 1999.

Emmanuelle Clément, Damien Lamberton, and Philip Protter. An analysis of a least squares regression method for American option pricing. *Finance and Stochastics*, 6 (4):449–471, 2002.

Les Clewlow and Chris Strickland. *Implementing Derivatives Models*. Wiley, Chichester, 1998.

Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.

Rama Cont and Peter Tankov. *Financial Modelling with Jump Processes*. Chapman & Hall/CRC, Boca Raton, 2004.

James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.

John C. Cox, Stephen A. Ross, and Mark Rubinstein. Option pricing: A simplified approach. *Journal of Financial Economics*, 7(3):229–263, 1979.

John C. Cox, Jonathan E. Ingersoll, and Stephen A. Ross. A theory of the term structure of interest rates. *Econometrica*, 53(2):385–407, 1985.

Robert C. Dalang, Andrew Morton, and Walter Willinger. Equivalent martingale measures and no-arbitrage in stochastic securities market models. *Stochastics and Stochastic Reports*, 29(2):185–201, 1990.

Freddy Delbaen and Walter Schachermayer. A general version of the fundamental theorem of asset pricing. *Mathematische Annalen*, 300:463–520, 1994.

Emanuel Derman and Iraj Kani. Riding on a smile. *Risk*, 7(2):32–39, 1994.

Emanuel Derman, Iraj Kani, and Neil Chriss. Implied trinomial trees of the volatility smile. *Journal of Derivatives*, 3(4):7–22, 1996a.

Emanuel Derman, Iraj Kani, and Joseph Z. Zou. The local volatility surface: Unlocking the information in index option prices. *Financial Analysts Journal*, 52(4):25–36, 1996b.

Jin-Chuan Duan and Jean-Guy Simonato. Empirical martingale simulation for asset prices. *Management Science*, 44(9):1218–1233, 1998.

Darrell Duffie. *Dynamic Asset Pricing Theory*. Princeton University Press, Princeton, 3 edition, 2001.

Darrell Duffie, Jun Pan, and Kenneth Singleton. Transform analysis and asset pricing for affine jump-diffusions. *Econometrica*, 68(6):1343–1376, 2000.

Bruno Dupire. Pricing with a smile. *Risk*, 7(1):18–20, 1994.

Nicole El Karoui, Monique Jeanblanc-Picqué, and Steven E. Shreve. Robustness of the Black and Scholes formula. *Mathematical Finance*, 8(2):93–126, 1998.

Heinz W. Engl, Martin Hanke, and Andreas Neubauer. *Regularization of Inverse Problems*. Kluwer, Dordrecht, 1996.

William Feller. Two singular diffusion problems. *Annals of Mathematics*, 54(1):173–182, 1951.

Jim Gatheral. *The Volatility Surface: A Practitioner's Guide*. Wiley, Hoboken, 2006.

Hélyette Geman, Nicole El Karoui, and Jean-Charles Rochet. Changes of numéraire, changes of probability measure and option pricing. *Journal of Applied Probability*, 32 (2):443–458, 1995.

Paul Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, New York, 2004.

Patrick S. Hagan, Deep Kumar, Andrew S. Lesniewski, and Diana E. Woodward. Managing smile risk. *Wilmott Magazine*, 1:84–108, 2002.

J. Michael Harrison and David M. Kreps. Martingales and arbitrage in multiperiod securities markets. *Journal of Economic Theory*, 20(3):381–408, 1979.

J. Michael Harrison and Stanley R. Pliska. Martingales and stochastic integrals in the theory of continuous trading. *Stochastic Processes and their Applications*, 11(3): 215–260, 1981.

Martin B. Haugh and Leonid Kogan. Pricing American options: A duality approach. *Operations Research*, 52(1):8–20, 2004.

David Heath, Robert Jarrow, and Andrew Morton. Bond pricing and the term structure of interest rates: A new methodology for contingent claims valuation. *Econometrica*, 60(1):77–105, 1992.

Steven L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Review of Financial Studies*, 6(2):327–343, 1993.

Yves Hilpisch. *Derivatives Analytics with Python: Data Analysis, Models, Simulation, Calibration and Hedging*. Wiley, Chichester, 2015.

Thomas S. Y. Ho and Sang-Bin Lee. Term structure movements and pricing interest rate contingent claims. *Journal of Finance*, 41(5):1011–1029, 1986.

John Hull and Alan White. Pricing interest-rate-derivative securities. *Review of Financial Studies*, 3(4):573–592, 1990.

John Hull and Alan White. Numerical procedures for implementing term structure models I: Single-factor models. *Journal of Derivatives*, 2(1):7–16, 1994.

John C. Hull. *Options, Futures, and Other Derivatives*. Pearson, New York, 10 edition, 2018.

Farshid Jamshidian. An exact bond option formula. *Journal of Finance*, 44(1):205–209, 1989.

Ioannis Karatzas and Steven E. Shreve. *Brownian Motion and Stochastic Calculus*. Springer, New York, 2 edition, 1991.

Peter E. Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*. Springer, Berlin, 1992.

Damien Lamberton and Bernard Lapeyre. *Introduction to Stochastic Calculus Applied to Finance*. Chapman & Hall/CRC, Boca Raton, 2 edition, 2008.

Hayne E. Leland. Option pricing and replication with transactions costs. *Journal of Finance*, 40(5):1283–1301, 1985.

Alan L. Lewis. A simple option formula for general jump-diffusion and other exponential Lévy processes. Manuscrito, Envision Financial Systems e OptionCity.net, 2001.

Francis A. Longstaff and Eduardo S. Schwartz. Valuing American options by simulation: A simple least-squares approach. *Review of Financial Studies*, 14(1):113–147, 2001.

Roger Lord, Fang Fang, Frank Bervoets, and Cornelis W. Oosterlee. A fast and accurate FFT-based method for pricing early-exercise options under Lévy processes. *SIAM Journal on Scientific Computing*, 30(4):1678–1705, 2008.

Roger Lord, Remmert Koekkoek, and Dick van Dijk. A comparison of biased simulation schemes for stochastic volatility models. *Quantitative Finance*, 10(2):177–194, 2010.

Robert C. Merton. Theory of rational option pricing. *Bell Journal of Economics and Management Science*, 4(1):141–183, 1973.

Robert C. Merton. Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, 3(1–2):125–144, 1976.

Marek Musiela and Marek Rutkowski. *Martingale Methods in Financial Modelling*. Springer, Berlin, 2 edition, 2005.

Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Springer, Berlin, 6 edition, 2003.

L. C. G. Rogers. Monte Carlo valuation of American options. *Mathematical Finance*, 12(3):271–286, 2002.

Steven E. Shreve. *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer, New York, 2004a.

Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, New York, 2004b.

Oldrich Vasicek. An equilibrium characterization of the term structure. *Journal of Financial Economics*, 5(2):177–188, 1977.

Paul Wilmott. *Paul Wilmott on Quantitative Finance*. Wiley, Chichester, 2 edition, 2006.